CIVICCAST

PUBLIC MEETINGS, ON THE AIR.

# CivicCast User Manual

**For station staff, volunteers and city IT staff - v1.0.0-beta.20 (native Windows line)**

The CivicCast Authors

2026-10-17

# Contents

**About this manual** **2**
Who Reads What . . . 2
Where the older guide names went . . . 2
How to read the notes in this manual . . . 2
Words used throughout . . . 3
How this manual was made . . . 3
Edition and package evidence . . . 3
Where to get help and report problems . . . 4

**Part I. Using CivicCast (for station staff and volunteers)** **5**
What CivicCast is and who uses it . . . 6
Signing in and finding your way around . . . 14
Before the meeting: schedules, agendas, and recording schedules . . . 27
Running the meeting . . . 52
After the meeting: assets, captions, summaries, approvals . . . 76
Publishing, and what residents see . . . 90
Reports, analytics, program guide export, underwriting, and apps . . . 104
When something looks wrong . . . 117

**Part II. Installing and running CivicCast (for IT staff)** **132**
Planning your station: hardware, network, storage, accounts . . . 133
Installing, first run, upgrading, uninstalling . . . 146
Configuring the station . . . 172
Running it day to day: service, logs, backups, updates . . . 206
Security and privacy . . . 225
Troubleshooting matrix . . . 241
Cable headend, streaming, CDN, federation, emergency alerts, and the API . . . 277

**Part III. How CivicCast is built** **295**
How CivicCast is built . . . 296

**Part IV. Reference** **341**
Appendix A: Command-line reference . . . 341
Appendix B: HTTP API overview . . . 365
Appendix C: Settings and environment variables . . . 431
Appendix D: Files, folders, ports and services . . . 466
Appendix E: Status and error glossary . . . 473
Appendix F: Roles and permissions matrix . . . 485
Appendix G: Checklists . . . 492
Appendix H: Measured evidence and known limits . . . 497
Appendix I: Glossary . . . 500
Appendix J: Licenses and third-party software . . . 508

Appendix K: Release history . . . . . 512
Appendix L: Index of screens . . . . . 516

# About this manual

CivicCast is a Windows program for a public-access (PEG) station or a small city to schedule, put meetings on the air, record them, and publish them for residents. This manual guides station staff through those tasks and the IT work that supports them.

### Who Reads What

| If you are... | Read |
|---|---|
| A volunteer, camera operator, video editor, records clerk or PEG station employee | **Part I** (chapters 1 to 8): what to click, what you will see, what to do when something looks wrong |
| The IT person at a small city who installs and runs CivicCast | **Part II** (chapters 9 to 15): planning, installing, configuring, running, securing, troubleshooting |
| An IT reviewer, integrator or the curious | **Part III** (chapter 16): how the system is built, with diagrams |
| Anyone looking something up | **Part IV**, the appendices: commands, web interface, settings, files and ports, status words, roles, checklists, evidence, glossary, licenses, release history |

### Where the older guide names went

Earlier releases split this manual into separate guides. Their names are still used in other documents, so here is where each one now lives in the CivicCast User Manual.

- **Admin Quick Guide** (also the Admin Guide): Part II, chapters 9 to 15, and Appendices A to F.
- **Meeting Operator Quick Guide** (also the Meeting Operator Guide): Part I, chapters 3 and 4.
- **Records Clerk Quick Guide** (also the Records Clerk Guide): Part I, chapters 5 and 6.
- **Technical Operations Reference**: Part II, chapters 11 to 15, Part III (chapter 16) and Appendices A to E.

### How to read the notes in this manual

> **Note:** background you may need.

> **Tip:** a faster or safer way to do something.

> **Warning:** something that can change what is on the air, delete data, or cannot be undone.

> **Known issue (version):** behavior checked in the named version differs from what the screen suggests; the note gives its scope and a safe next step. An explicit current-version statement describes a current limitation; an older label alone is historical.

> **For IT staff:** a pointer from the non-technical part to the technical chapter that has the detail.

### Words used throughout

- **Station** means the Windows computer running CivicCast and everything it controls.
- **Staff console** (also called the *operator console*) is the web page staff use to run the station. It opens at the station's address followed by `/operator/`.
- **Resident portal** is the public web page where residents watch live programs, browse recordings and read the schedule.
- **Channel** is one program stream, like a cable channel. A station can run several.
- **Asset** is one recording or media file stored in CivicCast.
- Schedule, Program Guide, recording and auto-schedule times use the station's configured time zone. Some reports and exports still use **UTC**; each chapter names the relevant time basis.

### How this manual was made

The manual was written from the product's own code and from a screen-by-screen inventory of the staff console, the resident portal and the installer, then checked against that code. Screens were photographed on a lab station that holds sample (mock) data only. Where the code could not settle a question, the manual says nothing rather than guessing. The version of the software this manual describes is shown on the cover; if you run a different version, details may differ.

### Edition and package evidence

This working source edition also includes unreleased Sprint 6 preparation, webhook, Roku and **Other stations** changes. For an installed Beta 19 package, use the manual bundled with that release. The source changes have not replaced the published installer or its package evidence.

The October 17 documentation correction reconciles operating instructions and refreshes the CLI, API and settings references. Existing installed Help may still contain the older wording until its application package is updated.

This unpublished Beta 20 candidate edition covers: remote guests in channel output, bulletin-board live video and audio, native viewer improvements, and the publication, import and underwriting work shipped in Beta 17 and 18. The manual also covers staff accounts, station backup and restore, network access, the restricted service account, station-local scheduling, worker recovery, broadcast outputs and recorded captions. The Beta 18 package record remains historical; use the Beta 19 verification record for claims about its installed package.

Check the releases page for available downloads. For an installed package, use its matching Help and verification record. A **historical observation** describes behavior or evidence from the named earlier release. A **Known issue (version)** names the version in which the behavior was checked; the label

alone does not say whether it remains true now. Historical host measurements remain distinct from installation checks of any package.

### Where to get help and report problems

Report problems and read release notes at the project page: https://github.com/scottconverse/civiccast-native. Check https://github.com/scottconverse/civiccast-native/releases for current downloads and publication status, then read the verification record for the exact package you use. Dated records for earlier packages remain available as release history.

# Part I. Using CivicCast (for station staff and volunteers)

These chapters are for people who run meetings, operate cameras, edit video, keep records and answer residents. You do not need to be technical. Each chapter lists what you need before you start, then walks through each task step by step.

## What CivicCast is and who uses it

This chapter explains what CivicCast does, which jobs the people at a small station do with it, and what happens to a meeting from the day it is put on the calendar to the day a resident watches it. It also gives a short word list for Part I. Everyone reading Part I should read it first.

### What CivicCast is

CivicCast is software for a public-meeting station: a city or county channel that records council and board meetings, is built to put them on a cable or web channel, and keeps an archive that residents can watch later. It runs on one Windows computer at the station. This manual calls that computer the **station computer**. The program keeps running in the background on that computer; you work with it through two web pages that open in a browser.

CivicCast does these jobs:

- **Keeps a library of meeting videos.** Each video file in the library is called an *asset*.
- **Schedules what airs on each channel.** A *channel* is one program stream the station sends out. A new station starts with three channels, named public, education and government.
- **Runs the channel.** A staff member starts and stops the channel's *outgoing feed* (the running stream), checks that a live camera or encoder is delivering a picture, and can put a live source on the air.
- **Records meetings** from a capture card or a network stream at times you set.
- **Makes captions.** Recorded-caption jobs turn speech into text that a person reviews before it is published with the recording. Live broadcasts can publish first-pass captions without a human review step (see Running the meeting).
- **Writes an AI summary on request.** The summary is built from the approved captions, and each quantitative claim (a count, an amount, a vote tally) must point to timestamp ranges in the transcript. A person approves it.
- **Publishes recordings to a resident website** and can copy them to archive and outside services you set up.
- **Watches its own health** and tells staff when something needs attention.

The two web pages are:

| Page | What it is | Who uses it |
|---|---|---|
| **Operator console** (the *console*) | The staff screens. A menu down the left side (the *sidebar*) leads to every screen. Chapter 2 shows how to sign in and find your way around. | Station staff and volunteers |
| **Resident portal** (the *portal*) | The public website. It has three pages: Home, Recordings and Schedule. No sign-in. | Residents |

The installer puts a shortcut named **CivicCast Operator Console** on the Desktop and in the Start

menu, and a second Start-menu shortcut named **CivicCast Public Portal**. The console and the portal are two different web pages served by the same program.

> **For IT staff:** The console and portal addresses, the Windows service that runs them, and how to make the portal reachable from residents' own devices are covered in Installing, first run, upgrading, uninstalling and Planning your station.

#### The people and their roles

CivicCast has five *roles*. A role is a set of things a signed-in person is allowed to do. The roles are named after jobs that exist at a small station, where one or two people usually do all of them. Think of a role as a hat. One person can wear all five hats.

| Role (as the console shows it) | The job, in plain words | What this job does in the console |
|---|---|---|
| **Setup admin** (`setup_admin`) | Sets the station up and keeps its settings right. | Creates the station's first account and recovery kit; adds camera or test sources, a backup folder and provider accounts on First Setup; edits Station Profile, AI Models, Custom Fields, Paywall and Control Room Setup; applies cable-headend settings and 24/7 settings on Channels; can create and cancel schedule items. |
| **Meeting operator** (`meeting_operator`) | Runs the meeting night. | Works the Live screen (check the source, run the checklist), starts and stops a channel's feed and puts a live source on air from Channels, runs Control Room, starts and stops scheduled recordings, builds agendas, adds recurring slots in the Program Guide. |
| **Records clerk** (`records_clerk`) | Looks after the official record of the meeting. | Approves, edits or rejects caption lines in the Review queue; approves AI summaries; places and clears legal holds; uploads videos; edits a recording's public title and description. |

| Role (as the console shows it) | The job, in plain words | What this job does in the console |
|---|---|---|
| **Publish operator** (`publish_operator`) | Decides what residents and outside services can see. | Creates and cancels schedule items and approves them to air; prepares a recording for playback; approves publishing on the Publish screen; removes a recording from the portal; manages Auto-schedule, the community board, program-guide export, underwriting and App Admin. |
| **Support admin** (`support_admin`) | The help desk: the person who keeps the station healthy and fixes it. | Creates the support bundle and runs repair, restore, update and rollback tools on Readiness; reads Reports; can read most screens. Mostly read-only elsewhere. |

The names in parentheses are the raw role names. You will see them in error messages, for example "This action requires one of these CivicCast roles: meeting_operator, setup_admin." The console never shows them any other way.

**Everyone signed in** Some screens ask for no role at all. The sidebar shows them to every signed-in person: Live, Facility, Control Room, Channels, CG Board, Schedule, Program Guide, Assets, Contributors, Review queue, Summary review, Publish, Playback policy, Analytics, Readiness, Alerts and Federation. The Manual and First Setup are open even to a person who is signed out.

> **Warning:** A screen that appears in your sidebar can still refuse you. The sidebar hides some screens by role, but each screen and each button is checked again when you use it. For example, anyone can open Schedule, but a person without the Publish operator, Setup admin or Support admin role gets a red "Could not load schedule." box. If a screen refuses you, you do not have the role it needs. Chapter 2 lists the wording you will see.

**How people get roles** The first account, created on First Setup, holds all five roles and remains the station's recovery account. A setup admin can add staff accounts, assign roles, disable sign-in, reset passwords and manage sessions from **Staff accounts**. Each staff member signs in with their own username and password and receives only their assigned roles. See Chapter 2 and Security and privacy.

The command-line tool also makes access tokens for scripts and tools. Those tokens are separate from browser sign-in.

> **For IT staff:** Role-limited keys, the role-to-screen table and the access rules by feature are in Security and privacy and the roles appendix (Roles and permissions).

### The life of a meeting

Here is the path a meeting takes through CivicCast. Each step names the console screens involved. Chapters 3 to 6 explain each step with exact buttons.

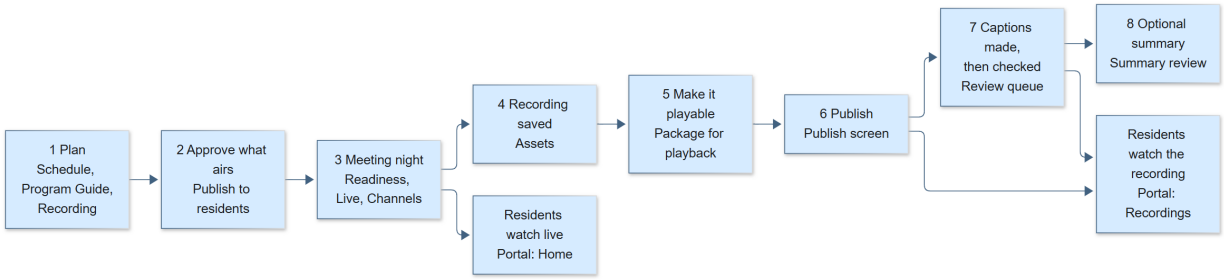



Figure 1.1. The life of a meeting in CivicCast. Boxes 1 to 8 are staff work in the operator console; the two boxes on the right are what residents see on the portal.

**How to read the figure.** Follow the arrows from the left. Steps 1 and 2 are done ahead of time. Step 3 is the meeting itself, and while it runs residents can watch on the portal's Home page. After the meeting, the recording appears in the Assets library (step 4). Before residents can watch a saved recording it has to be made playable in a browser (step 5) and then published (step 6). When recorded captions are enabled, publish approval also starts caption work for a recording with a stored source file (step 7). The recording becomes visible to residents at once; reviewed captions attach later according to the station's completion setting. The summary (step 8) is optional.

1. **Plan.** Put the meeting on the calendar. You can place a recorded video on a channel at a set time (Schedule, Program Guide or Auto-schedule). To capture a live meeting, set up a recording from a capture card or network stream (Recording). Details: Before the meeting.
2. **Approve what airs.** A recorded program placed on the Schedule does not air, and residents do not see it, until someone approves it with **Publish to residents**. Saving a schedule item and approving it are two separate steps. A live meeting goes on air in step 3, not here.
3. **Meeting night.** Check Readiness (the "can we broadcast right now?" page). On Live, check that the camera or encoder is delivering, then run the checklist. On Channels, start the channel's feed and, for a live meeting, put the checked live source on the air. Details: Running the meeting.
4. **Recording saved.** A finished recording becomes an asset in the Assets library. After you end a live session on the Live screen, CivicCast looks for the recording file, checks and packages it, and shows "Recording saved as asset" followed by the asset's ID.
5. **Make it playable.** *Packaging* makes a copy of the video that a web browser can stream. An asset with no playable copy cannot be watched on the portal.
6. **Publish.** On the Publish screen a publish operator ticks where the recording should go. The Portal option makes the recording public to residents. The other options copy it to archives and outside services.

7. **Captions.** When recorded captions are enabled, approving publication queues caption work for a recording with a stored source file, even if the Portal row is not selected. CivicCast holds the resulting caption lines for a person to approve, correct or reject. Station Profile can disable recorded captions, select English only or English and Spanish, and choose whether reviewed English attaches first or waits for every selected language. See Recorded-caption settings.
8. **Summary (optional).** From an asset's page a records clerk can ask for an AI summary. This console workflow uses only approved caption lines, including saved corrections that were subsequently approved. A person reviews and approves the summary.

After the meeting, the portal shows residents the recording on **Recordings** and on its own watch page, with captions when they are attached. The portal's **Schedule** page shows what airs on each channel over the next 72 hours.

> **Packaging recordings:** A meeting captured on **Recording** can arrive in Assets as **Recorded**, with **Not servable yet** until a streamable copy exists. **Package for playback** is offered for both **Validated** and **Recorded** rows with a stored source file. If a streamable copy already exists, use **Repackage for playback** to replace it after changing a trim or source. A recording that Live finalizes is checked and packaged by that step. See Chapter 5.

#### What residents see

The portal is an English-only website with a dark look and no sign-in. Its header has three links: **Home**, **Recordings** and **Schedule**.

- **Home** shows whether the station is on air. When it is, a video player appears. When it is not, you see "The station is standing by" or "No live broadcast is on air." It also lists what is coming up and the six newest recordings.
- **Recordings** is a searchable archive of every published meeting.
- A recording's **watch page** shows the video with captions, the meeting's agenda beside it if staff published one, the description, and a **Copy share link** button.
- **Schedule** shows what airs on each channel.

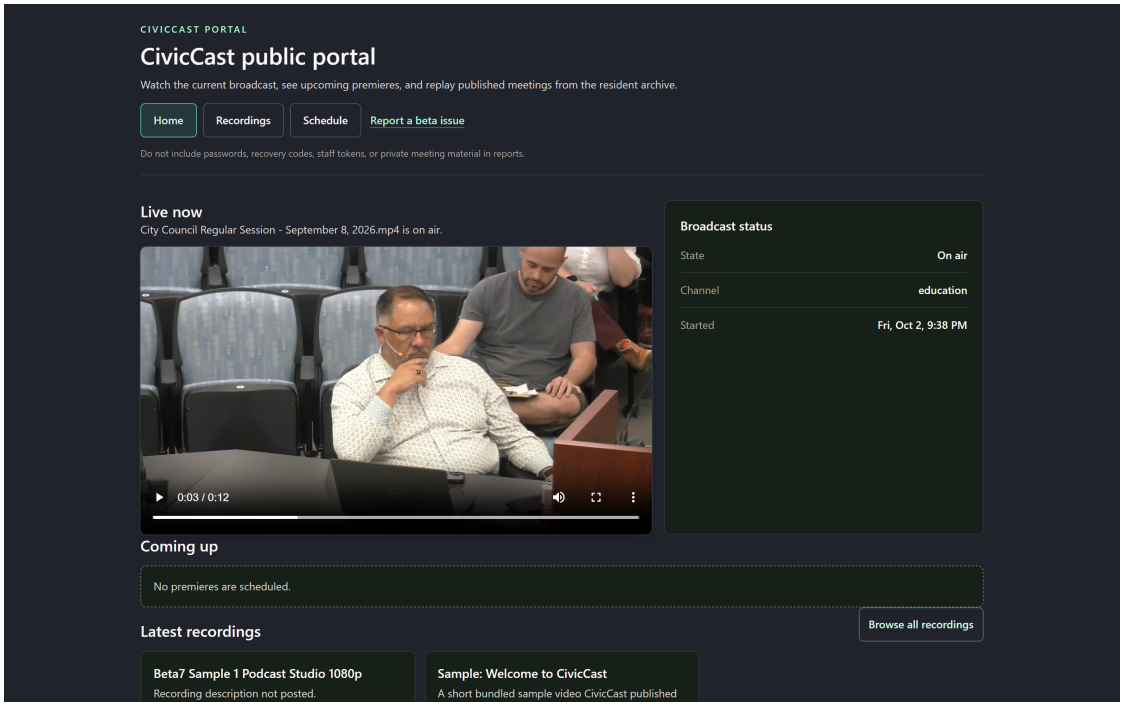
*Figure 1.2. The resident portal's Home page.*

#### Word list for Part I

These are the words used most in Part I. Each is explained again where it first matters.

| Word | What it means in this manual |
|---|---|
| **Station computer** | The one Windows computer where CivicCast is installed. |
| **Console** | The staff web page. The sidebar on its left is the menu. |
| **Portal** | The public web page for residents. |

| Word | What it means in this manual |
|---|---|
| **Channel** | One program stream the station sends out. A new station has three: public, education, government. |
| **On air** | A channel's feed is running and sending out what is playing. |
| **Outgoing feed** | The running stream for one channel. Staff start and stop it on the Channels screen. |
| **Slate** | A standby card the channel shows when no program is playing. |
| **Source** | A live picture CivicCast can receive, such as a camera or encoder. Source types shown on the Live screen are RTMP, RTSP, NDI and SRT. |
| **Asset** | One video file in the library (the Assets screen). |
| **Recorded / Validated** | Two of an asset's states. A *Validated* asset passed CivicCast's file checks, as uploads do. A *Recorded* asset is a finished live capture. |
| **Package for playback** | Make a copy of an asset that web browsers can stream. |
| **Publish** | Make a recording visible to residents, and optionally copy it to archives and outside services. |
| **Premiere** | A recorded asset placed on a channel at a start time. |
| **Caption** | The text of what is said, shown with the video. |
| **Cue** | One caption line with a start and end time. The Review queue shows cues one at a time. |
| **Summary** | An AI-written overview of a meeting. Every quantitative claim must link to caption timestamps. |
| **Readiness** | The answer to "can we broadcast right now?" on the Readiness screen. |
| **Role** | A set of things a signed-in person may do. There are five. |
| **Recovery kit** | The one-time printout or file with the admin username, the admin password and eight emergency codes, created on First Setup. |
| **Test mode** | The mode a new station starts in. The First-run defaults card on First Setup shows a **Mode** heading with "Test mode" under it. |
| **Headend** | The cable company's equipment that receives a channel's signal. |




The resident portal Home page while a channel is on air. The header shows the links Home, Recordings and Schedule. The Live now panel has a video player beside a Broadcast status card.

### Source edition and release evidence

This manual describes the unpublished Beta 20 candidate source; installation verification is pending. Use the download and installation instructions in Chapter 10 with a package on the releases page, and follow its matching verification record. The Beta 18 record tracks that earlier package's checks. Keep each historical result with its named release.

**Historical Beta 11 operating result:** one CivicCast host ran Public, Government, and Education channels together for 72 hours and recorded **99.78% live caption-input coverage**. This remains a Beta 11 host measurement; it does not establish performance of the current package. See the 72-hour host performance report for the channel totals and method.

**Live captions:** Whistle is the CPU primary, publishes first-pass recognition, and uses Whisper fallback for the affected channel when needed. CivicCast automatically probes Whistle recovery after an initial 30-second cooldown and retries with bounded backoff; a successful probe returns later chunks to Whistle without a service restart. Probe timing is not a fixed recovery guarantee. Optional NVIDIA CUDA applies to supported Whisper work; Whistle remains CPU-only.

The evidence appendix keeps historical measurements with their original builds and runs.

### If it did not work

This chapter contains no steps to follow, so there is nothing here to fail. If a later chapter's screen shows a message you do not understand, chapter 8 (When something looks wrong) lists the common ones.

**Related**

- Signing in and finding your way around
- Before the meeting
- Running the meeting
- After the meeting
- Publishing, and what residents see

## Signing in and finding your way around

This chapter shows you how to open the operator console, sign in, recover a lost password, sign out, and find the screen you need. It also explains the colors and status words you will see, how to use the Manual built into the console, and how to set up a brand-new station the first time. Everyone who uses the console reads this chapter; the first-run section is for the person setting up a new station.

### Before you start

- For **first setup, first-admin sign-in and recovery**, use a browser on the station computer (a remote-desktop session to that computer also works). Staff accounts can sign in from the station or, when the separate staff hostname is configured, from an approved staff network. See the network access guide and staff accounts guide.
- You need the **admin username and password** from the station's recovery kit. The kit is the printout or saved file made when the station was first set up.
- CivicCast has to be running in the background. If it is not, the shortcut opens a browser error page instead of the console.
- If nobody has set this station up yet, go straight to Set up a brand-new station.

> **For IT staff:** The Windows service, the console address and the sign-in limits are in Installing, first run, upgrading, uninstalling and Security and privacy.

### Open the console

1. On the station computer, double-click the **CivicCast Operator Console** shortcut on the Desktop. The Start menu has the same shortcut.
2. Wait for the browser to open.

The Desktop shortcut opens the local operator console. A signed-out browser now has a separate **Staff sign-in** page. Use **First setup** at `/setup` only to create or recover the first administrator; ordinary staff sign in at `/login`.

> **Note:** First-admin setup and recovery remain local-only. A staff account can use `/login` on the configured staff hostname from an allowed network; the public hostname does not expose staff sign-in.

### Sign in

1. Staff: open `/login`. First administrator: open `/setup` on the station computer and choose **Admin sign-in**.
2. Enter the account's username and current password. For the first administrator, use the password chosen during setup (or the new password from the most recent recovery/reset); an older printed password may no longer work.
3. Click **Sign in**. The button does nothing until both boxes are filled.

You should be taken to the **Readiness** screen. If a screen sent you here because you were signed out, you go back to that screen instead.

> **Note:** Signing in on one browser does not sign out any other browser or device that is already signed in.

> **Note:** A normal sign-in takes you straight to Readiness, so you will not see this on that path. After you create the first admin on a new station (see Set up a brand-new station), the **Setup complete** card says "CivicCast saved a fresh console token in this browser for <your admin display name>." A *console token* is the private pass your browser keeps so the station knows it is you. The sentence means you are signed in.

> **Warning:** Closing the browser does not sign you out. The browser keeps its token until you sign out, an administrator revokes the session, or the 12-hour session expiry is reached. On a shared or borrowed computer, always click **Sign out** before you leave.

#### Use a recovery code

Use this only when the admin password is lost. If you have the password, use **Admin sign-in** instead. Each recovery code works once, and the station has eight.

1. Find your printed or saved recovery kit and pick one unused code.
2. On the sign-in page, find the card named **Use recovery code**.
3. Type your username in **Admin username**.
4. Type the code in **Recovery code**.
5. Type a new password in **New admin password**. It needs at least 12 characters.
6. Type the same password in **Confirm new admin password**. If the two differ, you see "Passwords do not match."
7. Click **Recover account**. The button changes to **Confirm — consume recovery code**, and a red message appears: "This permanently consumes one recovery code — only 8 exist for this station. Click Recover account again to confirm." That message refers to the renamed **Confirm — consume recovery code** button.
8. Click **Confirm — consume recovery code**.

You are signed in and taken to the same place as after a normal sign-in. Every other browser or device that was already signed in stays signed in.

> **Warning:** The code is used up when the station accepts it. If you change any box after the first click, the button goes back to **Recover account** and you must click twice again.

> **Warning:** Your printed kit may still list an old password. After a recovery, write the new password somewhere safe. While you can still sign in, a Setup admin can make a fresh set of eight codes with **Regenerate recovery kit** in the **Security** card on **Station Profile**; the old codes stop working at once, and the new kit lists the codes only, not the password. If the password and all recovery codes are lost, a local Windows administrator can recover the account offline with

> the administrator-reset command in Security and privacy. That reset preserves station data, revokes existing first-admin sessions and old recovery codes, and requires a fresh recovery kit afterward.

The Beta20 source candidate includes station-state recovery repair so the normal service can replace the session automatically without manually changing permissions. Expected behavior is a return to normal sign-in, with no permission edits needed; private-key protection is unchanged. Final installed proof is pending.

#### Sign out

1. Look at the right end of the top bar.
2. Click **Sign out**. The button reads "Signing out…" for a moment.

You should land on the sign-in page. This signs out the browser you are using and no other.

The **Sign out** button appears only after the console has confirmed who you are. It is grey and unusable while a new station's recovery kit is waiting to be confirmed.

#### Find your way around the console

**The top bar** The top bar runs across the whole screen. From the left it shows the **C** logo, the word **CivicCast** and the installed version number, in the form `v1.0.0-beta.N`. On a wide screen the middle shows a pill reading **No live meeting broadcast** and a clock reading "Local <time> / Next No events scheduled". On the right are a theme button (**Switch to dark theme**), a round badge with your initials, and **Sign out**.

Hover over the round badge to see your name and your roles, in the form "<your display name> / <your role names>". Setup admins can also review account role assignments in **Staff accounts**.

> **Known issue (beta.12; first recorded in beta.11):** The **No live meeting broadcast** pill and the words "Next No events scheduled" still never change. They stay the same during a live meeting and with a full schedule. Do not use them to decide whether the station is on air or what comes next. Use **Readiness**, **Channels** and **Schedule** instead.

**The sidebar** The sidebar on the left is the menu. At its top a card reads **Public meetings** over **CivicCast station**. Below it are six sections. Click a section's name to open or close it. At the bottom is a link **Report a beta issue** and a tag **Operator-first beta**.

> **Known issue (beta.12; first recorded in beta.11):** The card at the top of the sidebar always reads "CivicCast station" and "Public meetings". It does not show your station's name. The name you chose appears on the sign-in page, in the **Setup complete** card.

The six sections start like this. The Help, Setup and System Health sections start closed, and the section that holds the screen you are on always opens.

| Section | Screens in it | You use it to |
|---|---|---|
| **Help** | Manual | Read the manual built into the console. |
| **Setup** | First Setup, Staff accounts, Backup and restore, Import station, Network access, Control Room Setup, Station Profile, Cable Commissioning, AI Models, Custom Fields, Paywall | Set the station up and change its settings. |
| **Run Meeting** | Live, Facility, Control Room, Remote Contribution, Channels, Other stations, CG Board, CG Designer, Schedule, Auto-schedule, Program Guide, Recording | Plan and run what goes on air. |
| **Review Records** | Assets, Missing Media, Media Lifecycle Settings, Contributors, Review queue, Summary review, Agendas | Look after videos, captions, summaries and agendas. |
| **Publish** | Publish, Playback policy, Analytics, Reports, EPG Export, Underwriting, App Admin | Decide what residents see, and read audience numbers. |
| **System Health** | Readiness, Alerts, Emergency Alerts, Federation | Check that the station is healthy. |

Altogether the console has 42 sidebar entries. On a phone or a narrow window, under 768 pixels wide, the sidebar becomes a drawer. Click the button **Open navigation** to open it. Pick a screen, or click outside it, to close it.

**Which screens you can see** You hold the roles of the account you signed in with. The account made on First Setup holds all five roles, and sees all 42 entries. A person with fewer roles sees fewer entries. Nineteen entries are shown to everyone signed in: Manual, First Setup, Live, Facility, Control Room, Channels, CG Board, Schedule, Program Guide, Assets, Contributors, Review queue, Summary review, Publish, Playback policy, Analytics, Readiness, Alerts and Federation. The other twenty-three need at least one of the roles marked below.

| Screen | Setup admin | Meeting operator | Records clerk | Publish operator | Support admin |
|---|---|---|---|---|---|
| Staff accounts | yes | - | - | - | - |

| Screen | Setup admin | Meeting operator | Records clerk | Publish operator | Support admin |
|---|---|---|---|---|---|
| Backup and restore | yes | - | - | - | yes |
| Import station | yes | - | - | - | - |
| Network access | yes | - | - | - | yes |
| Control Room Setup | yes | - | - | - | - |
| Station Profile | yes | yes | - | - | yes |
| Cable Com-missioning | yes | - | - | - | yes |
| AI Models | yes | yes | - | - | - |
| Custom Fields | yes | - | - | - | - |
| Paywall | yes | - | - | - | - |
| Remote Contribution | yes | yes | - | - | yes |
| Other stations | yes | yes | - | yes | yes |
| CG Designer | yes | - | - | yes | yes |
| Auto-schedule | yes | - | - | yes | yes |
| Recording | yes | yes | - | - | yes |
| Missing Media | - | yes | - | yes | yes |
| Media Lifecycle Settings | yes | - | yes | yes | - |
| Agendas | - | yes | yes | - | - |
| Reports | - | - | - | - | yes |
| EPG Export | yes | - | - | yes | - |
| Underwriting | yes | - | - | yes | yes |
| App Admin | yes | - | - | yes | - |
| Emergency Alerts | yes | yes | - | - | yes |

> **Warning:** The sidebar is a convenience, not a lock. A screen in your sidebar can still refuse you, and a button on it can still fail when you press it. Each screen and each button is checked again when you use it. Roles are added together: one matching role is enough.

These screens open for every role but refuse some things:

| Role | Screen | What happens |
|---|---|---|
| Records clerk, Publish operator | Alerts | The event list does not load; it requires Setup admin, Support admin or Meeting operator. The runtime safe-to-air health indicator is available to every operator role. |
| Records clerk | Federation | **Approve**, **Reject** and **Block** are refused. |
| Setup admin without Publish operator or Support admin | Federation | **Approve**, **Reject** and **Block** are refused. Creating the station key works. |
| Support admin | Alerts | **Save**, **Add destination** and **Delete** are refused. |

**Find a screen by what you want to do**

| I want to | Go to |
|---|---|
| Check whether we can broadcast | Readiness |
| Put a recorded video on a channel | Schedule, then **Publish to residents** |
| Set up a repeating weekly slot | Program Guide |
| Record a meeting from a capture card or stream | Recording |
| Run a live meeting | Live, then Channels |
| Upload or find a video | Assets |
| Check caption lines | Review queue |
| Approve an AI summary | Summary review |
| Decide what residents can see | Publish |
| See audience numbers | Analytics |
| Read the manual | Manual |

> **Known issue (beta.11):** Some screens carry a different name at the top of the page than in the sidebar. If you cannot find a heading, look at this list.
>
> | Sidebar name | Page heading |
> |---|---|
> | Readiness | Safe to broadcast |
> | Alerts | Alerts & monitoring |
> | Federation | ActivityPub federation |
> | CG Designer | CG Board Designer |
> | Recording | Scheduled recording |

> Contributors Contributor submissions
>
> Some on-screen messages also say "System Health" when they mean the **Readiness** screen.

> **Note:** Schedule, Program Guide, Auto-schedule and Recording use the station's configured time zone. The browser may show other timestamps in its own zone; Chapter 3 identifies the zone used by each time-entry screen.

#### Read the colors and status words

CivicCast uses five phrases to answer "can I act on this?". Each has a color. Wherever you see one, it means the same thing.

| Phrase | Color | What it means |
|---|---|---|
| **Ready** | Green | The required checks passed. |
| **Check before meeting** | Amber | Something optional or recoverable needs attention. |
| **Do not broadcast yet** | Red | A required check failed for tonight's broadcast. |
| **Not set up yet** | Amber | An optional provider or feature has no sign-in details or proof yet. |
| **Needs IT help** | Red | The next step needs administrator, certificate, database or service work. |

A channel's outgoing feed has its own words. **On air** is green. **Needs attention** is red. The others are amber: **Showing slate**, **Starting**, **Stopping**, **Finishing current item**, **Changing source** and **Stopped**. A feed that is not on air during a meeting is amber on purpose.

Other words you will meet: **Not run yet** (a check has not been run), **Undeliverable** (a message could not be delivered), **Live on the portal**, **Archive pending**, **Archive verified**, **Waiting for media**, **Needs changes**, **Under review**, **Draft**, **Publishing**, **Complete** and **Needs action**. Each screen's chapter explains the words on that screen. The statuses appendix (Statuses and what they mean) lists them all.

> **Known issue (beta.11):** The word **Ready** does not mean the same thing everywhere. On Readiness it means the whole station is ready. In the media readiness panel it means one video's playable copy is ready. On Assets, **Packaged** and **Published** are different words again. Read the screen's own text each time.

> **Known issue (beta.11):** Not every status uses the five phrases. You will see plain lowercase words such as `approval-only` on Federation, `critical`, `warning` and `info` on Alerts, `forced_slate` on Emergency Alerts, and job states such as `arming` and `finalizing` on Recording. Readiness also uses an extra phrase, **Ready with optional items**.

**Messages and confirmation boxes**

- **Pop-up messages** (toasts) appear at the bottom of the screen and close by themselves after 4.5 seconds. Click **Dismiss notification** to close one sooner.
- **Confirmation boxes** are the centered boxes that ask before a risky action. Focus starts on **Cancel**. Pressing Escape or clicking the dim background cancels. The action button names what it does, such as **Stop feed**.
- A **sample setup banner**, a red box reading "CivicCast could not finish first-run sample setup", can appear at the top of any screen except First Setup. It means the sample video or starter schedule item failed, and nothing else. The buttons are **Retry sample setup** (for a Setup admin or Publish operator) and **Dismiss**.
- **Loading this CivicCast screen...** shows while a screen loads. **Page not found** with the text "This operator route does not exist in this build." appears for an address that does not exist. Its buttons are **Manual**, **First Setup**, **Recording**, **Reports** and **Readiness**.

> **Known issue (beta.11):** Pop-up messages appear on only a few screens (Schedule, the trim editor, Program Guide, CG Board and Media Lifecycle). On other screens a successful save or command is silent. If nothing seems to happen, check the screen for a changed value before you press again.

> **Warning:** Not every risky button asks first. For example, **Start Live Stream** on Live, **Approve & put on air** on Channels, **Reject** on a caption line, and **Save automation settings** on Channels act at once. Alerts uses an inline **Delete** then **Confirm delete?**, and Emergency Alerts uses a tick-box for a forced slate. Treat every button as real.

> **Note:** No banner tells you that the console has lost contact with the station. Each screen reports its own loading failure.

#### Open the Manual

1. In the sidebar, open the **Help** section.
2. Click **Manual**.

You should see a page headed **Operator manual** with a contents list on the left. The Manual works without signing in, works without an internet connection, and is the only screen you can open while a new station's recovery kit is waiting to be confirmed.

- To find a section, type in **Filter by section title**. It narrows the contents list to sections whose

*title* contains your words. It does not search the text inside sections. If nothing matches you see `No section title matches "<text>".`
- Click a contents entry to jump to that section.
- **Report a beta issue** at the bottom of the sidebar opens this manual at Report a beta issue.

> **Note:** The Manual screen displays an offline copy assembled with the release. Check the version on this manual's cover if you are unsure which operating guide you have.

#### Use the keyboard or a screen reader

- The first stop when you press **Tab** on a fresh page is a link **Skip to main content**. It appears only while it has focus. Press Enter to jump past the top bar and sidebar.
- Every control you can reach with Tab shows a visible outline.
- After you change screens by clicking or pressing a key, focus moves to the main area of the new screen. It does not move on the first page load.
- On a narrow window, the **Open navigation** drawer keeps Tab inside it, and Escape closes it.
- The sidebar is labelled **Primary navigation**, and the screen you are on is marked as the current page. Each section header is labelled **Show <name> navigation** or **Hide <name> navigation**.
- Error boxes, toasts and the on-air banner on Readiness are announced to screen readers.
- The theme button switches between light and dark.

> **Known issue (beta.11):** The theme resets to light every time you reload the page and does not follow your computer's own dark-mode setting. The browser tab always reads "CivicCast Operator" on every screen, so look at the heading on the page to see where you are.

**The resident portal** The public portal has a hidden link **Skip to main content** that appears on the first Tab press. Its header has links **Home**, **Recordings** and **Schedule**, with the current one marked. After you change pages, focus moves to the new page's heading. Tap targets are at least 44 pixels tall. The portal is English only and always uses a dark look.

#### Set up a brand-new station

Do this once, on the station computer, when CivicCast is installed and nobody has set it up. The person who does it becomes the station's first admin and holds all five roles.

**Before you begin:** choose the person who will be the admin. Have a printer, or a safe place to save a file. Plan to finish in one sitting, because you cannot leave the recovery kit before you confirm it.

1. Open the console as described in Open the console. You see **First setup** and a card named **Durable storage**.
2. Click **Prepare storage** on the **Durable storage** card. It reads "Preparing…" and then shows **Storage ready** and "CivicCast has a local database for meeting records, captions, summaries, and subscriptions." Until then the page says "Create the first admin after storage is ready." If the card already shows **Storage ready** when you arrive, there is nothing to click; go on to step 3.
3. Fill in the form that appears:

- **Station name**: the name residents will recognize.
- **Admin display name**: the person responsible for setup and recovery.
- **Admin username**: a local sign-in name for the first admin.
- **Admin password**: at least 12 characters. A counter shows how many you have typed.
- **Confirm admin password**: type it again.
- **Where will you keep the recovery kit?**: a note for your records, such as "printed, stored in the clerk safe". It is not a file path, and nothing is saved there.
- **Resident portal URL**: optional for rehearsal. Set it before public launch.

4. Click **Create first admin**. The button stays grey until the required boxes are valid.
5. Read the green **Recovery kit ready** panel. It shows your admin username, your admin password and eight emergency recovery codes. It says: "This screen and the saved/printed copy are the only place your admin password and these codes are ever shown together."
6. Click **Print kit** to open your print dialog, or **Save kit** to download a text file named `civiccast-recovery-kit-<kit id>.txt` to your Downloads folder.
7. Tick **I have saved or printed this kit — the admin password and the recovery codes — and stored it away from this computer.** The box stays grey until you have used **Print kit** or **Save kit**. After **Print kit**, it unlocks when the print dialog closes, whether you printed or cancelled.
8. Click **Continue to the console**. It reads "Recording confirmation..." while it works.

You should see a green **Setup complete** card, a **First-run defaults** card, and a block of setup tools.

> **Warning:** The saved kit file contains the admin password in plain text, together with all eight codes. Keep it, or the printout, where only authorized people can find it. Do not email it, and do not leave it in a folder that syncs to the internet.

> **Warning:** Until you confirm the kit, `/setup` holds you on First Setup. Every sidebar entry except **Manual** is grey, **Sign out** is disabled, other console addresses return to First Setup, and your browser warns before you close the page. The kit lives in that browser tab. The station can never show the codes again. If you return to `/setup` without having confirmed, a warning named **Recovery kit never confirmed** appears there with a button **I found the kit — it is stored safely**. That button only records your word; it does not check anything.

**What a new station starts with** You did not choose these. A new station starts with three channels (public, education and government), the default channel government, the time zone "local", **Test mode**, a sample video, and a starter schedule item. The sample content is made in the background after you create the admin. The **First-run defaults** card shows the channels, the time zone, the mode, the storage folders, and whether sample content and the initial schedule are enabled.

> **Known issue (beta.11):** The **First-run defaults** card shows the starting channels, time zone, mode, sample content and initial schedule, but this page has no control to change **Test mode** or disable those defaults. Before a public meeting, remove the sample video and starter schedule item if you do not want them to appear. See Chapter 10 and Before the meeting.

**The setup tools** Below **Setup complete** are four tools for a Setup admin: **Camera or test media** (six choices, with **Bundled sample video** selected first), **Backup destination**, **Storage and viewing estimate**, and **Provider setup** (cloud storage, archive and notice providers). Part II explains each one. If you do not hold the Setup admin role, the provider boxes are disabled and a warning says so.

> **Note:** **Verify backup** writes, reads and deletes a small test file in the folder you name, to prove CivicCast can write there. It is a check of the folder. See Running it day to day for how backups are made.

#### If it did not work

| What you see | Cause | What to do |
|---|---|---|
| A browser error page instead of the console | CivicCast is not running, or the browser is not on the station computer. | Ask your IT person to start it. See Troubleshooting matrix. |
| "Invalid admin username or password." | The username or password is wrong. | Check the kit and retype both boxes. If the password is lost, use a recovery code. |
| "Invalid staff username or password." | The staff credentials are wrong, or the account is disabled. | Retype your credentials at `/login`. Ask a Setup admin to check your account or reset its password. |
| "Too many staff sign-in attempts. Wait and try again." (possibly followed by a retry time) | Too many failed staff sign-ins. | Wait for the stated retry time, then use the correct credentials at `/login`. |
| "Invalid recovery code or admin username." | The code or username is wrong, or the code was already used. | Try another unused code. A wrong guess does not use up a code. |
| "Too many sign-in attempts from this station. Wait N seconds, then try again with the correct password, or use a printed recovery code." | Too many wrong guesses. | Wait the time shown. A correct password or recovery code still gets through. |
| "Too many failed attempts to authenticate with the staff API. Wait N seconds, then try again." | Too many requests with a bad sign-in. The station's default is 10 failures in 60 seconds. | Wait, then try again. "Staff API" means the station's staff-side service. |
| "The station is cooling down after too many requests" | The same slowdown, shown on First Setup. | Click **Try again**. The sign-in form on that card still works. |
| **You were signed out**, "This browser's console session is no longer valid..." | Your session expired after 12 hours, an administrator revoked it, or the station's sign-in state was reset. | Sign in again. A Setup admin can review and revoke station sessions in **Staff accounts**. |

| What you see | Cause | What to do |
|---|---|---|
| "This browser's console sign-in is no longer valid" | The station rejected the sign-in this browser still held. | Click **Sign in again**, then use `/login` for a staff account or `/setup` for the first administrator. |
| "First setup can only be done from the station computer itself" | You opened first setup or first-admin recovery from another computer. | Open `/setup` on the station computer, or inside a remote session on it. Staff accounts use `/login` from the approved staff network when configured. |
| "Could not read setup state." with more text | The station did not answer properly. | Wait a moment and reload. If it repeats, tell your IT person. |
| "Sign in with the local station admin account, then try again." | The request carried no sign-in. | Sign in on First Setup. |
| "This action requires one of these CivicCast roles: …" | Your account does not hold a role the action needs. | Ask your admin. See below. |
| "Staff identity is required for this action." | You are signed out. | Sign in. |
| "Could not verify your staff identity (…)" | The screen could not check who you are. | Sign in again at `/login` with your staff account, or use `/setup` if you are the first administrator. |
| "Durable storage is not ready." | The station's database has not been prepared. | Click **Go to Setup**, then **Prepare storage**. |
| **Page not found** | The address does not exist in this build. | Use a button on the page, or the sidebar. |

The role message lists raw role names. They map to the names in the console like this:

| Raw name | Console name |
|---|---|
| `setup_admin` | Setup admin |
| `meeting_operator` | Meeting operator |
| `records_clerk` | Records clerk |
| `publish_operator` | Publish operator |
| `support_admin` | Support admin |

> **Known issue (beta.11):** The message "First setup can only be done from the station computer itself" also appears for first-admin sign-in and recovery when `/setup` is opened remotely. Those `/api/setup/*` actions remain local-only. Staff use `/login` and, when configured, the CIDR-restricted staff hostname.

> **Note:** Setup admins can add staff, assign roles and disable access in **Staff accounts**. See the staff accounts guide for account and session management.

**Related**

- What CivicCast is and who uses it
- Before the meeting
- Running the meeting
- When something looks wrong
- Roles and permissions

## Before the meeting: schedules, agendas, and recording schedules

This chapter covers the work you do before a meeting starts and before a program airs. You will put programs on a channel's schedule, repeat a program every week, let CivicCast fill a time of day by rule, set up automatic recording of a meeting, publish an agenda that residents can read next to the video, and review programs that residents and community producers send in. Station staff with the publish operator, meeting operator, or records clerk role normally do this work.

### Before you start

You need to be signed in to the operator console (Chapter 2 shows how). The left-hand menu groups the screens in this chapter in two sections: **Run Meeting** (Schedule, Auto-schedule, Program Guide, Recording) and **Review Records** (Contributors, Agendas).

What you can do on each screen depends on your role. The table shows who can load its data and who can change things. Sidebar visibility is separate: Schedule, Program Guide and Contributors appear for every signed-in person. Your station administrator gives out roles (Appendix: roles lists all of them).

| Screen | Who can load its data | Who can make changes |
|---|---|---|
| Schedule | Publish operator, setup admin, support admin | Publish operator, setup admin |
| Program Guide | Any signed-in person | Meeting operator, support admin |
| Auto-schedule | Publish operator, setup admin, support admin | Publish operator, setup admin |
| Recording | Setup admin, meeting operator, support admin | Setup admin, meeting operator |
| Agendas | Records clerk, meeting operator | Records clerk, meeting operator |
| Contributors | Publish operator, meeting operator, support admin | Publish operator, meeting operator |

> **Known issue (beta.11):** The menu shows some screens to roles that cannot use them. A meeting operator or records clerk who opens **Schedule** sees a red box, "Could not load schedule." A support admin can open **Contributors** but every button there fails. A publish operator who fills in the form on **Program Guide** is told "This action requires one of these CivicCast roles: meeting_operator, support_admin." only after pressing **Add to guide**. If a screen refuses you, ask your station administrator which role you need.

You also need at least one video in the library, and at least one channel set up on the station. Videos are managed on the **Assets** screen (Chapter 5).

> **For IT staff:** how roles map to people and how channels are created is in Part II, Configuring the station.

#### Three words you must know: Scheduled, Published, Cancelled

Every program you place on a channel is an *item* on the schedule. An item is always in one of three states. These words are used on several screens and they do not mean what they sound like, so read this section first.

| State | What it means | Does it air? |
|---|---|---|
| **Scheduled** | You placed it, but nobody has approved it to air. It is a draft. | No |
| **Published** | It has been approved to air. It will play when its start time arrives. | Yes |
| **Cancelled** | It was withdrawn. It will not air. | No |

The program that actually plays on a channel is built only from **Published** items. A **Scheduled** item never airs by itself.

The diagram shows the two ways an item gets onto the air. If you add an item yourself on **Schedule** or **Program Guide**, it starts as Scheduled, and someone with the publish operator or setup admin role must approve it by pressing **Publish to residents**. If **Auto-schedule** adds the item, it starts as Published and there is no separate approval step. The section on Auto-schedule explains why that matters.

> **Warning:** Do not assume a program is on the air because it appears on a schedule list. Look for the word **Published** (on the Schedule screen, a Scheduled program shows "Not yet visible to residents" and a Published one shows "Visible to residents").

**What residents see** Residents see two different lists, built from two different sources.

- **Coming up** on the portal home page lists programs placed with **Schedule** (or Contributors) that are **Published** and start in the future. It never lists a Scheduled item.
- **Channel schedule** (a page of the portal, see the example below) lists the next 72 hours of airings that came from **Program Guide** recurring slots. It lists an airing as soon as the guide has built it, whether or not anyone has published it.

*Figure: the Channel schedule page that residents see.*

> **Known issue (beta.11):** The Channel schedule page can show an airing that will not air, because it reads the guide's own list, not the published state. The guide also keeps listing an airing if you cancel that item on the Schedule screen, because nothing updates the guide when an item is cancelled. Items you add directly on the Schedule screen never appear on the Channel schedule page at all; they appear in **Coming up** after they are published. The two pages do not explain the difference to residents.

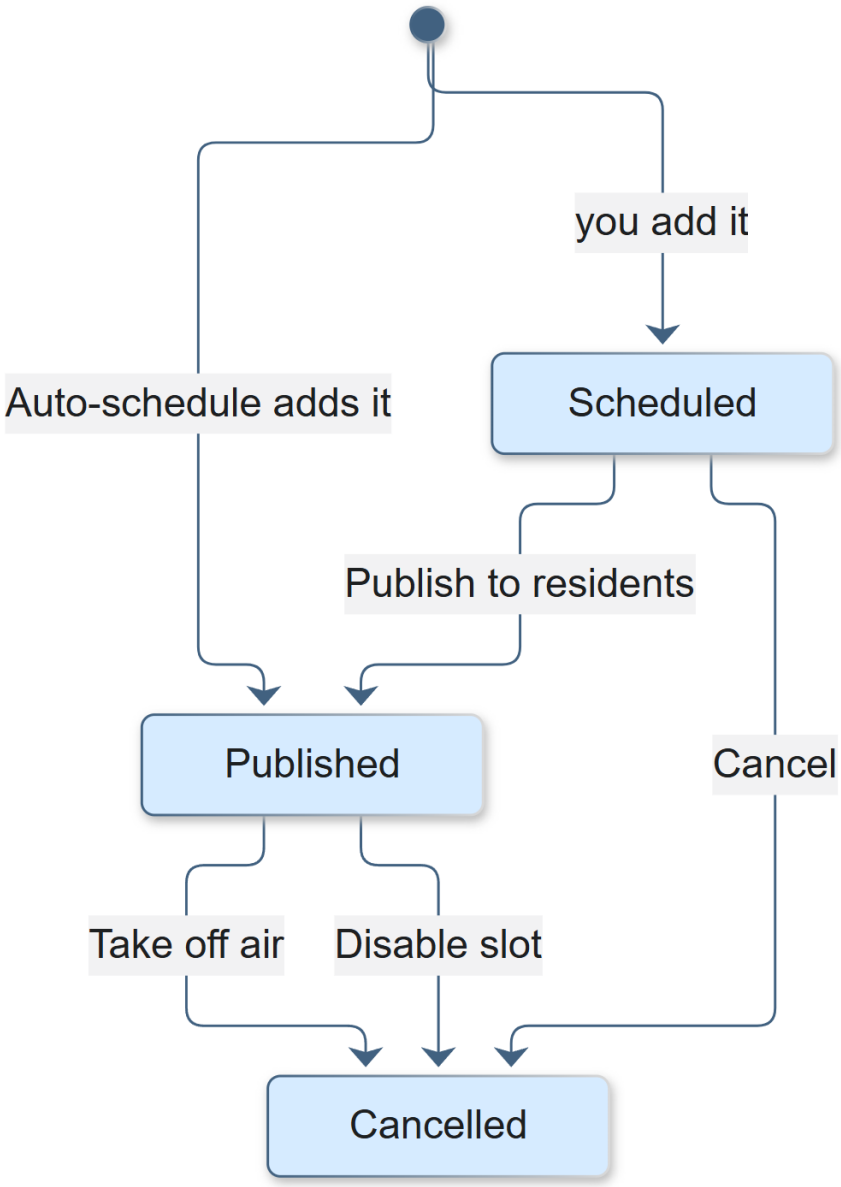
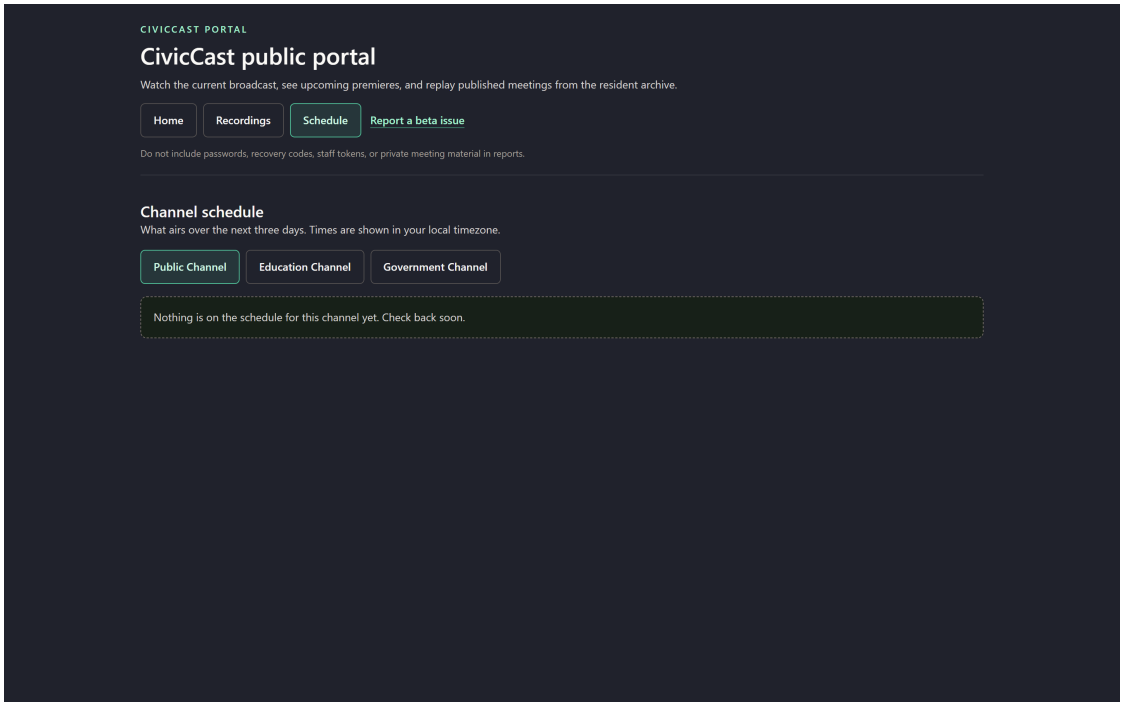



Figure: the life of a schedule item. Items you add by hand start as Scheduled. Items that Auto-schedule adds start as Published.




The resident portal's Channel schedule page, which lists the next three days of programs for one channel.

#### Which time zone are you typing in?

Schedule, Program Guide, Auto-schedule and Recording use the station's time zone. Set a named zone, such as `America/Denver`, in **Station Profile > Timezone**, and check the zone shown beside the form before saving. These times do not depend on the computer you use to open the console.

| Where you type | Time zone it uses |
|---|---|
| Schedule: **Start at** | The station's time zone. |
| Program Guide: **First airing**, **Repeat until (optional)** | The station's time zone; recurring programmes keep that local clock time. |
| Auto-schedule dayparts: **Start**, **End** | The station's time zone, set in the **Timezone** box on the **Station Profile** screen (the First Setup form has no such box). If it is still "local" or was never set, UTC. |
| Recording: start time, weekly time and weekday boxes | The station's local date, time and weekday. |
| Contributors: producer's **Requested air date** | The producer's own computer, with no zone saved (see Known issue under Contributors). |

*UTC* (Coordinated Universal Time) is the world's reference clock. CivicCast still uses UTC internally for timestamps; you do not need to convert station scheduling forms to UTC yourself.

> **Daylight-saving changes:** A recurring start in the hour skipped when clocks move forward is omitted for that date. A clock time that occurs twice when clocks move back runs once, at its earlier occurrence. A one-time booking in a nonexistent hour is rejected with an explanation. Check the preview when booking near a clock change; ordinary recurring programmes keep their station-local time without seasonal edits.

Existing weekly recording schedules created with the older UTC-only form retain their UTC timing; CivicCast does not silently move them on upgrade. These schedules are identified as UTC. New weekly recording schedules use the station's time zone. Review an older schedule before replacing it with a station-local schedule.

Changing **Station Profile > Timezone** does not move airings or captures that CivicCast has already planned. Those commitments keep their original timestamps; the changed zone applies when the recurring schedule is extended beyond that existing horizon. Review already-planned entries separately if they must move.

#### Put a recording on a channel once

Use the **Schedule** screen to place one recording on a channel at one time. This creates a *premiere*: a recorded video placed on a channel at a start time, for a length of time.

1. In the left menu, under **Run Meeting**, click **Schedule**. The page heading is "Schedule".
2. Click **New scheduled item** (top right). A panel opens on the right.
3. Under **Mode**, click **Premiere**.
4. In the **Asset** drop-down, choose the video. Each entry reads "title · asset id".
5. In the **Channel** drop-down, choose the channel by its display name.
6. In **Start at**, type the date and time in the station's time zone. Check the local-time preview and zone before saving.
7. In **Duration (min)**, type the length in whole minutes. It starts at 60. The largest allowed length is 14 days.
8. In **Notes (optional)**, type anything you want recorded. The box says the note is "visible in the audit log" (the station's record of who did what).
9. Click **Schedule premiere**. The button reads "Scheduling…" while it works.

You should see the panel close and a message, "Scheduled.", with a line such as "Premiere · City Council · Tue, Oct 6, 7:00 PM".

> **Warning:** The item you just made is **Scheduled**. It is not on the air and residents cannot see it. Go on to the next task, "Approve a program to air".

> **Note:** Embargo release is unavailable in this build. Use **Premiere**, then publish the scheduled item to residents separately. An Embargo item is left out of the channel's program and **Coming up**, and the approval check refuses it with "embargo entries publish at a single moment and cannot be committed to air".

> **Known issue (beta.11):** The **Asset** drop-down lists only videos in the Validated state. A video that CivicCast recorded itself (a live meeting or a scheduled recording) is in the Recorded state and is **not offered**, even though the approval step accepts Recorded videos. If the list is empty you will see "No validated assets." with the advice "Upload and validate an asset in the Assets tab first." The console has no action that changes a Recorded video into a Validated one. A video you upload on the **Assets** screen becomes Validated, so uploading the file again makes it appear. Auto-schedule (below) accepts Recorded videos.

> **Tip:** The yellow box in the panel reads "Timezone check". During the weeks when clocks change, compare the time with your station calendar before you press **Schedule premiere**.

**Read the Schedule screen** The top of the screen shows the week and the station time zone used for its dates and times.

1. Click **week** to see a grid of the week, or **list** to see every item as a row. On a phone the screen opens in the list.
2. Use **‹** (previous week), **Today**, and **›** (next week) to move through weeks.
3. In the list, each row shows the time, a mode label, a state label (Scheduled, Published or Cancelled), the title, the channel, and, for a premiere that is Scheduled or Published, either "Not yet visible to residents" or "Visible to residents". In the week grid, Cancelled items are greyed and struck through; in the list they only carry the Cancelled label.

The list shows every item, not only this week's. Clicking an item in the week grid switches to the list; it does not open a detail panel.

> **Note:** The Schedule screen says, "Conflicts are checked when you submit." An overlapping item can be entered in the form, then rejected by the server when you submit it, with "Time slot conflicts." and the conflicting item's ID, channel and start time. Choose a different time or channel, or cancel the conflicting item, then submit again.

#### Approve a program to air

A program on the Schedule screen airs only after someone with the publish operator or setup admin role approves it. The approval first runs a safety check. This is the real step that puts a program on the air.

1. On **Schedule**, click **list**.
2. Find the row that says "Not yet visible to residents".
3. Click **Publish to residents** on that row. The row says "Checking…" for a moment.
4. Read the review panel. At the top it says **Safe to air** or **Not safe to air yet**, with the start time and length. It also shows:
    - a line about problems with the video file, if there are any;
    - "Clashes with N other program(s) already scheduled:" followed by a list, if another program

overlaps this one;
- "Dead-air gap before this program (it can still air):" followed by a list, if the channel has nothing scheduled just before this program. *Dead air* means time when nothing is playing. A gap is for information only and does not stop the approval.

5. If the panel says **Safe to air**, click **Publish to residents** in the panel. The button reads "Publishing…". Click **Cancel** in the panel instead if you want to back out.

You should see the message "Visible to residents." with the program title, and the row now reads "Visible to residents". The item is **Published**. CivicCast also tells the channel's playout engine to read the schedule again, and can start a stopped channel unless an intentional operator stop is active. After an intentional stop, press Start explicitly to resume the feed.

> **Warning:** The final **Publish to residents** click has no further confirmation. It approves the program to air on the channel, not only to appear on the portal.

If the panel says **Not safe to air yet**, the approve button stays grey. Fix the reason shown (move or cancel the clashing program, or fix the video) and start again.

If your role cannot approve, the **Publish to residents** button is grey and a note explains that you can view the schedule but publishing needs the publish operator or setup admin role.

> **Known issue (beta.11):** The words "to residents" hide what this button does. It is the approval to air on the channel. The Channels screen has the same approval under another name, **Commit programs to air** with **Review & prepare** and **Approve & put on air** (Chapter 4). Either place works.

> **Known issue (beta.11):** The screen shows "Visible to residents." whenever the approval was saved, even if the message to the playout engine failed. The approval stays saved, and the engine picks the schedule up on its own cycle. The Channels screen shows "Couldn't reach the engine" for that case.

#### Cancel a scheduled item

1. On **Schedule**, click **list**.
2. On the row of a **Scheduled** item, click **Cancel**.
3. In the box titled `Cancel scheduled item for "<name>"?`, click **Cancel scheduled item**. (The other button, **Cancel**, closes the box and changes nothing.)

You should see the message "Cancelled." and the row's state label change to Cancelled. The row's **Cancel** button goes away.

> **Warning:** This cannot be undone. The box says you would need to create a new scheduled item to air it again.

> **Known issue (beta.11):** A **Published** row has no **Cancel** button on this screen. To take a program that you approved by hand off the air, use **Take off air** on the Channels screen (Chapter 4); it asks for a reason. Programs that Auto-schedule placed have no approval record, and the console has no button to remove one. The station's programming interface can cancel a Published item; ask your IT person (Part II, Integrations).

#### Keep a channel running continuously

For a repeating rotation of two recordings:

1. Add both recordings to **Schedule** on the same channel. Give each item its intended duration and put the second immediately after the first: its start is the first item's start plus its duration.
2. Use **Publish to residents** or the channel's **Commit to Air** control to approve both items. Items left **Scheduled** do not air.
3. Ask the station administrator to set `CIVICCAST_SCHEDULE_LOOP=1` in the Windows service environment, then restart the service between programs. The default is a finite schedule; loop mode repeats the published sequence. See service environment settings.
4. Watch the first transition on the channel. The loop preserves gaps between schedule entries, so use contiguous entries for uninterrupted programming.

For an automatically maintained schedule, use **Auto-schedule** below: create a saved search, a daypart and an enabled rule, then **Preview** and **Compile now**. The worker extends the rolling schedule hourly unless `CIVICCAST_AUTOSCHEDULE=off`. Daypart times use the station's configured time zone.

In **Reports**, planned UTC is the schedule time; actual on-air UTC comes from the engine's source-transition proof. The difference is shown as early, on time or late. VOD playback preroll is separate; a preroll time is not stored as an additional as-run timestamp.

#### Repeat a program on a schedule (Program Guide)

Use **Program Guide** when a recording should air again and again, for example every Friday night. You place a recording on a channel as a *recurring slot*: a recording, a channel, a start time, and a rule for how often it repeats. A background job turns each slot into real schedule items for the next 72 hours.

> **Warning:** The items the guide builds are **Scheduled**, not **Published**. Adding a slot does not put anything on the air. You must approve each one on the **Schedule** screen (task "Approve a program to air"). A station that adds a weekly slot and walks away gets nothing on the air.

1. In the left menu, under **Run Meeting**, click **Program Guide**.
2. In the **Channel** drop-down at the top, choose the channel.
3. Click **Add to guide** (top right). A panel opens, headed "Program guide" and the channel.
4. In **Recording**, choose the video. The entries read "title · asset id".
5. Under **Repeats**, click one card:
   - **Once**: "Airs a single time at the start time."
   - **Daily**: "Airs every day at this time."

- **Weekly**: "Airs every week on this weekday." (This is chosen when the panel opens.)
- **Weekdays**: "Airs Monday through Friday."

6. In **First airing**, type the date and time in the station's time zone. Check the date, time and station zone shown with the form.
7. If you chose Daily, Weekly or Weekdays, you may fill **Repeat until (optional)** with the last date it may air. Leave it blank to repeat with no end.
8. Either keep **Use the recording's own length** ticked, or untick it and type **Duration (min)** (at least 1).
9. In **Guide title (optional)**, type a title if you want residents to see a different title than the recording's. It can be up to 200 characters.
10. Click **Add to guide**. The button reads "Adding…".

You should see the message "Added to guide." with a line such as "City Council · Weekly" (the line shows the guide title if you typed one, otherwise the recording's ID). The new slot appears under **Recurring slots**. Airings that fall in the next 72 hours are built at once and appear under **Next 7 days**, grouped by day, with the status **Scheduled**. Later airings are added as they come inside that 72-hour reach.

The guide is rebuilt in the background. By default it looks 72 hours ahead and runs about every five minutes. The **Refresh guide** button builds it immediately, for every channel, not only the one shown.

> **For IT staff:** the background job, its 5-minute interval and its 72-hour reach are set by environment variables; see Part II, Running it day to day.

The **Weekdays** rule uses the station's Monday-through-Friday calendar. A Friday evening remains Friday even when its internal UTC timestamp falls on Saturday.

> **Known issue (beta.11):** The status in **Next 7 days** stays **Scheduled** even after you publish the item, so this list cannot tell you whether a program is approved. Check **Schedule** for that. Items you added directly on the Schedule screen are also listed here, with the status shown as the lower-case word `manual` and the detail "Scheduled directly (no recurring slot)."

> **Known issue (beta.11):** Like the Schedule screen, the **Recording** drop-down lists only Validated videos. A video that CivicCast recorded is not offered (see the Known issue under "Put a recording on a channel once").

> **Known issue (beta.11):** You cannot change a slot after you add it, even though the station's programming interface allows it. To change one, disable it and add a new one.

**Fix a skipped airing** When the guide cannot place an airing, the row in **Next 7 days** shows an amber status.

| Status | What happened | What to do |
|---|---|---|
| **Skipped · conflict** | Another program overlaps that time on the channel. The detail text under the title names it. | Cancel or move the other program on **Schedule**, then place this airing by hand (see below). |
| **Skipped · not playable** | The recording is missing, is not Validated or Recorded, has no file on the station, or has an unknown length. | Fix the recording on **Assets**, then place this airing by hand (see below). |

1. Read the status and the detail text under the title.
2. Fix the cause as shown in the table.
3. On **Schedule**, click **New scheduled item** and place the recording at that airing's time (Put a recording on a channel once), then approve it.

The guide records a skipped airing once and never looks at it again. We read the code: the program that builds the guide ignores any airing time it has already recorded, skipped or not. Fixing the cause and clicking **Refresh guide** therefore does not turn that skipped airing into a scheduled one. Later airings of the same repeating slot are still built as normal. **Refresh guide** shows a message such as "Guide refreshed." with counts like "N scheduled · N conflicts · N not playable"; those counts cover only airings that were newly recorded.

### Stop a repeating program

1. On **Program Guide**, find the slot under **Recurring slots**.
2. Click **Disable**.
3. In the box titled `Disable "<name>"?`, click **Disable slot**. (**Cancel** closes the box.)

You should see "Slot disabled." with a line such as "City Council · 3 future airing(s) cancelled", and the slot shows a **Disabled** label.

> **Warning:** Disabling a slot cancels every future airing it built. It cancels them even if you already approved them. Airings that already played, or are playing, are not touched.

> **Note:** To schedule this program again after disabling its slot, add a new slot with **Add to guide**.

#### Fill a channel by rule (Auto-schedule)

Use **Auto-schedule** when you want CivicCast to pick recordings for a time of day by itself, for example "the newest council meeting every weeknight at 6 PM". You build three things, then connect them:

- A **saved search**: a named description of which recordings qualify, such as "meeting body is City Council".
- A **daypart**: a repeating window of time on a channel, such as weeknights 18:00 to 22:00.

- A **rule**: connects one saved search to one daypart, and says how to pick.

For each matching day in a rule's window, CivicCast places **one** recording. It starts at the start time of the daypart and uses the recording's own length. If anything is already scheduled anywhere inside that day's daypart, that day is left alone.

> **Warning:** Programs placed by Auto-schedule are written as **Published**. They are approved to air with no separate approval step. Reviewing the **Simulate** preview is the only check you get. In the standard configuration, enabled rules also compile hourly, so a saved rule can put programs on the air without anyone pressing a button.

You need the publish operator or setup admin role to change anything here. A support admin can look and run **Simulate**, but the **Add**, **Edit**, **Delete** and **Compile now** controls do not appear.

### Create a saved search

1. Under **Run Meeting**, click **Auto-schedule**.
2. In **Saved searches**, click **Add saved search**.
3. Type a **Name**, for example "Recent council meetings".
4. Fill any of the other boxes you need. A blank box means no limit:
    - **Meeting body (exact)**: must match the recording's meeting body text exactly, including capital letters. Example: City Council.
    - **Title contains**: a word that must be in the title. Example: budget.
    - **Min length (minutes)** and **Max length (minutes)**.
    - **Include states**: tick the kinds of recording that may be picked. "Validated" and "Recorded" are ticked to begin with. Leave them.
    - **Newest published first**: ticked to begin with. It decides whether the newest or the oldest published recording comes first in the results, which matters for the **First match** pick (see "Create a rule").
5. Click **Create saved search**. The button is grey until the **Name** is filled.

To change or remove one later, click **Edit**, or click **Delete** and then **Confirm delete?**. If rules use the search, the confirm step adds the red line "Used by N rule(s) — deleting it stops them scheduling."

### Create a daypart

1. In **Dayparts**, click **Add daypart**.
2. In **Channel**, type the channel's ID exactly. The placeholder shows `public`.
3. Type a **Name**, for example "Prime time".
4. Set **Start** and **End**. They begin at 18:00 and 22:00. Use 24-hour times in the **station's time zone**. The note says "00:00 = midnight (end of day). An end before the start wraps past midnight."
5. Tick the **Days** the daypart applies to. Monday to Friday are ticked to begin with.
6. Click **Create daypart**.

> **Known issue (beta.11):** The **Channel** box is free text, unlike the drop-downs on Schedule and Program Guide. A mistyped channel ID makes a daypart for a channel that does not exist. In testing we could not confirm whether the station refuses that. Copy the ID from the Channels screen.

> **Known issue (beta.11):** The line under **Dayparts** says times follow "CIVICCAST_STATION_TZ; UTC if unset". In plain words: the times use the station's time zone, which is set in the **Timezone** box on the **Station Profile** screen (the First Setup form has no such box). A new station starts with the value "local", which CivicCast treats as UTC, so "18:00" means 18:00 UTC until someone sets a real zone name there.

### Create a rule

1. In **Auto-schedule rules**, click **Add rule**.
2. Type a **Name**, for example "Fill prime with council".
3. Choose a **Pick strategy**:
    - **Newest first**: the recording with the most recent published date.
    - **First match**: the first recording the saved search returns.
    - **Random**: any one of the recordings that match.
4. In **Saved search**, choose **Choose...** then the search.
5. In **Daypart**, choose the daypart. The entries read "name (channel)".
6. In **Rolling window (days, 14–60)**, type how many days ahead to fill. It starts at 30. A number outside 14 to 60 shows the red message "Rolling window must be a whole number from 14 to 60 days."
7. In **No-repeat window (days)**, type how many days must pass before the same recording may be picked again. 0 (the starting value) allows repeats.
8. Click **Create rule**.

### Preview and compile

1. On the rule's card, click **Simulate**. Nothing is written. You see "Would schedule N of M upcoming slots." and up to 30 lines, each with a date, a title and a label:
    - **Will air**: the day is open and a recording was picked.
    - **Already scheduled**: something is already scheduled in that daypart.
    - **No eligible video**: no recording matches the saved search.
    - **No usable duration**: the recording picked has no length.
2. If you are happy, click **Compile now** in the **Compile schedule** card. There is no confirmation box.

You should see "Published N schedule items across M rules." The programs now show on **Schedule** as **Published**.

### Retire a rule

1. On the rule's card, click **Delete**, then **Confirm delete?**.

> **Warning:** Deleting a rule stops it from placing new programs. It does not remove programs it already placed. The delete changes only the rule, and the screen gives no hint about this. The programs stay Published and will air.

> **Known issue (beta.11):** There is no way on this screen to switch a rule off without deleting it. A **disabled** label exists on rule cards, but nothing here sets it. You also cannot set a rule's priority or its active dates; every new rule is saved with priority 100 and no date limits. If you edit a rule, set it up again with the same care as when you created it.

#### Record a meeting automatically (Recording)

Use **Recording** to capture a live input or a network stream at set times, or to start a capture by hand. A *live input* is a video card in the station computer (SDI, HDMI or NDI). A *network stream* is video that arrives over the network from a camera or encoder (RTSP, SRT, HLS, RTMP or MPEG-TS). The page heading is "Scheduled recording". When a capture ends, CivicCast makes a new library asset in the Recorded state from the file.

You need the setup admin or meeting operator role to make changes. A support admin can look only. A person with any other role sees "Forbidden — scheduled recording is an operator / setup-admin / support-admin surface. Ask your station admin for access."

**Create a recording schedule** A *recording schedule* is a saved plan: one source, one time pattern, one length.

1. In the left menu, under **Run Meeting**, click **Recording**.
2. In the **New schedule** card, type a **Schedule ID (slug)**: a short code name. Use only lowercase letters, digits and hyphens, for example `evening-news`. You cannot change it later.
3. Type a **Name**, for example "Evening news". Two schedules cannot have the same name.
4. In **Source kind**, choose one. Under **Live inputs** are SDI, HDMI and NDI. Under **Network streams** are RTSP, SRT, HLS, RTMP and MPEG-TS. Changing the kind clears the next box.
5. Fill the box that appears next:
   - For **SDI** or **HDMI**, choose the card in **Input ID**. The drop-down lists the capture devices CivicCast found. Click **Inspect capture devices** to look again after you plug a card in.
   - For **NDI**, type the name in **Input ID** (the placeholder is `studio-ndi`).
   - For a network stream, type the address in **URI** (the placeholder is `rtsp://camera.local/stream`).
6. Under **Recurrence**, choose **One-shot** (one capture) or **Weekly**.
7. For **One-shot**, enter the start in the station's local time. For **Weekly**, click **Weekdays**, **Weekend**, **Every day** or **Clear**, tick the station-local days you want, and enter the local time, for example `19:00`.

8. In **Duration (HH:MM:SS)**, type the length with two digits in each part. It starts at `01:00:00`. The minimum is one minute (`00:01:00`). A value such as `1:5:0` is refused; type `01:05:00`.
9. In **Quality preset (encoder profile)**, leave `default`, or type a name your administrator gave you. The screen suggests default, copy, h264-1080p, h264-720p, hw-h264-1080p and hw-h264-720p.
10. In **Loudness regime**, choose how loud the recording should be. The choices are **Inherit station default**, **Copy source audio**, **Cable / ATSC A/85 (-24 LKFS)**, **Broadcast / EBU R128 (-23 LUFS)** and **Streaming (-16 LUFS)**. One line under the choice explains it.
11. Leave **Target series (optional)** blank unless your administrator tells you what to put there. We could not confirm what it does.
12. Keep **Enabled** ticked so the schedule runs.
13. Check the **Next fire** (or **Next 3 fires**) preview in the station's time zone before saving.
14. Click **Create schedule**.

You should see the new schedule appear in the **Schedules** table above the form.

> **Time-zone check:** For a Monday 7 PM meeting, select Monday and `19:00` in the station's zone. Weekly recordings keep that local time across daylight-saving changes. A nonexistent recurring time is skipped; a repeated time runs once at its earlier occurrence. Use **Next 3 fires** to confirm the planned dates.

> **Known issue (beta.11):** After **Create schedule** works, the form is not cleared and no "saved" message appears. Look for the new row in the **Schedules** table. If you click **Create schedule** a second time, the station answers "Recording schedule '<id>' already exists. Use PATCH to update." That means the first click worked.

> **Known issue (beta.11):** "Quality preset" is a free text box and the screen does not know which names are valid. Its note says "Contact your station admin for the full list of available presets." We could not confirm what the station does with a name it does not know.

> **Known issue (beta.11):** If the stream address contains a user name and password, the screen does not warn you. We could not confirm how CivicCast stores or hides it. Ask your IT person before you save such an address.

> **For IT staff:** the capture-device list, recording folders and the runtime switch are described in Part II, Running it day to day.

By default, CivicCast adds a row for each planned capture to the **Recordings** table up to ten minutes before its start time, and prepares it about 30 seconds before the start. Preparing does not record extra video: capture starts at the scheduled time and is bounded by the remaining recording window. If an input stalls or reconnects, the resulting playable media can be shorter than the elapsed wall time; inspect the recording's outcome and media duration. The job moves through these states:

| State | Meaning |
|---|---|
| `scheduled` | Planned, not started |
| `arming` | Getting the input ready |
| `recording` | Capturing now |
| `finalizing` | Closing the file and making the asset |
| `done` | Finished; the asset exists |
| `failed` | Stopped with an error; the **Failure** column says why |
| `skipped` | Not run, for example because the schedule was disabled |

These labels appear in lower case on the screen.

In beta.12, a job prepared ahead of time remains in `arming` until its planned start. The scheduler checks it again on later passes, moves it into `recording` when due, and finishes it at the planned end. A scheduled job should not remain in `arming` throughout its recording window.

If you disable the schedule before capture starts, both scheduled and prearmed jobs become `skipped`; CivicCast also stops a prearmed capture process. Disabling the schedule leaves a recording that has already started running. Use **Stop** on that recording if you want to finish it early.

**Start a recording now** **Record now** starts a capture from a saved schedule right away. There is no one-off capture without a schedule, so create a schedule first.

1. In the **Schedules** table, find the schedule.
2. Click **Record now** on its row. The button reads "Starting…".

You should see a new row in **Recordings** that goes from `arming` to `recording`. The capture lasts for the schedule's **Duration** unless you stop it earlier. There is no success message; the row is the confirmation.

> **Known issue (beta.11):** The empty-screen text says "use Record Now for a one-off capture". The button is named **Record now** and exists only on a saved schedule's row. The button is grey if the schedule is disabled.

**Stop a recording**

1. In the **Recordings** table, find the job in state `arming`, `recording` or `finalizing`.
2. Click **Stop**, then click **Confirm stop**. (**Cancel** backs out.)

If the job was in `recording`, the file recorded so far is finished into an asset. The state goes to `finalizing`, then `done`, and an asset link such as "asset id >" appears. Click it to open the asset. If the job was still in `arming`, nothing has been recorded yet: it ends as `failed` with the reason "Stopped before recording started; capture pipeline never reached the 'recording' phase." and no asset is made.

> **Warning:** Stopping ends the capture for good. The recording stops at that moment.

> **Known issue (beta.11): Stop** is also shown on a job still in `scheduled` state, but the station refuses it ("Cannot stop job '…': state is 'scheduled'; only ['arming', 'finalizing', 'recording'] are stoppable."), and the screen shows nothing, so the click appears to do nothing. To stop a capture that has not started, remove its schedule's **Enabled** tick (see below).

### Change or remove a schedule

- Click **Edit** on the row, change the form, and click **Save changes**. The schedule ID is locked. Click **Cancel edit** to leave without saving.
- To switch a schedule off, edit it, untick **Enabled** and save. This also cancels jobs that are still waiting to start.
- To delete it, click **Delete**, then **Confirm delete**. Earlier recordings stay in history. We could not confirm whether a job that was already planned still starts after you delete its schedule, and the screen does not show delete errors. Untick **Enabled** first.

### Find finished recordings

1. Use the **Recordings** filters: **State**, **Schedule ID**, and **Limit** (1 to 500, 50 to begin with). Click **Refresh** to reload.
2. The screen reloads by itself every 5 seconds while any job is active; the label reads "Live · refreshing every 5 s". Click **Pause** to stop it and **Resume** to start it again.
3. For a failed capture, set **State** to `failed` and read the red text in the **Failure** column.

#### Publish an agenda for a meeting (Agendas)

An *agenda* is the list of numbered items for one meeting. When you publish it, residents see it next to the video on that recording's watch page, and clicking an item with a time jumps the video to that moment. The records clerk and meeting operator roles can use this screen.

An agenda belongs to one recording. Each recording can have only one agenda. Agendas stay private drafts until you publish them.

### Create an agenda

1. In the left menu, under **Review Records**, click **Agendas**.
2. In the **Create new agenda** card, type an **Agenda ID (slug)** such as `council-2026-01`. Use only lowercase letters, digits, hyphens and underscores, starting with a letter or digit.
3. Type the **Meeting asset ID**: the ID of the recording this agenda belongs to. Find it on the **Assets** screen, in small type under the recording's title. Type it exactly, in lowercase.
4. If the agenda also exists as a document, type its web address in **Source doc URL (optional)**. Residents can read it next to the video.
5. Click **Create agenda**.

You should see the new agenda selected. Its card shows the ID, a **draft** label, and "meeting asset: <id>". If you have more than one agenda, the **Pick an agenda** drop-down at the top switches between them.

> **Known issue (beta.11):** The IDs are made-up codes, and a wrong character (for example a capital letter) shows raw technical text instead of a plain message. There is no list of recordings to choose from. We could not confirm whether the station checks that the recording exists.

> **Known issue (beta.11):** If you press **Create agenda** for a recording that already has an agenda, you see "An agenda already exists for (station_id='…', meeting_asset_id='…')." Select the existing one with **Pick an agenda**.

**Add items by hand**

1. In the **Items** section, fill the **Add item** form:
   - **Item ID (slug)**: a code name such as `item-01-call-to-order`.
   - **Order**: the item's position in the list, as a whole number from 0 up.
   - **Number (label, optional)**: what residents see, such as `1.a`.
   - **Title**: the item's name.
   - **Video timecode (seconds, optional)**: the moment in the video where the item starts, in whole seconds. For 1 hour 5 minutes, type 3900.
   - **Doc anchor (optional)**: a place in the source document, such as `#item-1a`.
   - **Notes (optional)**: private to staff. The box says they are not shown to viewers.
2. Click **Add item**.

You should see the item in the **Agenda items** table. Use **Edit** on a row to change it (the button becomes **Save item**, and **Cancel** leaves edit mode). Use **Delete**, then **Confirm delete**, to remove a row.

> **Known issue (beta.11):** The **Order** box starts at 0 every time. If you add a second item without changing it, the station refuses with "Another agenda item already occupies (agenda_id='…', order=0)." Change **Order** to the next free number (0, 1, 2, and so on) for each new item.

> **Known issue (beta.11):** The timecode must be typed as seconds. The screen does not accept hours and minutes.

**Fill the agenda automatically** The **Bulk actions** section has four ways to fill an agenda without typing each item.

- **Sync from chapters** makes an item for each eligible *chapter marker* on the recording: a named point in time added in the trim editor (open it from the recording's page on **Assets**). The result shows the number of source chapters, new items and skipped items with their reasons. Source chapter times are converted to the trimmed playback timeline: a marker at source 32 seconds

with a 2-second trim-in becomes a 30-second agenda link. Markers outside the selected trim are omitted. Existing occupied positions are preserved; repeated synchronization does not duplicate them. If you change a trim after synchronization, review and update the existing agenda times before publishing; re-sync does not overwrite your agenda edits. A zero-new-item result therefore explains whether chapters were absent or their positions were already occupied.

- **Import from doc (paste plain-text agenda, one item per line)**: paste the agenda text and click **Import**. Each non-blank line becomes one draft item. A leading number such as "3.a" or "VII" becomes the **Number**. It reads "Imported N item(s)." and the help line says "Taken literally, one item per line — nothing to review." Lines whose position already has an item are skipped.
- **Or upload a PDF agenda...**: choose a PDF and click **Import PDF**. CivicCast makes a best-effort guess from the text in the PDF (numbered items, ALL-CAPS headings and times). Each item gets a **Confidence** mark: green is 90% or more, amber 50 to 89%, red under 50%. If any item is under 90%, the banner says "Some items were a best-effort guess (see the Confidence marks below) — review them before publishing." A PDF that is a scan of paper, with no text in it, finds nothing: "Couldn't find any recognizable items in that PDF. Try pasting the agenda's text instead."
- **Import from an external agenda system**: look up a meeting in Legistar, PrimeGov, CivicClerk, or a "JS-rendered portal (CivicPlus, Granicus, other)" and import it. See the Known issue below.

> **Warning:** Importing a PDF or an external meeting into an agenda that is already **published** moves it back to **draft**. Review it and publish again.

> **Known issue (beta.11):** The **Import from an external agenda system** block is shown on every station but is **switched off** unless your IT person turns it on. If it is off, **Find meetings** answers in a yellow banner: "Agenda import is not enabled. Set CIVICCAST_AGENDA_SOURCE to 'legistar', 'primegov', 'civicclerk', or 'js_portal' to turn it on." You cannot fix that from this screen. Ask your IT person. "Tenant / site code" is the short name your city has in that system (the screen's example is `longmont`). The JS-portal choice also needs an optional add-on; if it is missing, the screen says "JS-portal runtime: not installed." and **Find meetings** is grey.

> **For IT staff:** turning on agenda import is covered in Part II, Integrations.

#### Publish, unpublish or delete an agenda

1. Check the **Items** table. An agenda needs at least one item before it can be published. Until it has one, **Publish** is grey with the hint "Publish needs at least one item — add one below or sync from chapters."
2. Click **Publish** (the button reads "Publishing..."). The label changes from **draft** to **published**.
3. To take it down, click **Unpublish**. It goes back to **draft** with no confirmation, and residents no longer see it.
4. To remove it, click **Delete**, then **Confirm delete**. The line "Confirming will also delete every item under this agenda." appears first.

> **Warning:** Delete removes the agenda and every item in it. There is no undo. Once the red **Confirm delete** button is showing, the only way to back out is to choose a different agenda in **Pick an agenda**.

> **Known issue (beta.11):** If the station refuses to publish an empty agenda, the message ends with "Add at least one item (DC-1) before publishing." The "DC-1" is an internal code; ignore it.

> **Known issue (beta.11):** The screen does not tell you that changes to items of an agenda that is **already published** go live as soon as you save. Only a PDF or external import reopens it as a draft. Edit a published agenda only when you are sure.

**What residents see** On the recording's watch page, residents see a card headed "Agenda" beside the video. It shows each item's number, title and time. Clicking an item that has a time jumps the video to it. An item with no time shows "—" and cannot be clicked. If you gave a **Source doc URL**, there is an "Agenda document" link, and if the address ends in `.pdf`, the document appears in a viewer. Residents never see your **Notes**. While the agenda is a draft, residents see nothing at all.

> **Known issue (beta.11):** The operator screen does not say where residents see the agenda, and it only links to the public page when the station was built with a public portal address set. Open the recording on the resident portal (Chapter 6) to check how it looks.

#### Review programs sent in by the community (Contributors)

Community producers can send a program to the station through the resident portal. Staff review each one on the **Contributors** screen. The page heading is "Contributor submissions". Producers never get a login to the operator console.

**What a producer does on the portal** Tell producers to open the station's resident portal and scroll to **Submit a program** at the bottom of the home page. They:

1. Fill **Producer name**, **Email**, **Program title** and **Description**. **Organization**, **Tags** (separated by commas) and **Requested air date** are optional.
2. Choose a **Video file**.
3. Click **Send to review**. The button reads "Uploading" while it works.
4. Copy the **Receipt** and **Status token** shown on the page.

Later, the producer pastes both into **Check submission status** and clicks **Check status** to see the state of the submission.

> **Known issue (beta.11):** The receipt and status token are shown only on the page and are lost if the producer reloads it. The page does not say so. Tell producers to copy both before they leave.

> **Known issue (beta.11):** The form shows an agreement summary but has no "I agree" box. Pressing **Send to review** counts as accepting the agreement, in the name typed in **Producer name**. Ask your station's lawyer or administrator whether that is acceptable.

> **Known issue (beta.11):** The form says nothing about file type, size, length or how long review takes. It lets the producer pick any video file. By default the station refuses files over 2 GiB (about 2 gigabytes); the producer then sees "Contributor media exceeds the station's upload limit of …". Your IT person can change the limit. The station also limits the total size of the intake folder and how much one address can send.

> **Known issue (beta.11):** The contributor workflow does not email the producer when the state changes. The producer learns about the state only by using **Check status**.

**Review a submission** You need the publish operator or meeting operator role to press the buttons.

1. In the left menu, under **Review Records**, click **Contributors**.
2. Read the three tiles at the top: **Needs action** (submissions waiting on you), **Total submissions** and **Status notices**.
3. Use the search box ("Search title, producer, or tag…") or the tabs **All**, **Submitted**, **Reviewing**, **Needs changes**, **Accepted**, **Scheduled** and **Declined** to find a submission. The oldest are listed first.
4. Read the card: the title, the producer, the producer's description, the **Media** file name and size, the **Media gate** (the file check; it reads "not run" until you accept, then "passed") and the **Agreement**.
5. Click **Start review** to mark it as under review. This is only a label; it does not check anything.
6. Edit the **Review title**, **Review tags** and **Review description** boxes if you want to correct what goes into the library.
7. Click one of the action buttons below.

| Button | What it does | When it works |
|---|---|---|
| **Start review** | Marks it **UNDER REVIEW** and saves your note | Submitted |
| **Request changes** | Marks it **NEEDS CHANGES** and saves your edits and note | Submitted, Reviewing, Needs changes |
| **Accept** | Checks the file, then copies it into the **Assets** library as a Validated asset. The card says **ACCEPTED**. | Submitted, Reviewing, Needs changes |
| **Send to schedule** | Creates a draft **Scheduled** entry on the Schedule screen for the new asset | Accepted only |

| Button | What it does | When it works |
|---|---|---|
| **Decline** | Marks it **DECLINED** with the reason typed in **Decline reason** | Anything except Declined, Scheduled, Published |

There are **no confirmation boxes**. Each click acts at once.

**Accept** checks the video with a tool called ffprobe. A corrupt or unsupported file is refused and the state does not change. The producer's original stays in the intake folder; CivicCast copies it to the library.

> **Warning: Decline** on a submission that already has a schedule entry cancels that entry first, without asking. If that cannot be done, the decline is refused. A submission that is already Published cannot be declined ("Cannot decline a submission that has already been published.").

> **Known issue (beta.11): Send to schedule** does **not** put anything on the air. It makes a **Scheduled** entry. You must then open **Schedule** and approve it with **Publish to residents**. The screen does not say so, and the message producers see, "Your program has a real spot on the schedule and will air automatically.", is untrue until you do.

> **Known issue (beta.11): Send to schedule** uses settings you cannot see first: the channel is whatever the form stored (the portal form always sends `public`), the length is the **Minutes** box beside the button (30 to begin with, never less than 60 seconds) rather than the video's real length, and the start is the producer's **Requested air date**, or the moment you click if there was none. A start time in the past is likely not what you want. We read the code and tested the data checks, but did not run this on a live station. They show that the screen sends the producer's date without a time zone, and the Schedule rules refuse a time with no zone. So expect **Send to schedule** to fail with "Could not build a schedule item from this handoff" for any submission that has a **Requested air date**.

> **Tip:** The more reliable path is to **Accept**, then use **New scheduled item** on the Schedule screen (Put a recording on a channel once). The accepted file is Validated, so the **Asset** drop-down offers it. You choose the channel, start time and length yourself. A schedule conflict on Send to schedule shows "Schedule conflict on channel '…': …".

> **Known issue (beta.11):** The producer cannot see your notes. The producer sees only a fixed message for each state. For **Request changes** that message is "The operator needs changes before this program can move forward." and does not include what you wrote in **Operator note or change request**. Only a **Decline reason** is passed on, and only when you decline. Write a decline reason you are happy for the producer to read. If you need to tell a producer what to fix, contact them yourself.

> **Known issue (beta.11):** Every failure on this screen, including a refused **Accept** or **Decline**, is shown under the title "Contributor queue could not load." with the real reason underneath, and a **Retry** button. Read the reason, not the title.

> **Known issue (beta.11):** The "Status notification outbox" and the **Status notices** tile suggest messages were sent. They are only a log of what the status page will say. They are not emails. The **Producer activity** tiles show a count of submitted, scheduled and declined programs for each producer.

> **For IT staff:** upload limits, the intake folder, and the file that holds submissions are described in Part II, Running it day to day.

#### If it did not work

**Schedule and Program Guide**

| What you see | Cause | What to do |
|---|---|---|
| "Could not load schedule." with "This action requires one of these CivicCast roles: publish_operator, setup_admin, support_admin." | Your role cannot read the schedule | Ask your station administrator for the publish operator role |
| "Durable storage is not ready." with **Go to Setup** | The station's database is not set up | Click **Go to Setup** and prepare storage (Part II, Installing) |
| "No validated assets." | No Validated video in the library | Upload a video on **Assets**. A CivicCast recording is not offered (see Known issue above). |
| "Time slot conflicts." | Another program overlaps on that channel | Choose a different time or channel, or cancel the other item |
| "Could not schedule." plus a detail | The station refused the item, for example the video no longer exists | Read the detail and fix it |
| "Could not check whether this is ready to publish." | The safety check failed to run | Try again; if it repeats, tell your IT person |
| "Could not publish this to residents." | A new clash appeared (409), or the video became unplayable (422) | Run **Publish to residents** again and read the new review |
| "Could not add to guide." with "meeting_operator, support_admin" | Your role cannot write to the guide | Ask for the meeting operator role |

| What you see | Cause | What to do |
|---|---|---|
| **Skipped · conflict** or **Skipped · not playable**<br>A program is in **Next 7 days** but is not on the air | See the table under "Fix a skipped airing"<br>It is Scheduled, not Published | Fix the cause, then place that airing by hand on **Schedule**<br>Approve it on **Schedule** |

**Auto-schedule**

| What you see | Cause | What to do |
|---|---|---|
| "Rolling window must be a whole number from 14 to 60 days." | The number is out of range | Type a whole number from 14 to 60 |
| "This rule points at a saved search or daypart that no longer exists." | You deleted one of them | Edit the rule and choose again |
| "Compile failed." or "Simulation failed." | The station refused the request | Read the detail in the red box; check **Go to Setup** if it mentions storage |
| "Viewing and managing … requires the publish operator, setup admin, or support admin role." | Your role is not allowed here | Ask your station administrator |

**Recording**

| What you see | Cause | What to do |
|---|---|---|
| "Slug required: lowercase letters, digits, hyphens." | Schedule ID has a bad character | Use only lowercase letters, digits and hyphens |
| "Duration must be HH:MM:SS — minutes and seconds need two digits (e.g. 01:05:00 not 1:5:0)." | Wrong time format | Type `01:05:00` |
| Invalid weekly time or "Pick at least one weekday." | Weekly fields incomplete | Enter a 24-hour `HH:MM` time in the zone shown by the form and tick a day |
| "No SDI capture input was detected or configured." (it names the source kind you chose, for example HDMI) | No card found | Check the card is installed, click **Inspect capture devices**, then ask your IT person |

| What you see | Cause | What to do |
|---|---|---|
| Amber banner "Scheduled recording runtime is unavailable in this deployment…" after **Record now** | The recorder part of the station is not running | Ask your IT person. Schedules can still be saved. |
| "An overlapping recording is already armed/active on this source" | A capture is already running on that input | Wait for it, or stop it |
| "Recording schedule '…' already exists. Use PATCH to update." | You pressed **Create schedule** twice | The first one worked; look in **Schedules** |
| A recording has state `failed` | The capture stopped with an error | Read the text in **Failure** and tell your IT person (Chapter 8) |

**Agendas**

| What you see | Cause | What to do |
|---|---|---|
| "Agendas require the records clerk or meeting operator role. Ask your station admin for access." | Your role cannot use this screen | Ask your station administrator |
| "Meeting agenda '…' already exists. Use PATCH to update." | That agenda ID is taken | Pick a new ID |
| "Another agenda item already occupies (agenda_id='…', order=…)." | Two items share an Order | Give the new item an unused Order |
| "Couldn't find any recognizable items in that PDF. Try pasting the agenda's text instead." | The PDF has no readable text | Paste the text and use **Import** |
| "Only plain-text and PDF agendas import here today (DOCX and other formats are a follow-up)." | Unsupported file type | Use a PDF or paste text |
| "Agenda import is not enabled…" | The outside-system import is off | Ask your IT person |

**Contributors**

| What you see | Cause | What to do |
|---|---|---|
| "Contributor queue could not load."<br>"The broken-media probe could not read this file – it is likely corrupt or truncated: …" after **Accept** | Any failure on the screen, or no permission<br>The producer's file is damaged | Read the reason underneath, then **Retry**<br>**Decline** with a reason and ask the producer to send it again |
| "The contributor's uploaded file is no longer on disk and cannot be accepted. Ask the contributor to resubmit." | The file was removed from the intake folder | Ask the producer to resubmit |
| "Could not build a schedule item from this handoff" or "Schedule conflict on channel '…': …" after **Send to schedule** | See the Known issues above | Use **New scheduled item** on **Schedule** |
| Producer sees "Choose a video file before submitting." | No file was chosen | Choose a file |
| Producer sees "This address has already used its contributor upload budget for this station." or "Too many contributor uploads from this address. Wait before trying again." | Upload limits were reached | Wait, or ask your IT person to raise the limits |
| Producer sees "This station's contributor upload storage is full…" | The intake folder is full | Tell your IT person |

**Related**

- Chapter 2: Signing in and finding your way around for roles and the left menu
- Chapter 4: Running the meeting for the Channels screen, where you can also approve a program to air and take a program off the air
- Chapter 5: After the meeting for assets, chapters and the trim editor
- Chapter 6: Publishing, and what residents see for the resident portal
- Chapter 8: When something looks wrong
- Appendix: roles

## Running the meeting

This chapter is for the people who work the room on meeting night: the camera operator, the meeting operator, and the volunteer who watches the screen while the council meets. It shows you how to check that the station is ready, run a live session, start and stop a channel, take a channel live from a camera, put scheduled programs on the air, drive the production control room, show community announcements between programs, and bring in remote guests. It also tells you, control by control, what each button really does. In this version of CivicCast several buttons that sound like "go on air" do something smaller than their names say, and a few things happen without any "Are you sure?" box.

### Before you start

**Where the screens are.** Everything in this chapter is in the left-hand menu of the operator console, in the section called **Run Meeting**: **Live**, **Facility**, **Control Room**, **Remote Contribution**, **Channels**, **CG Board** and **CG Designer**. The **Readiness** screen is in the section called **System Health**, which is folded shut until you click its name. If you have yet to sign in, see Signing in and finding your way around.

**Who can press what.** CivicCast has five roles. The table lists the role each control in this chapter needs. If your sign-in lacks the role, the screen usually shows a grey or amber note instead of working buttons.

| What you want to do | Role you need |
|---|---|
| Create, run and end a live session (Live) | Meeting operator |
| Check a meeting source (Live) | Meeting operator or Setup admin |
| Change a meeting source's address (Live) | Setup admin |
| Start, stop and restart a channel's feed (Channels) | Meeting operator |
| Take a channel live, return to schedule (Channels) | Meeting operator or Setup admin |
| Approve a program to air, take it off (Channels) | Publish operator or Setup admin |
| Change "Keep this channel on air" and other 24/7 settings | Setup admin |
| Lower-third banner (Channels) | Meeting operator (the screen only enables it for this role) |
| Open a Control Room session and fire cues | Meeting operator |
| Create a remote-guest room | Setup admin |
| Open a room, invite, admit, put guests on air | Meeting operator |
| Add and review community bulletins (CG Board) | Publish operator or Setup admin |
| Change the board layout (CG Designer) | Publish operator or Setup admin (Support admin can look only) |
| Facility router previews | Meeting operator or Support admin (see the Known issue in the Facility task) |

> **Note:** The first administrator account carries all five roles. Staff accounts carry only their assigned roles, so another person's sign-in may not enable every action in the table. See Signing in and finding your way around.

**What must already be set up.** CivicCast cannot do these things from the screens in this chapter, so someone with the Setup admin role must have done them first.

- A meeting source (a camera or encoder CivicCast can pull video from) has been added. If none exists, the **Live** screen shows a card titled "Choose a camera or test source" with a link **Open camera and test media setup**.
- Each channel has an *outgoing-feed configuration*. This is the saved description of where the channel's video is sent (the resident web player, the *cable headend* - the equipment at the cable company that puts your channel on the cable system - and so on). Without one, the **Start** button on **Channels** stays grey.
- A recording location exists (the Live screen lists it as "Recording targets").

> **For IT staff:** setup of sources, outgoing feeds and recording locations is in Configuring the station and Cable headend, streaming, CDN, federation, emergency alerts, the API.

**Times on these screens.** No screen in this chapter asks you to type a time. Times that CivicCast shows you (for example, in the list of programs on **Channels**) are formatted by your web browser in the time zone of the computer you are using. The one exception is the Control Room "locked by … since …" message, which shows the time in a raw year-month-day form that is normally UTC.

**Several different things called "on air"** The most important idea in this chapter: CivicCast has several separate controls that all say "on air", and each changes something different. Read this table once before meeting night.

| What you press | What it really changes |
|---|---|
| **Live** > **Start Live Stream** | Re-checks the selected source, saves the session and queues a takeover of the channel to that source. Watch the outgoing feed to verify it has taken air. |
| **Channels** > **Start**, **Stop**, **Restart feed**, **Finish current item, then stop** | Sends a command to the channel's feed program. This is what starts and stops the channel's actual video. |
| **Channels** > **Take live** | Switches the channel from its schedule to a camera or encoder source right now. |
| **Channels** > **Approve & put on air** | Approves one scheduled program so it plays at its scheduled time. It does not play it at the moment you click. |

| What you press | What it really changes |
|---|---|
| **Remote Contribution** > **Take channel live** | Switches the channel to its configured live source after confirmation. **Put on air** routes the admitted guest mix into that channel; **Off air** hands it back. |
| **Remote Contribution** > **Put on air** | Adds an admitted guest to the composed picture and sound, and routes that mix to the channel. |
| **Remote Contribution** > **Off air** | Removes a guest from the mix; the last guest returns the channel to its schedule. |
| **Control Room** > **Fire cue** | Sends a command to your production gear (OBS, vMix, a camera, a router). It does not put anything on a channel. |

**A few words used here**

- **Channel:** one outgoing program stream, such as "government". It has its own schedule and its own feed.
- **Feed (outgoing feed):** the background program that builds a channel's video and sends it to its outputs. Starting and stopping a channel means starting and stopping this feed.
- **Slate:** a plain card the channel shows when nothing is scheduled. The channel screens call this state "Showing slate".
- **Source:** a camera, encoder or test video CivicCast can pull live video from.
- **Session:** a period of work that has a start and an end, in the Live room or the Control Room.

#### Check that the station is ready

Do this early on meeting night, before anyone is in the room. The same screen is covered in more depth in When something looks wrong.

1. In the left menu, under **System Health**, click **Readiness**. The page's own title is "Safe to broadcast".
2. Read the banner called "On air right now". It lists each channel with a state word and a small colour tag, and it updates itself about every 5 seconds.
3. Read the card under the title. Its tag is **Ready** (green), **Check before meeting** (yellow) or **Do not broadcast yet** (red). The card's heading can also read "Ready with optional items", which means only optional items still need attention. The card's text says what to do next.
4. To run a full test, click **Check broadcast readiness**. This needs the Meeting operator role.

You should see a card titled "Broadcast readiness check result" with a line "Rehearsal result" that reads **Passed**, **Failed** or **Not run**.

> **Warning: Check broadcast readiness** has no confirmation box. It creates a private test live session called "Private first-broadcast rehearsal" on the channel `government`, copies the bundled sample video (if there is none, the result reads **Not run**), and saves a short test recording. If needed, it also creates the sample source and test recording location. The resident portal reconciles live-session state with actual program output. Run this rehearsal before a real meeting rather than during one.

> **Known issue (beta.11):** the Readiness page has no Refresh button. Only the "On air right now" banner updates by itself. The checklist, the feed cards and the other panels change only after you use one of their own buttons or reopen the page.

#### Choose and check a meeting source

The **Live** screen walks you through one meeting from "pick a source" to "end and save the recording". The first step is to choose which source CivicCast will test.

1. In the left menu, under **Run Meeting**, click **Live**.
2. Read the top of the page: the heading "Live", then the sentence "Run pre-flight and start only after CivicCast verifies the source, storage, and network from the server." On the right are small tags showing the session state ("No session" until you create one) and, later, "Pre-flight ready" or "Pre-flight blocked".
3. Read the **Safe to broadcast** panel. It says **Ready**, **Check before meeting** or **Do not broadcast yet**, and has a **Resident preview** link that opens the resident portal in a new tab.
4. Open the **Broadcast channel** menu and choose the channel for this meeting. Your browser remembers the choice. Once you create a session, the menu is locked and says "Channel is fixed while a session exists."
5. Under **Source to check**, select the source to test and associate with the next **Start Live Stream** session. Selecting it alone does not switch the current channel broadcast.
6. Click **Check source**. The button reads "Checking source…" for up to about 8 seconds while CivicCast tries to read video from the source.

You should see the source's tag change to **Delivering** and text such as "Checked just now" or "Checked 2 seconds ago". The other tags are **Needs re-check** (video was seen, but longer ago than 30 seconds), **Not answering** (the last check failed; the reason appears in a red box titled "Live action failed.") and **Not checked**.

> **Tip:** a source's **Delivering** answer expires after 30 seconds by default (your IT person can change this to anything from 5 to 300 seconds). Run **Check source** again immediately before you do anything that depends on it, especially **Take live** on the Channels screen.

The **On-air preview** plays this channel's outgoing HLS broadcast. Before a live takeover it shows scheduled output; it is not a separate preview of the selected incoming source. Enable the channel's web output on **Channels** if the preview is unavailable. Use **Check source** and **Run pre-flight** for incoming-source status. A passed source check alone does not mean the source is on air.

Live Room pre-flight checks the selected source's configured network address, including the sample source. A sample file cannot substitute for an unreachable live input. The installer's separate private rehearsal uses the bundled local sample. For a meeting, select a valid, reachable source on the session's channel and make sure its fresh check passes.

If a Setup admin needs to change a source's address, they click **Edit source**, change the fields and click **Save source**. CivicCast warns: "Saving this change clears what CivicCast knows about this source. You will need to choose Check source again before it can take air." That warning appears only after you change the address, the type or the stored credential. Anyone else sees "Editing a source needs the setup admin role. Ask your station admin to change the address or type."

#### Run a live session

A live session is CivicCast's record of one meeting being broadcast. These steps are in the order the buttons enable themselves. *Pre-flight* is a set of nine tests CivicCast runs before it lets you mark the session on air.

The browser remembers the current meeting. Reopening **Live** restores its session and channel from the station. After a meeting ends, choose **New live session**, then **Create live session** for the next meeting.

1. Choose your channel and source and run **Check source** until it says **Delivering** (see the previous task).
2. In the **Session controls** box, click **Create live session**. The tag at the top changes from "No session" to **Idle**. The session is always named "Council live room".
3. Click **Start pre-flight**. The tag changes to **Pre-flight**. This button only moves the session to the next step. It checks nothing.
4. Tick the box "Operator confirms the meeting details and acknowledges the server-side pre-flight result."
5. Click **Run pre-flight**. The **Pre-flight checklist** at the bottom of the page fills with nine rows. You can run it as many times as you like.
6. Read every row. A row that failed has a red border and a "Next step." line telling you what to fix. Fix it, then click **Run pre-flight** again until the tag at the top says **Pre-flight ready**.
7. When you are ready, click **Start Live Stream**.

You should see the session tag change to **On air**. If the checks fail at this moment, a red box titled "Live action failed." says "Go on air blocked: a fresh source-bound server-side pre-flight did not pass. No broadcast was started. Correct the failed checks and run pre-flight again." CivicCast runs the checks again, including a new source check, when you click, so the click can take several seconds.

> **Warning: Start Live Stream** has **no confirmation box**. The resident portal checks the session against the channel's outgoing feed. An absent, stopped or non-program feed is shown as starting, interrupted or failed rather than as a working live broadcast. Watch the channel's actual output when starting a meeting (see Start a channel).

**Start Live Stream** now queues the actual channel takeover as well as saving the session. A successful

response confirms the command was queued; verify the selected source in the **On-air preview** and the channel output. A stopped channel starts through the same takeover path.

The nine rows in the checklist are: **Network reachable**, **Recording storage**, **AI runtime**, **Live source**, **Recording target**, **Operator confirmation**, **Syndication**, **Internet Archive** and **NAS handoff**. Five must pass before you go on air: Network reachable, Recording storage, Live source, Recording target and Operator confirmation. AI runtime may be **not configured** without blocking, but a probed runtime that **fails** also blocks. Syndication, Internet Archive and NAS handoff report status and never block; they describe what will happen to the recording after the meeting. A row for one of those three can show a red border while the broadcast can still go ahead. What the five required checks test:

| Row | What it checks |
|---|---|
| Network reachable | The station can open a connection to at least one of two public internet addresses, 1.1.1.1 and then 8.8.8.8 (it waits up to 3 seconds for each by default). A station with no internet access cannot pass. |
| Recording storage | At least 50 GiB (about 50 gigabytes) is free on the drive CivicCast keeps its uploaded files on. This is the CivicCast data drive, not necessarily the recording drive. |
| Live source | The source you selected belongs to the session's channel and passes a fresh check. |
| Recording target | A real (not test) local recording location is configured. |
| Operator confirmation | The box in step 4 is ticked. |

> **Known issue (beta.11):** three more things on this screen are easy to misread. First, the **Safe to broadcast** panel near the top is worked out from the last test on the Readiness screen, not from the checklist on this page, so it can say "Check before meeting" while this page says "Pre-flight ready". Second, the hint on a failed Operator confirmation row says to tick the box "below", but the box is in **Session controls**, above the checklist. Third, the empty checklist says "Run pre-flight to populate the nine-check contract." It means "Click **Run pre-flight** to check the camera, the recording drive, the internet and your confirmation."

When you reopen **Live** in this browser, it recovers the saved meeting so **End Live Stream** remains available for an on-air session. If recovery fails, use **Retry session recovery**. Starting another meeting while one is on air is blocked.

#### End a live session and find the recording

1. When the meeting is over, click **End Live Stream**. It is enabled only while the session is **On air**.
2. A box titled "End the live stream?" appears. Click **End live stream** to confirm, or **Cancel**.

3. Watch the **Recording finalization** panel that appears. It shows **Waiting**, then "Attempt N of M. The recording is being checked and packaged.", then "Recording saved as asset <id>. Find it in the Assets library."

If it fails, the panel shows the reason (or "Finalization failed."). Fix the cause, then click **Retry finalization**. See After the meeting for what to do with the saved recording.

**End Live Stream** queues a return to scheduled output and closes this meeting's takeover. It keeps the channel running. To take the entire channel off the air, use **Stop** or **Finish current item, then stop** on Channels. Watch the outgoing feed to verify the handback.

> **Known issue (beta.11):** the saving step looks for a recording file named after the session in the recording location (for example `council-live-room.mp4`). Nothing on this screen creates that file, and in testing we could not confirm which part of CivicCast does. If the panel stays on **Waiting** or fails, ask your IT person to check that the recording location is receiving the meeting recording. By default CivicCast gives up waiting for the file 30 minutes after **End Live Stream**, and the panel then reads "No recording file was found for this session (expected ...)". While CivicCast is still retrying by itself the tag reads "Retrying". **Retry finalization** appears only after CivicCast has given up. It is shown to every role, but only a Meeting operator can use it.

#### Start a channel

The **Channels** screen controls each channel's actual video feed.

1. In the left menu, under **Run Meeting**, click **Channels**. The heading is "Channels" and a tag says "refreshes every 30s".
2. In the row of channel cards under the heading, click the card for the channel you want. Each card shows the channel's name and its id (for example `government`). The first channel is selected for you.
3. Find the box **Outgoing channel feed**. It shows a state tag. The words you can see there are **On air**, **Showing slate**, **Starting**, **Stopping**, **Finishing current item**, **Changing source**, **Stopped** and **Needs attention**.
4. Click **Start**.
5. A box titled "Start the outgoing feed for <channel>?" says "<channel> goes live to its configured outputs and becomes visible to residents." Click **Start feed**.
6. The button reads "Queuing..." for a moment. The screen then checks the feed every 2 seconds for 20 seconds.

You should see the state tag change to **Starting** and then **On air**, or **Showing slate** if nothing is scheduled right now. "Starting" means CivicCast is preparing the first program, which can take a while on a cold start.

> **Warning: Start** makes the channel visible to residents and to every output set up for it (the web player, and any cable or streaming output). Start a channel only when you mean it to be public.

Clicking **Start** only *queues* a command. The feed program picks it up afterwards. If it does not, after

20 seconds a red box says "Start was queued but the feed did not start." and tells you to check that the CivicCast egress service (the background program that sends the channel out) is running on this station (System Health), then try **Start** again.

If **Start** is grey, a line under the button says why:

- "No outgoing-feed configuration for <id>. Apply a headend preset or the local rehearsal preset first." The channel has never been set up. Ask a Setup admin.
- "Outgoing feed for <id> is disabled in its egress configuration. Enable it in Outgoing feed configuration, then start."
- "Checking for an outgoing-feed configuration…" Wait a moment.
- "Could not check the outgoing-feed configuration for <id>. Retry the check, then start." Click **Retry check**.

If you see "Outgoing feed controls require the meeting operator role.", your sign-in cannot use these buttons.

#### Stop, restart or finish a channel

Use the same **Outgoing channel feed** box.

| Button | Box that appears | Confirm button |
|---|---|---|
| **Stop** | "Stop the outgoing feed for <channel>?" - "This takes <channel> off the air. Residents watching lose the stream until the feed is started again." | **Stop feed** |
| **Restart feed** | "Restart the outgoing feed for <channel>?" - "The stream drops briefly for residents while <channel> restarts." | **Restart feed** |
| **Finish current item, then stop** | "Finish the current item, then stop <channel>?" - "<channel> plays out its current item and then goes off the air until the feed is started again." | **Finish, then stop** |

> **Warning:** all three commands change what residents see right now. **Stop** drops the stream at once. Use **Finish current item, then stop** at the end of a meeting if you want the program to end cleanly.

> **Stop is intentional:** Stop and Finish current item, then stop suppress automatic restarts for the current service session, even when Keep this channel on air is enabled. Press Start to resume. A service restart clears this suppression and applies the configured auto-start setting again.

> **Known issue (beta.11):** the screen refreshes every 30 seconds, and only **Start** is watched more closely. The screen does re-read the feed state once right after you confirm **Stop**, **Restart feed** or **Finish current item, then stop**, but the feed program may not have acted yet, so the state tag can take up to about 30 seconds to change. Do not click the button again straight away. Wait for the tag to change.

#### Keep a channel on air all day (the 24/7 setting)

The box **Run this channel 24/7** holds the setting **Keep this channel on air**. Only a Setup admin can change it. Everyone else sees "Automation settings require the setup admin role."

The checkbox's own hint reads "Starts the feed automatically after restarts and crashes." The same box has **Allow software (CPU) encoding fallback**, a choice of **Station slate** or **Community bulletins** under "Between programs, show", a **Slate message**, and two optional output names for network video (**NDI output name (optional)**) and a video card (**SDI output device (optional)**). To save any change, click **Save automation settings**. It is enabled only after you change something, and "Unsaved changes" appears beside it.

What "Keep this channel on air" really does, from the code we read:

- By default, about every 2 seconds, CivicCast's automation looks at every enabled channel with this setting on.
- If the channel is dark and no intentional stop is active, CivicCast queues Start. Failed starts retain their existing retry cooldown.
- An intentional Stop or Finish current item, then stop is remembered for this service session. Automation cannot override it; an explicit Start clears it.

> **Warning: Save automation settings** has **no confirmation box**. Saving with **Keep this channel on air** ticked, on a channel whose configuration is enabled, can put the channel on the air without anyone pressing **Start**.

> To keep a channel off air across service or computer restarts, untick **Keep this channel on air** and save before stopping it. The session-local stop suppression is not a permanent change to that setting.

> **Known issue (beta.11):** if a channel has no outgoing-feed configuration, this box says "This channel has no outgoing-feed configuration yet. Create one from the channel egress runbook (or the setup flow) first; then automation settings appear here." A non-technical person cannot do that. The real fix is for a Setup admin to apply a headend preset (see the Cable headend box

> on the same screen, or ask IT).

#### Live captions during the meeting

Live captions are a station setting, not a control on the Live screen. On a new station, **Show live captions on air** is off. A Setup admin can enable it in **Station Profile**; see Live captions: what the settings change.

Whistle runs live recognition on the CPU and sends first-pass captions without waiting for a second recognition to agree. Primary inference is serialized across the station's channels. If Whistle fails or exceeds its 30-second request deadline, Whisper supplies backup captions while CivicCast automatically tries a fresh Whistle process. A successful recovery returns that channel to Whistle without restarting the station. Whisper also transcribes recordings and can use supported NVIDIA CUDA acceleration. On **System Health**, temporary delay is distinguished from sustained overload so an operator can see when captioning is catching up.

Beta.12 starts Whistle without loading the Whisper backup first. A backup that cannot load therefore does not prevent Whistle from starting. Whisper loads when a channel actually needs fallback; that first switch can take longer, and insufficient memory or a backup startup failure can still interrupt that channel's captions.

When the switch is enabled, recognition resumes on the next worker scan, but the channel's caption route is added when the channel next starts. Stop and start each affected channel after enabling it. When switched off, recognition stops and queued audio drains on the next worker scan; the graph route is removed at the next channel start. The switch does not affect captions created later for recordings. For overload and fallback recovery steps, see Captions late or missing.

#### Take a channel live from a camera (live takeover)

A *takeover* puts a live source on a channel right now, ahead of whatever is scheduled. You then hand the channel back to its schedule when you are done.

1. On the **Live** screen, click your source's card and click **Check source** until it says **Delivering**.
2. Go to **Channels** (if a live session is open on **Live**, do this in a second browser tab) and click the channel. Find the box **Live takeover**. It says "This channel is on its scheduled program."
3. Click **Take live**. If it is grey, the line beside it says "No live source is ready yet." Go back to step 1. By default the button goes grey again 30 seconds after your last good **Check source**. The Channels screen asks for this state every 15 seconds, so after a fresh **Check source** the button can take up to about 15 seconds to turn on.
4. A small form appears with the question "Why are you going live? (optional)". Type a reason if you like.
5. Click **Confirm take live**, or **Cancel**. The button reads "Going live…" while CivicCast works.

You should see a red badge reading "Live takeover" with the name of the person and how many minutes it has been live.

> **Warning: Confirm take live** overrides the schedule and changes what is on the air. There is no pop-up. The second click is the confirmation. If the source's last good check is older than the window (30 seconds by default) when you confirm, the code we read refuses the takeover before it re-checks anything. The error can then read something like "Live ingest path '<channel>:local' is disabled.", which names CivicCast's built-in placeholder path and not your camera. Click **Check source** again and retry. If the channel is already under takeover, the error says "Channel '<id>' is already under live takeover."

To hand the channel back:

1. In the same box, click **Return to schedule**.
2. Optionally type a note (the example text is "e.g. meeting adjourned").
3. Click **Confirm return to schedule**, or **Stay live** to cancel.

> **Warning: Confirm return to schedule** changes what is on the air. The takeover history marks the takeover "Returned" as soon as the command is queued, not when the feed has actually switched back.

> **Known issue (beta.11):** the screen never says which source it will use, and the source card you picked on **Live** is not sent with the request. In the code we read, **Take live** uses the first **Delivering** source in CivicCast's list for that channel, which may not be the one you selected. The red "Live takeover" badge shows only who took over and how long ago, not the source, and it appears as soon as CivicCast records your request, which can be before the picture has switched. The **Source:** line in the **Outgoing channel feed** box should name the live source once the switch happens (it can take up to 30 seconds to update; we did not watch this on a running station), and a Setup admin can read the source in "Takeover history". Always watch the channel's actual output. A takeover is planned for up to 3,600 seconds (one hour). In testing we could not confirm what happens when that hour ends, so return to the schedule yourself well before then.

A Setup admin also sees "Takeover history", which lists past takeovers with "Live" or "Returned", who did it, when, and the reason. If none exist it says "No live takeovers have been recorded for this channel."

#### Put a scheduled program on the air (commit to air)

Programs that you want to play on a channel have to be *committed*. The box **Commit programs to air** on **Channels** does this. It lists the next 24 hours of programs for the selected channel. The same safety check is offered on the **Schedule** screen under the name **Publish to residents** (see Before the meeting); it does the same thing.

1. On **Channels**, find **Commit programs to air**. It says "Review the safety check, then approve a program to put it on this channel."
2. Under "Next 24 hours", find the program. Its tag is **Ready to review**, **Committed**, **Skipped (conflict)**, **Skipped (media)** or **Cancelled**. Only programs that are linked to a schedule item are listed.

3. Click **Review & prepare**. This is a dry run. It changes nothing.
4. Read the result. A green **Safe to air** tag means CivicCast found the media and no clash. A red **Not safe to air yet** tag comes with the reason: a message about missing media, or "Clashes with N other program(s) already scheduled:" followed by what overlaps. A gap before the program is listed as "Dead-air gap before this program (it can still air):".
5. If it says **Safe to air**, click **Approve & put on air**. To back out of the review, click **Cancel**.

You should see the program move to the "Recent commits" list with a tag: **Preparing**, **Queued to air**, **Couldn't reach the engine** or **Rolled back**.

> **Warning: Approve & put on air** has **no confirmation box**. It is a single click once the review says **Safe to air**. It publishes the schedule item to residents, saves an approval record in your name, and then queues a **Start** for the channel if the feed is stopped, or a reload if it is running, so the schedule is read again. The program plays at its scheduled time, not at the moment you click. It does not override an intentional operator stop in the current service session; press Start explicitly to resume that feed.

To undo an approval:

1. In "Recent commits", click **Take off air** on that program.
2. Type a reason in the box "Why are you taking this off air?". The button stays grey until you do.
3. Click **Confirm take-off**, or **Keep on air** to cancel.

> **Warning: Confirm take-off** cancels the program's schedule item. It will not play, not merely pause. The engine is told to read the schedule again, and the approval is marked **Rolled back** with your reason.

> **Known issue (beta.11):** four things in this box are loose with words. The tag **Queued to air** means the command was queued, nothing more. The tag "On air (confirmed)" is never shown, because no code sets that state. A row for a future program says "aired <time>" even though it has not played. And if the list of past approvals cannot be loaded (for example, for a role that is not allowed to see it), the box still says "Nothing has been committed to air on this channel yet." instead of showing an error.

Without the Publish operator or Setup admin role, **Review & prepare** is grey and the box says "You can review the schedule here. Putting a program on air or taking it off requires the publish operator or setup admin role."

#### Put a lower-third banner on a channel

A *lower third* is the strip of text across the lower part of the picture, such as "Town Council – Live". The box **Lower-third banner** on **Channels** holds one line of text for the selected channel.

1. In **Lower-third text**, type your line (up to 240 characters).
2. Click **Put on air**. The box asks "Put this banner on air?"

3. Click **Confirm: put on air**, or **Cancel**.
4. To remove it, click **Take off air**, then **Confirm: take off air**.

> **Warning:** this changes what is on the air, but not at once. The confirmation line explains that the change applies "to this channel's next pipeline build (a fresh start or a scheduled content swap)" and "Does not hot-change an already-live broadcast's on-screen text." ("Pipeline" here means the chain of software steps that builds the channel's picture.)

> **Known issue (beta.11):** the tag beside the heading says **On air** as soon as you save, even though the picture has not changed. Treat it as "Saved: on". The banner reaches the picture only when the channel next restarts or swaps content. For this chapter we have not watched a banner appear on a live channel. The box works only for a channel that has an outgoing-feed configuration. Otherwise it says there is none yet. The screen's note says a Setup admin may also change the banner, but the button is enabled only for a Meeting operator.

#### Change the cable headend (for Setup admins)

The box **Cable headend delivery** sets where a channel's cable-TV signal is sent. This is normally done once during setup, not on meeting night. If you are a Setup admin:

1. Choose a **Headend preset**, fill in the destination fields the preset needs, and click **Apply headend preset**. (For some presets the button reads **Enable web preview** instead.)
2. A box titled "Apply this headend preset?" spells out what will change. Click **Apply preset**, or **Cancel**.

> **Warning:** applying a preset changes what the channel outputs. If the channel is standing by on its slate, it restarts at once and every output drops for a few seconds. If a program is on air, nothing is interrupted and the change waits for the next stop and start. The box warns when other outputs, including the cable feed, will be removed. Do not apply a preset during a meeting unless IT has told you to.

> **For IT staff:** presets, destinations and the **Verify stream (TSDuck)** check are covered in Cable headend, streaming, CDN, federation, emergency alerts, the API.

#### Open a Control Room session

The **Control Room** drives your production equipment (OBS, vMix, Blackmagic ATEM switchers, HyperDeck recorders, PTZ cameras, CasparCG, routers and similar) during the meeting. It works through a small helper program that CivicCast calls the "TSR control service". In this chapter we call it the *production-control helper*.

You work on a *control surface*, which is a named bank of buttons. Each button is a *cue*, one action such as "Take scene" or "Router take". You open a *session* on the surface, test a cue with a *dry run* (a preview of what it would send), and then confirm it. A session is either **Test Mode**, in which nothing reaches your equipment, or **On-Air Mode**, in which cues are really sent.

> **Note:** the Control Room produces a source. It does not put anything on a channel. The page says so: "This console produces the source. Taking it to air on a channel is a Playout (S5) action — it is not fired from here." (Playout is the part of CivicCast that runs a channel; "S5" is an internal code name.) Use **Take live** on **Channels** for that.

> **Known issue (beta.11):** CivicCast's own readiness text (under **Technical detail** in the "Control-room readiness" card) says the Control Room's readiness is worked out from CivicCast's settings and test profiles, and that it is "not clean Windows install evidence, simulator evidence, real OBS/vMix/ATEM/NDI evidence, or station-device evidence." In plain words, the Control Room has not been proven against real production equipment in this beta. Rehearse in **Test Mode**, and keep your hardware's own controls within reach.

**Read the readiness panel**

1. In the left menu, under **Run Meeting**, click **Control Room**. The heading is "Production Control Room".
2. Look in the readiness panel for a card called "TSR control service". If it is blocked, the helper is not running or cannot be reached. Tell your IT person. A separate amber bar that says "Production control unavailable — the TSR control service is not running or not configured. Cues cannot fire until it is restored." appears only after a device probe, a cue dry run or a cue fire fails for the same reason. It does not appear when the page first opens.
3. Read the card "Control-room readiness". It has a headline tag and a second tag, either "Equipment verified" or "Equipment check pending". Under it are five counters: **Devices**, **Surfaces**, **Cues**, **Open sessions** and **On-Air**.
4. If any check is blocked or has a warning, it appears as its own card with a recovery sentence. A blocked check has a button **Open Control Room Setup**. Only a Setup admin can use that screen.

> **Known issue (beta.11):** the headline can never read **Ready**. The "equipment verified" value is permanently false in this version, so the best the headline can say is **Check before meeting**, and the amber note "You can register switchers and run dry runs now. On-air readiness is confirmed once a check against the room's actual devices passes." never goes away. No screen offers that check. Read **Check before meeting** here as "CivicCast has not verified your equipment", not "something is broken". A red **Do not broadcast yet** does mean a real blocked check.

**Check your equipment** Under **Devices** each piece of gear is a small chip with its name, kind and tags such as **Reachable**, **Unreachable**, **Disabled** or **Not probed**, plus a health tag: **Healthy**, **Never probed**, "Stale — probe again" (older than 5 minutes) or **Unreachable**. A Setup admin or Support admin can click **Probe** on a chip to test it (the button reads "Probing…"). If there are no devices, the screen says "No devices to operate yet." and tells you to register one in Control Room Setup.

**Open a Test Mode session**

1. Open the **Control surface** menu and pick a surface ("Select a surface..." is the empty choice).
2. Leave **Test Mode** selected, and click **Open Test Session**. The page shows a blue banner: "TEST MODE - device actions are blocked and recorded as test-only audit events."
3. Click a cue's button. This is a dry run only. A plan card appears with the cue name, the command text, the tag "Ready to send" or "Not ready", and a line such as "take-delay Nms · post-roll Nms".
4. If the cue is marked as needing confirmation, click **Test... (needs confirm)**, then **Confirm test action**. Otherwise click **Record test action**.
5. Watch the **Fired-cue audit** list. The row shows **Planned**. Nothing was sent to any device.
6. When you are done, click **End session**.

You should see the banner "Test action recorded." Test Mode never touches your equipment.

**Open an On-Air Mode session**

1. Wait until the readiness panel has no blocked checks.
2. Open the **Control surface** menu and pick a surface.
3. Select **On-Air Mode**.
4. Pick a **Safe-state cue**. This is the recovery cue, the one that puts the room back in a known safe state if something goes wrong. The menu lists only cues that are marked as needing confirmation.
5. Tick the box "I understand On-Air cue actions may be sent to production devices".
6. A list called "On-Air prerequisites" shows "Ready:" or "Needs attention:" for **Control-room readiness**, **Safe-state cue selected** and **On-Air responsibility acknowledged**. When all three are ready, click **Open On-Air Session**.

You should see an amber banner: "ON-AIR MODE - cue actions can be sent to production devices. Safe-state cue: <cue id>." The banner shows the cue's internal id, not its name. The **Safe State** panel under it shows the name. While the session is open, the screen shows the time remaining and the exact deadline.

> **Warning:** in an On-Air session a cue really is sent to your equipment, and that can change the picture going to the channel. Always click the cue first (the dry run) and read the plan card.

If someone else already has the surface open, you see: "A session is already open on this surface, locked by <name> since <time>. A setup admin or support admin can force-close it to release the lock."

An On-Air session expires **30 minutes** after it opens. The countdown warns you as the deadline approaches. At expiry, ordinary cue controls pause. The session owner can still use **Panic: Run Safe State** while the session remains open; end the session to release the surface, then open a new one to continue. If a normal cue request reaches the server after expiry, CivicCast refuses that cue and leaves the session open, so the owner can still run Panic or end the session. Cues already sent are not undone. After a page refresh, CivicCast restores the selected surface and its open session, but not an earlier dry-run result; dry-run a cue again before firing it.

### Fire a cue

1. Click the cue's button. The plan card appears (the dry run).
2. Read the command text and check the tag says **Ready to send**.
3. Click **Fire... (needs confirm)**, then **Confirm fire**. A cue that is not marked as needing confirmation shows **Fire cue** instead, and one click sends it.
4. Read the banner: "Cue fired." The audit list adds a row tagged **Fired**, or **Failed** if the helper could not send it.

> **Warning:** **Confirm fire** and **Fire cue** change what your production gear is doing. There is no pop-up. The two-step button is the confirmation. If the cue or the device changed since your dry run, CivicCast refuses with "The cue preview is stale ... Dry Run the cue again before Live Fire."

**If a cue fails: Safe State and rollback** The **Safe State** panel appears once a session is open. In an On-Air session it names the recovery cue and has two buttons. In a Test Mode session it reads "No safe-state cue is configured for this session." and has no buttons, because Test Mode does not ask you to pick one.

- **Dry Run Safe State** previews the recovery cue.
- **Panic: Run Safe State** sends the configured safe-state cue immediately through the recovery action. It does not wait for a dry run and remains available to the session owner after the On-Air deadline, while that session remains open.

If a cue fails in an On-Air session, use **Panic: Run Safe State** in the Safe State panel. A good result reads "Rolled back to Safe State."

> **Warning:** **Panic: Run Safe State** has no confirmation box. It sends the recovery cue to your equipment at once.

### End a session

1. Click **End session**.
2. A box titled "End the control room session?" appears. For an On-Air session it says "This releases your operator lock on this control surface. Any cue already sent is not undone. Panic can still run the safe-state cue until you end the session." For Test Mode it says "This releases your operator lock on this control surface. No operator can fire cues on it until a new session is opened." Click **End session**, or **Cancel**.

The selected surface is remembered in this browser tab. Refreshing the page restores the open session for its owner without opening a duplicate; dry-run results are temporary, so repeat a dry run before firing. Another operator sees the active session as read-only. A Setup admin or Support admin can release that lock after confirming; releasing a lock does not undo cues already sent. An operator can end their own session after confirming. Ending a session releases its surface lock.

A Support admin can also type a note in **Operator note** and click **Create support bundle**. This builds a troubleshooting file with private details removed, and shows its location. Everyone else sees "Support bundles require support admin."

#### Preview a router take (Facility)

The **Facility** screen is titled "Facility router". A *router* here means the video switching box that decides which camera or feed goes to which input, and a *take* is one switch.

> **Known issue (beta.11):** in this version the Facility screen is a **preview only, using sample data**. It cannot change a real router. The page says so with a tag, "hardware send disabled". The router, the sources and the destination it lists are a fixed built-in example, the same on every station: a "Control room router" at the address 192.0.2.10 (an address reserved for examples, which no real router uses), the sources "Council chamber" and "Bulletin board", and the destination "CivicCast capture". Do not treat them as your equipment. In the code we read, there is no screen or command that loads your own router.

What it does show is the exact command that *would* be sent. To look:

1. In the left menu, under **Run Meeting**, click **Facility**.
2. Click an endpoint card under **Router endpoints** (a tag shows "ready" or "off").
3. Click a route button such as "Council chamber to CivicCast capture", or use **Manual crosspoint** (choose a **Source** and a **Destination**, then click **Preview take**).
4. Read the **Take preview** card: the target, the protocol, the command, and a "Proof boundary" line that says what the preview does not do.

You should see the command text, and the tag "ready" or "blocked". No hardware is contacted, and nothing changes on any channel.

Two more previews need a channel chosen in **Target channel**: **Preview scheduled take** (a take timed 15 minutes from now with a 15-second lead, using a made-up item) and **Preview L-bar and squeeze-back** (a plan for shrinking the picture to make room for a graphics frame). Neither runs anything.

> **Note:** nothing on this screen can change anything on the air. It sends nothing and has no confirmation boxes. The only real way to do a router take in this version is a Control Room cue of the kind "Router take" on a "TCP device".

> **Known issue (beta.11):** the "Take preview" card says "Operator action. Confirm the previewed route, then send the command from the router panel." There is no send control anywhere. Ignore that sentence. Also, the screen allows some roles that the server then refuses. A sign-in with only the Setup admin role can click the route buttons, **Preview take**, **Preview scheduled take** and **Preview L-bar and squeezeback** and get a refusal ("This action requires one of these CivicCast roles: meeting_operator, support_admin."). A Support admin has the opposite problem: the server allows these previews, but the screen greys out the scheduled-take and L-bar buttons for that role.

#### Show community bulletins between programs (CG Board)

CG board and legacy slides now wrap supported title, organization, and message text into their allocated areas and reduce text size when needed. Long content can appear small; preview at the intended resolution before approval. Ticker scrolling behavior is unchanged.

*CG* is short for "character generator", the part of a broadcast system that puts text and graphics on screen. In CivicCast the **CG Board** is the "community board" a channel can show between programs: a card with station and community announcements. It is **not** the lower-third banner (that is on **Channels**, see above).

The board is shown in the gaps between programs when the channel's filler is set to **Community bulletins** (on **Channels**, in "Run this channel 24/7", under "Between programs, show"). Otherwise it shows the **Station slate**. If no bulletins are approved, the channel falls back to the slate.

Saved manual ticker text is included in the public board snapshot and the outgoing board picture. Changes appear when the board next refreshes. If a bulletin cannot be rendered, CivicCast reports the preparation error and uses the station slate rather than repeatedly relaunching a broken bulletin encoder.

1. In the left menu, under **Run Meeting**, click **CG Board**.
2. Use the **Channel** menu to choose a channel ("Public board" is the first choice).
3. The **Template library** shows three layouts, and **Read-only board preview** shows a preview of the board's zones. Clicking a template only changes this preview.
4. In **Community bulletins**, click **Add bulletin**.
5. Fill in **Organization**, **Submitted by**, **Title** and **Message**. All four are required. The limits are 160 characters for the first two, 200 for the title and 500 for the message.
6. Click **Add bulletin** again (the second button, in the form). The form closes. The bulletin appears with the tag **Submitted**.
7. To let it air, click **Approve**. The tag changes to **Approved**.

To send a bulletin back, click **Request changes**, type a note and click **Send request**. The tag becomes **Needs changes**, with your note in amber ("Notes: …"). To remove a bulletin, click **Decline**, type a reason and click **Decline bulletin**.

> **Warning:** **Approve** has no confirmation box. Once approved, a bulletin is eligible to air as a slide between programs on the channel you chose in the **Channel** menu, if that channel's filler is **Community bulletins**. We did not watch a bulletin air for this chapter. **Decline** is how you pull one that is airing. A declined bulletin cannot be approved again from this screen. Add a new one instead.

> **Note:** This screen is for reviewing and approving between-program bulletins. The board preview is read-only here; use **CG Designer** to change templates, zones or feeds. Only a Setup admin or Publish operator can manage bulletins. You cannot set start and end dates for a bulletin here. If the station's database has not been prepared, adding fails with "Durable storage is not

> ready. Open Setup and choose Prepare storage before managing community bulletins."

#### Design the community board (CG Designer)

**CG Designer** (the page title is "CG Board Designer") sets up the board itself for one channel: a template, its zones, outside feeds, a live preview and a change history. A *zone* is one box on the board, such as a ticker, a schedule, a logo, or a sponsor message. This is normally done before the meeting, not during it.

1. In the left menu, under **Run Meeting**, click **CG Designer**. You can see this item only with the Publish operator, Setup admin or Support admin role.
2. Choose a **Channel**.
3. If the channel has no board, choose a **Template** and click **Create board**.
4. To add a zone, click **Add zone**, fill in **Region**, **Zone kind** and **Content source**, then click **Add zone** again. To delete a zone, click **Delete**, then **Confirm delete?**.
5. To bring in an outside source such as a news feed or a calendar, click **Add feed**, fill in **Feed kind**, **Trust tier**, **Feed label**, **Source URL** and **Refresh (minutes)**, and click **Register feed**.
6. To vet individual feed items, click **Review items** on a feed and click **Approve** on each item you want to allow.
7. Use **Board background audio** to choose an uploaded track with a measured duration and reduce its level if needed. The track plays across board slides while the board fills a gap between programs. **Delete** removes the selection. The visual preview does not play sound.
8. To place a configured camera or other live feed in a board zone, choose **Live video** as the zone's **Content source**, then choose a live source belonging to this channel. The video appears in that zone while the board is on air. The preview shows the layout without playing the live feed. Configure the source under **Live Sources** first; NDI sources are not offered for board zones.
9. Check the **Live preview** and read **Board history** to see who changed what.

> **Warning:** the board is what viewers see between programs. Changing the **Board template** menu takes effect **immediately, with no confirmation**. Deleting a zone asks only "Confirm delete?".

> **Package note:** Board audio and live-video zones are part of Beta 19, not the published Beta 18 package. Support admins can view the designer but cannot edit it.

#### Bring in remote guests (Remote Contribution)

**Remote Contribution** lets remote council members, presenters and public commenters join from a web browser, with nothing for them to install. CivicCast manages rooms, invites and guest admission, combines admitted guests into one picture and audio mix, and routes that mix to the selected channel when you put a guest on air.

> **For IT staff:** use **Diagnostics > Install VDO and media relay**, then commission the self-hosted VDO.Ninja HTTPS site, a public HTTPS WHIP route to the relay, and a reachable TURN service before the meeting. CivicCast uses its bundled FFmpeg for composition and its existing live-source path for channel output. See Cable headend, streaming, CDN, federation, emergency alerts, and the API.

You see this item in the menu only with the Meeting operator, Setup admin or Support admin role.

### Set up a room and send invites

1. In the left menu, under **Run Meeting**, click **Remote Contribution**. A Setup admin first creates a room: type a **Room name** (the example is "Council Chamber Guests"), choose an enabled channel from **Configured channel**, then click **Create room**.
2. Click the room in the **Rooms** list.
3. Click **Open room**. The room's VDO.Ninja director opens in an embedded panel; use it to inspect guests and keep it open while sending guest controls. You can copy its link for use in a separate switcher. If you reload the page, click **Open room** again.
4. Under "Invite a guest", type the **Guest name**, choose a **Contribution role** (**Council member**, **Presenter** or **Public comment**), and click **Generate invite link**.
5. Copy the box labelled "Guest link for <name> — send this", and send it to that guest. Like the director link, this box is shown only right after you generate the link. If you reload the page or select another room it is gone, and the invite list shows only the guest's name, role and **Used** or **Pending**. Generate a new link if you did not copy it.

> **Note:** each guest link works for one guest, once, and expires after 4 hours. The screen says "single-use" but does not mention the 4 hours. Guests who join as **Public comment** must first accept terms before they can join. "Sent invites" lists each link as **Used** or **Pending**. A room holds up to 6 guests by default.

The channel menu shows configured, enabled egress channels. If none are available, ask a Setup admin to enable a channel. Each channel can have one remote-guest room on air at a time.

### Admit and control a guest

1. When a guest opens their link, they appear under **Guests** with the tag **In waiting room**.
2. Click **Admit**. Admission keeps the guest in the waiting room; it does not air their media.
3. Click **Put on air**. CivicCast adds the guest to the live picture and audio mix, takes the channel to that source, and waits for an on-air handoff record. An error leaves the guest in the room without claiming the action succeeded.
4. Keep the embedded VDO.Ninja director open. **Mute guest audio** and **Restore guest audio** change CivicCast's live audio mix and send a targeted VDO.Ninja request. **Mute guest camera** and **Restore guest camera** send camera requests to that guest.
5. **Off air** removes the guest from CivicCast's live mix. If they were the last on-air guest, CivicCast

returns the channel to its scheduled output. The guest stays connected in the room.
6. **Disconnect guest** asks for confirmation, then sends VDO.Ninja's targeted hangup command.

> **Important:** VDO.Ninja's director iframe does not confirm whether camera or hangup requests succeeded. CivicCast records those browser commands as **sent, not verified**. Verify guest connection and camera state in the director. CivicCast's on-air, off-air and audio-mix actions are separate channel operations; watch the channel output during a live meeting.

**Take the channel live** **Take channel live** is a separate action for the channel's ordinary configured live source, such as a camera. Use **Put on air** on an admitted guest to route remote guests. Use **Off air** on the last guest to return to schedule.

After checking in the director that a guest has left, click **Mark left after checking** and confirm. This changes only the CivicCast session record; it does not send a provider command.

**Close the room**

1. Select the room and click **Close room**.
2. Confirm to send hangup requests, remove any on-air guests from the channel and close the room record. VDO.Ninja does not acknowledge hangup requests; verify in the director before marking each guest left. The final on-air guest's removal returns the channel to its schedule.

You can reopen the room and send new invites afterwards.

**Test guest connections (Support admins)** A Support admin sees **Diagnostics** with TURN, VDO, relay and coturn status. Use **Install VDO and media relay** to fetch the verified local components, and **Test TURN connectivity** to probe the configured TURN server now.

#### The meeting-night checklist

Print this page or copy it. The screens named here are covered earlier in the chapter.

**An hour before**

1. Open **Readiness**. The card should say **Ready** (green). If it says **Do not broadcast yet**, read its next step and tell your Setup admin.
2. On **Channels**, check the channel's state tag and "Now / next". Make sure the program you expect is scheduled (see Before the meeting).
3. If you plan to use a camera: on **Live**, pick the source and click **Check source** until it says **Delivering**.
4. If you use the Control Room: open a **Test Mode** session, dry-run and record one cue, and end it. Check that no amber "Production control unavailable" bar appears.
5. If you use remote guests: open the room, send the invite links, and ask guests to join early so they are in the waiting room.
6. On **Channels**, check whether **Keep this channel on air** is ticked, and whether you expect it to be.

7. On **Live**, check the recovered session. If the previous meeting has ended, choose **New live session**. Click **Create live session** for the new meeting.

**At the start**

8. If you want a lower-third banner, set it first (see step 11). Then, if the channel is stopped, click **Start**, confirm **Start feed**, and watch for **On air** or **Showing slate**.
9. On **Live**, create the session if needed, click **Start pre-flight**, tick the confirmation box, click **Run pre-flight**, and click **Start Live Stream** when the required checks pass. This queues the selected source onto the channel.
10. Verify the camera in the **On-air preview** and the outgoing channel feed. If you operate through **Channels** instead of Live Room, use **Check source**, **Take live**, **Confirm take live** there; do not request a second takeover for a channel already live.
11. A lower-third banner is meant to take effect at the channel's next start (we have not watched one appear). On a channel that is already running it appears only when the channel next restarts or swaps content.

**During the meeting**

12. Keep **Live** and Control Room in their own tabs for convenience. Both restore their saved session after refresh; repeat any cue dry run before firing.
13. If you use an On-Air Control Room session, open a new one before 30 minutes have passed.
14. Admit remote guests one at a time, then click **Put on air** for each guest you want in the live mix. Keep the VDO.Ninja director open and verify the channel output.
15. Watch the channel's own output, not the screen's tags. After **Stop**, **Restart feed** or **Return to schedule**, the tag can take up to 30 seconds to change.

**At the end**

16. Return the channel: **Return to schedule**, **Confirm return to schedule**.
17. Send disconnect requests for remaining guests, verify they left in the director, mark them left, then click **Close room**.
18. End any Control Room session: click **End session**.
19. If you want the channel off the air, click **Finish current item, then stop** (or **Stop**) and confirm. This suppresses automatic restarts for the current service session even if **Keep this channel on air** is ticked. Ask a Setup admin to untick that setting only if the channel must stay off across a service or computer restart.
20. On **Live**, click **End Live Stream**, confirm, and watch **Recording finalization** until it says "Recording saved as asset <id>." Then follow After the meeting.

**If it did not work**

| What you see | What it means and what to do |
|---|---|
| "Live-room controls require the meeting operator role. Source status and readiness checks remain visible." | Your sign-in lacks the Meeting operator role. Ask your Setup admin. |
| "Channel preview unavailable" | Enable the channel's HLS web output on **Channels**. Incoming-source readiness is shown separately; use **Check source** and **Run pre-flight** before taking air. |
| A red box titled "Live action failed." | Ask the station administrator to check the CivicCast service and database. Keep this screen open during an active session; refreshing can lose the in-page session. |
| "Go on air blocked: a fresh source-bound server-side pre-flight did not pass." | A required checklist row failed when you clicked. Fix it, run **Run pre-flight**, try again. |
| "LiveSession already exists: council-live-room" | A live session has already been created on this station. See the Known issue in "Run a live session". Ask IT. |
| "Start was queued but the feed did not start." | The feed program did not respond in 20 seconds. Check Readiness, then click **Start** again. |
| "No live source is ready yet." (Take live is grey) | No source passed **Check source** in the last 30 seconds (by default). Run **Check source** on Live. |
| "Channel '<id>' is already under live takeover." | The channel is already live. Use **Return to schedule** first if you want to change it. |
| "Outgoing feed controls require the meeting operator role." | Your sign-in cannot start or stop channels. |
| A program says **Not safe to air yet** | Read the reason: missing media, or a clash with another program. Fix it on **Schedule** and review again. |
| **Couldn't reach the engine** on a committed program | The approval was saved, but the command to the feed did not go through. Check that the feed is running, and ask IT. |
| "Production control unavailable — the TSR control service is not running or not configured." | The Control Room helper is down. Tell IT. |
| "On-Air Mode expired before this cue could fire. Open a new On-Air session to continue." | The 30-minute limit passed. Open a new On-Air session. |
| "The cue preview is stale … Dry Run the cue again before Live Fire." | The cue or device changed after your dry run. Click the cue again. |

| What you see | What it means and what to do |
|---|---|
| "A session is already open on this surface, locked by <name> since <time>." | Another operator owns the surface. Your own open session normally restores after refresh. If the owner is unavailable, a Setup admin or Support admin can confirm a force-close on Control Room to release the lock; this does not undo cues already sent. |
| "Remote contribution isn't configured yet." | Guest video software has not been set up. Tell IT. |
| A guest-control notice says **sent, not verified** | The director iframe does not report command completion. Check the guest in the embedded director; use **Mark left after checking** only after verifying they have disconnected. |
| "Durable storage is not ready. Open Setup and choose Prepare storage ..." | The station's database is not ready. Tell IT. |
| "This action requires one of these CivicCast roles: ..." | Your sign-in lacks a role for that button. The names listed are the role names in short form. |

**Related**

- Signing in and finding your way around
- Before the meeting: schedules, agendas, recording schedules
- After the meeting: assets, captions, summaries, approvals
- Publishing, and what residents see
- When something looks wrong
- Configuring the station
- Cable headend, streaming, CDN, federation, emergency alerts, the API

## After the meeting: assets, captions, summaries, approvals

This chapter covers everything that happens to a meeting video once the meeting is over: finding it in the library, trimming it, making it ready for residents to watch, correcting its captions, reviewing the AI summary, and keeping the library healthy. It is normally done by a records clerk, a video editor, or a station volunteer. Publishing the video to residents is the next chapter: Publishing, and what residents see.

### Before you start

You need to be signed in to the staff console (see Signing in). Almost every screen in this chapter is in the left-hand menu under **Review Records**: **Assets**, **Missing Media**, **Media Lifecycle Settings**, **Review queue** and **Summary review**.

Words used in this chapter:

- An *asset* is one video in the library, with its title, details and files.
- *Packaging* means making a ready-to-stream copy of the video so residents can watch it in a web browser. It does not show the video to residents. Publishing does that.
- A *cue* is one caption line: a short piece of text with a start time and an end time.
- *Retention* is how long the station must keep a record before it may be deleted.

CivicCast gives each person one or more roles. The role decides which buttons work. In beta.11 most buttons look clickable to everyone, and a wrong-role click gives an error message such as "This action requires one of these CivicCast roles: ..." instead of being greyed out. The table lists who can do what in this chapter.

| Task | Role that can do it |
|---|---|
| See the Assets list, open a recording, read the Review queue | Any signed-in staff member |
| Upload a video | records_clerk, meeting_operator, support_admin |
| Edit title, description, meeting body, retention | records_clerk, meeting_operator, support_admin |
| Edit trim and chapters; preview source media | records_clerk, meeting_operator, support_admin |
| Package for playback; replace the source file | publish_operator, setup_admin |
| Place or clear a legal hold | records_clerk, support_admin |
| Approve, edit or reject a caption cue; retry a failed caption job | records_clerk |
| Generate an AI summary | records_clerk, support_admin |
| Approve a summary | records_clerk |
| Missing Media list | meeting_operator, publish_operator, support_admin |

> **Note:** A sign-in whose token has the administrator or operator scope holds all five roles, so nothing is refused. Role differences show up when a person has a limited, role-only sign-in.

> **For IT staff:** Role names and how to create role-limited sign-ins are in Part II (security chapter) and the roles appendix.

#### Find a video in the Assets list

1. In the left menu, click **Assets**.
2. To look for a video, type part of its title or its asset ID in the **Search assets** box. To narrow the list, click one of the tabs **All**, **Validated**, **Recorded**, **Analyzing** or **Rejected**.
3. Click a video's title to open its detail page.

You should see a table with the columns Title, State, Status, Duration, Size, Codec and Published. The asset ID appears in small type under each title.

The **State** and **Status** columns describe the same video in two ways.

| State | What it means |
|---|---|
| Validated | The file passed CivicCast's checks. It is ready to trim, package or schedule. Every upload, accepted contributor video and watch-folder file starts here. |
| Recorded | A live meeting recording that CivicCast has finished and saved. |
| Analyzing | CivicCast is still checking the file. |
| Rejected | The file failed the check. |

| Status | What it means |
|---|---|
| Not packaged yet | The file is validated, but residents cannot stream it yet. Use **Package for playback**. |
| Packaged | A streamable copy exists. Residents still cannot see it until it is published. |
| Published | Live on the resident portal. |
| Not servable yet | A live recording that has no streamable copy yet, so residents cannot play it. |
| Missing file | CivicCast cannot find the video file on disk. |
| Validating, Ingesting, Transcoding, Queued for transcode | CivicCast is working on the file. |

> **Known issue (beta.11):** The Assets list shows only the first 50 assets, ordered with published videos first (most recently published first), then everything else by asset ID. A new upload is not published yet, so it sorts after the published videos. The search box and the tabs search only those 50. A station with more than 50 videos can upload a file and not see it in the list. The screen does not say this and has no "next page" button. Workaround: ask IT staff to list assets through the API with a larger page size.

> **For IT staff:** The list calls `GET /api/staff/assets`, which takes `limit` (default 50, maximum 500) and `offset`, and returns the real total in the `X-Total-Count` header. See the API appendix.

#### Upload a video

Use this for a video that was not recorded by CivicCast, such as a file from a camera card or another recording system.

1. On the Assets screen, click **Upload video**.
2. In the **Video file** box, choose the video file. (While the panel is open, the **Upload video** button reads **Cancel upload**.) The accepted types are MP4, MOV, MKV, WebM, AVI or MPEG-TS (file endings .mp4, .mov, .mkv, .webm, .avi, .ts, .m2ts).
3. Check the **Title**. CivicCast fills it in from the file name, turning underscores and dashes into spaces. Change it if it is not what residents should see.
4. Click **Upload**. A progress bar shows "Uploading {file}..." and a percentage. Click **Cancel** to stop.

When it finishes you should see "Uploaded: {title}". The text below says the video now appears in the table, and the new row shows State **Validated** and Status **Not packaged yet**. No further action is needed for the check itself; CivicCast runs it before it says "Uploaded".

> **Note:** If CivicCast cannot read the file as a supported video, it refuses it and keeps nothing. The error appears in the upload panel and no row is created. The panel does not show a size limit. The server's default limit is 10 GB, set by the station's IT person.

> **Tip:** After uploading, set the title, description and **Meeting body** on the video's detail page right away (see the next section). Residents search and filter by these.

#### Edit a video's public details

The title, description and meeting body are shown to residents on the portal. Edit them on the detail page.

1. On the Assets screen, click the video's title.
2. In the **Public metadata** card, change **Title** (up to 200 characters), **Description** (optional, up to 2000) and **Meeting body** (optional, up to 120). The meeting body is the group that held the meeting, for example "City Council". Residents filter the Recordings page by it. Leave it blank and the video is not tagged, so it never matches a meeting-body filter.

3. Click **Save metadata**. The button says "Saving…" and then "Saved".

> **Warning:** If the video is attached to a schedule item that is already published (committed to air), the save is refused. The message reads "Asset X cannot be edited: N linked schedule item(s) already published. Unpublish or cancel the named items before retrying the edit."

The same card holds the retention controls. The four cards are **Default**, **Permanent**, **Meeting (long)** and **Short**, plus an optional **Retention deadline**. The screen reminds you: "Records officer review required. State presets provide a starting point, but local schedules, litigation holds, and official-minutes rules can require longer retention." It also says deletion is never automatic: expired items are flagged for the records clerk to review. A button, **Convert to a length + unit or forever term…**, switches the card to a length-and-unit form (days, weeks, months, years or forever).

> **Known issue (beta.11):** The console has no screen that lists the videos flagged for records review. The list exists only in the API (`GET /api/staff/records/disposition-queue`), and it is a list only: the code says an action screen is a later follow-up. Ask IT staff to read it for the records clerk.

> **Note:** Below the retention controls there is a custom-fields editor for any extra fields your station has defined. It is not described here because the fields differ from station to station.

#### Trim a video and mark chapters

Trimming sets where the video starts and ends. It does not change the original file.

1. Open the video's detail page and click **Edit trim & chapters**. On the Assets list you can click **Edit trim** instead, for a Validated row.
2. Use the timeline to move the playhead. The buttons step back or forward by one second (**-1s**, **+1s**) or one frame (**-1f**, **+1f**), or jump to the start and end.
3. At the moment the video should start, click **Set IN**. At the moment it should end, click **Set OUT**. You can also drag the **IN** and **OUT** handles on the timeline.
4. To mark a chapter, move to that moment and click **+ Mark**, then type a name for it. Remove a chapter with its remove button.
5. Click **Save trim & chapters**. A toast says "Saved." and the editor closes. To leave without saving, click **Discard**.

The preview has video playback controls. Use **Play selected trim** to inspect the chosen IN/OUT range. For packaged recordings, CivicCast plays the HLS copy and translates its playhead into the original source timeline, so trim points and chapter times retain their source positions. If the preview fails, use its retry action; the original recording is preserved.

> **Known issue (beta.11):** Chapters you mark are saved with the video, but we found nothing in the packaging code that writes them into the packaged video. The resident portal shows the meeting's agenda, not these chapters (see Publishing, and what residents see). To expose these

> markers to residents, use **Sync from chapters** on an agenda, review its times, and publish that agenda.

> **Warning:** For an uploaded video, the trim you save is used when the video is packaged. Set the trim before you click **Package for playback**. After changing an already packaged video, use **Repackage for playback** on the Assets screen to apply the new trim to the streamable copy. Existing English and Spanish caption tracks are rebuilt from their reviewed source cues before the new package replaces the old one. If a matching review is incomplete or no approved cues fall in the new range, repackaging refuses and keeps the previous playback package. For a live recording, the detail page says "Saving a trim re-renders the published recording automatically."

> **Note:** The trim also applies when the video airs from the schedule on a channel.

> **Note:** Caption review keeps timestamps from the original recording. Published English and Spanish captions are clipped to the range used to package the video and shifted to that playback timeline. For example, a source cue at 30 seconds appears at 28 seconds when the packaged trim starts at 2 seconds. Changing a saved trim takes effect in playback after repackaging; review records keep their original source timestamps.

If saving fails with a message about another writer, someone else changed the video while you were editing. Close the editor, open the video again and redo your change. If the video is already on a schedule item that is committed to air, the save is refused with the "cannot be edited … already published" message described under Edit a video's public details.

#### Package a video for playback

1. On the Assets screen, find a Validated or Recorded row with a stored source file and no streamable copy. An unpackaged Validated row says **Not packaged yet**; an unpackaged Recorded row says **Not servable yet**.
2. Click **Package for playback**. The button says "Packaging…" while it works.

You should see the Status change to **Packaged**. This needs the publish_operator or setup_admin role; otherwise the row says "A publish operator or setup administrator must package this recording."

CivicCast packages one video at a time. If another is already running you see "Another recording is already being packaged…". Wait and try again. If packaging fails, the message is the server's text or "Packaging failed. The original file was kept; try again."

The list offers **Package for playback** for Validated and Recorded assets with a stored source file, or **Repackage for playback** when a streamable copy already exists. This also lets you package a recording whose automatic packaging did not finish.

Packaged is not the same as published. Next, go to the Publish screen (Publishing, and what residents see).

#### Take a video off the public portal

1. Open the video's detail page.
2. In the **Technical** card, next to **Published**, click **Remove from portal**.
3. Read the dialog and confirm. It says: "Residents will no longer be able to view or find it there. This only affects portal visibility – the asset row, its media, and the Internet Archive / syndication copies are untouched."

The video disappears from the resident portal. Publish shows it as a draft awaiting portal approval again; previous delivery history is retained. This needs publish_operator or setup_admin.

#### Check a video's readiness, hold it, or replace its file

At the bottom of the **Technical** card is the **Media lifecycle** panel.

- A **readiness badge** says Ready, Transcoding (n%), Queued for transcode, Missing, Rejected or Not ready.
- **Loudness gate** shows "OK (-x.x LUFS)", "Failed — normalize before air" or "Not checked yet". LUFS is a measure of how loud the audio is. CivicCast checks against -16 LUFS with a margin of 1. The screen has no button for fixing a failed loudness result, so the line is for information.
- **Archival** lists three copies: the portal, the Internet Archive and a local NAS (a network storage box at the station). Each shows "verified" or "not verified", and the panel ends with "Archive-complete" or "Not archive-complete yet".

To place a **legal hold**, optionally type a reason, click **Place legal hold (blocks expiry)** and confirm "Place a legal hold on this asset?". A held video cannot expire under its retention rule. To release it, click **Clear legal hold** and confirm. Both actions are recorded in an audit log. This needs records_clerk or support_admin.

To replace the video file, choose a **Replacement video file** and confirm "Replace this asset's source file?". CivicCast renames the old file aside and never deletes it, and the video goes back to State **Validated**. This needs publish_operator or setup_admin.

> **Warning:** The replace-source dialog says viewers see the new file "immediately once processing finishes". In beta.11 that is not what the code does. The replacement resets the video to Validated and clears its transcode jobs, but it leaves the old streamable copy and the publish date in place. Residents keep seeing the old video. Use **Repackage for playback** in the Assets list to rebuild the streamable copy from the replacement source. Until that succeeds, residents keep seeing the earlier package.

> **For IT staff:** The repackage action uses `POST /api/staff/assets/{id}/package` for that asset. See the API appendix.

#### Audio tracks, loudness plan and caption status

Three read-only cards are on the **Channels** screen, not on Assets. They describe a channel on the air, not a recording.

- **Audio tracks (SAP / descriptive)** lists any secondary audio tracks, such as a second language (SAP) or described video, each marked "on air" or "disabled". If there are none, it says "Single audio program — no secondary (SAP / descriptive) tracks configured." There is nothing to edit in the console.
- **Loudness** shows, for each output of the channel, the target loudness and standard, and a "Measured -x.x LUFS" chip. The card says each output is set to its destination's standard: cable to ATSC A/85 (-24 LKFS) and streaming to -16 LUFS. It is visible only to setup_admin and support_admin; a meeting operator sees a refusal message in the card.
- **Captions** shows whether captions are reaching the output: "Captions on", "Captions off", "Caption proof failed" or "Not verified". It reads "on" only after a recent passing check. The card says: "This proves carriage at the egress boundary; it is not a claim of FCC Part 79 compliance." It refreshes every 30 seconds.

See Running the meeting for the Channels screen.

#### Offline captions: how a recording gets its captions

Configure recorded captions under **Setup > Station Profile > Recorded captions**. Choose whether processing is enabled, whether to include Spanish alongside English, and whether completion waits for all selected languages or only reviewed English. These are station choices. Captions for a recording are made in the background after you approve publishing on the Publish screen; there is no separate "make captions" button.

1. With recorded captions enabled, you approve publishing the recording. CivicCast queues a caption job for it first, even if the Portal row is not selected. If it cannot queue the job, publication reports the failure. A recording with no stored video file gets no caption job. When recorded captions are disabled, new caption jobs are not created and existing jobs pause until processing is enabled again.
2. The job transcribes the audio into English cue lines. On the video's detail page, the **Offline caption jobs** card shows **Transcribing...**. A full meeting can take several minutes, not seconds.
3. When transcription ends, the card shows **Awaiting review**. The cues are now in the **Review queue**.
4. You decide every English cue. If Spanish is selected, CivicCast also translates the reviewed English and queues the Spanish cues for review.
5. With **all selected languages** required, the selected tracks attach after their review is finished. With **reviewed English** required, the English track can attach first; selected Spanish captions attach later after their own review. English-only jobs do not need a translator or Spanish review.

> **Note:** Changing the completion setting never publishes unreviewed caption text. Disabling processing does not mark an uncaptioned recording as captioned.

If the job runs out of attempts, the card shows **Failed** with the reason in the **Last error** column. A records clerk can click **Retry** and confirm "Retry this offline caption job?". This restarts transcription from scratch with a fresh attempt budget and discards partial progress. Without the records clerk role the card says "Retrying a failed job requires the records clerk role."

The job can also be held. These are the reasons CivicCast puts on the row:

- Every English cue was rejected. Open the Review queue, choose English, and edit or approve cues. If the audio is truly unusable, ask a technical administrator to cancel the job.
- Every selected Spanish cue was rejected. Open the Review queue, choose Spanish, and edit or approve cues. The Spanish track attaches once review is complete and at least one cue is approved or edited. Under **Attach English after its review**, reviewed English can already be available.
- Spanish is selected but no translation model is available. The row says to install or repair it under "Settings > AI Models > Translation" and run 'civiccast doctor'. With **Wait for every selected language**, CivicCast retries with growing waits; after its attempts run out, the card shows **Failed** and a records clerk must click **Retry** after repair. With **Attach English after its review**, reviewed English attaches while the active job waits for Spanish. English-only jobs do not encounter this hold.

Published English and Spanish captions use the packaged playback timeline: cues outside its trim are excluded, crossing cues are clipped, and retained cues are shifted by the packaged IN point. Review records keep their original source timestamps. Repackaging rebuilds existing tracks from reviewed source cues before replacing the old package; incomplete matching review or an unusable caption range leaves the previous package intact.

#### Review and correct captions

The **Review queue** (page heading "Caption review") is where a person checks the machine's caption lines before the public sees them.

This queue is for recorded-caption jobs. Ordinary live captions go to the broadcast but are not saved here or turned into per-cue audio evidence. A live session therefore does not populate this queue; its empty state points operators to captioned recordings instead.

1. In the left menu, click **Review queue**. It opens on the **Pending** tab. The other tabs are **All**, **Edited**, **Approved** and **Rejected**, and a **Search** box above them filters by asset ID or caption text.
2. To work on one language, click **English** or **Spanish** in the **Language** row. **All languages** shows both.
3. Each card is one cue. It shows the recording's asset ID, the cue's time range, a language badge (EN or ES) and a status. The left box **Machine cue** is the computer's text and cannot be changed. The right box **Reviewed text** is yours to edit.
4. If the cue is wrong, change the **Reviewed text**, then click **Save edit**. The status becomes **Edited**, and the card leaves **Pending**. If you want that correction included in an AI summary, switch to **Edited** and click **Approve** on the saved cue. Complete the audio-evidence check below if the cue has low confidence.
5. If the cue is right, click **Approve**. The status becomes **Approved**.
6. If the cue should not be shown at all, click **Reject**.

You should see the card leave the Pending tab and the status pill change. The list refreshes after every decision.

What each button really does:

| Button | Result |
|---|---|
| **Approve** | Marks the cue Approved. It approves the text that is already stored: the machine text, or the text from your last **Save edit**. |
| **Save edit** | Stores what is in the **Reviewed text** box and marks the cue Edited. It is available only when the text has changed and is not empty. |
| **Reject** | Marks the cue Rejected and throws away any reviewed text. A rejected cue is left out of the captions. |

Approved and Edited cues both become captions, using the reviewed text. Rejected cues are left out. Pending cues hold their language's track back; selected Spanish review does not hold reviewed English back when **Attach English after its review** is selected.

> **Note:** The Review queue warns, "Save edit before approving if you want to keep these text changes." **Approve** uses the stored text; it does not save an unsaved draft. **Save edit** counts as a decision for the caption track. For AI summaries, open **Edited** and approve the saved correction as well; the console's summary workflow uses Approved cues only.

> **Warning: Reject** asks no question and discards the reviewed text. A rejected cue can be approved again later, which then approves the machine's original text, not your earlier correction.

**Low-confidence cues** A cue the machine was unsure of carries a **Low confidence** badge, and a yellow bar at the top says how many need attention. For these cues, **Approve** stays disabled until you do two things:

1. Click **Load review audio**. CivicCast fetches a clip covering the complete cue, with nearby context where available, and shows an audio player. Play it. The clip is bounded to 120 seconds. If the source cannot cover the whole cue, the message explains the missing coverage and how to regenerate or restore the needed source; a partial clip is not presented as complete evidence.
2. Tick **I compared this low-confidence cue with its audio evidence.** This box becomes available only after the audio can play.

This audio check applies to **Approve** only. **Save edit** has no such check, so a low-confidence cue can be edited and counted as decided without anyone playing the audio.

If the audio is not available, the card says "Audio evidence is unavailable. Approval of this low-confidence cue is blocked." You cannot approve that cue until the audio is available, so ask IT staff. If the recording's media file is missing, loading the audio fails with "The local meeting media is missing. Relink or restore the asset, then retry."

**Who can use it** Everyone can read the queue and play audio. Only a records clerk can approve, edit or reject. For anyone else a yellow note says "Caption review actions require the records clerk role. The queue stays visible for read-only review." and the boxes and buttons are disabled.

**If it did not work**

| What you see | Cause and fix |
|---|---|
| "No caption cues need review." | There are no cues at all for the chosen language, in any tab. Cues appear after a recording is approved for publishing on the Publish screen and the caption job has finished transcribing. |
| "No captions match the current search and filter." | Cues exist, but not under the tab or search you chose. After you decide the last cue on the **Pending** tab you will see this line. Click **All**, **Edited**, **Approved** or **Rejected** to see the decided cues. |
| "Spanish cues appear here after English captions are approved and translated." | Shown beside the **Language** row when the **Spanish** choice has no cues. The Spanish cues are made only after the English pass is fully decided. The Spanish text is a machine translation and needs review too. |
| "Caption review backend unavailable." | The station's database is not connected. Tell your IT person. |
| "Could not complete caption review action." after you clicked a button | The action failed and was not saved. Dismiss the message and try once more; if it keeps failing, ask your station administrator to check the CivicCast service and database. |
| "This is a low-confidence caption cue. Compare it with the retained audio, then explicitly acknowledge that review before approval." | Load and play the audio, tick the box, then Approve. |

#### Review an AI summary

CivicCast can write a draft summary of a meeting from its approved caption lines. A records clerk checks and approves the draft, then exports a signed-record PDF for station records.

**Make a summary**

1. Approve at least some caption cues for the recording in the Review queue. This console workflow uses only **Approved** cues. If you corrected cues with **Save edit**, open **Edited** and click **Approve** on those saved corrections, completing the low-confidence audio-evidence check when required. Edited cues that have not been Approved are excluded from the summary.
2. Open the recording's detail page and find the **AI summary** card, under the **Offline caption jobs** card.

3. Click **Generate summary**. The chip reads "queued" and then "generating…". The screen says generating locally can take 1-6 minutes on a CPU-only station, and you can close the tab; the job keeps running.
4. When it finishes the card says "Summary generated. Review it in Summary review".

If there are no approved cues the card says "No committed transcript cues yet. Approve caption review items for this recording first, then a summary can be generated from them." You need the records_clerk or support_admin role. If the job fails, the card shows **Failed** with the reason and a **Retry** button, which only a records clerk can use. The local AI program (Ollama) must be running with the summary model installed. If it is not, the job fails and the card says "Local Ollama AI runtime is not reachable. Start Ollama and retry, or configure a different summary model in AI model settings."

**Look at a summary** Local generation checks memory before loading the summary model and reserves 8 GiB for station operation. A resource-deferral message explains how to recover; wait for memory to become available or choose a smaller model before retrying. CivicCast unloads the local generation model after the request completes.

> **Note:** In one Beta 15 field run, 12.7 GiB was available against an 8 GiB model request plus an 8 GiB broadcast reserve, so the safeguard deferred generation. The guard worked, but summary editing, approval, export and publication were not completed in that run. Usability and full-workflow follow-up was recorded for Beta 17; this observation does not establish a defect in the current package. See the deferred field report.

1. In the left menu, click **Summary review** (page heading "Summary review", label "Summary + signed records").
2. Each card shows the asset ID, a status, the summary paragraph, and a list of **Sourced claims**. Under each claim are buttons labeled with a cue ID and its time range.
3. Click a cue button. The **Inline transcript player** highlights the range and loads the private source recording with playback controls. The player seeks to the cited source time; press Play to listen. Packaged recordings map the trimmed playback timeline back to source time. If the cited passage lies outside the available copy, the player explains that rather than seeking to a different passage. Use **Renew playback** if access expires. To read caption text, open the recording or Review queue.

The page shows **Pending review**, **Approved** and **Needs evidence** items. Approved summaries remain on the page so you can export or revisit their signed records. A **Needs evidence** card has a message explaining that the model's output could not be tied to caption cues with timestamps. A yellow bar counts items that need more evidence.

If nothing is waiting, the page says "No summaries need review." and tells you to use **Generate summary** on a recording's detail page.

**Edit, approve and export**

1. For a **Pending review** item, click **Edit summary** to revise its narrative. Click **Save changes** to keep the edit or **Cancel edit** to discard it. Sourced claims and their transcript references remain unchanged; compare the narrative with those claims before approval.
2. Click **Approve summary** only after checking the summary against its source cues. Approval is recorded with the identity from your signed-in staff account. After approval the card stays available with its status set to **Approved**.
3. On an approved card, click **Export signed record**. CivicCast creates a PDF/A-3B record and shows its record ID and digest. The timestamp is a deterministic test timestamp unless the station has configured a real timestamp authority.
4. Click **Download signed record** to save the PDF. Click **Verify signed record** to ask CivicCast to check its stored artifact and timestamp proof.
5. To return to an earlier export after leaving or refreshing the page, open the approved card and click **Load saved records**. CivicCast lists up to 10 recent exports for that summary; use each row's **Download** or **Verify** action.

Only a records clerk can edit, approve, export, download or verify. If an action fails, the page shows an error and a recovery step. Stale edits and approvals are rejected; reload and review the updated summary before trying again.

The summary review page does not provide a reject or regenerate action. To generate another draft, update the caption review as needed and use **Generate summary** on the recording's detail page.

> **For IT staff:** The approve route is `POST /api/staff/summaries/{id}/approve`; the request requires the displayed draft's `expected_audit_fingerprint`, and the server takes operator identity from the signed-in staff session. Approval atomically checks that the draft is still pending and unchanged; otherwise it returns 409 with reload guidance. Edits use `PATCH /api/staff/summaries/{id}` with the draft's expected audit fingerprint. Signed-record metadata is available from the records-clerk-only `GET /api/staff/records?summary_id={id}&limit=10` route; export, download and verify routes remain under `/api/staff/records`. See the API appendix.

#### Check for videos that are missing before a meeting

The **Missing Media** page is a warning list. It shows meetings on the schedule for the next 7 days whose video is not ready to play.

1. In the left menu, click **Missing Media**. Only meeting operators, publish operators and support admins see this entry.
2. Read each card. It names the meeting and its time, the channel, the video's asset ID and state, and gives a reason.
3. Click **Open asset** to go to the video's detail page and fix it.

The reasons are: "Referenced asset no longer exists.", "Asset is in state '{state}', not validated/recorded." and "Asset's backing file is missing." Every card shows "Not ready" whatever the reason. When nothing is wrong the page says "Nothing missing."

The page loads once when you open it. Leave it and come back to refresh. To fix a card you usually need to upload a replacement file or ask a publish operator to replace the file.

> **Known issue (beta.11):** "Every asset scheduled in the coming week is validated or recorded and ready for air" is too strong. The page checks only schedule items that are still drafts. Items already committed to air are not checked. A file deleted in the last hour may not be flagged yet, because the file check runs once an hour by default.

If you open the page with a role that is not allowed, you see "This action requires one of these CivicCast roles: meeting_operator, publish_operator, support_admin."

#### Watch folders, retention rules and storage (Media Lifecycle Settings)

This page holds three station-wide settings. In the left menu it is **Media Lifecycle Settings**, shown to setup_admin, publish_operator and records_clerk.

- **Watch folders.** A watch folder is a folder on the station's computer (a local disk, a USB drive or a network share) that CivicCast checks automatically and copies new videos from. Type the folder path in **Watch folder path** or click **Browse...**, then click **Add watch folder**. A new file is imported once it has stopped growing. The original stays where it is, and the new video appears in Assets as Validated. Click **Scan now** to check immediately, or **Remove** to stop watching (files already imported stay).
- **Retention automation.** A rule sets a video's retention policy by its meeting body. Fill in **Rule name**, **Meeting body (exact match)** and **Retention policy**, then click **Add rule**. Then click **Apply rules now**.
- **Storage budget.** A read-only view of how much disk the library uses, with a table by retention policy.

Each folder row shows **Not scanned yet**, **OK** with "Last poll" and "Last ingest" times, or **Degraded** with the reason, for example a drive that was unplugged.

> **Note:** Rules are applied only when someone clicks **Apply rules now**; expired assets are not deleted automatically and are flagged for records review. A rule whose **Meeting body** is blank matches nothing. Applying a rule sets the retention policy label on every video whose meeting body matches, replacing a policy someone chose by hand on that video. Applying a rule does not recalculate the retention deadline.

> **Warning:** The **Browse...** and **Scan now** buttons need the setup_admin role, adding or removing a watch folder needs publish_operator or setup_admin, and retention buttons need records_clerk or setup_admin. The page shows every button to every role, and a wrong-role click returns an error message.

> **For IT staff:** Watch folders, upload storage and the storage budget are set up in Part II (configuration and operations chapters). If the budget card says no budget is configured, the setting

> is an environment variable that only IT can change.

**If it did not work**

| What you see | Cause and fix |
|---|---|
| `"{name}" is not a supported video file. Accepted types: MP4, MOV, MKV, WebM, AVI, or MPEG-TS.` | The file ending is not one of the accepted types. Convert the video and upload again. |
| "Durable storage is not ready." with "Open Setup, prepare durable storage, then return here." | The station's storage has not been set up. Ask a setup administrator. |
| "Could not load assets." | Click **Retry**. If it fails again, tell your IT person. |
| "Save failed." on the detail page | Another person changed the video, or a linked published schedule item blocks the edit. Reload and retry, or unpublish the named items. |
| "Another recording is already being packaged…" | CivicCast packages one video at a time. Wait and click again. |
| "This action requires one of these CivicCast roles: …" | Your sign-in does not have the needed role. Ask your station administrator. |

**Related**

- Publishing, and what residents see
- Before the meeting: schedules, agendas, recording schedules
- Running the meeting
- When something looks wrong

## Publishing, and what residents see

This chapter explains how a recording goes from your library to the public: the Publish screen, what "published" really means, what each publishing step does, and the Playback policy and Paywall screens. The second half is a tour of the resident portal, the public website that residents use to watch meetings. It is written for the people who publish (normally a publish operator) and for anyone who needs to know what a resident sees.

### Before you start

- The recording must already be **packaged**, which means it has a ready-to-stream copy. If it is not, CivicCast refuses to publish it. See After the meeting.
- Approving and retrying a publishing step needs the **publish_operator** role. A setup administrator or a support administrator cannot publish unless their sign-in also holds publish_operator. Anyone signed in can look at the Publish screen. Without the role, the checkboxes and buttons are greyed out and the screen says "Publish operator role required to approve or retry surfaces."
- Words used here. A *surface* is one place a recording can be sent to, such as the resident portal or the Internet Archive (a free public online library). A *manifest* is the small file that tells a web browser how to play a video. The *resident portal* is the public web page, served from the station's own computer.

> **For IT staff:** The portal is served at the web root of the station and the staff console at `/operator/`. Settings for the outside services (Internet Archive, NAS, YouTube) are in Part II (integrations chapter).

### What "published" means

A recording is **published** when CivicCast sets its public publish date. From that moment it appears in the resident portal's Recordings list and has its own Watch page. That is the whole meaning. Three things are separate from it:

1. **Packaged** only means a streamable copy exists. Residents cannot see a packaged recording until it is published.
2. **Archive copies** (Internet Archive, local NAS) and **outside services** (YouTube) are other steps on the same screen. Publishing to the portal does not require them.
3. **Captions** are made after you approve. A recording can be public before its captions are reviewed.

Taking a recording off the portal is done on the video's detail page with **Remove from portal** (see After the meeting). That removes it from public view. It does not undo anything sent to other places.

### Publish a recording to the portal

1. In the left menu, click **Publish**.
2. Find the recording's card. You can narrow the list with the filter tabs: **All**, **Needs action**, **Draft**, **Archive pending**, **Archive verified**, **Reaching fewer places than planned** and **Complete**.
3. In the card, make sure only the **Portal** row is ticked. That is the default. The checkbox is labeled **Approve this surface**.

4. Click **Approve and Publish selected**.
5. Read the dialog titled `Publish "{title}" to residents?`. It says how many surfaces will publish for real, and that "The portal surface becomes publicly visible to residents immediately and starts offline caption transcription."
6. Click **Approve and Publish**. To back out, click **Cancel**.

You should see the **Portal** row change to "Succeeded". The card's state becomes **Complete**, or **Archive pending** if the recording is a public record (a recording kept with the Meeting (long) or Permanent retention policy) and archive copies are still needed. The **Portal live** tile at the top counts the recording.

At the top of the screen, tiles count your recordings: Total, Draft, Portal live, Archive verified, Degraded and Needs action. Each card shows three facts: **Canonical** (Portal public or Portal pending), **Archive** and **Published** (the date, or "Not public yet").

**What happens behind the button** When you approve publishing, CivicCast does these things in this order:

1. It checks the recording has a streamable copy. If not, it stops with "Publish preflight blocked: this asset has no manifest_url. Run the packager or fix ingest before approving publish."
2. This edition queues recorded-caption work only when the station's recorded-caption setting is enabled, and only for a recording with a stored video file. This happens even if the Portal row is not selected. When captions are disabled, approval can proceed without captions; existing jobs pause, and uncaptioned media is not marked caption-complete. The Beta 15 package predates this setting and attempts to queue caption work on approval. See Recorded-caption settings for the controls and whether reviewed English attaches first or waits for all selected languages. If enabled caption work cannot be queued, publishing stops with "Publish blocked: CivicCast cannot queue this recording's caption job … Nothing was published." and names the cause.
3. It runs the surfaces you ticked.
4. If the Portal row is selected, it sets the public publish date. If that fails, the Portal row shows "Failed", the message says "The recording remains private", and you can click **Retry this surface**.

> **Warning:** If the station's federation setting is on, every approval that leaves the recording public on the portal also posts a public notice, "New CivicCast recording published: {title}.", to the outside servers that follow the station. (Federation is a setting that shares public notices with other servers on the internet.) The Publish screen does not mention this, and a notice that has gone out cannot be taken back. Ask your IT person whether federation is on.

#### The publishing steps (surfaces)

Each card has nine rows. Each row shows a status dot, a name, badges such as **Required** or an approval word, a line such as "archive / Succeeded", a message and a **Next step**. All nine surfaces below are implemented; optional surfaces still need their provider settings and prerequisites.

| Row | What it does in the current source |
| --- | --- |
| Portal | Makes the recording public on the resident portal. Recorded-caption job behavior follows the source/package version and station setting described above. |
| Internet Archive | Mock by default. A real adapter requires the `CIVICCAST_PROVIDER_INTERNET_ARCHIVE=real` service setting and credentials. It uploads the recording when a local file resolves; without one, the adapter receives an asset/title payload rather than the media. |
| Local NAS rsync, Local NAS ZFS | These two rows share the mock-by-default local archive adapter. A real adapter copies to the configured mounted folder (local path or UNC) and verifies the copies; the names do not mean CivicCast runs rsync or manages ZFS. Without a local recording file the copy is metadata only. |
| YouTube Live, YouTube VOD | Mock by default. Real VOD upload requires a resolved recording file or the configured `CIVICCAST_YOUTUBE_MEDIA_ROOT/<asset_id>.mp4`. The Live adapter provisions and binds a YouTube broadcast and stream, but this Publish path does not send CivicCast's live output to YouTube's ingest address. |
| Podcast episode | Optional. Converts the asset's local recording to an MP3, saves it as an episode, and adds an audio enclosure to the channel RSS feed. Publish a new recording to Portal first, then select or retry Podcast. A missing recording or conversion error appears as a retryable failure. IT must set a public station URL and provide FFmpeg. |
| Subscriber notifications | Optional. When selected, CivicCast sends notices to confirmed subscribers through configured mail or webhook providers, records each delivery outcome and retries eligible failures. The row shows delivered, queued and failed counts. Mock providers are labeled simulated. |

| Row | What it does in the current source |
|---|---|
| Cable file package | Builds a ZIP with the video, captions, details and file fingerprints for the cable headend, in a folder IT has chosen. If no folder is set the row says it is "not set up (optional)". |

Provider selectors default to mocks, which do not send or archive media. The Setup Provider form saves values for station setup but does not configure these publishing adapters; IT must set the service environment described in Integrations. The capability matrix lists the current support boundary.

A step's status reads **Not run yet**, **Running**, **Succeeded**, **Failed**, **Blocked**, **Overridden** or **Not set up yet**. The recording as a whole shows one of **Draft**, **Preflight blocked**, **Publishing**, **Archive pending**, **Archive verified**, **Reaching fewer places than planned**, **Complete** or **Needs action**.

A recording that has not been packaged still gets a card. Its Portal row reads **Blocked** with the message "The portal cannot publish this asset until an HLS manifest exists.", the card reads **Preflight blocked**, and **Approve and Publish selected** stays disabled while that row is ticked.

The words **Required** and the kinds (canonical, archive, reach, record, audience) appear unexplained on screen. In plain terms: *Required* means an archive copy is needed before a public-record meeting counts as archived. *Canonical* is the portal. *Archive* is a stored copy. *Reach* is an outside service such as YouTube. *Record* is the cable package. *Audience* is podcast and notifications.

**Skip a required archive step** If a required archive copy cannot be made, a publish operator can skip it with an audit-logged reason.

1. In the archive row, tick **Use audit-logged archive override for this platform**.
2. Type the reason in the box. The screen needs at least 20 characters.
3. Approve as usual. The row becomes **Overridden** and counts as done.

**Retry a failed step** When a row shows **Failed**, read its message and **Next step**, fix the cause, then click **Retry this surface** on that row. A Portal retry queues caption work when recorded captions are enabled and the recording has a stored video file. Retrying another surface does not start caption work.

> **Known issue (beta.11):** If you approve a second time, CivicCast rebuilds the whole run. Any step you did not tick again goes back to "Not run yet", including the Portal row, and the card can read **Draft** and "Portal pending" even though the recording is still public. The screen's own text says archive and outside steps are opt-in "this time", which invites exactly this. Workaround: tick every step you want, all in one approval. To redo a single step, use **Retry this surface** on a Failed row instead of approving again: a retry changes only that row. If the Portal row has gone back to "Not run yet", its checkbox appears again; make sure it is ticked along with your other steps and approve again.

> **Known issue (beta.11):** The "Archive verified" state and the card text "IA and local NAS verified" (IA is the Internet Archive) can appear when the copies were simulated or overridden. Read the row-level note. A row that says "Simulated — nothing was actually archived. This is not the legal archive copy." has not archived anything. The same is true of YouTube: with the default setting the YouTube rows can say "YouTube Live RTMPS fanout proof succeeded." although nothing was sent to YouTube (RTMPS fanout is the live-stream hand-off), and the rows carry no "Simulated" note. The card's **Readiness check** list does warn that the YouTube preflight "is simulated (mock provider)".

> **Note:** If publishing stops, the message says some earlier steps may already be published. Check each surface status before retrying, and retry only the failed surfaces.

> **Known issue (beta.11):** The Publish screen always records the person as "Operator dashboard", not your name, in its audit trail. Do not promise per-person publishing records.

> **Warning:** Uploads to the Internet Archive and YouTube, once real, cannot be taken back from CivicCast.

**If it did not work**

| What you see | Cause and fix |
|---|---|
| "No assets are ready for publish review." | The library has no recordings at all. Upload or record a meeting, and package it. (A recording that is not packaged still shows a card; see above.) |
| "Publish dashboard needs a database." or "Durable storage is not ready." | The station's storage is not ready. Tell a setup administrator; they use Setup and **Prepare storage**. |
| "Publish preflight blocked: this asset has no manifest_url. ..." | The recording has no streamable copy. Use **Package for playback** on the Assets screen. |
| "Publish stopped. {detail} ..." | Read the detail text. It names the problem. |

#### Playback policy: what is and is not enforced

The **Playback policy** screen lets you record who is supposed to be allowed to watch a channel or a single recording, plus a lock for public records, and up to four "prerolls" (a short card or clip meant to play before a recording). It is for the publish_operator and support_admin roles. Anyone can look at it. Other roles can type changes but get a refusal when they click **Save policy**.

What you set:

1. Choose **Channel** or **Asset** under **Policy target**, and type the **Target ID**, the channel's or record-

ing's ID. The default is `government`. CivicCast accepts any ID made of lowercase letters, digits, _ and -, even one that does not exist, so a typing mistake quietly creates a policy for nothing. Copy the ID from the Channels or Assets screen. A policy on a recording wins over the policy on its channel.

2. Choose **Public**, **Authenticated** or **Invite only** under **Access tier**. **Invite group** and **OIDC provider** (a single-sign-on service) are free-text boxes. Invite only needs an invite group name before **Save policy** works.
3. Tick **Public-record asset** or **Public archive complete** if the record must stay public. Ticking either one switches the tier to Public, and the server rejects any public-record policy that is gated.
4. Optionally tick **Preroll enabled**, click **Add preroll**, and fill in at least the creative ID, the file address, an accessible label and a duration.
5. Click **Save policy**. There is no confirmation message. The only sign is the "Updated {date}" box changing.

The **Decision audit** at the bottom lists the last 8 allow or block decisions with **Refresh**.

> **Known issue (beta.11): Access tier** does not stop anyone from watching a video. The setting is read in only three places: the podcast feed, the app catalog, and a public "evaluate" service. The resident portal's video player and the video file server never read it. Choosing **Invite only** or **Authenticated** does not put a gate in front of the Watch page. Residents do not sign in on the portal for playback. Do not rely on this screen to restrict a recording. Use **Remove from portal** to withdraw a recording instead.

Exactly what is enforced today:

| Setting | Enforced? |
|---|---|
| Access tier on a **channel's podcast feed** | Yes. If the tier is not Public, the feed answers "Authenticated RSS is not enabled for this channel." unless **Authenticated RSS** is ticked, and then needs a signed viewer token, a coded pass for one listener ("A signed viewer token is required for this podcast feed."). The console has no button to make a viewer token; the server can issue one only through an API call. |
| Public-record lock | Yes, when saving. A public-record policy cannot be gated. |
| Access tier for the portal's video | No. |
| Prerolls | Saved, but we found nothing in the resident portal that plays them. We could not confirm that any app plays them. |

| Setting | Enforced? |
|---|---|
| Access tier in app listings | The tier is copied into the app catalog as information. We could not confirm that any app enforces it. |
| Decision audit | Fills only when something asks the policy service for a decision, which today is the podcast feed. It usually stays empty ("No playback decisions yet."). |

#### The paywall: optional paid access

Some stations may want some recordings behind a paid subscription. The **Paywall** screen is where a setup administrator configures it. It is off by default, and when off everything is public. It sits in the Setup part of the menu and is visible only to setup administrators. Everyone else sees "Forbidden — the subscription paywall is a setup-admin surface. Ask your station admin for access."

The screen has three cards: **Config** (the **Enable paywall** box, a provider, a signing secret), **Tiers** (plans that map to prices you create yourself in Stripe, a payment company) and **Comp access grants** (free passes you give to an email address).

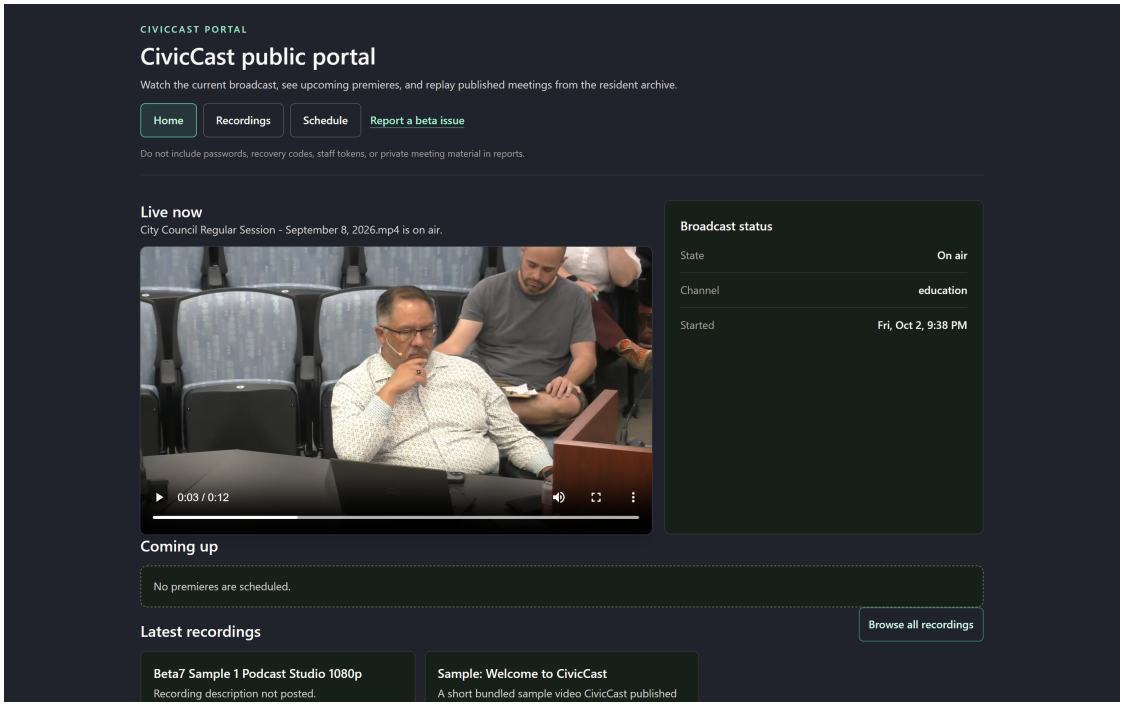
> **Beta 19 status:** Treat the Paywall as a test-only screen, not protected paid access. In the current code:
>
> - Turning it on does not select particular recordings. The Watch page asks every resident for a pass on every recording.
> - The video file itself is not protected. The gate is a screen shown in front of the player.
> - The "Email me a sign-in link" button sends through the configured mail provider and reports a delivery failure when that provider is unavailable.
> - The pages the portal uses to list plans and start payment do not exist on the server, so a resident cannot subscribe.
> - An ordinary save with the **Signing secret** box left blank preserves the stored secret. To remove it, use the explicit clear control. The secret is hidden when the page reloads.
> - The list of comp grants shows only grants made in the current browser session, and does not provide a history of grants from earlier sessions.

If you turn the paywall on for testing, the Watch page shows the gate described in the resident tour below. Residents will be blocked.

#### Resident tour: the public portal

The portal needs no sign-in. It is English only and always shows a dark theme. At the top are the title "CivicCast public portal", three buttons **Home**, **Recordings** and **Schedule**, and a link **Report a beta issue**. The page address changes as you move, so any page can be copied and shared. On a phone the same page stacks into one column.

*Figure: the portal Home page.*




The portal Home page on a computer while a channel is on air.

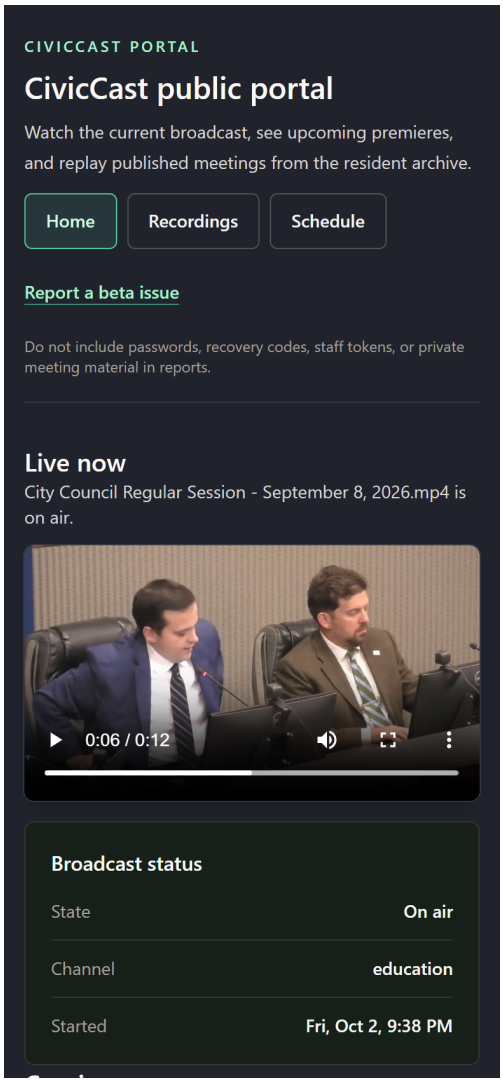
*Figure: Home on a phone.*

> **Known issue (beta.11): Report a beta issue** opens the staff console's help page in a new tab, not a page written for residents. The text under it, "Do not include passwords, recovery codes, staff tokens, or private meeting material in reports.", is written for staff. The portal has no help page, no privacy statement, and no contact details for the station.

**Home and Live now** Home has these parts, top to bottom:

- **Live now.** A sentence and a video, with a **Broadcast status** card beside it (State, Channel, Started). The portal checks again every 4 seconds, so a page left open follows the channel.

| What the station is doing | What residents see |
|---|---|
| On the air with web video | "{title} is on air." and the player. State reads "On air". |
| On the air, web video starting | "On air. The web preview is turned on but not serving yet." The page checks again by itself. |
| On the air, no web video | "On air, but web preview is not enabled for this channel." |
| Standing by | "The station is standing by. No program is on air right now." |
| Anything else | "No live broadcast is on air." State reads "Offline". |




The portal Home page on a phone-sized screen.

- **Coming up.** Cards for upcoming premieres, each with the title, the date and time, and the channel. Empty: "No premieres are scheduled."
- **Latest recordings.** Up to 6 of the newest published recordings, each with **Watch recording**, and **Browse all recordings** to go to the Recordings page. Empty: "No published recordings are available."
- **Follow new recordings** and **Submit a program**, covered below.

If nothing at all is posted the page says "Nothing is posted yet. Check back after the station schedules a premiere or publishes a recording." If a part cannot load, an amber list "Some portal sections need attention" says which. If all three fail the page says "The public portal could not load right now. Refresh the page, then contact the station if the problem continues."

The Home page reads the real emergency notice for its current channel automatically and checks again every five seconds. No special page link is needed. Its notice reflects alerts presented by the station. See Understand emergency alerts.

### Recordings: search and filters

1. Click **Recordings**.
2. Type words in **Search recordings** and click **Search** (pressing Enter works too). The search looks only at the title and the description, not at captions, agendas or the spoken words. It matches any part of a word, and capital letters do not matter.
3. To narrow results, use **Year** (the years of published recordings), **Meeting body** (the meeting body on the recording, such as "City Council") and any extra menus your station has made.
4. Use **Previous**, the page numbers and **Next** to move through the results. There are 12 recordings per page.

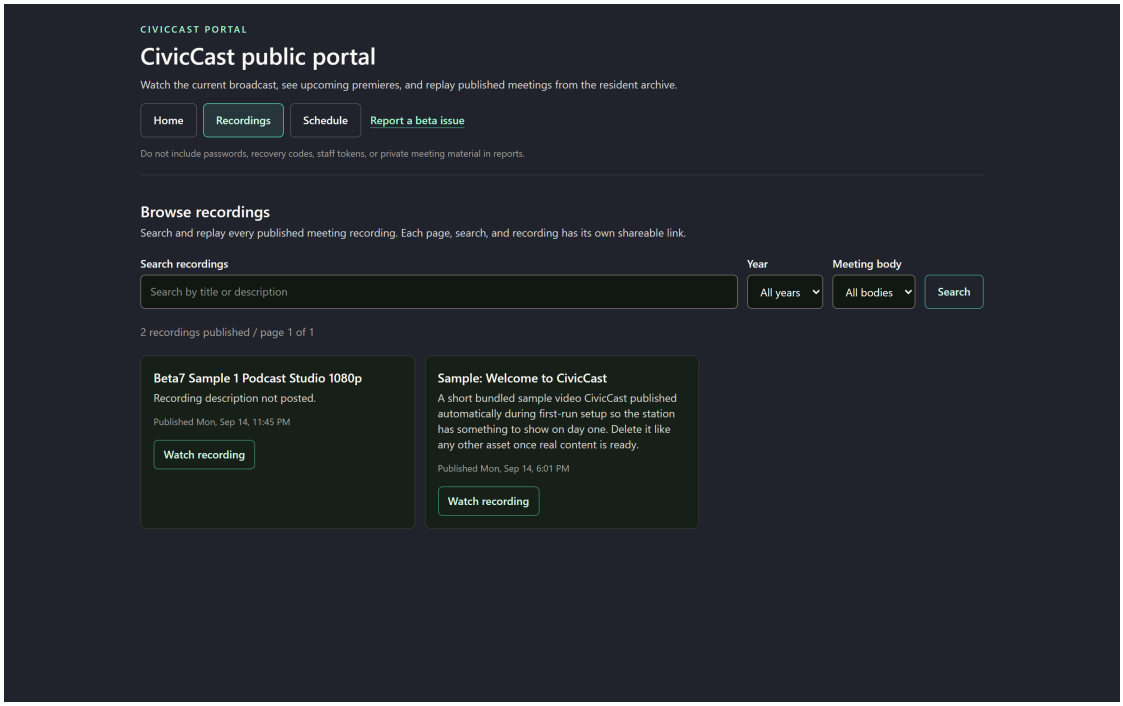
Each card shows the title, the description, "Published {date}" and **Watch recording**. A card with no description says "Recording description not posted." The line above the cards reads, for example, "2 recordings published / page 1 of 1" or "N recordings match this filter".

Meeting body and extra fields are set by staff on the video's detail page. A recording with no meeting body never matches a Meeting body filter. If the station has published nothing yet it says "No recordings have been published yet." and links to the channel schedule. If the list cannot load it says "Published recordings could not be loaded. Try again or contact the station." with **Retry**. When a search or filter matches nothing the page says "No recordings match this filter. Clear the search or facets to browse everything." There is no "Clear" button; the resident resets each control. When the search service is not ready, the page falls back to a plain list and shows "Showing recordings in a reduced-search mode — full search is temporarily unavailable, so some filters below may show fewer results than normal. Try again shortly for full search."

*Figure: the Recordings page.*

### Schedule

1. Click **Schedule**.




The Recordings page with the search box, the Year and Meeting body menus, and recording cards.

2. Click a channel's name at the top to switch channels.

The page reads "What airs over the next three days. Times are shown in your local timezone." It lists each day with time, title and length. The program on the air now has a green bar and the tag **On now**. Entries are not links. If nothing is listed the page says "Nothing is on the schedule for this channel yet. Check back soon." The Schedule page comes from the station's program log. **Coming up** on Home is a different list: committed premieres only.

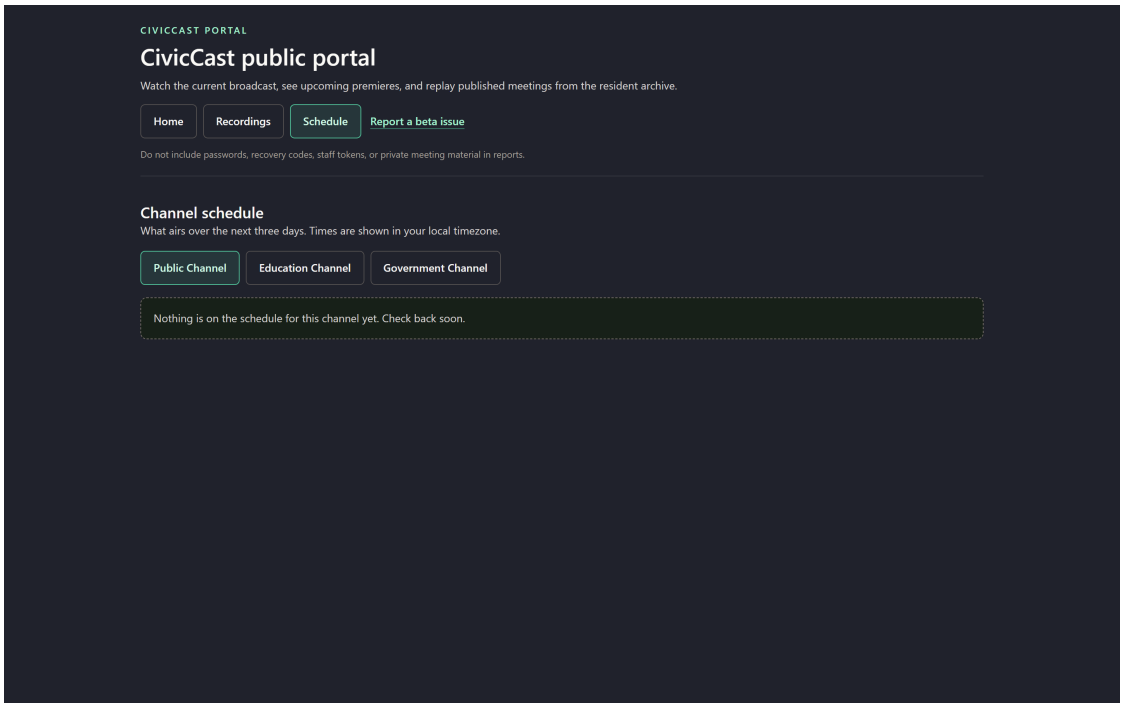
*Figure: the Schedule page.*

**Watch page: video, captions and agenda** A resident opens a recording by clicking **Watch recording**. The page shows **Back to all recordings**, the title, "Published {date} / {length}", the video player, the description, and **Copy share link**. Click **Copy share link**; the page says "Link copied." and puts the page address on the clipboard. The video is a normal browser player with play, volume and full screen controls.

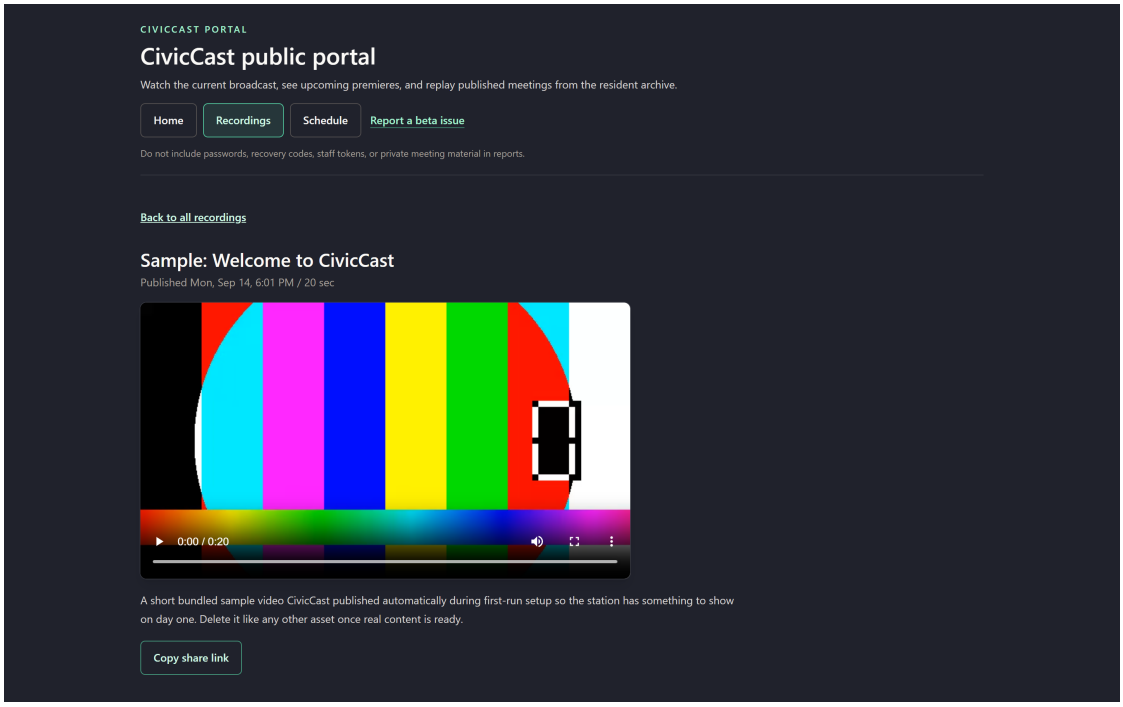
*Figure: a Watch page.*

**Captions.** If the video has caption tracks, a **Captions** bar appears under the player with buttons **Off** and one per track, such as English or Spanish. A track starts on only if the video marks it as the default. When a recording has no caption tracks there is no bar and no message. The portal cannot make captions; it only shows what staff approved (see After the meeting).

**Agenda chapters.** If staff published an agenda for the meeting, an **Agenda** card sits beside the video (below it on a phone). It lists the items by number, title and time. Click an item with a time and the video jumps there and plays. An item with no time shows a dash and cannot be clicked. If the agenda




The Schedule page with channel buttons. The channel shown has no entries yet.




A recording's Watch page with the video player and the Copy share link button.

has a document, an **Agenda document** link opens it in a new tab, and a PDF may show in the card. If there is no agenda, nothing is shown. The agenda is the only chapter feature: the player has no chapter marks on its timeline.

**Errors.** A recording that is not public shows "Recording not found" and "This recording does not exist or is no longer published. Browse the archive for the current recordings." Other failures show "This recording could not be loaded right now. Try again, then contact the station if the problem continues." with **Retry**. If the video cannot play, the player says one of: "Network error while loading the video. Check your connection and try again.", "The video could not be played. The stream may be unavailable." or, for an old browser, "Your browser does not support HLS playback. Please try a recent version of Chrome, Firefox, Safari, or Edge." (HLS is the streaming format the portal uses.) The player has no retry button; reload the page.

> **Note:** The share link uses the address the resident used to reach the portal. If your station is reached by an inside address, a copied link will not work for people outside.

> **Note:** A Beta 15 field run could not confirm that **Copy share link** changed the clipboard: the clipboard still held an earlier Library URL, and the action itself was not verified. This remains an unconfirmed Beta 15 observation; it does not establish a defect in the current package. See the Beta 17 deferred follow-up.

**What a resident sees when the paywall is on** Only if a setup administrator has turned the paywall on. The video is replaced by a gate. While it checks, a small label says "Checking access…". The gate shows "Subscription required", a reason line, a **Sign in by email** form with **Email me a sign-in link**, a "New here?" block with a plan list and **Subscribe**, and "Already signed in as {email}? Switch email".

In Beta 19, a resident can enter an email and request a sign-in link. A configured mail provider delivers it; the page reports an error if delivery fails. The plan list reads "Tier selection isn't configured yet on this station. Contact them for subscription details." With an email typed in, clicking **Subscribe** shows "This station hasn't finished setting up subscriptions yet. Please contact them." (With no email typed it says "Enter your email above before subscribing.") The agenda beside the video stays visible. See the Paywall status above.

**Subscribing to new recordings** At the bottom of Home is **Follow new recordings**. It offers an **Email address** box with **Subscribe**, a **Channel RSS feed** link, and a **Podcast RSS feed** link. RSS feeds are web addresses that a feed reader or podcast app checks for new items.

If a resident types an address and clicks **Subscribe**, a result box says "Subscription is waiting for confirmation." and "Open the confirmation link sent to this address." The small print says email uses a confirm step and a one-click unsubscribe link and carries no tracking pixels.

> **Known issue (beta.11):** Subscribing does not work end to end.
>
> - The signup is always for a channel named `government`, and both feed links point at that

name. A station with no channel by that name gets feeds for nothing.
- Unless IT has switched on a real mail connection, the confirmation email goes to a placeholder mailbox that never leaves the station computer.
- Even with a real mail connection, the confirmation link written into the email is only a path (`/subscribe/confirm?token=...`) with no web address in front of it. The portal confirms a subscription from `/?subscription=confirm&token=...` and we found nothing that serves the path in the email, so we could not confirm that a resident can finish confirming.
- In beta.11, a confirmed subscriber did not receive a publish notice. Current source offers the optional **Subscriber notifications** row with recorded delivery outcomes.
- In beta.11, the **Channel RSS feed** was valid but empty and the Podcast step on Publish was unavailable. Current source can add approved podcast episodes and playable enclosures to the feed.

Do not promise residents email alerts or working feeds in beta.11. The result box may also show a link labeled "Test one-click unsubscribe", which is a testing control.

**For IT staff:** Mail delivery is chosen with the provider setting `CIVICCAST_PROVIDER_MAIL` (default is the local placeholder). See Part II (integrations chapter).

The **Submit a program** form lower on Home lets a producer send a program to the station. It is a staff-facing intake feature and is not covered in this chapter.

#### Related

- After the meeting: assets, captions, summaries, approvals
- When something looks wrong

## Reports, analytics, program guide export, underwriting, and apps

This chapter covers the "business side" of a station: how many residents watched, what aired and when, the schedule file that outside TV-guide services read, sponsor messages, the resident apps, and the optional paid-access feature. A PEG station employee, a records clerk or a station manager normally uses these screens. You do not need any technical background, but a few steps need an IT person, and this chapter says so each time.

### Before you start

You need to be signed in (see Chapter 2). Five of the screens in this chapter sit in the left sidebar under **Publish**: **Analytics**, **Reports**, **EPG Export**, **Underwriting** and **App Admin**. The sixth, **Paywall**, sits under **Setup**.

CivicCast gives each person one or more *roles*. A role is a named set of permissions, such as "Support admin" or "Publish operator". The table shows who sees each screen in the sidebar and what each role can do on it.

| Screen | Shown in the sidebar to | What the role can do |
|---|---|---|
| Analytics | Everyone signed in | Only Support admin and Publish operator can load the numbers |
| Reports | Support admin | Read and download |
| EPG Export | Setup admin, Publish operator | Create, run and delete exports |
| Underwriting | Setup admin, Publish operator, Support admin | Spots, Flights, Placements: Publish operator or Setup admin. Affidavits: Support admin |
| App Admin | Setup admin, Publish operator | Both can view and track; only Setup admin can start a build |
| Paywall | Setup admin | Everything |

> **Note:** The first administrator account carries all five roles. A setup admin can create staff accounts with narrower roles; your assigned roles determine which screens and actions are available.

> **Warning:** Several screens in this chapter ask you to type dates. CivicCast treats those dates as **UTC**, the world reference clock. In the United States UTC is several hours ahead of local time. Mountain daylight time is 6 hours behind UTC. A meeting that starts at 7 p.m. Mountain daylight time is already 1 a.m. UTC *the next day*. When a report looks one day off, this is almost always why.

### Read your audience numbers (Analytics)

Analytics shows how many residents watched recorded meetings (called *VOD*, "video on demand") and live streams.

1. In the left sidebar, open **Publish**, then click **Analytics**.
2. Look below the toolbar. If a box says **Audience telemetry is off**, nothing is being counted. Go to "If it did not work" at the end of this chapter.
3. Click one of the range buttons: **7d**, **30d**, **Quarter** or **Year**. The page opens on **30d**.
4. Click **VOD**, **LIVE** or **ALL** to choose which panels to show. The page opens on **ALL**.
5. Choose a number from the **Metric** drop-down: **Viewer Count**, **Time Viewed** or **Peak Concurrent**.
6. Click **bar** or **line** to change the chart style.

You should see two panels, **VOD** and **Live**. Each starts with a line in the form `12 views · 3.4h watched`, followed by a chart called "Top by" your chosen metric and a chart called "over time". A `· peak 5` ending appears only on the **Live** panel, and only when the live numbers include a viewers-at-once count (see the Known issue below).

> **Note: Quarter** means the last 90 days and **Year** means the last 365 days, counted back from now. They are not calendar quarters or calendar years.

> **Note:** The two panels and their charts read totals that CivicCast recalculates in the background about every 5 minutes by default, so they can trail the tiles under them by a few minutes.

Under the panels, the page shows:

- Four tiles: **Asset views**, **View hours**, **Live peak** and **Podcast downloads**.
- A box called **Privacy boundary**, with the time the report was made.
- Tables named **Asset Time Series** and **Live Concurrent Viewers**.
- Six small tables: **Geography**, **Device**, **Platform**, **Caption Usage**, **Audio Usage** and **Subscription Growth**.
- A link, **Show rollup data table**, that opens a table of the raw totals.

**What the numbers mean**

- A **view** is one time someone starts playing a recording. The same person watching twice counts twice. The PDF report says this itself: "Viewer Count is a play count, not unique humans."
- The numbers cover **streaming only**: the resident website and apps. They do not include people watching on cable.
- Live "concurrent" numbers (how many people watch at the same moment) are estimates.
- **Podcast downloads** counts analytics events named `podcast_download`. Publishing an episode now creates playable audio and an RSS enclosure; the audio route does not itself emit that analytics event, so this tile can remain at 0 while downloads occur.

**What is collected, and what is not** The resident website sends a small anonymous message ("a beacon") for five things: playback started, playback heartbeat (a signal sent once a minute while a video plays, with the position in the video), playback finished, playback error, and which section of the schedule page someone opened.

CivicCast keeps: the event name, the time, which app sent it, the channel, the recording, and a short list of coarse extras.

CivicCast does **not** keep names, email addresses, IP addresses, session ids or any viewer identifier. The code states this as its privacy boundary, and the screen's **Privacy boundary** box repeats the rule as a machine phrase, `aggregate-only-no-session-ip-or-viewer-identity`. It means "counts only".

By default CivicCast discards events older than 366 days.

> **Note:** In beta.11 the resident website does not show residents a notice that views are counted. If your station wants one, you will need to post your own.

> **Known issue (beta.11):** Most of the extra tables stay empty. The resident website sends only the five messages above. It does not send the viewer's device, platform, country, caption language, audio track or how many people are watching live. The **Geography**, **Device**, **Platform**, **Caption Usage**, **Audio Usage** and **Subscription Growth** tables, the **Live peak** tile and **Live Concurrent Viewers** table therefore stay empty unless another app sends that information. This is expected, not a fault with your station. Watch time is also approximate: in both the file-based and the database-backed store we read, it is added up only from the position reported when a video finishes, so people who stop early add nothing.

> **Note:** When audience telemetry is off, the empty-state box says, "Ask your station administrator to enable audience telemetry in the service settings to collect Viewer Count and Time Viewed. The Reports tab (as-run / proof-of-performance) still works — it reads the program log, not the beacon." There is no Setup-screen switch; the station administrator changes the service settings. Until then, the Analytics report and PDF have no viewer-count or time-viewed data.

> **For IT staff:** The two settings are `CIVICCAST_PUBLIC_ANALYTICS_KEY` and `CIVICCAST_PUBLIC_ANALYTICS_ALLOWED_ORIGINS`; see Chapter 11 and Chapter 13. The resident website sends no key, so the website's own messages are accepted only when its address is listed in the allowed-origins setting (a comma-separated list of exact addresses such as `https://tv.example.gov`, matched against the browser's origin header). Setting only the key removes the "telemetry is off" box, but the website's messages are then refused.

**Download a spreadsheet**

1. Choose the range you want with **7d**, **30d**, **Quarter** or **Year**.
2. Choose **VOD** or **LIVE** (see the Known issue below about **ALL**).
3. Click **Export CSV**.

Your browser saves a file named like `analytics-rollups-vod-30d.csv`. CSV is a plain table you can open in Excel or Google Sheets.

> **Known issue (beta.11):** With **ALL** selected, **Export CSV** exports the **VOD** table only, and says nothing about it. The file holds the totals table, not everything the page shows. If the download fails, nothing appears on screen. Choose **LIVE** and export again to get the live table.

**Make the board report (PDF)**

1. Click **Generate Board PDF**. A small panel called **Include sections** opens. Four boxes are ticked: **Totals**, **Top content**, **Year-over-year** and **Live-event peaks**.
2. Untick any section you do not want.
3. Click **Download PDF**. The button reads **Generating…** while it works.

Your browser saves `audience-report-` followed by today's date (UTC). The PDF starts with the title, the reporting period and the time it was made, then three notes: streaming only, "a play count, not unique humans", and live numbers are estimated. Your ticked sections follow. **Top content** lists the ten most-played recordings by *recording id*, not by title.

If it fails, the page says "Could not generate the board PDF. Try again."

> **Known issue (beta.11):** The PDF always covers "now minus the range you chose" and ignores the **VOD**/**LIVE**/**ALL** buttons. Its title always reads "CivicCast station", even if your station has a name. If you need your station's name on the report, add it by hand before you send it on.

> **Known issue (beta.11):** The **Analytics** menu entry is visible to every role, but only Support admin and Publish operator can load it. Anyone else sees the frame of the page and the words "Report unavailable.", which does not say why. If you see this message and you expect access, ask your station admin.

#### Find out what aired, and when (Reports)

Reports lists what actually went out on your channels. CivicCast keeps a log of each change of source on air, called the *as-run log*. Cities and cable companies sometimes ask for this as proof that programs aired (a "franchise report": a report your cable franchise agreement may require).

Reports reads the on-air log. It does not count viewers; use Analytics for that.

1. In the sidebar open **Publish**, then **Reports**. Only the Support admin role sees this entry.
2. Click a tab: **Shows**, **As-Run** or **Hours by Category**.
3. In **From**, pick the first day you want.
4. In **Through**, pick the day *after* the last day you want. The range includes **From** and excludes **Through**. To see one day, set **From** to that day and **Through** to the next day.
5. Leave **Channel (optional)** on **All channels**, or pick one channel.
6. On the **As-Run** and **Hours by Category** tabs, check **Field key (required)**. It opens as `category`.

The page opens on today's date through tomorrow's, UTC midnight to UTC midnight. If the dates are wrong, you see "Pick a From date that is strictly before the Through date." and nothing loads.

> **Warning:** Dates here are UTC. A station on Mountain time should widen the range by a day when looking for an evening meeting.

**What each tab shows**

| Tab | Columns | Use it to |
|---|---|---|
| **Shows** | Asset, Plays, Total airtime, First aired (UTC), Last aired (UTC) | See each recording once, with how many times it aired |
| **As-Run** | Channel, Planned start (UTC), Actual on-air start (UTC), Start variance, End (UTC), Duration, Source, Asset, Category, Verified | Compare the intended schedule with each actual change on air |
| **Hours by Category** | Category, Entries, Total airtime, Hours | See airtime per category, such as "Council" |

The **Asset** column shows the recording's id, not its title. The **Shows** tab leaves out anything without a library recording (filler, slate and live). **Actual on-air start** comes from the engine's source-transition proof; **Start variance** shows how early or late it was against the planned start. VOD playback preroll does not change that timestamp. CivicCast does not persist a separate preroll timestamp.

The **Source** column uses these words:

- `program`: a recording from your library.
- `filler`: a community bulletin board graphics page.
- `live`: a live source.
- `slate`: the safety card the station shows in place of video, for example when a feed fails.
- `spot`: a sponsor message. In beta.11 this word never appears. See Underwriting below.

**Verified** reads "yes" for every row the playout engine wrote, because a row exists only when the engine confirmed that it went on air.

**Download a report file** On **Shows** and **As-Run**, click **Download CSV** or **Download XML**. Both use the dates and channel you chose. Your browser saves `civiccast-shows-` or `civiccast-as-run-` followed by the start date.

The **Hours by Category** tab has no download buttons. Read it on screen.

> **Known issue (beta.11):** The **Hours by Category** tab needs a *custom field* (an extra label you add to recordings, such as "category"). An unknown name shows: `No custom field named "{key}" is defined for this station. Define it in Setup > Custom Fields, then re-run the report.` Only the Setup admin role can open **Custom Fields**, while Reports is for the Support admin role. If you are not both, ask your Setup admin.

> **Note:** A copy of the as-run list is also available to the public at `/api/public/reports/as-run` with no sign-in. It leaves out Category and Verified. Reports does not tell you this, so do not treat the as-run list as private.

> **For IT staff:** The public as-run endpoint is described in Chapter 13.

#### Send your schedule to a TV-guide service (EPG Export)

An *EPG* (electronic program guide) is the schedule file that cable boxes, TV-guide services and apps read to show "what is on". EPG Export turns your published schedule for the next several days into such a file. Each *export* is a saved setup naming a channel, a file type, how many days ahead, and optionally a web address to send the file to.

> **Note: Program Guide**, under Run Meeting, is a different thing. It holds your recurring slots and builds the on-air schedule (see Chapter 3). **EPG Export** makes a file for outside services from the schedule that already exists.

### Before you start

- The items you want in the file must be **published** on the Schedule screen. The file contains only published items on that channel that start between now and the number of days ahead.
- You need the channel's id. The **Reports** screen has a channel drop-down that lists them as `slug (channel_id)`, or ask your IT person.
- Ask the guide service for a sample file and its address first. In testing we could not confirm that any particular service, such as TitanTV, accepts what CivicCast makes. The "X-List" format here is a generic eight-column table.

### Create an export and download the file

1. Open **Publish**, then **EPG Export**.
2. In **Config ID**, type a name using lowercase letters, numbers, `-` and _ only, such as `tv-guide-channel-1`. It cannot be changed later.
3. In **Channel ID**, type the channel's id.
4. In **Format**, choose **X-List (TV Guide / TitanTV)**, **XMLTV** (a common TV-guide file format) or **CSV**. Ask the guide service which one it reads.
5. In **Horizon (days)**, type how many days ahead to include. The box starts at 14. It must be more than 0.
6. Leave **Aggregator endpoint (optional)** empty. An empty address means "give me the file to download".
7. Click **Create config**.
8. In the **Configured exports** list, click **Generate now** on your export.
9. Click **Download document**.

You should see a green panel such as `12 slots · 3.0 KB (xlist)` (a single slot reads `1 slot`) with a **Download document** button. A *slot* is one scheduled program. Your browser saves `epg-export.xml`, `epg-export.csv` or `epg-export.txt` (X-List files end in `.txt`).

If the panel says `0 slots`, the channel has no published items in the window, or the channel id is wrong.

Each slot carries a start and end date and time in **UTC**, and the recording's title.

> **Known issue (beta.11):** The description, category and rating columns are always empty. The screen's wording suggests a full guide. Do not promise a service genre or rating information from this export in beta.11.

> **Note: Field map** accepts one `key=value` pair per line; blank lines and lines beginning with # are ignored. It renames headings in CSV and X-List only; XMLTV is unchanged. The current examples use valid columns: `title=Program Title` and `description=Meeting summary`. Available keys are `start_date`, `start_time`, `end_date`, `end_time`, `title`, `description`, `category` and `rating`.

**Send the file to an address instead** If you give an address in **Aggregator endpoint (optional)**, **Generate now** sends the file there.

1. Edit your export, or create a new one.
2. Type the service's address in **Aggregator endpoint (optional)**. It must start with `https://`.
3. Click **Save changes** or **Create config**.
4. Click **Generate now**.

You should see a panel that says `Pushed to {address} at {time}.`

> **Warning: Generate now** sends the whole guide at once, with no confirmation, and it cannot be taken back. CivicCast cannot log in to the receiving service, because no password or key can be stored. The file is sent as an open request. Nothing sends the file automatically on a schedule. Each press of **Generate now** sends it again.

If the send fails, the panel says "Push failed" with a reason, and gives you no file to download. To get a file, clear the address and run **Generate now** again.

The server accepts only `https` addresses and refuses `localhost` and private network addresses typed as numbers. It waits 5 seconds and does not follow redirects. The code does not look up where a web name points, so a name that points to a private address is not refused.

**Change or remove an export**

1. To change days ahead, address or field map, click **Edit**, change the box, and click **Save changes**. Clearing the address switches the export to download-only.

2. To remove an export, click **Delete**, then **Confirm delete**. Data the guide service already received stays there.

#### Record sponsor messages (Underwriting)

Underwriting means paid "sponsor acknowledgment" messages, such as "Support for this program comes from Acme Co-op". The screen uses these words:

- A **spot** is one short sponsor video, with the sponsor's name.
- A **flight** is a sponsor's run: which spot, which channels, and between which dates.
- A **placement** is a specific schedule slot a spot was assigned to.
- An **affidavit** is a sponsor-ready list of every time their spot aired, used for billing.

**Scheduling boundary:** Creating a flight does not insert video into the schedule automatically. IT staff must run the trafficking compiler against existing schedule slots through the station's programming interface. The scheduled asset must match the creative chosen for that placement. Commit that schedule through the ordinary approval controls. Once that exact slot, channel and asset actually air, CivicCast records a spot airing for the affidavit. Playing the same sponsor asset as an ordinary program does not count as a trafficked spot. A planned placement alone is not proof of airing.

**What does work**

| You can | Result |
|---|---|
| Create, edit and delete spots | Saved |
| Create, edit and delete flights | Saved |
| Tick the compliance attestation | Saved as a record that a person reviewed the spot |
| Read **Placements** | Empty unless IT staff trigger the compile request |
| Read **Affidavits** | Actual airings matching compiled placements; empty until a matching spot airs |
| Click **Download CSV**, **Download XML**, **Download PDF** | Downloads the selected affidavit using your staff session |

**Add a spot**

1. Open **Publish**, then **Underwriting**, then the **Spots** tab (Publish operator or Setup admin).
2. In **Spot ID**, type a name using lowercase letters, numbers, `-` and `_`, such as `acme-15s-2026q3`.
3. In **Underwriter**, type the sponsor's business name exactly as you will type it later when you look for it. The affidavit matches the name exactly, including capitals.
4. In **Asset ID (the :15 / :30 acknowledgment video)**, type the id of the sponsor video (the recording's id, from the Assets screen). The label's ":15 / :30" means a 15-second or 30-second message. Use lowercase letters, numbers, `-` and `_` here too.
5. Read the reminder box about 47 CFR 73.503. It says acknowledgments may name the sponsor, logo, location and a value-neutral description, and may not contain calls to action, prices,

comparisons or promotional language.
6. Tick **I attest this spot meets 47 CFR 73.503.** only after you have watched the spot.
7. Optionally write what you checked in **Review notes (optional)**.
8. Click **Create spot**.

The spot appears in the **Spots** list with "FCC 73.503 attested." or "NOT attested — operator must attest before traffic."

> **Note:** CivicCast does not check the content of a spot. Your attestation is the only gate. The screen cites a federal rule written for broadcast stations. Ask your station manager or counsel how it applies to your channel.

> **Note:** CivicCast does not check that the **Asset ID** is real. A wrong id saves without a warning.

**Add a flight**

1. Open the **Flights** tab.
2. In **Flight ID**, type a name such as `acme-2026q3-pubgov`.
3. In **Spot**, pick the spot from the drop-down.
4. Set **Start date** and **End date**. The window includes both days.
5. In **Channels**, type at least one channel id, separated by commas, such as `pub-1, gov-1`. The box has no "required" mark, but the server refuses a flight with no channel.
6. Leave **Frequency cap (per day, optional)** empty, or type a whole number from 1 to 1440.
7. In **Daypart (optional)**, leave **Any time** or choose a saved daypart by name, channel and hours. Use **Manage dayparts** to create or edit them in Auto Schedule. If an older flight refers to a deleted daypart, choose another or **Any time** before saving.
8. Click **Create flight**.

To delete a spot or flight, click **Delete**, then **Confirm delete**. Deleting a spot also deletes every flight and placement that used it. This cannot be undone.

**The Placements and Affidavits tabs** On **Placements**, set **From** and **Through** (UTC; **From** is included and **Through** is not), and optionally a channel and flight. On **Affidavits**, type the sponsor's name and a date range. The affidavit includes *both* the first and last day, in UTC.

An empty affidavit shows "No airings recorded for {name} between {from} and {to}." Check the exact sponsor name, UTC period, compiled placement, and whether its matching asset actually aired on the selected channel.

Use **Download CSV**, **Download XML** or **Download PDF** to save the current affidavit through your authenticated staff session.

> **Known issue (beta.11):** If your IT person switches on the setting `CIVICCAST_REQUIRE_FCC_ACK=1`, saving a spot without the tick fails with "Station policy requires fcc_compliant_ack=true". Without that setting an un-attested spot saves, despite the "NOT attested" wording.

> **For IT staff:** See Chapter 15 for the programming interface.

#### Build the resident apps (App Admin)

A resident app lets people watch your station on a phone or a TV box. CivicCast provides native viewer source apps for Roku, Apple, Android/Fire TV, Tizen and webOS, plus a web PWA. They browse live channels and replays, show station artwork, search the catalog, handle deep links and use platform playback controls. **App Admin** packages generic station-branded starter artifacts on the station computer, keeps a history with download buttons, and tracks app-store submissions; it does not build or submit the separate native source apps.

> **Warning:** App Admin does not put any app in an app store. Someone technical still has to sign and submit platform packages. The App Admin package is a generic starter; the native source apps are built with their platform toolchains. Apps read station branding from the station API when they run.

**Make a build** You must have the Setup admin role.

1. Open **Publish**, then **App Admin**.
2. Look at **Build profile**. It shows **App name**, **Tier**, **Store-ready** and an icon address. You cannot change these here. Change them on the **Channels** screen.
3. In **Platform target**, choose a platform. The names are lowercase: `web pwa`, `roku`, `tvos`, `fire tv`, `android tv`, `android mobile` and `ios ipados`.
4. In **Tier**, choose **unbranded** or **branded**.
5. Click **Queue build**.
6. In the box that asks "Queue a {platform} build ({tier} tier)?", click **Queue build**.
7. Wait. The button reads **Building...**.
8. When the build finishes, find it in **Build history** and click **Download**.

Your browser saves `{platform}-{record id}.zip`. Each row shows the time, the first 12 characters of a *SHA fingerprint* (a code that changes if the file is altered, useful for checking that a file you pass on arrived unchanged) and who made the build.

> **Known issue (beta.11):** **Tier** only labels the build record. The build uses the same steps for **unbranded** and **branded**, and does not read the station's settings. Do not expect the tier to change the file.

> **Known issue (beta.11):** The build runs inside a single web request. The page shows **Building...** with no progress bar and no time estimate, and it must stay open. In testing we could not confirm how long builds take.

> **Known issue (beta.11):** The **Store submissions** section stays on "No submissions tracked yet." The screen offers no way to add a platform, so the notebook has no rows. Only IT staff can create one through the programming interface.

> **Note: Store-ready: yes** is a flag someone set on the Channels screen. It does not test anything.

> **Note:** If you have the Publish operator role you see "Queueing a build requires the setup admin role." and can still view and download builds.

#### Turn on paid access (Paywall)

The **Paywall** screen, in the **Setup** section, lets a station hold some recordings behind a paid subscription (through a payment company called Stripe) or give a free "comp" pass to a named email address. It is off by default. When it is off, every recording is public. The Setup admin role can open it. How residents experience a paywall is described in Chapter 6.

A *tier* is a price level, for example "Basic monthly", linked to a price you made in Stripe. CivicCast never creates prices and never stores card numbers.

> **Beta 19 status:** The magic-link button sends through a configured mail provider and reports delivery failure. Paid access remains incomplete: the portal's tier-list and checkout routes are unavailable, and the gate covers the player rather than protecting the video file itself. The **Recently issued grants** list shows grants from the current browser session only. The screen preserves a saved signing secret when an ordinary save leaves the secret box blank; clearing it requires the explicit clear control.

> **For IT staff:** See Chapter 15 before enabling this on a live station.

#### If it did not work

| What you see | Where | What it means and what to do |
|---|---|---|
| `Audience telemetry is off` | Analytics | Counting is not switched on. Ask your IT person to turn it on (see the For IT staff note above) |
| `Report unavailable.` | Analytics | You may lack the role (Support admin or Publish operator), or the station is not ready. Ask your station admin |

| What you see | Where | What it means and what to do |
|---|---|---|
| `Reports require the support admin role. Ask your station admin for access.` | Reports | You need the Support admin role |
| `Pick a From date that is strictly before the Through date.` | Reports | Set **Through** to a later day than **From** |
| `No content has aired yet on this station. Reports will populate after your first scheduled meeting plays out.` | Reports | No air log rows in the range. Check the channel, widen the dates (UTC), and confirm something aired |
| `Durable storage is not ready yet.` | Reports, EPG, Underwriting, Alerts | The station's database is not ready. Tell your IT person |
| `Push failed: {error}...` | EPG Export | The guide service did not take the file. Check the address, then try again. Nothing changed on your station |
| `Could not run the export.` | EPG Export | The export could not run. Check the channel id and try again |
| `Missing Authorization header. Use Bearer <staff-token>.` | Staff programming-interface request | Sign in with the required staff role; use the screen's download controls for affidavit files |
| `Station policy requires fcc_compliant_ack=true ...` | Underwriting, saving a spot | Tick the attestation box after reviewing the spot |
| `Underwriting spot '...' already exists. Use PATCH to update.` | Underwriting | That Spot ID is taken. Use a different id, or click **Edit** on the existing spot |
| `App build tooling is not configured in this runtime...` | App Admin | This computer does not have the app build tools. Meeting recording and scheduled recording are not affected. Ask IT |
| `Could not download the artifact.` or `Build artifact file is no longer present on disk.` | App Admin | The ZIP file is missing from the station computer. Build again |

**Related**

- Chapter 2: Signing in
- Chapter 3: Before the meeting
- Chapter 6: Publishing, and what residents see
- Chapter 8: When something looks wrong
- Chapter 15: Cable headend, streaming, federation and the API

## When something looks wrong

This chapter helps you when a screen shows a red or yellow warning, an alert appears, or you are not sure the station is working. It covers the **Readiness** screen, **Alerts**, **Emergency Alerts** and **Federation**, then gives you a list of what to do and what not to do, a way to describe a problem to your IT person, and where to report a beta problem. Every operator role can read the runtime safe-to-air health indicator. The Alerts event list, its rule and destination editors, and Emergency Alerts have narrower permissions, described below. Fixing most problems is the IT person's job.

### Before you start

The four screens live in the left sidebar under **System Health**: **Readiness**, **Alerts**, **Emergency Alerts** and **Federation**. Most of them open for any role. Individual buttons are limited by role, and a button you cannot use shows a grey note that names the role needed.

> **Note:** The first administrator account carries all five roles. Staff members use their own accounts with assigned roles, so they may see fewer controls. A Setup admin manages accounts and roles in **Staff accounts**.

> **Known issue (beta.11):** One place has three names. The sidebar says **Readiness**, the page's heading says **Safe to broadcast**, and other text says **System Health**. They are the same screen.

### Check whether the station is ready (Readiness)

Readiness answers one question: "Can we broadcast right now?"

1. In the sidebar, open **System Health**, then click **Readiness**.
2. Wait while the page says "Checking station readiness…".
3. Read the banner at the top, **On air right now**. It shows each channel, a coloured label and a message. The banner refreshes by itself every 5 seconds.
4. Read the card below it. Its heading is the station's overall verdict, followed by a message and "Last checked" with the date and time.
5. Scroll down to **Required before broadcast** and **Optional and advanced**. Each row is one check, with a coloured label, a short message and a "Next step".

### What the colours and words mean

| Colour | Words on screen | What it means |
|---|---|---|
| Green | **Ready** | The required checks passed, and no optional check needs a look |
| Yellow | **Check before meeting** | Something needs attention or proof before the meeting |
| Red | **Do not broadcast yet** | A required check failed for tonight's broadcast |

A single check row can also say **Not set up yet** (an optional feature has no setup or proof) or **Needs IT help** (the next step needs administrator work on the computer).

A channel tile uses these words:

- **On air** or **Ready** (green).
- **Working** (yellow).
- **Needs attention** (red).

After the word you may see "on safety slate" (the channel is showing its fallback card) or "live captions off".

When live captions are enabled, the channel tile also reports the caption worker state, its last heartbeat, and which recognition provider is selected. **Digital silence in latest processed audio** means the latest processed audio chunk contained zero-valued samples; it does not claim that speech was recognized. **Worker stalled** means the expected worker has stopped reporting fresh progress. A missing or stale worker heartbeat remains unknown during the first 180 seconds after a channel goes on air; after that, a heartbeat older than 90 seconds is stalled. If audio remains pending for more than 120 seconds without a chunk completing, the worker is stalled; starting another batch retry does not reset this clock. A recent heartbeat with no pending audio is not a failure. Channels that are stopped or have live captions switched off are not caption failures.

The public `/health` probe continues to return HTTP 200 while the process answers. Its `status` is **degraded** when the schema is not current or live-caption readiness is failed, stalled, or not yet known. Its `live_captions` value is only a station-wide state; it never includes channel names. Signed-in staff see the per-channel worker details in **On air right now**.

Caption capacity distinguishes **Within capacity**, **Temporarily behind** and **Sustained caption backlog**. Temporary delay means captions can continue catching up within the runtime's recovery allowance. Sustained overload means the queue depth or age has remained beyond that allowance. Read these alongside the queue measurements and current provider rather than treating every late caption as a failure.

> **Known issue (beta.11):** A yellow station can show two different verdicts at once. The small pill at the top says **Check before meeting**, while the card below can say **Ready with optional items**. Both mean the required checks passed and an optional item needs a look. The second wording is not one of the five standard phrases.

> **Known issue (beta.11):** The page does not refresh by itself, except for the **On air right now** banner. There is no **Refresh** button. The checklist updates only after you use one of its own buttons or re-open the page after about 30 seconds.

**What each check means** These eight checks are required. If any one is red, the station says **Do not broadcast yet**.

| Check | In plain words | What the "Next step" says |
|---|---|---|
| **First admin** | The station has an administrator account | Open Setup and finish first-admin setup |
| **Recovery kit** | The one-time sheet that lets you get back in if the password is lost was made | Confirm someone printed or saved it |
| **Durable records storage** | The station's database is working and keeps its records | Open Setup and choose **Prepare storage** |
| **Backup** | A backup is set up | The message tells you what is missing |
| **Camera or meeting source** | A camera, encoder, Zoom or NDI source is added and passed its pre-flight check | Open Run Meeting and add or check the source |
| **Local recording** | A place to save recordings is chosen and passed a write, read and delete test | Choose where recordings are saved |
| **Resident portal** | The resident website address is set and the preview is confirmed | Open the preview and confirm residents can see the page |
| **Station policy** | The default station rules are active | Finish Setup |

The optional checks are:

| Check | In plain words |
|---|---|
| **Contributor upload storage** | How full the folder for resident uploads is. Yellow at 80 percent, red at 100 percent, when new uploads are refused |
| **Caption inference device** | Whether captions run on the graphics card or the main processor. It is always shown as informational, and never blocks |
| **Cable headend verification** | Whether the cable feed was tested. If no channel delivers to a cable headend, the row still appears and says so |
| **24/7 channel automation** | Whether channels set to run around the clock are running. If none are set up, the row still appears and says so |
| **YouTube**, **Subscriber notices**, **Federation**, **Internet Archive**, **Archive storage** | Optional publishing and archive features and whether each is set up |
| **Internal service certificates** | An advanced security item, marked "advanced". It is for IT staff |

**Run the broadcast readiness check** The **Check broadcast readiness** button runs a *private rehearsal*, a short test of the whole broadcast path with no public audience. You need the Meeting operator role.

1. Click **Check broadcast readiness** on the top card.
2. Wait for the **Broadcast readiness check result** card to appear.
3. Read the **Rehearsal result** line and the **Broadcast gate** line.

The rehearsal result is one of:

- **Passed**: a private session ran, passed its pre-flight check and finished a recording.
- **Failed**: the private session started but stopped before a recording was finished.
- **Not run**: something stopped the private session from starting.

The broadcast gate reads "All required items ready", or "N required item(s) need attention", or "N required item(s) not ready".

> **Warning:** The rehearsal makes a real private test session called "Private first-broadcast rehearsal" on the channel named `government`, copies a sample video, and saves a test recording. The button gives no warning and no "running" message. Run it before a meeting, not while a meeting is on the air. In testing we could not confirm whether the test recording is visible on the Assets screen.

> **Note:** Items listed under the gate link to matching rows lower on the page. If selecting one does not move the page to its row, scroll down and look for the same label.

> **Known issue (beta.11):** Some buttons on this page are grey with no reason shown, for example **Open maintenance window** and **Run failed-update rehearsal**. They need earlier steps to pass first. Only the role limits are explained on screen.

**The other panels on this page** Below the checklist, the page also holds tools for IT staff or trained staff. Do not press these unless you were asked to.

| Panel | What it is for |
|---|---|
| **Outgoing channel feed** | One card per channel with **Start**, **Stop**, **Restart feed** and **Finish current item, then stop**. The Meeting operator role is needed |
| **GStreamer engine repair** | Re-checks the video engine (called GStreamer) and, if it is broken, starts a signed repair of it. The page says this is never a reinstall. Setup admin or Support admin |
| **Backup and restore readiness** | **Check backup storage** and **Run real database restore drill** |

| Panel | What it is for |
|---|---|
| **Update and rollback** | Checks and practice runs for updating CivicCast and going back if an update fails. It does not install an update itself |
| **Support bundle** | Makes a redacted troubleshooting file for support (Support admin) |
| **Latest self-check** | The result of the station's automatic test. It runs daily and weekly; the first daily check runs overnight |
| **Machine health** | Processor use, memory, free disk space, whether the database is reachable and whether the service is running |

The feed buttons ask you to confirm before they act. They only *queue* a command for the channel's feed. Nothing tells you to wait.

> **Known issue (beta.11):** After **Restart feed**, the state label may stay on the old value until you re-open the page. The page does not poll these cards. Wait a minute, then reload Readiness.

> **Warning: Stop** takes a channel off the air. **Finish current item, then stop** does the same after the current program ends. Only press these on purpose, and never on a channel you do not intend to stop.

#### Watch for alerts (Alerts)

An *alert* is a warning the station raises by itself, such as "Disk space low" or "Channel off air". The **Alerts** screen lists them.

1. In the sidebar, open **System Health**, then click **Alerts**. You can also click **Review alerts** or **Open alerts** on the Readiness page.
2. Click **Active** to see current problems, or **Resolved** for past ones. The Active list refreshes every 10 seconds.
3. Read each row: the title (what went wrong), a severity (CRITICAL, WARNING or INFO), a summary, what it is about, and when it was first and last seen. "seen 3×" means it happened again.
4. Click **Acknowledge** on a row to say you have seen it. The row then shows "Acknowledged by" your name and the date.

An empty list says: "No active alerts. Everything the watch box monitors is healthy." The screen calls the station's own monitor "the watch box".

> **Note: Acknowledge** means "I have seen this". It does not fix the problem and does not close the alert. An alert closes itself when the problem goes away, and then moves to **Resolved**.

The alert titles you may see are:

- Channel off air
- Encoder stopped
- Server crashed
- Data format out of date
- Relay blocked
- Cable compliance check failed
- Missing media file
- Save to database failed
- Live takeover stuck over 2 hours
- AI engine is down
- Disk space low
- Computer clock out of sync
- Database unreachable
- CivicCast service is down
- Automatic self-check did not pass
- Live caption worker failed or stalled

Some alerts show a plain code with the dashes turned into spaces, for example "eas source unavailable", "asrun outbox degraded" or "caption tier degraded". "eas source unavailable" means an emergency-alert feed could not be reached; its rule card is titled **EAS source unavailable**. "asrun outbox degraded" concerns the on-air log that Reports reads.

**Rules and destinations** Under the list are two sections for administrators.

- **Alert rules** has cards for alert kinds with a rule, including **Live caption worker failed or stalled** and **EAS source unavailable**. On a card you can switch **Enabled** on or off, set **Severity** to Critical, Warning or Info, set **Re-alert after (minutes)** (how long CivicCast waits before warning you again while the problem continues), tick **Notify on resolve**, and choose destinations. Tick each email, text-message or webhook destination that should receive this alert, then click **Save**.
- **Where alerts go** lists *destinations*: an email address, a text-message number or a webhook (a web address that receives a message). Click **Add destination**, choose a **Type**, give it a **Name**, type the address in **Where to send (email, phone, or webhook URL)**, and click **Create destination**.

> **Beta.12 routing update:** A rule card's **Destinations** list shows the saved assignments. If an assigned destination no longer exists, it stays checked as **Unavailable destination** with its id; leave it checked to preserve the saved id, or uncheck it to remove it. Saving another rule setting without changing the destination checkboxes keeps its assignments.

The destination form has **Quiet hours start (UTC, HH:MM)** and **Quiet hours end (UTC, HH:MM)**. These are in UTC, not local time. A quiet-hours window holds back WARNING and INFO alerts. CRITICAL alerts ignore quiet hours. Sending still requires an enabled, assigned destination and a working de-

livery service.

> **Warning:** Do not click **Delete** on a destination unless you mean it. The first click turns the button into **Confirm delete?**, and a second click deletes right away. There is no cancel and no timeout.

**What happens in beta.12 when no destination is wired**

> **Known issue (beta.12):** A rule with no enabled destination assigned cannot deliver its alert. Set up routing on the rule card before relying on email, text or webhook notifications.
>
> - A new install ships every alert rule with *no destinations attached*.
> - Adding a destination on this screen does not attach it automatically. In each rule card's **Destinations** list, tick one or more destinations and click **Save** to attach them.
> - When an alert fires and its rule has no live destination, CivicCast writes a "suppressed" delivery note that reads "No enabled alert channel is configured for condition …". No screen shows that note.
> - The alert itself does still appear on the **Alerts** screen and in the red and yellow counts on **Readiness**.
> - There is no **Send test alert** button, so you cannot check a destination by pressing a button.
> - Some kinds of alert still have no rule, including "Automatic self-check did not pass", "asrun outbox degraded" and "caption tier degraded". They show on the **Alerts** screen as warnings but cannot be routed from **Alert rules**. **Live caption worker failed or stalled** and **EAS source unavailable** have warning rules; destinations still need to be attached by an administrator before those alerts can be delivered.
>
> Until a Setup admin assigns at least one enabled destination to each rule you rely on, those rules cannot notify anyone. Check **Readiness** and **Alerts** before every meeting and at the start of every shift.

> **For IT staff:** A rule takes `channel_ids` (the ids of destinations) through `PUT /api/staff/alert-rules/{rule_id}`, with the Setup admin role. The 14 rules that exist come with ids like `default:off-air`. See Chapter 14 and Chapter 15.

> **Role permissions:** The Alerts event list and acknowledgement actions require Setup admin, Support admin or Meeting operator. Setup admins and Support admins can view alert rules and destinations, but only Setup admins can change them. Publish operators and Records clerks can read the runtime safe-to-air health indicator, but cannot load the Alerts event list or its rule and destination editors.

#### Understand emergency alerts (Emergency Alerts)

The **Emergency Alerts** screen shows public-safety alerts the station has pulled in from outside feeds, and lets an operator put one on a channel. You can open it with the Setup admin, Support admin or Meeting operator role.

> **Warning:** CivicCast is **not** an Emergency Alert System (EAS) device. A banner on the page says so permanently: "Public-safety display — not an EAS device". It does not send the national EAS signal and never takes over programming by itself. The required national alert relay stays with your cable operator's certified equipment at the headend.

The feeds come from the National Weather Service (NWS), the federal alert system IPAWS, and AMBER child-abduction alerts. IPAWS and AMBER alerts are read in a standard format called CAP (Common Alerting Protocol); the code reads National Weather Service alerts from that service's GeoJSON feed instead. We did not test any live feed.

**What airs by itself, and what staff must do**

| Situation | What happens |
|---|---|
| A Beta.12 station with default settings | Nothing is polled and nothing airs by itself. The page says "No alert sources are configured. Ask your station administrator to configure an NWS, AMBER, or IPAWS (COG) feed." |
| IT staff turn on alert polling | CivicCast reads the configured feeds, by default once a minute, and lists active alerts under **Active alerts** |
| IT staff also turn on automatic display | Every active **severe** alert goes on every channel that is on air as a **crawl**. Every active **extreme** alert goes on as an **overlay**. A full-screen takeover is never automatic |
| An operator presses a button | The station queues a crawl, an overlay or (after a tick box) a full-screen takeover for the selected channel's next output update |

A *crawl* is a line of text that scrolls across the screen. An *overlay* is a message box placed over the picture. A *forced slate* is a full-screen message that replaces the programming.

> **Note:** There is no form to add an alert feed in the console. A station administrator must configure an NWS, AMBER, or IPAWS (COG) feed through the programming interface. Polling and automatic display remain off unless IT staff turn on the two station settings.

> **Known issue (beta.11):** Nothing on the page says that severe and extreme alerts go on air by themselves once IT turns on automatic display. The banner says only that CivicCast "never automatically pre-empts programming". Once automatic display is on, use **Clear** to take an alert down.

CivicCast draws the selected alert into the channel's GStreamer output: a scrolling crawl, a lower-half message panel, or an operator-confirmed full-screen slate. The resident home page also reads the

real alert for its current channel automatically, without a special page link, and refreshes it every five seconds. Clear, CAP cancellation, replacement and expiration update the presentation on the next successful delivery; clearing also prevents the same alert from being automatically shown again. If several alerts are active, the highest severity is shown. Other channel graphics remain in place when an alert changes or clears. Very long broadcast text is shortened; the resident notice carries the full public alert text. No cellular fallback is configured by this feature.

When an alert changes, CivicCast prepares the replacement before removing the previous warning. A failed render leaves that warning in place while a later update retries. Delivery failures can delay replacement or clearing. If Windows prevents deletion of a retired image after clearing succeeds, the warning can be absent until cleanup retries succeed. Watch the actual output when making changes. Temporary alert images are bounded during retries; this is not a cleanup of files left by older runs or a crashed process.

> **For IT staff:** The two settings are `CIVICCAST_EAS` and `CIVICCAST_EAS_AUTO_SURFACE`. `CIVICCAST_EAS` defaults to `off`; any other value (the code's own comment uses `inline`) starts the polling. `CIVICCAST_EAS_AUTO_SURFACE` is off unless set to `1`, `true`, `yes` or `on`, and it is read only when `CIVICCAST_EAS` is on. `CIVICCAST_EAS_POLL_SECONDS` sets the polling interval and defaults to 60. Polling skips any source that is disabled, is of the `manual` type or has no endpoint address. Sources are added with `PUT /api/staff/eas/sources/{id}`. See Chapter 15.

> **Output setup:** Keep emergency presentation off unless the station intends to use it. IT must enable `CIVICCAST_EAS=inline` with the GStreamer egress engine before starting the channel, so its graphics compositor is reserved. A display request is refused when emergency presentation or GStreamer is off. The installed runtime supports a CPU `compositor` fallback when D3D11 is unavailable; neither path requires a discrete GPU. Isolated MPEG-TS checks proved alert pixels and clearing on both CPU and D3D11 paths. This does not certify a cable headend or any live feed: verify the station's actual downstream output before relying on it.

### Check the feeds and active alerts

1. In the sidebar, open **System Health**, then click **Emergency Alerts**.
2. Under **Alert sources**, read each line. It shows the source name, its type (such as `nws-cap`, `ipaws-cap`, `amber-cap` or `manual`), the lowest severity it brings in (for example "≥ severe"), and **polling** or **disabled**.
3. Under **Active alerts**, read each alert: the event, a severity badge (extreme, severe, moderate, minor or unknown), a headline and the areas.
4. Under **On-channel now**, see what is currently shown on the channel you picked in **Channel**. Each line gives the channel name and the display type (`crawl`, `overlay` or `forced_slate`), not the alert's name.

> **Known issue (beta.12):** **polling** only means the source is switched on; a feed that is failing can still say **polling**. A failing feed raises an **eas source unavailable** alert on the Alerts screen. Beta.12 includes a warning rule for this alert, but a notification is delivered only after an administrator assigns an enabled destination and the delivery service is configured and working.

**Show an alert on a channel** You need the Setup admin or Meeting operator role.

1. In **Channel**, pick the channel. The first choice may be gov, a built-in guess. Pick your real channel.
2. Find the alert under **Active alerts**.
3. Click **Show crawl on** and the channel name, or click **Show overlay**.

A line for that channel appears under **On-channel now**. These buttons act at once with no confirmation and no success message.

### Take over the whole screen

1. Tick **Confirm full-screen takeover**.
2. Click **Forced slate**.

The tick resets after each use. The station refuses a forced slate without it.

> **Warning:** A forced slate replaces the picture for everyone watching that channel. Use it only on purpose.

### Take an alert down

1. Under **On-channel now**, click **Clear** on the line for that alert.
2. In the box, click **Clear alert**.

Residents on that channel go back to regular programming.

#### Understand federation (Federation)

*Federation* lets other websites that use a shared standard, called ActivityPub (the network behind Mastodon and similar sites, sometimes called "the fediverse"), follow your station and see when a new meeting is published, much like following a page on a social network. **Most stations do not need this.** It is off by default, and the screen says so.

1. In the sidebar, open **System Health**, then click **Federation**.
2. If you see **Federation is off** with a green tag, **Default-safe**, nothing is shared and the station is not advertised. You can stop here.

> **Known issue (beta.11):** There is no on/off switch in the console. The page says nothing about how federation is turned on. An IT person has to change station settings and restart CivicCast. The old built-in manual points to a switch that does not exist.

**Prepare to turn it on (Setup admin)**

1. Click **Generate station key**.
2. In the box **Generate the station key?**, click **Generate key**.
3. Click **Copy settings**, and give the copied lines to your IT person.

The key is the station's permanent identity on the network. Generating again reuses the same key. Creating a key does not turn federation on. It stays off until IT staff apply the settings and restart CivicCast, and it silently stays off if the web address or key file is missing.

> **For IT staff:** See Chapter 15 for the `CIVICCAST_ACTIVITYPUB_*` settings.

**After federation is on** The page shows counts (Pending, Accepted, Blocked, Rejected, Removed), your settings, and lists of followers and delivery attempts. A follower is another site that asks to follow you. The Publish operator or Support admin role can act on requests.

1. Click the **Pending** tab.
2. Click **Approve** to accept a follower.
3. Click **Reject** to refuse one. A box asks you to confirm with **Reject follower**.
4. Click **Block** to refuse one permanently. Confirm with **Block follower**.
5. For a delivery that failed, click **Replay delivery** to try again.

> **Warning: Block** cannot be undone from the console. The box says a blocked site "cannot re-follow until unblocked", but the screen has no **Unblock** control.

> **Known issue (beta.11):** If an **Approve**, **Reject** or **Block** fails, the page says, "Federation moderation failed. Ask your station administrator to check your access and the CivicCast service, then retry the follower action." The buttons show for every role, so a role mismatch can cause a failure. Ask your administrator if it keeps failing.

#### What to do, and what not to do

If you are not technical, this list is for you.

**Do**

- **Read the words, not only the colour.** Write down the exact message on the screen.
- **Check Readiness and Alerts before every meeting**, and at the start of every shift. Alerts appear in the console; notification destinations are not configured by default.
- **Click Acknowledge** on an alert you have seen, so your colleagues know someone has it.

- **Take a screenshot** of anything odd, including the time shown on the screen.
- **Note the time, the channel and what you were doing**, and write down whether the time is UTC or your local time.
- **Tell your IT person** early, with the details in the next section.
- **Run Check broadcast readiness before the meeting**, not during it.
- **Use the Manual** in the sidebar for how a screen works.

**Do not**

- **Do not rely on email or text alerts until they are configured and verified.** A Setup admin must assign an enabled destination to each rule you rely on, and the station's delivery service must be working. Local delivery checks do not prove that an external mail or SMS provider accepted a message.
- **Do not rely on Emergency Alerts as your EAS.** CivicCast is not an EAS device.
- **Do not press Stop, Restart feed, Repair, Update, Rollback or Restore buttons** unless you were trained to, or your IT person told you to.
- **Do not click Block or Delete casually.** Neither can be undone from the console.
- **Do not use the Paywall for live paid access** while its resident purchase and account workflows remain incomplete (see Chapter 7).
- **Do not type or paste passwords, recovery codes, staff passes (tokens) or private meeting material** into a report, an email or a public issue.
- **Do not keep pressing a button that did nothing.** Some actions only queue a request, and the page does not refresh by itself. Reload the page and look again.

#### Describe a problem to your IT person

A good report lets IT staff fix a problem without coming to your desk. Copy this list into an email or message, and fill it in.

```
What I was doing:
Which screen (the name in the sidebar):
Which channel (if any):
What I expected:
What happened instead:
The exact words on the screen (copy them, or attach a screenshot):
The colour or label (Ready, Check before meeting, Do not broadcast yet, Not set
 ↪  up yet, Needs IT help):
Date and time it happened (say if the time came from the screen, and whether it is
 ↪  UTC or local):
Is a meeting or a broadcast on the air now?  yes / no
Has it happened before?  yes / no
What I already tried:
```

If you have the Support admin role, you can also make a *support bundle*. This is a redacted troubleshooting file (the screen's own description of it). We did not check exactly what it contains. On **Readiness**, type a short note in the **Short note** box, click **Create support bundle**, then click **Download support bundle**. Give the file to your IT person. Do not post it publicly.

#### Report a beta issue

CivicCast is a beta. Check the releases page for available downloads and use the verification record that matches the installed package. Support is community-driven, with no contract and no guaranteed response time. Problems that stop broadcasting or installation, lose data or expose a secret are handled first.

1. In the operator console, click **Report a beta issue** at the bottom of the sidebar. It opens this **Report a beta issue** section in the Manual.
2. If you have a free GitHub account, open a new issue on the project's issue page: https://github.com/scottconverse/civiccast-native/issues.
3. If you do not have an account, ask your IT lead or anyone on staff who has one to post your description. Use the private contact in the project's security policy for security reports: https://github.com/scottconverse/civiccast-native/blob/main/SECURITY.md.

In every report, include:

- The CivicCast version. It is shown at the top of the console and in the health answer.
- Your Windows version.
- The screen where it happened.
- The exact message.
- What you were doing, and what you already tried.
- Whether it was a rehearsal, a test recording or a real meeting.

If the problem is with installing, also include the installer's file name and whether its checksum matched.

> **Warning:** Never include passwords, recovery codes, staff tokens, subscriber addresses or private meeting material in a report. Do not post a support bundle publicly. Say you have one, and wait for a maintainer to ask for it privately.

> **Warning:** If you think you have found a *security* problem, do **not** open a public issue. Use the private contact in the security policy linked above.

> **Support bundles:** Open **Readiness** to create a support bundle. **Create support bundle** requires the Support admin role. Keep the bundle private and share it only through an agreed support channel.

#### If it did not work

| What you see | Where | What it means and what to do |
|---|---|---|
| `Checking station readiness...` that never ends, or `Could not load System Health.` | Readiness | The page could not reach the station. Check you are signed in, then tell your IT person |
| `The live on-air signal could not load.` | Readiness | The live banner could not load. Reload; if it stays, tell IT |
| `Checking broadcast readiness requires the meeting operator role. Health checks remain visible.` | Readiness | You can read the checks but not run the test. Ask your station admin |
| `Alerts could not load.` or `This action requires one of these CivicCast roles: ...` | Alerts | The event list requires Setup admin, Support admin or Meeting operator. Ask an operator with one of those roles to check it. If your account has the required role, check your sign-in and retry; tell the station admin if it still fails |
| `Could not save the rule.` | Alerts | Your role cannot change rules. Only the Setup admin can |
| `Use 24-hour HH:MM, e.g. 22:00 (or leave blank).` | Alerts, destination form | Type quiet hours as hours and minutes, such as 22:00, in UTC, or leave blank |
| `Durable storage is not ready yet.` | Alerts and other screens | The station's database is not ready. Tell your IT person |
| `The Emergency Alerts console requires the setup admin, support admin, or meeting operator role. Ask your station admin for access.` | Emergency Alerts | Your role cannot open this screen |
| `Could not display the alert.` | Emergency Alerts | The server refused. The box usually shows the server's own reason instead of this sentence. For a full-screen takeover, tick **Confirm full-screen takeover** first |

| What you see | Where | What it means and what to do |
|---|---|---|
| `Could not load federation status.` | Federation | Click **Retry**. If it fails again, tell IT |
| `Could not verify your staff identity (...)` | Emergency Alerts | Sign in again at `/login` with your staff account; the first administrator uses `/setup` on the station computer |

**Related**

- Chapter 2: Signing in
- Chapter 4: Running the meeting
- Chapter 7: Reports, analytics, program guide export, underwriting and apps
- Chapter 14: Troubleshooting matrix
- Chapter 15: Cable headend, streaming, federation, emergency alerts and the API

# Part II. Installing and running CivicCast (for IT staff)

These chapters are for the one or two people who install and look after CivicCast at a small city or station.  They cover planning, installation, configuration, daily operation, security, troubleshooting and connecting to cable and streaming.

## Planning your station: hardware, network, storage, accounts

This chapter helps the person who runs IT at a small city decide which computer to use for CivicCast, what to open or leave closed on the network, how much disk space to set aside, who needs which accounts, and what to download and check before running the installer. Read it before Chapter 10. Nothing here changes the computer; it is all planning.

### Before you start

You need three decisions and one fact:

- **How many channels** the station will run at the same time (the lab station described below ran three).
- **Whether live captions matter** on the air. Captions are the part of the system most sensitive to the computer you choose.
- **Who will sit at the station computer** for first setup. The first administrator can only be created from a browser running on the station computer itself (Chapter 10, "First setup in the operator console").
- **The installation route:** use the project's release helper to assemble a supported package with its station-model resources: `scripts\download_windows_release_artifacts.ps1 -Install`. It verifies `station-models.json` and all matching `station-models.zip.*` parts before starting setup with `/S`.

### Choose the computer

**What the installer looks at** The installer's first-run window has a screen called **Checking This Computer**. It reads the processor, memory, graphics card and free disk space and shows them. Only one reading can stop you.

| Reading | How it is taken | What it does |
|---|---|---|
| Processor name and logical cores | Windows registry and `GetLogicalProcessorInformationEx` | Shown. Never blocks |
| Memory | `GlobalMemoryStatusEx`, in GB with one decimal | Shown. Never blocks |
| Graphics cards | Windows DXGI adapter list; the software adapter and exact duplicates are skipped | Shown as `<name> (<VRAM> GB)`. Only NVIDIA cards (PCI vendor 0x10DE) change the recommendation |
| Free space on the install drive | `GetDiskFreeSpaceExW` | **Blocks** when free space is less than the total of the default downloads |

A value the installer cannot read is shown as **Unavailable**. It is never replaced with a guess. If the whole check fails, the window says **Hardware check unavailable** and lets setup continue.

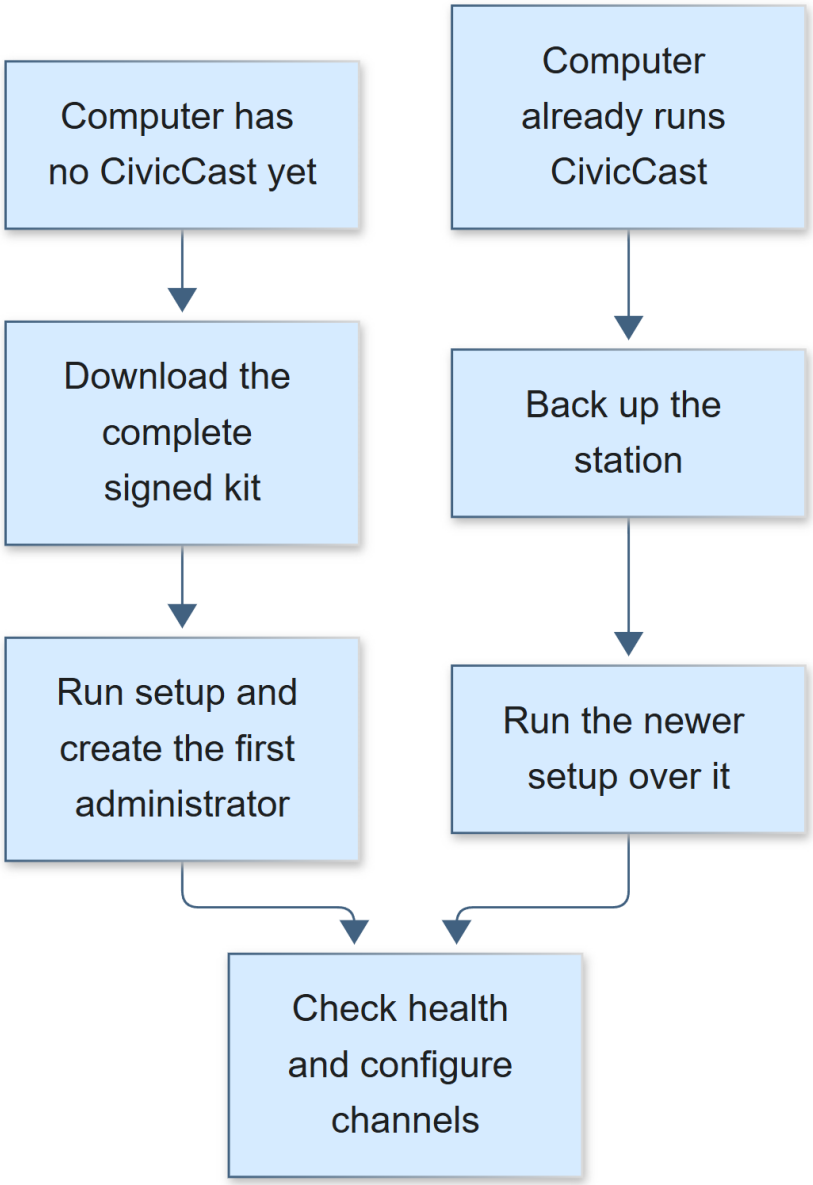



Figure 9.1. Use the complete signed kit for a first installation or an upgrade.

> **Known issue (beta.11):** The **Checking This Computer** screen runs after Windows setup has already copied and extracted the program. Its free-space check covers only optional downloads selected in the first-run window; it is not a pre-flight for the full installation. Use the storage table below to plan room for the staged packs, extracted models, conform cache and recordings, and follow the exact package's disk-space message.

**The operating system and software**

| Item | What the code and release files show |
|---|---|
| Windows, 64-bit | The installer, runtime and Visual C++ prerequisite are all x64. **No minimum Windows version is stated** in the installer configuration or code. The lab station ran Windows 11. We have no record of tests on Windows 10 or on Windows Server |
| Administrator rights | Setup installs for the whole machine (`perMachine`) and asks Windows for approval (UAC). The first-run window runs as the normal signed-in user |
| Visual C++ runtime | Setup installs a bundled offline copy. It may ask for a Windows restart (the runtime installer returns code 3010) |
| WebView2 | The setup window needs Microsoft WebView2. A bundled offline copy is installed silently if it is missing. No internet is needed for it |
| Windows Defender Firewall | Setup adds one rule with `netsh advfirewall`. If that fails, setup stops with code 119 |
| No WSL, no Docker | CivicCast Native runs as a Windows service. It does not use WSL or Docker |
| The older WSL-based CivicCast | If a computer already has the older "CivicCast Installer" (WSL edition), setup may stop with 135 or 127, and uninstalling CivicCast (Native) asks an ownership question. Plan to remove the older edition first |
| Clock and time zone | Keep the Windows clock synchronized and set the station's IANA time zone in Station Profile. Chapter 3 lists the time zone used by each entry form |

One more rule of thumb: dedicate the computer. The station runs a database, a web server, an AI engine and several video processes under the restricted Windows virtual account `NT SERVICE\CivicCastSupervisor`. We do not know of testing alongside other heavy software.

**Graphics card: what it changes** **Live captions use Whistle on the CPU by default.** Whistle publishes its first recognition without waiting for a second reading to agree. One station-wide lock serializes primary recognition across the station's channels. Requests have a 30-second deadline. If Whistle fails or times out, Whisper supplies backup captions while CivicCast automatically attempts to restore Whistle, one channel at a time. No service restart is needed after a successful recovery. Whisper fallback allows up to 120 seconds per request by default so slower CPU recognition can finish; its deadline and audio queues remain finite.

Whisper remains the engine for recorded-media captions and the configured fallback for Whistle. It can also be selected as the live primary through the service environment. CUDA is optional Whisper acceleration; it is not needed for the Whistle primary. AMD and Intel graphics do not provide CUDA acceleration. If `CIVICCAST_WHISPER_DEVICE` is unset, the service uses CUDA only when a supported NVIDIA GPU with at least 8 GB and the staged CUDA runtime are both present; otherwise Whisper runs on CPU. Set `CIVICCAST_WHISPER_DEVICE=cuda` or `cpu` to choose explicitly. `CIVICCAST_LIVE_CAPTION_ENGINE=whistle` selects the default Whistle primary with Whisper fallback; `whisper` selects Whisper for live captions. Restart the service after changing these settings. Needle usage telemetry is disabled; speech processing and model assets remain local.

The installer ships the signed Whistle component with the station packs. **Medium Whisper** is also required for fallback and recorded captions. **Large Whisper** and the CUDA runtime are optional; Large may be preselected on capable hardware.

A graphics card can accelerate **Whisper** and affect the default summary model. It is not required for live captions on the native Whistle path. It does not lighten video encoding (see the encoding paragraph after the tables).

| Your computer has | What the installer recommends | Effect on Whisper |
|---|---|---|
| No dedicated graphics card, or AMD or Intel graphics only | Medium; Large and CUDA remain optional | CPU with int8 by default |
| NVIDIA card with **less than 8 GB** of video memory | Medium; Large and CUDA remain optional | CPU with int8 by default |
| NVIDIA card with **8 GB or more** | Large and CUDA may be preselected | CUDA only when the CUDA runtime files are staged; otherwise CPU |

These Whisper choices affect the fallback, recorded captions, or an explicitly selected Whisper live-primary mode. Native live captions still default to Whistle.

> **Note:** The rule uses video memory as a stand-in for card capability. The code records that an older NVIDIA card with enough memory but no tensor cores can be *slower* on the card than on the processor (one older card missed all 30 deadlines in testing). The station operator can override the device with the `CIVICCAST_WHISPER_DEVICE` environment variable; see Chapter 11.

> **Historical beta.10 behavior:** The older native live path selected the highest installed Whisper tier. Beta.12 defaults to Whistle with Whisper fallback. The October 11 R9 report found that the separate acquisition screen downloaded Large Whisper despite an unchecked option. Use the complete release helper and its supported silent setup path; it does not depend on that acquisition screen.

The AI that writes summaries and translations is chosen by a second rule.

| Condition | Summary model chosen by default |
|---|---|
| Any NVIDIA graphics card detected **and** 16 GB of memory or more | `gemma4:12b` |
| Anything else | `gemma4:e4b` |

Do not assume every model is present on every station. Silent setup does not launch the first-run GUI acquisition flow or automatically fetch its models; the GUI offers acquisition separately. A displayed download or completed transfer does not by itself prove that a model was activated. The installed runtime tags and the installer pack-component IDs are different identifiers; use the installed model/runtime status when troubleshooting. The product catalog lists a minimum of 16 GB of memory for the 12B model and 8 GB for the smaller one. A historical no-dedicated-GPU benchmark used a computer with 32 GB of memory and 16 cores (32 threads): the smaller model finished a summary in 94 seconds warm and 128 seconds cold, while the 12B model took 366 seconds once and then failed twice. We have no measurement on a smaller computer.

**The default video encoder runs on the processor.** CivicCast uses OpenH264 software encoding by default. Compatible NVIDIA or Media Foundation hardware encoders can be configured when present; the channel profile and pre-flight determine whether one is used or a CPU fallback is allowed. The default output profile is 1280 x 720, 30 frames per second, H.264 at 6,000 kbps and AAC audio at 192 kbps. For multiple live channels with captions, size the processor for video playout and Whistle's station-wide serialized recognition, then observe the planned workload on the intended computer.

**What was measured on the lab station** The three-channel evidence in the verification record came from one lab computer. The project's oversight notes describe it as Windows 11 with an AMD Ryzen 7 7800X3D (8 cores, 16 threads) and an NVIDIA RTX 5070 Ti. We do not have its memory size on record.

| Measure on that computer | Result |
|---|---|
| Three channels on air for 8 hours | One service process the whole time; no restart; 0 watchdog firings |
| Processor load while preparing a long program for the first time | 95 to 98 percent (oversight notes) |
| Conform cache | 46 GB of its 60 GB budget used |

**Our recommendation** These follow from the evidence above. They are not tested minimums, because the project has not published any.

1. **For several live channels with captions**, size the processor for video playout and Whistle's station-wide serialized recognition, then observe and measure the planned workload on the intended computer.
2. **Without an NVIDIA card**, Whistle live captions still run on CPU. An NVIDIA card can accelerate Whisper fallback/recorded captions and may allow the larger summary model; neither the GPU nor CUDA is required for Whistle. Test the summary and caption workload on the intended machine.
3. **Do not run other large programs on the station** during meetings.

To see what CivicCast itself detects after install, open `http://127.0.0.1:8000/api/hardware` in a browser on the station (it needs no sign-in) or run `civiccast doctor` (Appendix A).

#### Plan the network and firewall

**What listens where** The application services listen on the station computer's loopback address. To provide remote resident and staff access, use the supported Caddy configuration with separate public and CIDR-restricted staff hostnames; do not expose port 8000 directly. See the network access guide.

| Port | Used by | Reachable from | Source |
|---|---|---|---|
| TCP 8000 | The CivicCast web server: operator console at `/operator/`, resident portal at `/`, the API, and `/health` | The station computer only | `children.py`, `core.py` |
| TCP 5432 (or the first free of 5433, 5434, 5435, 5544) | PostgreSQL database | The station computer only | `provision/models.py`, `port_select.py` |
| TCP 11434 | The bundled Ollama AI engine (version 0.30.6) | The station computer only | `children.py` |
| TCP 38474 | The setup window's one-copy-at-a-time guard | The station computer only | `main.rs` |
| UDP 17800 upward; UDP 18000 to 18499 | Internal media relays between the playout engine and ffmpeg or TSDuck. Source ports for outgoing transport streams sit 1,000 above the first range | The station computer only | `ts_relay.py`, `hls_relay.py` |

Do not open the internal ports on the firewall. If another program on the same computer already uses 11434 or 8000, plan to move it; we did not test what CivicCast does when a port is taken.

Setup adds a Windows Firewall rule named **CivicCast (Native) Portal/API (TCP 8000)**, but the application listener remains loopback-only. The rule does not expose it to other computers; configure the Caddy edge and its firewall separately for remote access.

**What the station connects out to**

| Purpose | Destination | When |
|---|---|---|
| First-run window downloads (signed runtime packs) | `github.com` release downloads (and the hosts GitHub redirects to) | Only if the window has to download something. Redirects that are not HTTPS are refused |
| First-run window downloads (caption weights) | `huggingface.co` | Same |
| First-run window downloads (AI model) | `registry.ollama.ai` | Same |
| Windows setup itself | None. The setup phase makes no network call; the Gate A run had networking disabled | Never |
| Video and publishing outputs | Whatever you configure: a cable headend (UDP or SRT), a CDN, a streaming service, email, a federation server | Chapter 15 |

The station does not need the internet to run. The Gate A run had networking disabled, and its station installed, came up and passed its checks. The code even hides the built-in `/docs` page because "a council-chamber station is frequently firewalled outbound and sometimes air-gapped".

The first-run GUI has a separate model-and-caption acquisition flow. Its optional selections control that download plan; required components remain included. Silent setup uses the complete signed kit rather than opening this GUI. A downloaded raw model directory does not replace the signed station resources needed for activation.

If a security appliance does TLS inspection or an allow-list, allow the destinations above needed by the selected GUI acquisitions, or use a complete local kit where available and block outbound access.

Two details matter if you control outbound traffic. A UDP transport stream to a headend leaves from a local port pinned 1,000 above the relay's listening port, so a firewall that filters by source port needs to know that. And the Gate A test harness added its own allow rules for `tsp.exe`, `ffmpeg.exe` and `ffprobe.exe`; the product does not add them. If a third-party firewall prompts on first use of those programs, allow them. The other outputs depend on what you configure (Chapter 15).

**Plan storage**

CivicCast keeps the program under the install folder (default `C:\Program Files\CivicCast (Native)`) and everything it creates under `C:\ProgramData\CivicCast`. The data folder follows the `PROGRAMDATA` environment variable and is on the system drive unless that variable is changed. We know of no test that moved it.

| What | Where | Size, with source |
|---|---|---|
| Program, runtime, video tools, AI engine | `<install folder>` | Signed runtime packs are staged here; check the matching release checksum and package manifest for exact sizes |
| Model packs: cached copy | `<install folder>\packs\.station-cache` | "about 21 GB" (the uninstall notice) |
| Model packs: extracted copy | `<install folder>\packs\captions-floor`, `components\`, `models\ollama\` | Extraction is about 1 to 1 with the pack size (code constant). The cached copy stays on disk too, so models take about twice the pack size |
| First-run downloads, if any | `C:\ProgramData\CivicCast\packs` and `components` | The download plan includes required components and the optional models selected in the GUI |
| Database | `C:\ProgramData\CivicCast\data\pgdata` | Holds records, not video; grows with use. We have no measured figure |
| Uploads and finished packages | `C:\ProgramData\CivicCast\data\uploads` | Operator-uploaded media |
| Conform cache (ready-to-play copies of programs) and working files | `C:\ProgramData\CivicCast\data\egress` (cache in `conform-cache`) | Budget **60 GB** by default; set `CIVICCAST_CONFORM_CACHE_GB` to change it (zero or less turns the cache off). Another budget keeps the three newest plan folders and about 5 GB (`CIVICCAST_PREPARED_PLAN_DIR_BUDGET_GB`) |
| Temporary live-caption work | `C:\ProgramData\CivicCast\data\caption-tap` | Short working audio is removed during normal processing; native live captioning does not create permanent per-cue review rows or evidence WAVs |

| What | Where | Size, with source |
|---|---|---|
| Scheduled recordings | In a `scheduled-recordings` folder under the recording target you set in the console | One program-hour is about 2 GB by the Setup screen's planning figure; the default output profile works out to about 2.8 GB per hour (6,192 kbps x 3,600 s, our arithmetic) |
| Logs | `C:\ProgramData\CivicCast\logs` | Supervisor, application and raw control-plane/PostgreSQL logs rotate at 10 MiB with 10 older copies. Older oversized raw logs may remain until rotation replaces them; `ollama.log` is not bounded. See Chapter 12 |
| Backups | A folder you choose | You decide |

Use the checksums for the matching release and the receipt for any local kit you received; pack sizes can change between releases. Activation checks staged station-pack sizes plus 2 GB of working room and reports "Not enough free disk space to activate this station..." when short. Add the conform cache budget (60 GB by default) and recording storage. The matching verification record identifies measured package observations.

> **Tip:** Do not rely on the default 60 GB cache fitting on a small system drive. Either give the station a large drive, or lower `CIVICCAST_CONFORM_CACHE_GB` before the first busy week. A single prepared program larger than the whole budget cannot be kept. The station then refuses it with the error "Conform-cache budget too small to retain '<file name>'; increase CIVICCAST_CONFORM_CACHE_GB or exclude this asset."

> **Known issue (beta.11):** The Setup screen's **Backup destination** control only proves that the folder accepts a test file (it writes, reads and deletes one). Its success message is "Backup destination accepted a write/read/delete proof." It does not copy station data there. See Chapter 12 for how backups are actually made. Use a local drive or a network path the restricted service account can reach; a drive letter mapped only in your session is not visible to a Windows service.

#### Plan accounts and permissions

| Who or what | What they need | Why |
|---|---|---|
| The person who installs | A Windows administrator account | Setup is per-machine and uses UAC. Run a silent install from an elevated prompt |

| Who or what | What they need | Why |
|---|---|---|
| The Windows service | Nothing to create | It is registered as **CivicCast Native Supervisor** (`CivicCastSupervisor`), runs as the restricted virtual account `NT SERVICE\CivicCastSupervisor`, starts automatically, and restarts after 5, 10 and 30 seconds if it fails |
| The person at the station for first setup | A normal Windows account on the station computer and a browser | First setup, sign-in and recovery only work from the station itself (a loopback check; other computers get HTTP 403) |
| The first CivicCast administrator | A display name, a username, a password of **12 characters or more**, and a place to keep the recovery kit | Created by the browser form in Chapter 10. This recovery account carries all five roles. A setup admin creates other staff accounts and assigns their roles (Chapter 13, Appendix F) |
| A person who keeps the recovery kit | A safe place that is not a public folder | The kit holds eight one-time recovery codes and, in the saved file, the administrator password in plain text |
| Security software | An exclusion or allow rule for `<install folder>` and `C:\ProgramData\CivicCast` if it blocks writes | The first-run window's own error text names "security software blocking the CivicCast folder" as the usual cause of a write refusal |
| Backup and publishing destinations | Credentials you hold | Chapter 15 covers provider credentials |

Windows accounts matter in one more place: the first-run window remembers that it has finished in `%USERPROFILE%\.civiccast` and in the browser storage of the Windows account that ran it. A different Windows account sees **Checking This Computer** again.

#### Download and verify what you need

**What is on the release page** This manual describes the unpublished Beta 20 candidate edition. Use the releases page for published packages and run the release-matched helper to obtain the installer, runtime packs and signed station-model resources together. The Beta 18 verification record tracks

that earlier package's results.

The complete offline kit contains the installer, its verification metadata, runtime packs, and signed station index/model packs. The exact file list comes from the matching release manifest; the Chapter 10 tree shows the working layout rather than every download or checksum file. The release helper uses `station-models.json` and every matching `station-models.zip.*` part, verifies the set and prepares the folders before setup. Raw model downloads or unsigned substitutes are not activation packs.

The assembled kit includes Whistle and the signed station-model resources. The first-run GUI separately offers optional model downloads; leaving Large unchecked excludes it from that GUI plan. See Chapter 10 for the complete-kit and GUI workflows.

### Check the files

1. Put `setup.exe`, `SHA256SUMS.txt`, `setup.exe.sidecar.json` and any `.ccpack` files in one folder. Open PowerShell there.
2. Compute the installer's hash and compare it with the same file's line in `SHA256SUMS.txt` and with the `sha256` value in the sidecar. All three must match exactly.
   ```
   Get-FileHash .\setup.exe -Algorithm SHA256
   Get-Content .\SHA256SUMS.txt
   Get-Content .\setup.exe.sidecar.json
   ```
   Hashes change with each package. Compare the whole value from the release page you downloaded from, not from an example in this manual.
3. Check the signature:
   ```
   Get-AuthenticodeSignature .\setup.exe | Format-List Status,
   ↪  SignerCertificate
   ```
   Expect `Status` to be `Valid` and the signer to be `CN=Scott Converse` (Azure Trusted Signing; the project's signing policy records this). Any other result is a reason to stop.
4. Check the packs the same way: `Get-FileHash .\*.ccpack -Algorithm SHA256` and compare each with `SHA256SUMS.txt`. The installer re-checks every pack's signature itself.

A **kit** has its own checks. Use the kit's own hash list or delivery manifest from whoever handed it over, and do not expect the GitHub sidecar in a kit. The installer inside a kit may carry a different file name than `setup.exe`; use the name in the kit's manifest.

**The Windows SmartScreen warning** Windows may show a blue **Windows protected your PC** screen. That screen is about download reputation, which a new publisher has not built up yet. It is not a verdict on the signature. Check the hash and the signature first (steps above). If both are right, choose **More info** and then **Run anyway**. If those choices are missing, or the publisher or hash differs from what you checked, stop. Your IT policy can also allow the signed build by publisher or by hash.

### Pre-install checklist

☐ I know this is a beta candidate and who in the city has accepted that.

- ☐ For a first install, I have prepared the matching kit with `scripts\download_windows_release_artifacts.ps1 -Install`; the helper verifies and assembles runtime and signed station resources before setup runs. The R9 asset set failed activation because required station packs were missing.
- ☐ I checked the installer's SHA-256 against two sources and its signature shows `Valid` for the expected signer.
- ☐ The computer is 64-bit Windows, dedicated to CivicCast, and not running the older WSL-based CivicCast.
- ☐ The install drive has about 55 GB free, plus room for the conform cache and recordings.
- ☐ I have an administrator account and the computer will stay powered and awake for the whole install (Gate A took about 33 minutes from start to a healthy station inside Windows Sandbox; a real computer will differ).
- ☐ Windows Defender Firewall is on and not locked by policy against adding a rule, or I have a plan for exit code 119.
- ☐ Security software will not quarantine `C:\Program Files\CivicCast (Native)` or `C:\ProgramData\CivicCast`.
- ☐ I know the plan for backups (Chapter 12) and where the recovery kit will be kept.
- ☐ Someone will be at the station to create the first administrator in a browser on that computer.
- ☐ If this is an upgrade, I have a separate backup and recovery path. Beta 12 has one observed normal upgrade from Beta 11 with the recorder check incomplete; see its verification record for the exact result.
- ☐ I have a way to copy text out of the setup window and out of `C:\ProgramData\CivicCast\install-progress.log` to send to support.

**If it did not work**

| What you see | Cause | What to do |
|---|---|---|
| The hashes do not match | A damaged or altered download | Do not run it. Download again from the exact release page, or ask the person who gave you the kit for a new copy |
| `Get-AuthenticodeSignature` is not `Valid` | The file is not the signed release | Stop. Do not install |
| Hardware check unavailable. "CivicCast could not check this computer's hardware. It will not guess…" | The probe failed | Setup can continue. If a download later runs out of room, free space and choose Retry |
| Not enough free disk space. "This drive doesn't have enough free space: needs X.X GB free, this drive has Y.Y GB." | The first-run window found too little room for its downloads | Free space on the drive, then close the window and open **CivicCast (Native)** again so the check repeats |

| What you see | Cause | What to do |
|---|---|---|
| "Not enough free disk space to activate this station…" in the setup details list (exit 123) | Activation needs the model-pack sizes plus 2 GB | Free the space the message names and run setup again. Nothing was deleted |

#### Historical measurements and package evidence

These host measurements belong to Beta 11 and are not Beta 16 capacity results. The 72-hour run covered Public, Government and Education without a service restart and recorded **99.78% live caption-input coverage**; the earlier 48-hour window recorded **99.88% rounded coverage**. See the Beta 11 72-hour host report and Beta 11 48-hour host report for the methods and run identities. Package checks belong to the verification record for the matching release; the Beta 18 record tracks its package results.

#### Related

- Chapter 10: Installing, first run, upgrading, uninstalling
- Chapter 11: Configuring the station
- Chapter 12: Running it day to day
- Chapter 13: Security and privacy
- Chapter 15: Cable headend, streaming, CDN, federation, emergency alerts, the API
- Appendix A: command-line reference

## Installing, first run, upgrading, uninstalling

This chapter walks through Windows setup, first run, first setup in the operator console, upgrade, repair and uninstall. It also explains setup exit codes and logs. Read Chapter 9 first.

### Before you start

- For a first install, use the release-matched helper linked in Install CivicCast on Windows. Its `-Install` option verifies `station-models.json` and every matching `station-models.zip.*` part, assembles the complete kit, then runs setup with `/S`.
- You checked the installer's hash and signature (Chapter 9, "Download and verify what you need").
- You have a Windows administrator account. Allow about 55 GB on the install drive for a prepared offline kit, or about 100 GB when the download preparation and installation share one drive, plus space for media and recordings.
- The computer will stay on, awake and connected to power for the whole install.
- For an upgrade, you also have a backup and you accept that the station goes off air while setup runs.

> **Note:** Use only a package listed on the releases page or named by the active owner-authorized handoff. A previous release's verification record does not qualify the package you install. Keep a separate backup and a station recovery path before upgrading.

### What the release and local kit contain

Setup has two phases. The **Windows setup** (`setup.exe`) runs with administrator rights and does the installation work: it copies the program, checks components, provisions the database, activates the station and registers the Windows service. Then the **first-run window** ("CivicCast (Native) Setup") opens as a normal user and offers a separate acquisition flow. Silent `/S` setup does not open that GUI or automatically fetch its models. A GUI download is not proof that a model is active.

| What you start with | Installer behavior | Package evidence |
|---|---|---|
| Complete offline kit: setup, runtime packs under `packs\`, signed index and station packs under `station\` | Verifies signed components and activates the local station | Use the verification record matching the package |
| Release download helper with `-Install` | Downloads and verifies the complete set, creates the folder layout, then runs setup | Use all matching archive parts and release checksum metadata |
| Raw caption or AI model downloads from the first-run GUI | Acquires required and selected optional components | Does not replace signed activation packs |
| Incomplete kit or mismatched station resources | Reports the missing or invalid component; activation cannot complete | Obtain the matching complete kit before retrying |

The complete kit has five runtime packs and a signed station index with six station packs. Whistle is part of that signed set. The release helper assembles the complete kit from the matching release assets.

```
CivicCast-kit\
  setup.exe              (a kit may use a different file name)
  packs\
    native-app-payload.ccpack
    native-server-binaries.ccpack
    native-ffmpeg-runtime.ccpack
    native-ollama-runtime.ccpack
    native-cuda-runtime.ccpack    (optional GPU libraries)
  station\
    station-index.json
    core.ccpack
    captions-floor.ccpack
    summary-gemma4-12b.ccpack
    summary-gemma4-e4b.ccpack
    translation-translategemma-4b.ccpack
    captions-whistle.ccpack
  QUICKSTART-OPERATOR.md
```

Place runtime packs in the `packs\` subfolder beside `setup.exe`. The four runtime packs other than `native-cuda-runtime.ccpack` are required; CUDA is optional. The signed station index lists `core`, `captions-whistle`, `captions-floor`, `summary-gemma4-12b`, `summary-gemma4-e4b` and `translation-translategemma-4b`. The helper places their matching `.ccpack` files under `station\`. Verify files against the matching release metadata or local kit receipt.

> **Note:** Windows setup stages signed runtime packs under the `packs\` subfolder beside `setup.exe`. The release helper verifies and assembles the separate station-model archive into the kit; setup itself does not fetch missing station packs. The first-run window's model and caption acquisition does not replace the signed station index or station packs needed for activation. Silent `/S` setup skips that GUI acquisition flow.

> **Tip:** Run setup from a local drive or a USB drive. A network share that you mapped to a drive letter in your own Windows session is usually not visible to the elevated setup (standard Windows behavior; not tested with CivicCast). Use the UNC path or copy the kit.

#### How the install runs

Each box is a step that writes a line before and after to `C:\ProgramData\CivicCast\install-progress.log`. If a step fails, setup stops, shows a dialog (when it is not silent) and exits with that step's code. On a failure after mutation, setup stops the service while attempting restoration of the saved application and station state. Successful restoration also restores the prior service configuration; incomplete restoration preserves the recovery journal and reports exit 136. Correct the reported cause before retrying setup. An incomplete kit cannot supply missing signed station packs; obtain the complete matching bundle first.

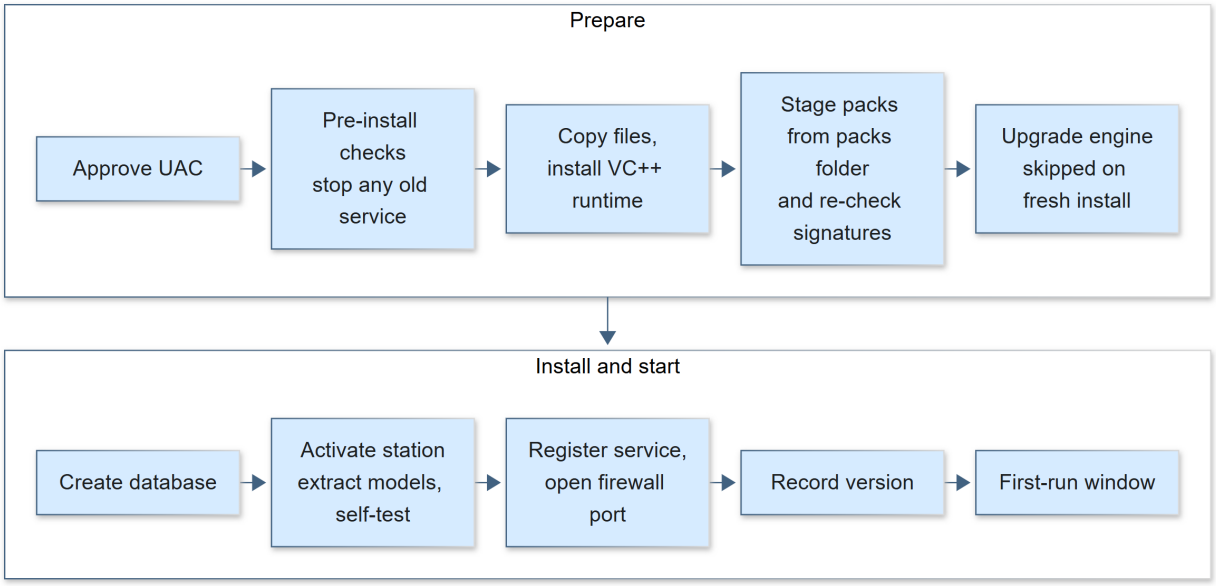



Figure 10.1. The order of the Windows setup steps and the first-run window.

#### Install from the kit

The page order of the standard Windows setup comes from a template that is not in the project's repository, so we could not confirm the exact sequence or wording of its standard pages. The text below that is in quotation marks comes from the project's own files.

### Step 1: Start setup and approve the prompt

1. Use the downloaded release helper with `-Install`. For an already prepared complete offline kit, run `Start-Process .\setup.exe -ArgumentList '/S' -Verb RunAs -Wait -PassThru` in PowerShell from the kit folder and check that `ExitCode` is 0. This supported route skips the separate acquisition window described below.
2. If Windows shows **Windows protected your PC**, follow Chapter 9 (check hash and signature first, then **More info**, then **Run anyway**).
3. Approve the administrator (UAC) prompt.

If CivicCast (Native) is already running, setup says "CivicCast (Native) is running! Please close it first then try again." or offers to end it ("Click OK to kill it"). If it cannot end it: "Failed to kill CivicCast (Native). Please close it first then try again".

If WebView2 is missing, setup installs the full offline copy it carries (the build configuration says "offlineInstaller", installed silently). The details list should show "Installing WebView2...", then "WebView2 installed successfully". The setup text also defines a line "Downloading WebView2 bootstrapper...", but that belongs to a download mode this build does not use, so we do not expect to see it; we could not confirm the exact lines on screen. On failure: "Failed to install WebView2! The app can't run without it. Try restarting the installer."

**Step 2: The "Already Installed" page (only if CivicCast is on the computer)** Before the normal setup pages or prerequisite installers, setup checks the installed CivicCast version. If the installed version is newer, setup exits with code 129 before installing prerequisites, stopping the service, replacing application files, or changing station data. Keep using the installed version, or use its complete matching signed setup kit if it needs repair. If setup cannot safely read or compare the installed version, it exits with code 130 before changing the installation; follow the message and contact support if it persists.

The corrected installer suppresses the generic **Already Installed** maintenance page so an upgrade cannot invoke the old uninstaller before recovery state is saved. Run the new matching setup directly over the existing installation; do not uninstall first. See "Upgrade" below. The older silent-installer downgrade message is historical; the current setup refuses a downgrade in the early version check for both graphical and silent setup.

**Step 3: Choose the install folder** The folder page asks "Choose Install Location". Its default is `C:\Program Files\CivicCast (Native)`. Above the browse box setup shows:

"CivicCast (Native) needs the space shown below on this drive for the program itself. After Setup finishes, the CivicCast setup wizard downloads additional components (captions and AI models) separately – their sizes are shown individually, before anything downloads, on that wizard's own screen."

The **Space required** figure is the installer's declared 5,400,000 KB (about 5.1 GiB) budget for the first setup phase; it is not the total space for every model. The next wizard shows the size of each selected optional download, and station activation checks the signed pack sizes plus 2 GB of working room on the install drive before extracting them. Keep space available on the install drive for both the program and its component cache. The database, media, recordings and logs use `C:\ProgramData\CivicCast`; some service state uses the service account's profile (see Chapter 16).

**Step 4: Watch the install** Start the install. The details list fills with lines. Setup also writes each step to `install-progress.log`. The labels D2, D3, D4 and K1 are the project's internal names for steps. The table shows what each line means.

| What the details list says | What is happening |
|---|---|
| "Preparing the existing CivicCast (Native) installation for a data-preserving upgrade…" | Upgrade only. The old service is being stopped gracefully |
| "No prior CivicCast Native bootstrap process to stop (fresh install)." | Fresh install |
| "Installing the offline Microsoft Visual C++ runtime prerequisite…" then "Microsoft Visual C++ runtime installation completed." | The bundled C++ runtime. If a restart is needed: "…completed; a Windows restart is required." Setup continues and marks the restart as pending. If a same-or-newer runtime is already present: "Microsoft Visual C++ runtime already present, same or newer — prerequisite satisfied; setup did not reinstall it." |

| What the details list says | What is happening |
|---|---|
| "Staging required native component packs from the 'packs' folder next to this installer…" then "Required native component packs staged and verified. Full detail: `C:\ProgramData\CivicCast\install-progress.log`" | Each pack's signature and contents are checked, the pack is copied into the install folder, checked again, and extracted. Nothing is downloaded |
| "Re-verifying the extracted native-server-binaries component pack against its signed pack file (D2)…" then "Native-server-binaries component pack verified against its signed pack file (D2)…" | Re-checks the extracted files. The same for native-app-payload, native-ffmpeg-runtime and native-ollama-runtime |
| "Running the CivicCast (Native) install/upgrade engine (D3)…" | Decides fresh install, same version, or upgrade. On a fresh install: "…no existing installation was found, so this is a fresh install — the install/upgrade engine was not applicable and did not run." |
| "Provisioning the CivicCast (Native) PostgreSQL server (D4)…" then "CivicCast (Native): database/messaging provisioning complete (or already provisioned; no-op)." | Creates the database. The word "messaging" is left over; the messaging server was removed from the product |
| "Activating the CivicCast (Native) station (K1)…" then "CivicCast (Native): using the station bundle beside setup.exe (`<folder>\station`)." | Extracts the speech and AI models and runs the self-test (next section). This is the longest step |
| "Registering the CivicCast (Native) supervisor service (D4)…" | Registers the service, starts it and waits for it to answer |
| "Registering the CivicCast (Native) portal/API firewall rule (D4)…" | Adds the firewall rule |
| "CivicCast (Native): recorded InstalledVersion `<installed version>` for the next install/upgrade run." | The last step before success |
| "CivicCast (Native) bootstrap install complete: required component packs staged and D2-verified, D3 install/upgrade engine run, PostgreSQL provisioned, service and firewall rule registered." | Setup finished |

Setup can take time while it extracts components and runs its self-test. Keep the window open and check `C:\ProgramData\CivicCast\install-progress.log` if the details list is quiet. Do not interrupt setup while a step is still running.

**The activation step and its self-test** Activation turns the extracted files into a working station. Setup looks for `<setup folder>\station\station-index.json` first. Otherwise it uses the embedded index and the model pack cache `<install folder>\packs\.station-cache`. Missing or invalid required signed resources stop activation with exit 123. The complete release helper prepares the matching resources; first-run raw-model acquisition is a separate workflow.

Activation then:

1. Checks the index signature and every pack's signature, size and SHA-256 (failure: child exit 66).
2. Does nothing if this exact station is already activated.
3. Checks free space: the sum of the pack sizes plus 2 GB of working room. If short: "Not enough free disk space to activate this station. The station's components need about N GB, plus 2 GB of working room, on the drive holding <folder> – but only M GB is free. Nothing has been changed or deleted. Free up space (or install to a drive that has it) and run setup again."
4. Deletes any old extracted component folders, then extracts the signed Whistle runtime, the Medium Whisper model to `packs\captions-floor`, the three AI model packs to `components\<id>`, and the optional Large Whisper model to `components\captions-large-v3`. The three AI model packs are merged into one store at `models\ollama`. It then checks that the required files are present.
5. Runs the self-test below. If every check passes it writes `activation-self-test.json` and then `station-set.json` in the install folder. Without them the station stays in a not-activated state.

The self-test runs the real programs once. The caption test is forced onto the processor; the AI test uses whatever the AI engine chooses:

| # | Check | Passes when | Time limit |
|---|---|---|---|
| 1 | The bundled Python imports CivicCast, the caption libraries (faster-whisper 1.2.1, ctranslate2 4.8.1) | It prints `native-core-ok` | 30 s |
| 2 | PostgreSQL tools `postgres` and `pg_ctl` run | Output names each | 30 s each |
| 3 | `ffmpeg` runs | Output contains "ffmpeg" | 30 s |
| 4 | The AI engine runs | Output contains "0.30.6" | 30 s |
| 5 | TSDuck `tsp`, only if present | Absent is allowed and logged | 30 s |

| # | Check | Passes when | Time limit |
|---|---|---|---|
| 6 | Caption test: a short recording, using Large if installed, else Medium, on the processor | The text contains “fellow americans” and “country” | 300 s |
| 7 | AI test: a private Ollama starts on a free local port; asks `gemma4:12b` to reply `CIVICCAST_OK`, `gemma4:e4b` to reply `CIVIC-CAST_FALLBACK_OK`, and `translategemma:4b` to translate “The council meeting is open.” into Spanish | Exact replies | Ready within 300 s; each request up to 300 s |

> **Known issue (beta.11):** If station activation fails, the dialog says the failing self-test is named in `C:\ProgramData\CivicCast\install-progress.log`, but that file records the activation step and returned code, not the failing check. Read and copy the setup window’s details list before closing it. Activation child code 67 covers more than self-test failures, including insufficient disk space or extraction errors; the details list distinguishes them.

**Service, firewall and finish** Setup registers **CivicCast Native Supervisor** (`CivicCastSupervisor`) to run automatically as the restricted Windows virtual account `NT SERVICE\CivicCastSupervisor`, starts it, and waits up to 120 seconds for Windows to report it running and up to 180 seconds for the web server to answer `/health`. On an existing default LocalSystem installation, setup first migrates the old service profile and CivicCast credentials before changing the account. See the restricted service identity guide. Setup then adds the firewall rule, records the version, fixes the “Apps” entry’s size and uninstall command, and writes three shortcuts: **CivicCast Operator Console** on the Desktop and in the Start menu folder “CivicCast (Native)”, and **CivicCast Public Portal** in that folder. The shortcuts open the local loopback listener at `http://127.0.0.1:8000/operator/` and `http://127.0.0.1:8000/`.

> **Known issue (beta.11):** The shortcuts are plain web links. If the service is stopped they show a browser error, and nothing tells the operator to start the service. See “If it did not work”.

**Step 5: The first-run window** After setup, the window titled **CivicCast (Native) Setup** appears (the final setup page wording can vary). If it does not appear, start **CivicCast (Native)** (the program is `CivicCast Native.exe` in the install folder). A silent `/S` install never opens it and does not automatically fetch the GUI's models. The window runs as the signed-in user and has up to four screens. The initial flow appears once per Windows account; use **Download AI models and captions** under **More options** to reopen that flow later.

**Checking This Computer.** "CivicCast is looking at this computer's hardware so it can recommend the right setup." It then lists Processor, Memory, Graphics and "Free disk space on <install target>", gives a one-sentence caption-engine recommendation (Chapter 9), and shows **Continue**. If free space is short it shows "Not enough free disk space" and hides **Continue**: "Free up space on this drive, then reopen CivicCast Installer." (The window is titled "CivicCast (Native) Setup"; reopen **CivicCast (Native)**.)

**What CivicCast Needs.** "Whistle is included with the signed station components and is the CPU primary for live captions. The Whisper models listed here provide live-caption fallback and recording transcription: Medium is required and Large is optional." It lists seven rows with sizes: CivicCast application runtime, Database & messaging services, Video and audio tools (installed by Windows setup), Whisper Medium (required), Whisper Large (optional), Whisper GPU acceleration (optional) and Local AI model (summaries & translation). The footer shows the total for required and selected optional downloads and **Continue**.

The GUI shows optional choices and reports found-local components. Its download plan includes your selected optional components and all required components. Leave Large Whisper unchecked to omit that optional transfer. Raw-model acquisition and signed station resources are separate: use the complete release download helper so activation has the signed packs it requires.

**Downloading** (or **Setting Up** if everything is found locally). "Keep CivicCast Installer open. If a download is interrupted, use Resume download." Each row shows a state:

| Row state | Text |
|---|---|
| Not started | "Waiting" / "Not started yet" |
| Running | "Downloading", then "N MB/s — T left" or "Measuring speed" |
| No new byte for 10 s | "Stalled — retrying" |
| Checking | "Verifying" |
| Done, already on disk | "Found locally — verified yes" |
| Done, downloaded | "Verified yes — checked against its signature" |
| You stopped it | "Stopped. X of Y is already downloaded and kept." with **Resume download** |

The buttons are **Stop downloading** ("CivicCast stopped downloading. Nothing already downloaded was lost."), **Resume download** or **Retry download** (per row) and **Open installer log**. Closing the window ends the program at once and keeps partial files.

> **Known issue (beta.11):** "Stalled — retrying" appears after 10 seconds without a new byte; the label alone does not show that a retry is in progress. If a row fails, choose **Resume download** or **Retry download** as shown. **Open installer log** opens the newest setup-progress or runtime-host log; it does not show the acquisition failure details stored in `%USERPROFILE%\.civiccast\installer-state.json`.

When a row fails the window shows one of these lines:

| Line shown | Cause | Do |
|---|---|---|
| "The connection dropped. Nothing is damaged." | Network failed | **Resume download** |
| "The downloaded file didn't match its signature and was discarded." | Hash mismatch | **Retry download** |
| "The download server didn't have this file. This is our problem, not yours." | Source not found | Retry will not fix it. Report it through the project's issue page |
| "The paused download couldn't pick up where it left off, so this file will start over. This is our problem, not yours." | Resume not possible | **Retry download** |
| "This drive doesn't have enough free space to finish this download. Free up some space, then choose Retry." | Disk full | Free space, **Retry download** |
| "Windows wouldn't let CivicCast save this file. Security software blocking the CivicCast folder is the usual cause; allow CivicCast in it, or start CivicCast with 'Run as administrator' once, then choose Retry." | Permission denied | Allow the folder in your security software, **Retry download** |
| "This file couldn't be saved to disk. The download folder may be unavailable or read-only. Check that the drive is connected and writable, then choose Retry." | Write failed | Fix the drive, **Retry download** |

| Line shown | Cause | Do |
|---|---|---|
| "This download stopped and CivicCast did not get a reason it can explain. Nothing on this computer is damaged. Choose Retry; if it stops again, use Open installer log and send that log to support." | Unknown | **Retry download** |

Two alert bars may also appear: "No files have started downloading yet. CivicCast is still waiting for the first byte..." after 30 seconds with nothing moving, and "CivicCast could not start downloading its components. Nothing is being downloaded right now..."

**The status panel.** When all rows are done (or on every later launch) the window shows "CivicCast Installer", a **Ready** or **Not ready** pill and the button **Open operator console**. The status line "CivicCast's native background service is running." comes from one check of `http://127.0.0.1:8000/health`. Under **More options** are **Repair this step**, **Download AI models and captions** (re-runs the download screens; safe on a healthy station) and **Show uninstall instructions**. The link **Report a beta issue** opens a GitHub new-issue page, which needs a GitHub account; do not paste passwords, recovery codes or private meeting material into it.

> **Known issue (beta.11): Ready** means only that the web server answered `/health`; it can show while optional Large Whisper and CUDA downloads are absent. **Show uninstall instructions** displays instructions but does not uninstall the station. Create the first admin in the operator console after installation, as described in the next step.

> **Tip:** The station does not depend on this window. Setup already started the service. If the first-run window is stuck on a download you do not need, close it and use the **CivicCast Operator Console** shortcut on the Desktop.

**Step 6: First setup in the operator console** Open the **CivicCast Operator Console** shortcut (or **Open operator console**) in a browser on the station computer. The address is `http://127.0.0.1:8000/operator/`. Use `/setup` on that computer to create or recover the first administrator. Ordinary staff use the separate `/login` page; when the staff hostname is configured, they can sign in from an approved staff network. See the network access guide and staff accounts guide. A remote-control tool remains suitable for first setup because its browser runs on the station.

1. Read the header: "First setup. Create the station identity, first local admin, and recovery kit before a public meeting."
2. If the **Durable storage** card shows a **Prepare storage** button, select it. Until storage is ready the page shows "Create the first admin after storage is ready".
3. Fill in the form: **Station name**, **Admin display name**, **Admin username**, **Admin password** (12

characters or more; a counter shows "(n/12)"), **Confirm admin password**, **Where will you keep the recovery kit?** (free text; nothing is written there) and, optionally, **Resident portal URL**.

4. Select **Create first admin**.
5. The **Recovery kit ready** panel appears. Select **Print kit** or **Save kit**. **Save kit** downloads `civiccast-recovery-kit-<kit id>.txt`, which contains the eight one-time recovery codes **and the administrator password in plain text**.
6. Tick "I have saved or printed this kit…" (it unlocks after you use **Print kit** or **Save kit**), then select **Continue to the console**. Navigation stays locked until the kit is confirmed, and the browser warns you if you leave first.

You should see "Setup complete" or "Signed in", the **First-run defaults** card, and the setup tools: **Camera or test media**, **Backup destination**, **Storage and viewing estimate** and **Provider setup**. Later sign-ins use **Admin sign-in** on the same page and land on the Readiness screen.

A lost password is recovered with **Use recovery code**: the first click arms it ("This permanently consumes one recovery code — only 8 exist for this station. Click Recover account again to confirm.").

> **Warning:** The saved recovery kit file contains the administrator password in plain text. Keep it off shared folders, cloud-synced folders and email. Print it, or store the file somewhere the city controls, and delete it from Downloads.

> **Known issue (beta.11):** The **First-run defaults** panel shows **Test mode**, the channel count, and whether sample content and the initial schedule are enabled. This page has no control that changes the mode or turns off those defaults. Before a public meeting, remove the sample video from **Assets** and the starter item from **Schedule** if you do not want them shown to residents; see Chapter 3 and Chapter 5. The saved mode label does not switch the station's playout behavior.

When first-run sample content is enabled, the bundled Welcome video gets a 20-second starter slot five minutes ahead. Use Schedule to approve it for air and start its channel before that time. If setup takes longer, schedule the same Welcome asset for a new future time; approving an expired item does not move its airtime. The corrected Beta 15 package played the sample in both a 20-second slot and a 60-second slot, then returned the channel to slate. An earlier Beta 15 candidate had repeated rollover attempts in the 60-second slot; the Beta 15 verification record distinguishes that initial observation from the corrected package result.

#### Silent install

`setup.exe /S` runs without dialogs. Add `/D=<folder>` to choose an install folder; keep that option last and unquoted. Run it from an elevated prompt, from the kit folder:

```
$p = Start-Process .\setup.exe -ArgumentList '/S' -Wait -PassThru
$p.ExitCode
```

A silent install shows no dialog and opens no window. A failure leaves only the exit code and the log. Exit 0 means success. After it finishes, run the checks in "Verify the install" and open the console from the shortcut.

#### Exit codes

When a step fails, the dialog shows text but not the number, and a silent run shows only the number. The number is also the last line setup writes to the log: "postinstall: FAILED, aborting with exit code N". Run `Get-Content C:\ProgramData\CivicCast\install-progress.log -Tail 20` to see it.

> **Known issue (beta.11):** Interactive failure dialogs explain the step but do not show its numeric exit code. Read `install-progress.log` for the code; the first-run window's **Report a beta issue** link opens the project's issue tracker and requires a GitHub account.

### Setup exit codes

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 0 | Any | Success | Verify (below) |
| 2 | Any | NSIS's own generic abort, with no specific step code | Read the log |
| VC++ code (for example 1602 or 1603) | C++ runtime | The bundled runtime would not install, and setup returns the runtime installer's own code. Code 1638 ("another version is already installed") is accepted only when the registry confirms a runtime is present; otherwise it fails the same way | Read the log. Run setup again. Code 3010 is not a failure: setup marks a restart as pending and continues |
| 82 | Uninstall | The teardown step (stop and remove the service, remove the firewall rule, clear registry state) did not finish, most often because the service stop was not confirmed. The uninstall aborts before anything is removed | Stop the service (services.msc, or `sc stop CivicCastSupervisor`) or restart Windows, then uninstall again |

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 110 | Stage packs | A required pack is missing or untrusted (inner code 74), or the optional GPU pack is present but untrusted (75) | Put the missing `.ccpack` files in the `packs` folder beside `setup.exe` and run again. The log line "step stage-packs: child reported:" names them. Setup safely prepares a partial install first. Nothing in `C:\ProgramData\CivicCast` was deleted |
| 111 | Re-check | The extracted server tools do not match their signed pack | Replace the kit copy (disk error or interrupted copy) and run again |
| 112 | Re-check | The extracted CivicCast program does not match its signed pack | Same |
| 121 | Re-check | The extracted ffmpeg tools do not match | Same |
| 122 | Re-check | The extracted AI engine does not match | Same |
| 113 | Upgrade engine | The upgrade failed **and** its own rollback failed | Setup then restores the pre-setup application and station state when automatic recovery succeeds. Read `upgrade\UPGRADE-RECOVERY.md` and `install-progress.log`; if setup reports exit 136, follow the recovery instructions below |

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 114 | Upgrade engine | This release has a database change that cannot be rolled back, so automatic upgrade was refused | The dialog says “Use the manual upgrade path with operator acknowledgement”. No such path is described anywhere in setup. Do not retry blindly; open an issue |
| 115 | Upgrade engine | Unexpected fault | Read the log and `upgrade\upgrade-engine.log` |
| 116 | Database | PostgreSQL could not be provisioned | Read the log and `provision\PROVISION-RECOVERY.md` |
| 117 | Database | Unexpected fault | Read the log |
| 118 | Service | The service could not be registered | Read the log |
| 119 | Firewall | The firewall rule could not be created | Read the log. Setup requires this rule |
| 120 | Pre-install | An existing service could not be stopped safely, its state could not be read, or the service exists but the old program is missing | Stop **CivicCast Native Supervisor** in Services, retry as administrator. If the old program folder was removed, repair or remove that install **without deleting** `C:\ProgramData\CivicCast` |
| 123 | Activation | See the table below | See below |
| 124 | Upgrade engine | The engine rolled back its database work; the installer then restores the pre-setup application and station state when automatic recovery succeeds | Read `upgrade\upgrade-engine.log` and `install-progress.log`. Fix the reported cause and run setup again; the previous service state is restored with the prior installation |

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 125 | Service | The service started but the web server is not answering | Read `install-progress.log`, `logs\control_plane.log`, `logs\control_plane-app.log` and `upgrade\upgrade-engine.log`, fix, run setup again |
| 126 | Service | Windows registered the service but could not start it | Read `C:\ProgramData\CivicCast\logs` and the Windows Application event log (Event Viewer, Windows Logs, Application) |
| 127 | Database | Setup cannot tell which CivicCast runtime owns the machine | If there is no older WSL edition, an administrator sets `HKLM\SOFTWARE\CivicCast\ActiveRuntime` to `native` and runs setup again. The exact command is in `provision\OWNERSHIP-RECOVERY.md` |
| 128 | Upgrade engine | An earlier failed upgrade's record is still on disk. Setup restores the pre-setup application and station state when automatic recovery succeeds | Move (do not delete) `C:\ProgramData\CivicCast\upgrade\upgrade-journal.json` somewhere safe, then run setup again |

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 129 | Version preflight | The setup version is older than the installed version. Setup stops before prerequisites, service changes, application-file replacement, or station-data changes | Keep using the installed version. If it needs repair, use the complete signed setup kit matching that installed version |
| 129 | Upgrade engine (defensive late refusal) | A newer version was detected after application files had been replaced and the database downgrade was refused | Setup automatically restores the pre-setup files and station state when recovery succeeds. Check `install-progress.log`; if restoration did not complete, keep the recovery records and follow exit 136 |
| 130 | Version preflight | Setup could not safely compare the installed version or identity; it stopped before changing the installation | Follow the setup message; retry from an administrator account or contact support with the message |
| 130 | Uninstall | You declined the ownership-transfer question (this code is phase-specific; see the setup log) | None; nothing was removed |
| 131 | Uninstall | The ownership transfer failed | Read the detail in the dialog |
| 132 | Uninstall | Blocked (active runtime plus an older WSL edition, or state unreadable) | Follow the dialog |
| 133 | Uninstall | Service stop unconfirmed during final cleanup; program files kept | Stop the service or restart Windows, then uninstall again |

| Code | Step | Meaning | What to do |
|---|---|---|---|
| 134 | Uninstall | Teardown incomplete, or a second uninstaller started while this one ran and cleared its plan to hand back the runtime-ownership setting (`ActiveRuntime`). The teardown case is a second line of defence: the uninstaller normally stops earlier, with 82 | Read the dialog and `install-progress.log`. Remove any leftover service, firewall rule and registry state by hand (Services, Windows Defender Firewall, `HKLM\Software\CivicCast\Native`) |
| 135 | Database | Another CivicCast product (the older WSL edition) is registered | Uninstall it from Settings, Apps (or run its `cutover-to-native`), then run setup again |
| 136 | Installer recovery | Setup could not prepare, complete or resume automatic restoration | Keep `C:\ProgramData\CivicCast` and `install-transaction` intact. Read `install-progress.log` and `install-transaction\recovery-error.txt`; retry setup or restart Windows so the recovery task can retry, then contact support if it still fails |

**Automatic setup recovery** Before replacing application files, setup creates a durable, access-restricted recovery record under `C:\ProgramData\CivicCast\install-transaction` and registers recovery to run after an interruption or restart. Once the existing service is stopped, setup makes a temporary copy of only the paths it can change: the application folder except `packs\.station-cache`, the local PostgreSQL cluster, `provision` and `upgrade` state, relevant registry and firewall state, the Windows service configuration and CivicCast shortcuts. Recordings and uploads are not copied because setup does not modify them. Setup calculates the copy size first

and stops before replacement unless that amount plus a 256 MB reserve is free.

If a later setup step fails, setup restores the previous application files, database and saved station state. It restores the prior service registration and start mode; if the service was running, setup starts it and checks that the saved version and database schema are healthy. A successful commit or restore removes the temporary copy. If automatic restoration is interrupted, the recovery task or the next setup run retries it. Exit 136 means recovery could not be prepared or completed; keep the station data and recovery records in place and use the log paths above. This transaction is a short-term setup safeguard, not a replacement for a separate station backup.

**Activation (exit 123)** Exit 123 covers every activation failure. The inner code in the log line "step d4-activate-station: returned N" tells them apart.

| Inner code | Meaning | What to do |
|---|---|---|
| (no index) | No `station-index.json` beside setup or embedded | Obtain `station-models.json` and all matching archive parts from the release page, then run `scripts\download_windows_release_artifacts.ps1 -Install` to assemble the kit before retrying |
| 66 | The index or a pack could not be obtained or verified | Read the missing component or verification reason. Use the complete matching signed station kit; raw model downloads do not replace activation packs |
| 67 | Any error after the packs verified: no free space, extraction failure, a missing file, or a self-test failure | Read the details list for the exact line. Free space if that is the cause |
| 78 | The embedded signing key was refused | This copy of setup is not a valid release build. Download it again from the release page |
| 64, 65 | The setup program is defective | Download it again |
| other | Unexpected | Read the log |

**Inner codes in the log** The log records each step's own code ("step X: returned N"). You may see: 64 arguments, 65 report could not be rendered, 66 acquisition, 67 activation or self-test, 68 install-tree verify, 70 service registration, 72 database-URL write, 73 uninstall preflight, 74 ownership transfer or required-pack failure, 75 provisioning or optional-pack failure, 76 repair made changes, 77 uninstall

blocked, 78 embedded trust key refused, 79 unrepairable, 80 teardown step failed, 82 service stop unconfirmed, 83 service would not start, 84 service running but not serving, 85 ownership unprovable, 86 unknown `--civiccast-` option, 87 another CivicCast product registered. Codes 69, 71 and 81 are not named in the files we read.

#### Logs

| Question | File (under `C:\ProgramData\CivicCast`) |
|---|---|
| What step did setup reach? | `install-progress.log` (time-stamped; read from the bottom and find the last "begin" with no "returned") |
| Which pack was missing? | The "step stage-packs: child reported:" line, and `install-manifest-report-<pid>-<time>.json` |
| Why an upgrade rolled back | `upgrade\upgrade-engine.log`, `upgrade\upgrade-journal.json`, `upgrade\UPGRADE-RECOVERY.md` |
| Database creation or ownership | `provision\PROVISION-RECOVERY.md`, `provision\OWNERSHIP-RECOVERY.md`, `provision\ownership-observation.txt` |
| Why the station will not run | `logs\supervisor.log` (10 MiB, 10 older files, flushed each record), `logs\control_plane.log`, `logs\control_plane-app.log`, `logs\postgres.log`, `logs\ollama.log` |
| First-run window | `%USERPROFILE%\.civiccast\runtime-host.log` and `installer-state.json` |
| Which self-test failed | The setup window's details list (not the log; see the Known issue above) |

Setup keeps its recovery journal and temporary copy under `C:\ProgramData\CivicCast\install-transaction` until the install commits or restoration completes. If automatic restoration cannot finish, keep that folder and all station data intact. Open the logs in Notepad, and when you report a problem to the project's issue page, remove anything private first. Do not paste passwords, recovery codes or tokens.

#### Verify the install

Run these on the station, then open the console. Each check says what a good answer looks like.

```
Get-Service CivicCastSupervisor | Format-List Name, DisplayName, Status,
 ↪  StartType
Invoke-RestMethod http://127.0.0.1:8000/health
Get-NetFirewallRule -DisplayName "CivicCast (Native) Portal/API (TCP 8000)"
Test-Path "C:\Program Files\CivicCast (Native)\station-set.json","C:\Program
 ↪  Files\CivicCast (Native)\activation-self-test.json"
Get-Content -LiteralPath 'C:\ProgramData\CivicCast\install-progress.log' -Tail 5
```

- The service is **CivicCast Native Supervisor**, `Status` **Running**, `StartType` **Automatic**.
- `/health` answers `status: healthy`, `schema: current` and the installed `version`. The HTTP status is always 200 while the process answers. A `degraded` body means database readiness or live-caption processing needs attention; inspect `schema` and `live_captions`, then authenticated System Health for details.
- The firewall rule exists.
- Both `station-set.json` and `activation-self-test.json` exist in the install folder.
- The last log lines include `postinstall: SUCCESS` and record the installed version.
- Settings, Apps shows **CivicCast (Native)** and the installed version.
- The operator console and the resident portal at `http://127.0.0.1:8000/` both load on the station.

Then sign in and open **Readiness** (the page headed "Safe to broadcast") and run a private rehearsal (Chapter 4, Chapter 8). Restart Windows once and confirm that the service starts automatically. This is an operational check for your station.

#### Upgrade

To upgrade, run the newer `setup.exe` over the existing installation. Use the release helper to assemble the runtime `packs\` and signed `station\` folders. Make a separate backup first (Chapter 12); the installer's upgrade backup is not your only recovery copy.

> **Warning:** Setup stops the CivicCast service before it replaces any file. That takes every channel off the air until setup finishes and the service starts again. Do not run an upgrade during a meeting or a scheduled program.

1. Close the first-run window if it is open.
2. Use the release helper with `-Install`, or run the new complete kit's `setup.exe /S` from an administrator PowerShell session. Approve UAC when requested and wait for setup's exit code. The native installer skips Tauri's generic **Already Installed** page and does not call the old uninstaller first.
3. Wait. Setup records the prior service and station state, stops the old service, makes its temporary recovery copy, replaces and verifies files, then runs the upgrade engine, which backs up, migrates and health-checks the database.
4. Confirm with "Verify the install".

Run setup directly over the existing installation. The generic reinstall page could invoke the old uninstaller before setup had a chance to save recovery state, so the corrected native installer suppresses that page and uses its own recoverable in-place transaction. It also clears Tauri's saved install-location marker on uninstall, so a later clean reinstall is not misidentified as an already-installed product. Take a separate backup first; the temporary recovery copy is limited to the setup paths described above and is removed after commit or restoration.

| Upgrade engine result | What setup does |
|---|---|
| Committed | Continues |
| Fresh install (nothing to upgrade) | Continues. Existing CivicCast data on the machine is adopted, not deleted |
| Same version already installed | Does nothing to the data; re-checks the packs; continues |
| Older setup over newer install, caught by the early version preflight | Exit 129 before prerequisites, service changes, application-file replacement, or station-data changes |
| Defensive D3 downgrade refusal after payload replacement | Exit 129. The installer restores the pre-setup application and station state when automatic restoration succeeds |
| Rolled back its own work | Exit 124. The installer restores the pre-setup application, database and service state when automatic restoration succeeds |
| Rollback also failed | Exit 113. Setup attempts to restore the pre-setup station; see `UPGRADE-RECOVERY.md` and `install-progress.log` |
| Release has an irreversible change | Exit 114 (refused) |
| Earlier failed upgrade's record found | Exit 128. Setup restores the pre-setup application and station state when automatic recovery succeeds; move the stale journal aside, then retry |

> **Historical Beta 13/Beta 14 behavior:** those installers could discover a downgrade only in the D3 upgrade engine, after replacing application files; their exit-129 wording could incorrectly say that setup stopped before changing anything. Beta 15 adds an earlier version check, so the normal downgrade refusal happens before prerequisites, service changes, application files, or station data are touched. The corrected Beta 15 source retains a defensive late D3 refusal and wraps replacement in automatic restoration of the prior application and station state. Package-specific observations remain in the Beta 15 verification record.

Exit 128 is a separate D3 stale-journal refusal. The installer restores the pre-setup state when automatic recovery succeeds; then move the stale `upgrade-journal.json` aside as instructed and retry. If setup reports exit 136, do not remove the recovery records or station data.

#### Repair

There is no separate repair button that does the work. Two paths exist.

**Re-run setup.** Run the same version's setup again with its `packs\` folder. The native installer skips the generic **Already Installed** page; setup stops the service, records a temporary recovery copy, re-verifies each extracted pack and rebuilds a damaged tree from the pack. The upgrade engine does nothing for the same version.

**The repair command.** For an operator working outside the setup window, the installed program has a repair mode. It checks every installed tree against its signed pack, re-lays corrupted files, repairs the runtime selector and re-registers the service. It never touches data. From an elevated PowerShell window, with the kit folder as `--installer-dir` (the folder that contains `packs\`):

```
& "C:\Program Files\CivicCast (Native)\CivicCast Native.exe" --civiccast-repair
 ↪  "C:\Program Files\CivicCast (Native)" --installer-dir "D:\CivicCast-kit" |
 ↪  Out-String
```

It prints a JSON report. The exit code is 0 if every tree already verified, 76 if something was repaired and now verifies, and 79 if something could not be repaired locally (the report names the remedy). Piping to `Out-String` makes PowerShell wait for this Windows-subsystem program. If repair stopped the service to rebuild a tree, it does **not** restart it: it prints the line to run, `sc.exe start CivicCastSupervisor`. Its "everything verified" answer does not check the database schema or the activation files. Use the verification record for package-specific test results.

> **Known issue (beta.11): Repair this step** runs the station bootstrap for runtime, FFmpeg, storage, service and dashboard lanes. For other failed or blocked lanes it only writes "CivicCast queued a repair pass for this installer lane"; it does not run a lane-specific repair. Use the listed retry step or contact the project through **Report a beta issue**.

#### Uninstall

Open Settings, Apps, Installed apps, **CivicCast (Native)**, **Uninstall** (or run `C:\Program Files\CivicCast (Native)\uninstall.exe`). The quiet form is the registry value `QuietUninstallString`: `"<install folder>\uninstall.exe" /S _?=<install folder>`.

1. **Ownership check.** The details list shows "Checking CivicCast Native runtime ownership before uninstall…". If CivicCast (Native) is the active runtime and the older WSL edition ("CivicCast Installer") is still installed, you are asked whether to transfer ownership to the WSL edition. **No** stops with exit 130 and removes nothing. **Yes** transfers ownership and continues (a failed transfer is exit 131). Any other blocked state is exit 132.
2. **Teardown.** "Removing the CivicCast (Native) supervisor service, firewall rule, and registry state…" The service is stopped and removed, the firewall rule is deleted, and the registry values (including the database password) are cleared. If the teardown does not finish with success (most often because the service stop cannot be confirmed), the uninstall aborts with exit 82 and **nothing is removed**: "stop it manually (services.msc, or 'sc stop CivicCastSupervisor'), or reboot this machine, then run Uninstall again."
3. **The model-pack notice.** "CivicCast (Native) is being removed, including the downloaded AI model packs (about 21 GB) kept in this folder. Your recordings, database and settings in `C:\ProgramData\CivicCast` are NOT affected and are being kept. If you reinstall later by running setup.exe on its own, it will need those model packs again and cannot download them – reinstall from the full CivicCast kit folder (setup.exe together with its station folder), or copy `<install dir>\packs\.station-cache` somewhere safe now if you want to reuse it." A silent uninstall writes the same text to the log.
4. **Removal.** The runtime, packs and then the whole install folder are deleted. If the uninstaller

cannot delete itself it schedules the removal for the next restart. The shortcuts are removed.

If the service stop was not confirmed, you also see exit 133 and the program files are kept. Exit 134 means teardown was incomplete and you remove the leftovers by hand.

| | Removed | Kept |
|---|---|---|
| Service and firewall | Service `CivicCastSupervisor`, rule "CivicCast (Native) Portal/API (TCP 8000)" | |
| Registry | `HKLM\SOFTWARE\CivicCast\Native` values (`DatabaseUrl`, `InstalledVersion`), the maintenance flag, the setup install-location key, and (when CivicCast Native was the only active runtime) the `ActiveRuntime` selector under `HKLM\SOFTWARE\CivicCast` | |
| Install folder | Everything in `C:\Program Files\CivicCast (Native)`: runtime, tools, the AI engine, `components`, `models`, `packs` and the **model-pack cache** `packs\.station-cache` | |
| Shortcuts | The Desktop and Start menu shortcuts | |
| Data | | All of `C:\ProgramData\CivicCast`: the database (`data\pgdata`), uploads and recordings (`data\uploads`), the egress working folder and conform cache (`data\egress`), packs and components the first-run window downloaded, logs, journals and recovery documents |

| | Removed | Kept |
|---|---|---|
| Per-user settings | | `%USERPROFILE%\.civiccast`. A checkbox on the uninstall page, "Also delete this account's saved installer settings (your recordings, database and settings in `C:\ProgramData\CivicCast` always stay on this computer).", clears the setup window's own saved settings (the folders `org.civiccast.native` under %APPDATA% and %LOCALAPPDATA%) |

> **Known issue (beta.11): the model-cache limitation.** Uninstall deletes `<install folder>\packs`, including the station-pack cache. If you reinstall without the signed station resources, activation can fail after staging the runtime packs. Use `scripts\download_windows_release_artifacts.ps1 -Install` to download and assemble the release's signed resources before reinstalling; from `setup.exe` alone setup may stop earlier at exit 110.

Because the database password is deleted from the registry, the surviving database cannot be reconnected with the old password. By design, a later install finds the existing database, issues a fresh password on it and keeps your data (the code calls this "adopt existing"). Check the verification record for package-specific uninstall and reinstall results.

> **Warning:** No uninstall step removes `C:\ProgramData\CivicCast`. If you want a clean computer, you must delete that folder yourself after you have backed up anything you need. Doing so permanently destroys the database, recordings and settings. The setup text gives no instructions for this.

#### If it did not work

| What you see | Cause | What to do |
|---|---|---|
| Setup stops with exit 110 after a page that promised downloads | No `packs` folder beside `setup.exe` | Run the release helper to assemble the kit, then confirm the runtime packs are under its `packs\` subfolder |
| Setup stops with exit 123 after a long wait | No `station` folder and no cached model packs, or a self-test or space failure | See the activation table and read the details list |

| What you see | Cause | What to do |
|---|---|---|
| The console shortcut shows a browser error | The service is not running | Press Windows key + R, type `services.msc`, press Enter, find **CivicCast Native Supervisor** and choose **Start**. Then read `logs\supervisor.log` and `logs\control_plane.log` |
| "First setup can only be done from the station computer itself" | The browser opened `/setup` away from the station | Use `/setup` on the station computer for first-admin actions. Ordinary staff use `/login`; remote staff sign-in requires the configured staff hostname and an approved network. |
| "The station is cooling down after too many requests." | Too many sign-in attempts | Wait, then **Try again** |
| "Could not read setup state." | The console reached the station but the station could not answer | Wait and reload. Check the service is running. Collect `install-progress.log` and do not run setup again until someone has read it |
| A required station pack is missing | Model acquisition in the first-run window cannot provide signed station packs | Check the error detail; confirm `station-models.json` and every archive part are available. Run `scripts\download_windows_release_artifacts.ps1 -Install` to verify and assemble the kit, then retry setup |
| The station will not stay up after a failed upgrade | Containment left the service on manual start | Fix the cause, run setup again |

#### Package status and verification

This chapter describes the unpublished Beta 20 candidate edition. Install only a package on the releases page, and follow its bundled Help and matching verification record. The Beta 18 record tracks the earlier package's checks. A historical Beta 15 field report found the installed `CivicCast Native.exe` unsigned while the outer setup was signed; that observation does not establish another package's status.

#### Related

- Chapter 9: Planning your station

- Chapter 11: Configuring the station
- Chapter 12: Running it day to day
- Chapter 13: Security and privacy
- Chapter 14: Troubleshooting matrix
- Appendix A: command-line reference

## Configuring the station

This chapter is for the person at the city who owns the CivicCast computer. It walks through every place a station is configured: First Setup and the admin account, the Station Profile, channels and their outputs, loudness, the playout engine, captions, AI models, cable commissioning, the Control Room, custom fields, the paywall, certificates and network exposure, storage, and the environment variables that sit behind all of it. Most of it is done once, before the first public meeting, by someone with the `setup_admin` role. A few things (the environment variables, the registry) need Administrator rights on the Windows computer itself.

### Before you start

You need:

- The station computer, installed and running (see Installing, first run, upgrading, uninstalling). The Windows service is `CivicCastSupervisor` ("CivicCast Native Supervisor").
- A browser on the station computer for first setup, first-admin sign-in and recovery. Staff accounts can sign in remotely only through the configured staff hostname and from an approved network; see the network access guide and staff accounts guide.
- Administrator rights on Windows for the registry and file steps in the environment-variable section.
- A plan for what the station does. A public-meetings station (residents watch on the web) needs far less configuration than a station that feeds a cable headend. Cable Commissioning and the headend presets are only needed for the second kind.

Most screens in this chapter live in the sidebar group **Setup**. Almost all of them can be used only by the `setup_admin` role. The first admin you create has every role (the token scope `admin` expands to all five roles), so on a one-person station you can ignore role names. Roles are described in the appendix on roles (Roles and what each can do).

Scheduling forms use the time zone in **Station Profile**; reports, exports and some dated controls retain their stated UTC basis. Check the label beside each date or time field before saving.

### Where settings live

Settings are kept in six different places. Knowing which place holds a setting tells you how to change it, how to back it up, and why a change in one place does not move anything in another.

| Where | What is kept there | How you change it |
|---|---|---|
| The database (PostgreSQL, `C:\ProgramData\CivicCast\data\pgdata`) | Each channel's output configuration, the AI model choices, custom fields, the paywall, control-room devices and cues, watch folders, retention rules, schedules, assets | The console screens in this chapter (or the staff API) |

| Where | What is kept there | How you change it |
|---|---|---|
| `station-state.json` and the files beside it | The Station Profile, first-admin and staff accounts and session hashes, recovery-kit state, commissioning progress, network settings, provider credentials (`provider-credentials.json`), proof references (`provider-proof-evidence.json`), `tester-ops-state.json` | Setup, Staff accounts, Network Settings and Station Profile screens |
| `C:\ProgramData\CivicCast\config\station-backups.json` | Backup destination, selected media roots, schedule, retention and last-run status | Station backup and restore screen; unattended passphrase is stored separately in the OS-protected secret vault |
| The service's environment (the registry value `Environment` on the service) | Tuning and switches that have no screen: timeouts, worker on/off switches, cache sizes, proxy and CORS rules, the control-room sidecar address, TSDuck location | You, by hand, in the registry (see Environment variables) |
| The Windows credential store ("keyring") | AI provider API keys, control-room device passwords | The AI Models and Control Room Setup screens, or `civiccast model set-provider-key` |
| Fixed or selected by the installer and network settings | The folder layout under `C:\Program Files\CivicCast (Native)` and `C:\ProgramData\CivicCast`, a loopback-only control-plane address (default `127.0.0.1:8000`), the Ollama address `127.0.0.1:11434` | Choose `127.0.0.1` or `::1` in Network Settings; the application cannot bind directly to a LAN or public address |

The service's `station-state.json` and related profile files live under its configured service profile. The installer sets that profile to `C:\ProgramData\CivicCast\profile`; when migrating a default LocalSystem installation, it copies the old profile and imports CivicCast Credential Manager entries before changing service identity. See the restricted service identity guide for migration details.

> **Backup:** Include the service profile and its state files in the station backup. Current native installations use `C:\ProgramData\CivicCast\profile`; older Beta 11 LocalSystem installations could use a profile outside `ProgramData`. Verify the configured path when recovering an older installation. See Running it day to day.

The folder layout is described in the installing chapter. The two folders that matter for configuration are `C:\ProgramData\CivicCast\data\uploads` (uploaded media and recordings, in a `recordings` sub-folder) and `C:\ProgramData\CivicCast\data\egress` (the playout work area: prepared copies of programs, slates, the live web-preview segments, the conform cache).

> **Known issue (beta.11):** The two folders above cannot be moved with a setting. The service sets `CIVICCAST_UPLOAD_DIR` and `CIVICCAST_EGRESS_WORK_DIR` for the control plane itself, derived from the `PROGRAMDATA` folder, and those values win over anything you put in the service environment. Plan disk space on the drive that holds `C:\ProgramData` (see Planning your station). The Station Profile has three editable storage folders; they do not move anything either (see Station Profile).

#### Run First Setup and create the admin account

First Setup is at `/setup` in the local operator console (`http://127.0.0.1:8000/operator/`). It creates the first administrator and one-time recovery kit; first-admin sign-in and recovery remain local-only. Staff accounts use the separate `/login` page and can use the configured staff hostname from approved networks. See the staff accounts guide.

1. On the station computer, open the console and choose **First Setup** in the sidebar.
2. Under **Durable storage**, click **Prepare storage**. This creates or activates the local database for meeting records, captions, summaries and subscriptions. If the service was started with a DATABASE_URL the button does nothing and just reports status.
3. Fill in **Station name** (up to 120 characters), **Admin display name** (up to 120), **Admin username** (up to 80), **Admin password** (at least 12 characters; a counter shows "(n/12)") and **Confirm admin password**.
4. In **Where will you keep the recovery kit?** type a note such as "printed, in the clerk's safe". This is a free-text reminder. It is not a folder and nothing is written there.
5. Optionally fill in **Resident portal URL**. A blank box is saved as nothing.
6. Click **Create first admin**. The **Recovery kit ready** panel appears with eight one-time recovery codes. The rest of the console is locked until you confirm the kit.
7. Click **Print kit** or **Save kit**, tick **I have saved or printed this kit...** and click **Continue to the console**.

After step 7 you see a green **Setup complete** card, the **First-run defaults** card and the setup tools (camera or test media, backup destination, storage estimate, provider setup).

What the form does not show you, but the server chooses for you when you click **Create first admin**:

| Setting | Value chosen | What it means |
|---|---|---|
| Number of channels | 3 | The channels `public`, `education` and `government` |
| Default channel | `government` | Used when the first-run sample content is added |
| Time zone | `local` | Until you set an IANA zone, `local` means UTC in the running service (see Station Profile) |
| Operation mode | `test` | Shown as "Test mode" on the First-run defaults card |
| Sample content | on | A short sample video is added to the library in the background |
| Starter schedule | on | A starter schedule is created with the sample content |

> **Known issue (beta.11):** The **First-run defaults** card shows Test mode, sample content and the starter schedule, but First Setup has no control to turn those defaults off. Remove the sample video and starter schedule item from Assets and Schedule before a public meeting if you do not want them shown to residents (see Before the meeting).

> **Known issue (beta.11):** The **Save kit** button downloads `civiccast-recovery-kit-<kit id>.txt` and that file contains the admin password in plain text as well as the eight codes. Keep it off the CivicCast computer, off email and off any synced cloud folder. The kit you regenerate later (Station Profile, Security) contains the codes only.

**Sign in again, and when the password is lost** At `/setup`, the station computer shows **Admin sign-in** and **Use recovery code** for the first administrator. A correct password signs in that browser and opens Readiness (`/health`) or the page you were returning to; it does not sign out other browsers. Ordinary staff use `/login`; remote sign-in is available only through the configured staff hostname from an approved network.

Too many wrong passwords are limited. The default is 10 failures per 60 seconds for each route and each connecting address (`CIVICCAST_AUTH_RATE_LIMIT`, `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS`). A correct password always gets through; a wrong one burns the budget.

To use a recovery code, click **Recover account**, enter one code and a new password (at least 12 characters), and click **Recover account** a second time. The second click is the real action.

> **Warning:** Using a recovery code permanently consumes it. If the password and all eight codes are lost, an elevated local administrator can use the offline `civiccast admin reset-`

> `password` procedure in Security and privacy. Do not reset first setup to recover a password. While signed in, you can also regenerate the recovery kit.

To make First Setup run again on a station that is already set up, the service must have the environment variable `CIVICCAST_ALLOW_FIRST_ADMIN_RESET=1`. Without it, a second First Setup is refused with "First-admin setup is already complete. Use the recovery flow or set CIVICCAST_ALLOW_FIRST_ADMIN_RESET=1 for an intentional local reset." Treat this variable as a station reset. We did not trace everything the reset replaces, so do not use it on a station with real records without help from support.

**The rest of the Setup page** Once signed in, the Setup page has four tools:

- **Camera or test media** has six choices (USB webcam or HDMI capture, phone or tablet broadcast app, hardware encoder or AV system, NDI source, bundled sample video, upload a short test video). **Save meeting source** creates a live source on the channel `government` (the channel is fixed in the screen). **Create sample media** makes a 2-second sample video and adds it to Assets. The stream address box is for an address such as `rtmp://...`; do not put camera passwords in it.
- **Backup destination** with **Verify backup**. See the warning below.
- **Storage and viewing estimate** is arithmetic in your browser: hours per meeting times meetings per month times 2 gives gigabytes stored. Nothing is measured.
- **Provider setup** holds the cloud and archive providers. See Provider setup and keys.

> **Note:** The Setup card's **Verify backup** checks whether CivicCast can write, read and delete a small test file in the selected folder; it does not create or schedule an encrypted station archive. Use **Station backup and restore** to create, verify and schedule `.ccbackup` archives, and review the station backup guide before relying on them. The destination check runs under the service account, so use a local path or share that account can write to; a drive letter mapped only in your own session is unavailable to the service.

#### Set the Station Profile

The Station Profile shows the station's identity and what this computer actually has, and holds two security actions. Only `setup_admin` can save; the roles `meeting_operator` and `support_admin` can read it.

| Field | What it changes | What it does not change |
|---|---|---|
| **Station name** (required, up to 120 characters) | The name shown in the console and on the community-board graphic that appears between programs when no channel branding is set | It is not the public site's title; channel and app names are set on Channels |

| Field | What it changes | What it does not change |
|---|---|---|
| **Timezone (IANA name, or "local")** | The zone used to enter, recur, preview and display schedules and auto-schedule dayparts, and to set the end-of-day deadline for retention terms | It does not change stored UTC timestamps in reports or exports |
| **Default channel id** | The channel the first-run sample content uses; unknown ids are healed to `government` when read | It does not change which channel residents see first |
| **Public base URL (optional)** | Stored and displayed | See the Known issue below: nothing reads it, and it cannot be cleared here |
| **Media library / Recordings / Backups** folders | The three paths stored in the profile; each Copy path button copies one | See the Known issue below: they do not move or choose where any file is written |
| **Show live captions on air** | The operator switch for live captions (the live speech-to-text tap on every on-air channel and the CEA-708 caption leg in the video). Turning it off stops recognition and drains queued audio on the next tap-worker scan; turning it on resumes recognition on the next scan. The caption routing in the video changes when each channel next starts | It does not affect captions on published recordings (those are the offline job, see Captions). New station profiles default to off |

The time zone is free text. Choose an IANA name such as `America/Denver`. The default `local`, a blank value, or an unknown name falls back to UTC; an unknown name is logged as a warning. Set the station's real zone here before creating schedules. A zone change does not rewrite already-generated occurrences; check the future calendar after saving.

> **Known issue (beta.11):** The three storage-folder fields are shown with a Save button, but the saved paths do not choose where media, recordings or backups are written. Editing them does not move any file. Use Assets to find recordings. The environment variables `CIVICCAST_STATION_MEDIA_LIBRARY`, `CIVICCAST_STATION_RECORDINGS` and `CIVICCAST_STATION_BACKUPS` override the displayed values; they do not move files either.

> **Known issue (beta.11):** The Station Profile's **Public base URL** field is stored but the program uses the service variable `CIVICCAST_PUBLIC_BASE_URL` to build RSS and federation addresses. Leaving the field blank does not clear a value already saved. The resident-portal preview link uses `CIVICCAST_RESIDENT_PORTAL_URL`.

The **Station box profile** card is read-only. It asks the program what this computer has: CPU, RAM, the recommended tier, the playout engine and its readiness, the clock (a row is yellow with "System clock is not NTP-synced." if the clock is not synchronized), DeckLink or SDI, TSDuck, the backup destination, AI model memory, and a "Cable-grade OS" line. Its readiness colors are shown as the raw words GREEN, YELLOW and RED. The card always asks for the `public-meetings` profile; there is no selector.

#### Security actions

- **Sign out other sessions** ends every other station-local console session except this browser, after a confirmation. It requires a signed-in station session with the `setup_admin` role; database and environment API tokens cannot use it. The **Staff accounts** screen also lets a setup admin review and revoke individual sessions.
- **Regenerate recovery kit** creates eight new codes and invalidates the old ones at once. The new codes exist only on that page. Save or print them before you leave the page, tick the confirmation and click **Done**. If you leave early the codes are lost, and the old ones no longer work.

**Recorded captions** The **Recorded captions** card on Station Profile controls captions for saved recordings separately from **Show live captions on air**. A setup admin can edit these settings; meeting operators and support admins can read them.

1. Choose **Enable recorded captions**. Turning this off stops new caption jobs from being queued and pauses existing work until processing is enabled again.
2. Keep English selected and choose whether to include **Spanish**, translated from reviewed English. Clear Spanish for English-only operation.
3. When Spanish is selected, choose **Wait for every selected language** to attach both reviewed tracks together, or **Attach English after its review** to make reviewed English available while Spanish translation and review continue. The job remains active until the selected Spanish work finishes.
4. Click **Save recorded-caption settings**. The card reports save failures; do not assume an unsaved change has reached the worker.

Only reviewed text is attached. These choices do not change live-caption performance thresholds or claim that an uncaptioned recording is captioned. Defaults are enabled processing, English and Spanish, and waiting for every selected language.

#### Configure channels, outputs and sinks

**How a channel is configured** A channel has an **egress configuration**, stored in the database. "Egress" means the signal CivicCast sends out. The configuration holds:

- **enabled**: whether the channel may be started.
- **auto_start** ("Keep this channel on air"): whether the automation starts the feed after a restart or crash.
- **allow_software_fallback** ("Allow software (CPU) encoding fallback"): see Encoders.
- **fill_policy**: what fills the gaps between programs, `slate` (the plain slate) or `bulletins` (approved community bulletins).
- **slate message** (1 to 240 characters), optional **NDI output name** and **SDI output device**.
- **sinks**: one or more output targets (below).
- **loudness target and tolerance** and the **canonical profile** (picture size, frame rate, codec, bitrates): see Loudness.
- the **lower-third banner** switch and text (up to 240 characters).

A new channel has no configuration until you create one. The Start button is disabled with "No outgoing-feed configuration for <id>. Apply a headend preset or the local rehearsal preset first." The normal way to create one is to apply a preset (next section). Creating it by preset gives the channel `enabled: true` and the slate message "CivicCast is preparing the channel."

The Channels screen (**Run Meeting** group) edits some of this: **Run this channel 24/7** (auto start, software fallback, fill policy, slate message, NDI name, SDI device) and **Cable headend delivery** (the presets). It does not edit individual sinks, loudness numbers or the picture profile. Those can be set only through the staff API (`PUT /api/staff/egress/channels/{id}/config` with a full configuration, `setup_admin` only; see Appendix: API).

> **Warning:** Changes to a channel's outputs do not reach a channel that is on air. A running pipeline reads its sinks, encode profile and loudness target only when it is built. A preset or a saved configuration lands at the next Stop and Start. **Apply headend preset** on a channel that is standing by on its slate restarts it at once, and every output (the cable feed included) drops for a few seconds while the pipeline rebuilds. On a channel that is airing a program nothing is interrupted, and the screen says "Preset saved. Restart the channel to put it on air."

### Apply a headend preset

1. Open **Channels** and select the channel.
2. In **Cable headend delivery**, choose a **Headend preset**.
3. Type the **Destination** (`udp://address:port` for network presets; a drop-folder path for the file preset; optional folder for the web preview).
4. Optionally set **Mux rate override (kbps)**. Tick **Keep the channel's other outputs alongside the headend feed** if the channel already has another output you want to keep.
5. Click **Apply headend preset** (or **Enable web preview**) and confirm the dialog.

You should see "Preset applied and going on air", "Preset saved. Restart the channel to put it on air", "Nothing to change" or "Preset saved for the next start", with a sentence of detail.

The presets in the program (each carries its sources and the note "Built from published vendor documentation; not field-proven against a real cable headend until the first-station beta."):

| Preset (screen label) | Picture | Video / audio codec requested | Mux rate | Transport |
|---|---|---|---|---|
| Generic CBR SPTS over UDP | 1280x720, 30 fps | H.264 at 5,000 kbps, GOP 30 / AC-3 192 kbps | 8,000 kbps | UDP unicast |
| Comcast MTD - SD | 720x480, 30 fps | MPEG-2 at 3,180 kbps, GOP 15 / AC-3 192 kbps | 3,750 kbps | UDP multicast |
| Comcast MTD - HD | 1920x1080, 30 fps | H.264 at 10,000 kbps, GOP 30 / AC-3 384 kbps | 12,000 kbps (a placeholder; use your carriage agreement's rate) | UDP multicast |
| TelVue HyperCaster - IP transport stream input | 1280x720, 30 fps | H.264 at 5,000 kbps, GOP 30 / AC-3 192 kbps | 8,000 kbps; ports 1024 and up | UDP unicast |
| Harmonic Spectrum - transport stream ingest | 1920x1080, 30 fps | H.264 at 8,000 kbps, GOP 30 / AC-3 192 kbps | 10,000 kbps | UDP unicast |
| Leightronix UltraNEXUS - file handoff | 1280x720, 30 fps | H.264 at 8,000 kbps, GOP 30 / AC-3 192 kbps | none (file drop) | A file in a watched folder |
| Local rehearsal (web preview, HLS) | the channel's own profile is left alone | not applied | none | A local folder served to the resident portal |

Applying a cable preset replaces the channel's whole canonical profile and sets the channel loudness target to -24 (see Loudness). Applying the web-preview preset adds only an `hls` output and leaves the encode profile and loudness alone.

> **Known issue (beta.11):** The native GStreamer engine maps MPEG-2 to its H.264 fallback and AC-3 to its AAC fallback. The Comcast MTD - SD preset therefore does not produce MPEG-2, and the cable presets do not produce AC-3. This is current source behavior; no real cable-headend output was verified for this release. Ask your cable operator whether the codecs actually emitted are acceptable, and complete Cable Commissioning before relying on a preset.

**Sink kinds** A **sink** is one output of a channel. The native station uses the GStreamer engine and supports the sink kinds below. Output destinations and any required credentials must be configured for the receiving service.

| Kind | URI form | What it is for | Notes from the code |
|---|---|---|---|
| `udp-ts` | `udp://host:port` (a port is required) | A headend or encoder that takes a transport stream over UDP | The cable preset creates this. Datagrams are 1316 bytes (7 transport packets). A constant `-muxrate` is added as an allowed extra argument. Passes through the TS relay when TSDuck is present (below) |
| `local-ts` | `udp://...` or `file://...` | A local transport-stream feed for another program on the same computer | |
| `file` | A path or `file://` URI | Writing a transport stream to a file | The file preset creates this |
| `srt` | `srt://...` | An SRT receiver | See the secret note below |
| `hls` | A local folder path or `file://` | The resident-portal web preview: a playlist and 2-second segments written to the folder | Delivered by a supervised ffmpeg helper because the shipped GStreamer has no HLS element. The folder must be inside the HLS root (below) |
| `sdi` | Not a network address | SDI output through a supervised relay using your own DeckLink-capable ffmpeg | Needs `CIVIC-CAST_SDI_FFMPEG` |
| `rtmp` | `rtmp://` or `rtmps://` | FLV through `rtmp2sink` | Configure the destination and stream-key handle; the worker resolves the secret when opening the output |

Each sink also has: a **label** (1 to 80 characters, unique within the channel), **latency** (0 to 60,000 ms, default 2,000), **extra output arguments** (only `-mpegts_flags`, `-muxrate`, `-pkt_size`,

`-flush_packets` and `-max_delay` are allowed; shell characters and extra inputs are rejected), a **loudness regime** and optional target and tolerance, and an **EAS tone strip** switch (default on; it strips the 853/960 Hz attention tone on internet outputs and leaves a cable feed untouched).

Secrets never go in the URI. A URI that carries a passphrase, key, token or password is refused at save time. For an SRT passphrase the sink carries a `secret_ref`. The native automation resolves that reference by reading the **environment variable of that name** from the service's environment. So an SRT sink with `secret_ref: SRT_PASSPHRASE_GOV` needs `SRT_PASSPHRASE_GOV=<passphrase>` in the service environment (see Environment variables).

The **HLS root.** The public web preview is served without sign-in from whatever folder the channel's `hls` sink names, so the folder is confined. By default it must be under the egress work folder (`C:\ProgramData\CivicCast\data\egress\live-hls\<channel>` when you leave the destination blank). `CIVICCAST_LIVE_HLS_ROOT` names a different root; every web-preview folder must then be under it. A drive root, a Windows system folder or a network share name is refused.

**The transport-stream relay** For each `udp-ts` sink, CivicCast can run a **TS relay**: a long-lived TSDuck `tsp` process between the encoder and the headend. The encoder sends to a local port (`127.0.0.1`, starting at 17800); the relay fixes the continuity counters and the PCR clock and forwards to your real destination from one pinned source port (the local port plus 1000). The point is that a plan change or encoder relaunch does not look to the headend like a new stream. The relay touches only `udp-ts` sinks.

The switch is `CIVICCAST_TS_RELAY`: `auto` (default; relay when `tsp` is found, otherwise pass straight through with a warning), `on` (log an error when `tsp` is missing, still pass through) or `off`. Any other value is an error. `CIVICCAST_TS_RELAY_BASE_PORT` moves the first local port. A separate helper feeds the `hls` sink and uses ports from 18000 (`CIVICCAST_HLS_RELAY_BASE_PORT`).

TSDuck is found in this order: `CIVICCAST_TSDUCK_PATH` (a folder or the file), the CivicCast-managed copy (`CIVICCAST_TSDUCK_HOME`, else `%LOCALAPPDATA%\CivicCast\tsduck` of the service account), the copy that the server-binaries pack ships (`tsp.exe` under `packs\native-server-binaries\payload\tsduck\bin`; the Gate A run found it there), then the PATH. The **Enable cable verification** button on Channels downloads TSDuck on demand, and the download has no confirmation step. We did not test the button on a station whose pack copy is present; by the lookup order it should not be needed there.

> **Known issue (beta.11):** The **Channels** screen's text for the TSDuck download says "Turn this on and CivicCast downloads the free TSDuck toolkit". The control is a download button with no confirmation and it needs an internet connection. Press it only on a station that is allowed to download software, and not during a meeting. The source does not state the download size.

**Encoders: hardware and software** The picture is encoded by a GStreamer encoder named from the profile's codec. The default `h264` (and `libx264`, `openh264`) maps to **openh264enc**, a software (CPU) encoder bundled with CivicCast. Other mapped names are `x264enc`, `x265enc` and, for hardware, `nvh264enc`/`nvh265enc` (NVIDIA) and, on Windows, `mfh264enc`/`mfh265enc` (Media Founda-

tion, which stands in for the VAAPI names).

On Windows CivicCast runs a pre-flight before each start:

- A software encoder is used as is. Nothing is checked.
- A hardware encoder that is present is used. HEVC works through Media Foundation.
- A hardware encoder that is **absent**: for H.264, if **Allow software (CPU) encoding fallback** is ticked the channel encodes on the CPU with `openh264enc` and logs "No hardware video encoder was found; 'allow_software_fallback' is on, so this channel is encoding on the CPU (openh264enc). This is slower and may not keep up with live on a weak CPU." If it is not ticked, the start is refused with "No hardware video encoder was found on this machine. To broadcast on the CPU instead (slower), turn on 'Allow software (CPU) encoding fallback' in this channel's settings, then start the channel again. HEVC/H.265 needs hardware and is not available this way."
- HEVC without hardware is always refused ("There is no software HEVC in this build").

`openh264enc` runs as four slices on four threads so that high-motion pictures stay faster than real time.

> **Note:** The software-fallback checkbox only matters when the channel's configured codec names a hardware encoder that this computer does not have. With the default profile and the presets above, the channel already uses the software `openh264enc` and the checkbox changes nothing. It is off by default, so a configuration that asks for hardware fails loudly instead of silently dropping to the CPU.

Hardware **decoding** is off by default and the CPU decoders are preferred. `CIVICCAST_GST_ALLOW_HARDWARE_DECODE=1` (values `1`, `true`, `yes`, `on`) turns hardware decoders back on for the playout worker.

If the GStreamer files on the computer are corrupt and cannot be repaired in place when the service starts, the service falls back to the older FFmpeg concat engine so the channel keeps airing, raises an operator alert and marks the channel degraded. You do not choose this; it is automatic.

**Loudness and the leveling ride** Each channel has one **loudness target** (default -16 LUFS) and **tolerance** (default 2 LU). Each sink can name its own **regime**:

| Regime | Target | Standard label |
|---|---|---|
| `streaming` | -16 | Streaming -16 LUFS (ITU-R BS.1770) |
| `atsc-a85` | -24 | ATSC A/85 -24 LKFS (CALM Act) |
| `ebu-r128` | -23 | EBU R128 -23 LUFS |
| `inherit` (default) | the channel's target | |

An explicit target on a sink wins over its regime. A sink whose target differs from the channel's gets its own re-encode of the audio; a sink equal to the channel's copies the program audio. The Channels

screen's **Loudness** card shows this plan read-only. It is visible to `setup_admin` and `support_admin` only. To set a different target or regime you apply a preset (cable presets set the channel to -24 and the headend sink to `atsc-a85`; the web preview sets that sink to `streaming`) or use the API.

The level of each program is set when CivicCast prepares ("conforms") it, before it airs. A public meeting is not steady: the chair, a speaker who steps back and a public-comment microphone can sit 15 to 20 LU apart inside one recording. A single average gain cannot land every stretch at the target. CivicCast therefore runs a slow, speech-following gain **ride** instead of a one-pass normalizer. Its numbers are fixed in the program, not settings: it follows speech over a 45-second window, moves the gain at most 0.5 dB per second, may lower a passage by 6 dB and raise it by 18 dB, caps the lift in a quiet gap at 8 dB, and limits the true peak at -1.5 dBTP (it then checks the finished file and re-encodes once with more headroom if the peak is too hot). The acceptance check it is built against is every 240-second window within 0.9 LU of the target and the whole program within 0.5 LU.

> **Known issue (beta.11):** The limiter in the loudness ride does not guarantee a cap on how much quiet passages are raised; a historical recording measured a quiet gap lifted by about 13.7 dB. Treat that figure as a past measurement, not a beta.11 measurement. If the ride cannot run (for example numpy is missing, or ffmpeg fails), CivicCast logs one warning and uses a single-pass `loudnorm` conform to the same target instead; the channel airs and only the leveling is missing.

**The playout engine: what you can and cannot set** On a native station these are fixed by the service and a value you put in the service environment is ignored: the engine (`CIVICCAST_EGRESS_ENGINE` is set to `gstreamer`), caption embedding (`CIVICCAST_EGRESS_EMBED_CAPTIONS=1`), the work and upload folders, and the portal folders. See Environment variables for what the service forces.

What you can tune (all optional, all in the service environment; defaults are what the code uses when the variable is not set):

- **Channel automation.** `CIVICCAST_CHANNEL_AUTOMATION` is `inline` (default) or `off`. It checks every channel every `CIVICCAST_CHANNEL_AUTOMATION_POLL_SECONDS` (default 2.0). With **Keep this channel on air** ticked, a channel with no running process gets a start command, retried every 30 seconds until it is live.
- **Preparation.** Each program is prepared in segments of at most `CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS` (default 1,800 s, 30 minutes). One preparation step may run at most `CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS` (default 300 s). The cache of whole prepared programs is limited by `CIVICCAST_CONFORM_CACHE_GB` (default 60) and the folder of prepared plans in use by `CIVICCAST_PREPARED_PLAN_DIR_BUDGET_GB` (default 5.0).
- **Starting and reloading.** `CIVICCAST_EGRESS_SEAMLESS_RELOAD` is on unless set to `0`, `false`, `no` or `off`; turning it off makes the encoder restart at every plan boundary (for diagnosis only). The worker waits for the picture to start `CIVICCAST_GST_PREROLL_TIMEOUT_S` seconds (default 30, clamped to 5-45) and for the first output `CIVICCAST_GST_FIRST_OUTPUT_TIMEOUT_S` (default 45, clamped to 10-120). A running worker that makes no output for `CIVICCAST_STALL_TIMEOUT_S` (default 10) quits so the service restarts it. A reload waits

`CIVICCAST_RELOAD_TIMEOUT_S` (default 10) for the new source and `CIVICCAST_RELOAD_COMMIT_TIMEOUT_S` (default 15; clamped 3-120 inside the engine) for the switch.

- **NDI and SDI.** CivicCast's bundled ffmpeg cannot contain the NDI muxer (a licensing limit). To publish a channel as NDI, set `CIVICCAST_NDI_FFMPEG` to your own NDI-capable ffmpeg and put a name in **NDI output name**. For SDI set `CIVICCAST_SDI_FFMPEG` to a DeckLink-capable ffmpeg and a device in **SDI output device**. `CIVICCAST_NDI_RELAY` and `CIVICCAST_SDI_RELAY` accept `inline` (default) or `off`. Without the variable, the console's readiness check says to set it.

> **Operator stops:** Stop and Finish current item, then stop suppress automatic starts for the current service session. An explicit Start clears that suppression. Disable **Keep this channel on air** if the channel must also stay off after a service or computer restart. Enabling that setting can start an eligible dark channel without another Start command.

**Repeat the published schedule** `CIVICCAST_SCHEDULE_LOOP=1` repeats the channel's published premiere sequence. The default is off, so a finite schedule ends after its last item. The loop preserves intentional gaps; make adjacent start times and durations contiguous for continuous programming. Set this variable through the service environment procedure below and restart between programs. Auto-schedule uses the station time zone, with `CIVICCAST_STATION_TZ` as the environment fallback. See the two-recording recipe in Before the meeting.

#### Configure captions

CivicCast captions in two different ways and they are configured separately. Native live captions use Whistle on the CPU by default; recorded-media captions use the separate Whisper workflow.

| | Live captions | Offline captions on recordings |
|---|---|---|
| What it is | A real-time speech-to-text tap on each on-air channel's audio, with the text embedded in the video as CEA-708 captions | A job that captions a published recording after the meeting |
| Switch | **Show live captions on air** (Station Profile); the tap rechecks it on each scan and video routing changes at the next channel start | **Enable recorded captions** in Station Profile; disabling pauses caption work and stops new jobs being queued |
| Where results go | Into the stream as captions; live history is bounded and ordinary live operation does not create permanent per-cue review records or evidence WAVs | To the review queue; attach after all selected languages are reviewed, or attach reviewed English first if that completion option is selected |

| | Live captions | Offline captions on recordings |
|---|---|---|
| Engine | Whistle CPU primary by default, with Whisper fallback; service environment can select Whisper directly | Whisper speech recognition and the translation model |
| Needs | CPU for Whistle and video playout; no CUDA GPU is required | Whisper; a translation model is needed only when Spanish is selected |

**The caption tiers** The speech model comes in two **tiers**:

- **Medium** (`faster-whisper-medium`) is the required Whisper floor for fallback and recorded captions. It is staged by the signed station pack and runs on the CPU.
- **Large-v3** (`faster-whisper-large-v3`) is optional. The first-run wizard offers it as a checkbox and may preselect it when an NVIDIA card with at least 8 GB of video memory is found. CUDA runtime files are optional and accelerate Whisper only; they are not required by Whistle.

At service start the station validates the staged Whisper tier and self-test receipt. The **saved selection** in AI Models controls the verified Whisper model used when a runtime is created. An activation default cannot silently override a different saved choice. The native live default remains Whistle regardless of whether Large is installed; Whistle's assets are resolved independently of the Whisper model directory.

The live caption runtime is created when the service starts. **AI Models** shows its service-start fallback tier separately from the saved selection. If you change the selection, restart CivicCast during a maintenance window to apply it to the live runtime. New recorded-caption jobs use the saved selection when they start; an already-running job keeps its existing model.

The device is chosen the same way. `CIVICCAST_WHISPER_DEVICE` in the **service** environment, if set, wins in either direction (`cuda` or `cpu`), with `CIVICCAST_WHISPER_COMPUTE_TYPE` (default `float16` for cuda, `int8` for cpu). Otherwise the station uses the GPU only when an NVIDIA adapter with at least 8 GB is found **and** both CUDA runtime libraries are staged; anything else is CPU with `int8`.

> **Note:** The AI Models caption selection concerns Whisper tiers. It does not change the native live engine from Whistle. The installer's optional-component choices control which optional models are downloaded.

**Live captions: what the settings change** **Live runtime:** Native stations use Whistle on CPU as the primary engine, with Whisper as fallback. Whistle inference is serialized by one station-wide lock across channels. The first recognition is published immediately, including low-confidence text; overlap is deduplicated without requiring two readings to agree. Whistle requests have a 30-second deadline. On failure, Whisper supplies backup captions while a fresh Whistle process is tried after an

initial 30-second cooldown. Further failed recovery attempts back off to 60, 120, 240 and at most 300 seconds. Only one recovery probe runs at a time, and successful recognition restores that channel to Whistle automatically. Whisper fallback requests allow 120 seconds by default; set `CIVICCAST_WHISPER_FALLBACK_TIMEOUT_SECONDS` to a finite value from 1 to 600 seconds to change that deadline. A failed Whisper child is closed and replayed once immediately; a repeated replacement failure starts a 30-second retry cooldown. `CIVICCAST_LIVE_CAPTION_ENGINE=whisper` selects Whisper as the primary engine instead.

Live captions are produced by the caption tap. The native station turns it on (`CIVICCAST_CAPTION_TAP=inline`) unless the service environment says `CIVICCAST_CAPTION_TAP=off`, in which case the tap does not run at all. The console switch is the safe-direction override: `off` in the environment forces live captions off whatever the profile says, but no environment value can turn them on against a profile that is off. When you turn the profile switch off, queued audio drains and speech recognition stops on the next tap-worker scan; caption routing is removed from the video when each channel next starts. When you turn it on, recognition resumes on the next scan, and each channel needs to start again before its video includes captions.

For the live tap, audio is collected in 5-second segments (`CIVICCAST_CAPTION_TAP_SEGMENT_SECONDS`, default 5.0), scanned every 2 seconds (`CIVICCAST_CAPTION_TAP_POLL_SECONDS`) and kept in a bounded queue. If recognition falls behind, old working audio can be shed so playout retains priority. Whisper-specific CPU thread and worker settings control its runtime; they do not override Whistle's station-wide inference lock. The playout worker has priority over caption work.

A separate proof loop decodes the emitted stream, compares the captions it finds with the expected ones, and only then marks a channel's captions "on" (a fresh pass is required; otherwise the Channels screen shows "Not verified"). It runs every 30 seconds with a 15-second limit.

> **Known issue (beta.11):** The poll and timeout of that proof loop are read from the **one-C** variable names `CIVICAST_CAPTION_PROOF_POLL_SECONDS` (default 30) and `CIVICAST_CAPTION_PROOF_TIMEOUT_SECONDS` (default 15). The two-C spellings `CIVICCAST_...` do nothing for these two. This is the reverse of the rule everywhere else (see the one-C spelling trap).

**Offline captions and Spanish** When recorded captions are enabled and the recording has a stored source file, publication queues a caption job. It transcribes in 30-second chunks (`CIVICCAST_OFFLINE_CAPTION_CHUNK_SECONDS`), polls every 60 seconds, retries after 300 seconds and permits 4 attempts by default. A reviewer approves the English cues. If Spanish is selected, the translation model creates Spanish cues for separate review. **Wait for all selected languages** attaches tracks after those reviews; **Attach English after its review** allows English to attach while Spanish work continues. The recording remains public while caption work proceeds.

> **Settings:** Use **Station Profile > Recorded captions** to disable recorded captions, select English only, or choose English-first completion. Legacy `CIVICCAST_OFFLINE_CAPTION_JOB=off` and `CIVICCAST_OFFLINE_CAPTION_SPANISH=0` are accepted defaults before profile settings are saved; they do not inherently prevent startup. Saved station settings take precedence.

> Spanish requires a working translation model only when selected.

**Caption evidence and retention** Live captioning uses `C:\ProgramData\CivicCast\data\caption-tap` as a temporary work area. Ordinary live operation does not create permanent review rows or evidence WAV clips for each live cue. Consumed audio chunks are deleted after processing; only the overlap needed for the next recognition is retained in memory. With Whistle, each channel retains at most 42 waiting completed five-second segments (210 seconds), plus in-flight inputs and the segment being written. The bound follows Whistle's finite processing grace. Other runtimes without that grace use a 12-segment working-file ceiling. This file ceiling is separate from the smaller backlog threshold used to detect sustained overload.

The live caption window is limited to the most recent 300 seconds and at most 512 cues. Delivery tracking follows that window instead of accumulating for the whole on-air session. Archive-wide caption review/evidence discovery is not a live-caption or broadcast-readiness prerequisite. These are implementation bounds, not controls on the configuration screen.

Original recordings, archived caption tracks and the recorded-caption review workflow are unchanged. Review recorded captions against the recording workflow; ordinary live captioning does not create an unattended review queue.

**Historical beta.10 behavior:** The older live path retained raw chunks for cleanup and created review evidence. Beta.11 ordinary live captioning does not create that permanent review/evidence archive or require archive discovery for readiness. This does not change original recordings or the recorded-caption review workflow.

#### Configure AI models

The AI Models screen (Setup group; `setup_admin` changes, `meeting_operator` reads) chooses the model behind three jobs. The default for each is a model that runs on this computer. The local AI engine is **Ollama 0.30.6**, started and watched by the CivicCast service on `127.0.0.1:11434`.

| Job | Default | Other choices |
|---|---|---|
| Whisper captions (live fallback and recorded media; optional direct live-primary mode) | `whisper-medium-faster` (needs 4 GB of memory) | `whisper-large-v3-faster` (8 GB) |
| Summary | `gemma4-12b-ollama` (16 GB) when the computer has a GPU and at least 16 GB of RAM; otherwise `gemma4-e4b-ollama` (8 GB) | `gemma4-31b-cloud` (Ollama Cloud), `gemini-2.5-flash-openrouter` (OpenRouter). Check the provider's current pricing and terms before enabling a hosted model. |
| Translation (English to Spanish) | `translategemma-4b-ollama` | `gemma4-31b-cloud` |

The rule behind the summary default is measured, not guessed. On a 32 GB computer with no

graphics card the 12B model took 366 seconds for one summary and then failed twice with memory errors, while the e4b model finished every attempt in 94 to 128 seconds. So 12B is only the default when a GPU is present. A summary is queued as a background job (poll every 15 s, retry after 120 s, 3 attempts; `CIVICCAST_SUMMARY_JOB`, `..._POLL_SECONDS`, `..._BACKOFF_SECONDS`, `..._MAX_ATTEMPTS`) so one slow generation does not time out a web request.

Before local summary generation, CivicCast requires the selected tier's catalog memory allowance plus 8 GiB of available RAM reserved for station work. If memory is insufficient, the job reports a recoverable deferral and explains whether to wait or select a smaller tier. CivicCast serializes its local Ollama generation requests and requests model unloading after generation, so an idle summary or translation model does not remain resident indefinitely.

How the Ollama child starts. The service starts Ollama only when the program `dependencies\ollama\ollama.exe` exists and a staged model store exists with a `manifests` folder (`models\ollama` first, then `packs\local-ai-model\models`, then `C:\ProgramData\CivicCast\packs\local-ai-model\models`, where the first-run window saves its downloads). If either is missing the service logs one line ("ollama child skipped (degraded AI, service healthy): …") and keeps running without AI. A skipped child is re-checked every 60 seconds, so models downloaded while the service is running are picked up without a restart. The child is started with `OLLAMA_NO_CLOUD=1`; hosted models are called by CivicCast directly over the internet, not through the local Ollama.

- **Choosing a local model** saves at once, with no confirmation. Options that need more memory than the computer has are disabled and marked "(exceeds this box)"; the screen does not show how much memory the computer has.
- **Choosing a hosted model** (Cloud or Frontier) only stages the choice. The page asks for the provider's API key, then for a tick acknowledging the provider's terms and costs, then **Apply cloud model**. Hosted models send meeting content to the provider; possible charges depend on the provider's current pricing and account terms. The server refuses a hosted choice without the consent tick.
- **Provider keys** go in the Windows credential store under the service name `civiccast.ai-provider` (handles `ollama-cloud-key` and `openrouter-key`). They are write-only: the screen never shows one again, and there is no button to remove one. To clear a key, run `civiccast model set-provider-key <ollama-cloud|openrouter> --clear`. To store a key without the console (for an offline station), run `civiccast model set-provider-key <provider>` with the key in the environment variable `CIVICCAST_PROVIDER_API_KEY` of that command's own shell (not the service). A hosted choice with no stored key shows "Hosted tier selected but no provider credential is stored — feature will defer until a key is saved."
- Hosted requests go to `https://ollama.com` and `https://openrouter.ai`. A station that blocks outbound traffic cannot use them.

For an air-gapped station, `civiccast model download --offline-bundle --bundle-dir <folder>` prints the manifest of model files with their SHA-256 values and `civiccast model import-offline` verifies a bundle against expected hashes (see Appendix: command line).

Recorded-caption translation uses the configured translation model to queue translated Spanish caption cues for review. This is a post-publication workflow; it does not translate the live English

caption stream. Review the translated cues before attaching or publishing the translated track.

The translation card reports saved Spanish review cues and their latest creation time. This persists across reloads; it describes the cues currently stored, rather than a lifetime total of translation-provider requests.

Model cards include identifiers such as `gemma4-e4b-ollama` and `whisper-medium-faster`. A saved caption-tier change applies to new recorded-caption jobs; an active job retains its original model. The live runtime keeps its service-start fallback tier until the service restarts. The screen reports that runtime separately from the saved selection and explains when a restart is needed.

#### Provider setup and keys

The Provider setup block on the Setup page lists twelve cards: Local resident portal, Backup destination, Internet Archive, YouTube, Subscriber notices, Local archive folder, Cloudflare R2, BunnyCDN, Fastly Object Storage, Akamai Object Storage, Podcast feed and Federation. Each shows a status (Ready, Check before meeting, Do not broadcast yet, Not set up yet, Needs IT help), a "Setup guide", and where it applies a credential form. Only `setup_admin` can save.

- **Save details** writes the values to `provider-credentials.json` beside `station-state.json`, as readable text in a file whose Windows access list is restricted. The screen says "CivicCast stores these locally and never shows secret values again." The Windows credential store is not used for these; it is used for AI provider keys and control-room device passwords. Protect the folder accordingly and keep it out of backups that leave the building unencrypted.
- **Test connection** exists for Cloudflare R2 and BunnyCDN only. The Fastly and Akamai setup text also says to run Test connection, but those cards have no such button.
- **Provision for me** on the Cloudflare R2 card takes a Cloudflare API token, verifies it, creates a bucket, enables a public domain, derives and saves the keys, and then health-checks. The token is not stored.
- **Record redacted proof** stores only the text you type (a file path, address or reference). The server rejects text containing `token=`, `secret=`, `password=` or `private_key=`. It does not run a live test.

The same providers can be configured with environment variables instead (for example `CIVICCAST_R2_ACCOUNT_ID`, `CIVICCAST_R2_ACCESS_KEY_ID`, `CIVICCAST_R2_SECRET_ACCESS_KEY`, `CIVICCAST_R2_BUCKET`, `CIVICCAST_R2_PUBLIC_BASE_URL`, `CIVICCAST_CDN_PROVIDER`, and the `CIVICCAST_BUNNY_*`, `CIVICCAST_FASTLY_*`, `CIVICCAST_AKAMAI_*`, `CIVICCAST_IA_*`, `CIVICCAST_YOUTUBE_*` and `CIVICCAST_SMTP_*` families listed in Appendix: settings). The CDN, archive and mail details are in Cable headend, streaming, CDN, federation, emergency alerts, the API.

For Internet Archive, local archive, YouTube, mail and subscriber webhooks, saving a credential in `provider-credentials.json` does **not** set the publishing adapter's selector or service environment. The adapter registry reads `CIVICCAST_PROVIDER_*` selectors and credentials from the running service environment. IT must set those using the service environment procedure and restart the service; use the provider behavior notes to see which Publish paths are actually available.

#### Run Cable Commissioning

Cable Commissioning is a four-step wizard for a station that sends a channel to a cable company's headend. Skip it for a station that only serves residents on the web. It is available to `setup_admin` (who can press things) and `support_admin` (who can only look). Progress is saved to `station-state.json` after each step, so you can leave and come back.

1. **Screen 8: First-run cable checks.** Optionally type the station name, then click **Run cable checks**. It runs 11 checks: operating system, disk space (a failure under 100 GB free), the GStreamer playout engine, the DeckLink/BMD Desktop Video SDK (a warning only), TSDuck (a warning if missing), the database (a failure unless durable storage is ready), the event bus, the backup destination (a warning), the time zone (a warning if it is UTC: "Set the station's real local timezone in Station Profile."), release integrity (always skipped) and the CasparCG helper (skipped or a warning). Only a **FAIL** blocks the next step; warnings do not. The screen always sends the `peg-cable` profile, so DeckLink wording appears on every station.
2. **Screen 9: Channel output setup.** Choose a channel, an output format (`720p30`, `1080i60`, `1080p30` the default, `SD480i60`), a headend profile, the destination `address:port`, an optional SDI device, a fill policy (Slate, Loop, Silence) and whether to check CEA-708 caption passthrough. Click **Save and continue**.
3. **Screen 10: Output proof.** Choose the test pattern ("Bars + tone" or "Slate"; "Live" behaves exactly like bars), a duration (1 to 1800 seconds, default 60) and click **Start proof run**, then confirm. It sends the test pattern to the channel's UDP output for that time while TSDuck checks the stream, then (if ticked) runs the caption check. The request holds the page until it finishes.
4. **Screen 11: Commissioning report.** Type the station name and click **Generate report**. The result is "Ready for broadcast" or "Commissioning incomplete" with the steps still needed. A proof with any blocker never reads ready. The same page can make a support bundle (a redacted diagnostic file).

The same flow runs from the command line: `civiccast cable doctor`, `civiccast cable commission --channel-id ... --headend-profile ... --destination ...`, `civiccast egress verify --channel-id ...` (see Appendix: command line).

> **Warning: Do not run the output proof while the channel is on air.** The proof pushes a test pattern to the channel's UDP destination for the whole duration. The dialog warns that it replaces the channel's real output; in the code it starts a second stream to the same address. We found no check that stops it on a channel that is airing.

> **Known issue (beta.11):** Screen 9 only saves your choices to the station record for the report. It does not configure the channel. Apply the headend preset on the Channels screen first. If the channel has no UDP output the proof fails with "channel '<id>' has no udp-ts sink to verify — apply a headend delivery profile first." The destination box is not tested for reachability. The cards are numbered Screen 8 to 11 although there are no Screens 1 to 7 on this page.

> **Known issue (beta.11):** The text under Screen 8 says warnings can be passed with "Continue-anyway". There is no such button; the step opens by itself when no check fails. The check and verdict words are shown raw (PASS, FAIL, WARNING, SKIPPED, partial), and the proof ends with an internal note about "rung 3" and "MASTER §13.2". It means the test checks the network signal only and is not a proof of SDI hardware.

#### Set up the Control Room

Control Room Setup (Setup group; `setup_admin` only) registers the production equipment a meeting is run with (switchers, deck recorders, PTZ cameras, relays), and builds **surfaces** (a named panel) of **cues** (one-press actions such as "take scene 2"). The operate screen, Control Room, uses what you configure here. Nothing here sends a command to a device except **Test connection**.

1. **Register a device.** Enter a label (up to 160 characters), a kind (OBS Studio, vMix, Blackmagic ATEM, HyperDeck, PTZ camera, OSC, TCP device, HTTP device, CasparCG, GPI (network relay), Serial (network relay)), the transport, a host and a port (1 to 65535), and a secret if the device has a password. The secret goes to the Windows credential store and is never shown again. Click **Register device**.
2. **Test connection.** The service asks the control helper to open a connection and records Reachable or Unreachable. A result older than 5 minutes shows "Stale — probe again". A device host must be `localhost`, a `.local` name or a private or link-local address; any other host is refused ("Device host must be localhost, .local, or a private/link-local IP unless a setup admin records a public-host override reason in the device profile.").
3. **Set timing.** In a device's profile set the **Take-delay (ms)** and **Post-roll (ms)** (0 to 600,000) and click **Save profile**.
4. **Create a surface** with a label and pick it in **Edit surface**. Surfaces are created for the role `meeting_operator`.
5. **Add cues** with a label, a device, an action (Take scene, Take input, Transition, Run macro, Play deck, Cue deck, Recall PTZ preset, Send OSC, Send HTTP, Push overlay, Clear overlay, GPI pulse, Serial send, Router take), a bank (0 to 99), a payload (JSON, pre-filled with a template) and **Confirm**. The server refuses an action the device kind does not support.

The readiness panel at the top lists the checks (control service, devices, profiles, enabled devices, device health, target safety, surfaces, cues, cue action safety, a safe-state cue, station-device evidence). A surface that has cues must have at least one cue with **Confirm** ticked; the program treats that cue as the safe-state cue, and readiness is blocked without it.

The helper behind this is the **TSR sidecar**, a separate small Node.js program bound to `127.0.0.1` (default port 7717). The service reaches it through `CIVICCAST_CONTROL_ROOM_TSR_URL` (for example `http://127.0.0.1:7717`). With the variable unset, every probe reports unreachable and every cue fails closed.

> **Known issue (beta.11):** We found nothing in the installer or the service that installs Node.js, ships the sidecar's dependencies or starts the sidecar. On a native station as installed,

> the **TSR control service** readiness card is blocked and cues cannot fire. To use the Control Room, IT has to install a compatible Node.js, install and start `civiccast\control_room\tsr_service(CIVICCAST_TSR_PORT=7717`, `npm install --ignore-scripts --omit=optional`, then `npm start`; the sidecar's own notes say its library targets Node 14, 16 or 18), and set the service variable. The button inside the blocked cards that says "Open Control Room Setup" points at the page you are already on, and the screen has no control that starts the helper.

> **Known issue (beta.11): Save profile** always sends empty options. Saving a profile replaces earlier settings, including a public-host override recorded elsewhere. The take-delay and post-roll boxes start at 0 and do not show the saved values. The headline can never read "Ready": equipment verification is fixed to "pending" in this build, so the best result is "Check before meeting". Nothing on the screen disables or enables a device, although a readiness hint talks about it.

The Control Room displays its On-Air session deadline and remaining time, restores the owner's open session after a refresh, and lets a Setup admin or Support admin force-close a locked session. A previous dry-run result is not restored; dry-run the cue again before firing it. See Running the meeting.

#### Define Custom Fields

Custom Fields (Setup group; `setup_admin`) are your own labels for programs, such as "Meeting type" or "Department". Each field becomes a box on every asset's detail page; if it is **Searchable** and **Exposed to public API** it also becomes a filter on the public Recordings page. This screen holds the definitions. The values are typed on each asset (roles `setup_admin`, `meeting_operator` and `records_clerk` can set them).

1. Open **Custom Fields**. In **Define a new field** enter a **Field key** (for example `meeting_type`; fixed after creation) and a **Label** (up to 200 characters).
2. Choose a **Type**: Text, Long text, List (pick one), Date, Number, Yes / no, Asset reference or Producer reference. For a List, type the choices one per line.
3. Set **Required**, **Searchable** and **Exposed to public API** (the last two are ticked by default) and an **Order** number.
4. Click **Create field**.

> **Known issue (beta.11):** A field shows as a public filter only when **both** Searchable and Exposed to public API are ticked, and only once a published recording has a value. Ticking only Searchable does nothing for residents.

> **Known issue (beta.11): Required** makes every asset without a value unsavable, including edits to its other fields, and the error uses internal ids ("Required field 'cf-x' ('Label') must be present."). Add a required field only when you are ready to fill it on every asset. Creating a field whose key makes the same internal id as an existing one overwrites that field's label, type, list and flags without a warning. The first **Delete** on a field with no values deletes it with no confirmation; on

> a field with values you get a red **Confirm delete (cascades values)** and the delete removes the saved values too. The up and down arrows swap Order numbers, so fields that all have Order 0 do not appear to move. The screen always creates fields for the station id `civiccast-station`, so do not set `CIVICCAST_STATION_ID`, or new fields will not appear in the list.

#### Configure the Paywall

The paywall (Setup group; `setup_admin`) is optional paid access to recordings through Stripe-hosted checkout, plus free "comp" passes. It is **off** by default; when off, every asset is public.

> **Beta 19 status:** Do not rely on the paywall for paid access. A configured mail provider sends magic links and reports delivery failure, but the public site's calls for tiers and checkout (`/api/public/paywall/tiers`, `/api/public/paywall/checkout`) have no matching server routes, and the public VOD media route does not enforce grants. The access check trusts the email address passed to it; the code does not prove the caller owns it.

> **Signing secret:** The server never returns the saved value, so the field stays blank after you open or reload this page. If a secret is saved, leave the field blank to keep it during an ordinary **Save**. To replace it, enter a new secret of at least 32 characters or generate one. Confirm the exact value currently in the field, then **Save**; editing the value clears confirmation, so confirm again. **Cancel** discards the pending replacement. To remove the saved secret, choose **Clear saved secret**, confirm, then **Save**. Clearing stops magic-link signing and Stripe webhook verification until you save a new one.

If you still want to look at it on a lab station:

1. Create the monthly or yearly prices in the Stripe website first. CivicCast never creates a price.
2. Tick **Enable paywall**. Provider **stripe** is the live one; **mock** is for tests and has no warning on a live station.
3. Click **Generate new secret** (32 random bytes, 44 characters, filled in the box), or paste your own of at least 32 characters. When replacing a saved secret, confirm the value currently in the box before **Save**; any edit requires a fresh confirmation. **Cancel** discards the pending replacement. Leave the field blank to keep the saved value when you save other settings. To remove it, choose **Clear saved secret**, confirm, and save. The Stripe webhook is `/api/webhooks/stripe`, and the code checks its `Stripe-Signature` header with this same secret. Stripe creates its own `whsec_` value for each endpoint, so whether Generate is the right value to give Stripe is not confirmed.
4. Add a tier for each price (**Tier ID** such as `basic`: lowercase letters, digits, `_` and `-`; **Display name**; **Stripe price id** starting `price_`; **Monthly** or **Yearly**) and click **Add tier**. Tier changes are local until you click **Save**.
5. Click **Save**.

**Comp access grants** issue a free pass to an email address for one asset, one series or everything, with an optional **Expires at** date (the end of that day, **in UTC**). The grant is created immediately on the

server, even if the paywall is not saved ON. The table lists only grants issued in this browser session; after a reload you cannot see or revoke earlier grants from this page. **Delete config** removes the saved configuration and turns gating off; we could not confirm whether it also removes grants and subscriptions.

#### TLS, certificates and network exposure

What the program does, as found in the code:

- The control plane, which serves the operator console, resident portal and API, stays on a loopback address (`127.0.0.1` by default; Network Settings also accepts `::1`) and speaks plain HTTP upstream. For remote HTTPS, use the supported Caddy configuration: a public hostname for resident pages and a separate, CIDR-restricted staff hostname. First setup and first-admin recovery stay local-only. See the network access guide.
- PostgreSQL listens on `127.0.0.1` (the first free of ports 5432, 5433, 5434, 5435 and 5544). Ollama listens on `127.0.0.1:11434`. The Control Room sidecar listens on `127.0.0.1`.
- The **local CA** is a small certificate authority for internal service-to-service TLS (mutual TLS). It holds two identities, `civiccast-api` and `civiccast-worker`. `civiccast cert rotate civiccast-api` creates the CA on first use (valid 3,650 days), then issues the named certificate (valid 90 days; "rotation due" starts when 30 days remain) with the key readable only by the account that ran the command. The folder is `CIVICCAST_CERT_ROOT`, default `.civiccast\certs` in the home folder of the account running the command. The only reader of these certificates is the CLI's `civiccast installer health-check`, which lists "Local CA mTLS" with the next step "Rotate internal service certificates every 90 days." We found no part of the station that serves or checks TLS with them.
- Behind a reverse proxy or CDN, set `CIVICCAST_TRUSTED_PROXY_CIDRS` (comma-separated CIDR list) so the real client address is used for rate limiting and audit. `CIVICCAST_TRUST_PRIVATE_PROXIES=true` trusts loopback and private-network hops too (default false; set it when a proxy such as nginx runs on the station itself). With `CIVICCAST_CDN_PROVIDER` set to `bunny` or `cloudflare`, the provider's published edge networks are added. With none of these set, forwarded headers are ignored.
- Browsers from another web origin are refused by default. `CIVICCAST_CORS_ALLOWED_ORIGINS` is a comma-separated list of exact origins (scheme, host, port); a wildcard is a startup error.

The installer-created TCP 8000 rule does not make the loopback-only application listener reachable from other computers. For supported remote access, deploy Caddy using the generated configuration; it terminates HTTPS and enforces the separate public/staff host policy. Do not expose port 8000 directly. See the network access guide.

The application itself does not terminate HTTPS; Caddy provides the supported HTTPS edge. The local CA remains for internal certificate operations and is not a browser certificate.

#### Storage and media lifecycle

Media Lifecycle Settings (Review Records group) has three cards. All three call the server separately, and the server's roles do not match the sidebar:

| Card | Roles that can use it (server) |
|---|---|
| Watch folders: list; add and remove | List: `meeting_operator`, `publish_operator`, `support_admin`. Add and remove: `publish_operator`, `setup_admin`. **Browse...** and **Scan now**: `setup_admin` only |
| Retention automation | List: `meeting_operator`, `publish_operator`, `support_admin`, `records_clerk`. Add, remove, **Apply rules now**: `records_clerk`, `setup_admin` |
| Storage budget | `meeting_operator`, `publish_operator`, `support_admin` |

The first admin has every role, so it sees all three work. A token with only `setup_admin` cannot load the watch-folder list or the budget; the card shows "This action requires one of these CivicCast roles: ...".

**Watch folders.** A watch folder is a folder on the station computer (a local disk, a USB drive or a network share) that CivicCast checks and imports video from. Type the path or use **Browse...**, which lists local drives only (network paths must be typed), then **Add watch folder**. A new folder is enabled, waits until a file has stopped growing for 10 seconds, and is checked about every 5 seconds (the worker's tick is 2.0 seconds, `CIVICCAST_WATCH_FOLDER_POLL_SECONDS`). Files are copied, never moved or deleted, into the uploads folder, and the imported asset takes the file name as its title. Whole drives (`C:\`), Windows system folders and administrative shares such as `\\host\C$` are refused. The service account must be able to read the folder.

**Retention rules.** A rule sets an asset's retention policy (`default`, `permanent`, `meeting`, `short`) from its exact **Meeting body** text. CivicCast never deletes an asset because of retention. A separate worker (`CIVICCAST_RETENTION_WORKER`, `inline` by default, every `CIVICCAST_RETENTION_POLL_SECONDS`, default 3600) only flags assets whose retention date has passed into the records clerk's disposition queue. Retention is also described in After the meeting.

> **Known issue (beta.11):** Rules do not run by themselves. The only thing that applies them is the **Apply rules now** button. A rule with a blank Meeting body is listed as "any" but matches nothing. Applying a rule changes the policy label; this step does not recalculate the retention deadline.

**Storage budget.** The card shows how much disk the library uses by retention tier. It shows "No budget configured (set CIVICCAST_MEDIA_STORAGE_BUDGET_BYTES)" until you set that variable to a number of bytes in the service environment. The uploaded file size limit is `CIVICCAST_UPLOAD_MAX_BYTES` (default 10 GiB, 10,737,418,240 bytes).

**Other storage workers**, all `inline` by default, each with an `off` switch (the switch is the variable named below): media lifecycle (`CIVICCAST_MEDIA_LIFECYCLE_WORKER`; every 300 seconds; ingest-time conversion is on and makes one `h264_720p_5mbps` proxy per validated asset, switch it

off with `CIVICCAST_MEDIA_LIFECYCLE_TRANSCODE_SEEDING_ENABLED=0`; a dry run that records what it would do is `CIVICCAST_MEDIA_LIFECYCLE_WORKER_DRY_RUN=1`; scheduled items with missing media are flagged 7 days ahead, `CIVICCAST_MISSING_MEDIA_HORIZON_DAYS`), media integrity (`CIVICCAST_MEDIA_INTEGRITY_WORKER`, every 3,600 seconds, flags a missing file) and the archive. For a local archive set `CIVICCAST_NAS_ARCHIVE_PATH` and `CIVICCAST_PROVIDER_LOCAL_NAS=real`. The archive path must be a reachable folder or the service logs why.

#### Environment variables

The current source inventory contains 443 environment variable names, including development and build settings. Almost all have a safe default. This section covers the ones a station's IT person will plausibly need, with the real default and what happens. Every variable below is read from the **service's** environment unless the table says it belongs to a command you run yourself.

**How to set one on the Windows service** The service's child programs (the control plane, the playout workers) get the service's environment, plus a short list of values the service forces. The supported way to give one variable to the service is the per-service `Environment` registry value, a list of `NAME=value` strings.

1. Open **PowerShell as Administrator** on the station computer.
2. Add the variable (this keeps the entries that are already there and replaces one of the same name):
```
$key = 'HKLM:\SYSTEM\CurrentControlSet\Services\CivicCastSupervisor'
$name = 'CIVICCAST_TS_RELAY'
$value = 'off'
$current = @()
try { $current = @((Get-ItemProperty -Path $key -Name Environment
 ↪  -ErrorAction Stop).Environment) } catch { }
$new = @($current | Where-Object { $_ -notlike "$name=*" }) + "$name=$value"
Set-ItemProperty -Path $key -Name Environment -Value $new -Type MultiString
```
3. Restart the service so a fresh process reads it: `Restart-Service CivicCastSupervisor`.
4. Read the value back: `(Get-ItemProperty $key).Environment`.

To remove a variable, run the same lines without the `+ "$name=$value"` part.

> **Warning:** Restarting the service stops PostgreSQL, the control plane and the Ollama engine (each is given up to 15 seconds to stop gracefully) and takes every channel off the air until the service is back and the channels have started again (channels with **Keep this channel on air** start themselves). Do it outside a meeting.

> **Warning:** Do not use `setx /M` or `[Environment]::SetEnvironmentVariable(..., 'Machine')`. The project's own lab found that a machine-wide variable set that way never reached the running service: a Windows service gets its environment when it is started, and

> Windows services are started by a process whose environment block is captured once at boot. The per-service registry value is read at that service's own start. This is how the project's lab scripts inject variables; the code comments in the station also name it as the place to set them.

> **Known issue (beta.11):** The native installer does not create this optional per-service `Environment` registry value. On a fresh install it may not exist; the steps above create it. No console screen lists the environment the station is running with, so confirm a change by its effect or its log line.

The commands you run yourself in a shell (such as `civiccast token issue` or `civiccast model set-provider-key`) read the **shell's** environment, not the service's. The program's launcher is `civiccast.exe` inside the station runtime (the build manifest places it under `runtime\Lib\site-packages\bin`; we did not confirm the path on an installed station). The CLI is described in Appendix: command line.

**What the service sets for you, and cannot be overridden** The service builds the control plane's environment from its own, then applies these values last, so they win over anything you set:

| Variable | Value the service sets |
|---|---|
| `CIVICCAST_EGRESS_WORK_DIR` | `%PROGRAMDATA%\CivicCast\data\egress` |
| `CIVICCAST_UPLOAD_DIR` | `%PROGRAMDATA%\CivicCast\data\uploads` |
| `CIVICCAST_SUPERVISED` | `1` |
| `CIVICCAST_EGRESS_ENGINE` | `gstreamer` (or `ffmpeg-concat` if the GStreamer files cannot be verified) |
| `CIVICCAST_EGRESS_EMBED_CAPTIONS` | `1` |
| `CIVICCAST_CAPTION_RUNTIME`, `CIVICCAST_CAPTION_TIER`, `CIVICCAST_WHISPER_MODEL_PATH` | `faster-whisper`, the tier chosen at start, the verified model folder |
| `CIVICCAST_CAPTION_TAP`, `CIVICCAST_CAPTION_TAP_DIR` | `inline` and the caption-tap folder, unless you set `CIVICCAST_CAPTION_TAP=off` |
| `CIVICCAST_WHISPER_DEVICE`, `CIVICCAST_WHISPER_COMPUTE_TYPE` | CUDA only with supported NVIDIA hardware (8 GB or more) and staged CUDA files; otherwise CPU/int8, unless explicitly set |
| `CIVICCAST_NATIVE_STATION`, `..._ROOT`, `..._MANIFEST`, `CIVICCAST_NATIVE_REPORTED_VERSION`, `CIVICCAST_LAN_ONLY_STATION` | The station identity and version; LAN-only mode keeps the API documentation pages off (they load fonts and scripts from the internet) |
| `CIVICCAST_OPERATOR_CONSOLE_DIST`, `CIVICCAST_PUBLIC_PORTAL_DIST` | The folders of the two web portals |

| Variable | Value the service sets |
|---|---|
| DATABASE_URL | Read from the registry value HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl, unless a non-blank DATABASE_URL is already in the service environment, which then wins (one log line says which source won, never the value) |

**The one-C spelling trap** Almost every CivicCast variable begins CIVICCAST_ (two C's). Sixteen names in the source begin CIVICAST_ (one C). They are typos in the source that stayed. For most of them nothing is wrong, because they are lab, test or internal. Two of them are real settings that you can only reach under the one-C spelling, and two more are legacy fallbacks:

- CIVICAST_CAPTION_PROOF_POLL_SECONDS and CIVICAST_CAPTION_PROOF_TIMEOUT_SECONDS are the **only** spellings the caption proof loop reads. Writing them with two C's does nothing.
- CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS and CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS used to be read only under the one-C names. Since beta.10 the two-C name is read first; the one-C name still works as a legacy fallback with a one-time deprecation warning, and if both are set and differ, the two-C value wins and a warning says so.

A name with the wrong number of C's is silently ignored: nothing validates unknown variable names. Copy names from this chapter or from the appendix of settings and check the log for the effect.

**Curated list** Defaults are what the code uses when the variable is unset or blank. "Off/on" values accept 1, true, yes, on unless stated.

**Identity and addresses**

| Variable | Default | Effect |
|---|---|---|
| CIVICCAST_STATION_STATE_PATH | Installer-configured service profile under C:\ProgramData\CivicCast\profile; when unconfigured, %LOCALAPPDATA%\CivicCast\station-state.json | Overrides the state file location; related state/credential files sit beside it |
| CIVICCAST_STATION_NAME | The profile's name, else "CivicCast Station" | Overrides the station name shown and used |
| CIVICCAST_STATION_TZ | The profile's zone, else local (= UTC) | An IANA zone for dayparts and retention dates; an invalid name gives UTC and a warning |

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_STATION_ID` | `civiccast-station` | Single-station id used by custom fields, paywall, agendas, recordings and as-run logs. Leave it unset |
| `CIVICCAST_PUBLIC_BASE_URL` | unset | Absolute `http(s)` address used to build RSS links and as the federation base; unset gives relative links |
| `CIVICCAST_RESIDENT_PORTAL_URL` | unset | The address the operator console's resident preview points to; unset uses the station's own portal |
| `CIVICCAST_LOCAL_MEDIA_BASE_URL` | unset (site-relative links) | An absolute media origin for live-manifest links, only when media is served from a different origin than the portal |

**Sign-in and network**

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_AUTH_RATE_LIMIT` / `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS` | 10 / 60 | Failed sign-ins allowed per window; must be positive whole numbers or the service will not start |
| `CIVICCAST_ALLOW_FIRST_ADMIN_RESET` | unset | `1` lets First Setup run again on a configured station |
| `CIVICCAST_STAFF_TOKENS` | unset | Fixed staff bearer tokens (`token:operator_id:...`); see Security and privacy |
| `CIVICCAST_TRUSTED_PROXY_CIDRS` | none | Comma-separated networks whose forwarded client address is trusted |
| `CIVICCAST_TRUST_PRIVATE_PROXIES` | `false` | Trust loopback and private-network hops as proxies |
| `CIVICCAST_CORS_ALLOWED_ORIGINS` | empty (no cross-origin access) | Exact origins allowed; a `*` is a startup error |

**Playout, outputs and relays** (details in Configure channels)

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_CHANNEL_AUTOMATION` | `inline` | `off` stops the automation driver that starts and supervises channels |
| `CIVICCAST_CHANNEL_AUTOMATION_POLL_SECONDS` | 2.0 | How often each channel is checked; must be positive |
| `CIVICCAST_EGRESS_SEAMLESS_RELOAD` | on | `0`, `false`, `no` or `off` makes the encoder restart at plan boundaries |
| `CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS` | 300 | Longest one preparation (ffmpeg) step; invalid or non-positive values are logged and replaced by 300 |
| `CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS` | 1800 | Longest span of a program prepared in one piece |
| `CIVICCAST_CONFORM_CACHE_GB` | 60 | Size limit of the cache of whole prepared programs |
| `CIVICCAST_PREPARED_PLAN_DIR_BUDGET_GB` | 5.0 | Limit for the prepared-plan folder |
| `CIVICCAST_GST_ALLOW_HARDWARE_DECODE` | off | Allows GPU video decoders in the playout worker |
| `CIVICCAST_GST_PREROLL_TIMEOUT_S` | 30 (5 to 45) | Wait for the picture to start |
| `CIVICCAST_GST_FIRST_OUTPUT_TIMEOUT_S` | 45 (10 to 120) | Wait for the first output |
| `CIVICCAST_STALL_TIMEOUT_S` | 10 | A worker with no output for this long quits so it can be restarted |
| `CIVICCAST_TS_RELAY` | `auto` | `auto`, `on` or `off` for the TSDuck relay on `udp-ts` outputs |
| `CIVICCAST_TS_RELAY_BASE_PORT` / `CIVICCAST_HLS_RELAY_BASE_PORT` | 17800 / 18000 | First local port used by each relay |
| `CIVICCAST_LIVE_HLS_ROOT` | the egress work folder | The only folder a web-preview folder may be under |
| `CIVICCAST_TSDUCK_PATH` / `CIVICCAST_TSDUCK_HOME` | unset / `%LOCALAPPDATA%\CivicCast\tsduck` | Where to find or keep TSDuck |
| `CIVICCAST_NDI_FFMPEG` / `CIVICCAST_SDI_FFMPEG` | unset | Your own NDI or DeckLink-capable ffmpeg |

**Captions and AI** (details in Configure captions and Configure AI models)

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_CAPTION_TAP` | forced `inline` | `off` turns live captions off entirely |
| `CIVICCAST_LIVE_CAPTION_ENGINE` | `whistle` for native live captions | `whistle` (CPU primary, Whisper fallback) or `whisper` (Whisper primary) |
| `CIVICCAST_WHISPER_DEVICE` / `CIVICCAST_WHISPER_COMPUTE_TYPE` | CUDA with supported NVIDIA GPU and staged runtime; otherwise `cpu` / `int8` | Forces `cuda` or `cpu` and the compute type |
| `CIVICCAST_WHISPER_CPU_THREADS` / `CIVICCAST_CAPTION_TAP_CPU_THREADS` | live: 1, up to 2 with 16 or more CPUs | CPU threads for captioning; live ignores 0 and caps above 2 |
| `CIVICCAST_CAPTION_TAP_SEGMENT_SECONDS` / `..._POLL_SECONDS` / `..._MAX_BACKLOG_SEGMENTS` | 5.0 / 2.0 / 2 | Live caption audio segment, scan period, tolerated backlog |
| `CIVICCAST_OFFLINE_CAPTION_JOB` | `inline` | `off` disables recorded-caption work by default until station settings are saved; use Station Profile for ongoing configuration |
| `CIVICCAST_OFFLINE_CAPTION_POLL_SECONDS` / `..._BACKOFF_SECONDS` / `..._MAX_ATTEMPTS` / `..._CHUNK_SECONDS` | 60 / 300 / 4 / 30 | Offline caption job timing |
| `CIVICCAST_SUMMARY_JOB` and `..._POLL_SECONDS` / `..._BACKOFF_SECONDS` / `..._MAX_ATTEMPTS` | `inline`; 15 / 120 / 3 | Summary job |
| `CIVICCAST_OLLAMA_BASE_URL` | `http://127.0.0.1:11434` | Read only by the alerting self-test; the summary and translation adapters use the loopback address directly |
| `CIVICCAST_PROVIDER_API_KEY` | unset | Read by `civiccast model set-provider-key` only (your shell, not the service) |

**Storage and lifecycle** (details in Storage and media lifecycle)

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_UPLOAD_MAX_BYTES` | 10,737,418,240 (10 GiB) | Largest accepted upload |
| `CIVICCAST_MEDIA_STORAGE_BUDGET_BYTES` | unset | Disk budget shown on the Storage budget card |
| `CIVICCAST_RETENTION_WORKER` / `CIVICCAST_RETENTION_POLL_SECONDS` | `inline` / 3600 | Flags expired assets for the records clerk's review; never deletes |
| `CIVICCAST_MEDIA_LIFECYCLE_WORKER` / `..._POLL_SECONDS` | `inline` / 300 | Readiness, ingest-time conversion and archive checks |
| `CIVICCAST_WATCH_FOLDER_WORKER` / `..._POLL_SECONDS` | `inline` / 2.0 | The watch-folder importer |
| `CIVICCAST_MEDIA_INTEGRITY_WORKER` | `inline` (checks every 3600 s) | Flags assets whose backing file has gone missing |
| `CIVICCAST_NAS_ARCHIVE_PATH` | unset | Local archive folder; needs `CIVICCAST_PROVIDER_LOCAL_NAS=real` |
| `CIVICCAST_BACKUP_DIR` | unset | A backup folder that overrides the one saved on the Setup page for the probe |

**Control Room and certificates**

| Variable | Default | Effect |
|---|---|---|
| `CIVICCAST_CONTROL_ROOM_TSR_URL` | unset | Address of the Node TSR sidecar, for example `http://127.0.0.1:7717`; unset fails every cue closed |
| `CIVICCAST_CERT_ROOT` | `.civiccast\certs` in the CLI user's home | Where the local CA and service certificates are kept (CLI only) |

For all other variables, the appendix of settings (Appendix: settings) has the generated list with the file where each is read.

#### If it did not work

| What you see | Likely cause | What to do |
|---|---|---|
| "First setup can only be done from the station computer itself" | You opened `/setup` away from the station | Use `/setup` on the station computer for first-admin actions. Ordinary staff use `/login`; remote staff sign-in requires the configured staff hostname and an approved network. |
| "First-admin setup is already complete. Use the recovery flow or set CIVICCAST_ALLOW_FIRST_ADMIN_RESET=1 for an intentional local reset." | The station is already set up | Sign in; use a recovery code if the password is lost |
| "Too many sign-in attempts from this station. Wait N seconds, then try again with the correct password, or use a printed recovery code." | The sign-in limit was reached | Wait the number of seconds shown |
| "Durable storage is not ready." or HTTP 503 on a screen | The database was not prepared or the service is still starting | Click **Prepare storage** on First Setup; check the service and `postgres.log` |
| The service stops soon after an environment edit | A value is invalid or a configured dependency cannot start | Read the exact control-plane error and correct the named value. `CIVICCAST_OFFLINE_CAPTION_JOB=off` is supported; it is not itself a startup error |
| "No outgoing-feed configuration for <id>. Apply a headend preset or the local rehearsal preset first." | The channel has no configuration | Apply a preset on the Channels screen |
| "Sink kind(s) [...] are not supported by the active GStreamer egress engine." | The configuration contains an unsupported sink kind | Use a supported kind: `srt`, `rtmp`, `udp-ts`, `local-ts`, `file`, `sdi` or `hls`; check the exact kind named in the error |
| "No hardware video encoder was found on this machine. To broadcast on the CPU instead (slower)..." | The profile names a hardware encoder this computer does not have | Tick **Allow software (CPU) encoding fallback**, or change the profile |

| What you see | Likely cause | What to do |
|---|---|---|
| “Start was queued but the feed did not start. The outgoing-feed worker did not report Starting or On air within 20s…” | The feed worker did not respond | Check the service in System Health, then try Start again |
| “channel ‘<id>’ has no udp-ts sink to verify — apply a headend delivery profile first.” | Commissioning proof on a channel with no UDP output | Apply the headend preset first |
| “Device host must be localhost, .local, or a private/link-local IP unless a setup admin records a public-host override reason…” | Control Room device on a public address | Use a private address; the screen has no way to record an override, so ask support |
| “The credential store is not available to persist the device secret.” / “The OS credential store is unavailable; provider keys cannot be saved here.” | The Windows credential store is not available to the service account | Ask support; use `civiccast model set-provider-key` from an account that has it |
| Control Room readiness is blocked at “TSR control service” | The sidecar is not installed or running, or the variable is not set | See Set up the Control Room |
| A setting you changed in the registry has no effect | The service was not restarted; the variable is one the service forces; or the name has the wrong number of C’s | Restart the service; check the forced list above; check the spelling |

**Package status**

These instructions describe the unpublished Beta 20 candidate edition. Use the release page and the Help and verification record that match the package on the station. The Beta 18 verification record tracks that earlier package’s checks.

**Related**

- Planning your station for the hardware, disk and network choices behind these settings.
- Installing, first run, upgrading, uninstalling for the folder layout, the service and the first-run wizard.
- Running it day to day for backups, logs and updates.
- Security and privacy for tokens, sign-in, the credential stores and what the paywall and provider keys expose.
- Troubleshooting matrix for symptoms and fixes.
- Cable headend, streaming, CDN, federation, emergency alerts, the API for delivery to a head-end, CDN and federation.
- Appendix: command line, Appendix: API, Appendix: settings and Roles.

## Running it day to day: service, logs, backups, updates

This chapter is for the IT person who keeps a CivicCast station running on native Windows. It explains the Windows service and logs, daily and weekly checks, disk use, backup and restore, and updates.

### Before you start

You need:

- An **Administrator** account on the station computer. Starting, stopping and restarting the service, reading `HKLM` registry values and reading the log folder all need elevation.
- The install folder, written `<INSTDIR>` in this chapter. The current default is `C:\Program Files\CivicCast (Native)`; a custom install can use another location. The installer places the program under it (`runtime\`, `packs\`, `dependencies\`, `models\`).
- The data folder, `C:\ProgramData\CivicCast`. Everything the station writes while it runs lives here (the code honours the `PROGRAMDATA` environment variable if it is set differently).
- An elevated PowerShell window for the commands below.

> **Note:** Earlier descriptions of CivicCast mention NATS JetStream. The native Windows station does not use it; do not monitor it or open a firewall port for it.

### What the service runs

CivicCast on Windows is one Windows service named **CivicCastSupervisor** (display name **CivicCast Native Supervisor**). It starts automatically at boot as the restricted virtual account `NT SERVICE\CivicCastSupervisor` and is registered so that Windows restarts it on failure: restart after 5 seconds, then 10 seconds, then 30 seconds, with the failure count resetting after 86,400 seconds (one day). Existing default LocalSystem installations migrate their profile and CivicCast credentials before the account changes; see the restricted service identity guide. The supervisor starts and watches a small number of child programs. Everything else (the playout workers, relays and caption tap) lives inside the control plane child.

The supervisor itself starts only three kinds of child, in this order: **Postgres** first, then the **control plane**, then **Ollama** if it is present. The control plane is the CivicCast application (a Python web service). It listens on a configured loopback address (`127.0.0.1` by default, or `::1`) and runs the playout workers, the transport-stream relay, the HLS relay and the caption tap itself. Those are not separate Windows services and you do not manage them individually. Supported remote HTTPS uses Caddy in front of this listener; see the network access guide.

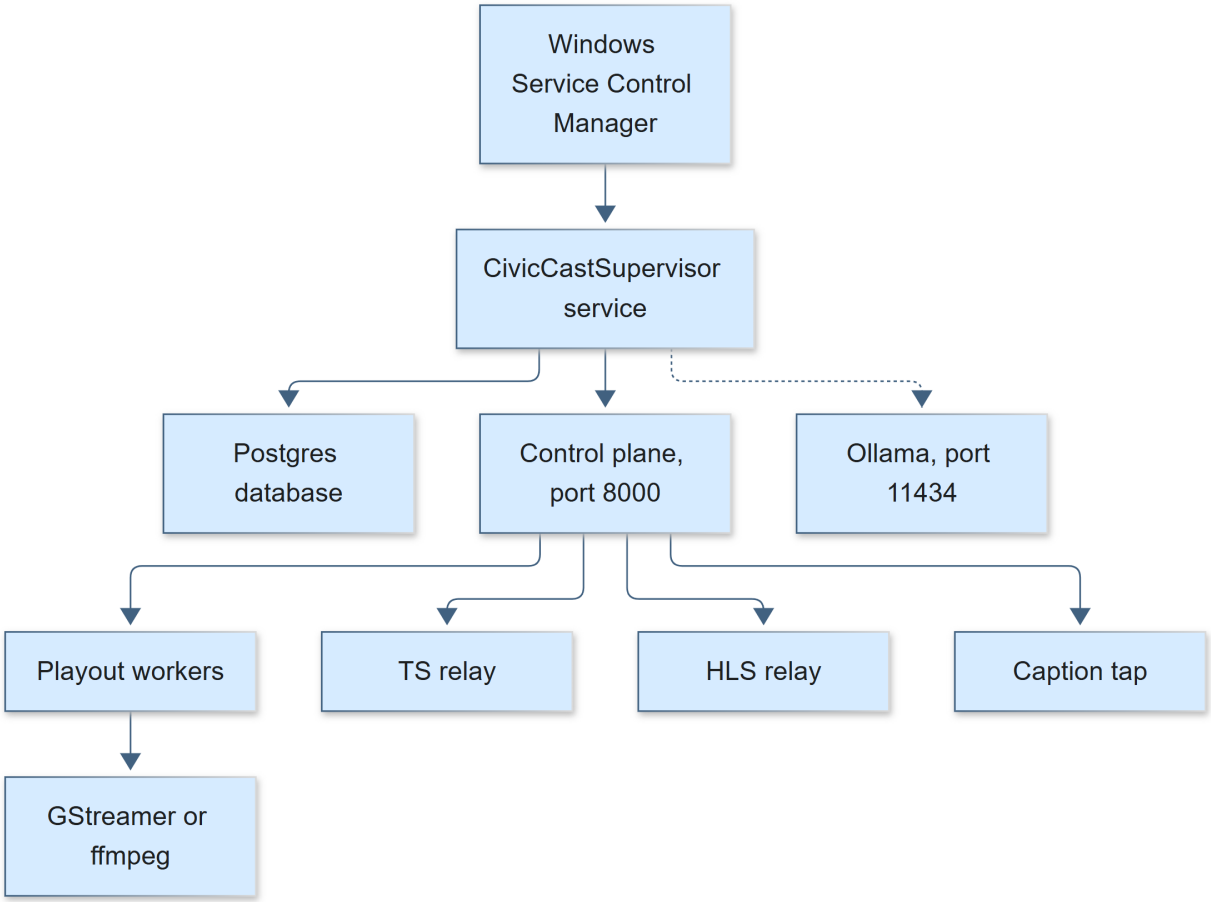



Figure: the processes CivicCast runs. A solid arrow means the parent starts and restarts the child; the dotted arrow means the child is optional.

| Piece | What it is | Who starts it | Notes |
|---|---|---|---|
| Postgres | The station database | Supervisor | Listens on `127.0.0.1` only. Uses the first free port of 5432, 5433, 5434, 5435, 5544. Password logins only (`scram-sha-256`), no TLS. |
| Control plane | The CivicCast application and its web API | Supervisor | `python -I -u -m uvicorn civiccast.app:create_app --factory --host <configured loopback> --port <configured port>`. Network Settings accepts `127.0.0.1` or `::1`; do not expose the app listener directly. |
| Ollama | The local AI model server used for summaries and similar tasks | Supervisor | Optional. Started only when both `dependencies\ollama\ollama.exe` and a model store with a `manifests` folder exist. Listens on `127.0.0.1:11434`. When skipped, it shows as `stopped` with a reason, and the supervisor re-checks every 60 seconds. |

| Piece | What it is | Who starts it | Notes |
|---|---|---|---|
| Playout workers | The processes that put your channel on the air | Control plane | The playout engine is GStreamer by default. The older ffmpeg concatenation engine is used only when `CIVIC-CAST_EGRESS_ENGINE` selects it. |
| TS relay | Sends a transport stream (for a headend encoder) | Control plane | Uses TSDuck (`tsp.exe`). Automatic mode runs only when `tsp` is present. |
| HLS relay | Sends a web stream | Control plane | Supervised `ffmpeg` process. |
| Caption tap | Listens to the channel audio for live captions | Control plane | Runs inside the control plane. Live captions are off by default; `CIVICCAST_CAPTION_TAP=off` forces them off. |

If CivicCast finds another CivicCast runtime active on the same computer (the older WSL-based product), the supervisor refuses to start the station and reports `blocked_wsl_active`. See If it did not work.

**Supervisor states** The supervisor keeps one of seven states internally. The operator console does not show these internal names; use `/health`, `civiccast runtime status` and `supervisor.log` to see the effects described below.

| State | Meaning |
|---|---|
| `starting` | Children are being started in order. |
| `ready` | Postgres and the control plane are up and the control plane answered its health check. |
| `degraded` | Something is wrong but the supervisor is still running: a child is failing to stay up, or a restart storm was detected (see below). |
| `blocked_wsl_active` | The dual-runtime guard found the older WSL runtime active and refused to start. |
| `blocked_probe_unavailable` | The guard could not tell whether the older runtime is active and refused to start. |

| State | Meaning |
|---|---|
| `maintenance` | The control plane is running in maintenance mode (no workers, no changes). |
| `stopping` | A stop was requested and children are being shut down. |

**How the supervisor restarts things**

- A child that exits is restarted after a delay that starts at 1 second and doubles up to 30 seconds, with about plus or minus 20 percent random jitter.
- A restart storm is 5 or more restarts inside 600 seconds. The supervisor then reports `degraded` and raises an alert of kind `service-down` from source `supervisor`.
- Readiness budgets at start: Postgres 60 seconds, control plane 180 seconds (with a rule that the control plane is declared stalled if it makes no CPU progress for 60 seconds), Ollama 60 seconds.
- A normal stop gives each child 15 seconds, then force-terminates it. Postgres is stopped with `pg_ctl stop -m fast`. The control plane is stopped by a console break signal so it can drain cleanly. A Windows Job Object kills every child if the supervisor itself dies.
- A stop watchdog forces the supervisor to exit 150 seconds after a stop begins (`CIVICCAST_SUPERVISOR_STOP_WATCHDOG_SECONDS`; a value of 0 or less disables it). When it fires it reports the service as stopped and writes an entry to the Windows Event Log.
- After 3 consecutive identical start failures the supervisor writes `C:\ProgramData\CivicCast\STATION-START-FAILED.md` describing the failure. The file is removed on the next good start.

#### Start, stop and restart the service

The supervisor has an internal control pipe, but beta.11 has no command-line tool that uses it. Operate the service with Windows itself.

1. Open an elevated PowerShell window.
2. Check the current state: `sc.exe query CivicCastSupervisor`
3. To start it: `sc.exe start CivicCastSupervisor`
4. To stop it: `sc.exe stop CivicCastSupervisor`
5. To restart it, stop it, wait until `sc.exe query CivicCastSupervisor` shows `STOPPED`, then start it.

You can do the same in the Windows **Services** app (`services.msc`). After a start, allow up to about three minutes for the control plane to become ready (its readiness budget is 180 seconds), then check health as described below.

> **Warning:** Stopping or restarting the service stops every playout worker and takes every channel off the air. Do it between meetings. A stop can take up to about 150 seconds in the worst case

> (the watchdog above).

> **Tip:** `sc.exe` error 1061 means the service cannot accept the command right now (typically a start or stop is still in progress); error 1062 means the service is not running. Neither needs action beyond waiting or starting it.

**Set environment variables for the service** CivicCast reads many settings from environment variables (the full list is in Appendix C). The service reads them from the standard Windows service `Environment` value, which is a multi-string value named `Environment` under `HKLM\SYSTEM\CurrentControlSet\Services\CivicCastSupervisor`. Each line is `NAME=value`. The children inherit them.

1. Open `regedit` as Administrator and go to that key.
2. Create or edit the `Environment` value (type **Multi-String Value**). Put one `NAME=value` per line.
3. Restart the service.

> **Known issue (beta.11):** The installer does not create a service `Environment` value for optional variables. Create it yourself when a setting requires one, and check it again after every upgrade.

> **Note:** The database address comes from the registry value `HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl`, which SYSTEM, Administrators and the station service identity can read. The supervisor copies it into the control plane's environment unless `DATABASE_URL` is already set in the service environment, in which case the environment wins. `supervisor.log` records which source won.

#### Read the logs

Service logs are in `C:\ProgramData\CivicCast\logs`. GStreamer worker logs are under each channel's egress work folder, in its `logs` subfolder.

On **System Health > Egress storage**, check worker-log bytes and file count, command-history rows and database storage, pending commands, and free space on the work volume. The panel warns below 20 GiB free. An unavailable measurement is shown as unavailable, rather than zero.

Each GStreamer stdout/stderr stream rotates at 10 MiB and keeps ten older files. An older oversized active log is reduced to its newest 10 MiB before the next worker starts. If the disk cannot accept diagnostics, CivicCast continues draining the worker output so logging does not stall playout. Free disk space to restore diagnostic logging; this does not make a full disk safe for recordings.

Consumed egress command history is trimmed in batches of up to 1,000 when a channel exceeds 10,000 consumed commands. Pending commands are preserved. Database space can be reused after pruning without immediately shrinking the database file. These limits do not delete recordings or retain live-caption audio evidence.

| File | What is in it | Rotation |
|---|---|---|
| `supervisor.log` | The supervisor: state changes, child starts, exits, restarts, readiness checks, the stop watchdog. The first line is `supervisor logging initialized` with the process id and the log destinations. | 10 MiB per file; keeps 10 older files (up to 11 files total). Each record is flushed to disk. |
| `control_plane-app.log` | The CivicCast application's own log (playout, schedule, captions, alerts). | 10 MiB per file; keeps 10 older files (up to 11 files total). Written only when the control plane runs under the supervisor. |
| `control_plane.log` | Raw standard output and error of the control plane, including the web server's access log (one line per request). | 10 MiB per file; keeps 10 older files (up to 11 files total). |
| `postgres.log` | PostgreSQL startup and server messages. The supervisor drains the inherited standard-output pipe after `pg_ctl` exits. | 10 MiB per file; keeps 10 older files (up to 11 files total). |
| `ollama.log` | Output of the Ollama child, when it runs. | None. |

After upgrading, an existing `control_plane.log` or `postgres.log` larger than 10 MiB rotates into an older copy without being shortened. That copy can exceed the normal size limit until later rotations replace it. New output follows the limits above; upgrading does not immediately reclaim space used by older logs.

The supervisor also writes some failures (for example the stop watchdog) to the Windows **Application** event log. In testing we could not confirm the source name Windows displays for them.

Other logs you may need:

- `C:\ProgramData\CivicCast\install-progress.log`: the installer's record of a first install.
- `C:\ProgramData\CivicCast\upgrade\upgrade-engine.log`, `upgrade-journal.json` and `UPGRADE-RECOVERY.md`: the upgrade engine, described below.

To follow a log live:

```
Get-Content C:\ProgramData\CivicCast\logs\supervisor.log -Tail 50 -Wait
```

#### Check that the station is healthy

**The health endpoint** The control plane answers `GET /health` (also at `/api/health`) with no sign-in. Run it from the station computer:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

The HTTP status is 200 whenever the control plane is alive, even if it is unhealthy. Read the body, not the status code. The fields are:

| Field | Values | Meaning |
|---|---|---|
| `status` | `healthy`, `degraded` | `healthy` requires a current database schema and no degraded or unknown live-caption verdict. This is not proof of every output or device. |
| `version` | text | The running CivicCast version. |
| `schema` | `current`, `behind`, `not-configured`, `unknown` | Whether the database matches this version of the program. |
| `live_captions` | `disabled`, `idle`, `unknown`, `degraded`, `healthy` | Coarse caption readiness; authenticated System Health identifies affected channels. Disabled captions and channels that are not on air do not themselves make readiness degraded. |
| `schema_db_revision`, `schema_expected_head` | text | The database's schema revision and the one this version expects. They differ when `schema` is `behind`. |
| `mode` | `normal`, `maintenance` | In `maintenance` the body also shows `workers_started: false` and `mutating_disabled: true`. |

> **Warning:** A monitoring tool that only checks for HTTP 200 will report a degraded or maintenance-mode station as fine. Alert on `status` not equal to `healthy`, and on `mode` not equal to `normal`.

Two more public endpoints give small answers: `GET /api/version` and `GET /api/hardware`.

**The Readiness screen** Signed in, open **Readiness** (page heading **Safe to broadcast**, section **System Health**). It is the station's own summary of whether it can go on air. Its controls are described in Chapter 8. The same data is available to staff as `GET /api/staff/runtime-safe-to-air` (any of

the five roles), `GET /api/staff/installer/system-health` and `GET /api/staff/system-resources` (see Chapter 15 for calling the API).

**Alerts** The station raises alerts for conditions including: off-air, encoder death, server crash, schema drift, relay blocked, compliance probe failure, missing media, commit failure, takeover stuck for 2 hours, AI runtime down, low disk, clock skew, database unreachable, service down, self-test failure, scheduled-recording failure or dropout, as-run outbox degraded, channel-automation failure, caption tier degraded, remote-contribution problems, and emergency-alert source unavailable.

> **Known issue (beta.11; destination editor updated in beta.12):** A fresh install seeds every alert rule with no destinations attached. With no destination, the evaluator records a "suppressed" delivery row and sends nothing. In beta.11 the rule editor could not set destinations; beta.12 adds a **Destinations** checklist to each rule card. To make an e-mail or SMS alert arrive, a Setup admin must create a destination with `POST /api/staff/alert-channels` and attach it to each rule with `PUT /api/staff/alert-rules/{rule_id}` (field `channel_ids`) or use the rule card. There is no **Send test alert** button. Until you have assigned at least one enabled destination to each rule you rely on, assume it cannot notify you and monitor from outside the station as well: poll `/health` and the Windows service state with your own monitoring tool.

**Self-tests** The station runs its own checks on a schedule. By default (the code's default settings) the **daily** self-test runs at 02:00 and the **weekly** self-test on Sunday at 03:00, both by the station computer's own Windows clock and time zone. If the service was down at that moment, the missed run happens after it comes back up.

- **Daily:** the install-time readiness path, a short playout continuity proof to a file, a backup-destination write/read/delete probe, and a ping of the AI runtime. The AI ping is advisory: if the optional AI runtime is down it warns but does not fail the box.
- **Weekly:** everything in the daily test plus a restore rehearsal, an SRT continuity proof, a TSDuck compliance probe (advisory) and an alert-channel check (advisory). A check whose tooling is not installed is left out, not recorded as a pass.

A non-passing result raises a `self-test-fail` warning and clears itself on the next clean run. Staff can read results at `GET /api/staff/self-tests` and, with the Setup admin or Support admin role, start one with `POST /api/staff/self-tests/run?kind=daily` or `kind=weekly`.

> **Note:** `civiccast dr run-drill` exercises a database backup/restore into a throwaway database; it is not a full station restore. The Setup/System Health backup-storage probe checks a small manifest round-trip, not restoration of the station database, media, configuration or provider state. Neither replaces the separate backup work later in this chapter.

**Command-line checks** These need the CivicCast command-line program. Use the full path because the installer does not add it to PATH:

```
$cc = "<INSTDIR>\runtime\Lib\site-packages\bin\civiccast.exe"
& $cc doctor
& $cc doctor --disk C:\ProgramData\CivicCast
& $cc runtime status
```

`civiccast doctor` accepts `--json`, `--disk <path>` (the folder whose drive is checked; it takes a path, and without it the check uses the home folder of the account running the command) and `--profile`. `civiccast runtime status` accepts `--json` and reports which runtime (native or the older WSL one) is active. The other commands used by operators are listed in Appendix A.

If `civiccast.exe` is missing, `<INSTDIR>\runtime\python.exe -m civiccast.cli` runs the same program.

#### Do the daily and weekly routine

This routine uses checks that exist in beta.11.

### Each day (about five minutes)

1. Run `Invoke-RestMethod http://127.0.0.1:8000/health` on the station. Confirm `status` is `healthy` and `mode` is `normal`.
2. Run `sc.exe query CivicCastSupervisor`. Confirm `RUNNING`.
3. Open **Readiness** in the console. Confirm it says the station is safe to broadcast. Read any warning in full.
4. Look at the last lines of `C:\ProgramData\CivicCast\logs\supervisor.log` for `degraded`, repeated restarts or `service-down`.
5. Check that `C:\ProgramData\CivicCast\STATION-START-FAILED.md` does not exist.
6. Check free space on the drive that holds `C:\ProgramData\CivicCast`.
7. Read the result of last night's daily self-test (`GET /api/staff/self-tests`, or the Readiness screen).

### Each week

1. Read the result of the Sunday weekly self-test.
2. Check the size of `ollama.log`; it is not rotated. The control-plane and PostgreSQL raw output logs rotate at 10 MiB and retain ten older files.
3. Check the size of `C:\ProgramData\CivicCast\data\egress` (see the next section). The native live-caption work area is bounded automatically and does not require weekly audio cleanup.
4. Run `civiccast egress trim-health --older-than-days 30 --dry-run` and decide whether to trim (see the next section).
5. Check the latest encrypted `.ccbackup` station archive and its verification result in **Backup and restore**. Keep a copy on separate storage; the optional database drill is not a station backup.
6. Check which media roots your station archive includes. Back up any excluded media separately; follow the station-backup workflow below rather than assuming the database drill includes those files.

#### Manage disk and cache

### Where space goes

| Folder | What it holds | Grows? | Trimmed automatically? |
|---|---|---|---|
| `C:\ProgramData\CivicCast\data\pgdata` | The Postgres database | With use | No |
| `C:\ProgramData\CivicCast\data\uploads` | Uploaded media | With use | No |
| `C:\ProgramData\CivicCast\data\egress` | Playout working files, including `conform-cache` | Yes | Yes, see below |
| `C:\ProgramData\CivicCast\data\caption-tap` | Temporary live-caption audio; `active.vtt` is in the channel's egress caption folder | Bounded in the native runtime | Processed chunks deleted; queued audio limited, see below |
| `C:\ProgramData\CivicCast\logs` | Logs | Yes | Only the rotated logs |
| `C:\ProgramData\CivicCast\upgrade` | Upgrade engine log, journal and pre-upgrade backups | Per upgrade | No |

> **Note:** A Beta 15 field run measured a 15.117 GiB drop in free space on the R9 station, but did not identify the responsible paths. This is a Beta 17 storage-accounting follow-up, not a confirmed CivicCast leak or a statement about Beta 16. See the deferred field follow-up.

**The conform cache** The **conform cache** is a folder where CivicCast keeps ready-to-play copies of your videos, converted to the channel's format. It is at `C:\ProgramData\CivicCast\data\egress\conform-cache`.

- **Size cap:** `CIVICCAST_CONFORM_CACHE_GB`, default 60 (gigabytes). A value of 0 or less disables the cache. When the cache is full, the oldest copies are removed first, and a copy that is used is refreshed so it is not the oldest.
- **Plan folders:** each channel's prepared playout plan folders are limited to the 3 most recent per channel, within a budget of `CIVICCAST_PREPARED_PLAN_DIR_BUDGET_GB` (default 5 gigabytes), and a folder younger than 24 hours is not removed. Leftover scratch files are removed after 1 hour.
- **Too small a cap:** if one video's conformed copy cannot fit in the budget, preparation of that video fails with: `Conform-cache budget too small to retain '<name>';`

`increase CIVICCAST_CONFORM_CACHE_GB or exclude this asset.` Raise `CIVICCAST_CONFORM_CACHE_GB` in the service environment and restart.

- **Failed background conforms:** after a failed or timed-out background (warm-up) conform the station waits 6 hours before trying that asset again. The time allowed for a background conform scales with the length of the asset and is capped at 7,200 seconds.

Historical measurement: in an eight-hour lab run on an earlier development build (three channels), the cache reached 46 GB of the 60 GB cap on a drive with 810 GB free, and the longest single preparation took 389 seconds. That is not a measurement of the current package or a sizing rule. Size the cap from your own library: it should hold the conformed copies of everything you expect to air in the next several days.

> **Known issue (beta.11):** There is no free-space guard on the conform cache. The 60 GB figure is a cap on the cache, not a promise that the disk has that much room. Make sure the drive has more free space than the cap plus the database, uploads and logs, and watch the **low disk** alert (which may have no destination).

**Playout health records** The playout engine stores health telemetry in the database. Nothing trims it automatically. The only trim is the command below. It needs `DATABASE_URL` set (see the next section for how to read it) and accepts `--dry-run` and `--json`:

```
& $cc egress trim-health --older-than-days 30 --dry-run
& $cc egress trim-health --older-than-days 30
```

**Caption data** **Live captions:** Ordinary live captioning creates no permanent per-cue review rows or evidence WAVs. Consumed audio is deleted after processing. Whistle's working-file ceiling is 42 waiting five-second segments per channel (210 seconds), plus in-flight inputs and the segment being written. Runtimes without Whistle's processing grace use a 12-segment ceiling. A small previous-audio overlap remains in memory for recognition. This finite file ceiling and the sustained-overload threshold serve different purposes.

The live caption file and delivery bookkeeping retain a rolling 300-second window with at most 512 cues. The live worker does not scan the caption-review archive, and archive evidence does not gate broadcast readiness. These bounds replace manual weekly live-caption cleanup. If working files exceed these bounds, report a fault rather than treating routine deletion by an operator as normal operation.

Original recordings, archived caption tracks and recorded-caption review remain available and are unaffected by these temporary live-caption limits. Analytics retention is separate: events are kept up to 366 days (`CIVICCAST_ANALYTICS_RETENTION_DAYS`, allowed range 1 to 366).

> **Historical finding (published beta.10):** The audit found unlimited live-caption review/evidence accumulation, uncovered raw chunks, growing `active.vtt`, and unpruned quarantine/collision folders. Beta.11 changes that live path; the ordinary published beta.11 runtime does not

> run the archive sweep or append per-cue evidence records. Its exact-package record contains only a brief output observation; the separate 36-hour dev7 overlay soak is development-station history, not package proof. Long unattended operation remains unproven for the published package.

#### Back up and restore

**Station backup and restore** Use **Station backup and restore** to create, verify and restore portable station archives. The encrypted `.ccbackup` contains the database, configured station files and portable CivicCast credentials, plus every file beneath the selected media roots. Choose an absolute destination, keep the recovery passphrase separately, and verify an off-station copy. A daily UTC schedule and bounded retention are available after saving the unattended passphrase in protected station storage. The station screen shows progress and the last run; the detailed inventory, restore steps and limitations are in the station backup guide.

Restore is an offline maintenance operation. Stop CivicCast and other station writers, provision the replacement service/database, then use `civiccast backup inventory` to inspect archive paths and `civiccast backup restore` with explicit target mappings and the `RESTORE` confirmation. The command prompts for the passphrase. Do not overlay a running station. A database restore can complete before later file or credential copies fail, so keep the source archive and rerun after correcting a path or permission problem.

The scheduled passphrase is stored separately from the archive and schedule. Keep it available to the operator who would rebuild the station. An upgrade rollback backup remains separate and is not a substitute for an off-station `.ccbackup`.

**Database restore drill** `civiccast dr run-drill` remains a separate database restore and crash-recovery drill. It proves a backup can be restored into a throwaway database; it is not the operator's station backup or live restore workflow.

**Run the disaster-recovery drill** The drill command is:

```
civiccast dr run-drill --out <dir> [--backup-dir <dir>] [--work-dir <dir>]
↪  [--database-url <url>] [--media-root <dir>]
```

| Option | Meaning |
|---|---|
| `--out` | Required. The folder the report is written to (`dr-drill-report.md` and `dr-drill-report.json`). |
| `--backup-dir` | Where the backup goes. Default `<out>\backup`. |
| `--work-dir` | Scratch folder for the restore and crash drills. Default `<out>\work`. |

| Option | Meaning |
|---|---|
| `--database-url` | The database to drill. Default: the DATABASE_URL environment variable. Only `sqlite://` and `postgresql://` addresses are accepted. |
| `--media-root` | A media folder to list in the backup manifest. Optional. |

The drill does the following:

1. **Backup.** It writes `database.pgdump` (`pg_dump --format custom` with a snapshot), `globals.sql` (`pg_dumpall --globals-only`) and `manifest.json` (table row counts and SHA-256 checksums). With `--media-root` it also records every media file's path, size and a hash of the first and last 64 KiB. It does not copy the media files.
2. **Restore.** It restores the backup into a throwaway database named `civiccast_drill_restore` on the same Postgres server (dropping and recreating that name), and it refuses to drop the production database.
3. **Verify.** It checks row counts, checksums, the schema revision, that the application can read the restored data, extensions, sequences and grants.
4. **Crash drill.** It starts a stand-in process and checks that the crash-recovery logic restarts it. This uses a stand-in and an in-memory store, and does not touch your real channels.

It exits 0 if everything passed, 1 if a drill failed, and 2 if there is no DATABASE_URL or the address scheme is unsupported (`No DATABASE_URL configured; pass --database-url or set $DATABASE_URL.` or `Unsupported DATABASE_URL scheme for the DR drill; use sqlite:// or postgresql://.`). Read the report: the code itself says media is a manifest and not a copy, the crash drill covers only automatic restart of a worker, and there is no hot failover.

> **Known issue (beta.11):** The drill calls `pg_dump`, `pg_dumpall`, `pg_restore` and `psql` by bare name, so Windows must find them through PATH. The installer does not add them to PATH, and the service adds only the FFmpeg folder. Run from a normal prompt and the drill can fail with `pg_dump could not be started: the executable "pg_dump" is a bare command name that could not be resolved through PATH.` The console's **Run real database restore drill** button calls the same code. The upgrade engine uses full paths and does not have this issue. Use the workaround below.

Workaround, from an elevated PowerShell window on the station:

```
$inst = "<INSTDIR>"
$env:PATH = "$inst\packs\native-server-binaries\payload\bin;$env:PATH"
$env:DATABASE_URL = (Get-ItemProperty
 ↪  'HKLM:\SOFTWARE\CivicCast\Native').DatabaseUrl
& "$inst\runtime\Lib\site-packages\bin\civiccast.exe" dr run-drill --out
 ↪  D:\civiccast-dr\2026-10-05 --media-root D:\civiccast-media
```

```
Remove-Item Env:\DATABASE_URL
```

> **Warning:** The database address contains the database password. Set it only in the window you are using, never write it to a file, a script or a support ticket, and clear it afterwards as shown.

The `--out` folder must have room for a full copy of the database. Choose a drive that is not the station's data drive when you can.

**What the archive covers** The station archive uses the installed integration's configured database, files, portable credential adapter and selected media roots. It is encrypted before publication; do not copy individual machine-protected DPAPI or Windows Credential Manager blobs as a substitute. Program files, packs and models are not part of the archive and must be reinstalled. Review the exact inventory and the portable credential boundaries in the station backup guide.

#### Recover from a disaster

The station archive is intended for a replacement installation: provision the station, stop all writers, inventory the archive, map old paths to their replacement locations, restore, import portable credentials and verify service health before returning to air. Use the station backup guide for the complete sequence and failure boundaries. Keep installer rollback and station disaster recovery as separate copies and procedures.

#### Update and upgrade

An upgrade runs the new `setup.exe` over the existing install. The current installer source first compares versions without stopping the service or replacing files. If the installed version is newer, setup exits 129. For an accepted upgrade, setup stops the service and takes a size-checked snapshot of the prior application and station state before replacing files. The engine then takes a verified database backup to `C:\ProgramData\CivicCast\upgrade\backups\pre-<new_version>`, upgrades the program and migrates the database. If a step fails, the engine attempts database rollback and the enclosing installer transaction then attempts to restore the pre-setup snapshot. When that restoration succeeds, the prior application, local database and service state are returned; if it cannot finish, keep the recovery records and station data intact.

It writes `upgrade-engine.log`, `upgrade-journal.json` and, if a rollback itself fails, `UPGRADE-RECOVERY.md`, all under `C:\ProgramData\CivicCast\upgrade`. Setup's exit codes tell you what happened:

| Setup exit | Engine code | Meaning | What to do |
|---|---|---|---|
| 0 | 0 | Upgrade committed | Check `/health` |

| Setup exit | Engine code | Meaning | What to do |
|---|---|---|---|
| 124 | 10 | The engine rolled back its database work; setup then attempts to restore the pre-setup snapshot | Read `install-progress.log` and the failed-run engine log and journal under `C:\ProgramData\CivicCast\install-transaction\failure-diagnostics` when present. If restoration succeeded, the previous service configuration is restored; fix the cause before running setup again. Keep recovery records if restoration did not finish. |
| continue | 11 | Fresh install, nothing to upgrade | None |
| continue | 12 | Same version already installed | None |
| 129 | 13 | An older setup was refused by the version guard | Use the setup matching the installed version. The normal read-only check exits before stopping the service or replacing files; a defensive late check also triggers snapshot restoration. |

| Setup exit | Engine code | Meaning | What to do |
|---|---|---|---|
| 113 | 20 | The engine's database rollback failed; setup also attempts to restore the prior installer snapshot | Read `install-progress.log` and retained diagnostics. `UPGRADE-RECOVERY.md` may not remain under `upgrade` after successful outer restoration. If restoration failed, keep `install-transaction` and station data intact. |
| 114 | 30 | A migration cannot be restored | Read the engine log; do not retry blindly |
| 128 | 31 | A terminal failure journal from an earlier attempt remains | Inspect `upgrade-journal.json` and any retained `install-transaction\failure-diagnostics` copy, then archive or move the prior terminal journal aside; do not delete it. |
| 136 | — | Installer recovery could not be prepared or completed | Keep `install-transaction` and station data intact; let the recovery task or next setup retry. |
| 115 | other | Other failure | Read the engine log |

Related provisioning exits: 75 maps to setup exit 116, 87 to 135 (another CivicCast product or the WSL runtime is present), 85 to 127.

Before upgrading an on-air station, take a separate backup and schedule a maintenance window with the recovery kit available.

> **Warning:** Do not upgrade during a meeting. The upgrade stops the service and takes every channel off the air.

After any upgrade, run through the daily routine, then run the drill.

#### Capacity guidance

The 72-hour three-channel host measurement is recorded in the Beta 11 host report. Historical Beta 12 upgrade observations are described in the verification record.

- In the historical eight-hour C16 lab run, the cache used 46 GB of its 60 GB cap and the longest conform took 389 seconds.
- That earlier run used a single service process and had 0 holes in 50 program changes. Its recorded limits included 13 live-caption catch-up discards, two self-recovered program-change aborts and one single-frame drop. These are historical C16 observations, not Beta 16 measurements. DeckLink SDI output and a real cable headend were not exercised in that run.
- The separate historical Beta 11, 72-hour, three-channel host measurement is documented in the host report.

Hardware sizing (CPU, memory, number of channels) is in Chapter 9 and is not repeated here.

> **Historical beta.10 finding:** An earlier audit reported a program-change watchdog gap. The current daemon includes a reload-stall watchdog that reissues a stuck rollover once, then restarts the channel if it remains pinned. That recovery is code behavior, not evidence of long-duration reliability; compare the as-run log with the schedule and consult the current verification record for package test scope.

#### If it did not work

| What you see | Cause | What to do |
|---|---|---|
| `sc.exe start` returns but `/health` never answers | Control plane still starting, or failed | Wait up to 3 minutes. Then read the last 100 lines of `supervisor.log` and `control_plane-app.log`. |
| `STATION-START-FAILED.md` appears | Three identical start failures in a row | Read the file. It names the failing child and error. Fix it, then start the service. |
| State `blocked_wsl_active` | The older WSL-based runtime is active on this computer | Run `civiccast runtime status` to see which runtime is active. The dual-runtime guard refuses to start the native station while the older runtime is active; see Chapter 10. |
| State `blocked_probe_unavailable` | The guard could not tell whether the WSL runtime is active | Run `civiccast runtime probe` and read its answer. |

| What you see | Cause | What to do |
|---|---|---|
| `/health` says `degraded`, `schema: behind` | Database older than the program | Do not edit the database. Check whether an upgrade stopped partway; read `upgrade-engine.log`. |
| `/health` says `mode: maintenance` | The control plane started without workers | Read `control_plane-app.log`. No channel plays in this mode. |
| `service-down` alert | Restart storm: 5 or more restarts in 10 minutes | Read `supervisor.log` for the repeating child and its exit reason. |
| `Conform-cache budget too small to retain '<name>'...` | The cache cap is smaller than that one file's copy | Raise `CIVICCAST_CONFORM_CACHE_GB` and restart. |
| `pg_dump could not be started: the executable "pg_dump" is a bare command name...` | PATH defect | Use the drill workaround above. |
| Disk full | Cache, logs, caption data or uploads grew | Stop the service, free space (logs first), then start. Do not delete files from `pgdata`. |
| Ollama shows `stopped` | `ollama.exe` or the model store is missing | The supervisor skips Ollama when its program or model store is absent. Nothing else depends on it. |
| Setup exit 135 during install | Another CivicCast product or the WSL runtime is present | See Chapter 10. |

#### Package status and verification

This chapter describes the unpublished Beta 20 candidate edition. For package results, use the matching release record: the Beta 18 record tracks the earlier package, while the Beta 15 verification record applies only to that package. Historical host measurements retain their Beta 11 or C16 run identity.

#### Related

- Planning your station
- Installing, first run, upgrading, uninstalling
- Configuring the station
- Security and privacy
- Troubleshooting matrix
- When something looks wrong
- Appendix A: command-line reference, Appendix C: settings, Appendix H: measured evidence

## Security and privacy

This chapter explains how to manage staff access, limit network exposure, protect resident data, and verify CivicCast packages. It covers roles, tokens, first-admin recovery, HTTPS, the public and staff APIs, retention, signatures and checksums. Complete first setup on the station computer and follow the network access guide before making the resident portal available remotely.

### Before you start

- You need Administrator rights on the station computer, and the Setup admin role in the console for the staff-token API calls.
- Most commands below need the CivicCast command-line program. The installer does not add it to PATH, so use the full path: `<INSTDIR>\runtime\Lib\site-packages\bin\civiccast.exe` (`<INSTDIR>` is the folder CivicCast was installed to). In this chapter `$cc` stands for that path.
- Token commands need the database address in the `DATABASE_URL` environment variable. Read it from the registry value `HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl` in an elevated PowerShell window, use it only in that window, and never write it to a file or a ticket, because it contains the database password:

```
$cc = "<INSTDIR>\runtime\Lib\site-packages\bin\civiccast.exe"
$env:DATABASE_URL = (Get-ItemProperty
 ↪  'HKLM:\SOFTWARE\CivicCast\Native').DatabaseUrl
```

### The security model in one page

- The application and database listeners stay on loopback. Remote HTTPS is available through the supported Caddy configuration, which separates the public resident hostname from a staff hostname restricted to configured CIDRs. Do not expose the application listener directly; see the network access guide.
- Anyone who can sign in to the console is a **staff user** with one or more of five roles.
- Staff sign in with a username and password at `/login`; the API returns a bearer token. Other staff API actions require `Authorization: Bearer <token>` and the assigned role.
- Residents use the **public** parts of the API, which need no sign-in.
- The installer configures the service as the restricted virtual account `NT SERVICE\CivicCastSupervisor`. Existing default LocalSystem installations migrate their profile and CivicCast credentials before the account changes. See the restricted service identity guide.

### Roles

CivicCast has five staff roles. A token carries one or more of them. The details of which screens each role opens are in Appendix F and in Chapter 2.

| Role id | Label in the console |
|---|---|
| `setup_admin` | Setup admin |
| `meeting_operator` | Meeting operator |
| `records_clerk` | Records clerk |
| `publish_operator` | Publish operator |

| Role id | Label in the console |
|---|---|
| `support_admin` | Support admin |

Rules the code enforces:

- For lifecycle and environment API tokens, the scope words `admin` and `operator` are aliases that grant all five roles. Console sessions use the roles assigned to the signed-in account.
- An API token with no scopes has no roles.
- A protected staff request without a token gets HTTP 401 with `Staff identity is required for this action.` The username/password login route is the exception and issues the session token.
- A request with a token that lacks the needed role gets HTTP 403 with `This action requires one of these CivicCast roles: <role ids>.`
- Setup admins assign roles to staff accounts in **Staff accounts**; CLI-issued database tokens receive scopes when issued.

#### Manage staff tokens

A **bearer credential** is a long secret string that proves an identity to the staff API. The operator console uses session credentials for people; programs and scripts use database or environment tokens. There are three kinds.

| Kind | Looks like | Where it is stored | How you retire it |
|---|---|---|---|
| Console session | Created when a first administrator or staff account signs in | `station-state.json`, as a salted hash | Expires after 12 hours; sign out, admin revocation, or eviction at the 20-session cap also ends it |
| Database token | `ccst_<id>_<secret>` | Database. Only a PBKDF2-SHA256 hash (210,000 iterations) plus a SHA-256 fingerprint is kept; the secret is shown once. | `civiccast token revoke` or `rotate` |
| Environment token | `ccenv1_` followed by 43 characters | The `CIVIC-CAST_STAFF_TOKENS` environment variable | Remove it from the variable and restart the service |

### Issue a database token

1. Open an elevated PowerShell window and set `$cc` and `DATABASE_URL` as shown above.
2. Run:

```
& $cc token issue --operator-id clerk-jones --display-name "Pat Jones" --scopes
↪  records_clerk,publish_operator --json
```

`--operator-id` and `--display-name` are required. `--scopes` takes a comma-separated list of role ids or the aliases above; its default is `operator`, which is all five roles. Give people only the roles they need. `--save-keyring` stores the token in the Windows credential store under the service name `civiccast.staff-token`. `--json` prints machine-readable output.

3. Copy the secret that is printed. It is printed once and cannot be shown again. Deliver it to its owner through a channel you trust, not through the same ticket that names the person.
4. Check it works. In PowerShell, on the station:

```
Invoke-RestMethod http://127.0.0.1:8000/api/staff/auth/me -Headers @{
↪  Authorization = "Bearer <the token>" }
```

You should see the operator's name and roles.

### List, revoke and rotate

- `& $cc token list` shows token metadata (never secrets). Add `--json` for scripts.
- `& $cc token revoke <token_id> --reason "left the clerk's office"` makes the token fail from then on. `--reason` defaults to `operator-requested` and is saved in the audit record. `<token_id>` is the public id shown by `token list`.
- `& $cc token rotate <token_id>` revokes that token and issues a replacement for the same operator with the same scopes. It prints the new secret once. `--save-keyring` saves it to the credential store.

The database records audit events for `issued`, `used`, `revoked` and `rotated`.

> **Known issue (beta.11):** A database token issued before the fingerprint was introduced cannot be matched. If `token list` shows a token that no longer authenticates after an upgrade, rotate it.

**Environment tokens** Generate one with `& $cc token generate-env`. To use it, add it to `CIVICCAST_STAFF_TOKENS` in the service environment (see Chapter 12) in the form `token:operator_id:Display Name:role[,role]`, with several entries separated by semicolons (`;`), and restart the service. Roles are required: an entry with no role stops the station from starting.

> **Known issue (beta.11):** On a station that has its database (every native station), the code checks database and console tokens first and accepts an environment token only when `CIVICCAST_STAFF_TOKENS_FALLBACK_WITH_DB=1` is also set in the service environment. The code treats that as a recovery setting. Without it, an environment token is rejected with `Invalid staff bearer token.` Prefer database tokens (`token issue`).

> **Warning:** The server cannot revoke an environment token. The console's **Sign out** only makes that browser forget it. To retire one, remove it from the variable and restart the service, which takes the channels off the air.

**Failed attempts are limited** A wrong token counts against the client's address: 10 failures in 60 seconds by default (`CIVICCAST_AUTH_RATE_LIMIT` and `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS`). After that the server answers HTTP 429 with a `Retry-After` header, and the console shows `Too many failed attempts to authenticate with the staff API. Wait N seconds, then try again.` A request with no token at all does not count. A valid token still works while the limit is active. The counts live inside the running process and reset when it restarts. CivicCast's own code ignores forwarded-address headers, so it counts failures against the address the web server reports. The web server (uvicorn) is started with its default settings, which replace that address with the `X-Forwarded-For` value when the connection comes from `127.0.0.1`. In a test with uvicorn 0.47 (not the shipped 0.51.0) we saw exactly that. So behind a reverse proxy on the same computer, failures are counted against the proxy's address if the proxy sends no such header, and against the visitor's address if it does.

> **Known issue (beta.11):** The memory the limiter uses to remember addresses has no upper bound (audit finding B-010). Keep the staff hostname restricted to approved CIDRs and retain bearer authentication and role checks; see the network access guide.

#### Understand sign-in, setup and recovery

**First setup and the recovery kit** Open `/setup` on the station computer for **First setup**. Creating the first admin there makes:

- the first-admin username and password (the password must have at least 12 characters), and
- a **recovery kit**: 8 one-time recovery codes.

The **Save kit** button downloads a plain-text file that contains the admin password and all 8 codes. The console blocks navigation and sign-out until the kit is confirmed.

> **Warning:** Treat the recovery kit like a master key. Print it and keep it in a locked drawer or a safe, or keep it in your password manager. Do not leave it in the Downloads folder, a shared drive or an e-mail.

The first administrator signs in at `/setup` on the station computer. Staff accounts use the separate `/login` page, including on the staff hostname from an approved CIDR when Caddy mode is configured. If the first-admin password is lost, **Use recovery code** burns one of the 8 codes and sets a new password; it takes two clicks and warns you each time. Other valid sessions remain signed in. Staff account creation, role assignment and session administration are covered in the staff accounts guide.

### What the server enforces

- **First setup and first-admin sign-in/recovery are local-only.** The routes under `/api/setup/*` check that the request came from the station computer. The generated Caddy configuration blocks these routes on both remote hostnames. Staff account sign-in uses `/api/staff/auth/login` and can be reached on the CIDR-restricted staff hostname; the public hostname does not forward staff routes.
- Station-local first-admin and staff sessions expire after **12 hours**. At most 20 console sessions are retained; newer sessions can evict the oldest. A Setup admin can list and revoke sessions, and staff password reset, disable or role changes revoke that account's sessions. See the staff accounts guide.
- The two API calls `POST /api/staff/installer/sessions/revoke-others` and `POST /api/staff/installer/recovery-kit/regenerate` (Setup admin) end the other console sessions and make a new kit (new codes; the old codes stop working at once). The console has buttons for both: **Sign out other sessions** and **Regenerate recovery kit** in the Security card of the Station Profile screen, visible to the Setup admin role.
- For a lost password with no usable recovery codes, use the local administrator procedure below. Do not use `CIVICCAST_ALLOW_FIRST_ADMIN_RESET`: it reruns setup and can replace station settings.

**Administrator password reset** The computer's owner or an authorized local Windows administrator can reset the existing first administrator without the old password or recovery codes. This is an offline maintenance operation, not a web API. It preserves the administrator's username, station settings, PostgreSQL database, recordings and other media. It revokes every first-admin console session and all old recovery codes. Separately issued staff API tokens are unchanged; retire those separately if compromise is suspected.

1. Schedule a short outage and close any manually launched CivicCast server. On the station computer, open **PowerShell > Run as administrator**.
2. Stop the supervisor and run the installed command (replace `<INSTDIR>` with your installation folder):
   ```
   Stop-Service CivicCastSupervisor
   & "<INSTDIR>\runtime\python.exe" -I -m civiccast.cli admin reset-password
   ```
3. Enter the new password twice at the hidden prompts (12–256 characters). Never put it in command arguments, environment variables, a script or a ticket. Cancelling or mismatching the prompts changes nothing.
4. After success, run `Start-Service CivicCastSupervisor`, sign in with the displayed existing username and new password, then open **Station Profile > Security > Regenerate recovery kit**. Save the new kit safely; all previous recovery codes are invalid.

The command checks the actual elevated Windows token, the stopped supervisor's supported service identity, and its registered environment. It holds the supervisor's exclusive Windows mutex throughout recovery, so service starts and another recovery cannot write concurrently. It targets the protected `station-state.json` from the service environment or an absolute local `CIVICCAST_STATION_STATE_PATH`; the invoking user's profile and environment are ignored. Relative,

network, linked and junction paths are refused. Custom service identities are not supported by this recovery procedure.

Before changing credentials it creates an ACL-protected `*.before-admin-reset.bak` beside the state file, then replaces the state atomically. The backup contains old credential hashes and session hashes: keep it restricted, do not attach it to support requests, and remove it after confirming access and your normal station backup. Restoring it also restores the old credentials and sessions. A missing or corrupt state file is refused, not reinitialized. If recovery reports a failure, retain the backup, correct the reported condition and retry; restart the supervisor when maintenance is complete. No database password or DPAPI secret is read; the new password is never echoed, logged or stored in plaintext.

The first-admin setup routes still rely on loopback at the application. A custom reverse proxy on the station can make remote requests appear local, so it must refuse every `/api/setup/*` path. The supported Caddy configuration does this explicitly on both hostnames and limits staff-host forwarding to configured source CIDRs. Do not put another proxy or CDN in front of Caddy without separately verifying the address restriction. The Beta 10 audit's Host/Origin observation (E-006) remains historical; complete first setup on the station computer and use the generated Caddy configuration for remote access.

#### Know what the station exposes to the network

**What listens, and where**

| Service | Bind address | Port | Notes |
|---|---|---|---|
| Control plane (web API, console, portal) | `127.0.0.1` by default; `127.0.0.1` or `::1` in Network Settings | 8000 by default | Loopback-only upstream; it cannot bind directly to a LAN or public interface. Caddy is the supported remote HTTPS edge. |
| Postgres | `127.0.0.1` | first free of 5432, 5433, 5434, 5435, 5544 | Password logins only (`scram-sha-256`). No TLS. |
| Ollama | `127.0.0.1` | 11434 | Environment `OLLAMA_NO_CLOUD=1` is set for it. |

The application upstream remains loopback-only. In Caddy mode, the public hostname serves resident pages and public routes; a separate staff hostname forwards requests only from configured CIDRs, with bearer and role checks still required. See the network access guide.

**What the firewall rule opens** The installer adds a Windows Firewall rule named **CivicCast (Native) Portal/API (TCP 8000)**. It is an inbound allow rule for TCP port 8000, limited to the program `<INST-`

`DIR>\runtime\python.exe`, enabled on **all** firewall profiles (domain, private and public) and with no restriction on the remote address.

The TCP 8000 firewall rule does not make the loopback-only application listener directly reachable from other computers. Caddy listener and firewall exposure are configured separately; open only the addresses and ports allowed by station policy. The broad installer rule remains a historical Beta 10 audit finding (E-009); review it with your Windows administrator rather than publishing port 8000.

Outbound traffic is not restricted by CivicCast.

### Other web protections

- **CORS** (which web sites may call the API from a browser) is deny-all by default. Allow specific sites with `CIVICCAST_CORS_ALLOWED_ORIGINS` as a list of exact origins. A wildcard `*` is refused as a startup error.
- Responses carry a `Content-Security-Policy` that begins `default-src 'self'` and adds narrower rules (for example `object-src 'none'` and `frame-ancestors 'none'`), plus `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` and `Referrer-Policy: same-origin`. A response that already sets one of these headers keeps its own value.
- Native stations set `CIVICCAST_LAN_ONLY_STATION=1`. With it, the API documentation pages `/docs` and `/redoc` are off, and `/openapi.json` needs a staff token.
- `CIVICCAST_AUTH_ACK=1` only silences a start-up warning about authentication. It does not change who can sign in.

#### Know the public and staff parts of the API

The web API has distinct groups of paths. Which group a path is in decides who may call it.

| Path group | Who can call it | What it is |
|---|---|---|
| `/api/staff/*` | Staff hostname from an allowed CIDR in Caddy mode, plus a staff bearer token and role | Staff sign-in and operator actions. The public hostname does not forward this path group. |
| `/api/setup/*` | Station computer only; some post-setup actions also require a staff token | First setup, first-admin sign-in and recovery. Caddy refuses these paths. |
| `/api/public/*` | Anyone who can reach the public hostname or local listener | Resident features. |
| `/api/webhooks/stripe` | Stripe, with a signature | Payment events. |
| `/media/*` | Anyone | Media files for the portal. |

| Path group | Who can call it | What it is |
|---|---|---|
| `/ap/*` | Anyone | ActivityPub (federation) endpoints. Federation mode is `disabled` unless `CIVICCAST_ACTIVITYPUB_MODE` is set. |
| `/health`, `/api/health`, `/api/version`, `/api/hardware` | Anyone | Health, version and hardware summaries. |

The staff middleware gates every path that begins with `/api/staff/`. The public asset lists show only assets that are published.

> **Known issue (beta.11):** There is no setting that turns off the public contributor routes (`/api/public/contribute`); the router is always included. Anyone who can reach the port can use them. See the limitations section.

> **Known issue (beta.11):** `GET /api/hardware` needs no sign-in and returns the computer's host name (audit finding B-012). Treat the host name as public.

#### Configure HTTPS and remote access

The control plane speaks plain HTTP to a loopback-only upstream. CivicCast provides a generated Caddy configuration for the HTTPS edge; it separates resident pages from staff access, blocks `/api/setup/*`, and restricts the staff hostname to configured CIDRs. Follow the network access guide: review the generated Caddyfile, configure DNS and the approved firewall edge, and test from both allowed and denied networks. Do not expose port 8000 directly or put an unverified proxy/CDN in front of Caddy. The application does not install or manage the Caddy service.

Do not publish `/api/public/contribute*` while the upload limitations below apply. The staff bearer token and role checks remain required even on the approved staff network.

**Internal certificates are a different thing.** The station contains a local certificate authority for mutual TLS between its own internal programs (the identities `civiccast-api` and `civiccast-worker`). Certificates last 90 days and show `rotation_due` in the last 30 days. Renew one with `& $cc cert rotate <identity>` (add `--json` for scripts). The root folder is `CIVICCAST_CERT_ROOT`, default `.civiccast\certs` under the service account's home. These certificates are not the HTTPS certificate for a browser and cannot be used as one.

#### Protect subscribers and viewers

### Viewer analytics

- The portal's analytics beacons are stored **without** an IP address, a session identifier or a viewer identifier. A stored event has an event id, event name, time, app target, channel id, content id and a short list of safe properties (such as country, region, state, device, platform and caption

language). String values are truncated to 160 characters.
- A beacon is accepted for storage only when `CIVICCAST_PUBLIC_ANALYTICS_KEY` or `CIVICCAST_PUBLIC_ANALYTICS_ALLOWED_ORIGINS` is set. Otherwise the beacon is accepted and dropped, which is "telemetry off". A beacon larger than 16,384 bytes is refused.
- Stored events are kept for up to 366 days (`CIVICCAST_ANALYTICS_RETENTION_DAYS`, range 1 to 366).

### Subscribers

- A subscriber's e-mail address is stored encrypted with AES-GCM. Confirmation and unsubscribe links carry signed tokens.
- The encryption key and the token secret come from the pair `CIVICCAST_SUBSCRIBE_TOKEN_SECRET` and `CIVICCAST_SUBSCRIBE_ENCRYPTION_KEY` (set both or neither). If you set neither, the station creates a durable file `subscribe-secrets.json` (in `CIVICCAST_CONFIG_DIR` or `.civiccast` under the service account's home).
- Alert destination URLs and signing credentials are stored separately in `alert-credentials.json`. On Windows, both local secret files are created with a restricted file ACL before any secret bytes are written. When older files have broader access, CivicCast replaces them atomically with the same bytes; if it cannot secure the replacement, the operation fails and leaves the original file in place.
- The e-mail confirmation token is not shown in the API reply.

> **Warning:** Back up that key. A database restored without it keeps encrypted addresses that nobody can read (Chapter 12).

> **Known issue (beta.11):** The confirmation token for a webhook subscription **is** returned in the API reply (a known gap).

> **Known issue (beta.11):** By default the e-mail, webhook, YouTube, Internet Archive and NAS providers are mock or local providers. Real ones are chosen with `CIVICCAST_PROVIDER_<KIND>=real`, where the kind is `MAIL`, `WEBHOOK`, `YOUTUBE`, `INTERNET_ARCHIVE` or `LOCAL_NAS` (see Appendix C); choosing a real provider without its credentials makes the station fail fast at start-up. Until you switch the mail provider, no subscriber e-mail is actually sent.

#### Retain and delete data

| Data | Kept | Source |
|---|---|---|
| Live-caption working audio | Processed chunks are deleted. Whistle retains at most 42 waiting five-second segments per channel (210 seconds), plus in-flight inputs and the segment being written; runtimes without that processing grace use a 12-segment ceiling. Native live operation does not run the optional age-based retention sweep. | `captions/tap_worker.py`, runtime wiring |
| Live-caption text and delivery tracking (native beta.11 runtime) | Rolling 300 seconds and at most 512 cues; no permanent per-cue review rows or evidence WAVs | `captions/stabilize.py`, `egress/caption_feed.py` |
| Recorded-caption review and original recordings | Existing recording/review workflow remains unchanged; live working limits do not delete archived tracks or recordings | `captions/vod.py`, recording workflow |
| Historical beta.10 caption-retention audit | The beta.10 audit found that `caption-retention-audit.jsonl` was not rotated. Native beta.11 live captioning does not enable the optional age-based retention sweep. | `captions/retention.py`, `captions/tap_worker.py` |
| Analytics events | Up to 366 days (default; range 1 to 366) | analytics store |
| Playout health telemetry | Not trimmed automatically; trim with `civiccast egress trim-health --older-than-days N` | CLI |
| Staff token audit events, alert history, as-run records | No deletion setting found | not found |

Live captions are off by default, and `CIVICCAST_CAPTION_TAP=off` forces them off. The native runtime separates transient live caption work from recorded-caption review. It does not make the live worker or broadcast readiness depend on archive-wide review/evidence retention discovery. No live review/evidence archive is created by ordinary live captioning; this does not change recorded captions or original recordings.

> **Historical finding (published Beta 10):** Audit findings B-003, B-004, B-005 and B-009 recorded unbounded live-caption evidence growth. The current native live path does not run that archive sweep, writes no per-cue evidence, deletes processed chunks and bounds the queued audio and rolling caption history. These source behaviors do not replace package-specific verification; see the release record matching the installed package.

Where residents' own data may be requested for deletion, we found no console screen or command that deletes a subscriber or a viewer's records. Ask the coder to confirm before you promise a deletion process.

#### Check what you install: signing and checksums

- **The installer** is signed with Authenticode through Azure Trusted (Artifact) Signing. The policy file `CODE_SIGNING_POLICY.md` names the signer as Scott Converse. Windows SmartScreen may still show a reputation warning for a new publisher. To check a download:

```
Get-AuthenticodeSignature .\CivicCast-Setup.exe | Format-List Status,
 ↪  SignerCertificate
Get-FileHash .\CivicCast-Setup.exe -Algorithm SHA256
```

The status should read `Valid`. Compare the hash with the line for that file in `SHA256SUMS.txt` on the matching release page. Check that the installer signature is valid and its hash matches the published `SHA256SUMS.txt`. Replace the file name with the real one.

- **Sidecar.** The release also has a `setup.exe.sidecar.json` file with checksum metadata. It is plain metadata and is not separately signed (`attestation` is `null`). `civiccast installer verify-package --artifact <file> --sidecar <file> [--json]` checks a file against it.
- **Packs.** The large components (`.ccpack` files) carry ed25519 signatures that the installer verifies. The installer also checks a SHA-256 for each file, rejects names containing `..` and rejects reparse points (Windows links). The pack index is pinned to the product version, so an older pack cannot be offered to a newer product.
- **Build downloads** are pinned by hash, and the application's Python packages are installed with `--require-hashes`.
- **No sigstore/cosign.** The owner decided against it (decision record ADR 0022).

> **Known issue (beta.11):** All packs are signed by one key that has no expiry and no rotation procedure (audit finding E-012). If a pack were ever to be signed by a stolen key, the installer would accept it. Download beta releases only from the project's GitHub release page.

> **Historical beta.10 audit finding (repository settings):** The 2026-10-02 audit reported that signing secrets were not restricted to a protected environment, action versions used mutable tags and `main` was not protected. Those observations describe repository settings at the audit date, not their current state. For every release, trust only the official release page, verify the installer's signature and compare its hash with the release checksum file.

#### Historical beta.10 security audit findings

The entries below summarize the whole-repository audit dated 2026-10-02 (`audit-lite-whole-repo-beta10-2026-10-02`). They preserve that audit's findings and severities; later beta.11 code and installer changes mean they are not a current release review. Current configuration and safety instructions appear in the operating chapters above. Finding numbers are the audit's.

| Finding | What is wrong | What you do |
|---|---|---|
| B-001 (Critical) | An anonymous visitor's contributor upload is accepted and written to disk **before** the size limits are checked, so a flood of uploads can fill the disk. | Keep the public contributor portal off the internet. Do not forward `/api/public/contribute*` at a proxy. Use the upload limits (`CIVICCAST_CONTRIBUTOR_MAX_UPLOAD_BYTES`, `CIVICCAST_CONTRIBUTOR_UPLOAD_DIR_MAX_BYTES`, `CIVICCAST_CONTRIBUTOR_UPLOAD_PER_IP_MAX_BYTES`, `CIVICCAST_CONTRIBUTOR_UPLOAD_RATE_LIMIT` default 10, `CIVICCAST_CONTRIBUTOR_SUBMISSION_RATE_LIMIT`) as a second layer. Watch free space. |
| E-001, B-002 | A webhook subscription can be pointed at an internal address (server-side request forgery) and its signing secret is guessable. | Webhook delivery happens only when `CIVICCAST_PROVIDER_WEBHOOK=real`. Leave it unset unless you need it, and keep the public subscribe routes off the internet. |
| E-006 | The loopback setup routes do not check `Host` or `Origin`. | Finish first setup before the computer is used for anything else. Do not browse the web from the station computer. |
| Proxy pitfall (design, not an audit finding) | A custom reverse proxy on the same computer can make requests appear to come from loopback. | Use the generated Caddy rules: refuse `/api/setup/*`; allow staff paths only on the separate CIDR-restricted staff hostname. |

| Finding | What is wrong | What you do |
|---|---|---|
| E-007 (historical Beta 10 audit) | The audit recorded that service and children ran as LocalSystem. | Current installer source configures the restricted virtual account; an isolated Windows Sandbox run with the current source overlay verified profile and CivicCast credential migration, virtual-account startup, and a healthy API with the current schema. This is development-source evidence, not qualification of a rebuilt or signed installer or production cutover. See the service identity guide. |
| B-006 | Any signed-in local user can wedge the supervisor's control pipe. Playout is not affected, but the supervisor's status, start, stop and restart verbs stop answering. | Allow only administrators to log in to the station. Windows `sc.exe` still starts and stops the service. |
| E-003 | The installer opens log and URL paths through `cmd.exe /C start`; an `&` in such a path can run a command. Exploiting it needs earlier write access to the installer's state. | Do not run the installer from a folder whose name or content an untrusted person controls. |
| E-004, E-005 | The dependency vulnerability scan checks a lock file that differs from what ships (37 of 81 pins differ), and an allow-list entry has expired. | Treat the shipped Python packages as not independently scanned. Watch the release notes for updates. |
| E-009 | The firewall rule is broader than needed. | See "What the firewall rule opens". |
| E-011 | The embedded Python is 3.12.10. The audit states, from memory and not verified online, that this was the last 3.12 build with binary installers, so the shipped interpreter may not get further stdlib security fixes. | Keep the station off untrusted networks until a later release replaces it. |

| Finding | What is wrong | What you do |
|---|---|---|
| E-012, E-002 | One pack-signing key with no expiry; build secrets and branch not protected. | See "Check what you install". |
| B-010 | The failed-login limiter has no memory bound. | Keep staff access on the separate hostname restricted to approved CIDRs; retain bearer authentication and role checks. |
| B-012 | `/api/hardware` shows the host name to anyone. | Treat the host name as public. |
| B-019 | `civiccast model set-provider-key --key <value>` puts the key on the command line, visible to other programs in the process list. | Supply the key in the `CIVICCAST_PROVIDER_API_KEY` environment variable instead. |
| B-003, B-004, B-005, B-009 | Historical beta.10 caption data growth (see retention). | Beta.11 removes automatic live review/evidence accumulation and bounds live working state; verify the candidate before deployment. |
| A-001, A-008 | The audit reported a program-change watchdog gap and no free-space guard on the 60 GB cache. Beta.11 includes a rollover watchdog and cache free-space guard; their code behavior is described in Chapter 12. | The audit findings are historical; current runtime reliability still depends on exact-package evidence. |
| C-001 | Automated tests were red in about 110 places at the time of the audit. | Historical audit result only; it does not describe the beta.11 test state. |

**Note:** The beta.10 audit also discussed operational items such as alerts with no destination, the disaster-recovery drill's `pg_dump` dependency and log rotation. See their current operating guidance in Chapter 12.

**Warning:** Keep the staff hostname restricted to approved networks and bearer-authenticated accounts, do not publish contributor uploads while their listed limits apply, and run only the signed installer from the official release page.

#### Package status and verification

This chapter describes the unpublished Beta 20 candidate edition. Use the matching release record for package checks; the Beta 18 record tracks its earlier package. A historical Beta 15 field report found the installed `CivicCast Native.exe` unsigned while the outer setup was signed; this does not establish the status of another package. See the deferred field follow-up.

**Report a security problem** Report a suspected vulnerability privately, as `SECURITY.md` in the repository describes. Do not open a public issue. The policy states that a report is acknowledged within 72 hours.

#### If it did not work

| What you see | Cause | What to do |
|---|---|---|
| `No DATABASE_URL configured` or a database connection error from `civiccast token ...` | `DATABASE_URL` not set in this window | Set it as shown in "Before you start". |
| `Staff identity is required for this action.` (401) | No `Authorization: Bearer` header | Send the header. |
| `This action requires one of these CivicCast roles: ...` (403) | The token lacks the role | Issue a token with the role. |
| `Too many failed attempts to authenticate with the staff API. Wait N seconds, then try again.` (429) | 10 bad tokens in 60 seconds from one address | Wait. Check the token. |
| A token you issued is rejected | Revoked, rotated, or issued before the fingerprint change | `token list`, then `token rotate`. |
| Console says **First setup can only be done from the station computer itself** | You opened the console from another computer | Use the station computer or a remote-desktop session on it. |
| **You were signed out** | The oldest of 20 sessions was evicted, or the station's sign-in state was reset | Sign in again. |
| `Invalid admin username or password.` | Wrong credentials | Use the recovery kit's password or one of the 8 codes. |
| A signed installer shows `NotSigned` or a hash that does not match `SHA256SUMS.txt` | Corrupt or altered download | Do not run it. Download again from the release page. |

**Related**

- Signing in and finding your way around
- Running it day to day
- Configuring the station
- Cable headend, streaming, CDN, federation, emergency alerts, the API
- Troubleshooting matrix
- Appendix C: settings, Appendix F: roles, Appendix H: measured evidence

## Troubleshooting matrix

This chapter is for the person who looks after the CivicCast station computer: usually the one or two IT staff at a small city. It explains how to identify a problem from a screen, log line or command result and what to do. It is organised by area. Each area is a table with four columns: the **symptom** you see, the **likely cause**, **how to confirm** it, and the **fix**.

### Before you start

- **Who can do what.** Reading logs and running the commands below needs an Administrator account on the station computer; the service runs as the restricted virtual account `NT SERVICE\CivicCastSupervisor`. Console buttons mentioned in the matrix need a role: **Support admin** for the Readiness support bundle; **Setup admin or Support admin** for "Repair GStreamer runtime & restore full egress"; **Setup admin** for settings and emergency-alert feeds; **Meeting operator** to start and stop channels and to retry a recording's finalization; **Publish operator** for the Publish screen. The roles are listed in Appendix: roles.
- **Open PowerShell as Administrator** before running any command in this chapter. Commands are shown for PowerShell.
- **Times.** Log lines use the station computer's clock and have no zone marker. Schedule, Program Guide and new recording entries use the configured station time zone; older weekly recording schedules can retain their UTC timing. See Before the meeting before comparing a log with a planned time.
- **Changing an environment variable.** Many settings in this chapter are environment variables. Set them in the service's `Environment` registry value as described in Running it day to day, then restart the service with `Restart-Service CivicCastSupervisor` between programs and check the log lines named in each row.
- **Restarting the service interrupts the air.** `Restart-Service CivicCastSupervisor` stops every channel. Do it between programs, or on a channel nobody is watching.
- **Local setup, remote staff access.** First setup and first-admin recovery are local-only. Staff can sign in remotely through the configured staff hostname from an approved network. See Signing in.

### Where things are logged

Uninstall leaves `C:\ProgramData\CivicCast` in place. Setup uses a durable recovery record and a temporary copy in `install-transaction` for the specific files and station state it may change; successful commit or restoration removes that copy. If recovery fails, keep the record and backup in place.

| What you want | File or place | Notes |
|---|---|---|
| Did the Windows service start, restart or give up? | `C:\ProgramData\CivicCast\logs\supervisor.log` | The service itself. Rotates at 10 MiB, keeps 10 old files, and is written to disk on every record. |

| What you want | File or place | Notes |
|---|---|---|
| The application's own log (channels, captions, publishing, recording) | `C:\ProgramData\CivicCast\logs\control_plane-app.log` | Most of the log lines quoted in this chapter are here. Old files are named `control_plane-app.log.1` and up. |
| Raw output of the web server process | `C:\ProgramData\CivicCast\logs\control_plane.log` | Crashes before the application logger starts land here. |
| The database server | `C:\ProgramData\CivicCast\logs\postgres.log` | Contains startup and server output; rotates at 10 MiB and keeps 10 older files. |
| Why the supervisor stopped trying to start | `C:\ProgramData\CivicCast\STATION-START-FAILED.md` | Written after 3 failed starts in a row with the same error. Removed after the next successful start. |
| What setup did, step by step | `C:\ProgramData\CivicCast\install-progress.log` | Read it from the bottom. Find the last "begin" with no matching "returned". |
| Automatic setup restoration | `C:\ProgramData\CivicCast\install-transaction\` (`transaction.clixml`, temporary `backup\`, and `recovery-error.txt` when recovery reports an error) | Do not delete these while setup recovery is incomplete. Exit 136 means recovery could not be prepared or completed. |
| Which pack was missing or untrusted | `C:\ProgramData\CivicCast\install-manifest-report-<pid>-<time>.json` | One per setup run. |
| Upgrade, rollback, provisioning | `C:\ProgramData\CivicCast\upgrade\` (`upgrade-engine.log`, `upgrade-journal.json`, `UPGRADE-RECOVERY.md`) and `provision\` (`PROVISION-RECOVERY.md`, `OWNERSHIP-RECOVERY.md`, `ownership-observation.txt`) | |

| What you want | File or place | Notes |
|---|---|---|
| One channel's encoder and relays | `C:\ProgramData\CivicCast\data\egress\<channel>\logs\` (`gst-worker.stdout.log`, `gst-worker.stderr.log`, `hls-relay.<sink label>.stderr.log`) | On the default GStreamer engine the worker logs are named `gst-worker.*`. On the FFmpeg fallback engine they are `ffmpeg.stdout.log` and `ffmpeg.stderr.log`. |
| First-run wizard | `%USERPROFILE%\.civiccast\runtime-host.log` and `installer-state.json` | Per Windows user, not per station. |
| Windows' own record of the service | Event Viewer, Windows Logs, Application, source `CivicCastSupervisor` | Setup exit 126 sends you here. |

Two commands you will use constantly:

```
Get-Content C:\ProgramData\CivicCast\logs\control_plane-app.log -Tail 200
Select-String -Path C:\ProgramData\CivicCast\logs\control_plane-app.log* -Pattern
↪  "Caption tap"
```

And one to ask the running station how it is:

```
curl.exe http://127.0.0.1:8000/health
```

The answer is a short JSON document with `status` (`healthy` or `degraded`), `version`, `schema` (`current`, `behind`, `not-configured` or `unknown`), `live_captions`, `schema_db_revision`, `schema_expected_head` and `mode`. `status` is `healthy` only when the schema is current and live captions are neither degraded nor unknown. Disabled captions or no active channel do not themselves make readiness degraded. For caption failures, open authenticated System Health for channel details. The HTTP status is 200 whenever the web server process is alive, so read the words in the body, not the number.

#### Install and activation (setup exit codes)

Setup returns a number when it fails. Interactive setup shows a dialog with words but not the number; the silent run (`/S`) shows only the number. The full table is in Reference tables. On a failure, setup first puts the service, if it exists, into "stopped, manual start", so a failed install does not leave a half-built station running. Running setup again is the supported recovery.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Setup stops at exit 110, or a dialog names `.ccpack` files | A required pack is missing or not trusted. A first install needs the `packs` and `station` folders beside `setup.exe`. | `install-progress.log` contains a `step stage-packs: child reported:` line naming the pack; the JSON `install-manifest-report-*.json` has the detail. | Put the full kit (the `packs` and `station` folders, with the files named in the log) beside `setup.exe` and run setup again. |
| Setup stops at exit 111, 112, 121 or 122 | After unpacking, the server tools, the program files, FFmpeg or Ollama do not match their signed list. The copy of the kit is damaged. | The last "begin" in `install-progress.log` is a verify step. | Replace your copy of the kit from the original download and run setup again. |
| Setup stops at exit 123 | Activation failed. The child code says which part: 64 arguments, 65 could not write the report, 66 pack missing or untrusted, 67 activation or self-test failed, 78 the embedded trust key was refused. | `install-progress.log` ends with `step d4-activate-station: returned 67` (or the other child code). | See the next two rows for exit 67. For 66 see exit 110. |
| Exit 123 child 67 on a computer that is short of disk | The same code is used for "not enough free disk space" and for unpacking failures. Activation needs the total size of the packs plus 2 GB of headroom. | The setup details pane shows "Not enough free disk space to activate this station…" | Free disk space and run setup again. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Exit 123 child 67 with enough disk | A self-test check failed. The checks include: the Python core imports; PostgreSQL, FFmpeg and Ollama report their pinned versions; TSDuck, if present; a CPU caption test on a sample file (300 second limit); and the three AI models answering (wait limit 300 seconds). | The failing check is named in the setup details pane while setup is open. | Keep the details pane text and the log, then run setup again. If it fails twice for the same reason, collect the items under "What to collect before asking for help". |
| Exit 116 or 117 | PostgreSQL could not be provisioned. | `provision\PROVISION-RECOVERY.md` and the end of `install-progress.log`. | Follow `PROVISION-RECOVERY.md`; run setup again. |
| Exit 127 | Setup cannot tell which CivicCast runtime owns this machine. | `provision\OWNERSHIP-RECOVERY.md`. | If there is no older WSL product, set `HKLM\SOFTWARE\CivicCast\ActiveRuntime` to the text `native`, then run setup again. |
| Exit 135 | The older WSL edition of CivicCast is still registered. | The dialog says so. | Uninstall it, or run its `cutover-to-native`, then run setup again. |
| Exit 118, 119 or 120 | Service registration failed (118), the firewall rule could not be made (119), or the old service could not be stopped before install (120). | `install-progress.log`. The dialog for 118 and 119 names no log path. | Run setup again as Administrator. For 120, stop the service in `services.msc` first. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Exit 125 | The service started but the station did not become ready to serve. The dialog names the database schema as the usual cause. | `install-progress.log` and `upgrade\upgrade-engine.log`; then `logs\control_plane.log`. | Fix the cause the logs name, then run setup again. Your data is untouched. |
| Exit 126 | The service is registered but Windows could not start it. | Event Viewer, Application log; `logs\`. | Fix the start-up error it names, then run setup again. |

> **Known issue (beta.11):** The exit-67 dialog says the failing check is named in `install-progress.log`, but the log contains only the activation return code. The child's detailed error appears in the setup details pane. If the setup window has closed, the failing check is lost; capture the details pane before closing it.

> **Known issue (beta.11):** Exit 67 also covers insufficient disk space and extraction errors, although the dialog says "NOT a missing-files problem". Check free disk space and the details pane before you chase a self-test.

> **Known issue (beta.11):** Interactive failure dialogs explain the step but do not show its numeric exit code. Read `install-progress.log` for the code; the installer table in Chapter 10 lists the current codes.

> **Historical Beta 10 observation:** A clean-install run on an earlier package observed the first playout packets more than 60 seconds after the first start, while the channel reported its fallback slate as connected. That timing does not predict startup on later packages. Use `/health`, the channel's output and the verification record matching the installed package.

#### Sign-in

The **First Setup** screen (`#/setup`) is for creating the first administrator and for that administrator to sign in or recover access. Staff use the separate sign-in page at `/login`. When Caddy remote access is configured, staff can sign in through the staff hostname from an approved CIDR; first-admin setup and sign-in remain local to the station.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| "Invalid admin username or password." | Wrong username or password. | The red alert on the card. | Use the username and password from the recovery kit. If the password is lost, use **Use recovery code**: it consumes one of the 8 one-time codes and sets a new password of 12 or more characters. |
| "Invalid staff username or password." | The username or password is wrong, or the account is disabled. | The alert on the Staff sign-in page. | Check the credentials. Ask a setup administrator to confirm the account is enabled or reset its password in **Staff accounts**. |
| "Too many staff sign-in attempts. Wait and try again." | Staff sign-in is rate-limited after repeated failed attempts. | HTTP 429 and the message on the Staff sign-in page; the response includes `Retry-After`. | Wait, then try again with the correct credentials. |
| "Too many sign-in attempts from this station. Wait N seconds, then try again…" | The setup sign-in limiter is cooling down. | The text names the wait. | Wait the number of seconds shown, then use the correct password or a recovery code. |
| "Too many failed attempts to authenticate with the staff API. Wait N seconds, then try again." | 10 bad tokens in 60 seconds from one client address. A request with no token at all does not count. | The text (the console writes it itself), and an HTTP 429 from the API. The API's own `detail` is "Too many failed staff authentication attempts. Wait and retry." with a `Retry-After` header in seconds. | Wait. The limit is set by `CIVICCAST_AUTH_RATE_LIMIT` (default 10) and `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS` (default 60). |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The card "First setup can only be done from the station computer itself" | You opened the console from another computer, or through a remote-viewer that is not running on the station. | The page is HTTP 403. | Sign in on the station computer itself, or in a remote-desktop session on it. The sign-in and recovery routes accept only loopback requests. |
| Notice "You were signed out", or "This browser's console sign-in is no longer valid" | The station rejected the token (HTTP 401). Most often this browser's session was the oldest and was dropped because the station keeps only 20 console sessions. | The notice appears after a screen change. | Sign in again. Nothing is wrong with the station. |
| A screen says "This action requires one of these CivicCast roles: …" | The token lacks a role (HTTP 403). | The text lists role ids such as `meeting_operator`. | Sign in as someone who holds the role. A Setup admin can assign the required roles in **Setup > Staff accounts**. |
| The browser shows its own error page for `http://127.0.0.1:8000/operator/` | The service is not running, or this is not the station computer. | `Get-Service CivicCastSupervisor` | See "Station will not start". |

> **Password recovery:** If the password and all eight recovery codes are lost, use the offline local-administrator `civiccast admin reset-password` procedure in Security and privacy. Stop the supervisor first as instructed there. Do not use `CIVICCAST_ALLOW_FIRST_ADMIN_RESET` for password recovery: it reruns first setup and can replace station settings.

#### Station will not start

The service is `CivicCastSupervisor` ("CivicCast Native Supervisor"). It runs as the restricted virtual account `NT SERVICE\CivicCastSupervisor`, starts automatically, and Windows restarts it after 5, 10 and 30 seconds if it stops. It starts two programs of its own, PostgreSQL and then the control plane (the web server and station engine), plus an optional Ollama AI engine on `127.0.0.1:11434`. The control plane gets 180 seconds to become ready, PostgreSQL gets 60 seconds, and a stop gives each child 15 seconds before it is ended.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| `Get-Service CivicCastSupervisor` says Stopped, and the console will not load | The supervisor gave up. After 3 failed starts with the same error it writes a marker file. | `C:\ProgramData\CivicCast\STATION-START-FAILED.md` exists. In `supervisor.log`: `station could not start: <ErrorType>: <detail>` and `consecutive failed start count is now N`. | Read the reason in the marker file. Fix it, then `Start-Service CivicCastSupervisor`. The marker is removed after a successful start. |
| The marker file's reason is "Native station activation self-test receipt does not match this distribution" | `station-set.json` or `activation-self-test.json` in the install folder is missing, stale or from another build. The service will not start without them. | The reason text. | Run setup again so activation re-creates them. Do not edit either file by hand. |
| `supervisor.log` says `singleton not acquired (<status>): <detail> -- another supervisor owns the station` | A second copy of the supervisor is running. | The line itself. | Stop the other copy. Check `Get-Service` and Task Manager for a second `CivicCast` process. |
| State is `blocked_wsl_active` or `blocked_probe_unavailable` | The older WSL edition is active, or the check for it could not run. | `supervisor.log`. | Remove or cut over the WSL edition (see exit 135 above). |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| `curl.exe http://127.0.0.1:8000/health` returns `"status":"degraded"` | Database readiness or live-caption processing needs attention; captions can degrade readiness even with a current schema. | Read `schema` and `live_captions`; authenticated System Health shows channel details. | For `behind`, see "Upgrade problems"; for schema `unknown` or `not-configured`, see "Database problems". For caption `unknown` or `degraded`, inspect the affected enabled on-air channel's worker activity and fallback state. This probe does not certify every output or device. |
| `supervisor.log`: `ffmpeg/ffprobe not staged at ... (degraded media handling, ...)` | FFmpeg is missing from the install folder. The station runs, but media preparation is degraded. | The line. | Run setup again to restore the files. |
| `supervisor.log`: `ollama child skipped (degraded AI, service healthy): ...` | The AI engine could not start. Summaries and translation are unavailable; the station is otherwise healthy. | The line. Port 11434 belongs to CivicCast's own Ollama. | Check that another program is not holding `127.0.0.1:11434`. Run setup again if the files are damaged. |
| `supervisor.log` (level ERROR): `... degrading egress to the FFmpeg concat engine (CIVICCAST_EGRESS_ENGINE=ffmpeg-concat) so the channel keeps airing.` | The GStreamer files are damaged. The station tried an in-place self-repair, it did not restore them, and the station switched to the older FFmpeg concat engine. | The line. | On **Readiness**, press **Repair GStreamer runtime & restore full egress** (Setup admin or Support admin). If that fails, run setup again. |

> **Warning:** Do not hand-edit `station-set.json`, `activation-self-test.json` or `STATION-START-FAILED.md`. Re-running setup rebuilds them.

#### Channels will not start, or sit on the slate

First, the words. The console shows an egress state in plain words: **Stopped**, **Starting**, **On air**, **Changing source**, **Showing slate**, **Finishing current item**, **Stopping** and **Needs attention**. They correspond to the internal states STOPPED, STARTING, ON_AIR, TRANSITIONING, FALLBACK_SLATE, DRAINING, STOPPING and ERROR. The log line that records every change is:

```
channel <id>: egress state -> <STATE> (source=..., pid=..., last_error=...)
```

The `last_error` part is the reason. Search the application log for it: `Select-String -Path C:\ProgramData\CivicCast\logs\control_plane-app.log* -Pattern "egress state ->"`.

> **Note:** A Beta 15 field run found that some worker labels and process/PID status did not agree with fresh process evidence. The reason was not established; use both the station logs and Windows process state when investigating, and do not treat this as a confirmed Beta 16 defect. See the Beta 17 deferred follow-up.

A channel that shows **Showing slate** is on air with the "technical difficulties" picture instead of its program. The station chose that rather than dead air. It is a symptom, not a fault in itself. The `last_error` tells you which of these it is:

| `last_error` text (start of it) | What it means | Fix |
|---|---|---|
| "Preparing the scheduled program; airing the fallback slate so the channel stays up while it conforms." | The next program is not yet in a form that can air (it is being conformed: re-encoded to the station's standard picture and sound). A cold file can take minutes. | Wait. If it happens every time, look at "Program change problems". |
| "No valid source plan is available; generated fallback slate." | Nothing is scheduled and nothing else is available to air. | Schedule something, or check **Auto-schedule** and the Program Guide. |
| "Scheduled source plan expired during preparation; aired fallback slate while automation resolves the current schedule." | The preparation took longer than the item's slot. | Automation resolves it on its own. If frequent, see the preparation times below. |
| "Live source failed repeatedly; aired fallback slate instead of an infinite crash-loop." | The real source failed 5 times in a row without staying healthy for 60 seconds. Automation tries the real source again about every 30 seconds. | Look at `gst-worker.stderr.log` for the source's own error. Fix the source (network stream, file, capture device). |
| "caption storage refused: ..." | The channel could not store caption data, usually a full disk or a permissions problem. | See "Disk full". |

| `last_error` text (start of it) | What it means | Fix |
|---|---|---|
| "egress encoder unavailable; aired fallback slate: …" | No video encoder could start. See "GPU and encoder problems". | See that section. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| **Needs attention** (ERROR) | The start failed and even the slate could not be encoded, or the configuration is invalid or a secret could not be resolved. With no FFmpeg at all there is no path to air anything. | `egress state -> ERROR ... last_error=...` in the application log. | Fix the named cause, then press **Start** again. |
| The channel stays **Starting** and then restarts | The engine produced no output. The worker quits itself so the daemon can restart it. | `gst-worker.stderr.log`: `CTRL first-output: no output within 45s of PLAYING - quitting for daemon restart`. The limit is `CIVICCAST_GST_FIRST_OUTPUT_TIMEOUT_S`, clamped to between 10 and 120 seconds. | Check the source and encoder rows. If it is only the first start on a new install, wait: see the Known issue above on slow first packets. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| On air, then silently stops sending | A running worker stopped producing output. | `gst-worker.stderr.log`: `CTRL stall: no output for 10s - quitting for daemon restart`. Application log: `channel <id>: worker exited (exit_code=..., state=..., desired=..., pending_reload=..., deliberate_kill=...)`. | The daemon restarts the worker itself. Look for repeated exits. |
| The picture is frozen but the audio goes on | Video-only freeze. | `gst-worker.stderr.log` lines of the form `CTRL output: <total> buffers (+delta) since PLAYING [mux-in 5.0s: video=+0 audio=+235]`: `video=+0` with audio climbing. | The channel heals the web stream first. After that, see the next row. |
| Log level CRITICAL: `Freeze escalation budget exhausted ... moving to the fallback slate for a bounded cooldown before one program retry` | The HLS stream stayed frozen after relay recovery and the hourly restart budget is spent. | Channel state, recovery slate and the CRITICAL log. | CivicCast requests fallback slate unless an operator override is active, then makes one programme retry after the cooldown ends one hour after the latest budgeted restart. Check output and logs; investigate if recovery cannot proceed or the retry fails. Do not assume the warning itself means a manual restart is required. |

**Program change problems**

A program change is the moment a channel moves from one scheduled item to the next. CivicCast tries to do it without a break: it prepares the next item ahead of time and swaps the source at the boundary (a "seamless reload"). If that fails it retries, and then falls back to a restart or to the slate. In the 8-hour run on build C16, 50 program changes happened with no holes. Those numbers come from an earlier internal build, not the published files.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| A change was late by a few seconds, and the log shows a retry | A first-attempt abort at a very short tail boundary. | `Boundary reload for <channel> did not land (...); re-resolving and re-arming (retry N of 2) ...` | None needed. In the lab it healed in about 3 seconds. After the 2 retries are spent: `... retries were spent; arming the fallback slate at the boundary...` |
| The channel restarted at a change instead of swapping | The seamless swap did not land. | `Seamless content-reload for <channel> did not land (...); falling back to restart.` | Look at the `(...)` reason. A restart means a short interruption, not a failure. |
| The slate came up for several minutes at a change | Preparation of the next item was cold and slow. | `Content-reload source preparation for <channel> took N. Ns for N segment(s)`, or `... preparation FAILED ... falling back to restart.` In the lab the longest preparation times were 449 seconds (C15) and 389 seconds (C16). | A 3 to 5 minute slate after a short plan followed by a long item was seen on an earlier internal build, and a look-ahead warm-up was added afterwards. Make sure the next items are in the conform cache (see "Loudness"). |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The log says the rollover is “past due” | The change should have happened already. | `Channel automation rollover horizon for <channel> is past due by Ns but the seamless reload is still settling; waiting.` or `Channel automation rollover reload for <channel> did not land within Ns (current_proof_event_id unchanged); retrying.` | These are the system retrying. Escalate only if they repeat for many minutes. |
| The channel is pinned at **Changing source** | Reload stall. | `Channel <id>: reload stall -- pinned TRANSITIONING for Ns with nothing in flight ... Re-issuing the rollover now`; if that fails: `reload stall not cleared by the re-issue ... Restarting the channel's egress.` | The watchdog does both steps itself. If it is still stuck afterwards, stop and start the channel. |
| The worker said the commit did not finish | The reload commit did not complete. | `gst-worker.stderr.log: CTRL reload: commit did not finish.` | The daemon falls back. |
| A single dropped frame at a join between two pieces | A known single-frame drop. | `CTRL mux diagnostic: dropped future-dated video arrival.` | None. It is one frame. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| A change aired only the tail of the closing item | The plan was resolved against a short remainder. | `Plan for <channel> resolved to a N. Ns tail of the closing scheduled item.` | Check the schedule's station time zone and the slot's planned start and end. |

> **Historical Beta 15 observation:** An initial Beta 15 candidate showed repeated rollover attempts after a 20-second Welcome clip ended inside a 60-second slot. The corrected Beta 15 package passed both the 20-second and 60-second slot checks and returned to slate. The Beta 15 verification record preserves both results; they do not establish Beta 16 package behavior.

> **Note:** A Beta 15 field run found scheduled speech windows on three channels showing slate with silent audio and header-only captions. The cause was not established, and the observation does not identify Whistle as the cause or establish Beta 16 behavior. This is a Beta 17 follow-up; see the deferred field report.

> **Historical Beta 10 observation:** An unexplained rebuild appeared during the eight-hour C16 development run. Its cause was not established; that observation is not evidence about the current package. Keep the logs if you see an unexplained rebuild.

#### No audio, low audio, loudness

CivicCast conforms every program to a loudness target. There are three targets: streaming at -16 LUFS, ATSC A/85 at -24 LKFS (the cable headend standard in the United States) and EBU R128 at -23 LUFS. The headend presets use A/85. For speech, a "leveling ride" raises quiet stretches by up to 18 dB over 4-minute windows. The default public-stream target is -16 LUFS plus or minus 1.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Speech sounds uneven across a long meeting | The leveling ride failed and the item was conformed with the simpler loudnorm filter instead, which does not hold the level across every 4-minute stretch. | `Speech leveling ride failed for '<name>' (window: ...): ... Conforming with loudnorm instead, which does not hold the level across every 4-minute stretch.` | Re-record or re-upload if the source is damaged. Otherwise accept it; the item still aired. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| A program is noticeably quiet and the log mentions the loudness gate | The source is quieter than the ride can lift. 18 dB of lift reaches a source down to about -35 LUFS; below that, the window is listed in the report as out of reach and is not counted as a failure. | `Speech leveling for '<name>' (window: ...): the kept attempt misses the loudness gate (worst 4-minute stretch N LU, whole program N LU).` | Fix the recording level at the microphone. A check on one public window read -19.4 LUFS and passed on re-check; that is the quiet-source class. |
| A true-peak error in the log | A peak exceeded the hard bound. | An ERROR line from the loudness module. | Collect the log and the asset id. The program still aired. |
| Conform runs again and again for the same file | The conform cache is too small, so files are evicted and rebuilt. The cache size is `CIVICCAST_CONFORM_CACHE_GB`, default 60. The older default of 20 caused thrash: one 4.4-hour asset took about 11 GB. | `Conform-cache warm failed for '<name>' ... retrying after Ns` reports a failed attempt. Retry delay starts at 60 seconds and doubles up to 30 minutes; success resets it. Upcoming-plan work is prioritized over opportunistic warming. `0` turns the cache off. | Keep the setting at its default or raise it, and keep the disk free. In the 8-hour run 46 GB of the 60 GB was used. |
| Silence on air, picture present | The source has no audio track, or the audio leg of the worker failed. | `gst-worker.stderr.log`: in the `CTRL output:` lines, `audio=+0`. | Check the source file or capture device. |

> **Known issue (beta.11):** There is no free-space guard for the 60 GB conform cache. Watch free space. See “Disk full”.

#### Captions late or missing

There are two separate caption paths. **Live captions** are off by default in beta.11. Enabling **Show live captions on air** in Station Profile starts speech recognition on the next tap-worker scan; stop and start each channel before expecting captions in its video. Turning the switch off stops recognition and drains queued audio on the next scan; caption routing is removed when each channel next starts.

The service setting `CIVICCAST_CAPTION_TAP=off` overrides the profile and keeps the tap off. **Recorded captions** run after publication and use the separate review workflow; see the Publishing and Recording sections.

**Live captions and recovery** The native live runtime uses Whistle on CPU by default. It publishes the first recognition without waiting for another reading to agree, and serializes primary inference across channels on the station. A Whistle request has a 30-second deadline. If it fails or times out, Whisper supplies backup captions while CivicCast automatically starts a fresh Whistle process for a recovery attempt. Successful recognition switches that channel back to Whistle. Whisper is also the recorded-caption engine. Supported NVIDIA CUDA acceleration is optional for Whisper; Whistle uses the CPU.

Whisper fallback allows 120 seconds per request by default. `CIVICCAST_WHISPER_FALLBACK_TIMEOUT_SECONDS` accepts finite values from 1 to 600 seconds. This deadline permits slower CPU responses to finish while still containing a stalled process. It does not expand the bounded audio queue. Caption status distinguishes temporary delay from sustained overload and paused recognition.

The audio tap has bounded queues and can discard older working audio or pause caption recognition under sustained overload to keep playout responsive. If captions lag, pause, or show gaps, use the table below to check the logs and reduce competing CPU or disk work.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| No captions on the live picture at all | Live captions are off by default, the service override is `off`, or the current channel has not started since the switch was enabled. | Station Profile checkbox **Show live captions on air**; the **Channels** caption status card; confirm `CIVICCAST_CAPTION_TAP` is not `off`. | Enable the profile switch; recognition resumes on the next worker scan. Then stop and start the channel so its video includes captions. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| `Whistle failed for <channel>; switching to Whisper` appears | Whistle failed or missed its 30-second deadline; that channel is using backup captioning during recovery. | Check the engine and recovery countdown on **System Health**; search `C:\ProgramData\CivicCast\logs\control_plane-app.log*` for the failure and recovery messages. | CivicCast retries Whistle after 30 seconds, with further failed probes backing off to at most 300 seconds. Successful recognition restores Whistle automatically. A failed Whisper child gets one immediate replay, then a 30-second cooldown after another failure. Bounded failure diagnostics are retained for investigation. |
| Captions have gaps and the log says `Caption tap catch-up ... Discarded` | Recognition fell behind and the bounded tap shed old working audio to preserve playout. | `Select-String -Path C:\ProgramData\CivicCast\logs\control_plane-app.log* -Pattern "Caption tap"`. | Reduce other heavy work on the station and watch the output. An NVIDIA GPU is optional for Whisper and does not accelerate Whistle. Do not treat a gap-free run as established until it has been observed under your workload. |
| Caption status shows delayed, overloaded or paused | Delayed means the queue is temporarily behind; overloaded indicates sustained backlog; paused means the tap has engaged its protective pause. | Check queue depth, queue age and engine recovery on **System Health**, and search the control-plane log for `Caption tap overload`. | Let a temporary delay catch up. For persistent overload, reduce competing CPU/disk work; caption recognition resumes according to its backoff. Restart the channel only if its caption state does not recover. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The caption status card says “Not verified” or “Caption proof failed” | The decode-back proof is absent, stale or failed. | Check the card on **Channels** and search the control-plane log around the same time. | Check the live-caption rows above and the channel logs. |
| Captions on H.265 channels | The caption inserter is H.264 only on native Windows. | The log: “HEVC/H.265 cannot embed captions on native Windows – the caption inserter is H.264-only…”. | Use H.264 for channels that must carry captions. |

Ordinary live captioning does not create permanent per-cue review records or evidence WAVs. Temporary working audio is cleaned up as it is processed; bounded live cues are not a substitute for caption review of the original recording. Recorded-caption review continues to use the Review queue.

#### Recording failures

Scheduled recording runs inside the station (`CIVICCAST_SCHEDULED_RECORDING`, `inline` or `off`; it looks for new jobs every 10 seconds and plans 600 seconds ahead). Recordings are made by FFmpeg, one process per job.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The recording job shows “ffmpeg exited before recording started for job …” | The input could not be opened. | The text in the job’s **Failure** column. The channel logs. | Check the input address or capture device and its permissions. |
| “ffmpeg did not create a recording segment for job …” or “ffmpeg created a zero-byte recording file for job …” | The input opened but delivered nothing, or the target folder is not writable. | The Failure column. | Check the input is live, and that the folder has space and is not read-only. |
| “No local recording target is configured.”, “No production local recording target is configured; only the installer rehearsal target was available.” or “No usable local recording target is configured.” | No recording folder is set up. | The Failure column. | Configure a local recording target; see Configuring the station. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| "Recording input preset … is not available for … Open Recording and select a detected or configured input preset." | The chosen input is not on this machine. | The Failure column. | Choose a detected input on **Recording**. |
| "Capture file … does not exist." or "… is empty." | A capture that ended with no media. | The Failure column. | Check the capture source. |
| "Unsupported scheduled recording encoder profile" | The encoder profile name is not one of: copy, default, inherit, h264-1080p, hw-h264-1080p, h264-720p, hw-h264-720p. | The text. | Pick one of those. |
| An amber banner "Scheduled recording runtime is unavailable in this deployment." (HTTP 503) | `CIVICCAST_SCHEDULED_RECORDING` is `off`, or the runtime did not start. | The banner. | Set it to `inline` and restart the service. |
| HTTP 403 on the recording schedule | `CIVICCAST_STATION_ID` does not match (default `civiccast-station`). | The response. | Set the matching value. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| A recording finished, but the Live room or assets show a failed finalization | The post-recording worker failed. The failure codes are `recording.never_appeared` (waited 1800 seconds), `recording.not_local`, `probe.failed`, `finalize.invalid_trim`, `package.failed`, `cdn.upload_failed`, `worker.interrupted` and `internal.error`. | `GET /api/staff/live/finalizations` or the Live room. | Press **Retry finalization** in the Live room, or call `POST /api/staff/live/sessions/{id}/finalization/retry`. The worker makes 3 attempts by default (`CIVICCAST_FINALIZATION_MAX_ATTEMPTS`), waiting 30 seconds before the second and doubling the wait after each further failure (`CIVICCAST_FINALIZATION_BACKOFF_SECONDS`). The retry call needs the Meeting operator role and answers 409 if the finalization is running or already complete. |
| The schedule ran at the wrong hour | The station time zone or the entry's planned time may be wrong. | Compare the planned time, station zone and computer clock. | Correct the Station Profile zone or schedule entry, then check the next occurrence. |
| A recording alert, `scheduled-recording-failure` or `scheduled-recording-dropout` | A job failed, or the input dropped out during the job. | **Alerts** screen. | See the rows above. |

**Known issue (beta.11):** **Stop** is also offered on a recording job that is still in the `scheduled` state (not yet started). The station refuses it with HTTP 409 ("Cannot stop job … only ['arming', 'finalizing', 'recording'] are stoppable"), and the screen does not show the error, so nothing seems to happen. **Stop** does work on a job that is arming, recording or finalizing. To stop a job that has not started from running, the usual way is to untick **Enabled** on its schedule; we did not confirm that this also cancels a job that already exists.

> **Known issue (beta.11):** Creating a recording schedule gives no confirmation, and creating the same one again returns 409 "already exists". Check the list before you try again.

#### Publishing problems

The Publish screen records "operator-dashboard" as the operator for every approval. Publish targets are the resident portal, Internet Archive, Local NAS (rsync or ZFS), YouTube Live, YouTube VOD, a podcast episode, subscriber notifications and a cable file package. Podcast and subscriber notifications are optional and show their outcomes when selected.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Approving fails with "Publish preflight blocked: this asset has no manifest_url. Run the packager or fix ingest before approving publish." (HTTP 409) | The recording has not been packaged for the web yet. | The error text. | Wait for packaging, or retry finalization (see above). |
| "Publish blocked: CivicCast cannot queue this recording's caption job … Nothing was published." (HTTP 409) | The caption job for this recording could not be queued. | The error text. | Check disk space and the caption pack. Then approve again. |
| A target shows a success message but nothing arrived at YouTube or the archive | Providers default to **mock**. Each of `CIVICCAST_PROVIDER_INTERNET_ARCHIVE`, `_LOCAL_NAS`, `_YOUTUBE`, `_MAIL` and `_WEBHOOK` is `mock` unless set to `real`. | Check the environment variables. | Set the variable to `real` and add the credentials listed in Cable headend, streaming, CDN, federation, emergency alerts, the API. |
| A target you did not tick was reset on a second approval | Approving again rebuilds the whole run and resets unticked targets. | The Publish screen. | Tick every target in one approval. |
| The cable file package row says "not set up (optional)" | `CIVICCAST_CABLE_PACKAGE_OUTPUT_DIR` is not set. | The row. | Set it (see Chapter 15) and approve again. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| A subscriber did not receive a new-recording notice | The subscriber is not confirmed, the optional surface was not selected, or its provider is mock or failed. | Check the subscriber-notification row and its delivery list. | Confirm the subscription, configure the provider, then retry eligible deliveries. Federation is a separate feature and is off by default. |

> **Known issue (beta.11):** The Publish screen does not always say when a mock provider did the work. An Internet Archive or Local NAS row carries a "Simulated" note, but the card's headline can still read "Archive verified" (HELP-02), and a YouTube row from the mock provider shows a success message with no simulated note (HELP-03). Before you rely on a publish target, send one test and look at the far end.

> **Known issue (beta.11):** Approving a recording can announce it to federation followers as a hidden side effect when ActivityPub is on. See Chapter 15.

> **Known issue (beta.11):** A second approval resets any target you left unticked (HELP-01). Tick everything in a single approval.

#### The portal does not show video

Residents can reach the portal through the configured public Caddy hostname. Staff use the separate hostname restricted to approved CIDRs. The application itself remains bound to loopback at `http://127.0.0.1:8000/`; the installer firewall rule does not make that listener directly reachable from another computer. Follow the network access guide and test from both allowed and denied networks.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The live page says offline or "standing by" | `curl.exe http://127.0.0.1:8000/api/public/live/current` reports a state: `offline`, `on_air`, `on_air_no_web_output` or `standing_by`. | Read `state` and `reason` in the answer. The reasons are "no HLS output configured", "HLS output configured but not serving yet" and "fallback slate, no program on air". | The first needs an `hls` output on the channel (see Chapter 15). The second means the web stream has not started: see the HLS relay row. The third is the slate: see "Channels will not start". |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The channel is on air, but the web stream is empty | The HLS relay did not start or died. | Application log: `HLS relay up for <channel>`, `HLS relay could not start for <channel> ... ffmpeg is not available`, `HLS relay for <channel> ... restarting the relay child`. The relay's own log is `data\egress\<channel>\logs\hls-relay.<sink label>.stderr.log`. | Restore FFmpeg by running setup again; stop and start the channel. |
| A recording page has no video | The recording has no `manifest_url`, so it was never packaged, or the web copy failed. | Assets screen status; finalization codes above. | Retry finalization. |
| Recordings play on the station but are slow or missing for the public | The CDN selector is on and uploading failed (`cdn.upload_failed`). | Failure code in the finalization list. | Check the CDN settings in Chapter 15. Review the matching package record's scope, then test one recording and verify it at the CDN before relying on that path. |
| Video is not found at all at `/media/live/<channel>/playlist.m3u8` | The channel has no HLS output, or is not running. | `curl.exe -I http://127.0.0.1:8000/media/live/<channel>/playlist.m3u8` | See above. |

> **Known issue (beta.11):** The `finalization-worker-runbook` in the repository says the local media base address defaults to `http://127.0.0.1:8000`. In the code the default is empty, which means addresses relative to the site. We follow the code.

#### Disk full

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Alert `disk-low`: "Low free space: media X GB (min 20)" | Free space on the media or backup drive fell below the limit. The default thresholds are 20 GB for media and 20 GB for backup. | **Readiness** shows "Media space" and "Backup space" in GB free. | Free space. The conform cache (up to 60 GB), old recordings and caption data are the large items. |
| Setup refuses with "Not enough free disk space to activate this station..." | Activation needs the total of the pack sizes plus 2 GB. | The setup details pane. | Free space. |
| Uploads fail with HTTP 507 on the contribute page | The drive is full. | The response. | Free space. |
| Disk fills slowly over weeks | Recordings and logs accumulate, and the conform cache can grow to 60 GB with no check on free space. Ordinary live captions do not create a permanent audio-evidence archive. | Folder sizes under `C:\ProgramData\CivicCast\data`. | Plan the disk size for the long term; see Planning your station. |

> **Known issue (beta.11):** The anonymous contributor upload spools a file to disk before any size limit applies, and can fill the drive (audit finding B-001). If the contribute page is open to the public, watch free space closely.

#### Database problems (and why there is no NATS)

CivicCast used to have a message broker called NATS. It was removed (decision 0023, owner decision of 2026-08-20). An in-process event bus replaced it. There is no NATS child to start, to check or to repair. If an old note tells you to look for one, ignore it.

PostgreSQL listens on `127.0.0.1` on the first free port of 5432, 5433, 5434, 5435 and 5544, chosen when the station was provisioned. The control plane finds its database address from the registry value `HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl`, readable by SYSTEM, Administrators and `NT SERVICE\CivicCastSupervisor`, unless the environment variable `DATABASE_URL` overrides it.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| **Readiness** machine health row says the database is **Unreachable**, or alert `db-unreachable`: "Database is not reachable from the host." | PostgreSQL stopped or cannot be reached. | `logs\postgres.log`. | Restart the service. If it fails again, read the PostgreSQL log for the cause (disk full, damaged files). |
| Alert `service-down`: "The CivicCast egress service is not running." | The egress engine is not running. | The alert. | Check `supervisor.log`. |
| Log: `DATABASE_URL source: environment override ...` or `... registry (HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl)` | This is an informational line that says which source won. | The line. | If an old `DATABASE_URL` machine variable points at a database that is gone, remove it. |
| A command-line tool says "DATABASE_URL must point at the CivicCast database before running staff token lifecycle commands." | The `civiccast` command line did not find the database address. | The message. | In an Administrator PowerShell, set `$env:DATABASE_URL` from the registry value for that shell only. |
| `/health` shows `schema` as `behind` | The database has not been migrated to this program's version. | `curl.exe http://127.0.0.1:8000/health` | See "Upgrade problems". |

#### GPU and encoder problems

The default encoder path is GStreamer. On Windows, a hardware H.264 request is mapped to the Media Foundation encoder (`mfh264enc`), and the default H.264 software encoder is `openh264enc`. HEVC (H.265) needs hardware; there is no software HEVC in this build.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| The channel does not air its program (it shows the slate, or ends in **Needs attention**). The message "No hardware video encoder was found on this machine. To broadcast on the CPU instead (slower), turn on 'Allow software (CPU) encoding fallback' in this channel's settings, then start the channel again. HEVC/H.265 needs hardware and is not available this way." appears in `last_error`, usually after `egress encoder unavailable; aired fallback slate:` | The pre-flight probe found no hardware encoder. | `last_error` on the channel, and the application log. | Either turn on **Allow software (CPU) encoding fallback** in the channel's settings, or install the drivers for the graphics card. |
| "No hardware HEVC/H.265 encoder is available on this machine. HEVC requires a hardware encoder – there is no software HEVC in this build…" | An H.265 channel on a computer with no hardware HEVC encoder. | The text. | Use H.264. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Log warning: "No hardware video encoder was found; 'allow_software_fallback' is on, so this channel is encoding on the CPU (openh264enc). This is slower and may not keep up with live on a weak CPU." | Software fallback is on. | The warning. | This is informational. On a weak CPU, expect trouble in "Program change problems" and "Captions". |
| Log: `egress encoder unavailable (...); falling back to the technical-difficulties slate rather than dead air.` | Neither the GStreamer nor the FFmpeg encoder could start for the program, so the slate is being encoded instead. | The line, and `last_error` "egress encoder unavailable; aired fallback slate: ...". | Fix the reason after the colon. |
| The channel is in ERROR with no slate | FFmpeg is absent from the machine, so even the slate cannot be encoded. | `last_error` is a message about FFmpeg not found. | Run setup again to restore the files. |
| Live GPU captions will not run | The graphics card has less than 8 GB of video memory, is not NVIDIA, or the CUDA pack is missing. | See "Captions". | See "Captions". |

#### Upgrade problems

> **Upgrade evidence (Beta 12):** One normal upgrade from the pinned Beta 11 baseline exited 0 and recorded a healthy service/API, current schema 0089, a saved preservation marker, and initial HLS audio/video plus Whistle observations on three channels. The post-upgrade recorder failed during its first JSON save; no five-snapshot PASS was recorded. Fresh-install and repair checks remain unrun, and no sustained-operation result is claimed. Take a separate backup of `C:\ProgramData\CivicCast` before an upgrade. See the Beta 12 verification record.

An upgrade is the same `setup.exe` run over an existing install. The generic **Already Installed** page is suppressed; setup does not uninstall first. The installer first checks versions without stopping the service or replacing files. For an accepted upgrade, setup saves a temporary, size-checked snapshot before replacing files, re-verifies packs, and runs the database upgrade. If a later step fails, the

engine rolls back its database work and the enclosing installer transaction attempts to restore the prior application and station state. Exit 136 means recovery could not be prepared or completed; keep `install-transaction` and station data intact and let the recovery task or next setup run retry.

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Setup exit 113 | The engine could not restore its database backup. Setup also attempts to restore the pre-setup application and station snapshot. | `install-progress.log`; when present, failed-run engine logs and journal under `install-transaction\failure-diagnostics`. | If outer restoration succeeds, the prior station state is back. A generated `UPGRADE-RECOVERY.md` may not remain under `upgrade` after that restore. If recovery did not finish, keep the recovery files and station data. |
| Setup exit 114 | This release has a database change that cannot be rolled back, so the automatic upgrade was refused. | The dialog. | No manual path is written in the dialog (HELP-107). Ask for help before you proceed. |
| Setup exit 115 | An unexpected fault in the upgrade step. | `install-progress.log`. | Run setup again; collect the logs if it repeats. |
| Setup exit 124 | The engine rolled back its database work; setup attempts to restore the pre-setup application and station state. | `install-progress.log` and, when present, the failed-run engine log and journal under `install-transaction\failure-diagnostics`. | Read the reason, fix it, then run setup again. If restoration succeeded, the previous service configuration is restored too; keep recovery records if it did not. |
| Setup exit 128 | A terminal failure journal from an earlier upgrade remains; when installer restoration succeeds, it returns the pre-setup state and preserves that earlier journal. | `upgrade\upgrade-journal.json` and any retained copy under `install-transaction\failure-diagnostics`. | Inspect and archive or move the prior terminal journal aside (do not delete it), then run setup again. |

| Symptom | Likely cause | How to confirm | Fix |
|---|---|---|---|
| Setup exit 129 | An older setup was refused. The normal read-only version preflight exits before stopping the service or replacing files; a defensive late check also attempts pre-setup restoration. | The dialog and `install-progress.log`. | Use the setup matching the installed version. If the late recovery did not complete, keep the recovery files and station data. |
| Setup exit 136 | The installer could not prepare, complete or resume automatic restoration. | `install-transaction\recovery-error.txt` and `install-progress.log`. | Keep station data and recovery files. Restart Windows or retry setup to let automatic recovery run; contact support if it repeats. |
| After the upgrade `/health` says `schema` is `behind` | The migration did not run. | `curl.exe http://127.0.0.1:8000/health`. | Run setup again; see exit 125. |
| Uninstall exit 82, 133 or 134 | The service could not be confirmed stopped (82, 133), or teardown is incomplete (134, remove by hand). | The dialog. | Stop the service in `services.msc` or restart Windows, then uninstall again. For 134, remove the leftovers in `services.msc`, Windows Firewall and the registry. |

#### What to collect before asking for help

Collect these before you ask anyone for help. Together they answer most questions.

1. **The time and what you saw.** Write down the clock time with its zone, the channel, and the words on screen. Say whether anyone changed a setting just before.
2. **The whole `C:\ProgramData\CivicCast\logs` folder** (`supervisor.log`, `control_plane-app.log`, `control_plane.log` and `postgres.log`, including numbered rotated copies), copied while the problem is fresh. Rotation can overwrite old lines.
3. **`C:\ProgramData\CivicCast\install-progress.log`**, the `upgrade` and `provision` folders, and any `STATION-START-FAILED.md` or `install-manifest-report-*.json`, for install and upgrade problems.
4. **The channel's own logs**: everything in `C:\ProgramData\CivicCast\data\egress\<channel>\logs\`.

5. **The health answer**: the output of `curl.exe http://127.0.0.1:8000/health` and of `Get-Service CivicCastSupervisor`.
6. **The version**: it is in the health answer, and the installed version is recorded in the registry value `InstalledVersion` under `HKLM\SOFTWARE\CivicCast\Native`.
7. **The support bundle**, if the console still loads: on **Readiness**, as a Support admin, create the support bundle. It is a JSON file named `<bundle id>.json` in a `support-bundles` folder next to `station-state.json` (the folder can be moved with `CIVICCAST_SUPPORT_BUNDLE_DIR`), and the console offers it for download. It holds the version, platform, which settings are present (without their values), setup, storage and health information, alerts, recent samples and the tail of some logs. Lines that contain a secret marker are dropped.
8. **Command-line status**, if you can run it: `runtime\python.exe -m civiccast.cli runtime status` and `... doctor`, using the `runtime` folder in the install folder. We have not run these on a station; the code supports them. The command list is in Appendix: command line.

> **Known issue (beta.11):** The support bundle does not collect the logs most useful for a GStreamer station. It reads `%USERPROFILE%\.civiccast\runtime-host.log` and each channel's `ffmpeg.stdout.log` and `ffmpeg.stderr.log`, while the default GStreamer engine writes `gst-worker.*` logs. `supervisor.log` and `control_plane-app.log` are not included; add them by hand (steps 2 and 4).

> **Note:** Use **Report a beta issue** for the project issue tracker, or follow the no-GitHub option in Chapter 8. Do not post a support bundle publicly; wait for a maintainer to request it through a private channel.

#### Reference tables

### Setup exit codes

| Code | Step | Meaning |
|---|---|---|
| 82 | Uninstall | The service stop was not confirmed. |
| 110 | Stage packs | A required pack is missing or untrusted (child code 74), or the optional GPU pack is untrusted (child code 75). |
| 111 | Verify | The unpacked server tools do not match their signed list. |
| 112 | Verify | The unpacked program files do not match. |

| Code | Step | Meaning |
|---|---|---|
| 113 | Upgrade | The engine could not restore its database backup; the installer also attempts full pre-setup restoration. See `install-progress.log` and any retained `install-transaction\failure-diagnostics`. |
| 114 | Upgrade | The release has a migration that cannot be rolled back; the automatic upgrade was refused. |
| 115 | Upgrade | An unexpected fault. |
| 116 | Provision | PostgreSQL could not be provisioned. See `PROVISION-RECOVERY.md`. |
| 117 | Provision | An unexpected fault. |
| 118 | Service | The Windows service could not be registered. |
| 119 | Firewall | The inbound rule could not be created. |
| 120 | Pre-install | The existing service could not be stopped, or its state could not be read. |
| 121 | Verify | FFmpeg does not match. |
| 122 | Verify | Ollama does not match. |
| 123 | Activation | Child codes 64, 65, 66, 67 and 78, or another. |
| 124 | Upgrade | The engine rolled back its database work; setup attempts to restore the prior application and station snapshot. |
| 125 | Service | The service started but the station is not serving (child code 84). |
| 126 | Service | The service would not start (child code 83). |
| 127 | Provision | Runtime ownership unknown (child code 85). See `OWNERSHIP-RECOVERY.md`. |

| Code | Step | Meaning |
|---|---|---|
| 128 | Upgrade | An earlier failed upgrade's journal is still on disk. |
| 129 | Upgrade | An older setup was refused; the normal version preflight runs before service stop or file replacement. |
| 130 | Uninstall | You declined the ownership-transfer prompt. |
| 131 | Uninstall | Ownership transfer failed. |
| 132 | Uninstall | Blocked: an active runtime together with a WSL product, or unreadable state. |
| 133 | Uninstall | Service stop unconfirmed; files kept. |
| 134 | Uninstall | Teardown incomplete; remove the leftovers by hand. |
| 135 | Provision | The older WSL edition is registered (child code 87). |
| 136 | Installer recovery | The automatic restoration transaction could not be prepared or completed. Keep `install-transaction` and station data intact. |
| 3010 | Visual C++ | The Visual C++ runtime installed and Windows needs a restart. We could not confirm that setup itself returns this number. |

Child codes seen: 64 arguments, 65 report write, 66 pack missing or untrusted, 67 activation or self-test, 68 verify, 73 and 74 uninstall preflight, 75 provision or pack, 78 embedded trust key refused, 83 service start, 84 not serving, 85 ownership, 87 other product. A child code of 77 appears in the setup code but its meaning is not named in any text we read. A first install with only `setup.exe` and no kit stops at exit 110 (see the first row of the install table).

**Log line index**

| Quote this fragment | Where | Section |
|---|---|---|
| `channel <id>: egress state -> <STATE>` | `control_plane-app.log` | Channels |

| Quote this fragment | Where | Section |
|---|---|---|
| `Boundary reload for ... did not land` | `control_plane-app.log` | Program changes |
| `Seamless content-reload for ... did not land` | `control_plane-app.log` | Program changes |
| `reload stall -- pinned TRANSITIONING` | `control_plane-app.log` | Program changes |
| `Freeze escalation budget exhausted` | `control_plane-app.log` | Channels |
| `CTRL first-output: no output within 45s` | `gst-worker.stderr.log` | Channels |
| `Caption tap is behind` | `control_plane-app.log` | Captions |
| `Caption tap catch-up ... Discarded` | `control_plane-app.log` | Captions |
| `Caption tap overload ... PAUSED` | `control_plane-app.log` | Captions |
| `Speech leveling ride failed` | `control_plane-app.log` | Loudness |
| `HLS relay could not start` | `control_plane-app.log` | Portal |
| `TS relay up for` and `TS relay failed to start` | `control_plane-app.log` | Chapter 15 |
| `degrading egress to the FFmpeg concat engine` | `supervisor.log` | Station will not start |
| `station could not start:` | `supervisor.log` | Station will not start |
| `singleton not acquired` | `supervisor.log` | Station will not start |

#### Rotation and source-preview checks

- If a channel stops after the last scheduled program, check that its items are published and that `CIVICCAST_SCHEDULE_LOOP=1` is set in the service environment for a repeating rotation. The loop preserves gaps: adjoining entries need contiguous start times and durations. See the continuous scheduling recipe.
- If the trim or summary source preview cannot play, confirm the original file remains available and package the recording for HLS playback. Use the preview's retry or renewal action after correcting the source. A summary citation outside a trimmed copy needs a package covering that passage; it should not be reviewed against a different time range.
- If **Publish** counts items needing action, its **Needs action** filter includes required blocked outputs as well as failed outputs. Open each listed next step to resolve it.

#### If it did not work

If nothing here matches, do the following in order. First, run `curl.exe http://127.0.0.1:8000/health` and write down the answer. Second, read the last 200 lines of `control_plane-app.log` and `supervisor.log` and look for lines at WARNING, ERROR or CRITICAL level. Third, if the station is not on air and you are able to restart it, restart the service between programs. Fourth, collect the items in "What to collect before asking for help". Do not delete anything under `C:\ProgramData\CivicCast`; it holds the database and every recording.

#### Package status and verification

This chapter describes the unpublished Beta 20 candidate edition. For package-specific results, use the record matching the installed version. The Beta 18 record tracks the earlier package; the Beta 15 verification record applies only to the corrected signed Beta 15 package.

#### Related

- Signing in and finding your way around
- Running the meeting
- Publishing, and what residents see
- When something looks wrong
- Installing, first run, upgrading, uninstalling
- Running it day to day
- Cable headend, streaming, CDN, federation, emergency alerts, the API

## Cable headend, streaming, CDN, federation, emergency alerts, and the API

This chapter is for the IT person who connects CivicCast to the outside world: the cable company's equipment, a content delivery network, other servers on the internet, weather and emergency feeds, and other software that wants to talk to the station. It describes current source behavior and identifies connections that remain **off by default**.

### Before you start

- **Who can do what.** Channel and headend settings need **Setup admin**. Publishing needs **Publish operator**. The roles are listed in Appendix: roles.
- **The station service stays on loopback.** The control plane listens on `127.0.0.1` port 8000. Remote resident and staff access uses the supported Caddy setup with separate public and CIDR-restricted staff hostnames; do not expose the application listener directly. Everything in the "HTTP API" section below is reached from the station, through an approved remote session, or through those configured paths.
- **Do not put a same-host reverse proxy in front of the setup routes.** The station decides that a request came from the station itself by looking at the address it came from. The code comments say that if a reverse proxy is ever placed in front of the app, the check "would need to move". A proxy on the same computer connects from the loopback address, so it would make every outside request look local. Do not forward `/api/setup` through a proxy. A proxy on a different computer cannot reach `127.0.0.1:8000` at all.
- **Environment variables.** Most switches here are environment variables. The code comments name the service's `Environment` registry value as the place to set them, and the station's processes inherit the supervisor's environment; the steps are in Running it day to day, under "Set environment variables for the service". Restart the service afterwards with `Restart-Service CivicCastSupervisor` in an Administrator PowerShell. We did not run that on a station. Restarting the service stops every channel; do it between programs.
- **Times.** Schedule, Program Guide and new recording entries use the station's configured time zone. Log lines use the station computer's clock and have no zone marker; keep that distinction in mind when comparing a schedule with a log.
- **Beta status of the interfaces.** Nothing in the code or in the repository's documents promises that any interface in this chapter will stay the same from one release to the next. Treat all of them as beta.

### Remote Contribution controls

Remote Contribution creates rooms only for configured, enabled egress channels. A guest publishes from VDO.Ninja into MediaMTX over WHIP. CivicCast reads each admitted guest from the relay over RTSP, uses its bundled FFmpeg to tile the guest pictures and mix their audio, publishes the combined feed back to the relay over SRT, then takes that feed through its existing live-source and channel egress path. **Put on air** and **Off air** control this route; the last guest off air returns the channel to scheduled output. The channel handoff is confirmed against the egress on-air proof before the action reports success.

The director iframe also sends targeted guest camera and hangup requests. VDO.Ninja does not acknowledge these commands, so CivicCast records them as **sent, not verified**. The channel audio-

mute control changes CivicCast's own mix as well as sending a director request. Verify connection and camera state in the director. Closing a room removes its on-air guests and stops new invitations; a guest's connection record remains until an operator verifies that they have left.

Commission the media relay and service environment before inviting guests:

In **Remote Contribution > Diagnostics**, a Support admin can select **Install VDO and media relay**. The service account needs Git for Windows on its PATH to fetch the pinned VDO.Ninja checkout. CivicCast verifies that checkout's commit before activating it, then verifies the pinned MediaMTX Windows binary and MIT license before activating the relay. The relay starts under CivicCast supervision when Remote Contribution is enabled; its RTSP and SRT listeners bind to loopback. The VDO source still needs its HTTPS site and signalling service, and the relay's WHIP listener needs an HTTPS reverse proxy. Configure WebRTC candidates for the public host and allow the media UDP port through the station firewall. Keep the public WHIP route restricted to your intended guest use; the installed relay config accepts publisher-created stream paths.

| Setting | Purpose |
|---|---|
| `CIVICCAST_REMOTE_CONTRIBUTION=on` | Enables remote-contribution supervision and broadcast routing. |
| `CIVICCAST_REMOTE_CONTRIBUTION_VDO_URL` | Browser-accessible HTTPS base URL of the self-hosted VDO.Ninja instance. |
| `CIVICCAST_REMOTE_CONTRIBUTION_MEDIAMTX_URL` | Browser-accessible HTTPS base URL of MediaMTX's WHIP listener. |
| `CIVICCAST_REMOTE_CONTRIBUTION_RTSP_URL` | Optional RTSP override; managed relay default is `rtsp://127.0.0.1:8554`. |
| `CIVICCAST_REMOTE_CONTRIBUTION_SRT_URL` | Optional SRT override; managed relay default is `srt://127.0.0.1:8890`. |
| `CIVICCAST_MEDIAMTX_COMMAND` | Optional relay command override. If omitted, CivicCast starts the verified managed MediaMTX install. |
| `CIVICCAST_VDO_COMMAND` | VDO.Ninja server command when CivicCast supervises that separate process. |
| `CIVICCAST_TURN_HOST`, `CIVICCAST_TURN_PORT` | Reachable TURN service used by guests behind firewalls. |

The managed MediaMTX configuration allows WHIP publishing and local RTSP and SRT access to publisher-created paths. Keep the RTSP and SRT listeners local to the station. The browser-facing WHIP listener needs the station's HTTPS reverse proxy and network routing. Test one guest on the station's output before a public meeting. If the relay is absent or the mix fails to produce video and audio, **Put on air** returns an error instead of marking the guest live.

#### Cable headend: UDP transport streams

A *headend* is the cable company's room of equipment that receives your channel and puts it on the cable. Many small stations send the headend a *UDP transport stream* (a continuous stream of MPEG

packets sent over the network). CivicCast does this with an output of kind `udp-ts` and a destination address of the form `udp://host:port`.

Agree the stream settings with your cable company, select the matching preset below, and use **Readiness** to inspect the local output proof. The cable company's receiving equipment must also be configured for that stream.

**Headend presets** A preset fills in the picture and sound settings and the stream rate for a type of receiver. Apply one from the channel's settings on the **Channels** screen. The same function is available as `POST /api/staff/egress/channels/{id}/config/headend-profile`, and `GET /api/staff/egress/headend-profiles` lists them.

| Preset | Picture and sound | Stream rate | How it is sent |
|---|---|---|---|
| `generic-udp-spts` | H.264, 720p at 30 frames a second, 5000 kbps; AC-3 sound at 192 kbps | 8000 kbps | UDP, one receiver (unicast) |
| `comcast-mtd-sd` | MPEG-2, 720x480, a keyframe every 15 frames, 3180 kbps | 3750 kbps | UDP, multicast |
| `comcast-mtd-hd` | H.264, 1080p at 30, 10000 kbps; AC-3 at 384 kbps | 12000 kbps (a placeholder rate) | UDP, multicast |
| `telvue-hypercaster-ip` | H.264, 720p at 30, 5000 kbps | 8000 kbps | Unicast; the port must be 1024 to 65535 |
| `harmonic-spectrum-ts` | H.264, 1080p at 30, 8000 kbps | 10000 kbps | UDP, unicast |
| `leightronix-file-drop` | H.264, 720p | none | A file drop, not a live stream |
| `local-rehearsal-hls` | Adds a web (`hls`) output only | not applicable | The folder must be under the egress work folder or the folder named by `CIVIC-CAST_LIVE_HLS_ROOT`; network (UNC) paths are refused |

Things the presets enforce: the loudness target for headend outputs is ATSC A/85 at -24 LKFS; the datagram size is 1316 bytes (seven 188-byte transport packets); a multicast preset refuses a unicast address. Applying a preset reports when the change takes effect on a running channel: `restart_queued`, `restart_required`, `next_start` or `unchanged`. The GStreamer output for `udp-ts` turns on automatic multicast for a multicast address and needs an explicit port.

**The transport-stream relay** When the encoder is restarted, for example at a program change that cannot be spliced, a plain UDP stream starts over: the headend sees the stream's counters and clock reset and can lose the picture. To avoid that, CivicCast puts a small relay program, TSDuck's `tsp`, between the encoder and the headend. The relay never restarts while the channel is on, so the headend sees one continuous stream.

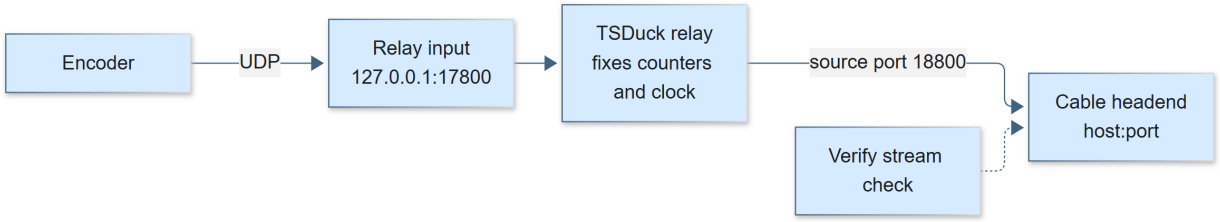



Figure: the path of a `udp-ts` output with the relay on. The encoder sends to a loopback address on the station; the relay forwards to the headend you configured.

The encoder never talks to the headend directly when the relay is on. It sends to `127.0.0.1` on a listen port. The relay listens there, repairs the continuity counters and the program clock reference, and sends the stream to the headend address you configured, always from the same source port. There is one relay for each pair of channel and `udp-ts` destination. Listen ports start at `CIVICCAST_TS_RELAY_BASE_PORT` (default 17800) and go up by one for each relay, and ports are reused when a relay is freed. The source port the headend sees is the listen port plus 1000, so the first relay sends from port 18800. If the cable company filters by source port, tell them these numbers.

The relay is controlled by `CIVICCAST_TS_RELAY`:

| Value | Meaning |
|---|---|
| `auto` (default) | Use the relay if TSDuck is found; otherwise send directly from the encoder. |
| `on` | Use the relay; if TSDuck is missing, log an error and send directly. |
| `off` | Never use the relay. |

The relay applies to `udp-ts` outputs only, on every encoder engine. The code looks for TSDuck's `tsp.exe` in this order: the file named by `CIVICCAST_TSDUCK_PATH`; the managed copy in `%LOCALAPPDATA%\CivicCast\tsduck` (or the folder in `CIVICCAST_TSDUCK_HOME`); the copy in the server-binaries pack at `payload\tsduck\bin\tsp.exe`; then your `PATH`. The version pinned for the product is TSDuck 3.44-4676. TSDuck is optional: the installer's self-test records its absence as a note and continues.

What the log says (in `C:\ProgramData\CivicCast\logs\control_plane-app.log`):

| Log fragment | Meaning |
|---|---|
| `TS relay up for <channel>: 127.0.0.1:<port> -> <host>:<port> (pinned source port <port>).` | The relay is running. |
| `TS relay auto mode: TSDuck (tsp) not found — udp-ts output is direct from the encoder; relaunches reset the TS session at the headend (#151). Install TSDuck to enable seamless splices.` | No TSDuck, so no relay. Each encoder restart resets the stream at the headend. |
| `CIVICCAST_TS_RELAY=on but TSDuck (tsp) is not available — ...` | You forced it on and TSDuck is missing. |
| `TS relay failed to start for <channel> -> <destination>; udp-ts falls back to ...` | The relay program would not start; output went direct. |
| `TS relay skipped for <channel>: udp-ts uri ... has no host:port.` | The destination address has no port. |

> **Known issue (beta.11):** The comments at the top of the relay's code describe it for the older FFmpeg concat engine. The code that starts it runs for every engine, including the default GStreamer engine. If you read the source, trust the behavior above.

**Check the stream with the verify probe** The station can capture a few seconds of what it is sending and test it with TSDuck. Use the **Verify stream (TSDuck)** button on **Channels** (Setup admin; shown only when the channel has a `udp-ts` output), the command `civiccast egress verify --channel-id <channel> --seconds 10`, or `POST /api/staff/egress/channels/{id}/compliance-probe`. It runs for about 10 seconds. The checks are: constant mux rate (within 5 percent), transport-stream sync, continuity, the program tables, a program clock reference present, and a single program. Without TSDuck the verdict is `not-run`. `GET /api/staff/egress/headend-readiness` reports whether TSDuck is available and the last results, and `POST /api/staff/egress/headend-device-probe` tests whether a headend appliance answers on a TCP port.

A passing check means the stream is well formed. It does not prove that the cable company's receiver accepts it, and the verify result itself says what it does not claim.

> **Note:** The 8-hour run on build C16 reports 41 "verify checks" (40 OK, 1 raw failure). Those were the lab's own captions, timing and playback checks, not a TSDuck stream test and not a cable-headend test. We could not confirm that the run used a `udp-ts` output at all, so it is not evidence for this section.

#### Web streams: HLS

*HLS* is the way video reaches a web browser: a playlist file (`playlist.m3u8`) and a rolling set of 2-second video pieces. The resident portal's live page uses it.

The shipped GStreamer runtime has no HLS writer. So for an output of kind `hls`, the code redirects the encoder's feed to a loopback UDP address `udp://127.0.0.1:<port>`, where the port is 18000 plus a number derived from the output's address (range 18000 to 18499, base set by `CIVICCAST_HLS_RELAY_BASE_PORT`). A supervised FFmpeg process reads that feed and writes the video pieces (`seg%09d.ts`) and `playlist.m3u8`, with a window of about 12 seconds (6 pieces). The playlist is served at `/media/live/<channel>/playlist.m3u8`; recordings are served at `/media/vod/<asset>/...`. The relay's own log is `C:\ProgramData\CivicCast\data\egress\<channel>\logs\hls-relay.<output label>.stderr.log`.

> **Note:** A Beta 15 field run found a synthetic CG playlist listing segment files that returned HTTP 404. The serving cause was not established, and the observation does not establish Beta 16 behavior. This is a Beta 17 follow-up; see the deferred field report.

| Log fragment | Meaning |
|---|---|
| `HLS relay up for <channel>` | The web stream is being written. |
| `HLS relay could not start for <channel> ... ffmpeg is not available` | FFmpeg is missing. Run setup again to restore it. |
| `HLS relay for <channel> ... restarting the relay child` | The FFmpeg process stopped and is being restarted. |

What residents see is decided by `GET /api/public/live/current`. Its `state` is one of `offline`, `on_air`, `on_air_no_web_output` or `standing_by`, and the `reason` can be "no HLS output configured", "HLS output configured but not serving yet" or "fallback slate, no program on air". `/api/public/channels/<channel>/live.m3u8` redirects to the live playlist, and `/api/public/channels/<channel>/captions.vtt` serves the current live captions.

> **Note:** Live captions are off by default in a new station profile and can have gaps under load. See the captions section of the troubleshooting matrix.

#### CDN and provider options

A *CDN* (content delivery network) is a hosting company that serves your recordings from many places so that the station's own internet line is not the bottleneck. CivicCast can upload finished recordings to one. The selector is `CIVICCAST_CDN_PROVIDER`:

| Value | Meaning |
|---|---|
| `off` (default) | No CDN. Recordings are served from the station. |

| Value | Meaning |
|---|---|
| `bunny` | Bunny. Variables: `CIVICCAST_BUNNY_STORAGE_ZONE`, `CIVICCAST_BUNNY_ACCESS_KEY`, `CIVICCAST_BUNNY_CDN_HOSTNAME`. |
| `cloudflare_r2` | Cloudflare R2. Variables: `CIVICCAST_R2_ACCOUNT_ID`, `CIVICCAST_R2_ACCESS_KEY_ID`, `CIVICCAST_R2_SECRET_ACCESS_KEY`, `CIVICCAST_R2_BUCKET`, `CIVICCAST_R2_PUBLIC_BASE_URL`. |
| `fastly` | Fastly Object Storage. Variables: `CIVICCAST_FASTLY_REGION`, `..._ACCESS_KEY_ID`, `..._SECRET_ACCESS_KEY`, `..._BUCKET`, `..._PUBLIC_BASE_URL` (each starting `CIVICCAST_FASTLY`). |
| `akamai` | Akamai (Linode) Object Storage. The same five variables, starting `CIVICCAST_AKAMAI`. |
| `stub` | A stand-in that writes files to a local folder (`CIVICCAST_CDN_STUB_ROOT`); the code marks it for tests and proofs only. |

The variables are listed in `docs/ops/cdn-and-providers.md` and confirmed in the provider factory code. Set the selector and that provider's variables in the service environment, and restart the service. If the selected provider's variables are missing, the code stops the application at start-up with a message naming them. When a recording is finalized, the finalization worker uploads its HLS files and sets the recording's `manifest_url`. A failed upload shows up as the failure code `cdn.upload_failed` (see Recording failures).

There is a separate *surge switch* that would send the live stream to the CDN during heavy viewing. It is **off** unless `CIVICCAST_LIVE_SURGE_THRESHOLD` is set. When it is used, `CIVICCAST_LIVE_SURGE_BUFFER_SECONDS` (default 15, capped at 30) sets the delay.

If a CDN or other server sits in front of the portal and you use the analytics features, set `CIVICCAST_ANALYTICS_TRUSTED_PROXY_CIDRS` to the address ranges of that server so viewer addresses are counted correctly.

After configuring a CDN, publish one recording and open it at the destination to confirm its credentials, destination path and playback settings.

**Publishing providers** Separate from the CDN, each publishing destination has a provider setting that is either `mock` or `real`:

| Variable | Destination |
|---|---|
| `CIVICCAST_PROVIDER_INTERNET_ARCHIVE` | Internet Archive |
| `CIVICCAST_PROVIDER_LOCAL_NAS` | A mounted local folder or UNC path |
| `CIVICCAST_PROVIDER_YOUTUBE` | YouTube Live and YouTube video upload |
| `CIVICCAST_PROVIDER_MAIL` | Email |
| `CIVICCAST_PROVIDER_WEBHOOK` | Webhooks |

All of them default to `mock`. A mock provider does not send anything. The selectors and adapter credentials are read from the service environment. Values saved by the Setup Provider form to `provider-credentials.json` are not copied into that environment and do not enable a real adapter; see service environment configuration for how IT sets values and Provider setup and keys for the form's separate role.

The real Internet Archive and local archive adapters use a resolved recording file when available. If it cannot resolve, the fallback is an asset/title payload rather than the full media. The local archive adapter writes a canonical copy and dated snapshot to a mounted filesystem path and checks the files by SHA-256 read-back; it does not run rsync or manage a ZFS target. YouTube VOD needs a local recording file or `CIVICCAST_YOUTUBE_MEDIA_ROOT/<asset_id>.mp4`. YouTube Live creates and binds a YouTube broadcast/stream, but the Publish path does not connect CivicCast's live output to the returned ingest address.

`CIVICCAST_PROVIDER_MAIL=real` is used by email signup/confirmation and optional notices from the Publish screen. `CIVICCAST_PROVIDER_WEBHOOK=real` enables webhook notices for confirmed subscribers. The staff-only subscription dispatch-test route can exercise either provider. Publishing records notice outcomes and retries eligible failures. Podcast publishing uses local FFmpeg, `CIVICCAST_PUBLIC_BASE_URL`, and the selected asset's managed recording; it does not use an external provider.

> **Known issue (beta.11):** The Publish screen does not always say when a mock provider did the work. An Internet Archive or Local NAS row carries a "Simulated" note, but a YouTube row from the mock provider shows a success message with no simulated note (HELP-03). Check the far end after your first real publish.

> **Beta.11 history:** The Publish screen then listed subscriber notifications without sending them, and the Podcast surface did not create audio. Current source can send selected notices and create selected podcast episodes when configured. See Publishing.

#### Cable file packages

Some cable stations take recordings as files, not as a live stream. A *cable file package* is a ZIP that holds the media, its captions, a `manifest.json` and a `SHA256SUMS` file (checksums, so the receiver can check nothing was damaged).

1. Set `CIVICCAST_CABLE_PACKAGE_OUTPUT_DIR` to the folder where packages should be

written.
2. Set `CIVICCAST_CABLE_CAPTIONS_DIR` to a folder that holds a caption file named `<asset-id>.vtt` or `<asset-id>.srt` for each recording that has captions.
3. Restart the service.
4. Approve the recording on **Publish** with the cable file package ticked. The package is made from the recording's local file; a recording with no local file (`file_path`) cannot be packaged.

You can also run the command `civiccast cable package` from the command line; see Appendix: command line.

> **Known issue (beta.11):** If `CIVICCAST_CABLE_PACKAGE_OUTPUT_DIR` is not set, the Publish row reads "not set up (optional)" and has no retry afterwards (HELP-08). Set the folder first, then approve; do not approve first and fix it later.

> **Note:** The package is a file. Its creation is not proof that any cable company accepted it. The `leightronix-file-drop` preset (above) is a different thing: it configures a channel's output, not this ZIP.

#### Federation (ActivityPub)

*ActivityPub* is the open standard that lets servers such as Mastodon follow each other. With it turned on, someone on a compatible server can follow the station and see new recordings announced. It is **off by default**.

CivicCast turns federation on only when all of these are true: the mode is not `disabled`; a public base address is set that begins with `https://` (or `http://` on a loopback address, for lab use); a private key path is set; and the key can be read. If any is missing it stays off without an error, so a half-configured station looks the same as an unconfigured one.

1. **Make a key.** In an Administrator PowerShell, from the install folder:
   ```
   .\runtime\python.exe -m civiccast.cli activitypub keygen --private-key-path
   ↪  "C:\ProgramData\CivicCast\federation\actor.pem" --base-url
   ↪  "https://your-public-address" --handle station --mode limited
   ```
   Store the key where only Administrators and SYSTEM can read it. The command prints the environment settings to add in the next step. The command is in the code; we did not run it on a station.
2. **Set the variables** (in the service environment) and restart the service:

| Variable | Meaning |
|---|---|
| `CIVICCAST_ACTIVITYPUB_MODE` | `disabled`, `open`, `limited` or `approval-only` |
| `CIVICCAST_ACTIVITYPUB_BASE_URL` | The public address of the station (if unset, `CIVICCAST_PUBLIC_BASE_URL` is used) |
| `CIVICCAST_ACTIVITYPUB_HANDLE`, `..._DISPLAY_NAME` | The account name and the name shown to others |

| Variable | Meaning |
|---|---|
| `CIVICCAST_ACTIVITYPUB_PRIVATE_KEY_PATH`, `..._PUBLIC_KEY_PEM` | The key |
| `CIVICCAST_ACTIVITYPUB_AUTHORIZED_FETCH` | Require signed requests to read the actor |
| `CIVICCAST_ACTIVITYPUB_BLOCKLIST`, `..._BLOCKLIST_FILE`, `..._ALLOWLIST`, `..._ALLOWLIST_FILE` | Servers to refuse, or the only servers to accept |
| `CIVICCAST_ACTIVITYPUB_INBOX_RATE_LIMIT`, `..._INBOX_RATE_WINDOW_SECONDS` | Defaults: 60 requests per 60 seconds |
| `CIVICCAST_ACTIVITYPUB_RETRY_BACKOFF_SECONDS` | Delay between delivery retries |
| `CIVICCAST_ACTIVITYPUB_LAB_ALLOW_LOCAL` | Lab only: allow local addresses |

The public routes are `/.well-known/webfinger`, `/.well-known/nodeinfo`, `/nodeinfo/2.0`, `/ap/actor`, `/ap/inbox`, `/ap/followers` and `/ap/outbox`. Remote actors and their inboxes must be public HTTPS addresses. The staff routes are under `/api/staff/activitypub/` (status, followers, approve, reject and block, outbox, deliveries, retry and replay, keygen).

The control plane listens on `127.0.0.1` only, so servers on the internet cannot reach these routes until you provide a path from your public address to port 8000. The same caution as above applies: a same-host reverse proxy changes how the station sees addresses.

> **Known issue (beta.11):** The console cannot switch federation on. The **ActivityPub** screen shows status and followers only. You must use the variables above.

> **Known issue (beta.11):** There is no control to unblock a server you blocked. Approve, Reject and Block need the Publish operator or Support admin role, and the moderation error shows the same generic text even when the real cause is a missing role.

> **Known issue (beta.11):** Approving a recording on **Publish** announces it to followers when federation is on. This is not shown on the Publish screen.

> **Warning:** Federation lets other servers send requests to the station's public address. Do not turn it on until the public path is designed, and read Security and privacy.

#### Emergency alerts feeds

CivicCast can read public alert feeds and show an emergency overlay or put up a slate. **It is not an Emergency Alert System (EAS) device**, and the console says so on the Emergency Alerts screen. Do not use it as your legal EAS path.

Two things must be switched on, and the installer sets neither:

| Variable | Effect |
|---|---|
| `CIVICCAST_EAS` | The poll worker runs only when this is set to something other than `off` (the default is `off`). |
| `CIVICCAST_EAS_AUTO_SURFACE` | Automatic display of severe and higher alerts needs `1`, `true`, `yes` or `on`. Without it, severe and higher alerts are not put on air automatically. |
| `CIVICCAST_EAS_POLL_SECONDS` | How often to poll; default 60. |

> **Known issue (beta.11):** The screen inventory for Emergency Alerts says severe alerts air automatically. In the code they do so only when both `CIVICCAST_EAS` is on and `CIVICCAST_EAS_AUTO_SURFACE` is on. With a stock install, nothing is polled and nothing is surfaced. We followed the code.

**Add a feed** Create or replace a feed source with `PUT /api/staff/eas/sources/{source_id}`. This needs the Setup admin role. The body is:

| Field | Meaning |
|---|---|
| `source_id` | A name. It must equal the one in the address. |
| `label` | A human-readable name. |
| `kind` | `ipaws-cap`, `nws-cap`, `amber-cap` or `manual`. |
| `endpoint_url` | The feed address. |
| `geocode_filter` | A list of area codes to keep. |
| `severity_floor` | The lowest severity to act on: `unknown`, `minor`, `moderate`, `severe` or `extreme`. Default `severe`. |
| `poll_seconds` | 15 to 3600; default 60. |
| `enabled` | `true` or `false`. |
| `credential_ref` | The **name of an environment variable** that holds the credential, not the credential itself. |
| `notes` | Free text. |

For any feed that has a `credential_ref`, the station sends `Authorization: Bearer <value of that variable>`. NWS feeds are requested as `application/geo+json`; the others as `application/cap+xml, application/xml`. A `manual` source is needed before an alert can be entered by hand (`POST /api/staff/eas/alerts/manual`, Setup admin; an alert can be attributed only to a source of kind `manual`). Forcing the slate needs an explicit operator confirmation. Reading the feeds needs Setup admin, Support admin or Meeting operator; displaying an alert needs Setup admin or Meeting operator.

The public overlay data is at `/api/public/cg/emergency-overlay`.

> **Known issue (beta.11):** A feed that fails raises the alert kind `eas-source-unavailable`, but the console still shows the feed as "polling". Watch the **Alerts** screen, not that word.

With `CIVICCAST_EAS=inline` configured before channel startup, the GStreamer channel reserves an emergency compositor layer. Configured crawls, overlays and forced slates are drawn into the outgoing video, including MPEG-TS output. Automatic presentation separately requires `CIVICCAST_EAS_AUTO_SURFACE`; neither polling nor automatic presentation is enabled by default.

#### Webhooks and subscriptions

There are three different webhook features. Keep them apart.

**Resident subscriptions** Residents can sign up on the public portal with `POST /api/public/subscribe/email` or `POST /api/public/subscribe/webhook` (body: `webhook_url`, `target_type`, `target_id`). There are also confirm, unsubscribe and RSS routes (`/api/public/subscribe/rss/<type>/<id>.xml`). Staff can send a test with `POST /api/staff/subscribe/dispatch-test` (Publish operator or Support admin).

When a subscriber webhook is delivered, the station sends a JSON body with `asset_id`, `title`, `portal_url`, `podcast_url`, `summary` and `published_at`, and two headers: `x-civiccast-signature: sha256=<hex>` (an HMAC-SHA256 of the body written with sorted keys and compact separators) and `x-civiccast-asset-id`. The timeout is `CIVICCAST_WEBHOOK_TIMEOUT_SECONDS` (default 30), with retries set by the `CIVICCAST_WEBHOOK_RETRY_*` variables.

> **Beta.11 history:** Only staff test dispatch could send subscriber notices. Current source can send them when the optional Publish surface is selected.

With `CIVICCAST_PROVIDER_WEBHOOK=real`, register a public HTTPS endpoint. CivicCast sends that endpoint a JSON challenge containing `type: "civiccast.subscription_confirmation"`, a `confirmation_token` and a random `signing_secret`. Neither secret nor token is returned in the signup reply. The receiver keeps the secret and confirms through `GET /api/public/subscribe/confirm?token=<confirmation_token>` on the station before notices can send. Verify notice signatures over the exact JSON bytes with that secret. A pending registration can be retried for a fresh challenge; the previous token then stops working. Existing subscriptions with the old identifier-derived secret must register and confirm again. Challenges, notices and retries reject non-public addresses and DNS results, pin connections to a validated IP with hostname TLS verification, and do not follow redirects or use environment proxies. Unsafe destinations return 400; failed challenge delivery returns 502. Mock mode records a simulated challenge and contacts nobody. See `docs/ops/cdn-and-providers.md` for the receiver contract.

**Alert destinations** The **Alerts** screen can hold destinations of three kinds: email, text message and webhook. A webhook destination receives a JSON body with `event_id`, `condition`, `severity`, `state`, `resource_ref`, `summary`, `detail`, `first_observed_at` and `last_observed_at`. The

body is signed with the destination's `secret` using HMAC-SHA256, and the header `X-CivicCast-Signature` carries the hex digest with no `sha256=` prefix. The timeout is 10 seconds. A failed delivery is retried after 120, 240, 480 and 960 seconds and then given up on after 5 attempts.

> **Known issue (beta.11; editor limitation fixed in beta.12):** On a new install every alert rule is created with no destination attached, so alerts may never be delivered until a Setup admin assigns destinations. In beta.11 the rule editor could not attach them; beta.12 adds a **Destinations** list to each rule card. There is no **Send test alert** button.

**Payments (inbound)** `POST /api/webhooks/stripe` accepts a payment provider's notification. It requires a `Stripe-Signature` header, rejects a body over 1 MiB, and answers 401 for a bad signature. It is not reachable from the internet unless you build a path to it.

> **Beta 19 status:** A configured mail provider sends sign-in links and reports delivery failures. The public portal still asks for paywall tiers and a checkout, but the server has no routes for them (the API list shows only `access`, `magic-link` and `verify`), and public VOD media is not grant-protected. See Publishing.

#### Station import from Cablecast

A **Setup admin** can open **Setup > Import station** to bring Cablecast show and schedule metadata into CivicCast. Enter the Cablecast API URL and, if that server requires them, its username and password. Choose **Preview import** first. The preview lists shows and schedule entries to add, along with conflicts and skipped records; it does not change CivicCast data. The password is cleared from the screen after the preview.

Review the proposed rows and acknowledge them before choosing **Apply this import**. The screen reports the created show and schedule counts and any rows that failed. **Import history** lists batches, and **Roll back this batch** removes the rows created by that batch after a second confirmation. If an Apply request reports an error, check batch history before retrying because some rows may already have been committed. Import progress names the current phase; the server does not provide a percentage.

This workflow imports metadata and media references, not the media files themselves. Check that referenced media is available to the new station before scheduling it on air. The screen supports Cablecast; other vendor file adapters are separate workflows.

#### Other stations

Open **Run meeting > Other stations** to browse published programs and check channels at a trusted CivicCast station. A **Setup admin**, **Meeting operator**, **Support admin** or **Publish operator** can view this screen. Choose a station from **Other station**, then use **Watch on ...** to open a published program on that station's public portal in a new tab. The list contains at most 100 published programs; it does not copy media to this station.

The channel section shows the remote station's reported state, current program and last health-report time. **Refresh channel status** requests a fresh snapshot. A missing report does not establish

whether the channel is on air. If a catalog or status request fails, retry that section; the other section remains usable.

A **Setup admin** or **Meeting operator** can choose **Start** or **Stop** for a shared channel. Check the station and channel named in the confirmation before continuing: these actions can affect the remote station's on-air stream. A successful request says the command was **queued**. Use **Refresh channel status** to confirm execution. If the request fails, refresh status before retrying because the remote station may already have received it.

An empty station list means no peers are configured. A station administrator provisions a service-owned `federation.json` beside `station-state.json`, or selects its location with `CIVICCAST_FEDERATION_CONFIG`. Each peer has a fixed HTTPS public origin. Remote status and commands additionally require its staff origin, a server environment variable holding a remote staff token, and an explicit list of channel IDs. The screen does not accept addresses or credentials. Remote staff-token roles are still enforced by the peer. Use the configuration example below; this is separate from ActivityPub above.

Example `federation.json`:

```
{
  "stations": [{
    "station_id": "neighbor",
    "label": "Neighbor station",
    "public_origin": "https://watch.neighbor.example",
    "staff_origin": "https://staff.neighbor.example",
    "token_env": "CIVICCAST_NEIGHBOR_STAFF_TOKEN",
    "channel_ids": ["public", "government"]
  }]
}
```

Replace the example origins and channel IDs with the peer's actual values. Both HTTPS origins need trusted certificates; configure the staff proxy to allow the central station's network. Set the named service environment variable to a remote staff bearer token with the required remote role, using the service environment procedure in Running it day to day. Never put the token itself in this JSON. File changes take effect on the next request; service-environment changes require a planned service restart. For catalog-only sharing, omit `staff_origin`, `token_env` and `channel_ids`. This source feature is not included in the published Beta 19 package.

#### External agenda import

The agenda screen can look up a meeting in an outside agenda system and import it. This is **off by default**. To turn it on, set `CIVICCAST_AGENDA_SOURCE` to `legistar`, `primegov`, `civicclerk` or `js_portal`, then restart the service. The other settings are `CIVICCAST_AGENDA_SOURCE_CLIENT`, `CIVICCAST_AGENDA_SOURCE_TOKEN` and `CIVICCAST_AGENDA_SOURCE_TIMEOUT_S` (default 10 seconds). A value other than those (or `off`) is rejected with a message listing the allowed values.

The tenant or site code that staff type on the screen must be 1 to 64 letters, digits, hyphens or underscores. The portal address for a `js_portal` source must not carry a user name or password, and must not point at a private or local address. A `js_portal` source also needs an optional add-on; the

screen reports "JS-portal runtime: not installed." when it is missing.

#### The HTTP API

Everything the console shows comes through the station's HTTP API, and the same calls are open to your own scripts. Calls under `/api/staff/` need a staff token; calls under `/api/public/` and a few others do not. The current generated route inventory is in `ops/docs-sprint/inventory/generated/api.md` and in the API appendix; use those tables for route details.

**Which areas are stable?** None is declared stable. We searched the code and the repository's documents for a versioning or stability policy and found none. The routes behind the console change when the console changes. Some installer routes are hidden from the published schema (`include_in_schema=False`); treat those as internal. The documented error format is: the `detail` field is a string for domain errors (404, 409, 503), an object for structured conflicts, and an array for validation errors (422), so a script should read the status code first and then the type of `detail`.

| Area | Authentication | Use for scripts? |
|---|---|---|
| `/health`, `/api/version`, `/api/hardware` | None | Yes. Simple and unlikely to change. |
| `/api/public/*` (channels, live, assets, schedule, programlog, embed, podcast, subscribe, reports, search) | None | Yes, with the beta caution. The resident portal's own pages call `/api/public/live/current` and `/api/public/assets`, among others. Some `/api/public/channels/<channel>/...` routes (for example `live`, `schedule` and `now-next`) return built-in sample data, not what is on air, so do not use them for real programming. |
| `/media/live/*`, `/media/vod/*` | None | The files that the players fetch. |
| `/.well-known/*`, `/nodeinfo/2.0`, `/ap/*` | None | Federation only; see above. |
| `/api/staff/*` | Staff token with a role | Possible, but it follows the console and may change without notice. |
| `/api/setup/*` | None, from the station computer only | No. Console sign-in and first setup only. |
| `/api/webhooks/stripe` | A payment signature | Inbound from the payment provider only. |

**Get a staff token** For scripts, issue an access key with the command line. (The console sign-in keeps its own token inside the browser; the console has no screen for entering a script's key.) You need an Administrator PowerShell on the station computer and the database address.

```
cd "<install folder>"
$env:DATABASE_URL = (Get-ItemProperty
 ↪  'HKLM:\SOFTWARE\CivicCast\Native').DatabaseUrl
.\runtime\python.exe -m civiccast.cli token issue --operator-id integrations
 ↪  --display-name "Integrations script" --scopes operator --json
```

`<install folder>` is where setup put CivicCast. The command prints the secret **once**; copy it into a password manager at once. The command needs DATABASE_URL; without it the message is "DATABASE_URL must point at the CivicCast database before running staff token lifecycle commands." The value is held in the registry where SYSTEM, Administrators and the station service identity can read it. Close the window afterwards so the variable is gone.

The scopes `operator` and `admin` each grant all five roles (Setup admin, Meeting operator, Records clerk, Publish operator and Support admin); a token with no scopes grants none. To give a script only what it needs, name the roles instead, comma-separated: `setup_admin`, `meeting_operator`, `records_clerk`, `publish_operator`, `support_admin` (the code also accepts the hyphenated and short forms such as `meeting-operator` and `meeting`). See Security and privacy. Other token commands are `token list`, `token revoke` and `token rotate`.

The installed package places the `civiccast.exe` command under `<install folder>\runtime\Lib\site-packages\bin\civiccast.exe`. If that executable is missing, run the same command through `<install folder>\runtime\python.exe -m civiccast.cli`.

> **Warning:** A token with all five roles can change the station and stop channels. Keep it out of scripts that others can read, and revoke it when you are done.

**Worked example with curl** Windows includes `curl.exe`. Put the token in a variable for this window only, so that it does not end up in a script file or in your history:

```
$env:CC_TOKEN = Read-Host "Paste the token"
```

1. **Check the station is up and which version.** No token is needed.
   ```
   curl.exe http://127.0.0.1:8000/health
   curl.exe http://127.0.0.1:8000/api/version
   ```
2. **Ask who the token belongs to.** It returns the name and the roles of the token.
   ```
   curl.exe -H "Authorization: Bearer $env:CC_TOKEN"
    ↪  http://127.0.0.1:8000/api/staff/auth/me
   ```
3. **List the channels and read one channel's state.**
   ```
   curl.exe -H "Authorization: Bearer $env:CC_TOKEN"
    ↪  http://127.0.0.1:8000/api/staff/egress/channels
   curl.exe -H "Authorization: Bearer $env:CC_TOKEN"
    ↪  http://127.0.0.1:8000/api/staff/egress/channels/<channel>/state
   ```
4. **Read the station's health, the same data as Readiness.**

```
curl.exe -H "Authorization: Bearer $env:CC_TOKEN"
 ↪  http://127.0.0.1:8000/api/staff/installer/system-health
```

5. **Read what residents see.** No token.
   ```
   curl.exe http://127.0.0.1:8000/api/public/channels
   curl.exe http://127.0.0.1:8000/api/public/live/current
   curl.exe http://127.0.0.1:8000/api/public/assets
   ```
   `/api/public/channels` returns the station's built-in channel line-up (`public`, `education` and `government`) with each channel's live output addresses filled in from its egress settings; it is not a list of channels you created.
   `/api/public/assets` lists only recordings that have been packaged for the web (a `manifest_url`) and have a publish date.
6. **Queue a command for a channel.** This needs the Meeting operator role. The body is a JSON document with an `action` of `start`, `stop`, `reload` or `drain` (the schema also lists `takeover` and `handback`, which have their own routes). The station answers with HTTP 202, which means "accepted", not "done". A `start` for a channel with no saved configuration, or one that is disabled, is refused with 409. Read the state afterwards.
   ```
   curl.exe -X POST -H "Authorization: Bearer $env:CC_TOKEN" -H "Content-Type:
    ↪  application/json" --data "{\"action\":\"reload\"}"
    ↪  http://127.0.0.1:8000/api/staff/egress/channels/<channel>/commands
   ```
7. **Download the full schema.** On a station the interactive pages `/docs` and `/redoc` are not served, and `/openapi.json` needs a staff token.
   ```
   curl.exe -H "Authorization: Bearer $env:CC_TOKEN"
    ↪  http://127.0.0.1:8000/openapi.json -o openapi.json
   ```

Other things worth reading with a token: `GET /api/staff/runtime-safe-to-air`, `GET /api/staff/assets`, `GET /api/staff/schedule`, and `POST /api/staff/schedule/{id}/cancel` to cancel a schedule item.

### What the answers mean

| Status | Meaning | What to do |
|---|---|---|
| 200, 201 | Done | |
| 202 | Accepted; a worker will do it | Read the state afterwards. |
| 401 | The token is missing, wrong or revoked | Check the `Authorization: Bearer` header and that the token was not revoked. |
| 403 | The token lacks a role. The text begins "This action requires one of these CivicCast roles:" | Use a token that holds that role. |
| 404, 409, 503 | A domain answer; `detail` is text | Read the text. |
| 422 | The request body is wrong; `detail` is a list of field errors | Fix the fields. |

| Status | Meaning | What to do |
|---|---|---|
| 429 | Too many failed tokens: 10 failures in 60 seconds from one address (by default). The `detail` reads "Too many failed staff authentication attempts. Wait and retry." | Wait; the `Retry-After` header gives the seconds. A request with no token at all does not count. |

**Historical Beta 10 audit note:** A whole-repository audit reported test/feature divergence at that time. It is not a test result for the current package. Use the current verification record for this release's checks and try an integration on a test channel before automating it.

#### If it did not work

- **The relay does not start.** Check `CIVICCAST_TS_RELAY`, whether `tsp.exe` exists in one of the places listed above, and the log fragments in the relay table. See also Troubleshooting matrix.
- **The headend sees a reset stream at program changes.** The relay is off or TSDuck was not found: look for `TS relay auto mode: TSDuck (tsp) not found`.
- **The web player shows nothing.** Read `/api/public/live/current` and its `reason`, then the HLS relay log.
- **A CDN upload fails.** Look for `cdn.upload_failed` in the recording's finalization status.
- **Federation does nothing.** One of the four conditions above is missing. There is no error message.
- **No emergency alerts appear.** `CIVICCAST_EAS` is `off` by default. Check it first.
- **A call returns 401 or 403.** Check the token and the role.
- **A call from another computer cannot connect.** The station answers on `127.0.0.1` only.

#### Package status and verification

This chapter describes the unpublished Beta 20 candidate edition. Use the release record matching the package on the station; the Beta 18 record tracks that earlier package's checks. Commission each external service and headend for the station's actual configuration.

#### Related

- Troubleshooting matrix
- Configuring the station
- Running it day to day
- Security and privacy
- How CivicCast is built
- Publishing, and what residents see
- Running the meeting

# Part III. How CivicCast is built

This part explains how the system is put together, with diagrams, for IT staff, integrators and reviewers.

## How CivicCast is built

This chapter explains how CivicCast is put together: which programs run on the station computer, what each one is responsible for, how a meeting recording and a 24-hour channel move through the system, and what the software does when something fails. It is written for the IT person at a city or station, for an integrator connecting CivicCast to other equipment, and for a technical reviewer who wants to check the design. You do not need to read it to run a station. The earlier chapters tell you what to click.

Each figure has a caption and a plain-English explanation. This chapter follows the current architecture for captions, installation and model selection, recording and private source playback. Where a value is a default that a setting can change, the text says so.

### How to read the figures

- A **rounded box** is a program or a part of a program. A **cylinder** is stored data. An **arrow** is a call, a read or a write, in the direction of the arrow.
- A **dashed arrow** is something that happens only sometimes, or only if a setting is turned on.
- **Default** values are marked "default" in the text. The environment variable that can change a value is named in the Appendix on settings.
- "Station" means the one Windows computer that runs CivicCast. "Control plane" means the web server and background workers that make up the CivicCast application. "Worker" means a separate program the control plane starts to play out one channel.

### The system and the people and equipment around it

Figure 16-1 shows what talks to CivicCast. CivicCast runs on one Windows computer. People use it through a web browser. Video comes in from cameras, capture cards, network streams, watch folders and uploaded files. Video goes out to residents, to a cable headend or other downstream equipment, and to archive and distribution services.

**What this shows.** Staff open the operator console at `/operator/`; residents use the portal at `/`. The station takes camera and network sources, uploaded programmes and watch-folder media, then plays channels to configured outputs. A published recording can feed the resident portal, podcast RSS, subscriber notices, and configured archive or distribution services. Remote guests enter through a separately hosted VDO.Ninja site and MediaMTX relay before CivicCast composes them into a channel. A content delivery network is optional. Emergency and weather feeds appear on screen; CivicCast is not an EAS device. The Windows station runs with its bundled Whistle and Medium Whisper resources, while the wizard can acquire optional components.

The application listener remains loopback-only (default `127.0.0.1:8000`, configurable to `127.0.0.1` or `::1`). Remote HTTPS uses Caddy with separate resident and CIDR-restricted staff hostnames; first setup stays local-only. See the network access guide.

**Emergency presentation in the beta.12 candidate.** When an administrator enables `CIVICCAST_EAS` before starting a channel, the channel reserves an emergency layer in its existing video compositor. Channel health ticks select the applicable alert and send its presentation to the video worker. Clearing or expiring an alert removes only that emergency layer, preserving ordinary

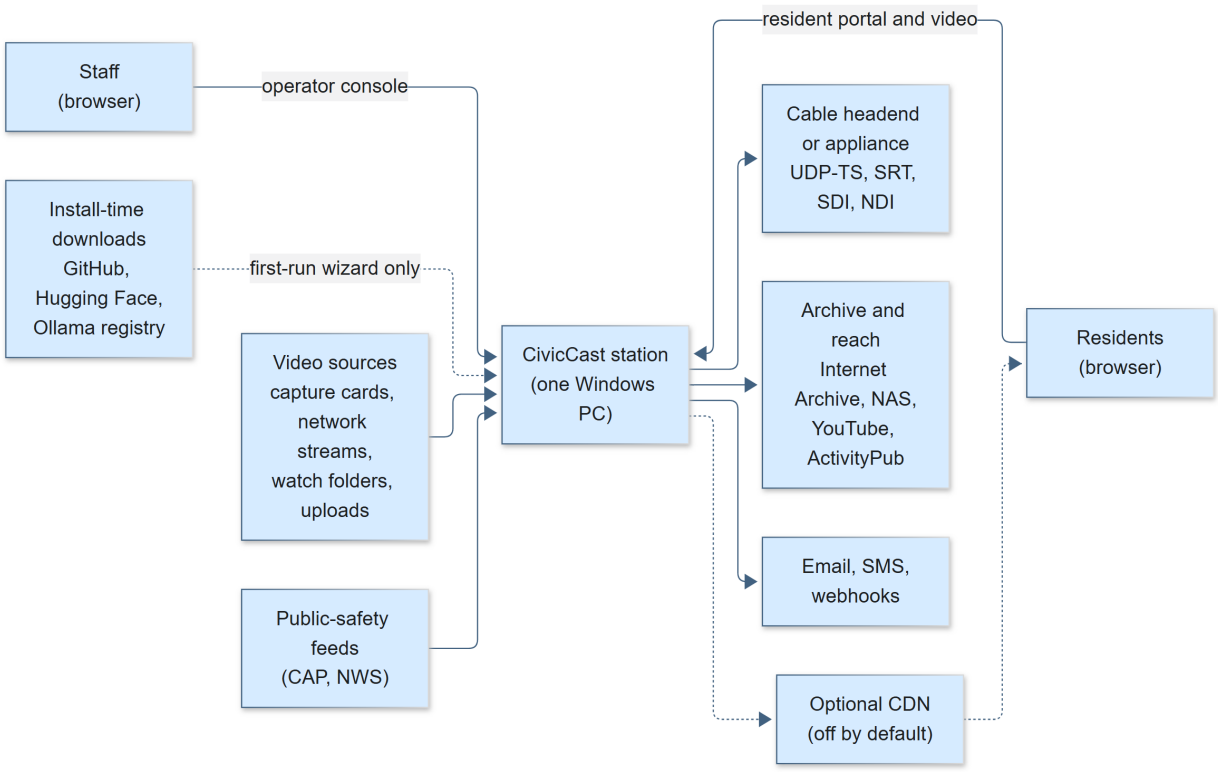



Figure 16-1. System context. Solid arrows are the normal flows. Dashed arrows are optional or occur only at install time.

graphics. A forced full-screen slate requires explicit confirmation. The resident page separately polls the channel-specific emergency endpoint and removes its notice when that endpoint returns no active alert. Isolated CPU and D3D11 video captures and a browser check exercised these paths; this does not establish real CAP-feed reception or acceptance by a cable headend. See Emergency alerts.

### The programs that run on the station

CivicCast is installed as a Windows service. A service starts when the computer starts, before anyone signs in, and keeps running when nobody is signed in. Figure 16-2 shows the programs that run, which one starts which, and the network addresses they use.

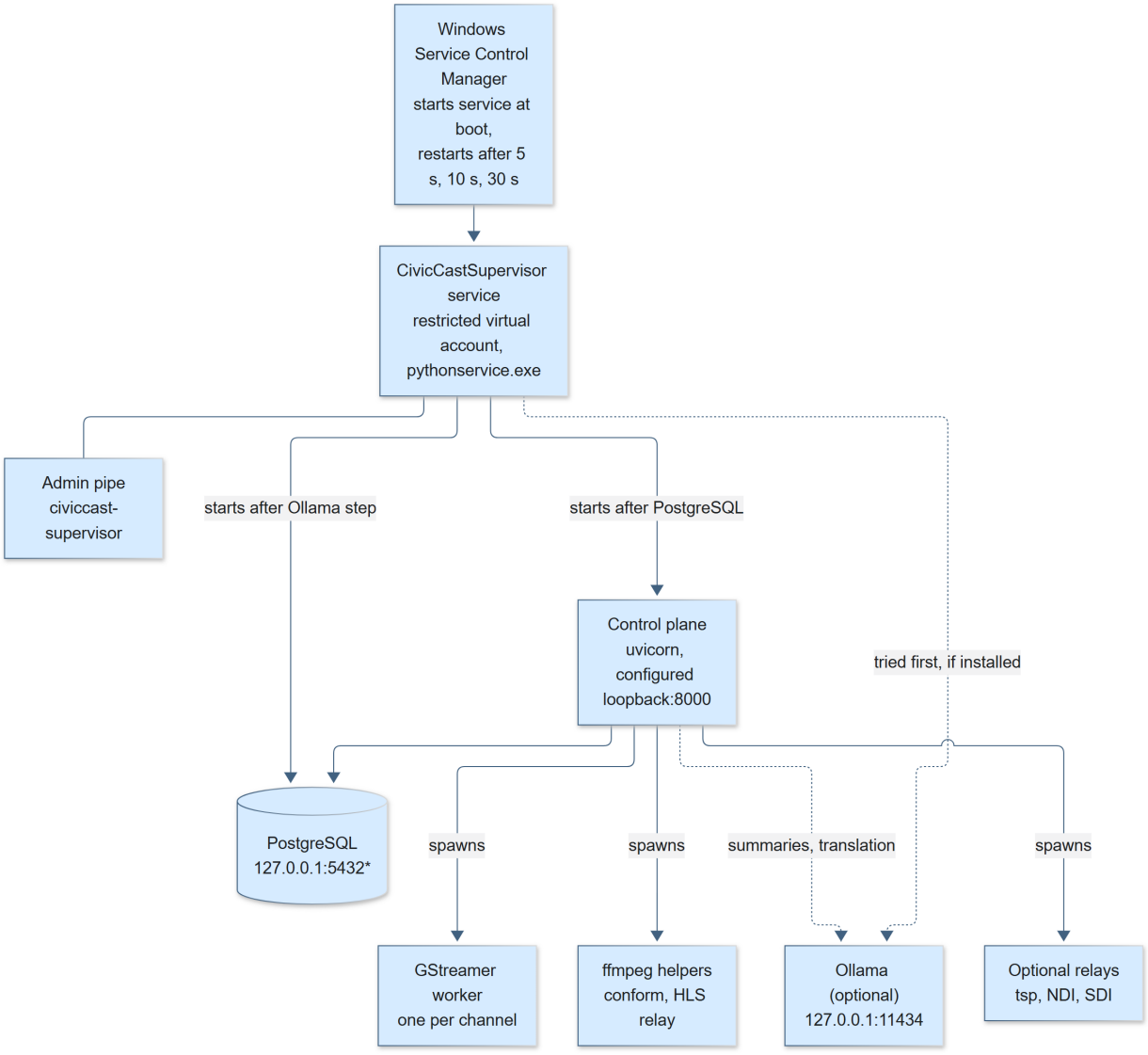



Figure 16-2. Processes on a native Windows station. *PostgreSQL uses the first free port of 5432, 5433, 5434, 5435 and 5544, chosen when the database was created.

**What this shows.** One Windows service, named `CivicCastSupervisor` (display name “CivicCast

Native Supervisor"), is the parent of everything else. Installer source configures it to run as the restricted virtual account `NT SERVICE\CivicCastSupervisor`; existing default LocalSystem installations migrate the old profile and CivicCast credentials before the service identity changes. See the restricted service identity guide. The supervisor is set to start automatically. It starts PostgreSQL and waits until the database answers a `SELECT 1` (up to 60 seconds by default), then starts the control plane and waits until `GET /health` returns 200 (up to 180 seconds by default, with a separate rule that gives up after 60 seconds of no CPU activity so a hung start is not mistaken for a slow one). Ollama is the local AI engine used for summaries and translation. If its program or a model store with a `manifests` folder is missing, the supervisor skips it and checks again about every 60 seconds.

The supervisor does not start the video workers. The control plane does, one GStreamer worker program per channel, and it starts helper programs (ffmpeg to prepare videos and to cut live HLS segments, and optional relays for UDP transport streams, NDI and SDI). This is deliberate: the part of the system that knows the schedule is the part that decides what each worker plays.

| Item | Value (default) |
|---|---|
| Service name | `CivicCastSupervisor` |
| Runs as, start type | `NT SERVICE\CivicCastSupervisor` (restricted virtual account), automatic |
| If the service stops unexpectedly | Windows restarts it after 5 s, then 10 s, then 30 s |
| Control plane address | Configured loopback, default `127.0.0.1:8000` |
| PostgreSQL address | `127.0.0.1`, first free port of 5432, 5433, 5434, 5435, 5544 |
| Ollama address | `127.0.0.1:11434` |
| Admin pipe | `\\.\pipe\civiccast-supervisor` |
| Single-instance lock | named mutex `Global\CivicCastSupervisorSingleton` |
| Firewall rule | inbound TCP 8000, program `<install folder>\runtime\python.exe` |

**How the supervisor protects the station.** It keeps four promises.

1. **One copy only.** A named lock (a Windows *mutex*) is intended to allow exactly one supervisor. Its current access-control list grants SYSTEM, Administrators and the CivicCast supervisor service SID.
2. **No orphans.** Every child is placed in a Windows *job object* set to kill all its members when the supervisor exits, and children cannot break out of it. If the supervisor crashes, Windows ends the whole process tree, including the video workers. A sweep by job name runs before the first start in case a previous job survived.
3. **Restart with backoff.** A child that stops is restarted after 1 second, then 2, 4 and so on, up to 30 seconds, with plus or minus 20 percent random jitter. Five restarts within 10 minutes put the supervisor in the `degraded` state: the service stays up, and an alert is raised. A restarted child waits until every child before it in the start order is ready again.

4. **Graceful stop.** On stop, children are stopped in reverse order. Each gets 15 seconds to stop cleanly (PostgreSQL with `pg_ctl stop -m fast`; the control plane with a Ctrl+Break) before Windows ends it.

The supervisor reports one overall state. The state names are `starting`, `ready`, `degraded`, `blocked_wsl_active`, `blocked_probe_unavailable`, `maintenance` and `stopping`. The two `blocked` states mean a guard that keeps the older WSL-based edition of CivicCast and this edition from ever running together has refused to let the station transmit. `maintenance` is a read-only mode the upgrade engine uses (Figure 16-14): the database and control plane come up, and no video worker is allowed to start. Once the supervisor is `stopping`, nothing can pull it back to running. A blocked state can only be released to `starting`, never straight to `ready`, so readiness is always proven again.

**An administrator can talk to the supervisor** through a Windows named pipe. Anyone who is signed in can ask for `status` and `version`. Only an administrator or SYSTEM can ask for `start`, `stop`, `restart`, `drain` or `runtime_set`. The check is made on the identity of the caller for each command, an unknown command is refused, and each command that changes anything is logged with the caller's account identifier. A message larger than 16 KiB closes the connection.

**Why it is built this way.** The design choices are recorded in ADR 0021 (the written record of the decision to run natively on Windows). The reasons it gives, and that we checked still hold, are:

- *A service rather than a program in someone's login session.* A service starts at boot. The ADR records a test in which video was being broadcast about 12 seconds after boot, more than six minutes before anyone signed in. A station that restarts after a Windows update or a power cut should return to air without anyone logging in.
- *One Windows product, no Linux layer.* The earlier edition ran inside WSL2 (a Linux layer on Windows). The ADR records the cost: thirteen release candidates, a per-user program whose only job was to keep the Linux layer awake, and a plan for SDI capture cards that WSL2 cannot reach. The WSL edition is not part of this repository.
- *Video workers outside the supervisor.* A worker that fails should be restarted by the program that knows what it was playing, not by a general-purpose process manager.

**What the ADR accepted as risk, and what is still true.** The ADR's least-privilege follow-up is implemented in installer source through the virtual service account and profile/credential migration. The ADR also says the operator console is a Tauri application; the operator console is a web page served by the control plane at `/operator/`, and Tauri is used only for the installer window.

#### Inside the control plane

The control plane is one Python program (the `civiccast.app` application, run by the `uvicorn` web server) that does three jobs at once: it answers web requests, it runs a set of background workers on threads, and it starts and watches the video workers. There is one such process per station. Figure 16-3 shows its main parts.

**What this shows.** Requests pass through several layers of middleware (code that runs on every request) and then reach a router. Relevant checks add security headers, verify staff bearer sessions

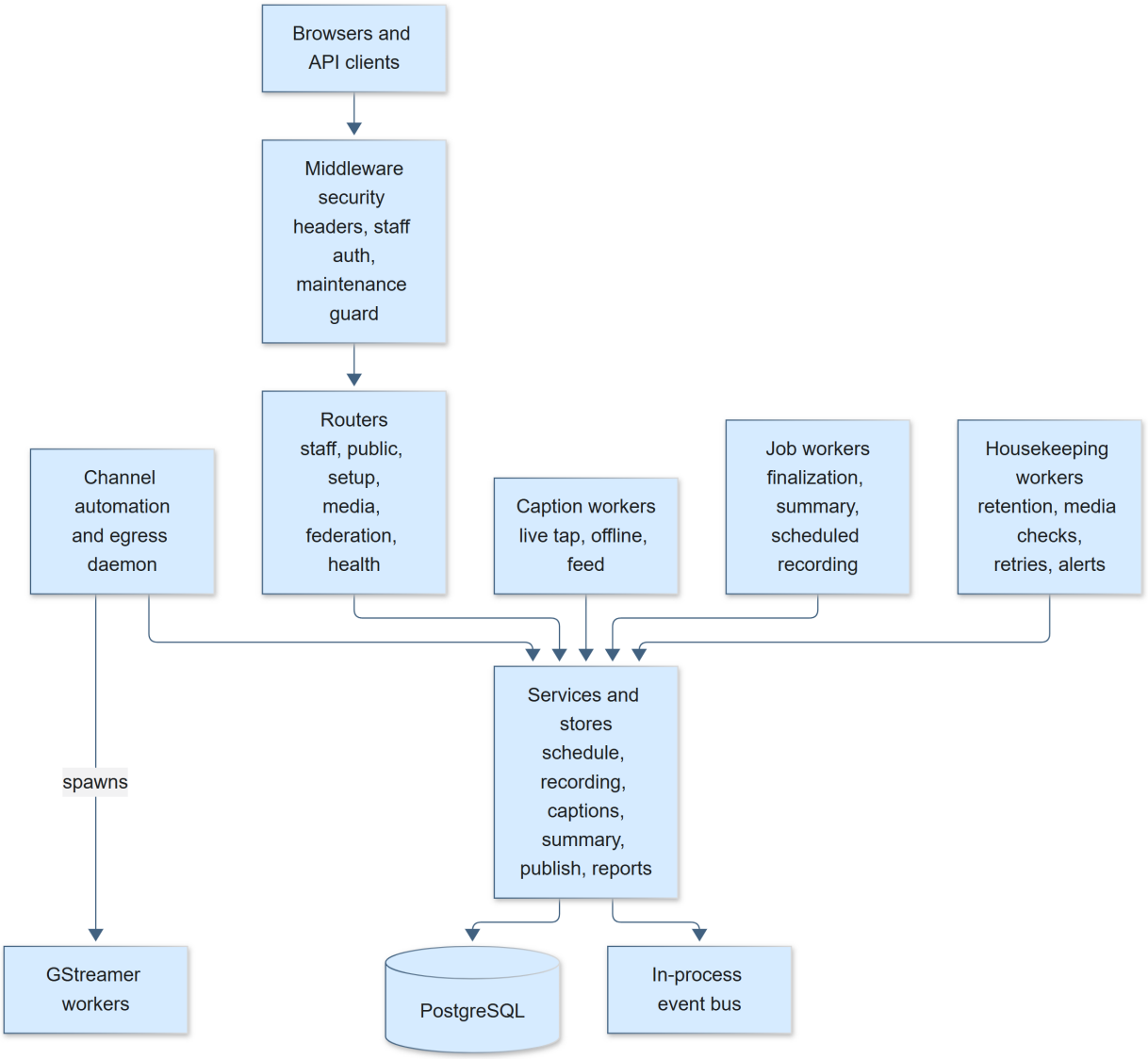



Figure 16-3. Parts of the control plane. The background workers are threads inside the same process as the web server; only the video workers are separate programs.

on protected `/api/staff/*` routes (the login route checks credentials), apply the local-only guard to `/api/setup/*`, and, during maintenance mode, refuse every POST, PUT, PATCH and DELETE request with HTTP 503. A cross-origin (CORS) layer is added only if an administrator lists allowed origins; none is allowed by default.

The routers are grouped by who may call them:

| URL prefix | Who may call it | What it holds |
|---|---|---|
| `/api/staff/...` | username/password login, then signed-in staff with a role | everything the operator console does: schedule, assets, recording, captions, summaries, publish, channels, alerts, reports |
| `/api/public/...` | anyone | what the resident portal and apps read: schedule, recordings, search, live channel list, subscribe, contribute |
| `/api/setup/...` | the station computer only; once setup is complete, all but `station-state`, `login` and `recover` also need a staff token (and, for most, the `setup_admin` role) | first setup, admin sign-in, recovery code |
| `/media/vod/...`, `/media/live/...` | anyone | video segments and playlists |
| `/ap/...`, `/.well-known/...` | remote federation servers | ActivityPub actor, inbox, outbox |
| `/health` | anyone | liveness, plus a readiness verdict |
| `/operator/`, `/` | anyone (the pages themselves) | the packaged operator console and resident portal web apps |

The two web apps are plain files bundled inside the program. The operator console is mounted at `/operator`. The resident portal is mounted at `/`, as a catch-all, so the control plane takes care that an unknown path that starts with `/api` gets a real JSON "not found" and not the portal's front page.

Behind the routers are *services* (the rules) and *stores* (the code that reads and writes the database). In a normal install every store is backed by PostgreSQL. The program refuses to start the staff write routes on a throwaway in-memory store unless an administrator explicitly allows it for testing.

The **in-process event bus** is a small piece of code in the same program that lets one part tell another that something happened. It is not a separate server. In beta.11 it has exactly one subject, `publish.asset.approved`, published when a publish approval is recorded, and nothing subscribes to

it. The workers coordinate through rows in the database and by polling, not through the bus. Earlier designs used a separate message server called NATS. ADR 0023 records why NATS was removed: nothing in the shipped product had ever used it for real traffic, yet it was a fourth process, a bundled program, a configuration file, a certificate and a health check that could report the station "not ready" for a service nothing depended on. The bus is kept behind an interface so a real network message system can be added later without rewriting the code that sends events. ADR 0023 says PostgreSQL's own notification feature stays for the low-volume "tell the screen a row changed" case; we found no use of it in the beta.11 code.

The **background workers** run on threads, each under a supervisor of its own that restarts the thread if it fails and can be switched off by an environment variable. The ones that matter most are:

| Worker thread | What it does |
|---|---|
| `civiccast-channel-automation` | every 2 seconds (default), checks every enabled channel, restarts channels that should be on air, and extends or replaces the program that is playing (Figures 16-6 and 16-7) |
| `civiccast-caption-tap-worker` | turns live audio from each channel into captions (Figure 16-10) |
| `civiccast-caption-feed` and `civiccast-caption-proof` | push finished captions into the running video worker, and check that captions really are in the emitted stream |
| `civiccast-offline-caption-worker` | captions finished recordings, after the meeting |
| `civiccast-summary-job-worker` | drafts meeting summaries with the local AI engine |
| the finalization worker | turns an ended live session into a recording (Figure 16-5) |
| `civiccast-scheduled-recording` | arms and runs scheduled recordings |
| `civiccast-program-log-materializer`, `civiccast-autoschedule-compile` | turn recurring program slots into schedule items |
| `civiccast-watch-folder-worker` | picks up new files from watch folders |
| retention, media-integrity and media-lifecycle workers | flag expired or missing media, and compute whether each asset is ready to air; they flag, and never delete |
| ActivityPub retry, webhook retry, alerting, analytics roll-up, bulletin expiry, contributor-upload reap, remote-contribution workers | housekeeping and retries |

When the supervisor has started the control plane in maintenance mode, none of these workers is started at all, because each one writes something. When the control plane is asked to stop, it first drains every live channel (waiting for each video worker to exit, then forcing it after a deadline), and it gracefully finishes any recording that is under way, so that a restart produces a finished recording

and not an orphan.

**Why threads in one process, and not many services.** The code keeps the control plane as one process on purpose. A station is one computer with one or two people to run it, and every extra process is another thing that can fail to start. The cost is that a fault that stops the control plane stops every worker thread with it. The supervisor then restarts the control plane, the channels start again (Figure 16-8), and the as-run log keeps its place (Figure 16-4).

### Where the data lives

CivicCast keeps its data in a database, in folders of files, in a few small files outside the main data folder, and in memory. Figure 16-4 shows each place and what is kept there.

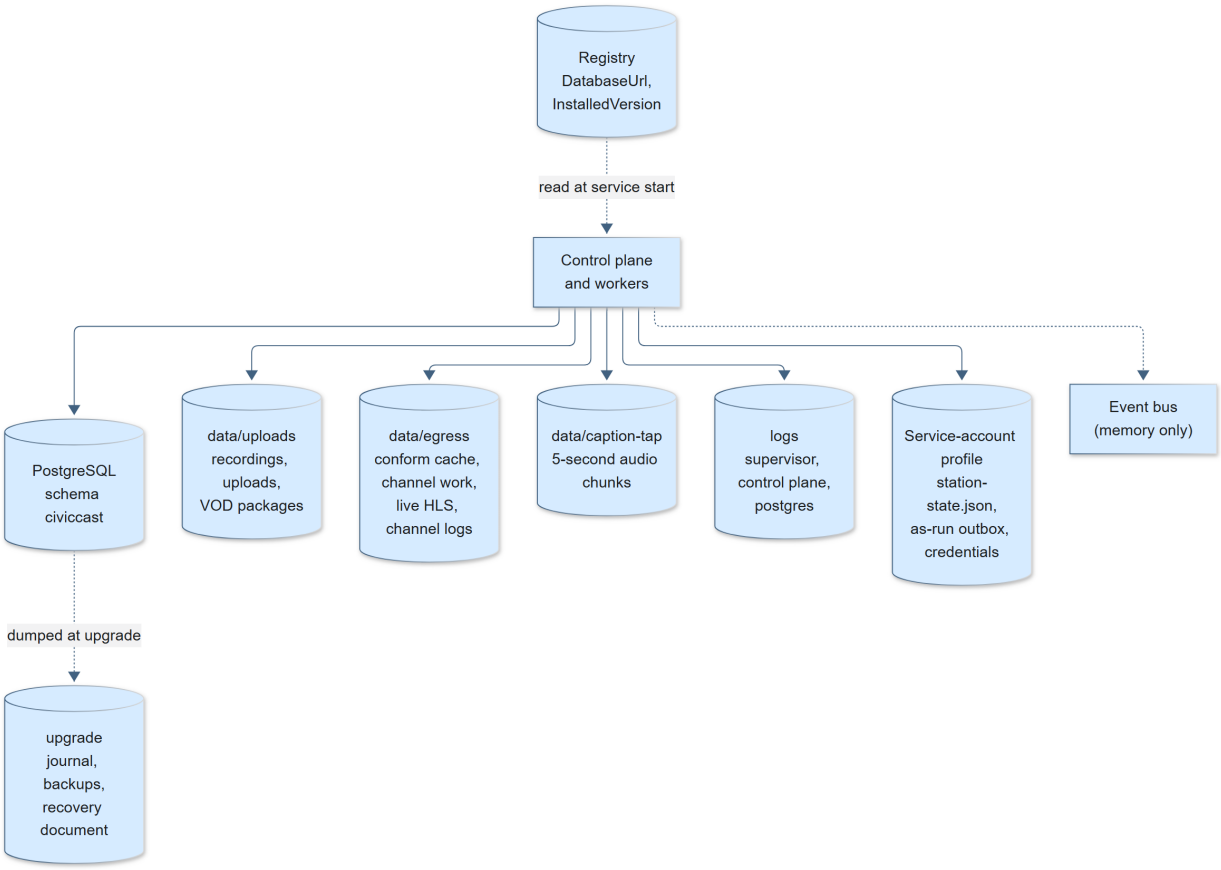



Figure 16-4. Data stores. The folders `data`, `logs` and `upgrade` are under `C:\ProgramData\CivicCast`. The registry values are under `HKLM\SOFTWARE\CivicCast\Native`.

**What each store holds.**

| Store | Where | What it holds |
|---|---|---|
| Database | PostgreSQL, database `civiccast`, schema `civiccast`, role `civiccast_svc`, files in `C:\ProgramData\CivicCast\data\pgdata` | everything that is a record rather than a file: assets, schedule items and program slots, recording schedules and jobs, live sessions, channel configuration, state, commands and the proof chain, caption review items, summaries and approvals, publish runs, signed-record exports, subscriptions, alerts, reports, staff token records |
| Uploads and recordings | `C:\ProgramData\CivicCast\data\uploads` | uploaded and captured media, finished recordings (in a `recordings` folder), and the packaged VOD copies (HLS playlists and segments, in a hidden `.civiccast-packages` folder, one per asset) |
| Channel work folder | `C:\ProgramData\CivicCast\data\egress` | the conform cache; for each channel the prepared plan folders, its worker logs and relay logs, the reload status file, generated slates and bulletin slides, and (for a channel that serves live HLS) its rolling segments; and the HLS relay's log files (5 MiB each, two kept) |
| Caption tap | `C:\ProgramData\CivicCast\data\caption-tap` | Temporary five-second audio chunks, deleted after processing; Whistle retains at most 42 waiting segments per channel (210 seconds), plus in-flight inputs and the segment being written. Runtimes without that processing grace use a 12-segment ceiling. |

| Store | Where | What it holds |
| --- | --- | --- |
| Logs | `C:\ProgramData\CivicCast\logs` | `supervisor.log`, `control_plane-app.log`, `control_plane.log` and `postgres.log` rotate at 10 MiB and retain 10 older files (up to 11 files each); the application log is written by the control plane, while the raw child logs are drained by the supervisor. |
| Installer records | `C:\ProgramData\CivicCast` | `install-progress.log`, the upgrade engine's journal, backups and recovery document under `upgrade`, and the database provisioning journal and recovery documents under `provision` |
| Downloaded components | `C:\ProgramData\CivicCast\packs` and `components` | optional Whisper and AI model files that the first-run wizard downloaded, because the installer window cannot write to the Program Files folder |
| Program files | the install folder, `C:\Program Files\CivicCast (Native)` by default | the embedded Python and the CivicCast program (`runtime`), PostgreSQL tools, ffmpeg, Ollama and optional GPU libraries (`packs`, `dependencies`), the AI model store (`models\ollama`), and the station records that prove activation passed |
| Registry | `HKLM\SOFTWARE\CivicCast\Native` | `DatabaseUrl` (the database address and password, readable by SYSTEM, administrators and the station service identity) and `InstalledVersion` |

| Store | Where | What it holds |
|---|---|---|
| Service-account profile | `C:\ProgramData\CivicCast\profile` for the configured station service | `station-state.json` (first-admin and staff password hashes, recovery-code hashes, session hashes and station profile), the as-run outbox, alert and subscription credentials, and local certificate authority files |
| Event bus | memory of the control plane | events passed between parts of the program; nothing is stored, and nothing is kept across a restart |

> **Historical Beta 11 behavior:** Before the station-operations identity migration, service-profile files could live under the LocalSystem system profile, outside `C:\ProgramData\CivicCast`. Installer source now points the supported service account at `C:\ProgramData\CivicCast\profile` and migrates the old profile and CivicCast credentials before switching a default LocalSystem installation. The station backup workflow includes configured state and portable credentials; see the station backup guide.

**Database or SQLite.** A native station runs on the PostgreSQL database provisioned by the installer. SQLite is a separate standalone backend; it is not the database prepared for an installed native station. The code also supports an in-memory store for tests; the program refuses to start its staff write routes on an in-memory store unless `CIVICCAST_ALLOW_EPHEMERAL_STORES=1` is set. The native service reads its database address from the registry at startup. If an environment variable named DATABASE_URL is set on the service, it takes precedence, and the supervisor logs which source it used (the variable name or the registry path, never the value). The native database accepts connections only from `127.0.0.1`, with password authentication (`scram-sha-256`) and one rule in its `pg_hba.conf`. It does not use TLS (`ssl = off`) because the connection never leaves the computer.

**How the schema is kept current.** One Alembic migration set, shared by every module, builds and upgrades the database. Each module keeps its own migration files in its own folder, and a single runner walks all of them (ADR 0008). The installer's provisioning step runs the migrations on a new database, and the upgrade engine runs them on an upgrade. At start-up the control plane compares the database revision to the one it expects and records one of `current`, `behind`, `ahead`, `not-configured` or `unknown`. It logs an error for `behind` or `ahead`, but it does not migrate by itself and does not refuse to start. `GET /health` always answers HTTP 200 while the program is running, and reports `status: "degraded"` unless the schema is `current`. Monitor the `status` field, not the HTTP code.

**What this design does for you.**

- *Records and files are separate.* A recording's metadata is in the database and its video is a file. The media-integrity worker re-checks that every file an asset points to still exists and flags the asset as missing if it does not; it never deletes anything. The retention worker flags expired assets for a records clerk to review; it also never deletes.
- *Writing a live recording and its record happens in one transaction.* When a live session ends, the session-ended event, the new recording asset and the move of the session to the `recorded` state are committed together (ADR 0011). The event's key makes a second attempt to finalize the same session a no-op, so a crash and retry cannot make two recordings, and a partial write cannot leave a session with no recording.
- *The record of what aired survives a database outage.* Every change of source is first written to a local SQLite file (`asrun_outbox.sqlite3`) with the strongest durability setting, then copied into the database. If the database is unreachable, the row waits in the file and is retried on every automation tick, and an alert is raised (ADR 0023 on the as-run outbox).
- *Nothing waits on a message server.* The workers coordinate through database rows and polling, and the in-memory bus carries one event today. Anything that must not be lost (a command to a channel, an as-run row, a finalization) is written to a table or file first.

**What it does not do.**

- **Station backup is separate from installer rollback.** The operator backup workflow creates encrypted `.ccbackup` archives with the configured database, station files, portable credentials and selected media; daily UTC scheduling, retention and offline restore are available. See the station backup guide. The pre-upgrade database copy remains temporary rollback protection, not a substitute for this archive.
- **Installer recovery is temporary protection, not a station backup.** Before replacing files, setup saves an access-restricted journal and a temporary copy under `C:\ProgramData\CivicCast\install-transaction`. The copy is limited to the application tree except `packs\.station-cache`, the native PostgreSQL cluster, `provision` and `upgrade`, relevant registry and firewall state, service configuration and shortcuts. It does not copy recordings or uploads. Setup checks the required copy size and leaves a 256 MB free-space reserve; after commit or successful restore it removes the temporary copy. If recovery cannot finish, the journal and copy remain for the startup recovery task or a later setup run.
- **Disposable command history and worker logs are bounded.** The normal command-consumption path trims up to 1,000 oldest consumed commands when a channel exceeds 10,000; it preserves pending commands and also trims on idle polls. GStreamer stdout and stderr rotate at 10 MiB with ten backups per stream, while pipe drainage continues if the log sink fails. System Health reports their storage use and work-volume headroom. The conform cache (60 GB by default), prepared plan folders (5 GB by default), relay logs and live-caption working data retain their existing limits. Original recordings and archived caption tracks are not deleted by these operational limits.
- **Uninstalling does not remove your data.** The uninstaller removes the program files, the service, the firewall rule and the registry values, but leaves `C:\ProgramData\CivicCast`. Because the uninstaller deletes the stored database password, a reinstall over a surviving data folder has to set a new password on the existing database. The installer does this using a short-

lived loopback-only access rule.

#### From a meeting to a resident: the life of a recording

This is the other half of the product. A meeting is captured, becomes a recording, is published to the resident portal, and gets captions and, if staff ask for them, a summary. Figure 16-5 shows the order in which things happen. It is not quite the order people expect, and the differences are worth knowing.

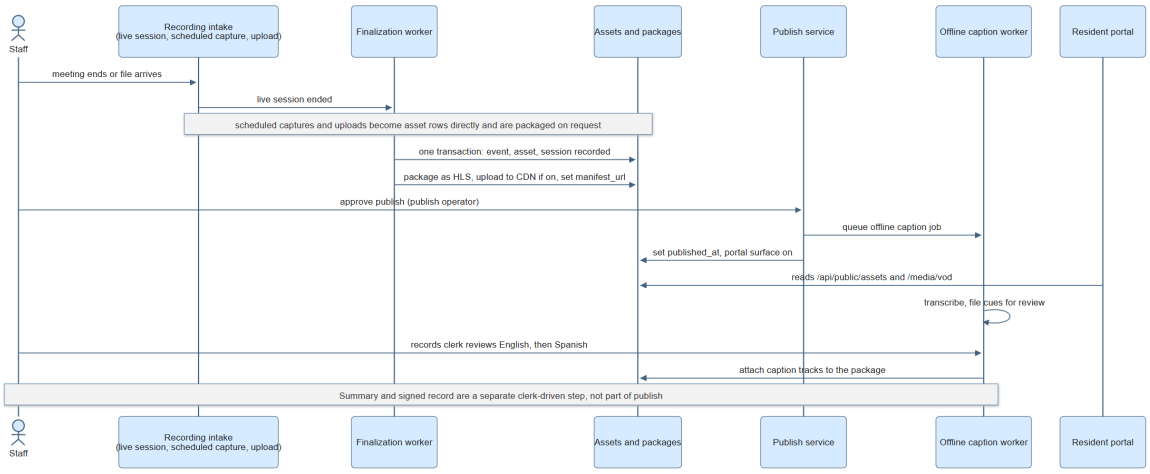



Figure 16-5. A recording from capture to resident. Captions are attached after the portal goes live.

**What this shows.**

1. **Three ways in.** A *live session* (an operator ends a broadcast, which moves the session from `on_air` to `ending`), a *scheduled recording* (a capture job that records an input such as a capture card, RTSP, SRT, HLS, RTMP, MPEG-TS or NDI stream), and an *upload or watch folder* (a file placed by a clerk or picked up from a folder; the watch-folder worker never deletes the source file, and flags the folder as degraded if it becomes unreachable). Each ends as a row in the assets table. Uploads start in the `validated` state, and recordings in the `recorded` state. New scheduling forms use the station's configured time zone; generated timestamps and reports retain UTC, and older weekly recording schedules can keep their UTC timing (see Chapter 3).
2. **Finalization.** For a live session, the finalization worker waits until the recording file's size has stopped changing (30 seconds by default), then writes the session-finalized event, the asset and the session's move to `recorded` in one database transaction (Figure 16-4). We could not find which program writes the recording file for a live session; the worker looks for `<live_session_id>.mp4` in the recording target folder.
3. **Packaging.** For a live-session recording, the same worker packages the video as HLS (an adaptive set of segments: 1080p, 720p, 480p and 240p, never scaled up, in 2-second segments), uploads it to a CDN if one is configured, and records the playlist address on the asset. Uploads and scheduled recordings are not packaged automatically; an operator asks for it.
4. **Approval and publish.** A publish operator approves. By default only the *portal* surface publishes, and archive and reach surfaces must be chosen by name (an older version published to

every surface when none was named, which was a safety defect that has been fixed). Approval is refused (HTTP 409) if the asset has no playlist address (`manifest_url`) yet. The caption job is queued first, and if it cannot be queued nothing is published; then the asset's published time is set and the one bus event is published. Publishing does not wait for captions to be reviewed.

5. **The resident portal** polls the public routes. It lists only assets that have a published time and a package, and the video route refuses an asset that is not published. There is no push channel.
6. **Captions come later.** The caption job transcribes the whole recording, files each cue in a review queue, and waits for a records clerk (Figure 16-10). The caption tracks are attached to the package only when review is finished.
7. **Summaries and records** are separate. See the next section.

**Where it differs from what you might expect.**

> **Note:** Finalization does not start captions or a summary. Captions start when publish is approved. A summary is made only when a records clerk asks for one. No broker event links these steps: they are linked through database rows and polling workers.

**Publish surfaces.** Each surface has a state (`blocked`, `not_configured`, `coming_soon`, `pending`, `running`, `succeeded`, `failed`, `overridden`), and the run as a whole shows one of nine dashboard states, from `draft` through `portal_live` and `archive_verified` to `complete` or `failed_needs_action`. A failure on one surface fails only that surface, and it can be retried on its own.

| Surface | Kind | Current behavior |
|---|---|---|
| Portal | canonical | Only default-selected; it makes the record public when its manifest exists. |
| Internet Archive, Local NAS | archive | Optional for other assets; public-record archive rows are required for archive completion but are not default-selected. Mock by default. A real IA/NAS provider uploads the full recording only when a local source file resolves; otherwise the service passes an asset/title payload, not the recording. Local NAS writes filesystem copies; it does not invoke rsync or ZFS. |

| Surface | Kind | Current behavior |
|---|---|---|
| YouTube VOD | reach | Mock by default. The real adapter uploads the recording when its local source path resolves; otherwise its configured media-root path must exist. |
| YouTube Live | reach | Mock by default. The real API adapter creates and binds a YouTube broadcast and stream, but the Publish surface does not connect CivicCast's live output to the returned RTMP ingest address. |
| Cable package | record | optional; `not_configured` when unset |
| Podcast | audience | Selected publishing converts a local recording into MP3, stores the episode, and adds its enclosure to the channel RSS feed. Missing media or encoding failure stays visible and retryable. |
| Subscriber notifications | audience | Optional delivery to confirmed subscribers when selected and configured; outcomes and retries are recorded. Signup confirmation and `/api/staff/subscribe/dispatch-test` are separate. |
| ActivityPub | federation | off by default; when enabled, a note is delivered only after the portal surface succeeded |

Provider selectors default to `mock`, which writes or sends nothing. **Setup > Provider setup** saves fields to `provider-credentials.json`; the Publish registry reads `CIVICCAST_PROVIDER_*` selectors and adapter credentials from the service environment, so saving that file alone does not enable real delivery. A provider result marked `simulated` is not a real external write. Keep public webhooks set to mock: the resident-supplied URL and identifier-derived secret make real webhook selection unsafe.

**Summaries and signed records** Summaries and records sit beside the publish path and are driven by a records clerk.

- **A summary is not automatic.** A clerk asks for one; the cues (the caption text with times) are supplied in the request. The local AI model, a Gemma model run through Ollama at `127.0.0.1:11434`, is called at temperature 0 with a 600-second limit, and the answer must be structured JSON: a narrative plus a list of *sourced claims*, each pointing at the transcript cues it came from. A claim that states a number, a vote or a dollar amount must cite transcript ranges. The program checks that each cited cue exists and that the cited time range lies inside it. It does not check that the claim's words match the cue's words: a person must read it. The record keeps the model's digest, the Ollama version and a fingerprint of the content.
- **Approval** is by a records clerk, who moves a pending draft with sourced claims to `approved`. Both stores enforce the pending-review state; the database update is conditional so a concurrent edit or approval cannot silently overwrite it. Refused or changed drafts return HTTP 409 and must be reviewed again.
- **A signed record** is a PDF/A-3B file built from an approved summary, with the claims, the provenance, the approval and a timestamp token attached as embedded files. Its visible page shows only a header, the identifiers, the fingerprint and the first 480 characters of the narrative; it does not include the transcript.

> **For IT staff:** The record name does not imply a PDF document signature. A real RFC 3161 authority is opt-in through `CIVICCAST_TSA_URL` or `CIVICCAST_TSA_ENABLE`; without that configuration, the token comes from a deterministic stand-in that carries a fixed date of 2026-05-14, not a real RFC 3161 timestamp from a timestamp authority. Verification checks the stored digest and the structure of the token. Do not describe an exported record as legally time-stamped or digitally signed until a real authority is configured and verified.

**Who touches what in this flow** Live sessions are ended by a meeting operator. Uploads are made by records clerks, meeting operators or support administrators. Packaging by hand is for publish operators and setup administrators. Publish approval is for publish operators. Caption review and summary work are for records clerks. The roles are explained in Figure 16-12.

#### Playing a channel: from schedule to air

A CivicCast channel is designed to keep airing through schedule gaps and unusable files by selecting fallback content. Worker outages, host failures and output faults can still interrupt delivery. Figure 16-6 shows the path from the schedule to the stream.

**What this shows.** Recurring program slots are turned into schedule items. A background thread (the *channel automation*) looks at every enabled channel every 2 seconds. It asks the *source plan provider* what should be airing now, and gives the answer to the *egress daemon*, the part that owns each channel's worker. The daemon asks the *preparer* to make the files ready, then starts or instructs the worker. The worker plays the prepared files through one video encoder to the outputs.

**The source plan.** A *source plan* is the list of things a channel should play from now on. Only schedule

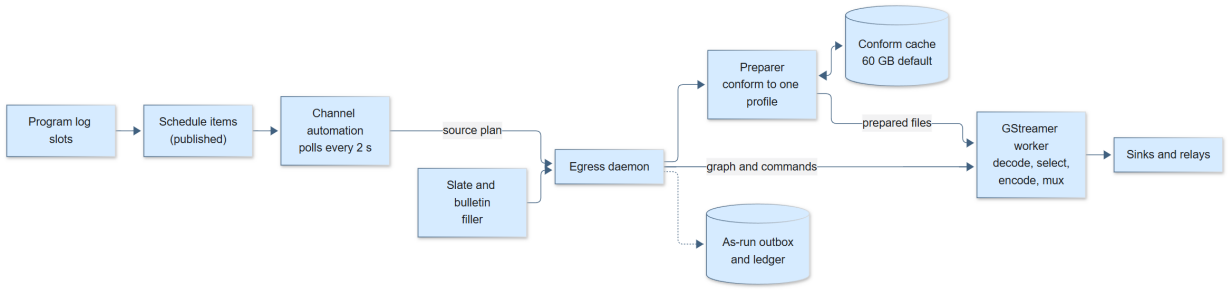



Figure 16-6. The playout pipeline for one channel. The daemon, the preparer and the automation run inside the control plane; the worker is a separate program.

items that are published and are premieres count. Each item is clipped to its slot. If the channel is switched on halfway through an item, the plan starts partway into it (called *join in progress*) so that the stream matches the clock. A gap of up to 30 seconds between items is absorbed. A longer gap is filled with filler. With the default GStreamer engine a plan holds one slice of up to 30 minutes, which keeps the worker's pipeline small; the automation extends it before it runs out (Figure 16-7). A plan segment has one of four kinds: a program (a recorded file), a *slate* (a generated card, used when nothing is scheduled), a *live* source (a network stream, used when an operator takes over the channel) and *cg* filler (bulletin slides). The slate is rendered as a short clip and repeated into one continuous file of 3,600 seconds, so a channel that falls back to slate does not need a second preparation. A live takeover is refused unless the live source has been observed as ready within the last 30 seconds by default (a source that merely exists is not treated as ready).

Scheduled segments carry their schedule-row occurrence ID through preparation. This distinguishes two scheduled uses of the same recording. A short program is allowed to air on its first selection; if the same occurrence is resolved again later with an advanced inpoint, the daemon treats it as the remainder of that program and resolves the slot's filler or next item instead of repeatedly restarting the short clip.

**Preparing a file ("conform").** Video files come in every size and shape. The preparer converts each one to a single *canonical profile* so that every change of program looks the same to the encoder. The default profile is 1280 by 720 pixels at 30 frames a second, H.264 video at 6,000 kbit/s with a keyframe every 60 frames, and AAC stereo audio at 192 kbit/s and 48 kHz, in an MPEG-TS container. A station can change the profile for a channel. The audio is also leveled (Figure 16-9). A conformed file is stored in the *conform cache*, a folder named `conform-cache` under the channel work folder (`C:\ProgramData\CivicCast\data\egress`), keyed by the source file's path, size and modified time, the profile, and the leveling method. A program that has aired before is a cache hit and costs no conversion. A program that has never aired is converted in the time before it is due, and the whole file is converted in the background so that the next airing hits the cache. The cache holds 60 GB by default and discards the oldest entries first. Each preparation step is limited to 300 seconds by default.

> **Note:** A station with long recordings should expect a delay the first time a recording airs, because that is when it is converted. The automation therefore starts preparing early. With the default settings it begins 690 seconds (11.5 minutes) before the projected end of the plan that is

> airing (the program, or its 30-minute slice when the program is longer): two 300-second preparation limits, plus 30 seconds to switch and a 60-second margin. A long file to be prepared next raises that lead time.

**The worker.** Each channel has one worker, a separate GStreamer program started with a description of the channel (a *playout graph*). It decodes the prepared files, chooses between its sources with an *input selector*, and keeps its video encoder and output multiplexers running across programme changes. Outputs can share MPEG-TS encoding or use separate audio and mux branches when their transport or audio settings differ. The graph has two sources: the program and a built-in black picture with silence. The encoder defaults to the OpenH264 software encoder because the shipped runtime does not include GPL-licensed x264. A hardware encoder (NVIDIA or Windows Media Foundation) is used only if the channel's profile asks for it by name, and the start is refused if a requested hardware encoder is missing unless the channel allows the software fallback. A channel with captions turned on must use H.264, because the caption inserter cannot embed captions in HEVC.

**Why one persistent pipeline.** The earlier design started a new encoder at every change of program, which starts a new MPEG-TS session each time. A cable plant's monitors log an error at every one of those splices. ADR 0015 chose to begin with a single long-running FFmpeg process; the code later moved to GStreamer, where the pipeline is never torn down at a program boundary, and the default engine is now GStreamer. ADRs 0015 and 0016 still describe the FFmpeg-concat design and the libx264 default, and are historical in those details. A station can still select the older engine with `CIVICCAST_EGRESS_ENGINE=ffmpeg-concat`. If the GStreamer files on the computer are damaged and cannot be repaired when the service starts, the supervisor selects the FFmpeg engine automatically, raises an alert and marks the channel degraded, so that the channel keeps airing.

**How the daemon and the worker talk.** Two channels are used, for two different reasons.

- *Operator intent* reaches the daemon through a table of commands in the database (start, stop, reload, drain, takeover, handback). The daemon reads it on every tick. ADR 0017 chose this over a direct call or a database notification because a command written to a table survives a restart of either side and a few seconds of delay does not matter for start and stop. The commands are marked consumed as a batch when read, so a command is delivered at most once.
- *Instructions to a running worker* go through a private Windows named pipe, `\\.\pipe\civiccast-worker-<channel>`, which the code restricts to SYSTEM and the service virtual account. The messages are small JSON lines (`reload`, `swap`, `caption`, `stop`) and each is acknowledged within 5 seconds. A `reload` acknowledgement only means "accepted"; the real outcome arrives later in a status file in the channel work folder, so a reload is never reported as done before it is.

**Changing programs without a gap** When one program ends and the next begins, CivicCast switches the worker to the new source while the output keeps running. Figure 16-7 shows the steps, and the watchdogs that stand behind them.

**What this shows.** The automation notices that the program that is airing will end soon and enqueues a `reload`. The daemon builds and prepares the next plan and sends it to the worker. The worker

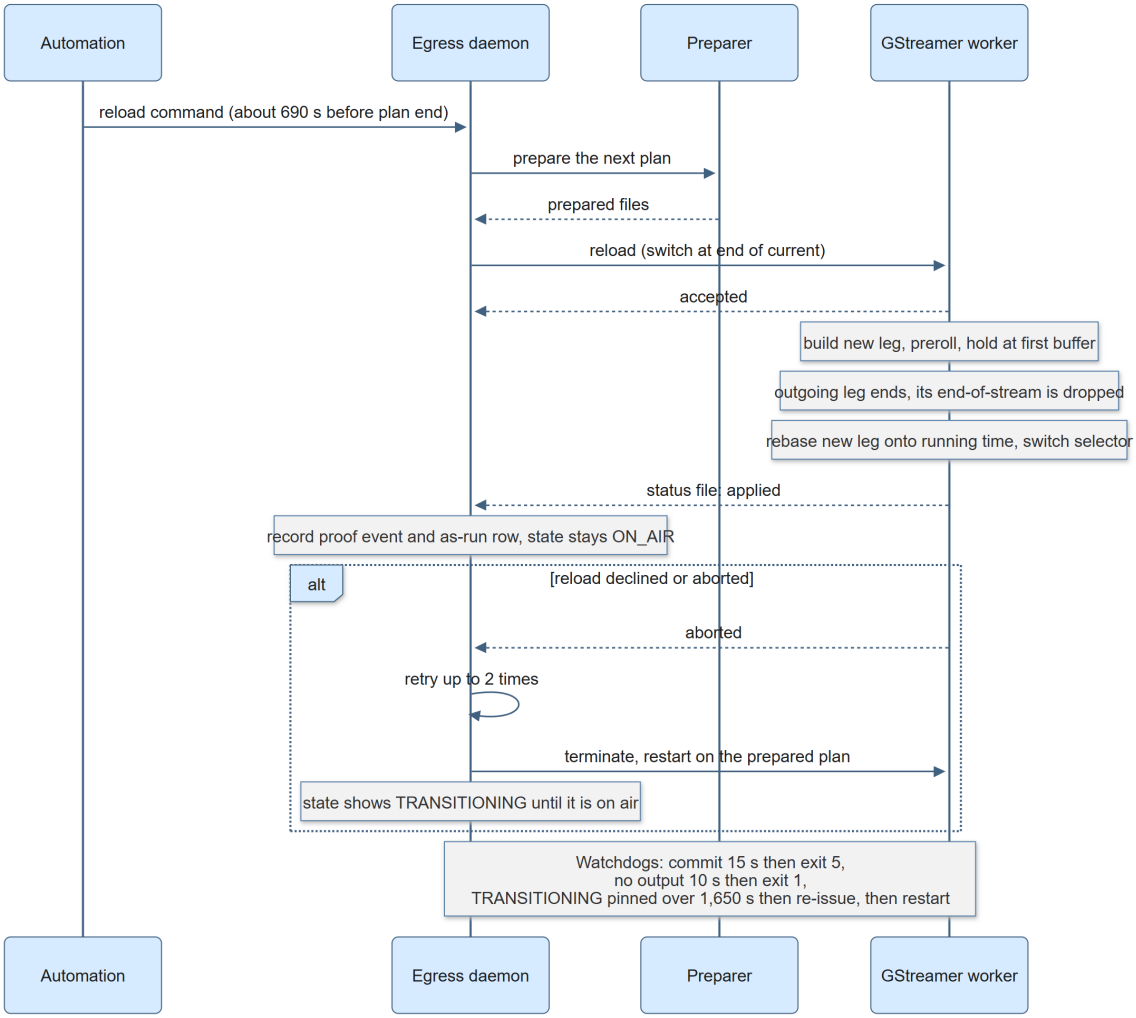



Figure 16-7. A program change (a “seamless reload”) and its watchdogs.

builds the new source beside the old one on the live pipeline and prerolls it, which means it decodes the first picture and sound so the new source is ready to run. The new source then waits, held by a blocking probe so that it does not use CPU or run ahead. When the old program reaches its natural end, the worker discards that end-of-stream signal (otherwise the encoder would see the stream end), shifts the new source's clock so that the output timeline continues without a jump, and switches both the picture and sound selectors. The old source is then removed on a separate thread. The airing item is never cut short and never decoded twice, and the encoder and multiplexer are never restarted. The daemon learns the result from the status file and then records the change in the *proof chain* (a durable log of what each channel did) and in the *as-run* record (what actually aired, which is a different thing from what was scheduled). The channel's state stays `ON_AIR` throughout; only the proof chain notes a transition.

If the worker declines or aborts (a source that cannot be decoded, a timeout), the daemon re-resolves the schedule and tries again, up to two times. If that also fails it falls back to the slower route: it ends the worker and starts a new one on the prepared plan. In that case the channel shows `TRANSITIONING` until the new worker is on air, and the stream is interrupted briefly.

**Unequal picture and sound endings.** The source waits for both streams to reach their natural end. If picture ends first, it repeats the final picture while the remaining sound plays. If sound ends first, it supplies silence while the remaining picture plays. The next programme starts at their common end instead of trimming the longer stream. This work stays inside the existing transition and watchdog handling; an operator stop cancels it. Outputs using loudness normalization have a declared three-second processing delay that the transition drain accounts for.

**The watchdogs.** Three independent timers exist because each one catches a failure the others cannot see.

| Watchdog | Where it runs | What it watches | What it does |
|---|---|---|---|
| Commit watchdog | inside the worker, on its own operating-system thread | the final switch took longer than 15 seconds (default; allowed range 3 to 120) | writes every thread's stack to the log, then ends the worker with exit code 5; the daemon treats this as an ordinary crash |
| Stall watchdog | inside the worker | no output left the multiplexer for 10 seconds (default), or one stream (picture or sound) stopped advancing | ends the worker with exit code 1; the daemon relaunches it |

| Watchdog | Where it runs | What it watches | What it does |
|---|---|---|---|
| Reload-stall watchdog | in the daemon | a channel pinned in `TRANSITIONING` with nothing in flight for longer than the plan length plus lead plus settle time (1,650 seconds at least) | logs an error, re-issues the reload once, and 300 seconds later, if still pinned, ends the worker so the crash relaunch re-plans from the schedule |

Two further guards act on the output a resident would see: if the audio and video start times of the newest HLS segment differ by more than 1 second on three probes in a row (probes are 30 seconds apart), the daemon restarts that channel's worker; and if the HLS playlist stops advancing for 30 seconds after the relay has already tried to heal itself, the daemon also restarts the worker. Each of these two guards is limited to three restarts per channel per hour, after which the shared fallback/-cooldown recovery below takes over.

**Why the watchdog that ends a stuck program change matters.** Before the reload-stall watchdog existed, a program change on the education channel in the lab once left the channel black and silent for about 34 hours. The beta.10 verification record names the watchdog that ends a stuck program change as the fix. The code shows why it can: the daemon measures how long a channel has been pinned in `TRANSITIONING`. The exact stall did not recur in the 8-hour lab run, which was on an earlier internal build.

**What a channel does when things go wrong** Every channel has one of eight states, shown in the operator console and stored with the channel. Figure 16-8 shows how they change.

**What this shows.** `STOPPED` and `ERROR` are off air. `STARTING` is the time spent preparing and launching a worker. `ON_AIR` is a worker airing a real program, and `FALLBACK_SLATE` is a worker airing the slate or bulletins because no real program can be aired right now. `DRAINING` is a graceful stop that lets the worker finish; `STOPPING` is an immediate one. A channel whose `auto_start` switch is on is started again by the automation after an application or computer restart, and a channel on `FALLBACK_SLATE` is moved to the real program as soon as the schedule yields one.

**What heals itself, and what does not.**

| Failure | What CivicCast does |
|---|---|
| The worker crashes or exits unexpectedly | The daemon relaunches it. The first crash relaunches at once. A second crash within 15 seconds waits for the 15-second cooldown. A worker that stays on air for 60 seconds clears the count. |

| Failure | What CivicCast does |
|---|---|
| A worker keeps crashing against the same source (5 crashes in a row) | The daemon stops trusting the source and airs the fallback slate. The automation retries the real source about every 30 seconds, so a source that recovers is picked up again. |
| A slate plan reaches its end | The daemon relaunches once onto the program that is now due. Past that one relaunch it goes to `STOPPED` and names the media to check. |
| A worker was slow to start under load (exit codes 3 or 4) | Relaunched, but counted at most once a minute toward the crash count, because a slow start is not a crash. |
| The database or the daemon restarted while a channel was on air | On start-up, rows still marked on air whose worker has gone are cleared, and a start is queued. |
| A program cannot be prepared in time | The channel airs the slate and the automation retries. |
| The HLS output freezes, or sound and picture drift apart | The guards described above restart the worker (limited to three times an hour). |
| The as-run record cannot be written (database busy or disk full) | The row is written to a local SQLite journal first, retried on every tick, and an alert `asrun-outbox-degraded` is raised. Playout is never stopped by a record-keeping error. |
| A channel is in `ERROR` (invalid configuration, a missing secret, ffmpeg not found) | **Does not self-heal.** An operator must correct the cause and press start. |
| A channel was stopped or drained by an operator | Stays stopped for the current service session, even with `auto_start` enabled. An explicit Start clears this suppression. After a service restart, the configured auto-start setting applies again. |
| The restart budgets of the output guards are spent | Enters a shared recovery cooldown and requests the existing fallback slate, unless an operator override is active. The cooldown ends one hour after the latest budgeted restart. CivicCast then attempts one return to the programme if that recovery slate is still active. Further failures remain subject to the rolling restart limits. An operator stop or drain cancels recovery. |

In the 8-hour lab run on a three-channel station, the service ran without a restart, no channel showed slate after start-up, and the watchdogs did not fire. Two program changes aborted at a sub-second boundary and recovered by themselves in about 3 seconds. That run was on an earlier internal build

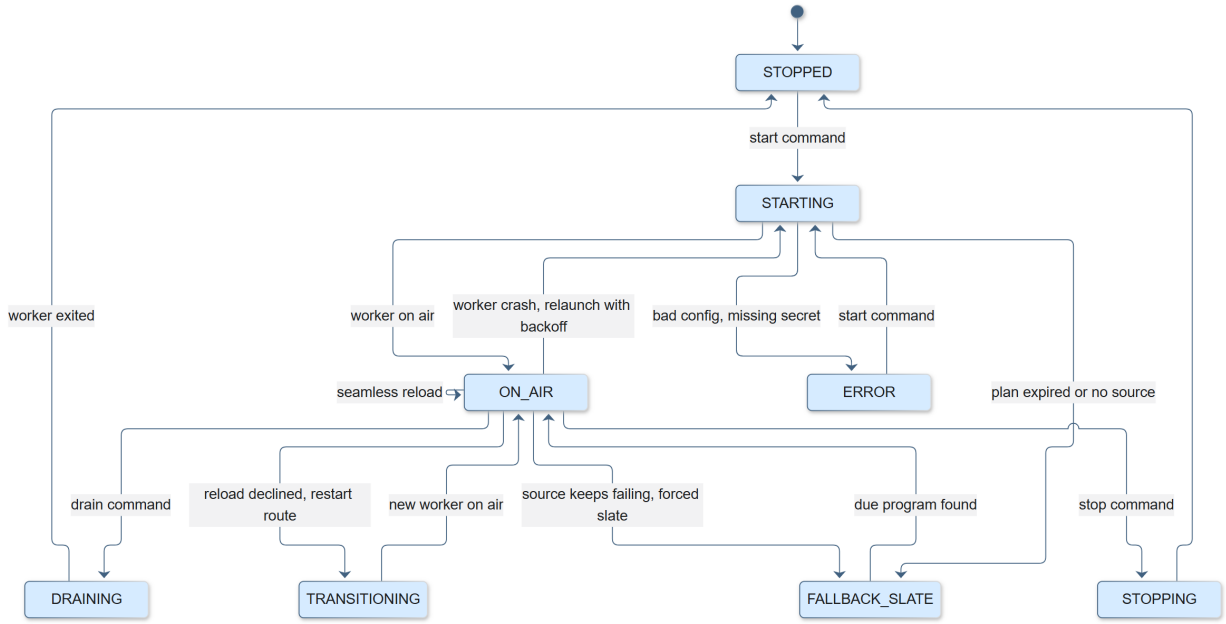



Figure 16-8. Channel states. The state is written by the egress daemon.

of the same engine, on one lab computer; it was not repeated on the published installer.

**Keeping the sound level steady** A public meeting is not steady. A chair at the dais, a speaker who steps back, and a resident at the public-comment microphone can sit 15 to 20 loudness units apart in one recording. CivicCast therefore levels each program once, when it is prepared, and plays it as is. Figure 16-9 shows the steps.

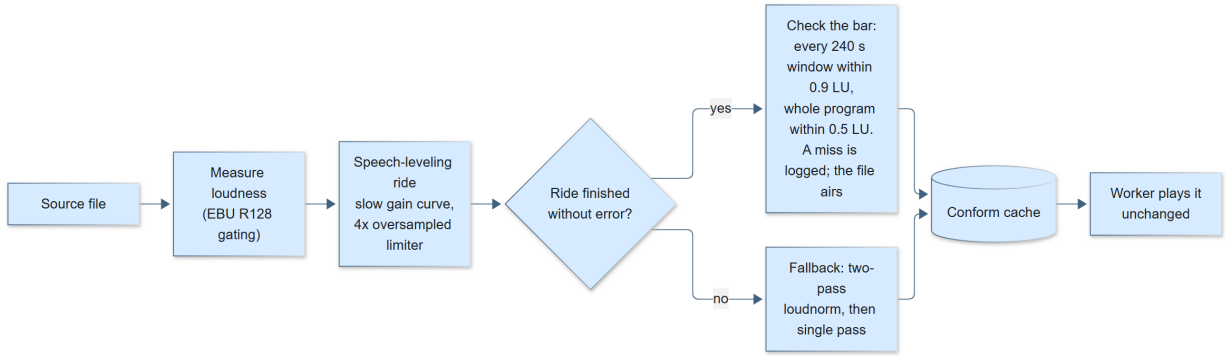



Figure 16-9. How program audio is leveled. The target is -16 LUFS by default.

**What this shows.** The measurement uses the standard EBU R128 method. A *speech-leveling ride* uses a 45-second window and moves the gain slowly (up to 0.5 dB a second, within a range of -6 dB to +18 dB in the code defaults, with a smaller cap for room tone) to follow speakers in a prepared file, instead of a single gain for the whole program, which cannot put every four-minute stretch on target. A limiter that works on a four-times oversampled copy of the signal holds the true peak at -1.5 dBTP, and a loop measures the result and corrects it up to four times. The result is then checked against a bar: every 240-second window within 0.9 LU of the target and the whole program within 0.5 LU. A

result that misses the bar is logged as a warning and still airs. This ride processes recorded source audio during preparation; it is not a gain guarantee for every live source or output. Only if the ride itself fails (an error in the leveler or the final mux) does the preparer fall back to the standard two-pass `loudnorm` filter, and then to a single pass if the measuring pass fails; it records which method was used.

The loudness target is a setting of the channel and not a fixed number, because streaming and cable expect different levels (ADR 0014). The default is -16 LUFS with a tolerance setting of 2 LU. The configuration also lets each output (a *sink*) name its own regime: streaming (-16), ATSC A/85 (-24) or EBU R128 (-23). Applying a cable headend profile sets the channel to -24. The GStreamer source builds a separate audio processing branch when an output needs a different target, using `audioloudnorm` before AAC encoding. It retains the configured target when HLS or cable delivery is routed through a loopback relay. Configured alert-tone filtering uses two band-reject filters around 853 Hz and 960 Hz on SRT and RTMP outputs, matching the existing destination policy.

> **Historical preparation limit (beta.11):** A stretch of a recording that is too quiet for the ride's largest boost is listed in the report and is not leveled. The 240-second-window measurements below come from historical lab runs; the station's leveling check also recorded two programs whose worst four-minute stretch was 1.94 LU and 1.39 LU. These preparation measurements do not describe the later per-output normalization. Sprint 3 records verification of the new emitted-output paths separately.

#### Captions: live and after the meeting

CivicCast captions in two different ways, for two different needs. The *live* path captions a channel while it airs, so captions can be embedded in the stream. The *offline* path captions a finished recording after it is published, with a human review step before anything is attached. Figure 16-10 shows both, and marks the points where audio can be thrown away.

**Native live path:** The runtime uses Whistle as the CPU primary, with Whisper fallback, immediate first-pass publication, and bounded temporary working state. It does not create permanent per-cue review records or evidence WAVs, and review-archive discovery is not a live-caption or broadcast-readiness prerequisite. Recorded captions still use their separate review workflow.

**The live path, step by step.**

1. **The audio tap.** Inside each channel's worker, a copy of the program audio is split off before it is encoded, converted to mono 16 kHz, and placed in a queue that never blocks the program: if it is full, it drops the oldest audio. This exists only if live captions are switched on for the station and a caption folder is set.
2. **Chunks on disk.** A writer collects the audio into five-second WAV files named `chunk-NNNNNN.wav` in `C:\ProgramData\CivicCast\data\caption-tap\<channel>`, writing each under a temporary name and renaming it so a half-written file is never read. Whistle's live tap retains at most 42 waiting completed segments (210 seconds) per channel, plus inputs currently being processed and the segment being written. Runtimes without Whistle's processing grace use a 12-segment working-file ceiling. Consumed chunks are deleted; the

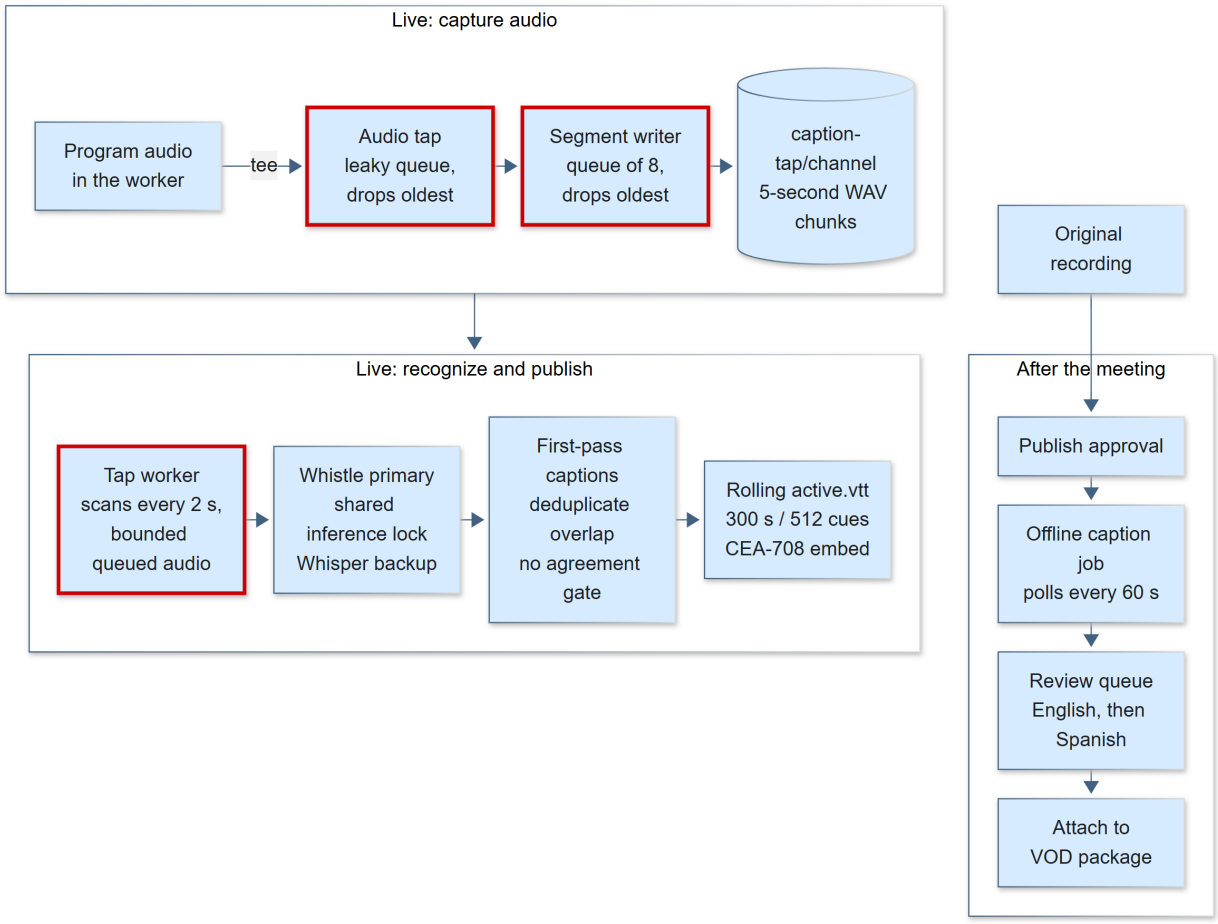



Figure 16-10. The caption pipelines. Boxes with a heavy red outline are where live audio can be shed under load. The offline path sheds nothing.

previous overlap needed for recognition remains in memory.

3. **The tap worker** scans every 2 seconds, takes the finished chunks of each channel, adds a five-second overlap from the previous chunk, and sends them to the speech recognizer.
4. **The recognizer** uses Whistle as the native live primary. Its child processes use the CPU; one station-wide inference lock allows one primary request at a time across that station's channels. A Whistle child startup or recognition attempt has a 30-second deadline, excluding time waiting for that shared lock. On failure or timeout, the affected channel uses Whisper while a bounded recovery probe tries a fresh Whistle process after an initial 30-second cooldown. Only one recovery probe runs across the station at a time, and it shares the primary-inference lock. Failed probes back off for 60, 120 and 240 seconds, capped at 300 seconds. A successful probe promotes the replacement for later chunks; probe text is not published. Recovery is automatic and does not require a service restart, but no fixed recovery time is guaranteed. Whisper fallback requests have a configurable 120-second default deadline. A failed Whisper child is closed and replayed once immediately; a repeated replacement failure starts a 30-second cooldown before another child is created. Whisper remains the recorded-caption engine and can use the staged NVIDIA CUDA runtime. Both engines run locally; neither is Ollama. Recording/offline model choices remain separate.
5. **The stabilizer** publishes the first recognition, including low-confidence text, and removes overlap already aired. It does not require two readings to agree. Recent cues are limited to 300 seconds and at most 512 cues. A per-session identifier and monotonic sequence distinguish fresh captions after a worker replacement without accumulating a session-long identifier map.
6. **Output.** The rolling result is written to `active.vtt` in the channel's work folder. Live cues are not filed in a permanent review queue and do not generate evidence WAVs. A separate *caption feed* worker reads `active.vtt` every 2 seconds and sends each new cue to the running worker, which turns it into closed captions (CEA-608/708 data) and inserts them into the video. Cues are split into pages of 32 columns by 2 rows without discarding words. Accepted caption pages are recorded with their actual transport timestamps in a bounded `captions/delivered.json` receipt: at most 512 pages and 180 seconds, reset with the worker. This is timing metadata, not an audio archive.
7. **The proof.** A decode-back check preserves the emitted MPEG-TS clock and maps both accepted and decoded captions to the actual capture's video timestamp origin. It compares captions wholly inside that captured window; missing clock information or no eligible expected captions cannot pass. The channel's caption status is `on` only while a recent check has passed; a missing, stale or failed check reads `not-verified` (ADR 0018).

> **Note:** Live captions are switched off in a new station profile until an administrator turns them on. The caption tap runs in its mode `inline` on a native station, but the audio tap and the embed are built only when the station's live-captions setting is on. Live captions are English only: the live path has no translation step.

**Where audio can be shed, and what happens to the captions.** The rule in the code is that captions are best effort and the channel on air always wins. When the station cannot recognize speech as fast as it arrives, audio is thrown away rather than letting captioning slow playout down.

| Where | What is dropped | What happens to the captions |
|---|---|---|
| The audio tap in the worker | The oldest audio when the queue (200 buffers or about 10 MiB) is full, or when the next stage holds 32 buffers | A gap in the chunks; playout is unaffected |
| The segment writer | The oldest waiting segment when its queue of 8 segments (about 40 seconds) is full | Missing five-second chunks; a slow disk is not fatal to air |
| The tap worker's backlog gate: short overload | Nothing yet. If a channel has more than 2 finished chunks waiting for fewer than 15 scans in a row (about 30 seconds), only the oldest 2 are processed and the rest wait | Captions continue, a little late |
| The backlog gate: catch-up shed | After 15 scans in a row over the limit, the oldest waiting chunks are deleted and only the newest 2 are kept | Captions keep running from the newest audio; the skipped speech is never captioned, and one warning is logged |
| The backlog gate: pause | If the station sheds 3 times within 5 minutes, the channel's captions are blanked and paused for 120 seconds, then 240, 480 and so on up to 900 seconds, with every new chunk deleted unrecognized during the pause | Captions are off for that channel; the status shows `paused` and the time to resume; the channel steps back down one rung at a time after sustained clean scans |
| Queued working-audio limit | Oldest waiting completed segments beyond the finite working-file ceiling (42 with Whistle at five-second cadence; 12 without its grace); in-flight inputs and the segment being written are protected | Captions resume from retained audio; discards are reported, without creating a permanent review archive |
| An operator switches captions off | Every chunk is deleted, the caption file is blanked | Captions off until switched on |
| A new session or an unreadable chunk | Previous-session audio or unusable working audio is removed in ordinary live mode | Captions restart cleanly; the error is logged without accumulating an audio quarantine archive |

> **Note:** Live captions remain best-effort. The tap can discard audio under sustained overload so playout keeps priority; first-pass publication does not guarantee that every spoken word will be captioned. Historical Beta 10 measurements are in Appendix H.

**The offline path.** When recorded-caption processing is enabled, publish approval queues a durable job row for the asset. A worker polls every 60 seconds (retrying with a 300-second backoff, at most four attempts) and transcribes the recording in 30-second chunks. Every cue goes to the review queue. A records clerk approves, edits or rejects each English cue; a low-confidence cue needs an explicit acknowledgement and valid audio evidence. If Spanish is selected, the reviewed English is translated and the Spanish cues receive their own review. The station can require all selected languages before caption attachment, or allow reviewed English to attach first while Spanish review continues. Only reviewed text is written into caption sidecars and segmented manifest tracks; configured CDN copies are updated after attachment.

**Recorded-caption settings. Setup > Station Profile > Recorded captions** controls enabled processing, English with optional Spanish, and the completion requirement. Disabled processing skips new jobs and pauses existing ones without failing startup. English-only processing does not need a translator. These settings are separate from live-caption performance and introduce no release-percentage threshold.

**Historical beta.10 evidence retention:** Its live path created per-cue review evidence and used age-based cleanup for evidence and raw chunks. In beta.11 ordinary live operation no longer creates that permanent review/evidence archive or scans it for retention/readiness. Original recordings, archived caption tracks and recorded-caption review are unchanged.

**Why the paths are separate.** Continuous live captions operate with bounded working resources rather than creating an ever-growing human-review obligation. The beta.11 runtime publishes recognition immediately, deduplicates overlap and releases consumed audio. Playout retains priority, inference concurrency remains bounded, and overload remains visible. Recording review remains a separate workflow using the original recording and complete caption tracks.

#### Where the video goes: outputs and relays

The worker shares encoded video across outputs. Outputs needing their own audio processing or RTMP transport have separate audio and mux branches: MPEG-TS for TS destinations, FLV for RTMP. A channel can have several *sinks* (outputs). Some sinks are written directly by GStreamer, and others go through a small helper program, a *relay*, started and watched by the control plane. Figure 16-11 shows the paths.

**What this shows.** The GStreamer source writes these kinds of sink directly: `udp-ts` and `local-ts` (a UDP socket, multicast supported), `srt` (with a passphrase fetched from a stored secret, and refused if the secret cannot be resolved), `file`, and `rtmp` through an FLV mux and `rtmp2sink`. RTMP stream-key handles are resolved in the worker rather than written into the serialized graph. The other kinds of output are made by relays, as follows.

**The TS relay (for cable headends).** Each time a worker is replaced, its multiplexer starts a new MPEG-TS session: new continuity counters and a new source port. A cable plant's monitors log an

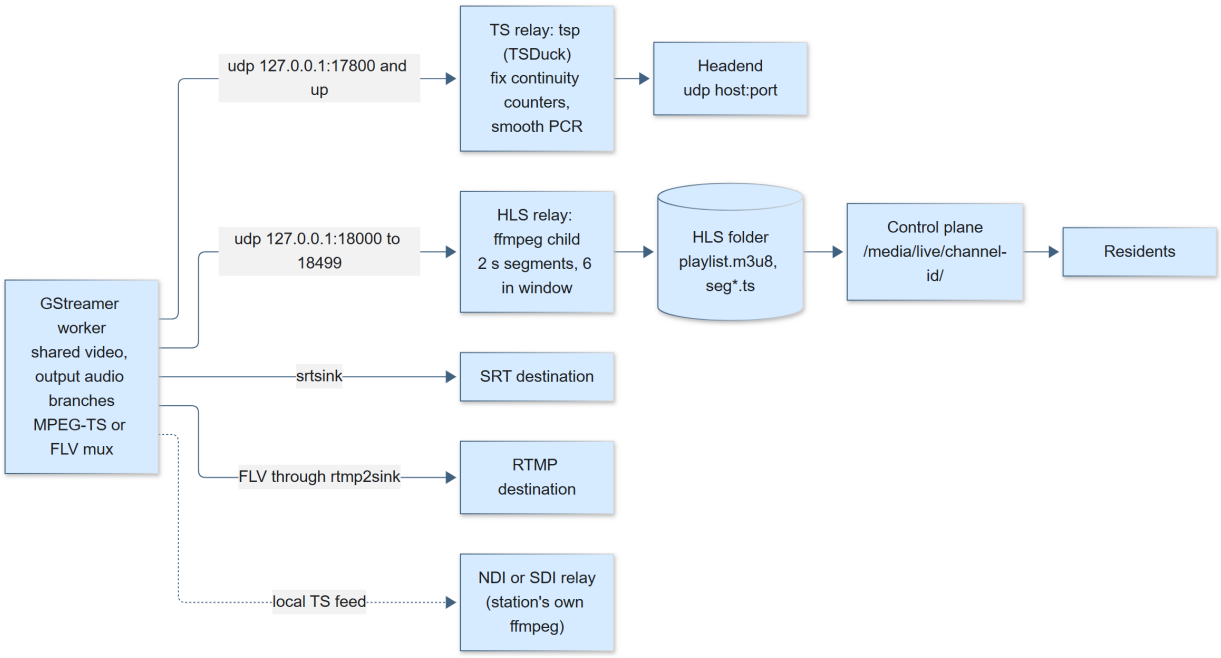



Figure 16-11. Output paths for one channel. The TS relay is used only for `udp-ts` sinks and only if TSDuck is present. A `file` sink writes the TS stream straight to a file, and is not shown.

error at each such change. When TSDuck is installed (it is an optional part of the server pack) and the channel has a `udp-ts` sink, the encoder instead sends to a loopback port (counting up from 17800), and a channel-lifetime `tsp` process rewrites the continuity counters (`continuity --fix`), re-stamps the PCR (`pcradjust`) and sends to the real destination from a fixed source port. This preserves the relay source port and repairs transport timing/counters across worker replacements; it does not generate missing programme packets or guarantee uninterrupted playback while an encoder is down. The mode is `CIVICCAST_TS_RELAY` (default `auto`: relay when `tsp` is found, and otherwise warn and send direct). `on` requires the relay but still passes the stream through rather than take the channel off air. Cable headend presets built into the program (`generic-udp-spts`, `comcast-mtd-sd`, `comcast-mtd-hd`, `telvue-hypercaster-ip`, `harmonic-spectrum-ts` and `leightronix-file-drop`) each set the encoder profile and the output sink and set the channel loudness target to -24; none of them has field proof. A seventh, local rehearsal profile only adds a web-preview HLS sink and leaves the encoder profile and loudness alone.

**The HLS relay (for residents and the portal).** The `hls` sink is rewritten to a local UDP sink on `127.0.0.1` (a port from 18000 to 18499 chosen from the sink's address). A supervised ffmpeg child copies the video and sound without re-encoding and writes segments (`seg%09d.ts`) with a six-segment sliding window. Two seconds is the requested segment duration; longer keyframe intervals produce longer segments. FFmpeg finishes each segment through a temporary file and does not independently delete old segments. A publisher validates the private playlist and its referenced files before copying it to `playlist.m3u8`. It protects both the public and staging windows, retains removed public segments for the preceding playlist's measured duration plus the segment's own duration, then prunes eligible older files. This gives viewers time to fetch segments from a recently

served playlist. The control plane serves the folder at `/media/live/<channel>/...` and refuses folders outside its approved media roots.

Live playlist requests copy the complete playlist under the same in-process lock used by its publisher. The lock is released before sending the response, so a slow client cannot hold up publication. Segment requests continue to stream their files. This synchronization applies to the station's single control-plane process; external programs reading the directory do not participate in the lock.

On a fresh or restarted relay, cleanup withdraws the old public playlist before removing media. Recently referenced segments retain their playback grace, and other retained numeric segments receive a conservative window of grace. New segment numbering starts beyond retained files so the new relay cannot overwrite a segment still being fetched by an old viewer.

**What the HLS relay does when it goes wrong.** A running process is not the same as a working output, so the supervisor looks at the playlist itself.

- A playlist that has not advanced for 30 seconds is judged stalled, and the relay child is restarted once (a one-shot self-heal). A fresh child gets 20 seconds' grace before it is judged.
- A child that starts while its input has no video (or no sound) would otherwise fix its output to the streams it saw and serve a half program for hours. The relay therefore requires every stream the sink carries, and checks that the newest segment holds both picture and sound after any restart; it retries three times quickly, then once a minute.
- If the playlist is still frozen 30 seconds after a self-heal, the daemon restarts the worker (at most three times an hour; Figure 16-7).
- Each relay child's error output is kept in `C:\ProgramData\CivicCast\data\egress\<channel>\logs\hls-relay.<sink>.stderr.log` (5 MiB, current and one previous), because evidence was lost in the earlier incidents that led to this design.
- The daemon can also reclaim a relay's UDP port from an orphaned relay process left by a control plane that died, killing it only if its command line matches the relay's exact input address and it is not a process the daemon owns.

**NDI and SDI.** NDI and SDI outputs depend on software the station brings itself. NDI cannot be bundled because of its licence, and SDI needs Blackmagic's driver and an ffmpeg build with the DeckLink muxer. When `CIVICCAST_NDI_FFMPEG` or `CIVICCAST_SDI_FFMPEG` names such a build and the channel has an NDI name or SDI device set, a separate supervised process reads the channel's local transport stream and republishes it as NDI or sends it to the card. It is a separate process on purpose: a dying relay must never take a cable channel off air. A relay that stops restarts after 5 seconds, then 15, then 60.

> **Note:** The verification record for the package you install states whether any physical DeckLink SDI capture/output, real cable-headend acceptance or NDI interoperability was exercised for that exact package. A transport-stream capture inside Windows Sandbox is not the same as a headend accepting the signal.

**Live takeover.** An operator can put a live network source (SRT, UDP, RTMP, RTSP or HTTP) on air in place of the schedule, and hand it back. The control plane first writes an audit record, then queues

a `takeover` command. Before either, it requires the live source to have been observed ready within the last 30 seconds (adjustable from 5 to 300 seconds); it re-probes if the observation is older, and refuses if anything is uncertain. The takeover is carried out as a seamless reload of the worker's source, not as a switch to a separate live input. While a takeover or a forced slate is active, the automation does not roll the channel over to the scheduled plan, so it does not fight the operator.

**The record of what aired.** At every actual change of source the daemon writes an as-run row: what was on, which asset, and when it began and ended (the slate and filler are labelled as such). This is the station's proof of performance. It is written through the local journal described under Figure 16-4.

### Who may do what: sign-in and roles

CivicCast decides who a caller is in the control plane, with a bearer token, and what they may do with a role. Figure 16-12 shows how an operator signs in at the console and how a request is checked.

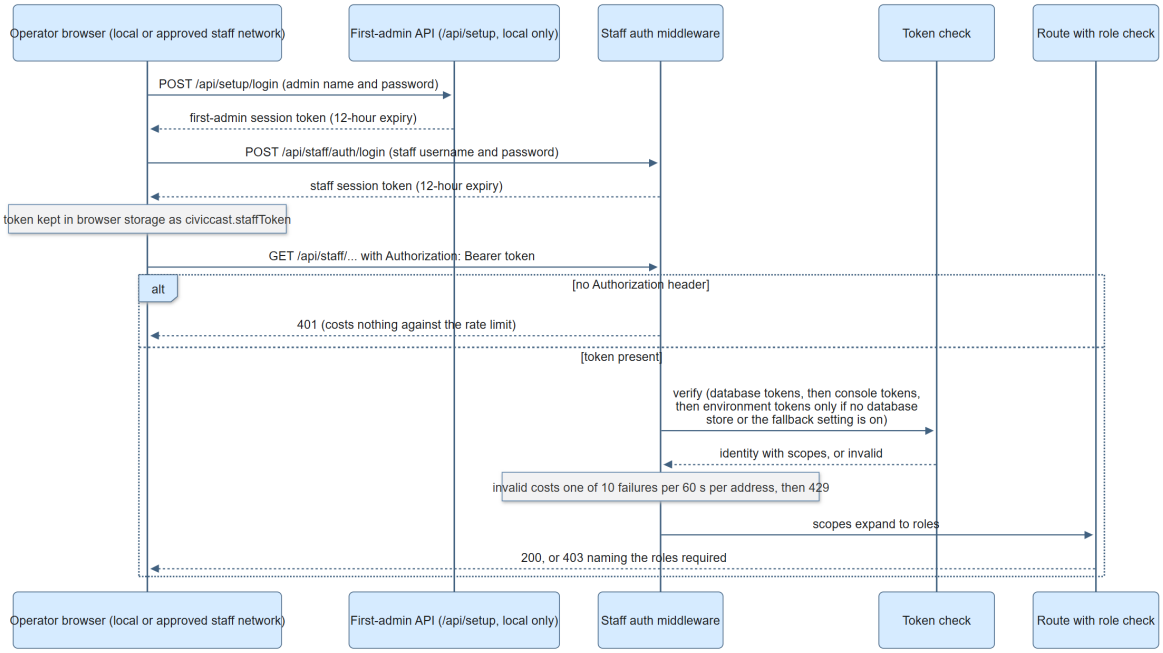



Figure 16-12. Sign-in and request authorization.

**What this shows.** First-admin setup and sign-in use the local First Setup screen. Staff use the separate `/login` page and `POST /api/staff/auth/login`; it returns a station-local session token after checking the username and password. The first administrator uses `POST /api/setup/login` on the station computer. Sessions last 12 hours, sign-in does not end another valid session, and at most 20 console sessions are retained. The browser sends its token on protected requests. The middleware checks `/api/staff/*` paths; `/api/setup/*` has a separate local-only guard, and public routes are intentionally available without staff sign-in. The three token kinds are:

| Kind | Looks like | Kept as | Used for |
|---|---|---|---|
| Console session token (first admin or staff) | `ccst_` plus a random string | salted HMAC-SHA256 in `station-state.json`; expires after 12 hours; at most 20 console sessions retained | people at the operator console |
| Lifecycle token | `ccst_` plus an id and a secret | a PBKDF2-SHA256 hash (210,000 iterations) in the `staff_tokens` table, with an audit table | scripts and the command line (`civiccast token issue`, `list`, `revoke`, `rotate`); can be revoked and rotated |
| Environment token | `ccenv1_` plus a random string | listed in `CIVIC-CAST_STAFF_TOKENS` with its roles | fixed configuration; cannot be revoked until it is removed from the setting; on a station that has the database token store it is accepted only when `CIVICCAST_STAFF_TOKENS_FALLBACK_WITH_DB=1` is also set |

The five roles are `setup_admin`, `meeting_operator`, `records_clerk`, `publish_operator` and `support_admin`. Console sessions use the roles assigned to that account; lifecycle and environment API tokens use their configured scopes, with `admin` and `operator` each expanding to all five roles. A token with no scopes has no roles. Each route lists the roles that may use it; one matching role is enough, and a wrong role gets a 403 that names the roles by their identifiers. In outline, from a scan of the route decorators (most of the roughly 490 operations in the API inventory name a role; the rest are open to any signed-in token, are public, or are checked inside the route; we did not re-count them for this edition): setup administrators configure the station, cable, graphics, playout and recording; meeting operators run live sessions, agendas, the program log and the facility; records clerks review captions, summaries and agendas; publish operators manage publishing, the schedule, program guide exports and moderation; and support administrators handle support bundles, reports and read-only alerts. The Appendix on roles has the full table. The navigation menu hides screens a role cannot use, but the menu is a convenience, not a security control: the route enforces the rule.

**Where sign-in is allowed from.** First setup, first-admin sign-in and recovery require the station computer (or a Remote Desktop browser running there). Staff sign-in can use the separate hostname

from a configured CIDR when Caddy mode is enabled. The public hostname does not forward staff routes; staff bearer and role checks still apply. See the network access guide.

**Why it is built this way.** The design assumes one station computer and a small number of people. Staff network access is explicitly restricted by the configured CIDR list and still requires a staff account; the cost is that remote staff need the approved network or a Remote Desktop session. Failed token checks are rate-limited by the address the web server reports; a request with no token is not counted, so a signed-out browser that loads the console cannot lock itself out.

The Setup admin can create staff accounts, assign roles, disable access, reset passwords, and review or revoke station-local sessions from **Staff accounts**. The first administrator remains the recovery account and cannot be demoted or disabled. Password resets, disablement and role changes revoke that account's sessions. See the staff accounts guide.

> **Warning:** The control plane speaks plain HTTP to its loopback upstream. Caddy provides the supported HTTPS edge and separates public and staff access; the database connection remains plain on loopback. The local certificate authority's certificates do not encrypt these connections. Browser sessions are stored in local storage, so closing a browser does not sign out; first-admin and staff session tokens expire after 12 hours or end sooner on sign-out or revocation. Lifecycle API tokens have their separate revoke/rotate controls.

#### Installing and activating

The installer is a standard Windows setup program built with Tauri and NSIS, with a series of steps (called *hooks*) that do the real work. Setup runs elevated (as an administrator). Figure 16-13 shows the steps in order.

**What this shows.**

1. **The kit.** The installer expects a folder holding `setup.exe`, a `packs` folder of signed `.ccpack` files and a `station` folder with the signed station index and the model packs. There is no download in this elevated phase: if the packs are not beside `setup.exe`, stage-packs fails (exit 110) and setup tells you which packs are missing.
2. **Packs and signatures.** A `.ccpack` is a ZIP file with a manifest listing every file's SHA-256 and size, and an Ed25519 signature over that manifest. The installer carries the public key it will trust (a development key is refused unless explicitly allowed). It checks the signature, every file and the component name and version before it extracts anything, then checks the extracted tree again. Four runtime packs are required: the server binaries (PostgreSQL tools, and TS-Duck if included), the application payload (Python, CivicCast and both web apps), ffmpeg and Ollama. The fifth runtime pack, NVIDIA CUDA libraries for Whisper, is optional. The signed station bundle also contains six station packs, including Whistle, the Medium Whisper floor and AI models.
3. **The database.** Provisioning creates a PostgreSQL 17 cluster with `initdb` in `C:\ProgramData\CivicCast\data\pgdata`, writes a configuration that listens on `127.0.0.1` only, creates the `civiccast` database and the `civiccast_svc` role with a generated password, picks the first free port of 5432, 5433, 5434, 5435 and 5544, restricts the data folder's access list, and

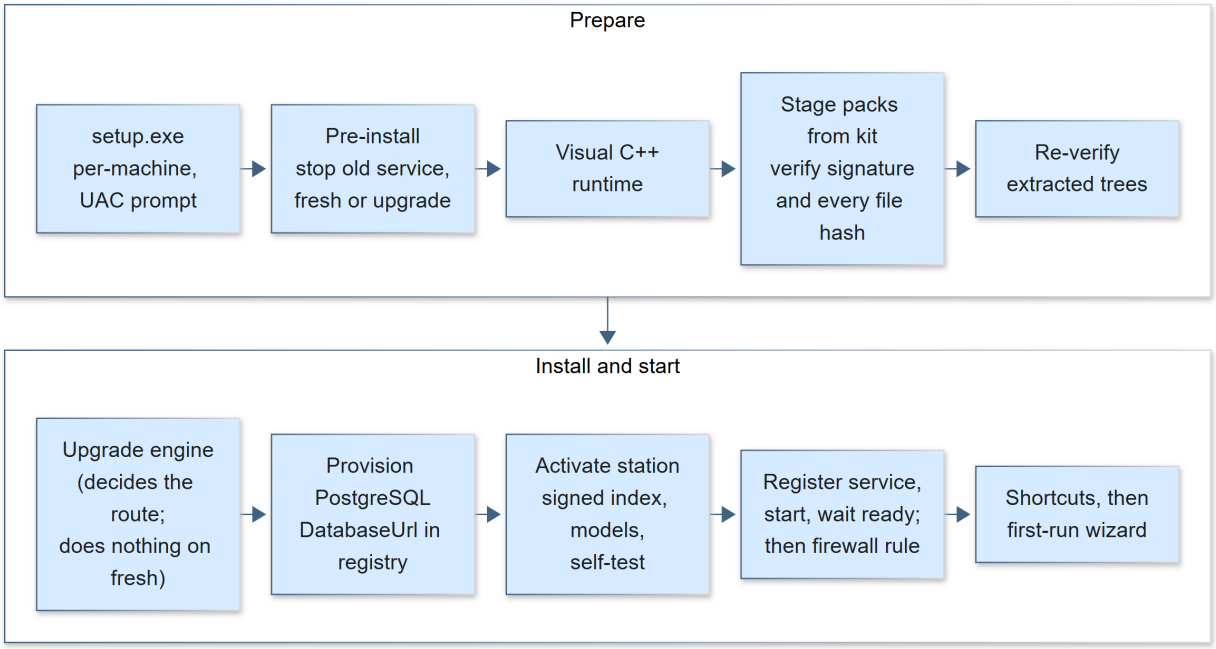



Figure 16-13. Installation and activation. Setup saves prior state before replacing files and automatically restores it after a failure when recovery succeeds; exit 136 reports that recovery could not be prepared or completed.

writes the connection string to the registry where SYSTEM, administrators and the station service identity can read it. A provisioning run that ends in failure is not retried; it leaves a recovery document in `provision`. Provisioning also refuses to continue if the older WSL-based edition is registered, or if it cannot tell which edition owns the machine.

4. **Activation.** Activation verifies the signed station index and each station pack's signature, size and hash, checks that there is room (the sum of the pack sizes plus 2 GB), extracts Whistle, Whisper and the AI models, and then runs a *self-test* before it declares the station activated. The self-test runs the Whisper caption model on a bundled sample recording (the text must contain the expected words), starts a private Ollama and checks its version, and sends test prompts to each AI model; per the installer inventory it also checks that embedded Python, PostgreSQL and ffmpeg run. Only then does it write `station-set.json` and `activation-self-test.json`. The service will not start the station without them.
5. **The service.** Setup registers `CivicCastSupervisor` under `NT SERVICE\CivicCastSupervisor` after any required legacy profile migration, starts it and waits up to 180 seconds for the station to answer `/health` with a current database schema, then adds the firewall rule, writes the installed version to the registry and creates Start Menu and Desktop shortcuts that open `http://127.0.0.1:8000/operator/`.
6. **The first-run wizard.** After setup, a non-elevated window checks the computer's hardware, shows the components the station still needs and downloads selected optional Large Whisper and CUDA files if missing. The signed setup supplies Whistle and the Medium Whisper floor. The wizard may preselect Large and CUDA for an NVIDIA graphics card with at least 8 GB of video memory; CUDA accelerates Whisper, not Whistle.

| Step | Setup exit code when it fails |
|---|---|
| Pre-install: could not stop the old service | 120 |
| Stage packs: required pack missing or untrusted | 110 |
| Verify extracted server binaries, app payload, ffmpeg, Ollama | 111, 112, 121, 122 |
| Provision the database | 116, 117; 127 or 135 if another CivicCast edition owns the machine |
| Activate: index or pack could not be verified; self-test or space failure | 123 (the activation tool itself exits 66, 67, 78, 64 or 65) |
| Register the service; start it and wait | 118; 126 if it will not start; 125 if it starts but does not serve |
| Firewall rule | 119 |
| Automatic setup recovery could not be prepared or completed | 136 |

Each of these steps writes a time-stamped line before and after to `C:\ProgramData\CivicCast\install-progress.log`.

**Why it is built this way.** Every pack is signed and checked twice because setup runs elevated and installs programs that run under the restricted station service identity. The self-test exists so a station that looks installed but cannot caption or summarize is stopped at setup rather than discovered at the first meeting. The service refuses to start without the activation records for the same reason.

> **Known issue (beta.11):** When the activation self-test fails, the dialog tells you to find the cause in `install-progress.log`, but the cause is shown only in the setup details pane, and exit 67 is also used for a full disk. A new install on a computer with a surviving data folder is handled by resetting the database password. The installer source also includes automatic setup recovery. Exit 136 is used when recovery cannot be prepared or a recovery attempt cannot be completed. See Chapter 10.

#### Updating, and going back

Running a newer `setup.exe` over an existing station upgrades it. The installer skips the generic **Already Installed** page. A read-only version check refuses an older setup before stopping the service or replacing files; accepted installs arm recovery, stop the service and snapshot the prior state before payload replacement. Figure 16-14 shows the database steps and what happens if one fails.

**What this shows.** The engine runs a series of steps and writes a *journal* after each one. The engine can resume a nonterminal journal if it is invoked again, but the setup wrapper handles an interrupted installer by restoring the pre-setup snapshot through its recovery task or a later setup. The engine first takes the *maintenance interlock*, a flag in the registry that the supervisor reads to start the station read-only. It stops writers and checks that the database is quiet, then takes a full backup and proves it by restoring it into a scratch database and comparing rows, checksums and the schema

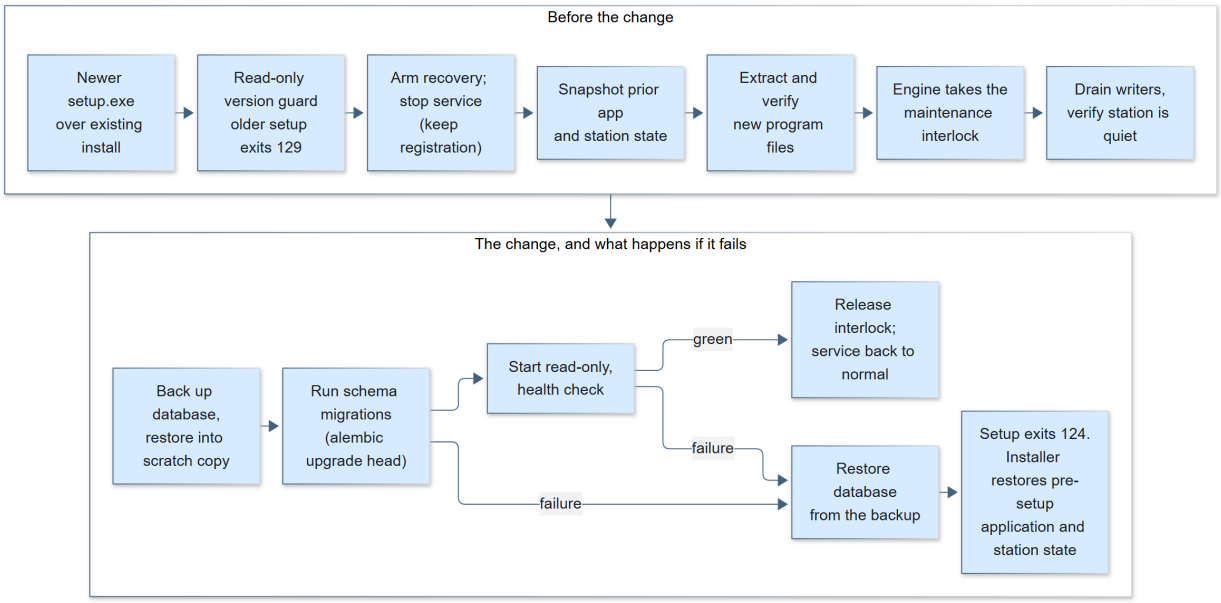



Figure 16-14. Upgrade and rollback.

revision. If that proof fails, the upgrade stops. It runs the migrations, starts the new code in maintenance mode, which brings up the database and the control plane and starts no video worker, and checks that the station is healthy. Only then does it release the interlock. The enclosing installer transaction retains a size-checked temporary snapshot of the prior application and station state until setup commits.

If the migration itself, or a step after it, fails, the engine restores the database from its backup. If that restore fails, the engine halts and writes `UPGRADE-RECOVERY.md`; setup then attempts to restore the whole pre-setup snapshot, including the old application files, local database and service state. Setup preserves the failed-run engine log and journal under `install-transaction\failure-diagnostics` before restoring the old `upgrade` directory, so a generated `UPGRADE-RECOVERY.md` may not remain at its original path. If immediate restoration fails, setup reports that and retains recovery records; a recovery attempt during startup or a later setup can return exit 136 if it cannot prepare or finish. A new release can also declare that its migration cannot be reversed; the engine then refuses to upgrade automatically (the engine exits 30, which setup reports as exit 114). The normal read-only version guard refuses an older setup before service or file changes (exit 129); a defensive engine check also routes through installer restoration if reached later.

**What a rollback does not do.**

- **The engine's own rollback does not put old program files back.** The installer's enclosing transaction does: when its temporary snapshot is intact, setup restores the previous application files, database, relevant app state and service state after a failed setup step. Keep a separate station backup; this temporary copy exists only until commit or successful restoration.
- **The backup does not contain your recordings.** It is a database dump with an integrity manifest, saved in `C:\ProgramData\CivicCast\upgrade\backups\pre-<version>`. The upgrade does not touch `data\uploads`, but the backup does not protect it either.

- **A failed upgrade journal may block the next engine run.** A terminal failure journal in the pre-setup `upgrade` directory makes the engine refuse a new run (exit 128) until someone archives or moves `upgrade-journal.json` aside; nothing removes that prior journal automatically. Setup restoration preserves the previous `upgrade` directory and keeps failed-run engine logs and the journal under `install-transaction\failure-diagnostics`.
- **There is no manual restore tool in setup.** The exit 114 message mentions a manual path with operator acknowledgement; we did not find the steps for it.

> **Warning:** Before any upgrade, take a full backup (the database and the whole of `C:\Program-Data\CivicCast`, plus the station files under `C:\ProgramData\CivicCast\profile` and the selected media roots in the `.ccbackup` inventory) and keep the station's recovery path available. Check the release verification record matching the package you plan to install.

#### Building and publishing a release

This is about how the files you download came to exist, and what checks stand behind them. Figure 16-15 shows the chain.

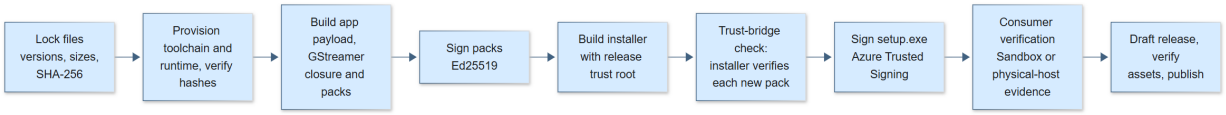



Figure 16-15. The release chain, from pinned inputs to a published release.

**What this shows.**

- **Pinned inputs.** Three lock files fix every external input: the runtime dependencies (ffmpeg n8.1.2 LGPL shared build, Node 24.15.0, Ollama 0.30.6, PostgreSQL 17.10-2 and TSDuck 3.44), the build toolchain, and the AI models (gemma4:12b, gemma4:e4b and translategemma:4b, each with its manifest and blob digests). A download is admitted only if its size and SHA-256 match.
- **The build refuses a dirty tree** and binds itself to one full commit identifier. It builds a GStreamer runtime closure and verifies it, builds each pack and signs it with an Ed25519 key held as a build secret, deleted afterward.
- **The trust-bridge check** runs the compiled installer's own code on the freshly signed packs, so a signing mistake is caught before anything is published.
- **Authenticode.** `setup.exe` is signed through Azure Trusted Signing (the key never leaves Microsoft's hardware), with an RFC 3161 timestamp, and the build fails unless Windows reports the signature as `Valid`. Checksums are regenerated from the signed file. Windows may still show a SmartScreen prompt, which is a matter of reputation, not signature.
- **No Sigstore or cosign.** The owner decided against them (ADR 0022). Provenance rests on the Authenticode signature, the Ed25519 pack signatures and the published checksums.
- **A software bill of materials** and a licence list are generated for the runtime and the application payload. The build refuses to produce them if any file has an unknown licence, and it rejects GPL-licensed GStreamer components.

- **Gate A** is one Windows Sandbox acceptance protocol. The current publisher also accepts physical-host verification receipts through a separate path; it then creates a draft release, checks the asset names and sizes, publishes it and records the result.

**Package and runtime evidence.** Use the release verification record matching the package for its installation checks. The Beta 18 record tracks that earlier package; use the Beta 19 verification record for its package checks. A historical Beta 11 three-channel, 72-hour host run recorded 99.78% live caption-input coverage; its host report preserves the original measurement.

#### Two example stations

CivicCast runs one station on one computer. Figure 16-16 shows two realistic shapes, a small station with one computer and one or two channels, and a larger station with more channels and more equipment around the same single computer.

**The small station.** One Windows computer with the service installed, an audio and video source (a capture card, or a network camera or encoder), and one or two channels. Staff can use the console locally or through the CIDR-restricted staff hostname when Caddy is configured. Residents reach recordings and the live channel through the public hostname when Caddy is deployed; the application itself remains loopback-bound. Native live captions use Whistle on CPU by default, with one primary request at a time across station channels. GPU acceleration is optional for Whisper fallback and recorded captions. Live audio remains best-effort and can be shed under overload (Figure 16-10).

**The larger station.** More channels and more video work share the same computer, so size the CPU and storage for the station's actual workload. An optional supported NVIDIA card and staged CUDA runtime can accelerate Whisper when used for recorded captions, fallback or direct live-primary mode; it is not required for Whistle. The original beta.10 eight-hour run and the separate dev7 soak describe earlier runtime history, not current-package capacity. A network storage target receives archive copies. Caddy can serve the resident portal on its public hostname and the staff console on a separate CIDR-restricted hostname. A cable headend takes a UDP transport stream (unicast or multicast; if TSDuck is installed, its relay preserves source-port and transport counter/timing behavior across worker restarts, without filling an encoder outage) or an SRT feed; NDI and SDI outputs depend on an ffmpeg build the station brings itself, because the NDI and Blackmagic software cannot be bundled. Staff outside the approved network can still use Remote Desktop to reach the station browser.

**Station boundaries.** The control plane, event bus and workers run on the station computer. If that computer fails, its channels go off air until it returns. The release verification record identifies the package and equipment used for operating checks; configure capture and cable-output hardware for your station's actual connections.

#### What CivicCast does not do yet

A technical reviewer should read this list before relying on the design. Each item is something we found in the code or in the beta.10 record. None of them is hidden in the sections above; they are collected here.

**One computer, one failure domain.**

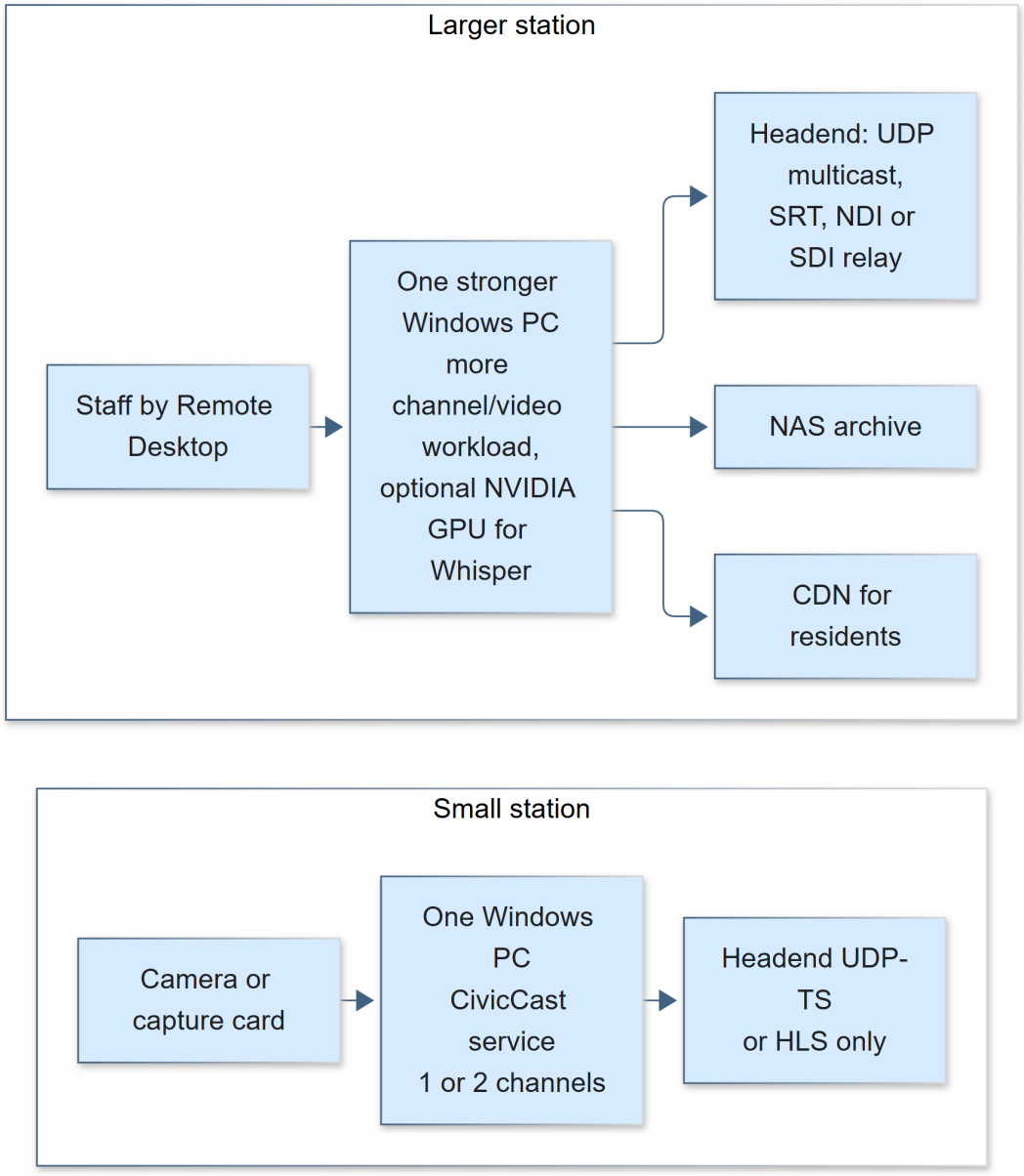



Figure 16-16. Two reference topologies. Both are one computer; the larger one adds equipment around it.

- CivicCast does not run a station across several computers. The control plane, the event bus, the staff rate limiter and every worker thread are in one process on one computer. There is no failover computer and no way to run a second control plane on the same database. If the computer stops, the channels stop. The supervisor restarts what fails *on* that computer.
- The control plane listens on a configured loopback address. Remote resident/staff access is available through the separate Caddy hostnames when configured; port 8000 should not be published directly.
- The application-to-database connection is plain PostgreSQL on loopback. Caddy can terminate HTTPS for remote browsers; the local CA certificates are not used for browser or database encryption.
- Installer source configures `NT SERVICE\CivicCastSupervisor`, a restricted virtual account that is not an administrator. See the service identity guide.

**People and access.**

- The first administrator is the recovery account. Setup admins can create staff accounts, assign roles, disable access, reset passwords and manage sessions; see the staff accounts guide.
- First-admin and staff console sessions expire after 12 hours. Lifecycle tokens have separate revocation/rotation controls and are not browser sessions.

**Data protection and recovery.**

- Station backup archives can be scheduled daily with bounded retention; program files, packs and models must still be reinstalled after a disaster.
- The archive contains configured station state and portable credentials; review the archive inventory and selected media roots before relying on it (see the station backup guide).
- The upgrade engine rolls back its database work; the current source installer's enclosing transaction also restores the prior application and station snapshot when that restoration succeeds. Its temporary copy excludes recordings and uploads and is removed after commit or successful restore. The read-only version guard refuses an older setup before service or file changes; this is not a database downgrade path. The offline `civiccast backup restore` command is separate from installer rollback. A terminal failed-upgrade journal can still block a later engine run until it is archived or moved aside.
- Worker stdout/stderr rotate at 10 MiB with ten backups per stream. Consumed channel commands are pruned in bounded batches while pending commands are preserved. These operational limits do not delete recordings; see System Health for storage use and low-disk warnings.

**Playout.**

- Freeze and audio/video recovery retain their three-restarts-per-hour budget, then use fallback and cooldown before a bounded retry. An intentional operator stop or drain suppresses automatic starts for that service session.
- RTMP, per-output normalization and configured SRT/RTMP alert-tone filtering are included in this source edition. Use the package's matching verification record for output-check status.

- Automatic H.264 output tries NVIDIA and Windows hardware encoders before retaining the CPU option. Explicit hardware requests follow the configured software-fallback policy. Availability and the selected channel profile determine which encoder starts; this is not a guarantee of GPU use. See preparation and encoder selection.
- A first-time program takes time to prepare, and a program change can wait on that preparation (the lead time is 11.5 minutes by default). An earlier internal run reported an unexplained rebuild of a long cached program and eleven daemon/automation test failures in both trees. Those are historical observations, not a measurement of this source edition.
- A single frame of video was dropped at one join in the lab run, and two program changes aborted and recovered by themselves.
- Command delivery from the staff API to the daemon is at most once, and the channel is driven by polling every 2 seconds.

**Captions, summaries and records.**

- Live captions are off in a new station profile, are English only, and can lose audio under heavy load. Captions embedded in the stream are not proven for CEA-708 broadcast compliance, and a channel whose profile uses HEVC cannot embed captions.
- Recorded-caption completion follows the station's selected languages and review requirement.
- The Internet Archive, NAS, YouTube and subscriber-notification surfaces are simulated until their providers are set to real. Podcast publication uses local FFmpeg and a configured public station URL. ActivityPub is off by default.
- Summary citations bind claims to transcript cue identifiers and times; staff review the wording against the source recording before approving it. Signed-record export embeds its timestamp proof in the PDF. Configure `CIVICCAST_TSA_URL` for an RFC 3161 timestamp authority; without a configured authority, the exporter uses its deterministic development timestamp. An embedded timestamp is separate from a PDF document-signing certificate.
- The public-safety display reads alerts and shows them; it is not an EAS device and does not relay the legally required EAS signal.

**Time.**

- New Schedule, Program Guide, recording and auto-schedule entries use the station time zone. Existing generated timestamps remain fixed when the zone or recurrence changes, and older weekly recording schedules can retain their UTC timing; see Chapter 3.

**Proof.**

- Commission the station's selected hardware and outputs for its actual use; package checks remain specific to the matching release record.

#### The design decisions behind the figures

The project records its major decisions as *architecture decision records* (ADRs). The table lists the decisions behind this chapter and compares them with the current source edition. An older ADR can

still describe a superseded implementation.

| ADR | Decision | Reason given | Current source behavior |
|---|---|---|---|
| 0001 | A NATS JetStream message server | durable streams, consumer groups, an open licence | No. Superseded by 0023 |
| 0008 | Synchronous SQLAlchemy 2.0 with psycopg 3, one migration runner with per-module migration folders, schema `civiccast` | the rest of the program is synchronous; N migration runners would be unworkable | Yes |
| 0010 | Live session state changes are conditional `UPDATE`s, forward only: `idle`, `preflight`, `on_air`, `ending`, `recorded` | two operators racing a change must produce exactly one winner | Yes |
| 0011 | Finalization is one transaction; the event key makes it idempotent | a crash must not leave an event with no asset, or two assets | Yes (the code also records trim points) |
| 0013 | Browser time zone for now, a station time zone later | a form without a zone is ambiguous across daylight saving | New Schedule, Program Guide, recording and auto-schedule entries use the station time zone. Older weekly recording schedules can retain UTC timing; changing the zone does not move existing generated timestamps |
| 0014 | Loudness target set per channel and per output, not a constant | web and cable expect different levels | Channel preparation and per-output targets are implemented. GStreamer creates an `audioloudnorm` audio branch when an output needs a different target |

| ADR | Decision | Reason given | Current source behavior |
|---|---|---|---|
| 0015 | One long-running FFmpeg process with a concat input | the cheapest way to keep one output session across programs | No. The default engine is now GStreamer; ADR 0015 describes the older engine |
| 0016 | A 1280 by 720, 30 fps, 6,000 kbit/s MPEG-TS profile with libx264 | a stable shape at every program change | The profile values remain the default. Automatic H.264 selects available hardware or OpenH264 on CPU; the bundle does not ship x264 |
| 0017 | Operator commands go through a durable table, polled by the daemon | commands must survive restarts; seconds of delay do not matter | Yes for the table and polling. The daemon runs as a thread inside the control plane, rather than the separate process described by the ADR |
| 0018 | Captions are `not-verified` until the emitted stream is decoded and checked | a caption file elsewhere in the product is not proof that captions are in the stream | Yes |
| 0020 | Fastly and Akamai through a generic S3-compatible adapter | one adapter for both | Yes (the CDN factory) |
| 0021 | Native Windows service, supervisor, one Windows product | the WSL cost record; a service starts at boot; WSL cannot reach capture cards | Yes, except that NATS and the “Tauri operator console” statements are out of date |
| 0022 | No Sigstore or cosign attestation | the owner’s decision; provenance is Authenticode and pack signatures | Yes |
| 0023 (NATS) | Remove NATS; keep an in-process bus behind an interface | NATS never carried real traffic and was a process, a port, a certificate and a health check | Yes |

| ADR | Decision | Reason given | Current source behavior |
|---|---|---|---|
| 0023 (as-run) | As-run rows go through a local durable journal first | the proof-of-performance record must survive a database outage | Yes. The journal sits in the service account's profile by default (see Figure 16-4) |
| 0024 | A watch folder never deletes its source files; an unreachable folder is a visible degraded state | never lose a source file; never fail silently | Yes |
| 0025 | A live source is ready only if it has been observed ready, within a time limit (30 seconds by default) | a configured camera that was unplugged for a week looked "ready" | Yes |

> **Note:** Two files in the ADR folder carry the number 0023. They are two separate decisions. The header of ADR 0001 still reads "Superseded by: N/A"; it was superseded by the NATS removal.

**Related**

- Chapter 9: Planning your station for hardware, network and storage.
- Chapter 10: Installing for the steps behind Figures 16-13 and 16-14.
- Chapter 12: Running it day to day for service, logs, backups and updates.
- Chapter 13: Security and privacy for the consequences of Figure 16-12.
- Chapter 14: Troubleshooting for symptoms of the failures in Figures 16-7 and 16-8.
- Chapter 15: Integrations for headends, streaming, CDNs, federation and the API.

# Part IV. Reference

Lookup material: commands, web interface, settings, files and ports, status words, roles, checklists, measured evidence, a glossary, licenses and release history.

## Appendix A: Command-line reference

CivicCast has one command-line program, `civiccast`. It is for the IT person, not for a clerk or operator: nothing in the operator console requires it. You use it to check the computer, to issue or revoke the access tokens that scripts and tools use, to run the disaster-recovery drill, and to run a few checks on the cable output. The command reference below is generated from the current source.

### Where the program is

On the Windows station, the installer puts CivicCast's own copy of Python and the `civiccast` program inside the install folder (normally `C:\Program Files\CivicCast (Native)`; if you chose another folder in setup, use that one). You do not need to install Python yourself. There are two equivalent ways to run it.

| Way | What you type (PowerShell, from any folder) |
|---|---|
| The launcher | `& "C:\Program Files\CivicCast (Native)\runtime\Lib\site-packages\bin\civiccast.exe" doctor` |
| CivicCast's Python | `& "C:\Program Files\CivicCast (Native)\runtime\python.exe" -m civiccast.cli doctor` |

Replace `doctor` with any command from the tables below. Add `--help` after any command or group to see its options on screen, for example `... civiccast.exe token --help`. A second launcher, `civiccast-runtime.exe`, sits beside it; it runs only the `runtime` group.

> **Tip:** To save typing, put the folder in a variable for the session: `$cc = "C:\Program Files\CivicCast (Native)\runtime\Lib\site-packages\bin\civiccast.exe"` and then run `& $cc doctor`.

### Before you run a command

1. **Use an Administrator PowerShell for anything that touches the station's data.** The station's database address is stored in the Windows registry at `HKLM\SOFTWARE\CivicCast\Native` (value `DatabaseUrl`). SYSTEM, Administrators and the restricted station service identity can read it.
2. **The `token` commands need the database address in the shell.** They print "DATABASE_URL must point at the CivicCast database before running staff token lifecycle commands." when it is missing. Set the environment variable DATABASE_URL for that PowerShell window from the registry value (the value contains the database password; do not paste it into tickets or chat).

3. **A command that changes something changes the live station.** `token revoke` cannot be undone; issue a new token instead.

> **Warning:** Do not run the `runtime cutover-to-native` or `runtime rollback-to-wsl` commands on a station that has never had the older WSL-based CivicCast edition. They exist to move a machine between the two editions; the supervisor refuses to start the station while the guard thinks the other edition is active.

#### How to read the tables

Each command has its own small table. The columns are:

| Column | Meaning |
|---|---|
| Option / argument | What you type after the command. `--name` options are optional unless the table says `(required)`. A word with no dashes (an *argument*) is positional: you type the value in that place. |
| Type | `str` is text, `int` a whole number, `path` a file or folder path, `boolean` a switch you add with no value. |
| Default | What the command uses when you leave the option out. Blank means no default. |
| Env var | An environment variable that supplies the value when the option is left out. Blank means none. |
| What it does | The program's own help text. |

Nearly every command accepts `--json`. It prints one machine-readable record instead of sentences, which is what you want in a script.

#### The commands you will use most

| Command | Use it to | Notes |
|---|---|---|
| `civiccast doctor` | Probe this computer: CPU, memory, disk, graphics card, operating system and the recommended hardware tier. | Add `--profile` for the full station-box profile (playout engine, clock, AI default, readiness). |
| `civiccast token issue` | Make an access token for a script or tool (see Appendix B). | The secret is printed once. Default scope is `operator`, which means all five roles; narrow it with `--scopes`. |
| `civiccast token list`, `revoke`, `rotate` | See, retire or replace tokens. | `list` never shows secrets. Needs DATABASE_URL. |

| Command | Use it to | Notes |
|---|---|---|
| `civiccast dr run-drill --out <folder>` | Run the real disaster-recovery drill: back up the database, restore it into a fresh copy, test crash recovery. | The same drill is a button on the Readiness screen. Read Appendix G. |
| `civiccast backup create`, `verify`, `inventory` | Create or check an encrypted `.ccbackup` archive and inspect its stored paths. | The command prompts for its passphrase; retain it separately from the archive. |
| `civiccast backup restore` | Restore onto a prepared station while CivicCast and other writers are stopped. | Requires explicit target path mappings and `--confirmation RESTORE`; see Chapter 12. |
| `civiccast cable doctor` | Run the read-only cable-headend readiness checks. | `--profile` defaults to `public-meetings`. |
| `civiccast cable commission` | Run the whole cable commissioning flow. | Sends a test signal to the headend address you give it. Do not run it while that channel is live. |
| `civiccast live-takeover state`, `take`, `return` | Check, begin or end a live takeover of one channel from the command line. | Uses the same takeover service and readiness gate as the Channels screen. |
| `civiccast egress verify` | Run a bounded TSDuck check of the channel's headend stream. | Needs the TSDuck tool (optional component). |
| `civiccast model state` | Print which AI models are installed and proven. | `model download` fetches models; `model import-offline` verifies an offline bundle. |
| `civiccast activitypub keygen` | Generate the station's federation key and print the settings to add. | Federation is off by default. |

#### Full command tables

The tables below are generated from the program itself.

47 commands. Generated by `ops/docs-sprint/tools/gen_cli_inventory.py`; do not edit by hand.

**activitypub** `civiccast activitypub keygen`

Generate ActivityPub key material and print non-secret env settings.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--private-key-path` (required) | path | | | Where to write the ActivityPub private key. |
| `--base-url` (required) | text | | | Public HTTPS base URL for this station. |
| `--handle` | text | civiccast | | Station account handle, e.g. civiccast. |
| `--mode` | text | approval-only | | Federation mode: open, limited, or approval-only. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**admin** `civiccast admin reset-password`

Recover first-admin access locally; revokes console sessions and recovery codes.

**backup** `civiccast backup create`

Create an encrypted backup using the configured destination and media roots.

`civiccast backup inventory`

Show the source paths that must be mapped on a replacement station.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `archive` (required) | path | | | |

`civiccast backup restore`

Restore while the station service and database writers are stopped.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `archive` (required) | path | | | |
| `--target-paths-json` (required) | path | | | JSON map of old absolute paths to new absolute paths |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--confirmation` (required) | text | | | must be exactly “RESTORE” |
| `--target-database-url` | text | | CIVICAST_TARGET_DATABASE_URL | Replacement database URL (prefer protected station configuration or environment) |

`civiccast backup verify`

Authenticate and verify every file in an archive.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `archive` (required) | path | | | |

**cable** `civiccast cable commission`

S3 screens 8–11: run the full cable commissioning flow end to end.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel to commission. |
| `--headend-profile` (required) | text | | | Headend profile id from the catalog. |
| `--destination` (required) | text | | | Headend destination address:port. |
| `--channel-name` | text | | | Human-readable channel name. |
| `--output-format` | text | 1080p30 | | 720p30 \| 1080i60 \| 1080p30 \| SD480i60. |
| `--sdi-device` | text | | | DeckLink device name, if SDI is used. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--fill-policy` | text | slate | | slate \| loop \| silence. |
| `--slate-asset-id` | text | | | Emergency slate asset id. |
| `--duration-seconds` | integer | 60 | | Output-proof run duration. |
| `--station-name` | text | | | Station name to stamp on the report. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast cable doctor`

S3 Screen 8: run the first-run cable commissioning checks (read-only).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--profile` | text | public-meetings | | Deployment profile to evaluate cable readiness for. |
| `--station-name` | text | | | Station name to stamp on the report. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast cable ndi-check`

Check whether this host can output NDI through FFmpeg.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast cable ndi-plan`

Build the FFmpeg command plan for local-file-to-NDI output.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--media` (required) | path | | | Local source media file to send to NDI. |
| `--ndi-name` (required) | text | | | NDI channel name shown to receivers. |
| `--muxer` | text | libndi_newtek | | FFmpeg NDI muxer to target after `civiccast cable ndi-check`. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast cable package`

Create a local cable file package from real media and captions.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--asset-id` (required) | text | | | CivicCast asset id for the recording. |
| `--title` (required) | text | | | Human-readable meeting or program title. |
| `--media` (required) | path | | | Local source media file to include in the package. |
| `--captions` (required) | path | | | Caption sidecar file, usually WebVTT or SRT. |
| `--output-dir` (required) | path | | | Directory where the cable package is written. |
| `--portal-url` | text | | | Optional CivicCast portal URL for the manifest. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast cable support-bundle`

S3 Screen 11: export a redacted support bundle (logs, config, health).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--output-dir` | path | | | Unused placeholder; bundles are server-managed. |
| `--note` | text | | | Optional operator note for the bundle. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**cert** `civiccast cert rotate`

Rotate one local-CA service certificate without printing private keys.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `identity` (required) | text | | | Known service identity to rotate. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**(top level)** `civiccast doctor`

Probe local hardware and report CPU, RAM, disk, GPU, OS, recommended tier.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit the probe as JSON (one object, machine-readable). |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--disk` | text | | | Filesystem path to probe for disk space (defaults to home directory). |
| `--profile` | boolean | False | | Render the full S1 StationBoxPro-file (playout-engine readiness, cable output, clock, AI default, PEG readiness) instead of just the hardware probe. |

**dr** `civiccast dr run-drill`

Run the real 0.5.0 disaster-recovery drill: backup, restore, crash-recovery.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--out` (required) | path | | | Directory to write dr-drill-report.md/.json into. |
| `--backup-dir` | path | | | Backup destination (default: <out>/backup). |
| `--work-dir` | path | | | Scratch dir for restore/crash drills (default: <out>/work). |
| `--database-url` | text | | | DATABASE_URL to drill (default: $DATABASE_URL). sqlite:// or postgresql://. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--media-root` | path | | | Media library root to manifest (optional). |

**egress** `civiccast egress caption-decode-proof`

Compare decoded emitted-stream captions with expected captions.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel id measured during the caption proof. |
| `--emitted-stream` (required) | path | | | Emitted stream file that was decoded. |
| `--expected-captions` (required) | path | | | Expected WebVTT or SRT caption file. |
| `--decoded-captions` (required) | path | | | Decoded WebVTT or SRT file from the emitted stream. |
| `--decoder-name` | text | ffmpeg-cc-decode | | Name of the decoder used to read the emitted stream. |
| `--timing-tolerance-seconds` | float | 0.75 | | Allowed start/end timing difference for each decoded caption cue. |
| `--output-path` | path | | | Optional JSON evidence file to write. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast egress continuity-proof`

Run a FileSink continuity and loudness proof for one egress source plan.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--source-plan-json` (required) | path | | | JSON file containing the ordered egress source plan to prove. |
| `--config-json` (required) | path | | | JSON file containing the egress channel config. |
| `--output-path` (required) | path | | | MPEG-TS FileSink output path for this proof run. |
| `--work-dir` | path | | | Directory for temporary continuity proof files. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast egress output test-pattern`

S3 Screen 10: drive a bounded test pattern through the GStreamer engine to the selected sink.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel to drive the test pattern to. |
| `--pattern` | text | bars | | bars \| live \| slate. |
| `--duration-seconds` | integer | 600 | | How long to run the test pattern. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast egress run`

Run the channel egress worker loop.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` | text | | | Channel id to poll. Repeat this option for more than one channel. |
| `--work-dir` | path | | | Directory for temporary egress plans and generated slate files. |
| `--poll-seconds` | float | 2.0 | | Seconds to wait between polling passes. |
| `--once` | boolean | False | | Poll once, print a report, and exit. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast egress srt-continuity-proof`

Run a representative SRT receiver continuity and loudness proof.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--source-plan-json` (required) | path | | | JSON file containing the ordered egress source plan to prove. |
| `--config-json` (required) | path | | | JSON file containing the egress channel config. |
| `--sender-url` (required) | text | | | Secret-free SRT caller URL for the sender. |
| `--receiver-url` (required) | text | | | Secret-free SRT listener URL for the receiver. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--receiver-output-path` (required) | path | | | MPEG-TS file written by the receiver. |
| `--work-dir` | path | | | Directory for temporary continuity proof files. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast egress trim-health`

Trim old egress health telemetry.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--older-than-days` | integer | 30 | | Delete egress health telemetry older than this many days. |
| `--dry-run` | boolean | False | | Show the cutoff without deleting health telemetry. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast egress verify`

Run a bounded TSDuck verification of the channel's headend stream (CA-7).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel whose headend stream to verify. |
| `--seconds` | integer | 10 | | How long to capture the live stream. |
| `--work-dir` | path | | | Directory for probe artifacts. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast egress verify-captions`

Standalone CEA-708 embed-through-decode-back proof (S11 -> S3).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--duration-seconds` | integer | 8 | | Bounded window to run the embed+decode-back proof for. |
| `--muxrate-kbps` | integer | 2000 | | Test-pattern mux rate in kbps. |
| `--work-dir` | path | | | Working directory for the embed run (defaults to the egress work dir). |
| `--caption-text` | text | | | Override the deterministic test caption text. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**installer** `civiccast installer beta-handoff`

Print fail-closed beta tester handoff readiness.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--release-manifest` | path | | | Release artifact manifest to evaluate for tester acquisition. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--clean-windows-evidence` | path | | | Clean Windows install proof JSON to include in the handoff status. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast installer health-check`

Run fail-closed first-run health checks for publish surfaces.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--profile` | text | public-meetings | | Deployment profile to verify. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast installer plan`

Print the first-run installer plan.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--profile` | text | public-meetings | | Deployment profile to plan. Default: public-meetings. |
| `--recommended-tier` | text | tier-1 | | Tier from civiccast doctor or the installer hardware probe. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast installer platform-plan`

Print the cross-platform installer bootstrap plan.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--os-family` | text | linux | | Platform family: linux or macos. Windows deployment readiness is decided separately, by the native station's own activation state (civiccast.installer.service), not this generic plan. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast installer summary`

Print fail-closed cross-platform installer readiness.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast installer verify-package`

Verify package bytes, sidecar hash, signed install manifest, and attestation.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--artifact` (required) | path | | | Package artifact path. |
| `--sidecar` (required) | path | | | Package sidecar JSON path. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**live-takeover** `civiccast live-takeover return`

Return one channel from a live takeover to its scheduled playout.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel to return to its scheduled source. |
| `--operator-id` (required) | text | | | Operator id recorded in the takeover audit. |
| `--notes` | text | | | Optional handback notes for the audit. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast live-takeover state`

Show whether a channel is under live takeover and whether it can take/return.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel whose takeover state to read. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast live-takeover take`

Begin a live takeover for one channel (queues the engine cut + records audit).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--channel-id` (required) | text | | | Channel to cut to a live source. |
| `--operator-id` (required) | text | | | Operator id recorded in the takeover audit. |
| `--operator-name` | text | | | Operator display name for the audit. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--reason` | text | | | Why the channel is being taken live. |
| `--path-id` | text | | | Specific live ingest path id (default: recommended path). |
| `--duration-seconds` | float | 3600.0 | | Planned live duration before automatic handback. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

**media** `civiccast media thumbnails-backfill`

Generate thumbnails for assets that don't have one yet.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

**model** `civiccast model download`

Download the real local AI models or print the offline bundle manifest.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--offline-bundle` | boolean | False | | Emit the air-gapped model bundle manifest instead of downloading. |
| `--profile` | text | public-meetings | | Deployment profile for the bundle manifest. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--dry-run` | boolean | False | | Show the model download plan without pulling models. |
| `--cache-dir` | text | | | Directory for the faster-whisper model snapshot. |
| `--bundle-dir` | text | | | Directory containing offline model artifacts. Required with –offline-bundle so the manifest can emit real SHA-256 values. |

`civiccast model import-offline`

Verify an offline model bundle from real file hashes.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--bundle-dir` (required) | path | | | Offline bundle directory. |
| `--expected` (required) | text | | | Expected filename=sha256 pair. Repeat for each model artifact. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast model set-provider-key`

Store (or clear) a cloud-provider API key for the hosted AI tiers (S13 D13).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `provider` (required) | text | | | Cloud provider to store a key for: ollama-cloud or openrouter. |
| `--key` | text | | | The provider API key. Omit to read it from the CIVIC-CAST_PROVIDER_API_KEY environment variable (preferred for headless/air-gap installs so the secret never appears in shell history). |
| `--clear` | boolean | False | | Remove the stored key for the provider instead of saving. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast model state`

Print installer-facing model proof state.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

**output** `civiccast output sdi-readiness`

S3: quick check – is the GStreamer SDI output path ready?

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

**runtime** `civiccast runtime cutover-to-native`

D7: cut over from WSL to native. Journal-backed; safe to re-run.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--state-dir` | path | | | Override the journal/evidence directory (default: `%ProgramData%\CivicCast\logs`). |
| `--interlock-owner` | text | | | Migration-owned interlock: verify (never take) a held record bound to this owner run id. |
| `--interlock-generation` | integer | | | Migration-owned interlock: the generation the record must still match at every phase boundary. |
| `--force-new` | boolean | False | | Explicitly discard an existing INCOMPLETE journal of the opposite direction and start fresh (CC-WS4-007) – only after confirming the in-flight run is genuinely abandoned. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast runtime probe`

Dump raw GuardInputs (diagnostics).

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast runtime rollback-to-wsl`

D8: roll back from native to WSL. Refuses without –ack. Runs INSIDE the D7a transfer interlock (CC-WS4-003), same treatment as cutover-to-native – it also mutates the selector.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--ack` | text | | | Must exactly equal the printed boundary statement. |
| `--exe-path` | text | | | Installed exe path for the restored Run entry. |
| `--state-dir` | path | | | Override the journal/evidence directory (default: `%Program-Data%\CivicCast\logs`). |
| `--interlock-owner` | text | | | Migration-owned interlock: verify (never take) a held record bound to this owner run id. |
| `--interlock-generation` | integer | | | Migration-owned interlock: the generation the record must still match at every phase boundary. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--force-new` | boolean | False | | Explicitly discard an existing INCOMPLETE journal of the opposite direction and start fresh (CC-WS4-007) – only after confirming the in-flight run is genuinely abandoned. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

`civiccast runtime status`

Read selector + interlock + run all probes + decide; report.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

**token** `civiccast token generate-env`

Generate a random secret for one CIVICCAST_STAFF_TOKENS entry.

`civiccast token issue`

Issue a staff bearer token and print the secret once.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--operator-id` (required) | text | | | Stable operator id. |
| `--display-name` (required) | text | | | Human-readable operator display name. |
| `--scopes` | text | operator | | Comma-separated token scopes. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--save-keyring` | boolean | False | | Save the issued token in the OS keyring. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast token list`

List staff token metadata without bearer secrets.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast token revoke`

Revoke a staff token so future staff-route requests fail closed.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `token_id` (required) | text | | | Public token id to revoke. |
| `--reason` | text | operator-requested | | Audit reason recorded with the revocation. |
| `--json` | boolean | False | | Emit machine-readable JSON. |

### `civiccast token rotate`

Revoke one staff token and issue a replacement for the same operator.

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `token_id` (required) | text | | | Public token id to rotate. |
| `--save-keyring` | boolean | False | | Save the replacement token in the OS keyring. |

| Option / argument | Type | Default | Env var | What it does |
|---|---|---|---|---|
| `--json` | boolean | False | | Emit machine-readable JSON. |

## Appendix B: HTTP API overview

CivicCast's web pages are built on an HTTP API: the operator console and the resident portal are both clients of it. IT staff and integrators can call the same API. This appendix explains who may call what; the generated tables at the end list every route.

### Where it is

On the station itself the API answers on the configured loopback address, by default `http://127.0.0.1:8000`. Network Settings also supports `::1`. The application stays loopback-only; remote access uses a separately deployed HTTPS reverse proxy. Network Settings generates a Caddy configuration with separate public and CIDR-restricted staff hostnames (see Appendix D). Do not publish application port 8000 directly. The same application serves the operator console at `/operator/` and the resident portal at `/`.

### The three groups of routes

| Group | Path starts with | Who may call it |
|---|---|---|
| Public | `/api/public/` | Anyone who can reach the server. No token. These feed the resident portal: channels and live status, schedule, recordings, search, captions, subscriptions, public agendas, the contributor form, the paywall status, and the app settings. |
| Staff | `/api/staff/` | Holders of a staff access token, and then only for the roles that route allows (see Appendix F). These feed the operator console. |
| Setup | `/api/setup/` | The station computer itself. First setup, sign-in, recovery and storage preparation. |
| Other | root and a few fixed paths | Protocol and utility routes (see below). |

Other protocol and utility routes include `/health`, federation discovery and ActivityPub routes,

media routes, `/api/hardware`, `/api/version`, and the signed Stripe webhook `/api/webhooks/stripe`. These are not all covered by the staff-token check; consult the generated route reference for current details.

> **Note:** `GET /health` always answers with HTTP 200 while the program is running. Read `status: healthy` requires a current database schema and no degraded or unknown live-caption verdict. Inspect both `schema` and `live_captions`; detailed caption status is available only to signed-in staff. This probe does not certify every station output or device.

#### How a caller proves who it is (the auth model)

1. **Staff routes need a bearer token.** Send the header `Authorization: Bearer <token>` with every request. A request with no header gets HTTP 401 and does not count against the lock-out limit.
2. **Where tokens come from.**
   - Staff sign in at `/login` using `POST /api/staff/auth/login`. This checks their account and returns a station-local console session token. Remote access uses the CIDR-restricted staff hostname when configured.
   - The first admin uses `POST /api/setup/login` on the station computer. It returns an `operator_console_token`; the same call from another computer is refused with HTTP 403.
   - Or issue a token with `civiccast token issue` (Appendix A). The console has no screen for pasting such a token in; it is for scripts and tools.
   - Or list tokens in the `CIVICCAST_STAFF_TOKENS` setting (an older method; the server cannot revoke these, so you retire one by changing the setting). On a station that has its database token store, as the native station does, these entries are honored only when `CIVICCAST_STAFF_TOKENS_FALLBACK_WITH_DB=1` is also set.
3. **A token carries scopes, and scopes become roles.** `admin` and `operator` mean all five roles; `setup_admin`, `meeting_operator`, `records_clerk`, `publish_operator` and `support_admin` (also written `setup`, `meeting`, `records`, `publish`, `support`) mean one role each. A token with no scopes has no roles.
4. **Each route names the roles that may use it.** A signed-in caller with the wrong role gets HTTP 403: "This action requires one of these CivicCast roles: …" followed by the role names in alphabetical order.
5. **Rate limit.** Ten failed token checks in 60 seconds from one client address earns HTTP 429 ("Too many failed staff authentication attempts. Wait and retry.") with a `Retry-After` header. The numbers are settings (`CIVICCAST_AUTH_RATE_LIMIT`, `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS`). A caller that sends the exact right token is not blocked by other people's failures.
6. **Read versus write.** Staff routes enforce their own roles; some reads accept any valid staff token, while sensitive reads and writes name allowed roles. Staff login is the account-authentication entry point, not an operation requiring an existing bearer token. Menu visibility does not grant permission to use an API (Appendix F).

7. **Maintenance mode.** While the service holds the station in maintenance mode (the installer does this during an upgrade, and during a hand-over between CivicCast editions), every request that changes data (POST, PUT, PATCH, DELETE) is refused with HTTP 503 and the body `{"error": "maintenance"}`. Reads still work.

> **Note:** A Beta 11 console limitation was the lack of a screen for creating operators or assigning roles. The Beta 18 source includes **Staff accounts** for creating staff users and assigning roles; see Security and privacy.

#### Getting the OpenAPI document

The OpenAPI document is the machine-readable list of every route, its inputs and its answers. On a native station it is served at `GET /openapi.json` and requires a staff token (the same bearer header as above):

```
Invoke-RestMethod -Uri http://127.0.0.1:8000/openapi.json -Headers @{
↪  Authorization = "Bearer <token>" }
```

The interactive pages `/docs` and `/redoc` are switched off on a native station, because they load their pictures and scripts from the public internet and a council chamber computer is often not allowed to reach it. The project's source repository carries a copy of the document at `docs/openapi.json`; the generated API reference below reflects the current source inventory.

#### Error answers

Branch on the HTTP status first. The `detail` field is text for ordinary errors, an object for a version conflict (HTTP 409 with version fields) and a list of field errors for bad input (HTTP 422). The common texts are in Appendix E.

#### Full route tables

432 paths, 522 operations, 84 groups. Generated by `ops/docs-sprint/tools/gen_api_inventory.py`; do not edit by hand.

Staff routes (`/api/staff/*`) require a bearer token and a role; public routes do not. See `docs/ops/staff-route-protection.md` and the role matrix in `inventory/screens/_shell-navigation-and-roles.md`.

### (root / protocol endpoints)

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/.well-known/nodeinfo` | Discover CivicCast NodeInfo | | 200 |
| GET | `/.well-known/webfinger` | Discover the CivicCast ActivityPub actor | | 200, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/ap/actor` | Read the CivicCast station ActivityPub actor | | 200 |
| GET | `/ap/followers` | Read accepted ActivityPub followers | | 200 |
| POST | `/ap/inbox` | Receive ActivityPub Follow and Undo requests | | 200 |
| GET | `/ap/outbox` | Read local CivicCast ActivityPub outbox | | 200 |
| GET | `/health` | Liveness plus station readiness | | 200 |
| GET | `/media/live/{channel_id}/{file_path}` | Get Live Media File | | 200, 422 |
| GET | `/media/vod/{asset_id}/{file_path}` | Get Media File | | 200, 422 |
| GET | `/nodeinfo/2.0` | Read CivicCast local federation metadata | | 200 |

**/api/hardware**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/hardware` | Get Hardware | | 200 |

**/api/public/agendas**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/agendas/{meeting_asset_id}` | Public read: published meeting agenda for one meeting asset | | 200, 404, 422, 503 |

### /api/public/app

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/public/app/analytics/events` | Ingest privacy-safe public app analytics | application/json | 202, 422 |
| GET | `/api/public/app/channels/{channel_id}` | Read one public app-platform channel config | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/catalog` | Read app-platform VOD catalog | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/catalog/playlists` | Read app-platform smart playlist definitions | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/live` | Read app-platform live playback state | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/now-next` | Read seeded app-platform now/next state | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/schedule` | Read app-platform schedule feed | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/schedule/epg` | Read app-platform EPG schedule export | | 200, 404, 422 |
| GET | `/api/public/app/channels/{channel_id}/schedule/epg/xlist` | Read TV Guide X-List style EPG export | | 200, 404, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/app/config` | Read the shared app-platform station config | | 200, 422 |

### /api/public/assets

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/assets` | List every asset in the active store | | 200 |
| GET | `/api/public/assets/{asset_id}` | Get a single asset by id | | 200, 404, 422 |

### /api/public/cg

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/cg/channels/{channel_id}/bulletins` | Read approved community bulletin-board items | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/display` | Read the complete CG portal display contract | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/feeds` | Read configured CG dynamic feed adapters | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/overlay-contract` | Read the CG linear overlay contract | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/render-plan` | Read the CG HLS render plan | | 200, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/cg/channels/{channel_id}/snapshot` | Read the multi-zone CG bulletin-board snapshot | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/stream.m3u8` | Read the CG HLS manifest | | 200, 422 |
| GET | `/api/public/cg/channels/{channel_id}/templates` | Read the CG template library | | 200, 422 |
| GET | `/api/public/cg/emergency-overlay` | Read the emergency-notification overlay state | | 200, 422 |
| GET | `/api/public/cg/idle` | Read the between-streams idle page state | | 200, 422 |

### /api/public/channels

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/channels` | List public linear channel profiles | | 200 |
| GET | `/api/public/channels/ctv/feed` | Read the reference connected-TV feed | | 200, 422 |
| GET | `/api/public/channels/{channel_id}/audio-tracks` | Public Audio Tracks | | 200, 422 |
| GET | `/api/public/channels/{channel_id}/captions.vtt` | Read the current public WebVTT captions for a channel | | 200, 404, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/channels/{channel_id}/live.m3u8` | Redirect to the channel's live HLS manifest | | 307, 404, 422 |
| GET | `/api/public/channels/{channel_id}/now-next` | Read public now/next state for a channel | | 200, 404, 422 |

**/api/public/contribute**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/contribute/agreements/current` | Read current contributor submission agreement | | 200 |
| POST | `/api/public/contribute/submissions` | Create an external producer submission for operator review | application/json | 201, 409, 422, 429 |
| GET | `/api/public/contribute/submissions/{submission_id}/status` | Read contributor-safe submission status | | 200, 403, 404, 422, 429 |
| POST | `/api/public/contribute/uploads` | Upload contributor media into the intake holding area | multipart/form-data | 201, 413, 422, 429, 503, 507 |

**/api/public/contribution**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/contribution/invites/{invite_token}` | Resolve Invite | | 200, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/public/contribution/invites/{invite_token}/accept-terms` | Accept Terms | | 200, 422 |

**/api/public/egress**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/egress/channels/{channel_id}/now` | Read viewer-safe current egress channel status | | 200, 404, 422, 503 |

**/api/public/embed**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/embed/{asset_id}` | Get embed snippet + manifest URL for a public asset | | 200, 404, 422 |

**/api/public/live**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/live/current` | Return the current on-air live session for residents | | 200, 422, 503 |

**/api/public/manual**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/manual` | Get the in-product operator manual | | 200 |

### /api/public/paywall

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/paywall/access` | Public read: does this email have access to this asset/series? | | 200, 422, 429, 503 |
| POST | `/api/public/paywall/magic-link` | Email a single-use magic-link to the viewer | application/json | 200, 422, 429, 503 |
| GET | `/api/public/paywall/verify` | Redeem a magic-link token (single-use) | | 200, 401, 422, 429, 503 |

### /api/public/playback-policy

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/public/playback-policy/evaluate` | Evaluate playback access and preroll policy | application/json | 200, 422 |

### /api/public/podcast

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/podcast/audio/{asset_id}.mp3` | Stream a published podcast enclosure | | 200, 422 |
| GET | `/api/public/podcast/{channel_id}.xml` | Read public podcast RSS feed | | 200, 422 |

### /api/public/programlog

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/programlog/channels/{channel_id}/guide` | Resident-facing channel guide: scheduled airings only, no internal detail | | 200, 422 |

**/api/public/reports**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/reports/as-run` | Public as-aired log | | 200, 422, 503 |

**/api/public/schedule**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/schedule/coming-up` | List committed (published) public premieres for the Coming Up widget | | 200, 422, 503 |

**/api/public/search**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/search` | Public custom-field search (exposed fields only) | | 200, 503 |

**/api/public/subscribe**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/public/subscribe/confirm` | Confirm a double opt-in subscription | | 200, 422 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/public/subscribe/email` | Start email double opt-in subscription | application/json | 200, 422 |
| GET | `/api/public/subscribe/rss/{target_type}/{target_id}.xml` | Read a public no-PII subscription RSS feed | | 200, 422 |
| GET | `/api/public/subscribe/unsubscribe` | Unsubscribe using a signed one-click link | | 200, 422 |
| POST | `/api/public/subscribe/unsubscribe` | One-click unsubscribe (RFC 8058) | | 200, 422 |
| POST | `/api/public/subscribe/webhook` | Start webhook subscription opt-in | application/json | 200, 400, 422, 502 |

**/api/setup/first-admin**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/setup/first-admin` | Complete local first-admin setup before staff auth exists | application/json | 200, 401, 403, 422 |

**/api/setup/login**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/setup/login` | Sign in with the local first-admin password | application/json | 200, 403, 422 |

**/api/setup/recover**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/setup/recover` | Recover the local first-admin account with a one-time recovery code | application/json | 200, 403, 422 |

**/api/setup/recovery-kit**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/setup/recovery-kit/acknowledge` | Record that the operator saved or printed the one-time recovery kit | application/json | 200, 401, 403, 422 |

**/api/setup/station-state**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/setup/station-state` | Read local setup state before staff auth exists | | 200, 403 |

**/api/setup/storage**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/setup/storage` | Read local durable storage setup state before staff auth exists | | 200, 401, 403 |
| POST | `/api/setup/storage` | Prepare installer-managed durable storage before staff auth exists | application/json | 200, 401, 403, 422 |

**/api/staff/activitypub**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/activitypub/deliveries` | List ActivityPub signed delivery attempts for staff evidence | | 200, 401, 422, 429 |
| GET | `/api/staff/activitypub/delivery-retries` | List the ActivityPub delivery retry queue (including dead letters) | | 200, 401, 429 |
| POST | `/api/staff/activitypub/delivery-retries/{retry_id}/replay` | Replay a dead-lettered ActivityPub delivery | | 200, 401, 404, 409, 422, 429 |
| GET | `/api/staff/activitypub/followers` | List ActivityPub followers for operator moderation | | 200, 401, 422, 429 |
| POST | `/api/staff/activitypub/followers/approve` | Approve a pending ActivityPub follower and deliver signed Accept | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/activitypub/followers/block` | Block an ActivityPub follower | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/activitypub/followers/reject` | Reject a pending ActivityPub follower and deliver signed Reject | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/activitypub/keygen` | Generate the station's ActivityPub federation key | | 200, 401, 429 |
| GET | `/api/staff/activitypub/outbox` | List local ActivityPub outbox activities for staff evidence | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/activitypub/status` | Read ActivityPub federation status and policy | | 200, 401, 429 |

### /api/staff/agenda

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/agenda/{agenda_id}/import-external` | Import agenda items from an external agenda system | application/json | 200, 401, 404, 422, 429, 502, 503 |

### /api/staff/agenda-sources

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/agenda-sources/js-portal/posture` | Report whether the optional js_portal (crawl4ai/Playwright) runtime is installed | | 200, 401, 429 |
| GET | `/api/staff/agenda-sources/{source}/{client_code}/meetings` | List upcoming/recent meetings from an external agenda system | | 200, 401, 404, 422, 429, 502, 503 |

### /api/staff/agendas

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/agendas` | List meeting agendas for the station | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/agendas` | Create a meeting agenda (always draft) | application/json | 201, 401, 409, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| DELETE | `/api/staff/agendas/{agenda_id}` | Delete a meeting agenda (cascades items) | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/agendas/{agenda_id}` | Get one meeting agenda | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/agendas/{agenda_id}` | Patch a meeting agenda (absent keys unchanged; status flips through the publish gate) | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/agendas/{agenda_id}/import` | Best-effort import of an uploaded agenda doc (text/plain or PDF) | | 200, 401, 404, 415, 422, 429, 503 |
| GET | `/api/staff/agendas/{agenda_id}/items` | List agenda items (default: ordered by operator-set order; timecode for the player) | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/agendas/{agenda_id}/items` | Create an agenda item | application/json | 201, 401, 404, 409, 422, 429, 503 |
| DELETE | `/api/staff/agendas/{agenda_id}/items/{item_id}` | Delete an agenda item | | 204, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/agendas/{agenda_id}/items/{item_id}` | Patch an agenda item (absent keys unchanged) | application/json | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/agendas/{agenda_id}/sync-from-chapters` | Seed agenda items from the meeting asset's chapter markers | | 200, 401, 404, 422, 429, 503 |

**/api/staff/ai-models**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/ai-models` | List the station-wide AI-model configuration (every feature) | | 200, 401, 429, 503 |
| GET | `/api/staff/ai-models/availability` | Per-feature availability of the effective AI model | | 200, 401, 429, 503 |
| DELETE | `/api/staff/ai-models/credentials/{provider}` | Clear a stored cloud-provider API key | | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/ai-models/credentials/{provider}` | Whether a cloud-provider API key is stored (boolean only, never the key) | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/ai-models/credentials/{provider}` | Store a cloud-provider API key (write-only; never returned) | application/json | 200, 400, 401, 404, 422, 429, 503 |
| GET | `/api/staff/ai-models/{feature}` | Catalog tiers + the operator selection for one feature | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/ai-models/{feature}/first-run-override` | Set/clear the commissioning-wizard first-run model override for one feature | application/json | 200, 400, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/ai-models/{feature}/select` | Record the operator AI-model selection for one feature | application/json | 200, 400, 401, 404, 422, 429, 503 |

**/api/staff/alert-channels**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/alert-channels` | List alert channels (redacted; no secrets) | | 200, 401, 429, 503 |
| POST | `/api/staff/alert-channels` | Create an alert channel (secret stays in the credential store) | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/alert-channels/{channel_id}` | Delete an alert channel | | 204, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/alert-channels/{channel_id}` | Update an alert channel | application/json | 200, 401, 404, 422, 429, 503 |

**/api/staff/alert-events**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/alert-events` | List alert events | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/alert-events/{event_id}/ack` | Acknowledge an alert event | | 200, 401, 404, 422, 429, 503 |

**/api/staff/alert-rules**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/alert-rules` | List alert rules | | 200, 401, 429, 503 |
| PUT | `/api/staff/alert-rules/{rule_id}` | Update a tunable alert rule | application/json | 200, 401, 404, 422, 429, 503 |

**/api/staff/analytics**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/analytics/export.csv` | Download a flat rollup CSV (PEG automation coverage floor) | | 200, 401, 422, 429 |
| GET | `/api/staff/analytics/reports/audience` | Build a packaged audience report — the franchise-authority deliverable. Use ?format=csv or ?format=xml for export-ready downloads. | | 200, 401, 422, 429 |
| POST | `/api/staff/analytics/reports/board-pdf` | Generate the one-click board-ready PDF (totals / top content / YoY / live peaks) | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/analytics/reports/overview` | Read aggregate-only station analytics report | | 200, 401, 422, 429 |
| GET | `/api/staff/analytics/rollups` | Read pre-aggregated viewership rollups (bar chart + time-series + stats) | | 200, 401, 422, 429 |

### /api/staff/app

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/app/builds` | List OTT app build records (newest first) | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/app/builds` | Build + record one OTT app target (local artifact, SHA-256 verified) | application/json | 201, 401, 422, 429 |
| GET | `/api/staff/app/builds/{record_id}` | Read one OTT app build record | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/app/builds/{record_id}/download` | Download a build artifact | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/app/channels/{channel_id}` | Read operator app-platform channel config | | 200, 401, 404, 422, 429 |
| PATCH | `/api/staff/app/channels/{channel_id}/branding` | Update operator app-platform channel branding | application/json | 200, 401, 404, 422, 429 |
| GET | `/api/staff/app/config` | Read operator app-platform station config | | 200, 401, 429 |
| PATCH | `/api/staff/app/config` | Update operator app-platform station config | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/app/store-submissions` | List external store submission trackers | | 200, 401, 429 |
| PATCH | `/api/staff/app/store-submissions/{app_target}` | Update an external store submission tracker (operator-maintained) | application/json | 200, 401, 422, 429 |

### /api/staff/assets

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/assets` | List every asset (operator library) | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/assets` | Create one asset (staff) | application/json | 201, 401, 409, 422, 429 |
| GET | `/api/staff/assets/broken` | List assets whose backing file is missing | | 200, 401, 429, 503 |
| GET | `/api/staff/assets/duplicates` | List groups of assets sharing identical file content | | 200, 401, 429, 503 |
| GET | `/api/staff/assets/readiness-dashboard` | Operator readiness dashboard across every asset | | 200, 401, 429, 503 |
| POST | `/api/staff/assets/upload` | Upload an asset file and run ffprobe ingest (staff) | multipart/form-data | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/assets/{asset_id}` | Get one asset (operator detail view) | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/assets/{asset_id}` | Update an asset's metadata (title, description, trim, chapters, retention) | application/json | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/assets/{asset_id}/custom-fields` | Get one asset's custom-field values | | 200, 401, 422, 429, 503 |
| PUT | `/api/staff/assets/{asset_id}/custom-fields` | Replace one asset's custom-field values (typed-validated) | application/json | 200, 401, 422, 429, 503 |
| PUT | `/api/staff/assets/{asset_id}/legal-hold` | Set or clear a legal hold (blocks retention expiry per CLAUDE.md §4.6) | application/json | 200, 401, 404, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/assets/{asset_id}/package` | Package validated or recorded local media for resident playback | | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/assets/{asset_id}/playback-grant` | Issue short-lived private playback access for an asset source | | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/assets/{asset_id}/readiness` | One asset's readiness badge state | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/assets/{asset_id}/relink` | Point an asset at a replacement file after validating it matches | application/json | 200, 401, 404, 409, 422, 429, 503 |
| PUT | `/api/staff/assets/{asset_id}/replace-source` | Swap out an asset's backing file; the old file is archived, not deleted | multipart/form-data | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/assets/{asset_id}/thumbnail` | Serve an asset's generated thumbnail image | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/assets/{asset_id}/unpublish` | Withdraw an asset from Portal visibility | | 200, 401, 404, 422, 429, 503 |

**/api/staff/audio-tracks**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/audio-tracks` | List Audio Tracks | | 200, 401, 422, 429 |
| DELETE | `/api/staff/audio-tracks/{track_id}` | Delete Audio Track | | 204, 401, 422, 429 |
| PUT | `/api/staff/audio-tracks/{track_id}` | Upsert Audio Track | application/json | 200, 401, 422, 429 |

### /api/staff/auth

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/auth/accounts` | List station-local staff accounts | | 200, 401, 429 |
| POST | `/api/staff/auth/accounts` | Create a station-local staff account | application/json | 201, 401, 422, 429 |
| PATCH | `/api/staff/auth/accounts/{operator_id}` | Change a staff account's roles, display name, or access state | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/auth/accounts/{operator_id}/reset-password` | Reset a staff password and end its existing sessions | application/json | 204, 401, 422, 429 |
| POST | `/api/staff/auth/login` | Sign in to a station-local staff account | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/auth/me` | Return the verified staff identity and product roles | | 200, 401, 429 |
| GET | `/api/staff/auth/sessions` | List station-local staff sessions | | 200, 401, 429 |
| DELETE | `/api/staff/auth/sessions/{session_id}` | Revoke one station-local staff session | | 204, 401, 422, 429 |
| POST | `/api/staff/auth/sign-out` | Sign out this browser's own staff session, leaving every other session signed in | | 200, 401, 429 |

### /api/staff/auto-schedule

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/auto-schedule/blocks` | List daypart blocks | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/auto-schedule/blocks` | Create a daypart block | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/auto-schedule/blocks/{block_id}` | Delete a daypart block | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/auto-schedule/blocks/{block_id}` | Get one daypart block | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/auto-schedule/blocks/{block_id}` | Update a daypart block | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/auto-schedule/compile` | Compile all enabled rules into schedule_items now | | 200, 401, 429, 503 |
| GET | `/api/staff/auto-schedule/rules` | List auto-schedule rules | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/auto-schedule/rules` | Create an auto-schedule rule | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/auto-schedule/rules/{rule_id}` | Delete an auto-schedule rule | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/auto-schedule/rules/{rule_id}` | Get one auto-schedule rule | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/auto-schedule/rules/{rule_id}` | Update an auto-schedule rule | application/json | 200, 401, 404, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/auto-schedule/rules/{rule_id}/preview` | Dry-run a rule: the slots it would fill, without writing | | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/auto-schedule/saved-searches` | List saved searches | | 200, 401, 429, 503 |
| POST | `/api/staff/auto-schedule/saved-searches` | Create a saved search | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/auto-schedule/saved-searches/{saved_search_id}` | Delete a saved search | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/auto-schedule/saved-searches/{saved_search_id}` | Get one saved search | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/auto-schedule/saved-searches/{saved_search_id}` | Update a saved search | application/json | 200, 401, 404, 422, 429, 503 |

**/api/staff/backups**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/backups` | List Station Backups | | 200, 401, 429 |
| POST | `/api/staff/backups/create` | Create Station Backup | application/json | 200, 401, 422, 429 |
| PUT | `/api/staff/backups/schedule` | Update Station Backup Schedule | application/json | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| PUT | `/api/staff/backups/schedule/passphrase` | Set Scheduled Passphrase | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/backups/status` | Station Backup Status | | 200, 401, 429 |
| POST | `/api/staff/backups/verify` | Verify Station Backup | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/backups/{backup_id}/download` | Download Station Backup | | 200, 401, 422, 429 |

**/api/staff/cable**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/cable/channels` | List operator channel profiles | | 200, 401, 429 |
| GET | `/api/staff/cable/channels/{channel_id}/now-next` | Read operator now/next state for a channel | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/cable/channels/{channel_id}/playout-plan` | Read channel schedule-to-playout plan | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/cable/channels/{channel_id}/proof-log` | Read channel playout proof log | | 200, 401, 404, 422, 429 |
| POST | `/api/staff/cable/commissioning/channel-setup` | S3 Screen 9: validate and persist the channel output setup | application/json | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/cable/commissioning/checks` | S3 Screen 8: run the first-run cable commissioning checks | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/cable/commissioning/output-proof` | S3 Screen 10: run the bounded output-proof test pattern + TSDuck probe | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/cable/commissioning/report` | S3 Screen 11: build and persist the final commissioning report | | 200, 401, 409, 422, 429 |
| GET | `/api/staff/cable/commissioning/state` | Resumable commissioning progress across all 4 steps | | 200, 401, 429 |

**/api/staff/captions**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/captions/external-ingest` | Add external caption appliance output to the review queue | application/json | 201, 400, 401, 409, 422, 429 |
| GET | `/api/staff/captions/offline-jobs` | List offline caption jobs (K3) for operator visibility | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/captions/offline-jobs/{job_id}/retry` | Manually retry a failed offline caption job (K3) | | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/captions/review-items` | List caption review queue items | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/captions/review-items` | Add a stable caption cue to the review queue | application/json | 201, 401, 409, 422, 429 |
| GET | `/api/staff/captions/review-items/{review_item_id}` | Get one caption review queue item | | 200, 401, 404, 422, 429 |
| POST | `/api/staff/captions/review-items/{review_item_id}/approve` | Approve a caption review item | application/json | 200, 401, 404, 409, 422, 429 |
| GET | `/api/staff/captions/review-items/{review_item_id}/clip` | Play bounded local audio around one caption cue | | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/captions/review-items/{review_item_id}/edit` | Edit a caption review item | application/json | 200, 401, 404, 422, 429 |
| POST | `/api/staff/captions/review-items/{review_item_id}/reject` | Reject a caption review item | application/json | 200, 401, 404, 422, 429 |
| GET | `/api/staff/captions/settings` | Read recorded-caption station settings | | 200, 401, 429 |
| PUT | `/api/staff/captions/settings` | Save recorded-caption station settings | application/json | 200, 401, 422, 429 |

### /api/staff/cg

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/cg/channels/{channel_id}/board` | Read the active CG board for a channel (board + zones + feeds) | | 200, 401, 422, 429, 503 |
| PATCH | `/api/staff/cg/channels/{channel_id}/board` | Update the active CG board (template / active) | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/cg/channels/{channel_id}/board` | Create (and activate) a CG board for a channel | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/cg/channels/{channel_id}/board/audio` | Delete Board Audio | | 204, 401, 422, 429 |
| GET | `/api/staff/cg/channels/{channel_id}/board/audio` | Get Board Audio | | 200, 401, 422, 429 |
| PUT | `/api/staff/cg/channels/{channel_id}/board/audio` | Set Board Audio | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/cg/channels/{channel_id}/board/audit` | List board-lifecycle audit events (newest first) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/cg/channels/{channel_id}/bulletins` | Read operator community bulletin submission queue | | 200, 401, 422, 429 |
| POST | `/api/staff/cg/channels/{channel_id}/bulletins` | Add a community bulletin submission to the channel board | application/json | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| PATCH | `/api/staff/cg/channels/{channel_id}/bulletins/{submission_id}` | Moderate a community bulletin (approve, decline, request changes) | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/cg/channels/{channel_id}/feeds` | List the channel's registered CG feed sources | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/cg/channels/{channel_id}/feeds` | Register a CG feed source | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/cg/channels/{channel_id}/feeds/{feed_source_id}` | Remove a CG feed source | | 204, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/cg/channels/{channel_id}/feeds/{feed_source_id}` | Update a CG feed source | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/cg/channels/{channel_id}/feeds/{feed_source_id}/items` | List a feed's current items with approval status (the review queue) | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/cg/channels/{channel_id}/feeds/{feed_source_id}/items/{item_id}/approve` | Approve a feed item for an approval-gated zone | | 201, 401, 404, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/cg/channels/{channel_id}/preview` | Render a live preview snapshot of the active board | | 200, 400, 401, 404, 422, 429, 503 |
| GET | `/api/staff/cg/channels/{channel_id}/preview.png` | Render a PNG preview snapshot of the active board | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/cg/channels/{channel_id}/zones` | Add a zone to the active board | application/json | 201, 401, 404, 422, 429, 503 |
| DELETE | `/api/staff/cg/channels/{channel_id}/zones/{zone_id}` | Remove a zone from the active board | | 204, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/cg/channels/{channel_id}/zones/{zone_id}` | Update a zone on the active board | application/json | 200, 401, 404, 422, 429, 503 |

**/api/staff/contribute**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/contribute/notifications/outbox` | Read queued contributor status notifications | | 200, 401, 429 |
| GET | `/api/staff/contribute/reports/producers` | Read producer activity report for station reporting | | 200, 401, 429 |
| GET | `/api/staff/contribute/submissions` | List operator contributor submission review queue | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/contribute/submissions/{submission_id}` | Read one contributor submission with operator review state | | 200, 401, 404, 422, 429 |
| POST | `/api/staff/contribute/submissions/{submission_id}/review` | Apply operator review decision to contributor submission | application/json | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/contribution**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/contribution/diagnostics` | Diagnostics | | 200, 401, 429 |
| POST | `/api/staff/contribution/diagnostics/turn-test` | Probe TURN reachability right now (does not wait for the next background poll) | | 200, 401, 429 |
| GET | `/api/staff/contribution/rooms` | List Rooms | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/rooms` | Create Room | application/json | 201, 401, 422, 429 |
| GET | `/api/staff/contribution/rooms/{room_id}` | Get Room | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/rooms/{room_id}/close` | Close the CivicCast room record (does not disconnect provider guests) | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/contribution/rooms/{room_id}/invites` | List Invites | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/rooms/{room_id}/invites` | Mint Invite | application/json | 201, 401, 422, 429 |
| POST | `/api/staff/contribution/rooms/{room_id}/open` | Open Room | | 200, 401, 422, 429 |
| GET | `/api/staff/contribution/sessions` | List Sessions | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/sessions/{session_id}/admit` | Admit Guest | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/sessions/{session_id}/drop` | Drop the guest and remove them from the live mix | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/sessions/{session_id}/media-control-request` | Record Media Control Request | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/sessions/{session_id}/mute` | Mute the guest in CivicCast's live audio mix | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/contribution/sessions/{session_id}/off-air` | Remove the guest from the live mix and return to scheduled output if last | | 200, 401, 422, 429 |
| POST | `/api/staff/contribution/sessions/{session_id}/on-air` | Route an admitted guest into the live channel and confirm the handoff | | 200, 401, 422, 429 |

**/api/staff/control-room**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| DELETE | `/api/staff/control-room/cues/{cue_id}` | Delete Cue | | 204, 401, 422, 429 |
| GET | `/api/staff/control-room/devices` | List Devices | | 200, 401, 429 |
| POST | `/api/staff/control-room/devices` | Create Device | application/json | 201, 401, 422, 429 |
| DELETE | `/api/staff/control-room/devices/{device_id}` | Delete Device | | 204, 401, 422, 429 |
| PATCH | `/api/staff/control-room/devices/{device_id}` | Update Device | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/control-room/devices/{device_id}/probe` | Probe Device | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| PUT | `/api/staff/control-room/devices/{device_id}/profile` | Upsert Profile | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/control-room/readiness` | Read control-room readiness (local configuration only; not station-device evidence) | | 200, 401, 403, 429, 503 |
| POST | `/api/staff/control-room/sessions` | Open Session | application/json | 201, 401, 422, 429 |
| DELETE | `/api/staff/control-room/sessions/{session_id}` | Close a session, releasing its operator lock on the surface | | 200, 401, 422, 429 |
| GET | `/api/staff/control-room/sessions/{session_id}` | Get Session | | 200, 401, 422, 429 |
| GET | `/api/staff/control-room/sessions/{session_id}/audit` | Session Audit | | 200, 401, 422, 429 |
| POST | `/api/staff/control-room/sessions/{session_id}/cues/{cue_id}/fire` | Fire Cue | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/control-room/sessions/{session_id}/cues/{cue_id}/plan` | Plan Cue | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/control-room/sessions/{session_id}/rollback` | Fire the session's configured safe-state/panic cue | | 200, 401, 422, 429 |
| GET | `/api/staff/control-room/surfaces` | List Surfaces | | 200, 401, 429 |
| POST | `/api/staff/control-room/surfaces` | Create Surface | application/json | 201, 401, 422, 429 |
| DELETE | `/api/staff/control-room/surfaces/{surface_id}` | Delete Surface | | 204, 401, 422, 429 |
| GET | `/api/staff/control-room/surfaces/{surface_id}` | Get Surface | | 200, 401, 422, 429 |
| PATCH | `/api/staff/control-room/surfaces/{surface_id}` | Update Surface | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/control-room/surfaces/{surface_id}/cues` | Create Cue | application/json | 201, 401, 422, 429 |
| GET | `/api/staff/control-room/surfaces/{surface_id}/session` | Read the open session and lock holder for a control surface | | 200, 401, 422, 429 |

**/api/staff/custom-fields**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/custom-fields` | List the station's custom-field definitions | | 200, 401, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/custom-fields` | Define a new custom field (setup_admin) | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/custom-fields/{field_id}` | Delete a custom-field definition (blocked if values exist unless confirm) | | 204, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/custom-fields/{field_id}` | Get one custom-field definition | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/custom-fields/{field_id}` | Edit a custom-field definition (label/options/flags; key is immutable) | application/json | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/doctor**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/doctor/audit` | Get Doctor Audit | | 200, 401, 429 |

**/api/staff/eas**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/eas/alerts` | List Alerts | | 200, 401, 422, 429 |
| POST | `/api/staff/eas/alerts/manual` | Create Manual Alert | application/json | 201, 401, 422, 429 |
| GET | `/api/staff/eas/alerts/{alert_id}` | Get Alert | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/eas/alerts/{alert_id}/display` | Display Alert | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/eas/decisions` | List Decisions | | 200, 401, 422, 429 |
| POST | `/api/staff/eas/decisions/{decision_id}/clear` | Clear Decision | | 200, 401, 422, 429 |
| GET | `/api/staff/eas/sources` | List Sources | | 200, 401, 429 |
| DELETE | `/api/staff/eas/sources/{source_id}` | Delete Source | | 204, 401, 422, 429 |
| PUT | `/api/staff/eas/sources/{source_id}` | Upsert Source | application/json | 200, 401, 422, 429 |

**/api/staff/egress**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/egress/channels` | List configured egress channels | | 200, 401, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}` | Read egress channel config, state, and latest health | | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/caption-proofs` | Recent CEA-608/708 caption decode-back proof samples (S11a) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/caption-status` | Live CEA-608/708 caption decode-back status (S11a) | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/egress/channels/{channel_id}/commands` | Queue an egress daemon command | application/json | 202, 401, 409, 422, 429, 503 |
| POST | `/api/staff/egress/channels/{channel_id}/compliance-probe` | Run a bounded TSDuck verification of the channel's headend stream | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/config` | Read an egress channel configuration | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/egress/channels/{channel_id}/config` | Create or replace an egress channel configuration | application/json | 200, 401, 422, 429, 503 |
| POST | `/api/staff/egress/channels/{channel_id}/config/headend-profile` | Apply a headend delivery profile to a channel's egress config | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/graphics-overlay` | Read the S15 graphics-overlay (lower-third) state for a channel | | 200, 401, 404, 422, 429, 503 |
| PUT | `/api/staff/egress/channels/{channel_id}/graphics-overlay` | Set the S15 graphics-overlay (lower-third) state for a channel | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/health` | Read recent egress health samples | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/egress/channels/{channel_id}/loudness-plan` | Resolve the per-sink loudness plan for a channel (S11b) | | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/proof` | Read recent egress as-aired proof events | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/schema-currency` | Whether the channel's persisted egress data matches the running schema | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/state` | Read the last-known egress daemon state | | 200, 401, 422, 429, 503 |
| DELETE | `/api/staff/egress/channels/{channel_id}/takeover` | Return a channel from live takeover to its schedule | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/egress/channels/{channel_id}/takeover` | Take a channel live (override the schedule) | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/takeover-audit` | Read the channel's live-takeover audit log (admin) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/egress/channels/{channel_id}/takeover-state` | Read the channel's manual-route (takeover) state | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/egress/headend-device-probe` | TCP reachability probe of a headend appliance | application/json | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/egress/headend-profiles` | List named cable-headend delivery profiles (vendor-doc sourced) | | 200, 401, 429 |
| GET | `/api/staff/egress/headend-readiness` | TSDuck availability and last stream-verification results | | 200, 401, 429, 503 |
| GET | `/api/staff/egress/ndi-readiness` | BYO-NDI readiness and supervised relay statuses | | 200, 401, 429 |
| POST | `/api/staff/egress/repair-gstreamer` | Repair the GStreamer runtime and restore full egress (no reinstall) | | 200, 401, 429 |
| GET | `/api/staff/egress/sdi-readiness` | BYO-SDI readiness and supervised relay statuses | | 200, 401, 429 |
| GET | `/api/staff/egress/storage` | Read egress worker log and command storage use | | 200, 401, 429, 503 |

**/api/staff/epg**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/epg/configs` | List EPG export configs for the station | | 200, 401, 429, 503 |
| POST | `/api/staff/epg/configs` | Create an EPG export config | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/epg/configs/{config_id}` | Delete an EPG export config | | 204, 401, 404, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/epg/configs/{config_id}` | Get one EPG export config | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/epg/configs/{config_id}` | Patch an EPG export config (absent keys unchanged) | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/epg/configs/{config_id}/generate` | Run the EPG export — download or push to endpoint | | 200, 401, 404, 422, 429, 503 |

**/api/staff/facility**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/facility/router-inventory` | Read operator-safe facility router inventory | | 200, 401, 429 |
| GET | `/api/staff/facility/router-panel` | Read the mobile-friendly facility router panel | | 200, 401, 404, 422, 429 |
| POST | `/api/staff/facility/router-schedule-plan` | Preview a schedule-triggered facility router take | application/json | 200, 401, 404, 422, 429 |
| POST | `/api/staff/facility/router-take-plan` | Preview a facility router take command | application/json | 200, 401, 404, 422, 429 |

**/api/staff/federation**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/federation/stations` | Stations | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/federation/stations/{station_id}/catalog` | Catalog | | 200, 401, 422, 429 |
| GET | `/api/staff/federation/stations/{station_id}/channels` | Channels | | 200, 401, 422, 429 |
| POST | `/api/staff/federation/stations/{station_id}/channels/{channel_id}/commands` | Command | application/json | 202, 401, 422, 429 |

**/api/staff/first-run**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/first-run/wizard` | Get V11 First Run Wizard | | 200, 401, 429 |
| GET | `/api/staff/first-run/wizard-v1.2` | Get V12 First Run Wizard | | 200, 401, 429 |

**/api/staff/installer**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/installer/acceptance-packet` | Generate a redacted station acceptance packet | | 200, 401, 429 |
| GET | `/api/staff/installer/backup` | Read backup setup and write-proof status | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/installer/backup` | Configure and verify a backup destination | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/installer/beta-handoff` | Read fail-closed beta tester handoff readiness (release engineering only) | | 200, 401, 429 |
| POST | `/api/staff/installer/cdn-concierge/r2` | Provision Cloudflare R2 CDN storage from one pasted API token | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/dr/run-drill` | Run the REAL 0.5.0 disaster-recovery drill (backup + restore + crash-recovery) | | 200, 401, 429 |
| POST | `/api/staff/installer/first-admin` | Complete first-admin setup and generate the recovery kit | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/installer/first-admin-contract` | Read the v1.3 first-admin and recovery-kit product contract | | 200, 401, 429 |
| GET | `/api/staff/installer/first-run-plan` | Read the profile-driven first-run installer plan | | 200, 401, 422, 429 |
| GET | `/api/staff/installer/health` | Run fail-closed first-run publish-surface health checks | | 200, 401, 422, 429 |
| POST | `/api/staff/installer/model-bundle` | Build an offline model bundle manifest with hash verification | application/json | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/installer/provider-credentials` | Save provider credentials from the setup wizard | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/provider-credentials/{provider_id}/test-connection` | Live-test a provider's saved credentials against the provider | | 200, 401, 422, 429 |
| POST | `/api/staff/installer/provider-proof` | Record redacted provider proof evidence | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/installer/provider-readiness` | Read provider setup and readiness cards | | 200, 401, 429 |
| POST | `/api/staff/installer/recovery-kit/regenerate` | Mint a fresh recovery kit for the already-signed-in admin | | 200, 401, 429 |
| POST | `/api/staff/installer/rehearsal` | Run a private first-broadcast rehearsal | | 200, 401, 422, 429 |
| GET | `/api/staff/installer/remote-contribution` | Whether the pinned VDO.Ninja (remote contribution) is installed + verified | | 200, 401, 429 |
| POST | `/api/staff/installer/remote-contribution/install` | Install the pinned VDO.Ninja for the remote-contribution tier (S17/S3 commissioning) | | 200, 401, 429 |
| GET | `/api/staff/installer/resident-preview` | Read the resident preview target | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/installer/restore` | Read restore rehearsal status | | 200, 401, 429 |
| POST | `/api/staff/installer/restore/rehearsal` | Run a backup restore rehearsal | | 200, 401, 429 |
| GET | `/api/staff/installer/safe-to-broadcast` | Read the current safe-to-broadcast report | | 200, 401, 422, 429 |
| GET | `/api/staff/installer/safe-to-broadcast-contract` | Read the v1.3 safe-to-broadcast product contract | | 200, 401, 429 |
| GET | `/api/staff/installer/sample-seed-status` | Read first-run sample content and starter schedule seeding status | | 200, 401, 429 |
| POST | `/api/staff/installer/sample-seed-status/dismiss` | Dismiss the first-run sample content seeding notice | | 200, 401, 429 |
| POST | `/api/staff/installer/sample-seed-status/retry` | Retry first-run sample content and starter schedule seeding | | 200, 401, 409, 429 |
| POST | `/api/staff/installer/sessions/revoke-others` | Sign out every other operator-console session, keeping this one signed in | | 200, 401, 429 |
| GET | `/api/staff/installer/source-setup` | Read plain-language camera and source setup guidance | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/installer/source-setup/live-source` | Create a meeting source from the operator setup wizard | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/source-setup/sample-upload` | Create a bundled sample video for private rehearsal | | 200, 401, 429 |
| GET | `/api/staff/installer/station-state` | Read first-admin setup and recovery-kit state | | 200, 401, 429 |
| GET | `/api/staff/installer/storage` | Read installer-managed durable storage setup state | | 200, 401, 429 |
| POST | `/api/staff/installer/storage` | Prepare installer-managed durable storage | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/support-bundle` | Generate a redacted support bundle | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/installer/support-bundle/{bundle_id}/download` | Download a generated redacted support bundle | | 200, 401, 403, 404, 422, 429 |
| GET | `/api/staff/installer/system-health` | Read operator-facing system readiness | | 200, 401, 422, 429 |
| GET | `/api/staff/installer/tsduck` | Whether TSDuck (cable stream verification) is installed | | 200, 401, 429 |
| POST | `/api/staff/installer/tsduck/install` | Download and install TSDuck on demand to enable cable verification | | 200, 401, 429 |
| GET | `/api/staff/installer/update-rollback` | Read update and rollback readiness | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/installer/update-rollback/failed-update-rehearsal` | Run controlled failed-update rollback rehearsal | | 200, 401, 429 |
| POST | `/api/staff/installer/update-rollback/maintenance-window` | Open an update maintenance window | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/update-rollback/post-update-proof` | Run post-update Safe to broadcast proof | | 200, 401, 429 |
| POST | `/api/staff/installer/update-rollback/preflight` | Run an update preflight checkpoint | | 200, 401, 429 |
| POST | `/api/staff/installer/update-rollback/rollback-artifact` | Configure a rollback artifact | application/json | 200, 401, 422, 429 |
| POST | `/api/staff/installer/update-rollback/rollback-rehearsal` | Run rollback artifact rehearsal | | 200, 401, 429 |

**/api/staff/live**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/live/finalizations` | List recording finalization worker statuses | | 200, 401, 429, 503 |
| GET | `/api/staff/live/ingest-plan` | Build the current live ingest plan for a channel | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/live/preflight-v1.1` | Get V11 Live Preflight | | 200, 401, 429 |
| GET | `/api/staff/live/recording-targets` | List recording targets | | 200, 401, 429, 503 |
| POST | `/api/staff/live/recording-targets` | Create a recording target | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/live/recording-targets/{recording_target_id}` | Get one recording target | | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/live/relay-configs` | List optional remote ingest or relay targets | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/live/relay-configs` | Create an optional remote ingest or relay target | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/live/relay-configs/{relay_config_id}` | Get one remote ingest or relay target | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/live/relay-configs/{relay_config_id}/health` | Update relay health from a station probe | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/live/sessions` | Create a live session (staff) | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/live/sessions/{live_session_id}` | Get a live session by id (staff) | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/live/sessions/{live_session_id}/end-broadcast` | Transition on_air -> ending (stamps ended_at) | | 200, 401, 404, 409, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/live/sessions/{live_session_id}/finalization` | Get recording finalization status for one session | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/live/sessions/{live_session_id}/finalization/retry` | Re-queue a failed recording finalization | | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/live/sessions/{live_session_id}/go-on-air` | Transition preflight -> on_air (stamps started_at) | application/json | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/live/sessions/{live_session_id}/preflight` | Run the pre-flight checklist evaluator | application/json | 200, 401, 422, 429, 503 |
| POST | `/api/staff/live/sessions/{live_session_id}/start-preflight` | Transition idle -> preflight | | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/live/sources` | List configured live sources | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/live/sources` | Create a configured live source (RTMP / RTSP / NDI / SRT) | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/live/sources/{live_source_id}` | Get one live source | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/live/sources/{live_source_id}` | Edit a configured live source | application/json | 200, 401, 404, 409, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/live/sources/{live_source_id}/probe` | Check whether a live source is delivering media right now | | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/media-lifecycle**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/media-lifecycle/audit-log` | Media lifecycle worker audit trail (dry-run entries included) | | 200, 401, 422, 429 |
| GET | `/api/staff/media-lifecycle/browse-folders` | List subdirectories of a path, for the non-technical watch-folder picker | | 200, 401, 422, 429 |
| GET | `/api/staff/media-lifecycle/missing-media` | Scheduled items whose backing asset is not ready within the horizon | | 200, 401, 429 |
| GET | `/api/staff/media-lifecycle/retention-policies` | List retention automation rules | | 200, 401, 429 |
| POST | `/api/staff/media-lifecycle/retention-policies` | Add a retention automation rule | application/json | 201, 401, 422, 429 |
| POST | `/api/staff/media-lifecycle/retention-policies/apply` | Apply enabled retention rules to every asset now | | 200, 401, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| DELETE | `/api/staff/media-lifecycle/retention-policies/{policy_id}` | Remove a retention automation rule | | 204, 401, 404, 422, 429 |
| PUT | `/api/staff/media-lifecycle/retention-policies/{policy_id}` | Update a retention automation rule | application/json | 200, 401, 404, 422, 429 |
| GET | `/api/staff/media-lifecycle/storage-budget` | Storage used vs the configured budget, by retention tier | | 200, 401, 429 |
| GET | `/api/staff/media-lifecycle/watch-folder-configs` | List configured auto-ingest watch folders | | 200, 401, 429 |
| POST | `/api/staff/media-lifecycle/watch-folder-configs` | Add a watch folder (local disk, USB, or NAS/SMB path) | application/json | 201, 401, 422, 429 |
| DELETE | `/api/staff/media-lifecycle/watch-folder-configs/{config_id}` | Remove a watch folder config | | 204, 401, 404, 422, 429 |
| PUT | `/api/staff/media-lifecycle/watch-folder-configs/{config_id}` | Update a watch folder config | application/json | 200, 401, 404, 422, 429 |
| POST | `/api/staff/media-lifecycle/watch-folder-configs/{config_id}/scan-now` | Scan one watch folder immediately, bypassing its poll interval | | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/migrate**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/migrate/apply` | Apply a dry-run plan: create the proposed shows + schedule items for real | application/json | 201, 401, 422, 429, 503 |
| GET | `/api/staff/migrate/batches` | List every import batch (applied and rolled back) | | 200, 401, 429, 503 |
| POST | `/api/staff/migrate/dry-run` | Fetch a source system's inventory and produce a typed import diff (no writes) | application/json | 200, 400, 401, 422, 429, 502, 503 |
| POST | `/api/staff/migrate/rollback` | Undo an apply: delete exactly the rows that batch created | application/json | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/network**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/network/settings` | Read station listener and proxy settings | | 200, 401, 429 |
| PUT | `/api/staff/network/settings` | Save station listener and proxy settings | application/json | 200, 401, 422, 429 |

**/api/staff/paywall**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/paywall/config` | Get the active paywall config for the station | | 200, 401, 429, 503 |
| PUT | `/api/staff/paywall/config` | Create or replace the paywall config for the station | application/json | 200, 401, 409, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| DELETE | `/api/staff/paywall/config/{config_id}` | Delete the paywall config | | 204, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/paywall/config/{config_id}` | Patch the paywall config (absent keys unchanged) | application/json | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/paywall/grants` | Issue a comp access grant | application/json | 201, 401, 409, 422, 429, 503 |
| DELETE | `/api/staff/paywall/grants/{grant_id}` | Revoke a grant | | 204, 401, 404, 422, 429, 503 |

**/api/staff/playback-policy**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/playback-policy/audit/events` | Read playback policy decision audit log | | 200, 401, 429 |
| POST | `/api/staff/playback-policy/viewer-tokens` | Issue a signed resident playback entitlement token | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/playback-policy/{subject_type}/{subject_id}` | Read one channel or asset playback policy | | 200, 401, 422, 429 |
| POST | `/api/staff/playback-policy/{subject_type}/{subject_id}` | Update one channel or asset playback policy | application/json | 200, 401, 422, 429 |

**/api/staff/playout**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/playout/commit` | Approve and air a committed occurrence | application/json | 201, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/playout/commits` | List a channel's commit-to-air reports (most recent first) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/playout/commits/{report_id}` | Get one commit-to-air report | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/playout/prepare-commit` | Dry-run a commit-to-air for an occurrence (no change made) | application/json | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/playout/rollback/{report_id}` | Roll back (undo) a committed airing | application/json | 200, 401, 404, 422, 429, 503 |

**/api/staff/podcast**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/podcast/episodes` | Create a podcast episode through Publish approval | application/json | 200, 401, 422, 429 |

**/api/staff/producer-ops**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/producer-ops/call-sheets` | List call sheets | | 200, 401, 429 |
| POST | `/api/staff/producer-ops/call-sheets` | Create a call sheet | application/json | 201, 401, 409, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/producer-ops/call-sheets/{call_sheet_id}` | Read one call sheet | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/producer-ops/call-sheets/{call_sheet_id}/assignments` | List crew assignments on a call sheet | | 200, 401, 422, 429 |
| POST | `/api/staff/producer-ops/call-sheets/{call_sheet_id}/assignments` | Assign a volunteer's crew role on a call sheet | application/json | 201, 401, 404, 409, 422, 429 |
| POST | `/api/staff/producer-ops/checkouts` | Check out an equipment item to a volunteer | application/json | 201, 401, 404, 409, 422, 429 |
| POST | `/api/staff/producer-ops/checkouts/{checkout_id}/return` | Return a checked-out equipment item | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/producer-ops/equipment` | List the equipment roster | | 200, 401, 422, 429 |
| PUT | `/api/staff/producer-ops/equipment/{equipment_id}` | Create or update an equipment roster entry | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/producer-ops/equipment/{equipment_id}/access-rule` | Read the access rule for an equipment item, if any | | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| PUT | `/api/staff/producer-ops/equipment/{equipment_id}/access-rule` | Set the training badge required to check out an equipment item | application/json | 200, 401, 404, 422, 429 |
| GET | `/api/staff/producer-ops/series-applications` | List producer series applications | | 200, 401, 429 |
| POST | `/api/staff/producer-ops/series-applications` | Submit a producer's request for a recurring series slot | application/json | 201, 401, 409, 422, 429 |
| GET | `/api/staff/producer-ops/series-applications/{application_id}` | Read one series application | | 200, 401, 404, 422, 429 |
| POST | `/api/staff/producer-ops/series-applications/{application_id}/review` | Apply a staff review decision to a series application | application/json | 200, 401, 404, 422, 429 |
| GET | `/api/staff/producer-ops/volunteers` | List the volunteer roster | | 200, 401, 422, 429 |
| GET | `/api/staff/producer-ops/volunteers/{volunteer_id}` | Read one volunteer roster entry | | 200, 401, 404, 422, 429 |
| PUT | `/api/staff/producer-ops/volunteers/{volunteer_id}` | Create or update a volunteer roster entry | application/json | 200, 401, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/producer-ops/volunteers/{volunteer_id}/badges` | List training badges earned by a volunteer | | 200, 401, 422, 429 |
| POST | `/api/staff/producer-ops/volunteers/{volunteer_id}/badges` | Grant a training badge to a volunteer | application/json | 201, 401, 404, 409, 422, 429 |
| GET | `/api/staff/producer-ops/volunteers/{volunteer_id}/checkouts` | List equipment checkouts for a volunteer | | 200, 401, 422, 429 |

**/api/staff/programlog**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/programlog/channels/{channel_id}/log` | Merged program log for one channel, including recorded skips | | 200, 401, 422, 429 |
| POST | `/api/staff/programlog/materialize` | Materialize all due occurrences now (refresh the guide) | | 200, 401, 429 |
| GET | `/api/staff/programlog/slots` | List program slots, optionally per channel | | 200, 401, 422, 429 |
| POST | `/api/staff/programlog/slots` | Create a program slot and materialize its upcoming occurrences | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/programlog/slots/{slot_id}` | Get one program slot | | 200, 401, 404, 422, 429 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| PATCH | `/api/staff/programlog/slots/{slot_id}` | Update a program slot (future occurrences re-materialize) | application/json | 200, 401, 404, 422, 429 |
| POST | `/api/staff/programlog/slots/{slot_id}/disable` | Disable a slot and cancel its future materialized airings | | 200, 401, 404, 422, 429 |

**/api/staff/publish**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/publish/assets` | List publish-dashboard status for every asset | | 200, 401, 429, 503 |
| GET | `/api/staff/publish/assets/{asset_id}` | Get publish-dashboard status for one asset | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/publish/assets/{asset_id}/approve` | Approve and run the v0.7 three-tier publish workflow | application/json | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/publish/assets/{asset_id}/preflight` | Check v0.7 publish readiness before operator approval | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/publish/assets/{asset_id}/surfaces/{surface_id}/retry` | Retry one v0.7 publish surface | application/json | 200, 401, 404, 409, 422, 429, 503 |

**/api/staff/recording**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/recording/input-presets` | List detected and configured SDI/HDMI recording inputs | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/recording/jobs` | List recording jobs for the station | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/recording/jobs/{job_id}/stop` | Stop a running recording job | | 200, 401, 404, 409, 422, 429, 503 |
| GET | `/api/staff/recording/schedules` | List recording schedules for the station | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/recording/schedules` | Create a recording schedule | application/json | 201, 401, 403, 409, 422, 429, 503 |
| DELETE | `/api/staff/recording/schedules/{schedule_id}` | Delete a recording schedule | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/recording/schedules/{schedule_id}` | Get one recording schedule | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/recording/schedules/{schedule_id}` | Patch a recording schedule (absent keys unchanged) | application/json | 200, 401, 404, 409, 422, 429, 503 |
| POST | `/api/staff/recording/schedules/{schedule_id}/record-now` | Start an ad-hoc recording against this schedule's source NOW | | 200, 401, 404, 409, 422, 429, 500, 503 |

**/api/staff/records**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/records` | List recent signed-record metadata | | 200, 401, 422, 429 |
| POST | `/api/staff/records` | Create a PDF/A-3B signed-record export | application/json | 201, 401, 409, 422, 429 |
| GET | `/api/staff/records/disposition-queue` | List assets flagged for retention disposition review | | 200, 401, 429, 503 |
| GET | `/api/staff/records/{record_id}/download` | Download a PDF/A-3B signed-record artifact | | 200, 401, 404, 422, 429 |
| GET | `/api/staff/records/{record_id}/verify` | Verify timestamp and audit metadata for a signed record | | 200, 401, 422, 429 |

**/api/staff/release**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/release/proof` | Get V11 Release Proof | | 200, 401, 429 |

**/api/staff/reports**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/reports/as-run` | As-aired log (engine-verified actual air times) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/reports/export` | Download as-run or shows as CSV or XML | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/reports/hours-by-category` | Franchise hours grouped by an S22 custom-field value (DC-3) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/reports/shows` | Per-show play counts + airtime | | 200, 401, 422, 429, 503 |

**/api/staff/runtime-safe-to-air**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/runtime-safe-to-air` | Continuous runtime safe-to-air signal (cached ~4s) | | 200, 401, 429, 503 |

**/api/staff/schedule**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/schedule` | List schedule items | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/schedule` | Create a schedule item (premiere \| embargo) | application/json | 201, 401, 409, 422, 429, 503 |
| GET | `/api/staff/schedule/{schedule_id}` | Get a single schedule item by id | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/schedule/{schedule_id}/cancel` | Cancel a schedule item | | 200, 401, 404, 422, 429, 503 |

**/api/staff/self-tests**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/self-tests` | Self-test history (daily/weekly) | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/self-tests/run` | Run a self-test on demand (daily or weekly check set) | | 200, 401, 422, 429, 503 |

**/api/staff/station**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/station/profile` | Read the station identity profile (name, timezone, storage roots, channel) | | 200, 401, 429 |
| PUT | `/api/staff/station/profile` | Edit the mutable station identity profile fields (including the live-caption switch) | application/json | 200, 401, 422, 429 |
| GET | `/api/staff/station/timezone` | Read the station timezone used by scheduling forms | | 200, 401, 429 |

**/api/staff/station-box-profile**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/station-box-profile` | Full StationBox-Profile: cable/PEG appliance-readiness report | | 200, 401, 422, 429 |
| GET | `/api/staff/station-box-profile/readiness` | PEG readiness roll-up only (cheap poll target for S8 alerting) | | 200, 401, 422, 429 |

**/api/staff/stream**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/stream/overlay-compositor-plan` | Preview a streaming overlay compositor plan | application/json | 200, 401, 422, 429 |

**/api/staff/subscribe**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/subscribe/dispatch-test` | Dispatch a simulated subscription notification for local proof | application/json | 200, 401, 409, 422, 429 |

**/api/staff/summaries**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/summaries/generate` | Generate a sourced summary draft from committed transcript cues | application/json | 201, 401, 422, 429, 503 |
| GET | `/api/staff/summaries/jobs` | List summary generation jobs for operator visibility | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/summaries/jobs` | Queue async summary generation from committed transcript cues | application/json | 201, 401, 422, 429, 503 |
| GET | `/api/staff/summaries/jobs/{job_id}` | Get one summary generation job's status/progress | | 200, 401, 404, 422, 429, 503 |
| POST | `/api/staff/summaries/jobs/{job_id}/retry` | Manually retry a failed summary generation job | | 200, 401, 404, 409, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/summaries/review-items` | List sourced summaries awaiting review or ready for export | | 200, 401, 429 |
| POST | `/api/staff/summaries/transcript.csv` | Export committed transcript cues as CSV | application/json | 200, 401, 422, 429 |
| PATCH | `/api/staff/summaries/{summary_id}` | Edit a pending summary narrative before approval | application/json | 200, 401, 404, 409, 422, 429 |
| POST | `/api/staff/summaries/{summary_id}/approve` | Approve a sourced summary for signed-record export | application/json | 200, 401, 404, 409, 422, 429 |

**/api/staff/system-resources**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/system-resources` | Recent system-resource samples | | 200, 401, 422, 429, 503 |

**/api/staff/underwriting**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/underwriting/affidavits` | Per-underwriter proof-of-airing for a period (billing-ready, DC-3) | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/underwriting/affidavits/export` | Download an affidavit as CSV / XML / PDF for billing | | 200, 401, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/staff/underwriting/compile` | Run the trafficking compiler for one date over operator-supplied candidates | application/json | 200, 401, 422, 429, 503 |
| GET | `/api/staff/underwriting/flights` | List flights, optionally narrowed by spot or active date | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/underwriting/flights` | Create a flight | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/underwriting/flights/{flight_id}` | Delete a flight (cascades placements) | | 204, 401, 404, 422, 429, 503 |
| GET | `/api/staff/underwriting/flights/{flight_id}` | Get one flight | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/underwriting/flights/{flight_id}` | Patch a flight (absent keys unchanged) | application/json | 200, 401, 404, 422, 429, 503 |
| GET | `/api/staff/underwriting/placements` | What will / did air — placements over a window | | 200, 401, 422, 429, 503 |
| GET | `/api/staff/underwriting/spots` | List underwriting spots for the station | | 200, 401, 422, 429, 503 |
| POST | `/api/staff/underwriting/spots` | Create an underwriting spot | application/json | 201, 401, 422, 429, 503 |
| DELETE | `/api/staff/underwriting/spots/{spot_id}` | Delete an underwriting spot (cascades flights + placements) | | 204, 401, 404, 422, 429, 503 |

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/staff/underwriting/spots/{spot_id}` | Get one underwriting spot | | 200, 401, 404, 422, 429, 503 |
| PATCH | `/api/staff/underwriting/spots/{spot_id}` | Patch an underwriting spot (absent keys unchanged) | application/json | 200, 401, 404, 422, 429, 503 |

**/api/version**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| GET | `/api/version` | Get Version | | 200 |

**/api/webhooks/stripe**

| Method | Path | Summary | Body | Responses |
|---|---|---|---|---|
| POST | `/api/webhooks/stripe` | Stripe webhook (signed) | | 200, 400, 401, 413, 503 |

## Appendix C: Settings and environment variables

CivicCast's low-level settings are *environment variables*: named values that Windows hands to a program when it starts. Most day-to-day choices are made on screens (Station Profile, AI Models, and so on); environment variables are for things the screens do not offer, such as how big the cache of prepared video may grow, or which ports the internal relays use. Changing one is an IT job.

### How the names are built

Every CivicCast setting starts with `CIVICCAST_` followed by capital letters and underscores, for example `CIVICCAST_CONFORM_CACHE_GB`. Values are plain text; a number is typed as digits. Yes/no switches usually accept `1` or `0` (some also accept `on`, `off`, `true`, `false`, `yes`, `no`; the table does not say which, so check the setting's own text before relying on a spelling). A few settings are also known by older or shorter names; the table lists each name exactly as the code reads it.

### The one-C spelling trap

The prefix is spelled **C-I-V-I-C-C-A-S-T** (two C's in the middle, nine letters). A setting typed **CIVICAST_** (one C, eight letters) looks almost identical and is silently ignored by anything that reads the correct name. This has already happened in the product's own code. The rule for beta.11:

1. **Use the two-C spelling for every setting**, with the exceptions in the next point.

2. **Four product settings are read only under the one-C spelling.** For these, the correct two-C spelling does nothing:
    - `CIVICAST_CAPTION_PROOF_POLL_SECONDS` (how often the on-air caption proof runs; default 30)
    - `CIVICAST_CAPTION_PROOF_TIMEOUT_SECONDS` (time limit for one proof; default 15)
    - `CIVICAST_GST_LEG_RATE_DIAG` (a diagnostic log line; on unless set to `0`, `off`, `false` or `no`)
    - `CIVICAST_WORKER_PERSISTENT` (set by the product itself; leave it alone)
3. **Two settings accept both spellings.** `CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS` (default 300) and `CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS` (default 1800) are read through a resolver. It uses the two-C name when that is set to something other than blank. Otherwise it uses the one-C name and writes a one-time "deprecated" warning to the log. When both are set and differ, the two-C value wins and a warning names both. A blank value counts as not set at either spelling. The one-C names will stop being read in a future release.
4. **Everything else in the table with one C is not for stations.** `CIVICAST_CI_SOURCE_SHA`, `CIVICAST_CLEANROOM_*`, `CIVICAST_EXPECTED_VERSION` and `CIVICAST_KEEP_PASSING_UI_EVIDENCE` belong to the project's build and test scripts. `CIVICAST_CONTROL_ROOM_TSR_URL` appears only in a test file (the setting the product reads is `CIVICCAST_CONTROL_ROOM_TSR_URL`). `CIVICAST_OBS_WEBSOCKET_PASSWORD` is read only by a laboratory test tool. `CIVICAST_WSL_DRIVE_MOUNT_ROOT` and `CIVICAST_TRANSLATE_WINDOWS_BACKUP_PATHS` belong to the retired WSL edition.

> **Tip:** After setting any value, restart the service and read the end of `C:\ProgramData\CivicCast\logs\control_plane-app.log`. A setting with the wrong name is not logged at all, which is why the spelling matters. How each setting treats a bad value differs (the cache size, for example, silently falls back to its default), and we did not check every setting.

#### How to set a setting for the service

CivicCast runs as the Windows service `CivicCastSupervisor`. The service starts the control plane and the other programs with its own environment plus a small set of fixed values it adds last. So a setting must be in the **service's** environment, not in your own PowerShell window.

1. Open Registry Editor as Administrator.
2. Go to `HKLM\SYSTEM\CurrentControlSet\Services\CivicCastSupervisor`.
3. Add or edit the multi-string value named `Environment`. One setting per line, written `NAME=value`, for example `CIVICCAST_CONFORM_CACHE_GB=80`.
4. Restart the service: in PowerShell, `Restart-Service CivicCastSupervisor`. Stopping it takes up to 15 seconds per child program; a channel on air goes off air while the service restarts.

> **Warning:** Restarting the service stops every channel. Do it between meetings.

> **Note:** Code comments in the product describe this `Environment` value as the place a station's settings live, but we found no installer code that writes it, so on a fresh install it may not exist and you create it. If you also set a machine-wide Windows environment variable, the service inherits it when it starts; the per-service value is the one the product documents.

**Settings the service sets for itself** The station sets these when it starts the control plane. Do not set them yourself.

| Setting | Value the service gives it |
|---|---|
| `CIVICCAST_EGRESS_WORK_DIR` | `C:\ProgramData\CivicCast\data\egress` |
| `CIVICCAST_UPLOAD_DIR` | `C:\ProgramData\CivicCast\data\uploads` |
| `CIVICCAST_SUPERVISED` | `1` |
| `CIVICCAST_LAN_ONLY_STATION` | `1` (turns off `/docs` and `/redoc`, puts `/openapi.json` behind a token) |
| `CIVICCAST_OPERATOR_CONSOLE_DIST`, `CIVICCAST_PUBLIC_PORTAL_DIST` | the folders holding the two web portals |
| `CIVICCAST_NATIVE_STATION`, `CIVICCAST_NATIVE_STATION_ROOT`, `CIVICCAST_NATIVE_STATION_MANIFEST` | the activated station |
| `CIVICCAST_CAPTION_RUNTIME`, `CIVICCAST_CAPTION_TIER`, `CIVICCAST_WHISPER_MODEL_PATH`, `CIVICCAST_WHISPER_DEVICE`, `CIVICCAST_WHISPER_COMPUTE_TYPE`, `CIVICCAST_EGRESS_EMBED_CAPTIONS` | caption model choices made at activation |
| `DATABASE_URL` | read from the registry value `HKLM\SOFTWARE\CivicCast\Native\DatabaseUrl` unless it is already set in the service environment (an environment value wins) |
| `OLLAMA_HOST`, `OLLAMA_MODELS`, `OLLAMA_NO_CLOUD`, `NO_PROXY` | for the AI engine: `127.0.0.1:11434`, the model folder, cloud use off |

`CIVICCAST_CAPTION_TAP` is the one exception: if you set it to the exact word `off` in the service environment, the service passes that through and live caption audio tapping stays off; any other value leaves the normal behavior.

`CIVICCAST_CAPTION_RUNTIME=faster-whisper` identifies the Whisper runtime used for recorded captions and fallback. Separately, `CIVICCAST_LIVE_CAPTION_ENGINE` defaults to `whistle` on a native station; IT can explicitly select `whisper`. These settings describe different parts of the caption path.

**A few settings worth knowing** Each default below is read from the code. The full list follows in the generated table.

| Setting | What it does | Default |
|---|---|---|
| `CIVICCAST_CONFORM_CACHE_GB` | How large the folder of prepared copies of programs (the *conform cache*) may grow, in gigabytes. | 60 |
| `CIVICCAST_AUTH_RATE_LIMIT` / `CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS` | Failed token checks allowed per client address, and the window in seconds. | 10 / 60 |
| `CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS` | Longest one video preparation step may run. Accepts either spelling. | 300 |
| `CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS` | How far ahead the playout engine prepares a program. Accepts either spelling. | 1800 |
| `CIVICCAST_AUTOMATION_PASS_WATCHDOG_SECONDS` / `..._REPEAT_SECONDS` | When one channel-automation pass has run so long that its stack is written to the log, and how often it repeats. | 30 / 60 |
| `CIVICCAST_HLS_RELAY_BASE_PORT` | First of 500 local UDP ports used by the web-video relays. | 18000 |
| `CIVICCAST_TS_RELAY` | Cable relay mode: `auto`, `on` or `off`. | auto |
| `CIVICCAST_TS_RELAY_BASE_PORT` | First local UDP port used by the cable relay. | 17800 |
| `CIVICCAST_LIVE_SOURCE_READINESS_TTL_SECONDS` | How long a "Delivering" source check stays fresh on the Live screen (clamped 5 to 300). | 30 |
| `CIVICCAST_WHISPER_FALLBACK_TIMEOUT_SECONDS` | Deadline for a Whisper fallback recognition request. | 120 |
| `CIVICCAST_REMOTE_CONTRIBUTION_MEDIAMTX_URL` | Commissioned MediaMTX public WHIP endpoint for Remote Contribution. | unset |
| `CIVICCAST_STAFF_TOKENS` | Older way to list staff tokens, one entry per token. | unset |
| `CIVICCAST_STATION_NAME`, `CIVICCAST_STATION_TZ`, `CIVICCAST_STATION_MEDIA_LIBRARY`, `CIVICCAST_STATION_RECORDINGS`, `CIVICCAST_STATION_BACKUPS` | Overrides for what Station Profile shows. | per Station Profile |

| Setting | What it does | Default |
|---|---|---|
| `CIVICCAST_ACTIVITYPUB_*` | Federation switches (mode, handle, base URL, key path). Off by default; see Chapter 15. | off |

> **Warning:** The Live screen's checks need 50 GiB free on the drive that holds `CIVICCAST_UPLOAD_DIR`, regardless of how small a meeting is. That number is built into the program and is not a setting.

#### The full table

How to read it. **Name**: exactly as the code reads it. **Prod refs**: how many places in the product source (tests excluded) use the name; a count of 1 or 2 means a narrow setting. **First location**: where to look in the source. **Documented**: whether the name appears in a prose document in the repository. **Notes**: only the one-C rows have a note.

A few rows are not real settings. Names ending in an underscore (for example `CIVICCAST_FASTLY_`) are family prefixes written in code comments. `CIVICCAST_ALERT` is the name of an installer script macro, not a setting. The generated table lists the names found in this source edition, including legacy one-C spellings; it is not a list of recommended settings.

443 distinct names (20 use the one-C `CIVICAST_` spelling). Generated by `ops/docs-sprint/tools/gen_env_inventory.py`; do not edit by hand.

`Documented` = the name appears in a prose doc under `docs/` or a root `.md` file. `Prod refs` counts non-test source references; the first location is shown.

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| `CIVICAST_ACTIVE_NETWORK_SETTINGS` | 1 | civiccast/native/supervisor/service.py:326 | no | one-C spelling: check it is not silently inert |
| `CIVICAST_CAPTION_PROOF_POLL_SECONDS` | 1 | civiccast/app.py:1767 | yes | one-C spelling: check it is not silently inert |
| `CIVICAST_CAPTION_PROOF_TIMEOUT_SECONDS` | 1 | civiccast/app.py:1769 | yes | one-C spelling: check it is not silently inert |
| `CIVICAST_CI_SOURCE_SHA` | 2 | scripts/collect_source_state.py:107 | yes | one-C spelling: check it is not silently inert |
| `CIVICAST_CLEANROOM_VBOX_REPORT` | 1 | scripts/run_clean_windows_install_proof.py:248 | no | one-C spelling: check it is not silently inert |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICAST_CLEANROOM_VBOX_REPORT_MAX_AGE_SECONDS | 1 | scripts/run_clean_windows_install_proof.py:442 | no | one-C spelling: check it is not silently inert |
| CIVICAST_CLEANROOM_VBOX_VM | 2 | scripts/run_clean_windows_install_proof.py:84 | no | one-C spelling: check it is not silently inert |
| CIVICAST_CONTROL_ROOM_TSR_URL | 1 | civiccast/apps/portal/e2e/operator/e2e/control-room-readiness.spec.ts:30 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_EGRESS_PREPARATION_TIMEOUT_SECONDS | 4 | civiccast/egress/automation.py:1599 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_EXPECTED_VERSION | 1 | scripts/policy/check_release_identity.py:229 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_GSTREAMER_SOURCE_SEGMENT_SECONDS | 2 | civiccast/egress/envvars.py:17 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_GST_LEG_RATE_DIAG | 2 | civiccast/egress/gstengine.py:656 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_KEEP_PASSING_UI_EVIDENCE | 5 | civiccast/apps/portal/e2e/operator/playwright.config.ts:14 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_OBS_WEBSOCKET_PASSWORD | 4 | civiccast/control_room/lpm_lab_stage45.py:240 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_PUBLIC_BASE_URL | 1 | civiccast/native/supervisor/service.py:361 | no | one-C spelling: check it is not silently inert |
| CIVICAST_TARGET_DATABASE_URL | 1 | civiccast/dr/station_backup_cli.py:84 | no | one-C spelling: check it is not silently inert |
| CIVICAST_TRANSLATE_WINDOWS_BACKUP_PATHS | 1 | civiccast/installer/service.py:1131 | yes | one-C spelling: check it is not silently inert |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICAST_TRUSTED_PROXY_CIDRS_EXACT | 1 | civiccast/native/supervisor/service.py:2022 | no | 3rd C spelling: check it is not silently inert |
| CIVICAST_WORKER_PERSISTENT | 1 | civiccast/egress/gstreload_policy.py:75 | yes | one-C spelling: check it is not silently inert |
| CIVICAST_WSL_DRIVE_MOUNT_ROOT | 1 | civiccast/installer/service.py:1130 | yes | one-C spelling: check it is not silently inert |
| CIVICCAST_ACCEPTANCE_PACKET_DIR | 1 | civiccast/installer/service.py:3517 | yes | |
| CIVICCAST_ACTIVE_NETWORK_SETTINGS | 2 | civiccast/app.py:2663 | no | |
| CIVICCAST_ACTIVITYPUB_ALLOWLIST | 1 | civiccast/activitypub/config.py:79 | yes | |
| CIVICCAST_ACTIVITYPUB_ALLOWLIST_FILE | 1 | civiccast/activitypub/config.py:80 | yes | |
| CIVICCAST_ACTIVITYPUB_AUTHORIZED_FETCH | 5 | civiccast/cli.py:2228 | yes | |
| CIVICCAST_ACTIVITYPUB_BASE_URL | 5 | civiccast/cli.py:2225 | yes | |
| CIVICCAST_ACTIVITYPUB_BLOCKLIST | 1 | civiccast/activitypub/config.py:75 | yes | |
| CIVICCAST_ACTIVITYPUB_BLOCKLIST_FILE | 1 | civiccast/activitypub/config.py:76 | yes | |
| CIVICCAST_ACTIVITYPUB_DISPLAY_NAME | 1 | civiccast/activitypub/config.py:86 | yes | |
| CIVICCAST_ACTIVITYPUB_HANDLE | 5 | civiccast/cli.py:2226 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_ACTIVITYPUB_INBOX_RATE_LIMIT | 1 | civiccast/activitypub/config.py:95 | yes | |
| CIVICCAST_ACTIVITYPUB_INBOX_RATE_WINDOW_SECONDS | 1 | civiccast/activitypub/config.py:97 | yes | |
| CIVICCAST_ACTIVITYPUB_LAB_ALLOW_LOCAL | 1 | civiccast/activitypub/config.py:94 | yes | |
| CIVICCAST_ACTIVITYPUB_MODE | 10 | civiccast/cli.py:2224 | yes | |
| CIVICCAST_ACTIVITYPUB_PRIVATE_KEY_PATH | 6 | civiccast/cli.py:2227 | yes | |
| CIVICCAST_ACTIVITYPUB_PUBLIC_KEY_PEM | 2 | civiccast/cli.py:2230 | yes | |
| CIVICCAST_ACTIVITYPUB_RETRY_BACKOFF_SECONDS | 1 | civiccast/activitypub/retry_worker.py:74 | yes | |
| CIVICCAST_ACTIVITYPUB_RETRY_MAX_ATTEMPTS | 1 | civiccast/activitypub/retry_worker.py:77 | yes | |
| CIVICCAST_ACTIVITYPUB_RETRY_POLL_SECONDS | 1 | civiccast/activitypub/retry_worker.py:71 | yes | |
| CIVICCAST_ACTIVITYPUB_RETRY_WORKER | 3 | civiccast/activitypub/retry_worker.py:15 | yes | |
| CIVICCAST_ADDSIZE_PACKS_KB | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:172 | yes | |
| CIVICCAST_AGENDA_IMPORT_SOURCE | 10 | civiccast/agenda_import/config.py:7 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_AGENDA_SOURCE_CLIENT | 1 | civiccast/agenda_import/config.py:118 | yes | |
| CIVICCAST_AGENDA_SOURCE_TIMEOUT_S | 2 | civiccast/agenda_import/config.py:120 | yes | |
| CIVICCAST_AGENDA_SOURCE_TOKEN | 4 | civiccast/agenda_import/config.py:119 | yes | |
| CIVICCAST_AKAMAI_ | 1 | civiccast/stream/cdn/factory.py:18 | yes | |
| CIVICCAST_AKAMAI_ACCESS_KEY_ID | 2 | civiccast/dr/station_backup_runtime.py:71 | yes | |
| CIVICCAST_AKAMAI_BUCKET | 2 | civiccast/dr/station_backup_runtime.py:73 | yes | |
| CIVICCAST_AKAMAI_PUBLIC_BASE_URL | 2 | civiccast/dr/station_backup_runtime.py:74 | yes | |
| CIVICCAST_AKAMAI_REGION | 2 | civiccast/dr/station_backup_runtime.py:70 | yes | |
| CIVICCAST_AKAMAI_SECRET_ACCESS_KEY | 2 | civiccast/dr/station_backup_runtime.py:72 | yes | |
| CIVICCAST_ALERT | 25 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:261 | yes | |
| CIVICCAST_ALERTING | 4 | civiccast/app.py:150 | yes | |
| CIVICCAST_ALERT_CREDENTIALS_FILE | 4 | civiccast/alerting/credentials.py:55 | yes | |
| CIVICCAST_ALLOW_DETERMINISTIC_STAFF_TOKEN | 6 | civiccast/auth/tokens.py:73 | yes | |
| CIVICCAST_ALLOW_DETERMINISTIC_SUBSCRIBE_SECRETS | 1 | civiccast/subscribe/secrets.py:86 | yes | |
| CIVICCAST_ALLOW_DEVELOPMENT_PACK_KEY | 5 | civiccast/apps/installer/src-tauri/src/native_packs.rs:247 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_ALLOW_EPHEMERAL_STORES | 22 | civiccast/app.py:1899 | yes | |
| CIVICCAST_ALLOW_FIRST_ADMIN_RESET | 5 | civiccast/installer/router.py:1567 | yes | |
| CIVICCAST_ALLOW_INSECURE_MANIFEST | 4 | civiccast/vod/models.py:20 | yes | |
| CIVICCAST_ANALYTICS_RATE_LIMIT_MAX_BUCKETS | 1 | civiccast/app_platform/router.py:559 | yes | |
| CIVICCAST_ANALYTICS_RETENTION_DAYS | 1 | civiccast/analytics/store.py:249 | yes | |
| CIVICCAST_ANALYTICS_ROLLUP_LOOKBACK_DAYS | 1 | civiccast/analytics/postgres_store.py:394 | yes | |
| CIVICCAST_ANALYTICS_ROLLUP_POLL_SECONDS | 1 | civiccast/analytics/postgres_store.py:393 | yes | |
| CIVICCAST_ANALYTICS_ROLLUP_WORKER | 2 | civiccast/analytics/postgres_store.py:385 | yes | |
| CIVICCAST_ANALYTICS_TRUSTED_PROXY_CIDRS | 5 | civiccast/cli.py:2407 | yes | |
| CIVICCAST_APP_BUILD_ARTIFACTS_DIR | 1 | civiccast/app_platform/build_router.py:82 | yes | |
| CIVICCAST_APP_BUILD_STORE_PATH | 1 | civiccast/app_platform/build_store.py:122 | yes | |
| CIVICCAST_APP_PLATFORM_CONFIG_PATH | 1 | civiccast/app_platform/store.py:230 | yes | |
| CIVICCAST_AUTH_STACK | 3 | civiccast/app.py:2742 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_AUTH_RATE_LIMIT | 2 | civiccast/auth/rate_limit.py:74 | yes | |
| CIVICCAST_AUTH_RATE_LIMIT_WINDOW_SECONDS | 2 | civiccast/auth/rate_limit.py:75 | yes | |
| CIVICCAST_AUTOMATION_PASS_WATCHDOG_REPEAT_SECONDS | 1 | civiccast/egress/automation.py:273 | yes | |
| CIVICCAST_AUTOMATION_PASS_WATCHDOG_SECONDS | 2 | civiccast/egress/automation.py:20 | yes | |
| CIVICCAST_AUTOSCHEDULE | 4 | civiccast/app.py:1557 | yes | |
| CIVICCAST_AVAILABLE_VERSION | 3 | civiccast/installer/service.py:1483 | yes | |
| CIVICCAST_BACKUP_DIR | 2 | civiccast/installer/service.py:208 | yes | |
| CIVICCAST_BOOTSTRAP_INSTANCE_ID | 1 | civiccast/app.py:2607 | yes | |
| CIVICCAST_BULLETIN_EXPIRY | 3 | civiccast/app.py:1585 | yes | |
| CIVICCAST_BUNNY | 1 | civiccast/stream/cdn.py:12 | yes | |
| CIVICCAST_BUNNY_ACCESS_KEY | 4 | civiccast/dr/station_backup_runtime.py:63 | yes | |
| CIVICCAST_BUNNY_CDN_HOSTNAME | 4 | civiccast/dr/station_backup_runtime.py:64 | yes | |
| CIVICCAST_BUNNY_STORAGE_ZONE | 4 | civiccast/dr/station_backup_runtime.py:62 | yes | |
| CIVICCAST_CABLE_CAPTIONS_DIR | 2 | civiccast/cable/package.py:174 | yes | |
| CIVICCAST_CABLE_PACKAGE_OUTPUT_DIR | 2 | civiccast/cable/package.py:173 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_CANDIDATE_WORKFLOW_RUN_ID | 2 | scripts/verify-release.sh:15 | yes | |
| CIVICCAST_CAPTION_AUDIO | 2 | scripts/run_real_ai_release_proof.py:81 | | |
| CIVICCAST_CAPTION_FEED_POLL_SECONDS | 1 | civiccast/app.py:1759 | yes | |
| CIVICCAST_CAPTION_MODEL_HASH_RECEIPT | 1 | civiccast/native/station_runtime.py:1459 | | |
| CIVICCAST_CAPTION_PROOF_PROCESS | 3 | civiccast/app.py:1771 | yes | |
| CIVICCAST_CAPTION_PROOF_PROCESS_CHILD | 1 | civiccast/egress/caption_proof_process.py:91 | | |
| CIVICCAST_CAPTION_RUNTIME | 5 | civiccast/captions/tap_worker.py:2730 | | |
| CIVICCAST_CAPTION_TAP | 20 | civiccast/app.py:1598 | yes | |
| CIVICCAST_CAPTION_TAP_ATOMIC | 2 | civiccast/captions/tap_worker.py:372 | | |
| CIVICCAST_CAPTION_TAP_BATCH_DIAGNOSTIC | 3 | civiccast/captions/tap_batch_diagnostic.py:23 | | |
| CIVICCAST_CAPTION_TAP_CATCH_UP_SHED_LIMIT | 1 | civiccast/captions/tap_worker.py:399 | | |
| CIVICCAST_CAPTION_TAP_CATCH_UP_SHED_WINDOW_SECONDS | 1 | civiccast/captions/tap_worker.py:407 | | |
| CIVICCAST_CAPTION_TAP_CPU_THREADS | 7 | civiccast/captions/runtime.py:154 | | |
| CIVICCAST_CAPTION_TAP_DIR | 25 | civiccast/app.py:1598 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_CAPTION_TAP_MAX_BACKLOG_SEGMENTS | 1 | civiccast/captions/tap_worker.py:383 | yes | |
| CIVICCAST_CAPTION_TAP_MAX_CHANNEL_WORKERS | 3 | civiccast/captions/tap_worker.py:234 | yes | |
| CIVICCAST_CAPTION_TAP_MAX_OVERLOAD_BACKOFF_SECONDS | 2 | civiccast/captions/tap_worker.py:443 | yes | |
| CIVICCAST_CAPTION_TAP_OVERLAP_SECONDS | 1 | civiccast/captions/tap_worker.py:374 | yes | |
| CIVICCAST_CAPTION_TAP_OVERLOAD_BACKOFF_SECONDS | 1 | civiccast/captions/tap_worker.py:440 | yes | |
| CIVICCAST_CAPTION_TAP_OVERLOAD_PERSISTENCE_SCANS | 1 | civiccast/captions/tap_worker.py:391 | yes | |
| CIVICCAST_CAPTION_TAP_PHASE_TIMING | 3 | civiccast/captions/phase_timing.py:19 | yes | |
| CIVICCAST_CAPTION_TAP_POLL_SECONDS | 1 | civiccast/captions/tap_worker.py:376 | yes | |
| CIVICCAST_CAPTION_TAP_SEGMENT_SECONDS | 5 | civiccast/captions/tap.py:72 | yes | |
| CIVICCAST_CAPTION_TIER | 3 | civiccast/ai_models/runtime.py:134 | | |
| CIVICCAST_CAPTION_TIER_EVENT | 4 | civiccast/app.py:1077 | yes | |
| CIVICCAST_CAPTION_TRUTH | 2 | scripts/run_real_ai_release_proof.py:82 | | |
| CIVICCAST_CDN_PROVIDER | 17 | civiccast/cli.py:2324 | yes | |
| CIVICCAST_CDN_STUB_ROOT | 3 | civiccast/stream/cdn_factory.py:22 | yes | |
| CIVICCAST_CERT_ROOT | 4 | civiccast/cli.py:917 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_CG_DEMO_FEEDS | 3 | civiccast/cg/router.py:76 | yes | |
| CIVICCAST_CHANNEL_AUTOMATION | 3 | civiccast/egress/automation.py:32 | yes | |
| CIVICCAST_CHANNEL_AUTOMATION_POLL_SECONDS | 3 | civiccast/egress/automation.py:772 | yes | |
| CIVICCAST_CLEANROOM_VM | 1 | scripts/run_vm_cleanroom_release.py:131 | yes | |
| CIVICCAST_COMPONENTS_BASE_URL | 1 | civiccast/apps/installer/src-tauri/src/acquisition_catalog.rs:216 | yes | |
| CIVICCAST_CONFIG_DIR | 3 | civiccast/alerting/credentials.py:55 | yes | |
| CIVICCAST_CONFORM_CACHE_GB | 5 | civiccast/egress/preparer.py:23 | yes | |
| CIVICCAST_CONSOLE_ENTRY_POINTS | 7 | civiccast/native/appspayload.py:62 | yes | |
| CIVICCAST_CONSOLE_LAUNCHERS | 7 | civiccast/native/appspayload.py:63 | yes | |
| CIVICCAST_CONTAINED | 1 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:355 | yes | |
| CIVICCAST_CONTRIBUTOR_MAX_UPLOAD_BYTES | 1 | civiccast/contributor/router.py:97 | yes | |
| CIVICCAST_CONTRIBUTOR_STORE_PATH | 2 | civiccast/contributor/store.py:448 | yes | |
| CIVICCAST_CONTRIBUTOR_SUBMISSION_RATE_LIMIT | 1 | civiccast/contributor/router.py:376 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_DIR | 2 | civiccast/contributor/store.py:457 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_CONTRIBUTOR_UPLOAD_DIR_MAX_BYTES | 2 | civiccast/contributor/router.py:124 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_MAX_AGE_HOURS | 2 | civiccast/contributor/store.py:510 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_PER_IP_MAX_BYTES | 1 | civiccast/contributor/router.py:159 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_RATE_LIMIT | 1 | civiccast/contributor/router.py:316 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_REAP | 1 | civiccast/app.py:1375 | yes | |
| CIVICCAST_CONTRIBUTOR_UPLOAD_REAP_POLL_SECONDS | 1 | civiccast/app.py:1367 | yes | |
| CIVICCAST_CONTROL_PLANE_URL | 2 | civiccast/native/supervisor/service.py:2057 | | |
| CIVICCAST_CONTROL_ROOM_TSR_URL | 5 | civiccast/app.py:3245 | yes | |
| CIVICCAST_CORS_ALLOWED_ORIGINS | 5 | civiccast/app.py:2368 | yes | |
| CIVICCAST_COTURN_COMMAND | 5 | civiccast/installer/contribution_install.py:24 | | |
| CIVICCAST_CUDA_BIN_DIR | 3 | civiccast/captions/runtime.py:60 | yes | |
| CIVICCAST_DATABASE_URL | 3 | civiccast/apps/installer/src-tauri/src/native_service_registration.rs:1821 | yes | |
| CIVICCAST_DB_CONNECT_TIMEOUT | 5 | civiccast/db/session.py:99 | yes | |
| CIVICCAST_DISTRIBUTION | 16 | civiccast/native/app/payload.py:64 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_EAS | 5 | civiccast/app.py:1815 | yes | |
| CIVICCAST_EAS_AUTO_SURFACE | 2 | civiccast/app.py:1822 | yes | |
| CIVICCAST_EAS_POLL_SECONDS | 1 | civiccast/app.py:1837 | yes | |
| CIVICCAST_EGRESS_DEGRADED_REASON | 1 | civiccast/native/station_runtime.py:96 | yes | |
| CIVICCAST_EGRESS_DRAIN_DEADLINE_SECONDS | 2 | civiccast/app.py:655 | yes | |
| CIVICCAST_EGRESS_EMBED_CAPTIONS | 11 | civiccast/app.py:1726 | yes | |
| CIVICCAST_EGRESS_ENGINE | 16 | civiccast/cli.py:1250 | yes | |
| CIVICCAST_EGRESS_PREPARATION_TIMEOUT_SECONDS | 3 | civiccast/egress/automation.py:976 | yes | |
| CIVICCAST_EGRESS_ROLLOVER_LEAD_SECONDS | 1 | civiccast/egress/automation.py:255 | yes | |
| CIVICCAST_EGRESS_SEAMLESS_RELOAD | 11 | civiccast/egress/daemon.py:2719 | yes | |
| CIVICCAST_EGRESS_WORK_DIR | 12 | civiccast/egress/automation.py:306 | yes | |
| CIVICCAST_EMAIL_FROM | 1 | civiccast/publish/providers.py:30 | yes | |
| CIVICCAST_EMAIL_SMTP_URL | 1 | civiccast/publish/providers.py:30 | yes | |
| CIVICCAST_EVIDENCE_DIR | 8 | civiccast/apps/portal/operator/e2e/a11y.spec.ts:20 | yes | |
| CIVICCAST_EXIT_D2_APP_PAYLOAD | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:577 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_EXIT_D2_FFMPEG_RUNTIME | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:599 | yes | |
| CIVICCAST_EXIT_D2_OLLAMA_RUNTIME | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:600 | yes | |
| CIVICCAST_EXIT_D2_SERVER_BINARIES | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:576 | yes | |
| CIVICCAST_EXIT_D3_FAULT | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:580 | yes | |
| CIVICCAST_EXIT_D3_HALTED | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:578 | yes | |
| CIVICCAST_EXIT_D3_REFUSED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:579 | yes | |
| CIVICCAST_EXIT_D3_REFUSED_DOWNGRADE | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:669 | yes | |
| CIVICCAST_EXIT_D3_ROLLED_BACK_FLAT | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:624 | yes | |
| CIVICCAST_EXIT_D3_STALE_JOURNAL | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:663 | yes | |
| CIVICCAST_EXIT_D4_ACTIVATION | 8 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:607 | yes | |
| CIVICCAST_EXIT_D4_FIREWALL | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:584 | yes | |
| CIVICCAST_EXIT_D4_OTHER_PRODUCT | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:705 | yes | |
| CIVICCAST_EXIT_D4_PROVISION_FAILED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:581 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_EXIT_D4_PROVISION_FAULT | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:582 | yes | |
| CIVICCAST_EXIT_D4_RUNTIME_OWNERSHIP | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:653 | yes | |
| CIVICCAST_EXIT_D4_SERVICE | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:583 | yes | |
| CIVICCAST_EXIT_D4_SERVICE_NOT_SERVING | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:638 | yes | |
| CIVICCAST_EXIT_D4_SERVICE_NOT_STARTING | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:644 | yes | |
| CIVICCAST_EXIT_INSTALL_RECOVERY | 5 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:360 | no | |
| CIVICCAST_EXIT_PACK_DELIVERY | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:575 | yes | |
| CIVICCAST_EXIT_UNINSTALL_BLOCKED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:692 | yes | |
| CIVICCAST_EXIT_UNINSTALL_DECLINED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:690 | yes | |
| CIVICCAST_EXIT_UNINSTALL_INCOMPLETE | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:694 | yes | |
| CIVICCAST_EXIT_UNINSTALL_TRANSFER_FAILED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:691 | yes | |
| CIVICCAST_EXIT_UNINSTALL_TREES_RETAINED | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:693 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_EXIT_UPGRADE_QUIESCE | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:592 | yes | |
| CIVICCAST_FAIL | 52 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:45 | yes | |
| CIVICCAST_FALLBACK_OK | 2 | civiccast/apps/installer/src-tauri/src/main.rs:5113 | yes | |
| CIVICCAST_FASTLY_ | 1 | civiccast/stream/cdn/factory.py:18 | yes | |
| CIVICCAST_FASTLY_ACCESS_KEY_ID | 2 | civiccast/dr/station_backup_runtime.py:66 | yes | |
| CIVICCAST_FASTLY_BUCKET | 2 | civiccast/dr/station_backup_runtime.py:68 | yes | |
| CIVICCAST_FASTLY_PUBLIC_BASE_URL | 2 | civiccast/dr/station_backup_runtime.py:69 | yes | |
| CIVICCAST_FASTLY_REGION | 2 | civiccast/dr/station_backup_runtime.py:65 | yes | |
| CIVICCAST_FASTLY_SECRET_ACCESS_KEY | 2 | civiccast/dr/station_backup_runtime.py:67 | yes | |
| CIVICCAST_FEDERATION_CONFIG | 1 | civiccast/federation/service.py:212 | yes | |
| CIVICCAST_FFMPEG | 2 | sandbox-lab/scripts/In-Sandbox-Report.ps1:3252 | yes | |
| CIVICCAST_FINALIZATION_ | 1 | civiccast/live/finalization_worker.py:1007 | yes | |
| CIVICCAST_FINALIZATION_BACKOFF_SECONDS | 1 | civiccast/live/finalization_worker.py:138 | yes | |
| CIVICCAST_FINALIZATION_MAX_ATTEMPTS | 1 | civiccast/live/finalization_worker.py:136 | yes | |
| CIVICCAST_FINALIZATION_NEVER_APPEARED_SECONDS | 1 | civiccast/live/finalization_worker.py:145 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_FINALIZATION_POLL_SECONDS | 1 | civiccast/live/finalization_worker.py:140 | yes | |
| CIVICCAST_FINALIZATION_RUNNING_LEASE_SECONDS | 1 | civiccast/live/finalization_worker.py:142 | yes | |
| CIVICCAST_FINALIZATION_SETTLE_SECONDS | 1 | civiccast/live/finalization_worker.py:134 | yes | |
| CIVICCAST_FINALIZATION_WORKER | 3 | civiccast/live/finalization_worker.py:117 | yes | |
| CIVICCAST_GSTREAMER_PYTHON | 2 | civiccast/native/gstreamer_runtime.py:96 | yes | |
| CIVICCAST_GSTREAMER_RUNTIME_ROOT | 4 | civiccast/native/gstreamer_runtime.py:97 | yes | |
| CIVICCAST_GSTREAMER_SOURCE_SEGMENT_SECONDS | 2 | civiccast/egress/envvars.py:15 | yes | |
| CIVICCAST_GST_ALLOW_HARDWARE_DECODE | 1 | civiccast/egress/gstdecode_policy.py:72 | yes | |
| CIVICCAST_GST_FIRST_OUTPUT_TIMEOUT_S | 2 | civiccast/egress/gstengine.py:345 | yes | |
| CIVICCAST_GST_PREROLL_TIMEOUT_S | 4 | civiccast/egress/gstengine.py:250 | yes | |
| CIVICCAST_HLS_RELAY_BASE_PORT | 1 | civiccast/egress/hlsrelay.py:342 | yes | |
| CIVICCAST_IA_ACCESS_KEY | 6 | civiccast/archive/internet_archive.py:43 | yes | |
| CIVICCAST_IA_ACCESS_VALUE | 1 | civiccast/publish/providers.py:28 | yes | |
| CIVICCAST_IA_COLLECTION | 2 | civiccast/archive/internet_archive.py:63 | yes | |
| CIVICCAST_IA_ITEM_PREFIX | 2 | civiccast/archive/internet_archive.py:64 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_IA_SECRET_KEY | 4 | civiccast/archive/internet_archive.py:44 | yes | |
| CIVICCAST_IDENTITY_MIGRATION | 1 | civiccast/native/identity_migration.py:20 | no | |
| CIVICCAST_INSTALLER_BOOTSTRAP_LOG | 1 | civiccast/installer/service.py:3499 | yes | |
| CIVICCAST_INSTALL_TRANSACTION | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:400 | no | |
| CIVICCAST_INTERNET_ARCHIVE_ACCESS_KEY | 5 | civiccast/dr/station_backup_runtime.py:50 | yes | |
| CIVICCAST_LAN_ONLY_STATION | 3 | civiccast/app.py:2187 | yes | |
| CIVICCAST_LIVE_CAPTION_ENGINE | 2 | civiccast/ai_models/runtime.py:167 | yes | |
| CIVICCAST_LIVE_HLS_ROOT | 9 | civiccast/egress/headend.py:67 | yes | |
| CIVICCAST_LIVE_MANIFEST_BASE_URL | 5 | civiccast/live/finalization_worker.py:123 | yes | |
| CIVICCAST_LIVE_SOURCE_PROBE_TIMEOUT_SECONDS | 4 | civiccast/live/sourceprobe.py:35 | yes | |
| CIVICCAST_LIVE_SOURCE_READINESS_TTL_SECONDS | 1 | civiccast/live/readiness.py:78 | yes | |
| CIVICCAST_LIVE_SURGE_BUFFER_SECONDS | 3 | civiccast/live/surge_service.py:137 | yes | |
| CIVICCAST_LIVE_SURGE_THRESHOLD | 7 | civiccast/app.py:2415 | yes | |
| CIVICCAST_LOCAL_MEDIA_BASE_URL | 8 | civiccast/cable/channel.py:40 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_MANAGED_STORAGE_DIR | 6 | civiccast/installer/service.py:206 | yes | |
| CIVICCAST_MEDIAMTX_COMMAND | 2 | civiccast/installer/contribution_install.py:150 | yes | |
| CIVICCAST_MEDIA_INTEGRITY_POLL_SECONDS | 2 | civiccast/schedule/media_integrity_worker.py:100 | yes | |
| CIVICCAST_MEDIA_INTEGRITY_WORKER | 2 | civiccast/schedule/media_integrity_worker.py:90 | yes | |
| CIVICCAST_MEDIA_LIFECYCLE_POLL_SECONDS | 2 | civiccast/schedule/media_lifecycle_worker.py:436 | yes | |
| CIVICCAST_MEDIA_LIFECYCLE_TRANSCODE_SEEDING_ENABLED | 2 | civiccast/schedule/media_lifecycle_worker.py:387 | yes | |
| CIVICCAST_MEDIA_LIFECYCLE_WORKER | 2 | civiccast/schedule/media_lifecycle_worker.py:426 | yes | |
| CIVICCAST_MEDIA_LIFECYCLE_WORKER_DRY_RUN | 3 | civiccast/app.py:1267 | yes | |
| CIVICCAST_MEDIA_STORAGE_BUDGET_BYTES | 2 | civiccast/schedule/media_lifecycle_router.py:929 | yes | |
| CIVICCAST_MISSING_MEDIA_HORIZON_DAYS | 2 | civiccast/schedule/media_lifecycle_worker.py:458 | yes | |
| CIVICCAST_MSVC_INSTALLATION_PATH | 5 | scripts/build_native_app_payload.py:1750 | yes | |
| CIVICCAST_MSVC_RUNTIME_DLL | 2 | scripts/build_native_app_payload.py:1747 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_NAS_ARCHIVE_PATH | 11 | civiccast/archive/local_nas.py:6 | yes | |
| CIVICCAST_NAS_TARGET | 1 | civiccast/publish/providers.py:32 | yes | |
| CIVICCAST_NATIVE_REPORTED_VERSION | 4 | civiccast/app.py:2204 | yes | |
| CIVICCAST_NATIVE_ROOT | 2 | scripts/verify_beta1_live_hls_media.py:359 | yes | |
| CIVICCAST_NATIVE_STATION | 14 | civiccast/app.py:940 | yes | |
| CIVICCAST_NATIVE_STATION_MANIFEST | 3 | civiccast/installer/service.py:6078 | yes | |
| CIVICCAST_NATIVE_STATION_ROOT | 4 | civiccast/ai_models/runtime.py:141 | yes | |
| CIVICCAST_NDI_FFMPEG | 6 | civiccast/egress/ndi_relay.py:8 | yes | |
| CIVICCAST_NDI_RELAY | 2 | civiccast/egress/ndi_relay.py:62 | yes | |
| CIVICCAST_NDI_RUNTIME_DIR | 1 | civiccast/installer/service.py:6296 | yes | |
| CIVICCAST_NDI_SENDER | 1 | civiccast/cable/ndi.py:119 | yes | |
| CIVICCAST_NETWORK_PROBE_TIMEOUT_SECONDS | 2 | civiccast/live/network_probe.py:57 | yes | |
| CIVICCAST_NETWORK_SETTINGS_PATH | 3 | civiccast/dr/station_backup_runtime.py:89 | | |
| CIVICCAST_NOTICE | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:290 | yes | |
| CIVICCAST_NOTIFICATION_DELIVERY_LEASE_SECONDS | 1 | civiccast/subscribe/outcome_store.py:90 | | |
| CIVICCAST_OBS_WEBSOCKET_PASSWORD | 1 | civiccast/dr/station_backup_runtime.py:75 | | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_OFFLINE_CAPTION_BACKOFF_SECONDS | 1 | civiccast/captions/vosk_job.py:410 | yes | |
| CIVICCAST_OFFLINE_CAPTION_CHUNK_SECONDS | 1 | civiccast/captions/vosk_job.py:416 | yes | |
| CIVICCAST_OFFLINE_CAPTION_JOB | 4 | civiccast/captions/settings.py:45 | yes | |
| CIVICCAST_OFFLINE_CAPTION_MAX_ATTEMPTS | 1 | civiccast/captions/vosk_job.py:413 | yes | |
| CIVICCAST_OFFLINE_CAPTION_POLL_SECONDS | 1 | civiccast/captions/vosk_job.py:407 | yes | |
| CIVICCAST_OFFLINE_CAPTION_SPANISH | 3 | civiccast/captions/settings.py:52 | yes | |
| CIVICCAST_OK | 7 | civiccast/apps/installer/src-tauri/src/main.rs:794 | yes | |
| CIVICCAST_OLLAMA_BASE_URL | 1 | civiccast/alerting/service_test.py:250 | yes | |
| CIVICCAST_OLLAMA_READY | 2 | scripts/run_real_ai_release_proof.py:145 | yes | |
| CIVICCAST_OLLAMA_REGISTRY_BASE_URL | 1 | civiccast/apps/installer/src-tauri/src/acquisition_catalog.rs:253 | yes | |
| CIVICCAST_OPERATOR_CONSOLE_DIST | 7 | civiccast/app.py:2788 | yes | |
| CIVICCAST_OPERATOR_CONSOLE_URL | 3 | civiccast/installer/service.py:6030 | yes | |
| CIVICCAST_PACK_PUBLIC_KEY_BASE64 | 2 | civiccast/apps/installer/src-tauri/src/native_packs.rs:244 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_PACK_SIGNING_KEY_ID | 2 | civiccast/apps/installer/src-tauri/src/native_packs.rs:242 | yes | |
| CIVICCAST_PAYLOAD_REPLACED | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:344 | yes | |
| CIVICCAST_PLAYBACK_POLICY_STATE_PATH | 2 | civiccast/playback_policy/store.py:160 | yes | |
| CIVICCAST_PLAYWRIGHT_EXECUTABLE | 3 | civiccast/apps/installer/playwright.config.ts:6 | yes | |
| CIVICCAST_PODCAST_MEDIA_DIR | 1 | civiccast/podcast/service.py:44 | yes | |
| CIVICCAST_PORTAL_BASE | 2 | civiccast/vod/routers.py:25 | yes | |
| CIVICCAST_PORTAL_TOKEN | 2 | civiccast/installer/service.py:623 | yes | |
| CIVICCAST_POSTCLEAR_ARMED | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:740 | yes | |
| CIVICCAST_PREFLIGHT_SCRIPT | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:746 | yes | |
| CIVICCAST_PREPARED_PLAN_DIR_BUDGET_GB | 1 | civiccast/egress/preparer.py:241 | yes | |
| CIVICCAST_PROGRAM_LOG_HORIZON_HOURS | 1 | civiccast/programlog/materializer.py:74 | yes | |
| CIVICCAST_PROGRAM_LOG_POLL_SECONDS | 1 | civiccast/programlog/materializer.py:73 | yes | |
| CIVICCAST_PROGRAM_LOG_WORKER | 3 | civiccast/programlog/materializer.py:15 | yes | |
| CIVICCAST_PROVIDER_CIVICCAST_PROVIDER_API_KEY | 6<br>2 | civiccast/agenda_import/config.py:8<br>civiccast/cli.py:964 | yes<br>yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_PROVIDER_CREDENTIALS_FILE | 1 | civiccast/installer/service.py:4704 | yes | |
| CIVICCAST_PROVIDER_INTERNET_ARCHIVE | 11 | civiccast/archive/internet_archive.py:6 | yes | |
| CIVICCAST_PROVIDER_LOCAL_NAS | 4 | civiccast/archive/local_nas.py:17 | yes | |
| CIVICCAST_PROVIDER_MAIL | 4 | civiccast/paywall/router.py:274 | yes | |
| CIVICCAST_PROVIDER_PROOFS_FILE | 1 | civiccast/installer/service.py:4711 | yes | |
| CIVICCAST_PROVIDER_WEBHOOK | 3 | civiccast/app.py:1287 | yes | |
| CIVICCAST_PROVIDER_YOUTUBE | 5 | civiccast/platform/providers.py:11 | yes | |
| CIVICCAST_PUBLIC_ANALYTICS_ALLOWED_ORIGINS | 4 | civiccast/app_platforms/router.py:569 | yes | |
| CIVICCAST_PUBLIC_ANALYTICS_KEY | 5 | civiccast/analytics/models.py:152 | yes | |
| CIVICCAST_PUBLIC_ANALYTICS_RATE_LIMIT_PER_MINUTE | 1 | civiccast/app_platforms/router.py:542 | yes | |
| CIVICCAST_PUBLIC_BASE_URL | 12 | civiccast/activitypub/config.py:59 | yes | |
| CIVICCAST_PUBLIC_PORTAL_DIST | 11 | civiccast/app.py:2777 | yes | |
| CIVICCAST_R2_ | 2 | civiccast/load/real_edge_ramp.py:24 | yes | |
| CIVICCAST_R2_ACCESS_KEY_ID | 6 | civiccast/dr/station_backup_runtime.py:58 | yes | |
| CIVICCAST_R2_ACCOUNT_ID | 6 | civiccast/dr/station_backup_runtime.py:57 | yes | |
| CIVICCAST_R2_BUCKET | 6 | civiccast/dr/station_backup_runtime.py:60 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_R2_PUBLIC_BASE_URL | 6 | civiccast/dr/station_backup_runtime.py:61 | yes | |
| CIVICCAST_R2_SECRET_ACCESS_KEY | 6 | civiccast/dr/station_backup_runtime.py:59 | yes | |
| CIVICCAST_REAL_BOUNDARY_TOKEN_FILE | 1 | civiccast/apps/portal-operator/e2e/real-boundary-smoke.spec.ts:160 | yes | |
| CIVICCAST_RECORDING_DRAIN_DEADLINE_SECONDS | 2 | civiccast/app.py:687 | yes | |
| CIVICCAST_RECORDING_INPUT_PRESETS_JSON | 1 | civiccast/recording/input_presets.py:19 | yes | |
| CIVICCAST_RELEASE_ENGINEERING | 1 | civiccast/installer/router.py:1023 | yes | |
| CIVICCAST_RELEASE_RUN_ID | 1 | scripts/verify-release.sh:11 | yes | |
| CIVICCAST_RELOAD_COMMIT_TIMEOUT_S | 2 | civiccast/egress/gstreamer/engine.py:800 | yes | |
| CIVICCAST_RELOAD_TIMEOUT_S | 1 | civiccast/egress/gstreamer/worker.py:724 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION | 8 | civiccast/app.py:1509 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION_HOME | 3 | civiccast/installer/contribution_install.py:53 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION_MEDIAMTX_URL | 4 | civiccast/app.py:3277 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_REMOTE_CONTRIBUTION_POLL | 2 | civiccast/live/contribution/coprocess.py:88 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION_RTSP_URL | 1 | civiccast/app.py:3278 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION_SRT_URL | 1 | civiccast/app.py:3282 | yes | |
| CIVICCAST_REMOTE_CONTRIBUTION_VDO_URL | 4 | civiccast/app.py:3304 | yes | |
| CIVICCAST_REPORTING_WORK_DIR | 2 | civiccast/reporting/serve.py:139 | yes | |
| CIVICCAST_REQUIRE_FCC_ACK | 9 | civiccast/app.py:3428 | yes | |
| CIVICCAST_RESIDENT_PORTAL_URL | 6 | civiccast/installer/service.py:209 | yes | |
| CIVICCAST_RETAINED_WHEEL_PATH | 11 | civiccast/native/apply_payload.py:65 | yes | |
| CIVICCAST_RETENTION_POLL_SECONDS | 2 | civiccast/schedule/retention_worker.py:114 | yes | |
| CIVICCAST_RETENTION_WORKER | 2 | civiccast/schedule/retention_worker.py:104 | yes | |
| CIVICCAST_ROLLBACK_ARTIFACT_PATH | 1 | civiccast/installer/service.py:1510 | yes | |
| CIVICCAST_ROOT_KEY | 5 | civiccast/apps/installer/src-tauri/src/native_service_registration.rs:191 | yes | |
| CIVICCAST_RUNTIME_ARTIFACT_MIRROR | 2 | scripts/provision_native_runtime_dependencies.py:43 | yes | |
| CIVICCAST_RUNTIME_BUILD_ID | 1 | civiccast/app.py:2610 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_RUN_POSTGRES_TESTS | 2 | civiccast/dr/report.py:22 | yes | |
| CIVICCAST_SCHEDULED_RECORDING | 2 | civiccast/recording/runtime.py:69 | yes | |
| CIVICCAST_SCHEDULED_RECORDING_CAPTURE_SUBDIR | 1 | civiccast/recording/runtime.py:79 | yes | |
| CIVICCAST_SCHEDULED_RECORDING_HORIZON_SECONDS | 1 | civiccast/recording/runtime.py:76 | yes | |
| CIVICCAST_SCHEDULED_RECORDING_POLL_SECONDS | 1 | civiccast/recording/runtime.py:74 | yes | |
| CIVICCAST_SCHEDULE_LOOP | 2 | civiccast/egress/source_plan.py:178 | yes | |
| CIVICCAST_SDI_FFMPEG | 3 | civiccast/egress/router.py:1092 | yes | |
| CIVICCAST_SDI_RELAY | 2 | civiccast/egress/sdi_relay.py:70 | yes | |
| CIVICCAST_SELF_HOSTED_RTX | 1 | scripts/run_real_ai_release_proof.py:73 | yes | |
| CIVICCAST_SETUP_NONCE | 1 | civiccast/installer/service.py:6018 | yes | |
| CIVICCAST_SMTP_FROM | 3 | civiccast/dr/station_backup_runtime.py:36 | yes | |
| CIVICCAST_SMTP_HOST | 3 | civiccast/dr/station_backup_runtime.py:35 | yes | |
| CIVICCAST_SMTP_PASSWORD | 3 | civiccast/dr/station_backup_runtime.py:39 | yes | |
| CIVICCAST_SMTP_PORT | 3 | civiccast/dr/station_backup_runtime.py:37 | yes | |
| CIVICCAST_SMTP_STARTTLS | 2 | civiccast/dr/station_backup_runtime.py:40 | yes | |
| CIVICCAST_SMTP_USERNAME | 3 | civiccast/dr/station_backup_runtime.py:38 | yes | |
| CIVICCAST_SOAK_BASE_URL | 3 | civiccast/load/live_soak.py:76 | yes | |
| CIVICCAST_SOAK_COMMIT | 2 | civiccast/load/live_soak.py:598 | yes | |
| CIVICCAST_SOAK_ROOT | 3 | civiccast/load/live_soak.py:75 | yes | |
| CIVICCAST_SOAK_THRESHOLD | 3 | civiccast/load/live_soak.py:77 | yes | |
| CIVICCAST_STAFF | 1 | civiccast/installer/wizard.py:20 | yes | |
| CIVICCAST_STAFF_TOKEN | 3 | scripts/generate-openapi-artifacts.py:319 | yes | |
| CIVICCAST_STAFF_TOKENS | 18 | civiccast/cli.py:2073 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_STAFF_TOKENS_FALLBACK_WITH_DB | 5 | civiccast/auth/tokens.py:171 | yes | |
| CIVICCAST_STALL_TIMEOUT_S | 7 | civiccast/egress/daemon.py:187 | yes | |
| CIVICCAST_STARTUP_HOOK_TIMEOUT_SECONDS | 1 | civiccast/app.py:670 | yes | |
| CIVICCAST_STATION_BACKUPS | 3 | civiccast/installer/station_state.py:422 | yes | |
| CIVICCAST_STATION_ID | 14 | civiccast/app.py:3447 | yes | |
| CIVICCAST_STATION_MEDIA_LIBRARY | 3 | civiccast/installer/station_state.py:421 | yes | |
| CIVICCAST_STATION_NAME | 2 | civiccast/installer/station_state.py:400 | yes | |
| CIVICCAST_STATION_RECORDINGS | 3 | civiccast/installer/station_state.py:422 | yes | |
| CIVICCAST_STATION_STATE_PATH | 11 | civiccast/dr/station_backup_runtime.py:107 | yes | |
| CIVICCAST_STATION_STORAGE_ROOT | 2 | civiccast/installer/station_state.py:1472 | yes | |
| CIVICCAST_STATION_TZ | 6 | civiccast/app.py:1032 | yes | |
| CIVICCAST_STEP | 110 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:105 | yes | |
| CIVICCAST_SUBSCRIBER_WEBHOOK_SECRET | 7 | civiccast/dr/station_backup_runtime.py:45 | yes | |
| CIVICCAST_SUBSCRIBE_ACCEPT_V08_LEGACY_SECRETS | 1 | civiccast/subscribe/secrets.py:171 | yes | |
| CIVICCAST_SUBSCRIBE_ENCRYPTION_KEY | 3 | civiccast/dr/station_backup_runtime.py:42 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_SUBSCRIBE_LEGACY_ENCRYPTION_KEYS | 2 | civiccast/dr/station_backup_runtime.py:44 | yes | |
| CIVICCAST_SUBSCRIBE_LEGACY_TOKEN_SECRETS | 2 | civiccast/dr/station_backup_runtime.py:43 | yes | |
| CIVICCAST_SUBSCRIBE_RATE_LIMIT | 1 | civiccast/subscribe/router.py:129 | yes | |
| CIVICCAST_SUBSCRIBE_RATE_LIMIT_WINDOW_SECONDS | 1 | civiccast/subscribe/router.py:130 | yes | |
| CIVICCAST_SUBSCRIBE_SECRETS_FILE | 3 | civiccast/dr/station_backup_runtime.py:88 | yes | |
| CIVICCAST_SUBSCRIBE_TOKEN_SECRET | 3 | civiccast/dr/station_backup_runtime.py:41 | yes | |
| CIVICCAST_SUMMARY_JOB | 2 | civiccast/summary/jobs.py:282 | | |
| CIVICCAST_SUMMARY_JOB_BACKOFF_SECONDS | 1 | civiccast/summary/jobs.py:295 | | |
| CIVICCAST_SUMMARY_JOB_MAX_ATTEMPTS | 1 | civiccast/summary/jobs.py:298 | | |
| CIVICCAST_SUMMARY_JOB_POLL_SECONDS | 1 | civiccast/summary/jobs.py:292 | | |
| CIVICCAST_SUPERVISED | 11 | civiccast/app.py:443 | yes | |
| CIVICCAST_SUPERVISED_ENV_VAR | 5 | civiccast/app.py:438 | yes | |
| CIVICCAST_SUPERVISOR_MODE | 5 | civiccast/app.py:719 | yes | |
| CIVICCAST_SUPERVISOR_MODE_CONTRACT | 7 | civiccast/app.py:620 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_SUPERVISOR_STOP_WATCHDOG_SECONDS | 1 | civiccast/native/supervisor/service.py:1070 | yes | |
| CIVICCAST_SUPPORT_BUNDLE_DIR | 2 | civiccast/installer/service.py:5136 | yes | |
| CIVICCAST_TEARDOWN_EXIT | 18 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:136 | yes | |
| CIVICCAST_TESTER_AVAILABLE_VERSION | 1 | civiccast/installer/service.py:1484 | yes | |
| CIVICCAST_TESTER_OPS_STATE_PATH | 4 | civiccast/installer/service.py:4652 | yes | |
| CIVICCAST_TRANSACTION_ARMED | 5 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:357 | no | |
| CIVICCAST_TRANSACTION_BEGIN | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:388 | no | |
| CIVICCAST_TRANSACTION_CALL | 6 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:362 | no | |
| CIVICCAST_TRANSACTION_CALLBACKS | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:431 | no | |
| CIVICCAST_TRANSACTION_COMMIT | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:419 | no | |
| CIVICCAST_TRANSACTION_RECOVER_PENDING | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:377 | no | |
| CIVICCAST_TRANSACTION_RESTORE | 4 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:410 | no | |
| CIVICCAST_TRANSACTION_RESULT | 8 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:358 | no | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_TRANSACTION_SCRIPT | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:359 | no | |
| CIVICCAST_TRANSACTION_SNAPSHOT | 2 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:403 | no | |
| CIVICCAST_TRANSCODE_FORMATS | 3 | civiccast/schedule/media_lifecycle_models.py:137 | yes | |
| CIVICCAST_TRANSCODE_OUTPUT_ROOT | 1 | civiccast/schedule/media_lifecycle_worker.py:474 | yes | |
| CIVICCAST_TRUSTED_ANALYTICS_RATE_LIMIT_PER_MINUTE | 1 | civiccast/app_platform/router.py:540 | yes | |
| CIVICCAST_TRUSTED_HOSTS | 5 | civiccast/subscriber/router.py:321 | no | |
| CIVICCAST_TRUSTED_PROXY_CIDRS | 5 | civiccast/common/trusted_proxy.py:22 | yes | |
| CIVICCAST_TRUSTED_PROXY_CIDRS_EXACT | 5 | civiccast/common/trusted_proxy.py:25 | no | |
| CIVICCAST_TRUST_PRIVATE_PROXIES | 5 | civiccast/common/trusted_proxy.py:36 | yes | |
| CIVICCAST_TSA_ENABLE | 3 | civiccast/dr/station_backup_runtime.py:78 | yes | |
| CIVICCAST_TSA_POLICY_OID | 2 | civiccast/dr/station_backup_runtime.py:77 | yes | |
| CIVICCAST_TSA_URL | 7 | civiccast/dr/station_backup_runtime.py:76 | yes | |
| CIVICCAST_TSDUCK_HOME | 3 | civiccast/egress/compliance.py:53 | yes | |
| CIVICCAST_TSDUCK_NETWORK_TESTS | 1 | civiccast/installer/tsduck_install.py:73 | yes | |
| CIVICCAST_TSDUCK_PATH | 18 | civiccast/egress/compliance.py:49 | yes | |
| CIVICCAST_TSDUCK_RCVBUF_BYTES | 1 | civiccast/egress/compliance.py:137 | yes | |
| CIVICCAST_TS_RELAY | 7 | civiccast/egress/ts_relay.py:28 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_TS_RELAY_BASE_PORT | 1 | civiccast/egress/ts_relay.py:51 | yes | |
| CIVICCAST_TURN_HOST | 5 | civiccast/installer/contribution_install.py:22 | yes | |
| CIVICCAST_TURN_PORT | 3 | civiccast/installer/contribution_install.py:22 | yes | |
| CIVICCAST_UPLOAD_DIR | 19 | civiccast/app.py:371 | yes | |
| CIVICCAST_UPLOAD_MAX_BYTES | 4 | civiccast/schedule/media_lifecycle_router.py:532 | yes | |
| CIVICCAST_UV_EXE | 4 | scripts/build_native_app_payload.py:960 | yes | |
| CIVICCAST_VDO_COMMAND | 2 | civiccast/installer/contribution_install.py:145 | yes | |
| CIVICCAST_VERAPDF_ARTIFACT_DIR | 1 | civiccast/records/pdfa.py:690 | yes | |
| CIVICCAST_VERSION | 43 | civiccast/_native_version.py:22 | yes | |
| CIVICCAST_VERSION_PREFLIGHT | 3 | civiccast/apps/installer/src-tauri/nsis-hooks-bootstrap.nsh:747 | no | |
| CIVICCAST_VOD_PACKAGE_DIR | 4 | civiccast/captions/vod_job.py:711 | yes | |
| CIVICCAST_WATCH_FOLDER_MAX_CONCURRENT_FOLDERS | 2 | civiccast/schedule/watch_folder_worker.py:250 | yes | |
| CIVICCAST_WATCH_FOLDER_POLL_SECONDS | 2 | civiccast/schedule/watch_folder_worker.py:240 | yes | |
| CIVICCAST_WATCH_FOLDER_WORKER | 2 | civiccast/schedule/watch_folder_worker.py:230 | yes | |
| CIVICCAST_WEBHOOK_RETRY_BACKOFF_SECONDS | 1 | civiccast/subscribers/retry_worker.py:81 | yes | |
| CIVICCAST_WEBHOOK_RETRY_MAX_ATTEMPTS | 1 | civiccast/subscribers/retry_worker.py:83 | yes | |
| CIVICCAST_WEBHOOK_RETRY_POLL_SECONDS | 1 | civiccast/subscribers/retry_worker.py:79 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| CIVICCAST_WEBHOOK_RETRY_WORKER | 3 | civiccast/subscribe/retry_worker.py:15 | yes | |
| CIVICCAST_WEBHOOK_SIGNING_VALUE | 1 | civiccast/publish/providers.py:31 | yes | |
| CIVICCAST_WEBHOOK_TIMEOUT_SECONDS | 3 | civiccast/subscribe/webhook.py:97 | yes | |
| CIVICCAST_WHISPER_BEAM_SIZE | 5 | civiccast/captions/runtime.py:374 | yes | |
| CIVICCAST_WHISPER_COMPUTE_TYPE | 6 | civiccast/captions/runtime.py:709 | yes | |
| CIVICCAST_WHISPER_CPP_EXE | 2 | civiccast/captions/tapes_worker.py:2805 | yes | |
| CIVICCAST_WHISPER_CPP_MODEL_PATH | 2 | civiccast/captions/tapes_worker.py:2806 | yes | |
| CIVICCAST_WHISPER_CPU_THREADS | 17 | civiccast/captions/runtime.py:118 | yes | |
| CIVICCAST_WHISPER_DEVICE | 9 | civiccast/captions/runtime.py:52 | yes | |
| CIVICCAST_WHISPER_FALLBACK_TIMEOUT_SECONDS | 1 | civiccast/captions/whistle.py:68 | yes | |
| CIVICCAST_WHISPER_MODEL_PATH | 6 | civiccast/ai_models/runtime.py:135 | yes | |
| CIVICCAST_WHISPER_NUM_WORKERS | 4 | civiccast/captions/runtime.py:775 | yes | |
| CIVICCAST_WHISTLE_CHANNELS | 1 | civiccast/ai_models/runtime.py:208 | yes | |
| CIVICCAST_WHISTLE_ROOT | 2 | civiccast/ai_models/runtime.py:173 | yes | |

| Name | Prod refs | First location | Documented | Notes |
|---|---|---|---|---|
| `CIVICCAST_YOUTUBE_CLIENT_ID` | 6 | civiccast/dr/station_backup_runtime.py:53 | yes | |
| `CIVICCAST_YOUTUBE_CLIENT_SECRET` | 9 | civiccast/dr/station_backup_runtime.py:54 | yes | |
| `CIVICCAST_YOUTUBE_MEDIA_ROOT` | 3 | civiccast/syndicate/youtube.py:15 | yes | |
| `CIVICCAST_YOUTUBE_PRIVACY` | 3 | civiccast/dr/station_backup_runtime.py:56 | yes | |
| `CIVICCAST_YOUTUBE_REFRESH_TOKEN` | 5 | civiccast/dr/station_backup_runtime.py:55 | yes | |
| `CIVICCAST_YOUTUBE_REFRESH_VALUE` | 1 | civiccast/publish/providers.py:29 | yes | |

## Appendix D: Files, folders, ports and services

Where CivicCast puts things on a Windows station, what runs, and what listens on the network. All paths assume the defaults: install folder `C:\Program Files\CivicCast (Native)` (called `<I>` below) and data folder `C:\ProgramData\CivicCast`. The data folder follows the Windows `PROGRAMDATA` setting if your machine moves it.

### Folders created by setup

**The install folder `<I>`** Everything here is program files. Changing it needs Administrator rights, and uninstalling deletes it.

| Path | What is in it |
|---|---|
| `<I>\CivicCast Native.exe` | The setup and first-run program. |
| `<I>\uninstall.exe` | The uninstaller. |
| `<I>\vc_redist.x64.exe` | The Microsoft Visual C++ runtime installer that setup runs first. |
| `<I>\runtime\` | CivicCast's own copy of Python 3.12, the CivicCast program, and both web portals. `runtime\python.exe` runs the program; `runtime\pythonservice.exe` hosts the Windows service. |
| `<I>\runtime\Lib\site-packages\bin\civiccast.exe` | The command-line program (Appendix A). |

| Path | What is in it |
|---|---|
| `<I>\packs\native-server-binaries\payload\` | PostgreSQL programs, and `tsduck\bin\tsp.exe` when TSDuck is shipped. |
| `<I>\packs\.station-cache\` | The cache of the large AI-model packs that setup unpacked from. About 21 GB, according to setup's own text. |
| `<I>\packs\captions-floor\` | The standard (Medium) caption model. |
| `<I>\packs\captions-whistle\` | The signed Whistle model and CPU engine for native live captions. |
| `<I>\dependencies\ffmpeg\bin\` | `ffmpeg.exe` and `ffprobe.exe`. |
| `<I>\dependencies\ollama\` | The Ollama AI engine (version 0.30.6). |
| `<I>\dependencies\cuda\` | Optional graphics-card libraries. |
| `<I>\components\<name>\` | The AI models: `summary-gemma4-12b`, `summary-gemma4-e4b`, `translation-translategemma-4b`, and the optional `captions-large-v3`. |
| `<I>\models\ollama\` | The merged AI model store that the Ollama engine reads. |
| `<I>\station\` | The signed station index and `core.ccpack`. |
| `<I>\station-set.json`, | Proof that activation and the self-test passed. |
| `<I>\activation-self-test.json` | The service will not start without them. |

**The data folder `C:\ProgramData\CivicCast`** This is everything that belongs to your station. **Uninstalling does not delete it** (no step in the uninstall code removes it).

| Path | What is in it |
|---|---|
| `data\pgdata\` | The PostgreSQL database cluster: all station records. Accessible to SYSTEM, administrators and the restricted station service identity. |
| `profile\` | The restricted service account's CivicCast profile, including `station-state.json` and related station files unless explicitly relocated. |
| `install-transaction\` | Automatic installer recovery snapshots and recovery records. Preserve it when recovery has not completed. |
| `data\uploads\` | Media uploaded by operators, and recordings that finish into the library (`CIVICCAST_UPLOAD_DIR`). |
| `data\egress\` | The playout engine's working folder: the plans, the prepared copies of programs, and the HLS web output. |

| Path | What is in it |
|---|---|
| `data\egress\conform-cache\` | The conform cache: prepared, ready-to-play copies of long programs. Limited to `CIVICCAST_CONFORM_CACHE_GB`, default 60 GB. |
| `data\caption-tap\` | Working folder for live captions. |
| `packs\` | Components the first-run wizard downloaded (same layout as `<I>\packs`). |
| `components\captions-large-v3\` | The optional Large caption model, if downloaded. |
| `logs\` | The logs (next section). |
| `install-progress.log` | Time-stamped notes from setup. First place to look when setup fails. |
| `install-manifest-report-<number>-<time>.json` | A report of which packs setup staged, one file per run. |
| `upgrade\` | `upgrade-engine.log`, `upgrade-journal.json` and `UPGRADE-RECOVERY.md` from the last upgrade. |
| `provision\` | `PROVISION-RECOVERY.md`, `OWNERSHIP-RECOVERY.md`, `ownership-observation.txt` and the database set-up journal. |

> **For IT staff:** Setup and Station Profile show **Installation**, **Database** and **Media uploads** from the installed service's resolved configuration. Native first-run storage defaults are under `C:\ProgramData\CivicCast\data`; existing saved storage choices remain explicit station settings. Use the displayed runtime paths when locating the actual files. Include `station-state.json` in backups as well: `CIVICCAST_STATION_STATE_PATH` can relocate it. The restricted native service uses `C:\ProgramData\CivicCast\profile`; other runtime accounts can use their `%LOCALAPPDATA%\CivicCast\` fallback.

**Per-user files from the first-run wizard**

| Path | What |
|---|---|
| `%USERPROFILE%\.civiccast\installer-state.json` | The wizard's saved progress. |
| `%USERPROFILE%\.civiccast\runtime-host.log` | The wizard's log. |

**Registry**

| Key | What |
|---|---|
| `HKLM\SOFTWARE\CivicCast\Native` | `DatabaseUrl` (readable by SYSTEM, Administrators and the restricted station service identity; it holds the database password) and `InstalledVersion`. |
| `HKLM\SOFTWARE\CivicCast` | `ActiveRuntime` (which CivicCast edition owns the machine), `Maintenance`, `NativeUninstallPostclearPending`. |
| `HKLM\SYSTEM\CurrentControlSet\Services\CivicCastSupervisor` | The Windows service. Its `Environment` value is where the product's code comments say service settings go; we found no installer code that creates it (Appendix C). |

**Shortcuts** Start menu: **CivicCast Operator Console** and **CivicCast Public Portal** (internet shortcuts to `http://127.0.0.1:8000/operator/` and `http://127.0.0.1:8000/`). Desktop: **CivicCast Operator Console**. If the service is stopped, these open a browser error page.

#### Logs

All service logs are in `C:\ProgramData\CivicCast\logs`.

| File | What it records |
|---|---|
| `supervisor.log` | The Windows service itself: starts, stops, restarts of the child programs, watchdog messages. Rotates at 10 MiB, keeps 10 old files, written to disk on every line. |
| `control_plane-app.log` | The CivicCast program's own log: the one to read first when something on a screen fails. |
| `control_plane.log` | Raw console output of the control plane. Rotates at 10 MiB and keeps 10 older files. |
| `postgres.log` | PostgreSQL startup and server output. Rotates at 10 MiB and keeps 10 older files. |
| `ollama.log` | The AI engine, when it is running (by the same naming rule: one log per child program). |

The Windows Event Log also receives service messages under the source `CivicCastSupervisor`. Setup problems are in `install-progress.log` (read it from the bottom up; find the last "begin" that has no matching "returned").

#### The Windows service

| Item | Value |
|---|---|
| Name | `CivicCastSupervisor` (display name "CivicCast Native Supervisor") |
| Starts | Automatically with Windows. |
| Runs as | The restricted virtual account `NT SERVICE\CivicCastSupervisor`. |
| If it crashes | Windows restarts it after 5 seconds, then 10, then 30. |
| What it launches | PostgreSQL, then the control plane (the web server and playout engine), and the Ollama AI engine when its files are present. They start in that order and stop in reverse; each gets 15 seconds to stop politely before it is ended. |
| Restarting a child | After a child program fails, the service retries with a delay that grows from 1 second to 30 seconds. Five restarts within ten minutes put the station in the `degraded` state (the service stays up). |
| Only one copy | A system-wide lock named `Global\CivicCastSupervisorSingleton` stops a second copy. |
| Control channel | A Windows named pipe, `\\.\pipe\civiccast-supervisor`. Not a network port. |

Control it from an Administrator PowerShell: `Get-Service CivicCastSupervisor`, `Restart-Service CivicCastSupervisor`, `Stop-Service CivicCastSupervisor`.

#### Ports

The "Address" column is the network address the program listens on. `127.0.0.1` and `::1` mean only programs on the same computer can connect.

| Port | Protocol | Who listens | Address | What it is for |
|---|---|---|---|---|
| 8000 | TCP | The control plane (`uvicorn civiccast.app:create_app`) | Configured `127.0.0.1` or `::1` | The operator console (`/operator/`), the resident portal (`/`) and the whole API; remote HTTPS access goes through Caddy. |

| Port | Protocol | Who listens | Address | What it is for |
|---|---|---|---|---|
| 5432, or the first free of 5433, 5434, 5435, 5544 | TCP | PostgreSQL | `127.0.0.1` | The database. Setup picks the first free port in that order and records it in the database address; the service reads the port from there. |
| 11434 | TCP | The Ollama AI engine | `127.0.0.1` | The local AI engine used for meeting summaries and Spanish translation. |
| 38474 | TCP | `CivicCast Native.exe` | `127.0.0.1` | A "one copy at a time" lock for the setup program. Not for any other use. |
| 18000 to 18499 | UDP | `ffmpeg` (one per web-video output) | `127.0.0.1` | The internal relay that turns the playout engine's stream into HLS web video. The start can be moved with `CIVICCAST_HLS_RELAY_BASE_PORT`; each output uses one port in a 500-port range. |

| Port | Protocol | Who listens | Address | What it is for |
|---|---|---|---|---|
| 17800 and up | UDP | `tsp.exe` (TSDuck), one per channel and cable destination | `127.0.0.1` | The cable relay, used only when a cable (`udp-ts`) output exists and TSDuck is present. Its outgoing source ports sit 1,000 higher (18800 and up). Start moved with `CIVICCAST_TS_RELAY_BASE_PORT`. |
| none | Named pipe | The supervisor | local | The service's control channel (above). |

There is **no** CivicCast listener on any non-loopback address in the supervisor's normal configuration.

> **For IT staff:** A firewall rule does not make the loopback-bound application remotely reachable. Use **Network access** to choose IPv4 or IPv6 loopback and generate the Caddy configuration. IT must deploy Caddy, valid certificates and the intended public/staff network access. Test both hostnames from their intended networks; first-admin setup and recovery remain local-only.

**Connections the station makes outward**

| Where to | Port | When |
|---|---|---|
| `1.1.1.1` and `8.8.8.8` | TCP 443 | The Live screen's "Network reachable" pre-flight check connects to these to prove the station has internet. A station with no internet cannot pass that check. |
| Your cable headend or encoder | the address and port you set | A `udp-ts` cable output, and the commissioning test. |
| Streaming destinations | the address you set | SRT, RTMP, HLS and NDI outputs you add on Channels. |
| Alert destinations | email, text-message or webhook address you set | Alerts, when destinations are connected. |

| Where to | Port | When |
|---|---|---|
| Hosted AI services | HTTPS | Only if you switch an AI job to a hosted model on AI Models. The Ollama engine itself is started with cloud use switched off. |

Remote Contribution's install action fetches pinned VDO.Ninja source and a verified Windows MediaMTX relay. IT still commissions the VDO HTTPS service, public WHIP route and a reachable TURN server; those services' public ports depend on the station network.

## Appendix E: Status and error glossary

Every status word an operator is likely to see, with what it means. Words are written exactly as the console prints them. Where the console prints a raw code instead of a plain word, the code is given. Some words are in two vocabularies for the same thing; the tables say so.

### Readiness phrases

Five phrases are used across the console for "can I act?". Colors: green, yellow (amber), red.

| Phrase | Meaning |
|---|---|
| Ready | The required checks passed for what you are about to do. (green) |
| Check before meeting | Something optional or recoverable needs attention. (yellow) |
| Do not broadcast yet | A required check failed for tonight's broadcast. (red) |
| Not set up yet | An optional provider or feature has no credential or proof yet. (yellow) |
| Needs IT help | The next step needs administrator, command-line, certificate, database or service work. (red) |

On Readiness the server's own card heading can also read "Ready with optional items", which is not one of the five. Station Profile shows the raw words `green`, `yellow` and `red`. Readiness check rows are tagged `required`, `optional` or `advanced`.

### Assets (Assets screen)

The list has two columns that describe the same file in two vocabularies.

| State column | Status column | Meaning |
|---|---|---|
| Analyzing (`pending_ingest`) | Validating | CivicCast is checking the uploaded file. Usually under a minute. |
| Ingesting | Ingesting | The file is being processed for ingest. |
| Validated | Packaged / Not packaged yet | The file passed the check. "Not packaged yet": use **Package for playback** to make it streamable. "Packaged": playable and ready to publish or schedule. |
| Rejected | Rejected | The file failed validation. Open the asset for the reason. |
| Recorded | Not servable yet | A live recording finished but has no public playback copy yet, so residents cannot stream it. |
| (any) | Published | Live on the resident portal. |
| (any) | Transcoding / Queued for transcode | A playback copy is being made or is waiting. The asset stays usable. |
| (any) | Missing file | CivicCast cannot find the file on disk. The console has no "relink" control; use **Replace source file** in the asset detail page. |

#### Publishing (Publish screen)

| Pill on an asset | Meaning |
|---|---|
| Draft | The portal step has not finished: it has not run yet, or it is still running. |
| Preflight blocked | A required check stops publishing. |
| Publishing | The fallback label returned when no earlier dashboard-state rule applies. |
| Archive pending | The portal copy is live; a required archive copy is still pending or running. |
| Archive verified | Portal live and every required archive copy is verified. |
| Reaching fewer places than planned | Portal live, but an optional reach surface failed. |

| Pill on an asset | Meaning |
|---|---|
| Needs action | A required surface failed. |
| Complete | Everything required is done. |

Each surface has its own state: Blocked, Not set up yet, Coming soon (shown as "not built yet"), Not run yet, Running, Succeeded, Failed, Overridden. Its approval is `pending`, `approved` or `overridden`. A badge marks a surface completed by a simulated (mock) provider rather than a real write. Selectors default to `mock`, which writes or sends nothing.

| Surface | What it does now |
|---|---|
| Portal | The only default-selected surface; it makes the record public when its manifest exists. |
| Internet Archive, Local NAS | Optional for other assets; public-record archive rows are required for archive completion but are not default-selected. A real provider uploads the full recording only when its local source file resolves; otherwise it receives an asset/title payload. Local NAS writes filesystem copies, not rsync or ZFS. |
| YouTube VOD | Mock by default. The real adapter uploads the recording when its local source path resolves; otherwise the configured media-root path must exist. |
| YouTube Live | Mock by default. The real adapter creates and binds a YouTube broadcast and stream, but CivicCast does not connect its live output to the returned RTMP ingest address. |
| Cable package | Optional; `not_configured` when unset. |
| Podcast | Optional Publish surface. Produces a local MP3 from a local recording, persists the episode, and exposes its enclosure in the channel RSS feed; failures can be retried. |
| Subscriber notifications | Optional, selected publish surface with delivery receipts and retries; requires confirmed subscribers and configured providers. Signup confirmation and `/api/staff/subscribe/dispatch-test` are separate. |
| ActivityPub | Off by default; when enabled, a note is delivered only after the portal surface succeeds. |

**Provider setup note: Setup > Provider setup** saves fields to `provider-credentials.json`, while the Publish registry reads `CIVICCAST_PROVIDER_*` selectors and adapter credentials from the service environment. Saving that file alone does not enable real delivery. Keep public webhooks set to mock: the resident-supplied URL and identifier-derived secret make real webhook selection unsafe.

**Captions**

| Where | Words | Meaning |
|---|---|---|
| Caption review queue, each line | Pending, Approved, Edited, Rejected | Nothing is attached to a recording's public video until every line is approved, edited or rejected. Lines are marked EN (English transcription) or ES (the Spanish translation pass). |
| Recording's caption job (Assets) | Transcribing…, Awaiting review, Complete, Failed | `pending`, `awaiting_review`, `complete`, `failed`. "Awaiting review" means the cues are in the review queue. |
| Channel card, Captions row | On / Off (switched off in the station profile) / Not verified; open channel caption proof | Whether captions in the live output have been proven by decoding them back. |
| Channel caption card | Captions off, Not verified, Captions on, Caption proof failed, Stale (older than 120 seconds) | The result of the decode-back proof. |
| Live caption runtime (raw words in status data) | within-capacity, delayed, overloaded, storage-refused, paused, disabled | The live caption worker's state. "delayed" means temporarily behind; "overloaded" means sustained backlog. |

**Summaries (Summary review screen)**

| Word | Meaning |
|---|---|
| Pending review | A draft waiting for a records clerk. It can be approved only if it has sourced claims. |
| Approved | A clerk approved it; a signed PDF/A-3B record can be exported. |
| Rejected | A clerk rejected it. |
| Needs evidence (`refused`) | The system could not support the summary with evidence from the captions. |

The generation job itself is `pending`, `running`, `complete` or `failed` in the API.

**Meetings, sessions and recordings**

| Where | Words |
|---|---|
| Live session | Idle, Pre-flight, On air, Ending, Recorded |
| Live source check | Delivering (media seen within the freshness time), Needs re-check (seen, but too long ago), Not answering (last check failed), Not checked |
| Live pre-flight rows | Network reachable, Recording storage, Live source, Recording target and Operator confirmation must pass before Start Live Stream works. AI runtime may be "not configured" without blocking, but a check that fails (probed and broken) does block. Syndication, Internet Archive and NAS handoff only report and never block going on air. |
| Remote relay | Not configured, Ready, Degraded, Offline; modes Local encoder, Cloud relay, Direct platform |
| Scheduled recording job | `scheduled`, `arming`, `recording`, `finalizing`, `done`, `failed`, `skipped` |
| Schedule screen item | Scheduled (created, not yet approved to air; a premiere also says "Not yet visible to residents"), Published (approved to air; "Visible to residents"), Cancelled (withdrawn). Premiere and Embargo are the two kinds. |
| Program Guide airing | `scheduled`, `skipped_conflict`, `skipped_asset`, `cancelled` (shown as Skipped (conflict), Skipped (media), Cancelled) |
| Private rehearsal result | Passed, Failed, Not run |
| Recording finalization | Waiting, running ("Attempt N of M"), completed ("Recording saved as asset …"), failed |

**Channels and the outgoing feed**

| Word | Meaning |
|---|---|
| Stopped | The feed is not running. |
| Starting | Preparing the first program. "Preparing source" in the process row; no process number yet. |
| On air | The feed is running a program. |

| Word | Meaning |
|---|---|
| Changing source | The feed is switching between programs or sources. |
| Showing slate | The feed is airing its fallback card because nothing is ready. |
| Finishing current item | A “finish, then stop” command is waiting for the current program to end. |
| Stopping | A stop is under way. |
| Needs attention | The feed reported an error (red). |

Raw API words: `STOPPED`, `STARTING`, `ON_AIR`, `TRANSITIONING`, `FALLBACK_SLATE`, `DRAINING`, `STOPPING`, `ERROR`. A stop or restart can take up to about 30 seconds to show on the page.

| Commit to air | Meaning |
|---|---|
| Preparing | The commit was accepted and is being prepared. |
| Queued to air | A start or reload was queued for the program’s scheduled time. |
| On air (confirmed) | A commit-to-air state is not shown with this exact label in the current console; use the channel’s current feed state and caption/output proof instead. |
| Couldn’t reach the engine | Dispatch failed. |
| Rolled back | An operator rolled the commit back. |
| Not ready to air / Committed / Ready to review | Badges on upcoming programs in the commit list. |

**Contributor submissions, alerts, emergency alerts and federation**

| Where | Words |
|---|---|
| Contributor submission | `submitted`, `under_review`, `needs_changes`, `accepted`, `declined`, `scheduled`, `published`. The media gate: `not_run`, `passed`, `failed`, `override_accepted`. |
| Alert delivery | Delivered (also sent, success), failed, Undeliverable (`dead_letter`), pending, queued |
| Emergency (CAP) alert severity | unknown, minor, moderate, severe, extreme |
| Emergency alert status | active, superseded, expired, cancelled |
| Emergency display mode and state | crawl, overlay, forced slate; pending, displayed, cleared, expired |

| Where | Words |
|---|---|
| Federation follower | pending, accepted, blocked, rejected, removed |
| Federation delivery retry | pending, delivered, `dead_letter` |

#### The station service (the supervisor's own state names)

These are the state names inside the Windows service. They are not what `GET /health` reports: that is a separate `healthy` or `degraded` answer about database and live-caption readiness. Read `schema` and the coarse `live_captions` field; authenticated System Health shows channel details. We could not confirm which of these names `supervisor.log` prints.

| State | Meaning |
|---|---|
| starting | The service is bringing up PostgreSQL and the control plane. |
| ready | Both are up. |
| degraded | Up, but five restarts happened in ten minutes. |
| maintenance | The installer or a migration is holding the station read-only (an upgrade, or a hand-over between CivicCast editions). Changes are refused with HTTP 503 until the hold is released. |
| blocked_wsl_active, blocked_probe_unavailable | The guard that protects against two CivicCast editions owning one machine refused to start the station. |
| stopping | A stop is under way. |

Child programs (PostgreSQL, control plane): pending, stopped, starting, ready, stopping, failed. `GET /health` returns `status` healthy or degraded and `schema` current, behind, not-configured or unknown.

#### Setup program exit codes

When setup fails it ends with a number. Interactive setup shows a message box (not the number); a silent install (`/S`) shows only the number. Follow the row for stale upgrade journals and keep recovery records when restoration has not finished; nothing in these steps intentionally deletes station data.

| Code | Step | What it means | What to do |
|---|---|---|---|
| 82 | uninstall | The service stop could not be confirmed. | Stop the service in services.msc or reboot, then uninstall again. |

| Code | Step | What it means | What to do |
|---|---|---|---|
| 110 | stage packs | A required component pack is missing or not trusted, or the optional GPU pack is not trusted. | Put the named `.ccpack` files in the `packs` folder beside setup.exe and run again. |
| 111, 112, 121, 122 | verify | The unpacked server (111), app (112), ffmpeg (121) or Ollama (122) files do not match their signature. | Replace your copy of the kit and run again. |
| 113 | upgrade | The engine could not restore its database backup; setup also attempts full pre-setup restoration. | Read `install-progress.log` and any failed-run diagnostics under `install-transaction\failure-diagnostics`. `UPGRADE-RECOVERY.md` may not remain under `upgrade` after successful outer restoration. Keep recovery records if restoration did not finish. |
| 114 | upgrade | This release has a database change that cannot be rolled back; the automatic upgrade was refused. | No manual steps are given by setup. Ask the project. |
| 115 | upgrade | An unexpected fault. | Read the log. |
| 116 | database | PostgreSQL could not be provisioned. | Read the log and `provision\PROVISION-RECOVERY.md`. |
| 117 | database | An unexpected fault. | Read the log. |
| 118 | service | The Windows service could not be registered. | Read the log. |

| Code | Step | What it means | What to do |
|---|---|---|---|
| 119 | firewall | The firewall rule could not be created. | Read the log. |
| 120 | before install | The existing service could not be proved stopped, or its state could not be read. | Fix it and retry as Administrator. |
| 123 | activation | The station could not be activated or its self-test failed (see the next table). | See below. |
| 124 | upgrade | The engine rolled back its database work; setup attempts to restore the prior application and station snapshot. | Read `install-progress.log` and, when present, the failed-run engine log and journal under `install-transaction\failure-diagnostics`. |
| 125 | service | The service started but is not serving (health check failed). | Read `install-progress.log` and `upgrade-engine.log`. |
| 126 | service | The service would not start. | Read the `logs` folder and the Windows Application event log. |
| 127 | database | Setup cannot tell which CivicCast edition owns this machine. | If there is no WSL edition, set `HKLM\SOFTWARE\CivicCast\ActiveRuntime` to `native`; see `OWNERSHIP-RECOVERY.md`. |

| Code | Step | What it means | What to do |
|---|---|---|---|
| 128 | upgrade | A prior terminal upgrade-failure journal remains; when restoration succeeds, setup returns the pre-setup state and preserves that earlier journal. | Inspect and archive or move `upgrade\upgrade-journal.json` aside (do not delete it), then run setup again. |
| 129 | upgrade | An older setup was refused by the read-only version guard, normally before the service is stopped or files are replaced. A defensive late check also attempts snapshot restoration. | Use the setup matching the installed version. If the late recovery did not finish, keep the recovery records and station data. |
| 130 | uninstall | You declined the ownership-transfer prompt. | None. |
| 131 | uninstall | The ownership transfer failed. | Read the detail in the message. |
| 132 | uninstall | Blocked (active edition conflict, or state unreadable). | As the message says. |
| 133 | uninstall | The service stop was not confirmed; files were kept. | Stop the service or reboot, then uninstall again. |
| 134 | uninstall | Teardown incomplete. | Remove the leftovers by hand (services.msc, firewall, registry). |
| 135 | database | The older WSL edition of CivicCast is registered on this machine. | Uninstall it or run its `cutover-to-native`, then run setup again. |

| Code | Step | What it means | What to do |
|---|---|---|---|
| 136 | installer recovery | Setup could not prepare, complete or resume automatic restoration. | Keep station data and `install-transaction` intact. Read `install-progress.log` and `install-transaction\recovery-error.txt`; restart Windows or retry setup so recovery can retry, then contact support if it repeats. |

Code 123 hides a second number from the activation step: 66 (pack or index missing or not trusted), 67 (anything wrong after the packs were verified, including too little free disk space, an extraction failure, a missing file or a self-test that did not pass), 78 (the embedded signing key was refused: download setup again), 64 or 65 (a defective setup program). The self-test runs a short check of the program, PostgreSQL, ffmpeg, Ollama, a caption transcription and three AI requests; each AI request may take up to 300 seconds on a slow machine.

> **Known issue (beta.11):** The activation message tells you to find the failed self-test in `install-progress.log`. The log records only "returned 67"; the cause is in the setup program's detail pane. Run setup interactively and read the detail list if you need the cause.

#### Common error messages

| Where | Message | Cause and fix |
|---|---|---|
| API, HTTP 401 | Missing Authorization header. Use Bearer <staff-token>. | No token sent. Staff sign in at `/login`; the first administrator signs in at `/setup`. |
| API, HTTP 401 | Invalid Authorization header. Use Bearer <staff-token>. | The header is not in the form `Bearer <token>`. |
| API, HTTP 401 | Invalid staff bearer token. | The token is wrong or from another station. |
| API, HTTP 401 | Staff bearer token has been revoked. | Issue a new one. |
| API, HTTP 401 | Staff identity is required for this action. | The route needs an identity and none was attached. |

| Where | Message | Cause and fix |
|---|---|---|
| API, HTTP 403 | This action requires one of these CivicCast roles: … | Wrong role. The names are the raw role ids in alphabetical order, for example `meeting_operator, setup_admin`. See Appendix F. |
| API, HTTP 429 | Too many failed staff authentication attempts. Wait and retry. | Ten failed token checks in 60 seconds; wait for `Retry-After`. |
| Sign-in | Invalid admin username or password. | Use the username and password from the recovery kit, or a recovery code. |
| Sign-in, HTTP 429 | Too many sign-in attempts from this station. Wait N seconds… | Wait, or use a printed recovery code. |
| Setup, HTTP 403 | First setup can only be done from the station computer itself. | Open the console in a browser on the station, not from another computer or a remote viewer's own computer. |
| API, HTTP 503 | Durable storage is not ready yet. (Some areas, such as Publish, CG Board and Contributors, say instead: Durable storage is not ready. Open Setup and choose Prepare storage, or set DATABASE_URL for a technical deployment.) | The database is not ready. Check the service and `postgres.log`. |
| API, HTTP 503 | `{"error": "maintenance"}` | The station is in maintenance mode, normally because an upgrade is running. Reads still work; changes resume when the hold is released. |
| Live | LiveSession already exists: council-live-room | The Live screen uses one fixed session id, so a second session cannot be created until IT resets it. |

| Where | Message | Cause and fix |
|---|---|---|
| Live | Go on air blocked: a fresh source-bound server-side pre-flight did not pass. No broadcast was started. Correct the failed checks and run pre-flight again. | Fix the red pre-flight items. |
| Remote Contribution | Remote contribution is not configured (no self-hosted VDO.Ninja URL). VDO.Ninja, a MediaMTX relay, and reachable TURN service must be commissioned before guests can join. | Those services are not set up. |
| Remote Contribution | A guest-control notice says **sent, not verified**. | The director iframe does not report command completion. Check the guest in the director and mark them left only after verifying they disconnected. |
| Emergency Alerts | A forced full-screen slate must be confirmed by an operator… | Tick the confirmation box before choosing a forced slate. |
| Control Room | On-Air Mode expired before this cue could fire. Open a new On-Air session to continue. | The On-Air session lasts 30 minutes; open a new one. |
| Agendas | Another agenda item already occupies (agenda_id=…, order=0) | Two items cannot share an Order number; use the next number. |
| Command line | DATABASE_URL must point at the CivicCast database before running staff token lifecycle commands. | Set DATABASE_URL in that PowerShell window (Appendix A). |
| Windows Event Log | DATABASE_URL is unset or empty, and the installer-persisted registry value … is also missing or empty | The station has no database address. Repair the installation. |

## Appendix F: Roles and permissions matrix

CivicCast has five roles. A role is a set of things a person is allowed to do. This appendix says which role may open which screen and which role may do which task. It is built from the console's own menu rules and from the server's role checks.

> **Note:** The menu hides screens a role cannot use, but that is only a convenience. The server checks the role again on every action. A person who types the address of a hidden screen is stopped by the screen itself or by the server.

#### The five roles

| Role (as the console labels it) | Id used in messages and tokens | In plain words |
|---|---|---|
| Setup admin | `setup_admin` | Sets the station up: the first admin and recovery kit, live sources, AI models, provider keys, alerts, paywall, channel and cable settings, and has write access to most configuration. |
| Meeting operator | `meeting_operator` | Runs a meeting: the Live room, feed Start/Stop/Restart, the Control Room, Remote Contribution, scheduled recording, the Program Guide, agendas. |
| Records clerk | `records_clerk` | Looks after the record: reviews captions line by line, approves summaries and exports signed records, edits agendas, sets legal holds. |
| Publish operator | `publish_operator` | Decides what residents see: schedules, packages and publishes recordings, the community board, EPG export, apps, and reviews contributor submissions. |
| Support admin | `support_admin` | Helps keep the station healthy: restore drills, update and rollback tools, support bundles, reports, read-only views of alerts and many screens. |

#### How a person gets a role

1. **The first admin gets all five.** The account created on First Setup, and anyone who signs in with that admin's password or a recovery code, gets the scope `admin`, which means all five roles.
2. **Tokens for tools get what you give them.** `civiccast token issue --scopes ...` makes a token; with no `--scopes` it gets `operator`, which is also all five. Name roles (for example -

`-scopes records_clerk`) to narrow it (Appendix A).

3. **Setup admins manage staff accounts and roles.** The **Staff accounts** screen creates staff accounts, assigns their roles, disables access and resets passwords. The first administrator remains the recovery account with all five roles. Each signed-in account sees menu entries allowed by its roles, as summarized below.
4. **Roles add up.** One matching role is enough.
5. **No scopes means no roles.** A token with no scopes is refused everywhere that checks a role.

#### Who can open each screen

Y means the menu shows the entry. - means it is hidden. S = Setup admin, M = Meeting operator, R = Records clerk, P = Publish operator, A = Support admin. While the console has not yet loaded who you are, role-limited entries stay hidden.

| Menu section | Screen | S | M | R | P | A |
|---|---|---|---|---|---|---|
| Help | Manual | Y | Y | Y | Y | Y |
| Setup | First Setup | Y | Y | Y | Y | Y |
| Setup | Staff accounts | Y | - | - | - | - |
| Setup | Backup and restore | Y | - | - | - | Y |
| Setup | Import station | Y | - | - | - | - |
| Setup | Network access | Y | - | - | - | Y |
| Setup | Control Room Setup | Y | - | - | - | - |
| Setup | Station Profile | Y | Y | - | - | Y |
| Setup | Cable Commissioning | Y | - | - | - | Y |
| Setup | AI Models | Y | Y | - | - | - |
| Setup | Custom Fields | Y | - | - | - | - |
| Setup | Paywall | Y | - | - | - | - |
| Run Meeting | Live | Y | Y | Y | Y | Y |
| Run Meeting | Facility | Y | Y | Y | Y | Y |
| Run Meeting | Control Room | Y | Y | Y | Y | Y |
| Run Meeting | Remote Contribution | Y | Y | - | - | Y |
| Run Meeting | Channels | Y | Y | Y | Y | Y |
| Run Meeting | Other stations | Y | Y | - | Y | Y |
| Run Meeting | CG Board | Y | Y | Y | Y | Y |
| Run Meeting | CG Designer | Y | - | - | Y | Y |
| Run Meeting | Schedule | Y | Y | Y | Y | Y |
| Run Meeting | Auto-schedule | Y | - | - | Y | Y |
| Run Meeting | Program Guide | Y | Y | Y | Y | Y |
| Run Meeting | Recording | Y | Y | - | - | Y |
| Review Records | Assets | Y | Y | Y | Y | Y |
| Review Records | Missing Media | - | Y | - | Y | Y |
| Review Records | Media Lifecycle Settings | Y | - | Y | Y | - |
| Review Records | Contributors | Y | Y | Y | Y | Y |
| Review Records | Review queue | Y | Y | Y | Y | Y |

| Menu section | Screen | S | M | R | P | A |
|---|---|---|---|---|---|---|
| Review Records | Summary review | Y | Y | Y | Y | Y |
| Review Records | Agendas | - | Y | Y | - | - |
| Publish | Publish | Y | Y | Y | Y | Y |
| Publish | Playback policy | Y | Y | Y | Y | Y |
| Publish | Analytics | Y | Y | Y | Y | Y |
| Publish | Reports | - | - | - | - | Y |
| Publish | EPG Export | Y | - | - | Y | - |
| Publish | Underwriting | Y | - | - | Y | Y |
| Publish | App Admin | Y | - | - | Y | - |
| System Health | Readiness | Y | Y | Y | Y | Y |
| System Health | Alerts | Y | Y | Y | Y | Y |
| System Health | Emergency Alerts | Y | Y | - | - | Y |
| System Health | Federation | Y | Y | Y | Y | Y |

Two extra pages are not in the menu: an asset's detail page and its full-screen trim editor. Both follow the Assets rules.

> **Known issue (beta.11):** A token that carries only `setup_admin` cannot open **Missing Media** or **Agendas**. Those screens require `meeting_operator`, `publish_operator` or `support_admin` for Missing Media, and `records_clerk` or `meeting_operator` for Agendas. Use a token with one of those roles.

**Screens a role can open but cannot use**

| Role | Opens, but is refused |
|---|---|
| Records clerk | Alerts event list (HTTP 403); Federation Approve, Reject and Block (HTTP 403). The coarse runtime-safe-to-air signal is readable. |
| Publish operator | Alerts event list (HTTP 403). Federation moderation and the coarse runtime-safe-to-air signal are allowed. |
| Setup admin on its own | Federation Approve, Reject and Block (403). Key generation is allowed. |
| Support admin | Alerts rule and destination changes (Save, Add destination, Delete return 403); the feed buttons on Readiness are disabled. |
| Meeting operator | Alerts rule and destination changes (an access note); Readiness restore, update and support-bundle controls are disabled with notes. |

When the server refuses a role, the screen often prints the raw role ids ("This action requires one of these CivicCast roles: …"), not the labels above.

#### Who can do the key tasks

The table summarizes the task permissions described in this manual. Read "any signed-in role" as: the server only needs a valid token, and no particular role. Sensitive reads can still require a named role; menu visibility alone is not permission.

| Task | Allowed roles |
|---|---|
| Look at lists and status (assets, publish status, Program Guide reads, channel state, readiness, caption queue, summary queue, federation status) | any signed-in role |
| Read the Schedule list and the commit history | Publish operator, Setup admin, Support admin |
| Upload a video; edit an asset's title and details; relink a file | Meeting operator, Records clerk, Support admin |
| Package an asset for playback; remove it from the portal; replace its source file | Publish operator, Setup admin |
| Set or clear a legal hold on an asset | Records clerk, Support admin |
| Approve, edit or reject a caption line; retry an offline caption job | Records clerk |
| Start a summary job or generate a summary | Records clerk, Support admin |
| Approve a summary; retry a summary job; export a signed record | Records clerk |
| Approve publishing for an asset; retry a publish surface | Publish operator |
| Create or cancel a scheduled item; commit programs to air; Auto-schedule rules | Publish operator, Setup admin |
| Add, change or disable a Program Guide slot | Meeting operator, Support admin |
| Accept, decline or send to schedule a contributor submission | Meeting operator, Publish operator |
| EPG Export setups and Underwriting spots and flights | Publish operator, Setup admin |
| Build app packages | Setup admin |
| Change app settings and branding; record store-submission notes | Publish operator, Setup admin |
| Create the live session, run pre-flight, Start Live Stream, End Live Stream, Retry finalization | Meeting operator |
| Add or edit a live source | Setup admin |
| Check a live source (the probe) | Setup admin, Meeting operator |
| Start, Stop, Restart or Finish-then-stop a channel's feed | Meeting operator |

| Task | Allowed roles |
|---|---|
| Run Check broadcast readiness (the private rehearsal) | Meeting operator |
| Set up recording schedules; Record now; stop a recording | Setup admin, Meeting operator |
| Open a Control Room session; fire a cue; roll back | Meeting operator |
| Plan (dry run) a cue; read a session | Meeting operator, Support admin |
| Close a Control Room session | Setup admin, Meeting operator, Support admin |
| Show or clear an emergency alert on a channel | Setup admin, Meeting operator |
| Add an emergency alert source or a manual alert | Setup admin |
| Create, change or delete an alert destination; change an alert rule | Setup admin |
| Read alert rules and destinations | Setup admin, Support admin |
| Read or acknowledge alert events | Setup admin, Meeting operator, Support admin |
| Read the coarse runtime-safe-to-air signal | All five operator roles |
| Manage staff accounts and assigned roles | Setup admin |
| Configure, create, verify or schedule station backups | Setup admin, Support admin |
| Configure Network access | Setup admin, Support admin |
| Browse Other stations | Setup admin, Meeting operator, Publish operator, Support admin |
| Send Other stations Start/Stop commands | Setup admin, Meeting operator; the remote token must also permit commands |
| Change which AI model does a job; set a provider key | Setup admin |
| Create, edit and delete agendas and agenda items | Meeting operator, Records clerk |
| Paywall settings and comp passes | Setup admin |
| Create the federation key | Setup admin |
| Approve, reject or block a follower; replay a delivery | Publish operator, Support admin |
| Verify the backup folder | Setup admin |
| Check backup storage, run the restore drill, update preflight, maintenance window, rollback tools, repair the playout engine | Setup admin, Support admin |
| Create and download a support bundle | Support admin |
| Reports (as-run log, shows, hours by category, export) | Support admin |
| Edit community board, zones, feeds; approve bulletins | Publish operator, Setup admin |

#### Role by API area

This role-by-area matrix is a Beta 11 source snapshot and has not been regenerated for the current edition. For each area it lists routes with a role requirement and which roles may call them; a dash means none in that snapshot. Routes with no role requirement are reads, plus `POST /api/staff/auth/sign-out`; any valid token may use them. Areas that have no role-gated route (`auth`, `first-run`, `release`, `doctor`) are not listed.

How this was made: the application was loaded and each route's role requirement was read from the Beta 11 source. These counts are historical reference, not a current-package authorization measurement. For a current request, rely on the live server's authorization response and the generated API reference in Appendix B.

| Area | Gated routes | S | M | R | P | A |
|---|---|---|---|---|---|---|
| `activitypub` | 5 | 1 | - | - | 4 | 4 |
| `agenda` | 1 | - | 1 | 1 | - | - |
| `agenda-sources` | 2 | - | 2 | 2 | - | - |
| `agendas` | 11 | - | 11 | 11 | - | - |
| `ai-models` | 8 | 8 | 4 | - | - | - |
| `alert-channels` | 4 | 4 | - | - | - | 1 |
| `alert-events` | 2 | 2 | 2 | - | - | 2 |
| `alert-rules` | 2 | 2 | - | - | - | 1 |
| `analytics` | 5 | 1 | 1 | 1 | 5 | 5 |
| `app` | 8 | 8 | - | - | 7 | - |
| `assets` | 12 | 5 | 8 | 7 | 5 | 7 |
| `audio-tracks` | 3 | 3 | 1 | - | - | 1 |
| `auto-schedule` | 17 | 17 | - | - | 17 | 7 |
| `cable` | 9 | 9 | 4 | 4 | 4 | 6 |
| `captions` | 6 | - | - | 6 | - | 2 |
| `cg` | 18 | 18 | - | - | 18 | 6 |
| `contribute` | 5 | - | 5 | - | 5 | 4 |
| `contribution` | 15 | 1 | 12 | - | - | 6 |
| `control-room` | 21 | 15 | 11 | - | - | 9 |
| `custom-fields` | 5 | 5 | 2 | 2 | - | - |
| `eas` | 9 | 9 | 6 | - | - | 4 |
| `egress` | 15 | 14 | 8 | - | - | 5 |
| `epg` | 6 | 6 | - | - | 6 | - |
| `facility` | 2 | - | 2 | - | - | 2 |
| `installer` | 33 | 29 | 1 | - | 2 | 18 |
| `live` | 12 | 6 | 7 | - | - | - |
| `media-lifecycle` | 14 | 9 | 5 | 5 | 8 | 5 |
| `migrate` | 4 | 4 | - | - | - | - |
| `paywall` | 6 | 6 | - | - | - | - |
| `playback-policy` | 2 | - | - | - | 2 | 2 |

| Area | Gated routes | S | M | R | P | A |
|---|---|---|---|---|---|---|
| `playout` | 5 | 5 | - | - | 5 | 2 |
| `podcast` | 1 | - | - | - | 1 | 1 |
| `producer-ops` | 21 | - | 21 | - | 21 | 21 |
| `programlog` | 4 | - | 4 | - | - | 4 |
| `publish` | 2 | - | - | - | 2 | - |
| `recording` | 9 | 9 | 9 | - | - | 4 |
| `records` | 1 | - | - | 1 | - | - |
| `reports` | 4 | - | - | - | - | 4 |
| `runtime-safe-to-air` | 1 | 1 | 1 | 1 | 1 | 1 |
| `schedule` | 4 | 4 | - | - | 4 | 2 |
| `self-tests` | 2 | 2 | - | - | - | 2 |
| `station` | 2 | 2 | 1 | - | - | 1 |
| `station-box-profile` | 2 | 2 | 2 | - | - | 2 |
| `stream` | 1 | - | 1 | - | - | 1 |
| `subscribe` | 1 | - | - | - | 1 | 1 |
| `summaries` | 5 | - | - | 5 | - | 3 |
| `system-resources` | 1 | 1 | - | - | - | 1 |
| `underwriting` | 14 | 12 | - | - | 12 | 2 |

> **Note:** Alerts, Reports and a few areas were not tested here beyond reading the role requirement. This table says who the server lets in, not whether the screen works for them. “Role-gated” does not include checks written inside a route’s own code.

## Appendix G: Checklists

Copy these into your own notebook or ticket system. Each box is one action. Where a screen name is given, it is the menu name. Schedule and recording entries use the station’s configured time zone; reports and exports may retain UTC, as noted below.

> **Warning:** Check the matching verification record for the source, hash and tests tied to the package you are using. These lists help prepare and operate a station; they do not certify the package.

### G.1 Go-live checklist

Do this once before the first public meeting, and again after any reinstall.

**The computer and the service**

- ☐ The Windows service `CivicCastSupervisor` is Running and set to start automatically (`Get-Service CivicCastSupervisor`).
- ☐ The top bar shows the version you meant to install.
- ☐ The console opens at `http://127.0.0.1:8000/operator/` on the station computer.

- ☐ Decide how residents and other staff will reach the station. Configure Caddy with separate public and CIDR-restricted staff hostnames if remote access is needed; test each from the intended network (Appendix D).
- ☐ Free disk space is comfortable on the drive holding `C:\ProgramData\CivicCast` (the conform cache alone may use up to 60 GB by default) and on the drive for `CIVICCAST_UPLOAD_DIR` (the Live screen needs 50 GiB free there).

**Accounts and the recovery kit**

- ☐ First Setup is done: station name, time zone, first admin.
- ☐ The recovery kit (admin username, password, the 8 one-time recovery codes) is saved or printed and stored where a second person can find it, and you pressed **I found the kit — it is stored safely**. Until you do, the whole menu except Manual is locked.
- ☐ Each staff member has an account with the needed roles and knows how to sign in. The first administrator remains the recovery account (Appendix F).

**Configuration**

- ☐ Station Profile: station name, time zone, default channel, public web address and the three storage folders look right and have space.
- ☐ Station Profile: decide the **Show live captions on air** switch (off by default). Turning it off stops recognition on the next tap-worker scan and removes caption routing when each channel next starts. Turning it on resumes recognition on the next scan; stop and start each channel before expecting captions in its video. Whistle is the native CPU primary; NVIDIA/CUDA is optional Whisper acceleration.
- ☐ AI Models: the caption, summary and translation models are the ones you intend. Remember that a local change takes effect the moment the choice changes.
- ☐ First Setup: **Verify backup** succeeded for your backup folder; this only checks storage. On **Backup and restore**, configure an encrypted station archive, select media roots and retention, create and verify a backup, and retain its passphrase separately (G.7).
- ☐ Readiness: no required row is red; read every row in "Required before broadcast" (First admin, Recovery kit, Durable records storage, Backup, Camera or meeting source, Local recording, Resident portal, Station policy).
- ☐ Readiness: press **Check broadcast readiness** and read the result. It creates a short private test session and recording on channel `government`.
- ☐ Channels: every channel you plan to air has an outgoing-feed configuration, and you have pressed **Start** once and seen the state change to On air, then **Stop** and seen Stopped.
- ☐ If you send a channel to a cable company: Cable Commissioning is complete, and you have read the warning that the test signal goes to the headend address.
- ☐ Emergency Alerts: you know that CivicCast is not an emergency-alert device, and who adds alert sources (setup admin only, through the API).

**Times** (station zone and UTC)

- ☐ New recording entries, Schedule and Program Guide use the station time zone. Check the dis-

played local time and weekday; older weekly recording schedules can retain UTC timing.
- ☐ Alerts: quiet hours are typed in UTC.
- ☐ EPG Export: dates and times in the file are UTC.
- ☐ Auto-schedule dayparts use the station time zone from Station Profile.

**Public side**

- ☐ The resident portal shows your station's channels, schedule and recordings.
- ☐ Subscribe (email and RSS) and Submit a program forms work the way you intend, or you have decided not to promote them.
- ☐ Paywall, Federation and Remote Contribution stay off unless you have set them up; each is off by default.

#### G.2 Meeting-night checklist

**Hours before**

- ☐ Readiness: the banner and top card are green or yellow with a reason you understand. Press **Check broadcast readiness** (needs the Meeting operator role).
- ☐ Missing Media: no meeting on this week's schedule is missing its video.
- ☐ Schedule or Program Guide: tonight's programs and the recording schedule are present. Check the station-local time and the older weekly-recording UTC exception above.
- ☐ Channels: the commit list shows tonight's programs; press **Approve & put on air** for those that need it. The program airs at its scheduled time, not at the click.
- ☐ Channels: **Stop** or **Finish current item, then stop** suppresses automatic starts for this service session. To remain off across a service or computer restart, ask a Setup admin to untick **Keep this channel on air**.
- ☐ Control Room (if used): start in Test mode. An On-Air session lasts 30 minutes, then must be reopened.
- ☐ Remote Contribution (if used): open the room, send single-use guest links (they expire after 4 hours).
- ☐ Captions: check the Captions row on the channel card (On, or Off because of the station profile).

**The Live room (if you use it)**

- ☐ Choose the broadcast channel and the meeting source. Press **Check source** until it says Delivering. A result goes stale after about 30 seconds.
- ☐ **Create live session**, **Start pre-flight**, tick the confirmation box in Session controls, **Run pre-flight**, fix any red row, **Start Live Stream**.
- ☐ Network reachable needs internet access (it tests 1.1.1.1 and 8.8.8.8). Recording storage needs 50 GiB free.
- ☐ **Do not refresh the browser tab while on air.** The live session is held only in that page; a refresh loses it and you cannot end the stream from that screen.

**During**

- ☐ Watch the on-air banner on Readiness; it refreshes every 5 seconds. Other panels do not refresh by themselves.
- ☐ A Stop or Restart can take up to about 30 seconds to show. Do not press it again.
- ☐ If the channel shows slate or "Needs attention", read chapter 8 and the logs (Appendix D).

**After**

- ☐ **End Live Stream** (if you used it) and wait for "Recording saved as asset …". Confirm on Channels that the feed is in the state you want.
- ☐ Assets: find the recording, **Package for playback**, check the title and details.
- ☐ Review queue: approve, edit or reject every caption line; nothing is attached to the video until all are decided.
- ☐ Summary review: approve the summary if you use one; export the signed record.
- ☐ Publish: choose the surfaces and approve. Read the status pill afterward (Appendix E).

### G.3 Daily checklist

- ☐ Readiness: top card, banner and required rows. Read "Latest self-check" (the daily self-check runs overnight).
- ☐ Alerts: anything active?
- ☐ Missing Media: empty?
- ☐ Machine health on Readiness: Media space and Backup space free, Database Reachable, Service Running.
- ☐ Review queue: lines waiting.
- ☐ Contributors: new submissions waiting.
- ☐ Each channel's feed state is what you expect.

### G.4 Weekly checklist

- ☐ Run the weekly self-check (**Run weekly self-check now**) and read its result.
- ☐ Look at free space and at the conform cache folder `C:\ProgramData\CivicCast\data\egress\conform-cache` against its limit.
- ☐ Open `C:\ProgramData\CivicCast\logs\supervisor.log` and scan for repeated restarts of a child program.
- ☐ Check **Check backup storage** on Readiness and confirm the scheduled station archive completed and verifies. A storage probe alone is not a backup (G.7).
- ☐ Analytics: audience counts and any gaps.
- ☐ Retention: Media Lifecycle Settings still match your retention rules and watch folders are working.
- ☐ Tokens: `civiccast token list` and revoke any you no longer need (Appendix A).

### G.5 Monthly checklist

- ☐ Run the real database restore drill (Readiness, **Run real database restore drill**, or `civiccast dr run-drill --out <folder>`), at a quiet hour.
- ☐ Restore one file from your own media backup to a different folder and play it.
- ☐ Rotate staff tokens used by scripts (`civiccast token rotate`).

- ☐ Look at the CivicCast release page for a newer release and read its notes before you decide (Appendix K).
- ☐ Recovery kit: confirm it is still where you stored it. It holds 8 one-time recovery codes and each recovery uses one.
- ☐ Check certificates on Readiness ("Internal service certificates" in the advanced list).
- ☐ If Federation is on: review followers and the delivery retry queue.
- ☐ Review Windows updates and reboot at a quiet hour; confirm the service comes back by itself and the channels return.

#### G.6 Before-upgrade checklist

> **Warning:** Read the verification record matching the package you plan to install. Before upgrading an on-air station, keep a separate backup and its recovery information.

- ☐ Read the release notes and the verification record for the version you are installing. Note what was and was not proven.
- ☐ Choose a time when no meeting is scheduled and no channel is needed.
- ☐ Create and verify a `.ccbackup` station archive, check its selected media roots and path inventory, retain its passphrase and recovery kit separately, and run the database drill the same day (G.7). Keep separate copies of files outside the archive.
- ☐ Prepare matching installer resources with `scripts\download_windows_release_artifacts.ps1 -Install`; it verifies and assembles the signed station bundle from the release manifest and archive parts. The R9 field attempt used an earlier public asset set that lacked those resources and failed activation.
- ☐ Readiness, Update and rollback panel: save the path to the installer you would roll back to (the field is labelled **Rollback artifact path**; its own example is an older installer) with **Save rollback artifact**, then run **Run rollback rehearsal**, then **Run update preflight**.
- ☐ On Readiness, **Open maintenance window** (60 minutes). It needs the update preflight and a passed rollback rehearsal first. During the upgrade itself the service holds the station in maintenance mode, and changes are refused with HTTP 503.
- ☐ Note the free disk space.
- ☐ Run the new `setup.exe` over the existing install. The installer first checks the installed version without stopping the service or replacing files, then arms recovery, stops the service and snapshots prior state before payload replacement. It re-checks the packs, backs up and migrates the database, then registers and starts the service. An older setup is refused (exit 129).
- ☐ If setup fails, note the exit number (Appendix E) and read `install-progress.log`. For upgrade failures, check the failed-run engine log and journal under `install-transaction\failure-diagnostics` when present. Follow Appendix E: exit 128 requires archiving or moving the stale journal aside; if restoration did not finish, keep `install-transaction` and station data intact before retrying setup.
- ☐ After: Readiness is green, the version in the top bar changed, **Run post-update proof**, then run the go-live items that touch channels and captions.

#### G.7 Station backups and disaster-recovery drill

Use **Backup and restore** for encrypted `.ccbackup` archives containing the database, configured station state, portable credentials and selected media roots. Configure a schedule and retention, verify archives, and save the passphrase separately. An archive cannot restore files it did not include; inspect its inventory before relying on it. Reinstall program files, packs and models from a matching installer.

Restore is an offline maintenance operation: stop CivicCast and other writers, prepare the replacement database and service, then use `civiccast backup inventory` and `civiccast backup restore` with explicit target mappings and `--confirmation RESTORE`. Do not overlay a running station. A database restore can finish before a later file copy fails; keep the archive and correct the reported path or permission problem before retrying. See Chapter 12.

The database drill and backup-storage probe are separate checks:

- Its drill backs up the real database, restores it into a completely fresh database, and checks row counts, content checksums and that the program's own stores can read the restored data.
- The drill does not replicate your media: it records a list of files and a hash of a bounded sample. Include the required media roots in station archives or maintain a separate file backup.
- The Setup/System Health backup-storage probe writes, copies and checksums a small manifest; it does not copy or restore station data. The database drill is separate.
- The drill restores into a scratch copy; it is not the offline station-archive restore command.

Checklist:

- ☐ Write down, in one place, what must survive a lost machine: the database, `C:\ProgramData\CivicCast\data\uploads`, `station-state.json` and its folder, the recovery kit, your own copy of `setup.exe` and the packs.
- ☐ Name who runs the backup, how often, and where the copy lives (a different computer or drive).
- ☐ Run the drill at a quiet hour: Readiness, **Run real database restore drill**, or `civiccast dr run-drill --out <folder>` (it writes `dr-drill-report.md` and `.json` into that folder).
- ☐ Read the report. Keep it with the date.
- ☐ Prove your media backup: restore a few files to another folder and play them.
- ☐ Prove the recovery kit: you can find it and the codes without the station computer. Do not use up a recovery code for practice unless you intend to.
- ☐ Rehearse a full rebuild on a spare machine: install the same setup, use the offline station-archive restore with reviewed path mappings, sign in, and run the go-live list. Record any required files outside the archive.
- ☐ Record how long it took and what surprised you; fix the procedure.

### Appendix H: Measured evidence and known limits

This appendix preserves historical evidence and its build identity. Use the verification record matching the package for its identity and checks. The Beta 18 record tracks that earlier package; the unpublished Beta 20 candidate needs its own package record. The older beta.10 record is `docs/releases/v1.0.0-beta.10-verification.md`, with evidence under `ops/beta10-oversight/`

in the repository.

#### Historical Beta 11: three channels, 72 hours, 99.78% live caption-input coverage

Public, Government and Education ran together for 72 hours without a service restart. All 216 scheduled checkpoints reported healthy service status. Recorded live caption-input coverage was **99.78%**, calculated from 1,695 seconds of logged discarded input across 777,600 channel-seconds. This is a historical Beta 11 measurement, not a Beta 16 measurement. The Beta 11 72-hour host report preserves channel totals and the run identity. Earlier Beta 11 48-hour evidence remains linked in that report.

#### H.1 Which build each result comes from

| Evidence | Build | What it covers |
|---|---|---|
| Clean-install Gate A lane, 10 of 10 criteria | The **published** beta.10 installer and packs (commit `b6520847`) | Install, first start, console and portal render, clerk workflow, captions, playout, 5-minute soak, install progress, completion |
| Eight-hour watched run, 3 channels | Earlier internal build `C16`; the four key engine files are identical to the published source | Playout, program changes, loudness, preparation times |
| About 59 hours of uptime, 2 channels | Earlier internal build `C15`, **not** the published build | Long-run stability of the engine |

No long run was done on the published beta.10 installer. That build contains later fixes (including the watchdog that ends a stuck program change) and installer fixes relative to the earlier internal builds.

#### H.2 Eight-hour watched run (2026-10-01 16:30 to 2026-10-02 00:30, Mountain time)

One lab station, three channels (education, government, public) airing real programs at the same time, captions embedded in the output.

| Measure | Result |
|---|---|
| Station service | One process for the whole 8 hours; no restart, no crash |
| Slate or filler after startup | None |
| Station errors and watchdog firings | 0 and 0 |
| Verification checks | 41 run, 40 passed, 1 raw failure (explained below) |
| Loudness | 16 of 16 four-minute windows passed on all three channels (target -16 LUFS, plus or minus 1) |
| Program changes | 50 watched, no holes, no errors; picture gap at most 0.033 s, sound gap at most 0.041 s |

| Measure | Result |
|---|---|
| Preparation of upcoming programs | 51 logged; the longest took 389 s, inside the lead time of about 11 minutes |
| Disk | 46 GB of the 60 GB conform-cache budget used; 810 GB free |

The one raw failure was a check that the education playlist advanced between two samples. It was judged a sampling blip by the run's own records, but a hiccup of under five seconds cannot be ruled out by direct observation, and the raw failure stands in the record.

#### H.3 About 59 hours of two channels on the air (the "accidental soak")

After a planned run ended on 2026-09-29, the lab station was left running. The station service started on 2026-09-29 at 02:59 and was still running, with no restart, when it was checked on 2026-10-01 at 14:10, which is about 59 hours. Roughly 51 of those hours were after the planned run ended and were not being watched.

- **Government and public channels:** 27 of 28 programs and 111 of 112 seamless program changes went through. Both channels stayed on the air to the end of the period, with no slate, no filler and no crash.
- **Education channel:** it got stuck in a program change on 2026-09-30 at about 03:44 and then showed black picture and silence for about 34 hours until the service was restarted. The watchdog that now ends that kind of stuck program change was built afterwards and is in the published Beta 10 build. The exact stall has not recurred since.
- **One caption file error:** the government channel logged one isolated "permission denied" error while renaming a caption file.

**How to state this fairly.** The same playout engine has run two channels on one lab station for more than two days. That is not the same as saying beta.10 has done so, because the long run used an earlier build. It is two channels on one station, not two stations, and it was assessed from service logs, program-change records and sampled output, not by inspecting every second of video.

#### H.4 Limits recorded for beta.10

1. **Two program-change first attempts aborted and recovered** on the government channel during the eight-hour run, each in about 3 seconds with no visible effect on air.
2. **One single-frame video drop** (0.033 s) at a join inside a program.
3. **Quiet source material is reported, not leveled** when it is too quiet for the loudness control's maximum lift.
4. **Eleven playout-engine unit tests fail** in the same way on the published tree and the installed tree; they were failing before this build and are unexplained.
5. **An unexplained rebuild of a long cached program** was seen in the previous internal run and is unresolved.
6. **Upgrades were not proven for beta.10.** Installing beta.10 over an earlier release was not tested; use the matching release record for the package you plan to install.

7. **A first install with neither the full kit (installer plus its `packs` and `station` folders) nor an earlier install was not proven for beta.10.** The code requires the kit. See Chapter 10 for the current package contents and evidence link.
8. **Several staff-console screens described behavior the beta.10 code did not have.** Current screen notes in Part I use the current scope where the mismatch remains.

#### H.5 What is not proven

- The beta.10 record contains no human field sign-off or run at a real station; this is not a statement about later field observations or packages.
- No 24-hour or 72-hour soak was run on the beta.10 published build. A separate dev7 overlay soak and Beta 11 package results are documented separately, not as beta.10 evidence.
- The beta.10 record does not establish physical broadcast video-card checks, real cable-headend acceptance or sustained production use at a real PEG station.

#### H.6 How to use this evidence

Read sections H.1 through H.5 as historical evidence for their named builds and runs. For the unpublished Beta 20 candidate, use the current operating chapters; installation verification is pending; keep each measured result associated with its recorded source and run.

## Appendix I: Glossary

Plain-English meanings of the terms used in this manual, in alphabetical order. A word in *italics* inside a definition has its own entry. Where a term is a screen name, it is the name in the console's menu.

**A**

**ActivityPub** The open standard that lets servers follow each other's published activity. CivicCast's optional Federation feature uses it.

**Admin (first admin)** The one local account made on First Setup. It signs in with a username and password and holds all five *roles*.

**Affidavit** On the Underwriting screen, a per-sponsor proof-of-airing report used for billing.

**Agenda** A numbered list of items for one recorded meeting, each optionally tied to a moment in the video. Once published it appears beside the player on the public watch page, or below it on a phone.

**AI engine** The Ollama program the station runs on the same computer for meeting summaries and Spanish translation. It listens only on the station computer. See *Ollama*.

**Alert** A problem the station has flagged, shown on the Alerts screen and counted on Readiness. Not the same as an *emergency alert*.

**Alert destination** Where an alert is supposed to be sent: an email address, a phone number for text messages, or a *webhook*. Quiet hours (typed in UTC) are set per destination.

**Alert rule** One condition the station watches (for example a channel going off air) with its settings: on or off, severity, how often to repeat, and whether to notify when it clears.

**API** The set of web addresses the console, the portal and other tools use to talk to the station. See Appendix B.

**App shell** A small generic viewing app that App Admin can package for a platform (web, Roku, Apple

TV, Fire TV, Android, iPhone and iPad). It is not an app-store release.

**Archive (surface)** A place where a copy of a published recording is kept: the Internet Archive or a local *NAS*. Required archive copies must succeed for the pill to read "Archive verified".

**As-run log** The log of what actually aired on the station's channels, written by the playout engine. The Reports screen reads it.

**Asset** One video the station holds: an uploaded file, a file picked up from a *watch folder*, an accepted contributor program or a live recording.

**Auto-schedule** Filling a channel's air time by rule instead of one item at a time. A rule joins a *saved search* and a *daypart*; a compile run places matching recordings.

**B**

**Backup destination** The folder you name on First Setup so CivicCast can check it can write there. It does not copy your data by itself.

**Bearer token** The secret a tool sends in the `Authorization: Bearer` header to prove who it is to the API.

**Beta candidate** A release labeled by GitHub as a *pre-release* for testing, not a production release. Check the project's releases page for the current native-line version and its publication status.

**Board (community board)** The between-programs picture a channel shows: zones for a ticker, schedule, logo, sponsor and approved *bulletins*.

**Bulletin** A short community announcement submitted for the board. It does not air until staff approve it.

**C**

**Cable file package** A folder made by `civiccast cable package` holding a recording, its captions and a manifest, for sending to a cable operator as files.

**CAP (Common Alerting Protocol)** The standard format for public-safety alerts that the Emergency Alerts screen reads from feeds such as NWS, AMBER and IPAWS.

**Caption** Text of what is said, shown with the video. Each timed line is a *cue*.

**Caption review queue** The Review queue screen, where a person approves, edits or rejects each caption line before it is attached to a recording's public video.

**CC BY 4.0** The license for CivicCast's documentation: you may copy and adapt it if you credit the source.

**CEA-708** A standard for captions carried inside a television signal. The `civiccast egress verify-captions` command tests embedding and reading back this kind of caption.

**CG (character generator)** Software that puts text and graphics on the picture. CG Board and CG Designer manage the community board.

**Channel** One outgoing program stream with its own schedule and feed, such as `government` or `public`.

**Chapter** A named point in a recording. An agenda can be filled from a recording's chapters.

**Check source** The Live screen button that runs one short test on a meeting source to see whether video is arriving.

**Commissioning (cable)** A four-step wizard that checks the computer, records the headend choices, sends a timed test signal and produces a report.

**Commit to air** Approving a scheduled program on Channels so it airs at its time. It does not start it at the click.

**Comp pass** A free access pass issued by staff to a named email when the *paywall* is on.

**Conform** Preparing a copy of a program in a standard form so it plays smoothly on a channel.

**Conform cache** The folder of prepared copies. It lives at `data\egress\conform-cache` under the data folder and is limited to 60 GB by default.

**Console token** The proof of sign-in the browser keeps after staff sign-in at `/login` or first-admin sign-in at `/setup`. It is stored in the browser until you sign out, an administrator revokes the session, or it expires.

**Contributor** An outside producer who sends in a program through the public "Submit a program" form. Staff review these on the Contributors screen.

**Control plane** The main CivicCast program: the web server and playout engine, listening on configured IPv4 or IPv6 loopback, normally `127.0.0.1:8000`.

**Control Room** The screen for driving production equipment (switchers, cameras, recorders) with one-press *cues*. Different from Control Room Setup.

**Crawl** A message that moves across the picture. One of three ways an emergency alert can be shown (crawl, overlay, forced slate).

**Cue (caption)** One timed line of caption text.

**Cue (Control Room)** A one-press action on a control surface, such as taking a camera scene. You dry-run it, then confirm to fire it.

**Custom field** A label the station invents for its programs, such as "Meeting type". It becomes a box on each asset and can become a public filter.

**D**

**Daypart** A recurring time window on a channel, such as weeknights 6 to 10 PM, that an *Auto-schedule* rule fills.

**Dead letter** A message that could not be delivered after the allowed tries. The console shows it as "Undeliverable".

**DeckLink** A family of video cards (Blackmagic) used for SDI input and output.

**Decode-back proof** A check that reads captions back out of what was actually sent, to prove they are in the picture.

**Degraded** Working, but not fully healthy. For the service it means five restarts in ten minutes. For `/health` it means the database schema is not confirmed current or live-caption readiness is degraded or unknown. Read `schema` and `live_captions`; this probe does not certify every output or device.

**Director view** A link, shown right after you open a Remote Contribution room, that you embed in your video switcher.

**Disaster-recovery drill** A test that backs up the real database, restores it into a scratch copy and checks the result.

**Dispatch** The step where a committed program's start is sent to the playout engine.

**DST (daylight-saving time)** The twice-a-year clock change. The Schedule screen warns you to confirm the local meeting time around it.

**E**

**EAS (Emergency Alert System)** The national alert system. CivicCast says on screen that it is **not** an EAS device.

**Egress** Everything that carries a channel out of the station: the outgoing feed, web video, cable output and streams.

**Embargo** A scheduled release moment for an approved recording, with no length.

**Emergency alert** A public-safety alert pulled in from a configured feed. Polling and automatic display are off by default. IT must enable `CIVICCAST_EAS` and `CIVICCAST_EAS_AUTO_SURFACE` before applicable severe or extreme alerts can surface automatically. With inline GStreamer presentation enabled before channel start, the emergency layer is drawn into output video; the resident portal also polls its applicable notice. A full-screen *forced slate* requires explicit operator confirmation. CivicCast is not a certified EAS device.

**Environment variable** A named setting handed to a program when it starts, such as `CIVICCAST_CONFORM_CACHE_GB`. See Appendix C.

**EPG (electronic program guide)** A file of upcoming programs that cable boxes and guide services read. The EPG Export screen makes it in X-List, XMLTV or CSV form.

**Event Log (Windows)** Windows' own record of events. The CivicCast service writes to it under the source `CivicCastSupervisor`.

**Exit code** The number a program returns when it ends. Setup's exit codes are in Appendix E.

**F**

**Facility** A preview-only screen for a video router. Its buttons show the command that would be sent; they do not send it.

**Federation** An optional feature that lets other servers on the fediverse follow the station and see when a meeting is published. Off by default.

**Fediverse** The network of servers (Mastodon and similar) that talk to each other using *ActivityPub*.

**Feed (outgoing channel feed)** The running stream for a channel. Start, Stop, Restart and Finish-then-stop control it.

**Feed (outside content)** An RSS, calendar, weather or social source that CG Designer can pull into a board zone.

**FFmpeg** The open-source video tool CivicCast uses to prepare and check media. `ffmpeg.exe` and `ffprobe.exe` are in the install folder.

**Finalization** After a live meeting ends: a worker finds the recording file and turns it into an *asset*.

**Firewall rule** A Windows setting that allows or blocks network traffic. Setup adds one for TCP 8000.

**First Setup** The screen that creates the first admin and recovery kit, and later serves that administrator's sign-in and recovery flow. Staff use `/login`.

**Flight** The date window in which an underwriting spot should run.

**Follower** A remote account that has asked to follow the station on the fediverse. Staff approve, reject or block it.

**Forced slate** A full-screen takeover with an emergency message. It needs an explicit operator confirmation.

**G**

**Gate A** The set of checks run in a clean Windows Sandbox before a release is published. It has a

clean-install lane, an upgrade lane and a download-only lane. The beta.10 verification record reports only its clean-install lane; check each release's verification record for the lanes run on that candidate.

**Gemma** The family of AI models used for summaries (12B and e4b) and, as TranslateGemma 4B, for Spanish translation. They run locally through *Ollama*.

**GitHub pre-release** A release on GitHub marked as not final. Beta.11 is published this way.

**GStreamer** The video framework CivicCast's playout engine is built on.

**H**

**Headend** The cable company's equipment that receives a channel from the station.

**HLS (HTTP Live Streaming)** The web video format residents' browsers play for live and recorded video.

**I**

**Inbox and outbox (federation)** Where the station receives follow requests and where it lists its own published activity.

**Ingest** Bringing a file or stream into the library and checking it. A file that fails the check is Rejected.

**Install folder** Where setup puts the program, normally `C:\Program Files\CivicCast (Native)`.

**Internet Archive** An outside archive. One of the archive copies a publish can make, if configured.

**Invite link** A single-use browser link for a remote guest. It expires after 4 hours.

**IPAWS** The U.S. federal alert system whose alerts, in CAP form, the Emergency Alerts screen can read.

**K**

**Kit (installer kit)** The folder holding `setup.exe` together with the `packs` and `station` folders that setup uses.

**L**

**LAN-only station** A station set to assume it cannot reach the public internet. A native station is set this way, so `/docs` and `/redoc` are off and `/openapi.json` needs a token.

**Legal hold** A flag on an asset that stops it from being removed under retention rules.

**LGPL and GPL** Two families of free-software licenses. CivicCast avoids shipping GPL-licensed code and ships LGPL libraries with their license text. See Appendix J.

**Live session** The Live screen's record of one meeting: idle, pre-flight, on air, ending, recorded.

**Live takeover** Putting a live source on a channel now, overriding its schedule, for up to an hour by default.

**Local AI** AI models that run on the station computer. Hosted models are separate and may incur charges under the provider's current pricing and account terms.

**Loopback (127.0.0.1 or ::1)** An address that means "this computer only". A program listening there cannot be reached from the network.

**LUFS** A unit of perceived loudness. Preparation follows the channel target; streaming commonly uses -16 LUFS and ATSC cable profiles use -24 LUFS. Different output regimes can have separate targets.

**M**

**Maintenance mode** A state in which the service holds the station read-only while the installer or a migration works. Requests that change data get HTTP 503; reads still work.

**Maintenance window** A 60-minute period you open from Readiness as part of an update; the console records it. Separate from the service's own *maintenance mode* during an upgrade, when changes are refused with HTTP 503.

**Media gate** The check on a contributor's file, shown as not_run, passed, failed or override_accepted.

**Meeting body** The name of the group that held a meeting, such as "City Council". Retention rules and Auto-schedule can match on it.

**Migration (database)** A change to the database layout made during an upgrade, with a backup first.

**Missing Media** A warning list of meetings in the next week whose video is not ready to play.

**N**

**NAS (network-attached storage)** A storage box on your network. One archive copy can go there.

**NDI** A way to send video over a network. An output kind on Channels.

**NWS** The U.S. National Weather Service, one source of weather alerts.

**O**

**Ollama** The local AI engine program (version 0.30.6 in the current runtime lock), started by the service on `127.0.0.1:11434`.

**On-Air Mode** A Control Room session type that really fires cues. It expires after 30 minutes. The other type, Test Mode, fires nothing.

**OpenAPI** The machine-readable description of every API route, served at `/openapi.json` with a token.

**Operator console** The staff web application at `/operator/`.

**OTT** "Over the top": viewing apps for Roku, Apple TV, Fire TV and similar. See *App shell*.

**Overlay** Text or graphics drawn over the picture. One of three emergency-alert display modes.

**P**

**Pack (.ccpack)** A signed bundle of program files or models that setup unpacks and checks.

**Package for playback** Making an asset streamable. Not the same as publishing.

**Paywall** An optional setting that holds some recordings behind a paid subscription (through Stripe) or a free comp pass. Off by default.

**PDF/A-3B** A long-term archive form of PDF. Summary review exports its signed record in it.

**PEG** Public, educational and government access television.

**Playback policy** Who is allowed to watch (everyone, signed-in residents, or an invite group), an optional public-record lock and up to four prerolls.

**Playout** The act of airing scheduled programs on a channel. The playout engine does it.

**Portal (resident portal)** The public web site at / where residents watch live and recorded meetings.

**PostgreSQL** The database program that holds all station records. It listens only on the station computer.

**Pre-flight** The Live screen's checklist run before a meeting goes on air.

**Premiere** A scheduled item that places a recording on a channel at a start time for a set length.

**Preroll** A short graphic or video card shown before playback.

**Private rehearsal** What **Check broadcast readiness** runs: a short private test session and recording.

**Program Guide** The screen of recurring slots that place a recording on a channel on a repeating pattern.

**ProgramData** The Windows folder `C:\ProgramData`, where `CivicCast` holds the station's data.

**Proof (egress)** A recorded result showing the output did what was claimed, such as a caption decode-back proof.

**Publish** Making a recording public through chosen *surfaces*. Approving it on the Publish screen does the work.

**R**

**Readiness** The menu name for the screen that answers "can we broadcast now?" (the page calls itself "Safe to broadcast").

**Recording schedule** A rule for capturing a live input at set times. New entries use the station's configured time zone; older weekly entries can retain UTC timing.

**Recovery code** One of 8 one-time codes in the recovery kit. Using one sets a new admin password.

**Recovery kit** The printed or saved sheet with the admin username, password and recovery codes, made on First Setup.

**Registry (Windows)** Windows' settings database. CivicCast keeps a few values under `HKLM\SOFTWARE\CivicCast`.

**Retention** How long an asset is kept, set by rules (including by *meeting body*). The Media Lifecycle Settings screen says nothing is deleted automatically: expired assets are flagged for a records clerk to review.

**Reverse proxy** A program in front of a web server that passes requests to it. CivicCast's application stays on IPv4 or IPv6 loopback. Network Settings generates a Caddy configuration for HTTPS public and CIDR-restricted staff hostnames; IT deploys and commissions that proxy.

**Role** A set of permissions. CivicCast has five. See Appendix F.

**Rollback** Returning to the previous version after a failed update.

**RSS** A feed format. Residents can follow new recordings by RSS; CG Designer can also read RSS sources.

**RTMP** A streaming protocol that some encoders and platforms use. A kind of live source and output.

**Rule (Auto-schedule)** A saved-search-plus-daypart pair that decides what to place when.

**S**

**Safe State** The recovery cue you pick before opening an On-Air Control Room session. The panic and roll-back buttons fire it.

**Saved search** A set of conditions that picks which recordings qualify for an Auto-schedule rule.

**Schema** The shape of the database. `/health` reports whether it is current.

**Scope** A word on a token that grants a role. `admin` and `operator` grant all five.

**SDI** A professional video cable standard used by cameras and cable equipment.

**Self-check** A set of automatic tests the station runs (daily and weekly); the result is on Readiness.

**Service (Windows)** A program that Windows starts by itself. CivicCast's is `CivicCastSupervi-`

sor.

**Setup (setup.exe)** The Windows installer. Also the name of a menu section.

**Silent install** Running setup with `/S`, which shows no windows and returns only an exit code.

**Slate** A fallback card a channel shows when nothing is ready to air.

**Slot** A recurring Program Guide entry, which a background job turns into real schedule items.

**Spot** A short underwriting sponsor message.

**SRT** A streaming protocol for video over unreliable networks. A kind of live source and output.

**Station** The whole CivicCast system on one computer, and the organization it serves.

**Station index** The signed list of the packs a station needs, in the `station` folder.

**Summary** An AI-written meeting summary built from approved captions, where every claim points to timestamped cues. A records clerk approves it.

**Supervisor** The service program that starts, watches and stops PostgreSQL, the control plane and the AI engine.

**Support bundle** A redacted troubleshooting file made on Readiness by a support admin.

**Surface** A destination in Publish (Portal, Internet Archive, NAS, YouTube, cable file package). In the Control Room, a named panel of cues.

**Syndication** Sending a recording or stream to outside platforms. A Live pre-flight row.

**T**

**Takeover** See *Live takeover*.

**TCP and UDP** Two ways programs send data over a network. A "port" is a numbered door for each.

**Tier (hardware)** The recommendation `civiccast doctor` gives for how capable the computer is.

**Time zone** Where "9 AM" is measured. Scheduling forms use the station's configured zone; reports and some other dated fields use UTC.

**Token** See *Bearer token* and *Console token*.

**Trim** Cutting the start and end of a recording. There is a full-screen trim editor.

**TSDuck** An optional tool for checking and smoothing a cable stream. Its program is `tsp.exe`.

**TSR control service** The helper the Control Room uses to talk to production equipment.

**TURN (coturn)** A relay server that helps remote guests connect. Remote Contribution needs one you set up.

**U**

**Underwriting** Paid sponsor acknowledgment messages. The screen keeps a catalog of spots and flights and a billing report.

**Upgrade** Running a newer setup over an existing install. Check the release verification record for the exact package to see whether its upgrade lane was tested.

**UTC** Coordinated Universal Time, the clock that does not change with seasons or places. Reports, exports and some dated controls use it.

**V**

**VDO.Ninja** A browser video service used for remote guests; you host it yourself.

**VOD (video on demand)** Recorded programs residents can watch any time.

**W**

**Watch folder** A folder CivicCast checks automatically and imports video files from.

**Webhook** A web address that receives a message when something happens. One kind of alert destination.

**WebVTT** A text format for captions that web players read.

**Whisper (faster-whisper)** The engine used for recorded captions, native live fallback, or explicitly selected live-primary operation. Medium is the standard installed Whisper model; Large v3 is optional. Native live captions default to Whistle instead.

**Whistle** The native live-caption primary, using the local CPU engine and model. Primary inference is serialized across station channels. Failures use Whisper fallback while bounded probes try to restore Whistle automatically; neither engine guarantees captions under overload.

**WSL (Windows Subsystem for Linux)** The Windows feature the older CivicCast edition ran on. The current CivicCast line is native Windows.

**X**

**X-List and XMLTV** File formats for program guides. EPG Export offers both and CSV.

## Appendix J: Licenses and third-party software

This appendix lists what CivicCast is licensed under and what other people's software ships inside it. It reports what the project's own files say. It is **not legal advice**, and it does not say whether your use of any component is allowed. Ask a lawyer who knows your situation.

### CivicCast's own license

| What | License |
|---|---|
| CivicCast code | Apache License 2.0 |
| CivicCast documentation (this manual) | Creative Commons Attribution 4.0 International (CC BY 4.0) |

CivicCast is an independent open-source project. It is not affiliated with, sponsored by or approved by Tightrope Media Systems, Cablecast or any other named vendor. Those names are trademarks of their owners; references to other products are for compatibility and comparison only.

### What the project says it has not done

- CivicCast has **not** received a third-party patent clearance or freedom-to-operate opinion.
- **You** are responsible for any patent, codec, broadcast, hardware, cloud-service, app-store or other license your deployment needs.
- The project's own notice names these areas for review by counsel: codec delivery, multi-site federation, dynamic ad insertion, automated recording derivation from live automation logs, real-time caption translation, and commercial distribution of binaries that include patent-encumbered media defaults.

#### The no-GPL posture

The project's rule for what it ships in the Windows runtime (recorded in its architecture decision ADR 0021, "No GPL in the shipped runtime") and how the build enforces it:

1. **GPL media plugins are left out on purpose.** The two GStreamer plugin bundles that carry GPL code are never installed. The build records six plugin files as excluded: the x264 and x265 video encoders, and the a52, dts, dvdread and resindvd plugins. Adding either bundle is a build-refusal condition.
2. **The database pack is checked.** The build refuses to continue if any file in the server pack (PostgreSQL and TSDuck) is recorded under a GPL-family license: "zero GPL/AGPL tolerance" for that pack.
3. **FFmpeg is the LGPL build.** The standalone `ffmpeg.exe` is an LGPL "shared" build, and the FFmpeg libraries inside GStreamer report LGPL with nonfree and version-3 features disabled and no x264 or x265 encoder registered. The Python video library (PyAV) used by captions is a CivicCast-built LGPL-only wheel; the ordinary prebuilt wheels are deliberately not authorized.
4. **Libraries offered under a choice of licenses use the non-GPL choice.** librtmp is offered as GPL or LGPL, and CivicCast records LGPL; cairo as LGPL or MPL-1.1, and it records LGPL; FreeType as the FreeType License or GPL-2.0, and it records the FreeType License. Two D-Bus type files with a similar dual license were removed from the shipped files.
5. **Device control stays out of the core.** The Control Room reaches production equipment through a separate helper (the TSR control service) so that no GPL or AGPL device-control code is linked into the core program (ADR 0019).
6. **Only confirmed licenses are recorded.** For each shipped file the project records a license only where it confirmed it against the upstream project's own license file. A file with no confirmed license is reported as a gap, not guessed. In beta.10's table no file is marked unresolved.

LGPL means you receive the right to use and replace the library, and CivicCast ships the license text with it. It is not the same as GPL, and it still has conditions; read the text.

#### What is bundled

| Component | Version / source record | License (as the project records it) | What it does here |
|---|---|---|---|
| CivicCast program and web portals | 1.0.0-beta.20 unpublished candidate source edition | Apache-2.0 | The station itself; use each installed package's matching component records. |
| GStreamer libraries and plugins | 1.28 series | LGPL-2.1-or-later for the libraries and almost all plugins; the Rust-based plugins (`hlssink3`, closed captions) are MPL-2.0 | The playout engine. |

| Component | Version / source record | License (as the project records it) | What it does here |
|---|---|---|---|
| Cisco OpenH264 | in the closure | BSD-2-Clause | H.264 video encoding. Its patent position is a separate question that the license text does not answer. |
| VisualOn AAC encoder | in the closure | Apache-2.0 for the encoder library; the GStreamer plugin that wraps it is LGPL-2.1-or-later | AAC audio encoding. |
| FFmpeg (`ffmpeg.exe`, `ffprobe.exe` and libraries) | n8.1.2 build | LGPL-3.0-or-later | Preparing and checking media. |
| PostgreSQL | 17.10 | PostgreSQL License | The database. |
| TSDuck (optional) | 3.44 | BSD-2-Clause | Checking and smoothing a cable stream. |
| Ollama | 0.30.6 | MIT | The local AI engine. |
| OpenSSL | 3.x | Apache-2.0 | Secure connections. |
| Node.js | 24.15.0 (listed in the runtime lock) | MIT | Listed in the runtime dependency lock. |
| Whisper large-v3 speech model (via faster-whisper) | pinned revision | MIT | Captions (the optional Large model). The standard Medium model is a separate pack whose license this table does not restate. |
| Whistle speech model and `cactus-needle` engine | Whistle engine 3.1.0; model and wheel hashes pinned in `civiccast/native/app_payload.py` | Apache-2.0 for both | CPU-first live captions. Whistle is included in the signed station pack set. |
| Gemma 4 (12B, e4b) and TranslateGemma 4B models | as pinned in the model lock | The publisher's license, carried inside each model package | Summaries and Spanish translation. Read the license inside the package; this manual does not restate it. |

| Component | Version / source record | License (as the project records it) | What it does here |
|---|---|---|---|
| Smaller libraries | various | BSD-2-Clause, BSD-3-Clause, MIT, Zlib, Libpng, bzip2, FreeType License (FTL), HPND-sell-variant, MIT-Modern-Variant, Unicode-TOU, SQLite's public-domain dedication, ICU, LGPL-2.1-or-later (glib, pango, libsoup, gettext, and others) | Text, fonts, images, networking and compression support. |
| Python packages | pinned by hash | Each under its own license, listed per package in the build's bill of materials | The CivicCast program's dependencies. |
| Web portal libraries | pinned | Not restated here | React, React Router, TanStack Query and hls.js in the portal bundles. |
| Microsoft Visual C++ Redistributable | the installer's own copy | Proprietary (Microsoft's terms) | Required by the Windows programs. CivicCast has no right to reproduce the text; it ships a pointer to where the real terms are. |
| NVIDIA CUDA and cuDNN libraries (optional GPU pack) | optional | NVIDIA's end-user license terms | Faster captions on an NVIDIA graphics card. The pack carries reference texts, not NVIDIA's copyrighted text. |

Remote Contribution separately downloads pinned VDO.Ninja source and the Windows MediaMTX relay when its install action is requested; these are not universal installed runtime components. The source pins and verifies the downloads (`civiccast/installer/contribution_install.py` and `mediamtx_install.py`); MediaMTX's license file is checksum-verified with the executable. Consult each fetched component's included license and source record. Installing them does not com-

mission public HTTPS, WHIP or TURN access.

> **Note:** Names and versions in the table are from the project's lock files and license tables. The exact contents of an installed package are in its `LICENSE-BOM.md` files and matching verification record; use those for that package's component inventory.

#### Where the license texts are

Each runtime pack carries a `LICENSE-BOM.md` (a per-file table of license, generated from the build) and a `licenses` folder with the license text for every license it uses. CivicCast also keeps the texts in its source tree, in `civiccast/native/license_texts`, one file per license identifier:

| File | License |
|---|---|
| `Apache-2.0.txt` | Apache License 2.0 |
| `BSD-2-Clause.txt`, `BSD-3-Clause.txt` | BSD licenses |
| `MIT.txt`, `MIT-Modern-Variant.txt` | MIT licenses |
| `LGPL-2.1-or-later.txt` | GNU Lesser General Public License 2.1 or later |
| `MPL-2.0.txt` | Mozilla Public License 2.0 |
| `FTL.txt` | FreeType License |
| `HPND-sell-variant.txt` | HPND (sell variant), used by fontconfig |
| `LicenseRef-Fontconfig-2.16.1.txt` | fontconfig's own notice |
| `Libpng.txt`, `Zlib.txt`, `bzip2-1.0.6.txt`, `blessing.txt` | libpng, zlib, bzip2 and SQLite |
| `Unicode-TOU.txt` | Unicode terms of use |
| `LicenseRef-Microsoft-VCRedist.txt` | A pointer, not a license text: the Microsoft redistributable's terms are proprietary |

The installer is signed with an Authenticode signature (publisher Scott Converse, through Azure Trusted Signing). To check a download yourself, run `Get-AuthenticodeSignature .\setup.exe` and expect Status Valid; compare the SHA-256 against the release's `SHA256SUMS.txt`.

## Appendix K: Release history

The native Windows release history and the source edition covered by this manual, newest first. The project's release-truth file and GitHub release page establish package publication state. "Superseded" means a newer release replaced it; it does not mean it was bad.

> **Note:** This manual covers the unpublished Beta 20 candidate edition. The Beta 18 verification record applies to that earlier signed package. A 72-hour Beta 11 host run recorded **99.78% live caption-input coverage** across three channels; see the host report. The earlier 48-hour window remains in its dated report.

| Release | Date | Status | What it was |
|---|---|---|---|
| **v1.0.0-beta.20** | — | Unpublished candidate edition; installation verification is pending | Consolidates Sprint 6 preparation, output, webhook, Roku and station-integration changes. |
| **v1.0.0-beta.18** | 2026-10-15 | Published package; historical record | Guided Cablecast station import; see its verification record. |
| **v1.0.0-beta.17** | 2026-10-15 | Superseded by beta.18 | Published signed package; fresh Sandbox install and service checks passed, while the product-engine TSDuck probe timed out. See the matching verification record. |
| **v1.0.0-beta.16** | 2026-10-14 | Superseded by beta.17 | Published signed package with recorded clean install, Beta 15 upgrade, restricted service identity, installed runtime smoke and uninstall checks in a Windows 11 Hyper-V guest. See the matching verification record. |
| **v1.0.0-beta.15** | — | Published package; historical record | Package observations and limits are in the Beta 15 verification record; no extended soak or normal cross-version upgrade is claimed. |

| Release | Date | Status | What it was |
|---|---|---|---|
| **v1.0.0-beta.14** | 2026-10-12 | See release page | Field fixes for installer/model selection, caption timing and recovery, recording, playback, resource protection and documentation. Exact checks are in the Beta 14 verification record. |
| v1.0.0-beta.13 | 2026-10-11 | Published GitHub pre-release | Automatic Whistle recovery and bounded diagnostics. Fresh installation, normal Beta 11 upgrade and one interrupted-install repair passed their recorded three-channel observations. |
| v1.0.0-beta.12 | 2026-10-10 | Superseded by beta.13 | Initial distribution and field failures, subsequent public package repair and upgrade observations are preserved in the Beta 12 verification record. |
| v1.0.0-beta.11 | 2026-10-08 | Superseded by beta.12 | Native live captions use Whistle on CPU by default, with Whisper fallback; the installer includes Whistle and required Medium Whisper while Large Whisper and CUDA are optional. See the Beta 11 verification record for this package's source and exact historical evidence. |

| Release | Date | Status | What it was |
|---|---|---|---|
| v1.0.0-beta.10 | 2026-10-02 (20:31 Mountain; 2026-10-03 02:31 UTC) | Superseded by beta.11 | Added program-change, loudness, looping and relay/reload work. Its eight-hour lab run used an earlier internal build; its clean-install result belongs to the beta.10 package. See Appendix H for historical evidence and limits. |
| v1.0.0-beta.9 | 2026-09-18 (changelog date) | Never published | A version bump and an installer rebuild; its work is in beta.10. |
| v1.0.0-beta.8 | none | Never published | Its work is in beta.10. |
| v1.0.0-beta.7 | 2026-09-15 (19:47 UTC) | Superseded by beta.10 | Published as a GitHub pre-release with a signed `setup.exe`. All three Gate A install journeys and an eight-hour physical-machine soak with captions off passed. Live captions stay off by default in that release. |
| v1.0.0-beta.6 | none | Not listed as a release; the changelog calls it rejected | A candidate that was not accepted. |
| v1.0.0-beta.5 | 2026-09-09 22:14 Mountain (2026-09-10 04:14 UTC) | Superseded | Published as a pre-release after several rejected candidates; all three Gate A lanes passed at publish time. The changelog later records it, with beta.6, as rejected. |

| Release | Date | Status | What it was |
|---|---|---|---|
| v1.0.0-beta.4 | 2026-09-04 (20:41 UTC) | Superseded | A download-only upgrade for stations already on beta.3: `setup.exe` and the runtime packs, no re-download of the AI model bundle. |
| v1.0.0-beta.3 | 2026-09-03 (07:14 UTC) | Superseded | The first downloadable release: `setup.exe` and the runtime packs attached to the GitHub release, verified by checksums. |
| v1.0.0-beta.2 | none | Never published | An internal baseline build used only as the starting point for upgrade tests. |
| v1.0.0-beta.1 | 2026-08-31 | Superseded | The first tagged release of the native Windows line: delivered on USB only, with no downloadable files. |

The same release-truth file also lists tags from a **different, retired product**: the older WSL-based CivicCast edition (v1.0.0-rc1 through rc15 and earlier). They are not releases of this line and are not installation targets for it. Of those, rc13 was withdrawn after a clean Windows test found installer bootstrap failures.

Published packages are listed on the project's GitHub releases page: https://github.com/scottconverse/civiccast-native/releases.

### Appendix L: Index of screens

Every screen of the operator console, then the resident portal pages and the setup program's windows. Use it to find which chapter explains a screen and who can see it.

**Roles** are written as letters: S = Setup admin, M = Meeting operator, R = Records clerk, P = Publish operator, A = Support admin. "All" means every role. Where a role has no letter, the menu hides the screen from it (Appendix F).

**How to open a screen.** In the operator console, click its name in the left menu. A screen's address is the console address plus `#/` and its path, for example `http://127.0.0.1:8000/operator/#/live`. The **Page title** column is filled in where the heading at the top of the screen differs from the

menu name.

**Operator console**

| Menu name | Menu section | Roles | Page title (if different) | Chapter |
|---|---|---|---|---|
| Manual | Help | All | Operator manual | Chapter 2 |
| First Setup | Setup | All | First setup and admin sign-in | Chapter 2 |
| Staff accounts | Setup | S | Staff accounts | Chapter 2 and Chapter 11 (Part II) |
| Backup and restore | Setup | S, A | Station backup and restore | Chapter 12 (Part II) |
| Import station | Setup | S | Import station | Chapter 15 (Part II) |
| Network access | Setup | S, A | Network Settings | Chapter 11 (Part II) |
| Control Room Setup | Setup | S | Control Room — setup | Chapter 11, Configuring the station (Part II) |
| Station Profile | Setup | S, M, A | | Chapter 11, Configuring the station (Part II) |
| Cable Commissioning | Setup | S, A | | Chapter 11 and Chapter 15 (Part II) |
| AI Models | Setup | S, M | | Chapter 11, Configuring the station (Part II) |
| Custom Fields | Setup | S | | Chapter 11, Configuring the station (Part II) |
| Paywall | Setup | S | Subscription paywall | Chapter 11 (Part II); what residents see is in Chapter 6 |
| Live | Run Meeting | All | Live | Chapter 4 |
| Facility | Run Meeting | All | Facility router | Chapter 4 |
| Control Room | Run Meeting | All | Production Control Room | Chapter 4 |
| Remote Contribution | Run Meeting | S, M, A | | Chapter 4 |
| Channels | Run Meeting | All | | Chapter 4 |

| Menu name | Menu section | Roles | Page title (if different) | Chapter |
|---|---|---|---|---|
| Other stations | Run Meeting | S, M, P, A | Other stations | Chapter 15 (Part II) |
| CG Board | Run Meeting | All | | Chapter 4 |
| CG Designer | Run Meeting | S, P, A | CG Board Designer | Chapter 4 |
| Schedule | Run Meeting | All | | Chapter 3 |
| Auto-schedule | Run Meeting | S, P, A | | Chapter 3 |
| Program Guide | Run Meeting | All | Program guide | Chapter 3 |
| Recording | Run Meeting | S, M, A | Scheduled recording | Chapter 3 |
| Assets | Review Records | All | | Chapter 5 |
| Missing Media | Review Records | M, P, A | | Chapter 5 |
| Media Lifecycle Settings | Review Records | S, R, P | | Chapter 5 |
| Contributors | Review Records | All | Contributor submissions | Chapter 3 |
| Review queue | Review Records | All | | Chapter 5 |
| Summary review | Review Records | All | | Chapter 5 |
| Agendas | Review Records | M, R | | Chapter 3 |
| Publish | Publish | All | Publish dashboard | Chapter 6 |
| Playback policy | Publish | All | | Chapter 6 |
| Analytics | Publish | All | | Chapter 7 |
| Reports | Publish | A | | Chapter 7 |
| EPG Export | Publish | S, P | | Chapter 7 |
| Underwriting | Publish | S, P, A | | Chapter 7 |
| App Admin | Publish | S, P | | Chapter 7 |
| Readiness | System Health | All | Safe to broadcast (System Health) | Chapter 8 |
| Alerts | System Health | All | Alerts & monitoring | Chapter 8 |
| Emergency Alerts | System Health | S, M, A | | Chapter 8 |
| Federation | System Health | All | ActivityPub federation | Chapter 8; setup detail in Chapter 15 (Part II) |

That is 42 menu entries in six sections. Two more pages are reached from the Assets list and are not in the menu: **asset detail** (`#/assets/<asset id>`) and the full-screen **trim editor** (`#/assets/<asset id>/trim`); both are in Chapter 5. An address the console does not know shows "Page not

found" with buttons for Manual, First Setup, Recording, Reports and Readiness. A few old or short addresses forward to the right screen (for example `/docs` and `/manual` to Manual, `/sign-in` and `/staff-login` to the staff sign-in page at `/login`, `/readiness` to Readiness, `/today` to Schedule, `/archive` to Assets).

> **Known issue (beta.11):** The top bar clock always says "Next No events scheduled" and the pill "No live meeting broadcast", whatever the schedule holds. They are placeholders. Use Schedule and Channels for the real answers.

#### Resident portal pages

These need no sign-in. They are covered in Chapter 6.

| Page | Address | What it shows |
|---|---|---|
| Home | `/` | What is live now, what is coming up, the latest recordings, the follow-by-email-or-RSS box and the Submit a program form. |
| Watch a recording | `/#/watch/<asset id>` | The video player with captions and the agenda sidebar; a subscription gate appears instead of the player if the station has set a paywall. |
| Browse recordings | `/#/recordings` | A filterable list of recordings. |
| Channel schedule | `/#/schedule?channel=<channel id>` | The coming schedule of one channel (default channel `public`). |

#### Setup program windows

Covered in Part II, Chapter 10 (Installing, first run, upgrading, uninstalling): the Windows setup program (`setup.exe`) with its pre-install checks, pack staging, upgrade and database, activation and self-test, and service and finish steps; and the first-run wizard windows "Checking This Computer", "What CivicCast Needs" and "Downloading" (or "Setting Up"). Setup exit codes are in Appendix E.